

SDAA C编程指南

产品版本号：v3.1.0

发布日期：2026.01.16

版权声明

本版权声明是太初（无锡）电子科技有限公司（以下简称“太初科技”）关于软件产品（下称“产品”）、编程指南以及软件使用手册的全部版本（包含现有版本及未来更新版本）及与软件产品、编程指南以及软件使用手册全部版本相关的源程序、源代码、示例、模型、场景、数据、域名、结构设计、图标、标志、标识、版面设计、文档资料等以及任何由太初科技基于软件技术维护或支持服务所提供的数据、资料等（下称“产品信息”）做出的法律声明。

免责声明：

太初科技不就产品和产品信息做出任何担保、条件、陈述和保证（无论是明示的还是默示的，无论是依据成文法、普通法、惯例还是其他任何原因），包括但不限于性能、结果、安全、不侵权、适销性、完整性、准确性、可不受干扰性、可销售品质以及特定用途，太初科技不应因下列原因产生的任何违约、损害赔偿、费用或问题承担赔偿责任，包括但不限于：

- （1）用户未按照软件使用手册/指南内容规范使用产品；
- （2）因用户自身设计问题导致的损失；
- （3）因用户将软件产品与非太初科技提供的任何其他软件、程序或设备的组合或使用而产生的损失；
- （4）因电力供应故障、通讯网络故障等公共服务因素或移动通讯终端病毒或黑客攻击、系统不稳定等第三人因素产生的损失；
- （5）因不可抗力等风险因素（是指不能预见、不能克服并不能避免且对一方或双方造成重大影响的客观事件，包括但不限于自然灾害如洪水、地震、风暴等以及社会事件如战争、动乱、政府行为等。）致使软件产品的使用出现中断或终止而产生的损失；
- （6）在太初科技已尽善意管理或善意提醒的情况下，因常规或紧急的设备与系统维护、设备与系统故障、网络信息与数据安全等因素产生的损失；
- （7）因您提供的产品反馈信息侵犯第三方权利致使软件产品侵权而发生的侵权索赔；
- （8）非因太初科技的原因引起的与软件产品有关的其它损失。

责任限制：

除法律规定不得排除或限制的赔偿责任外，无论相关责任是基于违约、违反保证、侵权（包括疏忽）、产品责任或任何其他诉讼或责任理论而产生的，太初科技在任何情况下都无需对用户因使用或无法使用产品和产品信息而产生的任何商业损害或间接经济损失承担赔偿责任（包括但不限于商誉损失、利润损失、信息损失、业务中断、计算机故障、第三方索赔或因无法使用软件产品而寻求替代方案的支出费用），即使太初科技已被告知出现这种损失、损害、索赔或费用的可能性。依据本声明以及太初科技的销售条款，太初科技在任何情况下承担的总赔付金额仅限于用户购买产品所支付的款项（如有）。

信息准确性：

上述产品信息属于太初科技所有，且太初科技保留不经通知随时对产品信息或对任何产品和服务做出任何更改的权利，但不承诺更新产品信息。产品使用指南/手册列出的性能测试和等级要使用特定芯片或计算机系统或组件来测量。经该等测试，产品使用手册/指南所示结果反映了太初科技产品的测试性能。系统硬件或软件设计或配置的差异会影响产品的实际性能。如上所述，太初科技不担保或保证产品性能及特定用途，用户需就产品进行必要测试以确认产品适合并适用于用户计划或自行设计的应用及应用程序。

知识产权通知：

产品及其产品信息的著作权、商标权、专利权等知识产权以及商业秘密属于太初科技及/或相关权利人专属所有或持有，受《中华人民共和国著作权法》《计算机软件保护条例》《中华人民共和国商标法》《中华人民共和国专利法》及适用之国际公约中有关著作权、商标权、专利权及/或其他财产所有权法律的保护。

任何单位和个人未经太初科技书面授权，不得以任何目的（包括但不限于学习、研究等非商业用途）修改、截取、编纂、上传、出版、发布等或以任何方式和媒介复制、转载和传播产品信息的任何部分，否则将视为侵权，太初科技保留依法追究其侵权损害赔偿责任。除了用户使用产品和产品信息权利，太初科技不就任何专利、版权、商标、商业秘密或任何其他知识产权或所有权授予任何明示或暗示的权利或许可。

太初科技的“太初元碁”“Tecorigin”等文字及/或标识，以及太初科技的其他标志、标识、徽记、产品和服务名称均为太初科技在中国和其他国家的商标，如有宣传、展示等使用需要，需事先取得太初科技的书面授权。

版权声明

© 2024 太初（无锡）电子科技有限公司 保留一切权利。

目录

1. 概述	1
1.1 图说SDAA C	1
1.2 什么是SDAA C?	1
1.3 产品优势	3
1.4 工作流程	4
1.5 设备端代码使用限制	4
2. 最新动态	8
2.1 v3.1.0更新记录	8
2.2 v3.0.1更新记录	8
2.3 v3.0.0更新记录	8
2.4 v2.3.1更新记录	8
2.5 v2.2.1更新记录	8
2.6 v2.1.1更新记录	10
2.7 v2.1.0更新记录	10
2.8 v2.0.2更新记录	11
2.9 v2.0.1更新记录	13
2.10 v2.0.0更新记录	13
2.11 v1.16.0更新记录	14
2.12 v1.15.0更新记录	14
2.13 v1.14.0更新记录	15
2.14 v1.13.0更新记录	18
2.15 v1.12.0更新记录	19
2.16 v1.11.0更新记录	19
2.17 v1.10.0更新记录	20
2.18 v1.9.0更新记录	20

3. 快速入门	21
3.1 概述	21
3.2 步骤一：安装软件	21
3.3 步骤二：编写程序	21
3.4 步骤三：编译程序	22
3.5 步骤四：运行程序	22
4. 异构计算模型	24
4.1 概述	24
4.2 SPMD	24
4.3 Kernel函数	27
4.4 存储空间	27
4.5 硬件架构	30
4.6 数据传输	31
4.7 SDAARuntime接口	33
5. 语言规范	34
5.1 语法	34
5.1.1 <<<...>>>	34
5.2 关键字	34
5.2.1 __host__	35
5.2.2 __global__	35
5.2.3 __device__	35
5.2.4 __local__	36
5.2.5 __scoped_local__	36
5.2.6 threadIdx	37
5.2.7 threadIdx	38
5.3 数据类型	38
5.4 头文件	39
5.5 命名空间	39

5.6 常用SDAARuntime接口.....	40
6. 环境变量.....	42
6.1 SDAA_SYNC_PRINT.....	42
7. 函数接口.....	43
7.1 概述.....	43
7.2 线程组.....	43
7.2.1 概述.....	43
7.2.2 ThreadGroup.....	43
7.2.3 thread_group_set_mask.....	45
7.2.4 thread_group_include.....	46
7.2.5 thread_group_exclude.....	47
7.2.6 thread_group_is_included.....	48
7.2.7 thread_group_get_size.....	49
7.2.8 thread_group_clear.....	50
7.3 计算核心SPE同步.....	50
7.3.1 概述.....	50
7.3.2 sync_threads.....	51
7.4 SPM内存分配和释放.....	53
7.4.1 概述.....	53
7.4.2 malloc.....	53
7.4.3 free.....	54
7.5 内存初始化.....	55
7.5.1 memset.....	55
7.6 数据搬运.....	57
7.6.1 DMA数据搬运.....	57
7.6.1.1 概述.....	57
7.6.1.2 阻塞型DMA数据搬运.....	58
7.6.1.2.1 memcpy.....	58

7.6.1.2.2 check_memcpy	60
7.6.1.2.3 memcpy_stride	62
7.6.1.2.4 check_memcpy_stride	65
7.6.1.3 非阻塞型DMA数据搬运	68
7.6.1.3.1 MemcpyHandle及配置接口	68
7.6.1.3.1.1 概述	68
7.6.1.3.1.2 MemcpyHandle	68
7.6.1.3.2 memcpy_async	69
7.6.1.3.3 memcpy_wait	73
7.6.1.3.4 check_memcpy_async	75
7.6.2 RMA数据搬运	78
7.6.2.1 概述	78
7.6.2.2 阻塞型RMA数据搬运	79
7.6.2.2.1 rma_get	79
7.6.2.2.2 rma_put	81
7.6.2.3 非阻塞型RMA数据搬运	84
7.6.2.3.1 RmaHandle及配置接口	84
7.6.2.3.1.1 概述	84
7.6.2.3.1.2 RmaHandle	84
7.6.2.3.1.3 rma_set_thread_id	85
7.6.2.3.1.4 rma_set_thread_group	86
7.6.2.3.2 rma_async_get	88
7.6.2.3.3 rma_async_put	91
7.6.2.3.4 rma_complete	94
7.6.2.3.5 rma_wait	96
7.6.2.4 典型场景示例	98
7.6.2.4.1 单个计算核心向多个计算核心发起非阻塞RMA操作	98
7.6.2.4.2 多个计算核心向单个计算核心发起非阻塞RMA操作	102

7.6.3 Broadcast数据搬运	106
7.6.3.1 概述	106
7.6.3.2 BroadcastHandle及配置接口	107
7.6.3.2.1 概述	107
7.6.3.2.2 BroadcastHandle	107
7.6.3.2.3 broadcast_set_thread_group	108
7.6.3.3 阻塞型Broadcast数据搬运	109
7.6.3.3.1 broadcast	109
7.6.3.4 非阻塞型Broadcast数据搬运	114
7.6.3.4.1 broadcast_async	114
7.6.3.4.2 broadcast_wait	121
7.6.3.5 典型场景示例	124
7.6.3.5.1 模拟核心阵列进行横向、纵向广播	124
7.6.4 跨步类型	125
7.6.4.1 概述	125
7.6.4.2 MemoryStride	126
7.7 原子操作	128
7.7.1 概述	128
7.7.2 atomic_inc	128
7.7.3 atomic_add	130
7.7.4 atomic_add_noret	131
7.7.5 atomic_sub	132
7.7.6 atomic_sub_noret	133
7.7.7 atomic_cas	135
7.7.8 atomic_cas_bool	136
7.8 矩阵乘	138
7.8.1 概述	138
7.8.2 阻塞型矩阵乘	139

7.8.2.1 matmul	139
7.8.2.2 check_matmul	142
7.8.3 非阻塞型矩阵乘	145
7.8.3.1 使用逻辑	145
7.8.3.2 调用型操作	146
7.8.3.2.1 matmul_init	146
7.8.3.2.2 matmul_load_weight	149
7.8.3.2.3 matmul_compute	153
7.8.3.2.4 matmul_store	157
7.8.3.2.5 matmul_wait	160
7.8.3.3 配置型操作	164
7.8.3.3.1 MatmulHandle	164
7.8.3.3.2 matmul_set_weight_row_padding	166
7.8.3.3.3 matmul_set_updating_weight	167
7.8.3.3.4 matmul_set_output_row_offset	168
7.8.3.3.5 matmul_set_flushing_output	169
7.8.3.3.6 matmul_wait_loading_weight	170
7.8.3.3.7 matmul_wait_loading_input	174
7.9 转置操作	177
7.9.1 概述	177
7.9.2 阻塞型转置	177
7.9.2.1 transpose	177
7.9.3 非阻塞型转置	180
7.9.3.1 TransposeHandle及配置接口	180
7.9.3.1.1 概述	180
7.9.3.1.2 TransposeHandle	180
7.9.3.2 transpose_async	181
7.9.3.3 transpose_wait	185

7.9.3.4 check_transpose_async	187
7.10 向量操作	191
7.10.1 概述	191
7.10.2 装入/存储类操作	192
7.10.2.1 simd_load	192
7.10.2.2 simd_store	193
7.10.2.3 simd_loadu	193
7.10.2.4 simd_storeu	194
7.10.2.5 simd_load_widen	195
7.10.2.6 simd_store_narrow	197
7.10.2.7 simd_loadu_widen	198
7.10.2.8 simd_storeu_narrow	199
7.10.2.9 simd_set	200
7.10.3 标量扩展类操作	201
7.10.3.1 simd_stretch	201
7.10.4 比较选择类操作	203
7.10.4.1 simd_seleq	203
7.10.4.2 simd_selle	204
7.10.4.3 simd_sellt	206
7.10.5 元素整理类操作	208
7.10.5.1 simd_concat	208
7.10.5.2 simd_ins	210
7.10.6 符号拷贝类操作	212
7.10.6.1 simd_copy_sign	212
7.10.7 类型转换类操作	213
7.10.7.1 simd_cvt_f2h	213
7.10.7.2 simd_cvt_h2f	214
7.10.8 融合乘加类操作	215

7.10.8.1 simd_fma	215
7.10.8.2 simd_fms	216
7.10.8.3 simd_fnma	217
7.10.8.4 simd_fnms	218
7.10.9 算数运算类操作	219
7.10.9.1 simd_add	219
7.10.9.2 simd_sub	221
7.10.9.3 simd_mul	223
7.10.9.4 simd_div	225
7.10.9.5 simd_rem	226
7.10.9.6 simd_fabs	228
7.10.9.7 simd_redsum	229
7.10.10 比较类操作	230
7.10.10.1 simd_cmplt	230
7.10.10.2 simd_cmple	231
7.10.10.3 simd_cmpgt	233
7.10.10.4 simd_cmpge	235
7.10.10.5 simd_cmpeq	236
7.10.10.6 simd_cmpne	238
7.10.10.7 simd_isnan	240
7.10.10.8 simd_max	241
7.10.10.9 simd_min	242
7.10.11 逻辑运算类操作	243
7.10.11.1 simd_and	243
7.10.11.2 simd_or	244
7.10.11.3 simd_not	246
7.10.11.4 simd_neg	247
7.10.12 按位运算类操作	249

7.10.12.1 simd_band	249
7.10.12.2 simd_bxor	250
7.10.12.3 simd_bor	251
7.10.12.4 simd_bnor	252
7.10.12.5 simd_bnot	254
7.11 性能采样	255
7.11.1 概述	255
7.11.2 perf_start	255
7.11.3 perf_stop	256
7.11.4 perf_print	258
7.11.5 clock	260
7.11.6 PerfData	261
7.12 设备信息查询	262
7.12.1 概述	262
7.12.2 get_heap_size	262
7.12.3 get_stack_size	263
7.12.4 get_local_size	263
7.13 程序调试	264
7.13.1 概述	264
7.13.2 printf	264
7.13.3 abort	266
7.13.4 assert	267
8. 数学函数	270
8.1 概述	270
8.2 单精度标量接口	270
8.2.1 ceilf	270
8.2.2 floorf	271
8.2.3 fabsf	272

8.2.4 sqrtf	273
8.2.5 erff	274
8.2.6 expf	275
8.2.7 expm1f	277
8.2.8 isnan	278
8.2.9 loglpf	279
8.2.10 log2f	280
8.2.11 logf	281
8.2.12 sigmoidf	282
8.2.13 sinf	283
8.2.14 cosf	284
8.2.15 tanf	285
8.2.16 tanhf	286
8.2.17 atanf	287
8.2.18 fmaf	289
8.2.19 fmaxf	290
8.2.20 fminf	291
8.2.21 fmodf	292
8.2.22 powf	293
8.2.23 roundf	295
8.3 双精度标量接口	296
8.3.1 ceil	296
8.3.2 floor	297
8.3.3 fabs	298
8.3.4 sqrt	299
8.3.5 erf	301
8.3.6 exp	302
8.3.7 expm1	303

8.3.8 isnan.....	304
8.3.9 loglp.....	305
8.3.10 log2.....	306
8.3.11 log.....	307
8.3.12 sigmoid.....	308
8.3.13 sin.....	310
8.3.14 cos.....	311
8.3.15 tan.....	312
8.3.16 tanh.....	313
8.3.17 atan.....	314
8.3.18 fma.....	315
8.3.19 fmax.....	316
8.3.20 fmin.....	317
8.3.21 fmod.....	318
8.3.22 pow.....	320
8.3.23 round.....	322
8.4 向量接口.....	323
8.4.1 simd_exp.....	323
8.4.2 simd_exp128.....	324
8.4.3 simd_log.....	325
8.4.4 simd_log128.....	327
8.4.5 simd_tanh.....	328
8.4.6 simd_tanh128.....	329
8.4.7 simd_sigmoid.....	330
8.4.8 simd_sigmoid128.....	331
8.4.9 simd_sqrt.....	332
8.4.10 simd_inv.....	334
8.4.11 simd_erf.....	335

8.4.12 simd_erf128	336
8.4.13 simd_atan	337
8.4.14 simd_atan128	339
8.4.15 simd_log2	340
8.4.16 simd_log2_128	341
8.4.17 simd_sin	342
8.4.18 simd_sin128	344
8.4.19 simd_cos	345
8.4.20 simd_cos128	346
8.4.21 simd_round	347
8.4.22 simd_round128	349
8.4.23 simd_trunc	350
8.4.24 simd_trunc128	351
8.4.25 simd_floor	352
8.4.26 simd_floor128	353
9. 高层次函数接口	355
9.1 概述	355
9.2 数学函数	355
9.2.1 batch_tanh	355
9.2.2 batch_sin	356
9.2.3 batch_cos	357
9.2.4 batch_log	358
9.2.5 batch_pow	360
9.2.6 batch_floor	362
9.2.7 batch_ceil	363
9.2.8 batch_axpy	364
9.2.9 batch_exp	366
9.2.10 batch_xor	367

9.3 激活函数.....	368
9.3.1 batch_gelu.....	368
9.3.2 batch_sigmoid.....	369
9.4 规约函数.....	371
9.4.1 sum.....	371
10. 程序编译.....	373
10.1 命令行选项.....	373
10.2 编译流程.....	374
10.2.1 LTO模式.....	374
10.2.2 非LTO模式.....	375
10.3 编译方式.....	376
10.3.1 概述.....	376
10.3.2 LTO模式.....	376
10.3.2.1 动态链接方式.....	376
10.3.2.2 静态链接方式.....	377
10.3.2.3 直接生成可执行文件方式.....	377
10.3.3 非LTO模式.....	377
10.3.3.1 动态链接方式.....	378
10.3.3.2 静态链接方式.....	378
10.3.3.3 直接生成可执行文件方式.....	378
10.4 编译示例.....	378
10.4.1 编译示例1.....	379
10.4.2 编译示例2.....	380
11. 程序调试.....	383
11.1 概述.....	383
11.2 打印运行时日志.....	383
11.3 TecoGDB调试工具.....	384
11.4 终止设备端程序.....	384

11.5 断言	385
12. 命令行工具	387
12.1 TecoGDB调试工具	387
12.2 sdaacfilt符号解码工具	387
13. 性能调优	389
13.1 概述	389
13.2 性能度量	389
13.2.1 性能指标	389
13.2.2 性能采样	390
13.3 性能优化	392
13.3.1 程序并行	392
13.3.1.1 任务级并行	392
13.3.1.2 线程级并行 (SPMD)	396
13.3.1.3 指令级并行 (SIMD)	399
13.3.2 函数接口	402
13.3.2.1 SPE同步接口	402
13.3.2.2 DMA数据搬运	404
13.3.2.2.1 SPE ID相关	404
13.3.2.2.2 阻塞型DMA数据搬运	409
13.3.2.2.2.1 概述	409
13.3.2.2.2.2 搬运地址	409
13.3.2.2.2.3 数据搬运量	412
13.3.2.2.2.4 数据搬运方向	415
13.3.2.2.3 非阻塞型DMA数据搬运	418
13.3.2.3 RMA数据搬运	421
13.3.2.3.1 SPE ID相关	421
13.3.2.3.2 非阻塞型RMA数据搬运	423
13.3.2.3.2.1 优化策略	423

13.3.2.3.2 rma_get 优化示例	423
13.3.2.3.3 rma_put 优化示例	428
13.3.2.4 Broadcast数据搬运	433
13.3.2.4.1 SPE ID相关	433
13.3.2.4.2 非阻塞型Broadcast数据搬运	435
13.3.2.5 原子操作	439
13.3.2.6 矩阵乘	443
13.3.2.6.1 阻塞型矩阵乘	443
13.3.2.6.2 非阻塞型矩阵乘	445
13.3.2.7 向量操作	450
13.3.2.7.1 性能数据	450
13.3.2.7.2 优化策略	453
13.3.3 数学函数	455
13.3.3.1 精度和性能数据	455
13.3.3.2 优化策略	460
13.3.4 程序编译	463
13.3.4.1 循环展开	463
13.3.4.2 链接时优化 (LTO)	471
13.3.4.3 浮点融合	474
14. 接口示例库	484
15. 入门实战	485
15.1 向量运算	485
15.1.1 概述	485
15.1.2 编程思路	485
15.1.3 编写程序	486
15.1.3.1 主机端	486
15.1.3.2 设备端	489
15.1.3.3 完整程序	491

15.1.4 程序编译.....	493
15.2 使用SPMD完成矩阵乘.....	493
15.2.1 概述.....	493
15.2.2 编程思路.....	494
15.2.3 编写程序.....	494
15.2.3.1 主机端.....	495
15.2.3.2 设备端.....	497
15.2.3.3 完整程序.....	499
15.2.4 编译程序.....	502
15.3 使用SUMMA算法实现矩阵乘.....	503
15.3.1 概述.....	503
15.3.2 编程思路.....	503
15.3.3 编写程序.....	504
15.3.3.1 主机端.....	504
15.3.3.2 设备端.....	507
15.3.3.3 完整程序.....	512
15.3.4 编译程序.....	519
15.4 使用CAS完成自定义原子操作.....	519
15.4.1 概述.....	519
15.4.2 编程思路.....	520
15.4.3 编写程序.....	520
15.4.3.1 主机端.....	520
15.4.3.2 设备端.....	522
15.4.3.3 完整程序.....	524
15.4.4 编译程序.....	525
16. 进阶实战.....	527
16.1 概述.....	527
16.2 算子开发实战.....	527

16.2.1 什么是算子开发.....	527
16.2.2 算子基本概念.....	527
16.2.3 算子开发流程.....	529
16.2.4 开发加法算子.....	530
16.2.4.1 算子分析.....	530
16.2.4.2 算子实现.....	531
16.2.4.2.1 实现流程.....	531
16.2.4.2.2 步骤一：申请主机端内存并初始化输入数据.....	532
16.2.4.2.3 步骤二：申请Global存储空间并搬运数据（主机端至Global）...	532
16.2.4.2.4 步骤三：申请SPM存储空间并搬运数据（Global至SPM）至SPM进行计算.....	533
16.2.4.2.5 步骤四：搬运数据（SPM至Global）.....	534
16.2.4.2.6 步骤五：搬运数据（Global至主机端）.....	535
16.2.4.2.7 步骤六：算子测试.....	535
16.2.4.2.8 完整程序.....	537
16.2.4.3 编译运行.....	539
16.2.5 开发矩阵乘算子.....	539
16.2.5.1 算子分析.....	539
16.2.5.2 算子实现.....	541
16.2.5.2.1 设备端矩阵乘.....	541
16.2.5.2.2 主机端矩阵乘.....	543
16.2.5.2.3 完整程序.....	545
16.2.5.3 编译运行.....	548
16.3 算子性能调优.....	549
16.3.1 概述.....	549
16.3.2 性能度量.....	549
16.3.3 性能优化.....	550
16.3.3.1 概述.....	550

16.3.3.2 计算相关优化.....	551
16.3.3.2.1 向量指令使用.....	551
16.3.3.2.2 指令流水线优化.....	553
16.3.3.2.3 矩阵乘法加速单元.....	555
16.3.3.3 访存相关优化.....	557
16.3.3.3.1 双缓冲优化.....	558
16.3.3.3.2 广播优化.....	561
16.3.4 优化实战.....	564
16.3.4.1 优化思路.....	564
16.3.4.2 优化程序.....	565
16.3.4.3 完整程序.....	567
16.4 算子调优实战.....	571
16.4.1 GEMM基础实现.....	571
16.4.1.1 实现原理.....	571
16.4.1.2 完整程序.....	573
16.4.1.3 性能数据.....	576
16.4.2 GEMM优化策略.....	577
16.4.2.1 SIMD.....	577
16.4.2.1.1 实现原理.....	577
16.4.2.1.2 完整程序.....	582
16.4.2.1.3 性能数据.....	590
16.4.2.2 矩阵乘法单元.....	591
16.4.2.2.1 实现原理.....	591
16.4.2.2.2 完整程序.....	596
16.4.2.2.3 性能数据.....	603
16.4.2.3 广播.....	604
16.4.2.3.1 实现原理.....	605
16.4.2.3.2 完整程序.....	610

16.4.2.3.3 性能数据	618
16.4.2.4 双缓冲	619
16.4.2.4.1 实现原理	619
16.4.2.4.2 完整程序	626
16.4.2.4.3 性能数据	635
17. 常见问题	637
17.1 如何使用CMake构建?	637
17.1.1 LTO模式	637
17.1.1.1 示例代码	637
17.1.1.2 静态库	638
17.1.1.3 动态库	640
17.1.2 非LTO模式	641
17.1.2.1 示例代码	641
17.1.2.2 静态库	642
17.1.2.3 动态库	643
17.2 链接LTO模式编译的库报错, libxxx.a: archive has no index; run ranlib to add one, 如何解决?	643
17.3 CUDA程序迁移	644
17.3.1 如何解决线程编号不对应?	644
17.3.2 如何解决调用Kernel函数不对应?	644
17.3.3 SDAA C中是否有与CUDA对应的线程束Warp?	645
17.3.4 如何在Kernel函数中调用数学函数接口?	645
17.3.5 CUDA的blockId如何修改为SDAA C代码?	646
17.3.6 CUDA的share memory如何修改为SDAA C代码?	646
17.4 使用VSCode打开SDAA C程序时, 如何实现代码高亮、代码自动补全和代码高亮提醒等功能?	646

1. 概述

1.1 图说SDAA C



图1 SDAA C图解

1.2 什么是SDAA C？

随着计算机技术的飞速发展，传统的CPU处理器体系已经无法满足天气预报、工程仿真和以深度学习为代表的人工智能等领域对计算资源的需求，从而促使计算机的处理器体系发生从单核到多核再到众核的革命性跨越。

目前，以众核作为加速核心的主流产品包括：NVIDIA GPU和AMD GPU等，对应的编程语言分别为：CUDA C/CUDA Fortran和OpenCL。同时还有譬如OpenACC导语之类的并行编程语法。以上三款产品在高并发、大数据计算方面能够基本满足用户对计算资源的需求，但在编程方面依然存在着亟待解决的问题：

- CUDA C/CUDA Fortran、OpenCL等编程语言在编程语法上符合当前主流编程语言的编程规范，学习成本较低。但是其对应的编程模型与硬件结合较为密切，需要用户深入了解硬件体系结构，否则可能无法有效使用加速核心的全部算力。
- OpenACC屏蔽了底层的硬件差异，不需要深入了解硬件体系结构，但是其自身的编程导语与主流的编程语言存在较大差异，需要用户全量学习OpenACC的编程语法，学习成本和使用门槛均较高。

为了解决主流众核加速产品在学习成本和使用方法上的问题，SDAA C编程语言应运而生。

SDAA C（Software Defined Accelerator Architecture C）是运行在SDAA异构并行计算平台上的编程模型。以太初AI加速卡为计算核心，支持C/C++编程语言，用户仅需学习少量的**SDAA C编程语法**即可快速上手编程。同时SDAA C最大化地隐藏了底层硬件的实现细节，用户无需深入了解硬件的体系架构就可以正常使用太初AI加速卡提供的强大算力，大幅度地降低学习成本，使用户可以专注于并行政程的开发。

SDAA异构并行计算平台

SDAA异构并行计算平台是太初元碁针对AI场景，自主研发的众核异构计算架构，能够充分发挥太初AI加速卡的计算能力，为科学计算、人工智能等领域提供完整的解决方案。



SDAA异构并行计算平台以太初元碁人工智能系列硬件（太初AI加速卡）为基础，基础软件层完成对硬件驱动及操作系统的高度抽象，并提供完整的运行时库，编程模型及配套的编译、调试和分析工具；加速库提供包括TecoDNN、TecoBLAS等基础且丰富的算子库；应用层整合了当前主流的深度学习框架。

SDAA C运行于基础软件层，对下屏蔽了硬件的物理特性，对上提供了丰富的编程接口，以及代码调试、调优工具。用户无需感知底层的硬件细节，仅需遵从SDAA C的编程规范即可使用太初AI加速卡，为高并发、大数据运算提供强有力的算力支撑。

SDAA异构并行模型

SDAA异构并行模型采用主从异构的物理架构，即Host-Device运行模式。

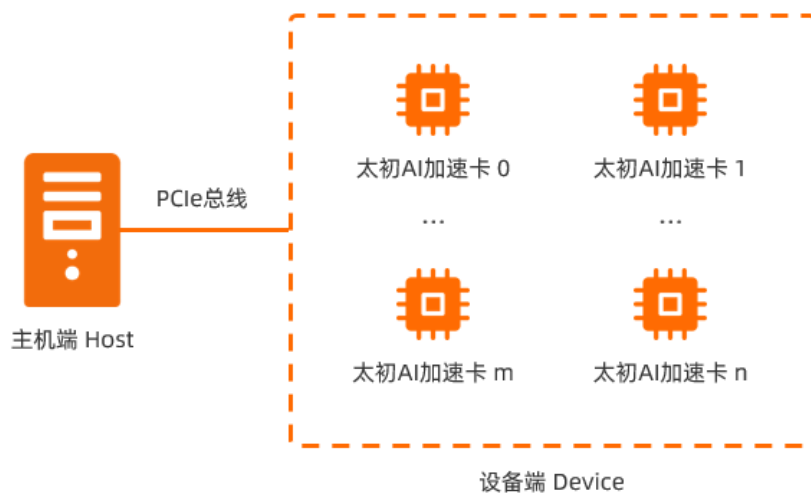


图2 SDAA异构并行模型

- 主机端（Host端）：由CPU构成，主要负责对设备端（Device端）内存资源的申请和释放，并控制设备端（Device端）处理任务的资源。
- 设备端（Device端）：由太初AI加速卡构成，负责处理大规模的数据运算。太初AI加速卡通过高效的运算部件（SPE）和访存部件（Global存储器等）弥补了通用处理器在处理高并发、大数据场景下计算和访存上的短板，提高了数据处理的效率。
 - 太初AI加速卡由计算核心阵列SPA（Synergistic Processor Element Array）和对应的Global高带宽存储器构成。
 - 每个计算核心阵列SPA是一个协从运算单元，由计算核心SPE（Synergistic Processor Element）构成。

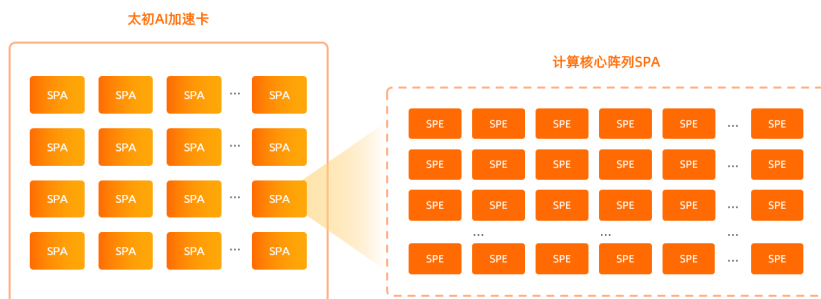


图3 太初AI加速卡

1.3 产品优势

SDAA C规避了主流加速器编程模型中存在的编程语法学习成本高、与硬件强相关造成的兼容性差等诸多缺点，是一款上手简单，同时内容丰富的编程模型。

学习成本低

SDAA C支持C/C++标准语法，用户仅需学习少量的SDAA C编程语法即可快速上手编程。同时SDAA C最大化地隐藏了底层硬件的实现细节，用户无需深入了解硬件的体系架构就可以正常使用太初AI加速卡提供的强大算力，大幅度地降低学习成本，使用户可以专注于并程序的开发。

丰富的函数接口

SDAA C提供了包括：矩阵乘、向量运算函数、数据搬运及原子操作等丰富的[函数接口](#)，可以更好地辅助用户使用太初AI加速卡的计算资源，简化应用程序开发，优化应用程序性能。

丰富的数学函数

SDAA C内置了丰富的数学函数接口，集成了包括：正弦计算、余弦计算、正切计算、指数计算、对数计算、高斯误差计算等常用标量数学计算接口。同时为了满足用户对于应用程序运行性能的追求，还提供了相应的向量数学函数计算接口，具体可参考[数学函数](#)。

程序调试

程序调试是应用程序开发过程中的必要环节，优秀的程序调试手段可以帮助程序开发人员在一定程度上推进应用程序开发的进度。SDAA C提供打印计算核心SPE接口运行时日志功能，可以帮助用户快速定位设备端的运行时问题，具体可参考[程序调试](#)。

性能调优

为了获取更好的系统性能并满足不断增加的业务需求，程序的运行性能通常是应用程序开发人员重点关注的内容之一，SDAA C提供了性能调优工具和基本的并行编程优化技巧，可以辅助程序开发人员优化应用程序，最大化的利用太初AI加速卡的物理资源，提升应用程序的执行速度，具体可参考[性能调优](#)。

1.4 工作流程

SDAA C将Host端的应用程序切分成一系列的加速任务，每个加速任务会通过SDAA C的编程接口发送到Device端的太初AI加速卡上并行执行。加速任务在Device端执行完毕后将计算结果返回至Host端，供用户进一步处理。

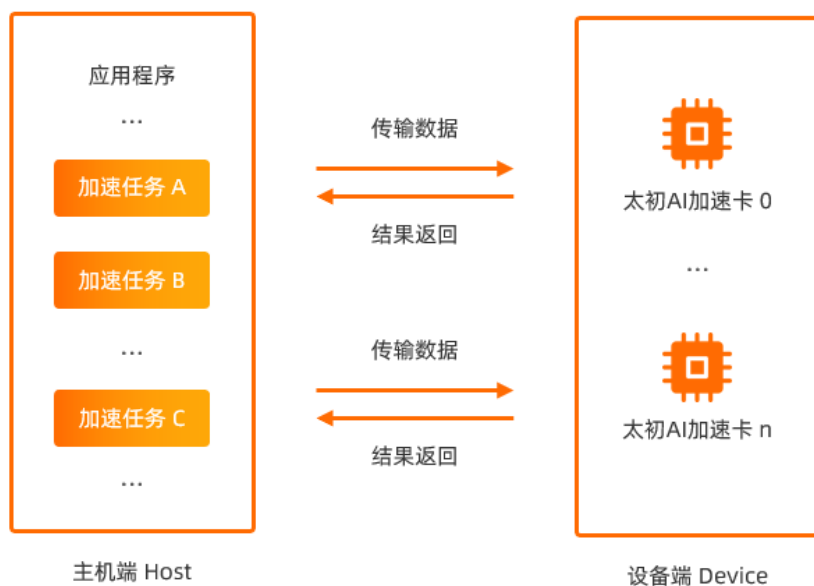


图1 SDAA C工作流程

1.5 设备端代码使用限制

SDAA C基于C/C++语法，并在此基础上进行了部分优化扩展。用户在设备端编程时，可以基本遵循C/C++编程规范。SDAA C的设备端代码不支持以下内容：

- Exceptions and try/catch blocks 异常处理
- RTTI
- STL标准库
- 全局对象的构造与析构
- 虚函数表跨越Host与Device端
- 使用new关键字
- `__global__`修饰的函数中使用可变参数
- 线程私有操作
- 局部静态变量
- C/C++原生的原子操作（您可以使用SDAA C提供的原子操作接口，可参考[原子操作](#)）
- 以下终止进程相关函数

```
void exit(int status);  
int atexit(void (*function)(void));
```

- 所有文件操作相关函数

```
int fclose(FILE *stream);  
FILE *fopen(const char *pathname, const char *mode);  
FILE *fdopen(int fd, const char *mode);  
FILE *freopen(const char *pathname, const char *mode, FILE *stream);  
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);  
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);  
int fprintf(FILE *stream, const char *format, ...);  
int fscanf(FILE *stream, const char *format, ...);  
int fgetc(FILE *stream);  
char *fgets(char *s, int size, FILE *stream);  
int getc(FILE *stream);  
int ungetc(int c, FILE *stream);  
int fputc(int c, FILE *stream);  
int fputs(const char *s, FILE *stream);  
int putc(int c, FILE *stream);  
int fflush(FILE *stream);  
...
```

SDAA C对C++11新特性支持情况如下表：

C++11 功能特性	提案	SDAA C是否支持
C99 preprocessor	N1653	是

C++11 功能特性	提案	SDAA C是否支持
<code>static_assert</code>	N1720	是
Right angle brackets	N1757	是
Extended <code>friend declarations</code>	N1791	是
<code>long long</code>	N1811	是
Compiler support for <code>type traits</code>	N1836 N2518* N2984 N3142	是
<code>auto</code>	N1984	是
<code>Delegating constructors</code>	N1986	是
extern template	N1987	是 (LTO模式支持)
<code>constexpr</code>	N2235	是
<code>Template aliases</code>	N2258	是
<code>char16_t</code> and <code>char32_t</code>	N2249	是
<code>alignas</code>	N2341	是
<code>alignof</code>	N2341	是
Defaulted and deleted functions	N2346	是
<code>Strongly-typed enum</code>	N2347	是
<code>Atomic operations</code>	N2427	否
<code>nullptr</code>	N2431	是
Explicit <code>conversion operators</code>	N2437	是
ref-qualifiers	N2439	是
Unicode <code>string literals</code>	N2442	是
Raw <code>string literals</code>	N2442	是
<code>Inline namespaces</code>	N2535	是
<code>Inheriting constructors</code>	N2540	是

C++11 功能特性	提案	SDAA C是否支持
Trailing function return types	N2541	是
Unrestricted unions	N2544	是
Variadic templates	N2242 N2555	是
Expression SFINAE	N2634	是
Dynamic initialization and destruction with concurrency	N2660	否
Initializer lists	N2672	是
Non-static data member initializers	N2756	是
Attributes	N2761	是
Forward (opaque) enum declarations	N2764	是
User-defined literals	N2765	是
Rvalue references	N2118 N2844 CWG1138	是
Lambda expressions	N2550 N2658 N2927	是
Range-for loop	N2930 N3271	是
noexcept	N3050	是
Defaulted move special member functions	N3053	是
override and final	N2928 N3206 N3272	是
decltype	N2343 N3276	是

2. 最新动态

2.1 v3.1.0更新记录

更新内容

优化代码及修复代码已知问题，不涉及文档内容变化。

2.2 v3.0.1更新记录

更新内容

优化代码及修复代码已知问题，不涉及文档内容变化。

2.3 v3.0.0更新记录

新增内容

描述	文档
程序编译的命令行选项，新增 <code>--sdaa-arch=pcx_100</code> ，用于生成《PCX编程指南》相关代码。	命令行选项

2.4 v2.3.1更新记录

新增内容

描述	文档
原子操作 <code>atomic_add</code> 新增支持的数据类型。	atomic_add
<code>sdaacfilt</code> 符号解码工具的sdaacfilt示例，补充交互模式下的命令行说明。	sdaacfilt符号解码工具

2.5 v2.2.1更新记录

新增内容

描述	文档
非阻塞型Broadcast数据搬运： - 新增 <code>broadcast_async</code> 接口。 - 新增 <code>broadcast_wait</code> 接口。	<code>broadcast_async</code> <code>broadcast_wait</code>
跨步类型：新增 <code>MemoryStride</code> 结构体。	<code>MemoryStride</code>

更新内容

描述	文档
更新DMA数据搬运接口形式： - 阻塞型DMA数据搬运接口： - 更新 <code>memcpy_stride</code> 接口形式。 - 更新 <code>check_memcpy_stride</code> 接口形式。 - 非阻塞型DMA数据搬运接口： - 更新 <code>MemcpyHandle</code> 接口形式。 - 更新 <code>memcpy_async</code> 接口形式。 - 更新 <code>check_memcpy_async</code> 接口形式。	<code>memcpy_stride</code> <code>check_memcpy_stride</code> <code>MemcpyHandle</code> <code>memcpy_async</code> <code>check_memcpy_async</code>
更新Broadcast接口形式： - 更新 <code>BroadcastHandle</code> 接口。 - 更新阻塞型Broadcast数据搬运的 <code>broadcast</code> 接口。 - 更新非阻塞型Broadcast数据搬运接口： - 更新 <code>broadcast_async</code> 接口。 - 更新 <code>broadcast_wait</code> 接口。	<code>BroadcastHandle</code> <code>broadcast</code> <code>broadcast_async</code> <code>broadcast_wait</code>
更新非阻塞转置接口： - 更新 <code>TransposeHandle</code> 接口。 - 更新 <code>transpose_async</code> 接口。 - 更新 <code>check_transpose_async</code> 接口。	<code>TransposeHandle</code> <code>transpose_async</code> <code>check_transpose_async</code>

描述	文档
设备信息查询接口： <code>get_heap_size</code> 、 <code>get_stack_size</code> 、 <code>get_local_size</code> ，在使用时，不需要手动包含 <code>sdaa_device_info.h</code> 头文件。	get_heap_size get_stack_size get_local_size

删除内容

描述	文档
非阻塞型DMA数据搬运接口：删除 <code>memcpy_set_stride</code> 接口。	N/A
BroadcastHandle及配置接口：删除 <code>broadcast_set_stride</code> 接口。	N/A
跨步类型：删除 <code>Stride</code> 、 <code>stride_set_section_num</code> 、 <code>stride_set_glbmem_stride</code> 、 <code>stride_config</code> 接口。	N/A
非阻塞转置接口：删除 <code>transpose_set_section_size</code> 、 <code>transpose_set_stride</code> 接口。	N/A

2.6 v2.1.1更新记录

新增内容

描述	文档
新增设备信息查询头文件 <code>sdaa_device_info.h</code> 。	头文件
函数接口新增设备信息查询接口。	设备信息查询接口

2.7 v2.1.0更新记录

新增内容

描述	文档
新增硬件架构章节。	硬件架构
新增命名空间介绍章节。	命名空间

描述	文档
程序编译的命令行选项，新增 <code>-h</code> / <code>--help</code> 。	命令行选项
进阶实战，新增GEMM算子调优实战。	算子调优实战

2.8 v2.0.2更新记录

新增内容

描述	文档
语言规范，新增scl.h头文件。	头文件
新增 <code>simd_storeu_narrow</code> 向量接口。	<code>simd_storeu_narrow</code>
数学函数，新增向量取整接口： <code>simd_round</code> <code>simd_round128</code> <code>simd_trunc</code> <code>simd_trunc128</code> <code>simd_floor</code> <code>simd_floor128</code>	<code>simd_round</code> <code>simd_round128</code> <code>simd_trunc</code> <code>simd_trunc128</code> <code>simd_floor</code> <code>simd_floor128</code>

描述	文档
新增高层次函数接口： - batch_tanh - batch_sin - batch_cos - batch_log - batch_pow - batch_floor - batch_ceil - batch_axpy - batch_exp - batch_xor - batch_gelu - batch_sigmoid - sum	高层次函数接口
新增命令行工具章节，包括： - TecoGDB调试工具 - sdaacfilt符号解码工具	命令行工具
性能调优，新增部分取整接口精度和性能数据： - simd_round - simd_round128 - simd_trunc - simd_trunc128 - simd_floor - simd_floor128	精度和性能数据

更新内容

描述	文档
更新powf、pow、logf、log、simd_log以及simd_log128的边界值说明。	• powf • pow • logf • log • simd_log • simd_log128

2.9 v2.0.1更新记录

新增内容

描述	文档
新增 <code>simd_loadu_widen</code> 向量接口。	simd_loadu_widen

2.10 v2.0.0更新记录

新增内容

描述	文档
新增性能采样的结构体变量。	PerfData
新增进阶实战内容。	进阶实战

更新内容

描述	文档
性能采样的 <code>perf_start</code> 接口，使用方式更新。	perf_start
性能采样的 <code>perf_stop</code> 接口，使用方式更新。	perf_stop
性能采样的 <code>perf_print</code> 接口，输出的内容更新。	perf_print
优化性能调优章节内容。	性能调优
接口示例库调整为一级目录。	接口示例库

描述	文档
最佳实战更新为入门实战。	入门实战

2.11 v1.16.0更新记录

更新内容

描述	文档
向量操作的符号拷贝类接口simd_cop_sign，新增half数据类型。	simd_copy_sign
向量操作的比较操作接口simd_cmpeq，结果向量新增float和half数据类型。	simd_cmpeq
更新数学函数向量接口的边界值说明。	simd_exp128 simd_log128 simd_tanh128 simd_sigmoid128 simd_erf128 simd_atan128 simd_log2_128 simd_sin128 simd_cos128

2.12 v1.15.0更新记录

更新内容

描述	文档
非阻塞型转置，修改跨步单元的大小限制。	transpose_set_section_size
非阻塞型转置，增加跨步参数说明。	transpose_set_stride
同一SPA内Global存储到SPM存储之间的非阻塞型转置操作，修改源矩阵行和列的限制。	transpose_async

描述	文档
更新边界值说明。	• 向量操作 • 数据搬运 • 单精度标量接口 • 双精度标量接口 • 向量接口
更新示例代码结果注释说明。	转置操作 、 向量操作 、 性能采样 、 程序调试 、 性能调优

2.13 v1.14.0更新记录

新增内容

描述	文档
新增内存初始化接口 <code>memset</code>	memset

描述		文档
新增向量操作接口	装入/存储类操作	simd_set
	算数运算类操作	simd_add
		simd_sub
		simd_mul
		simd_div
		simd_rem
		simd_redsum
	比较类操作	simd_cmplt
		simd_cmple
		simd_cmpgt
		simd_cmpge
		simd_cmpeq
		simd_cmpne
		simd_isnan
		simd_max
		simd_min
	逻辑运算类操作	simd_and
		simd_or
		simd_not
		simd_neg

描述		文档
	按位运算类操作	simd_band
		simd_bxor
		simd_bor
		simd_bnor
		simd_bnot
新增性能采样接口	clock	clock

描述		文档
新增数学函数	单精度标量接口	ceilf
		expm1f
		isnan
		log1pf
		fabsf
		floorf
		fmaf
		fmaxf
		fminf
		fmodf
		powf
		roundf
	双精度标量接口	ceil
		expm1
		isnan
		log1p
		fabs
		floor
		fma
		fmax
		fmin
		fmod

2.14 v1.13.0更新记录

更新内容

优化代码及修复代码已知问题，不涉及文档内容变化。

2.15 v1.12.0更新记录

新增内容

描述	文档
新增 <code>__scoped_local__</code> 关键字	__scoped_local__
TecoCC支持的编译命令行，新增 <code>--stack-on-global</code> 选项	支持的命令行选项

2.16 v1.11.0更新记录

新增内容

描述	文档
标量数据类型，新增bfloat类型	数据类型
新增环境变量 <code>SDAA_SYNC_PRINT</code>	SDAA_SYNC_PRINT
装入/存储类向量操作，新增 <code>simd_load_widen</code> 函数。	simd_load_widen
装入/存储类向量操作，新增 <code>simd_load_narrow</code> 函数。	simd_load_narrow
新增融合乘加类向量操作，包括： <code>simd_fma</code> 函数 <code>simd_fms</code> 函数 <code>simd_fnma</code> 函数 <code>simd_fnms</code> 函数	融合乘加类操作
数学函数的向量接口，新增 <code>simd_sin</code> 函数。	simd_sin
数学函数的向量接口，新增 <code>simd_sin128</code> 函数。	simd_sin128
数学函数的向量接口，新增 <code>simd_cos</code> 函数。	simd_cos

描述	文档
数学函数的向量接口，新增 <code>simd_cos128</code> 函数。	simd_cos128
设备端支持 <code>abort</code> 接口。	终止设备端程序
设备端支持断言 <code>assert</code> 接口。	断言

2.17 v1.10.0更新记录

新增内容

描述	文档
数学函数的单精度标量接口，新增 <code>log2</code> 函数。	log2
数学函数的双精度标量接口，新增 <code>log2f</code> 函数。	log2f
数学函数的向量接口，新增 <code>simd_log2</code> 函数。	simd_log2
数学函数的向量接口，新增 <code>simd_log2_128</code> 函数。	simd_log2_128

2.18 v1.9.0更新记录

更新内容

本版本为首次正式发布版本。

3. 快速入门

3.1 概述

本入门指导以在设备端打印 `Hello AI Card` 为例，概要介绍SDAA C的编程流程，帮助您快速上手SDAA C编程。

在使用SDAA C进行编程前，您需要先在Linux系统中安装tecocc编译工具，在主机端编写应用程序后，再使用tecocc编译工具进行编译，最后运行编译后的二进制文件。



图1 SDAA C编程流程

3.2 步骤一：安装软件

TecoCC作为TecoToolKit的组件之一，具体安装方法可以参考《环境安装手册》。

3.3 步骤二：编写程序

在使用SDAA C进行应用程序开发时，用户不需要分开编写主机端和设备端的程序，而是通过SDAA C提供的关键字将应用程序切分成一系列的加速任务。每个加速任务会通过SDAA C的编程接口发送到Device端的太初AI加速卡上并行执行。

? 说明：

1. 为使编译器可以正常识别SDAA C提供的关键字，应用程序的文件名需要以scpp为后缀进行命名。
2. 详细的语言规范，可参考[语言规范](#)。

本文以打印设备端 `Hello AI Card` 为例，创建一个以scpp为后缀的文件，例如test.scpp。test.scpp中应用程序如下：

```
__global__ void kernel_entry()
{
    printf("Hello AI Card\n");
}
```

```
int main()
{
    sdaaSetDevice(0);
    kernel_entry<<<1>>>();
    sdaaDeviceSynchronize();

    return 0;
}
```

上述应用程序中：

- `__global__`：函数标识符，修饰Kernel函数，可参考[__global__](#)。
例如：此示例中 `kernel_entry` 函数前有 `__global__` 关键字修饰，表示该函数是Kernel函数，在设备端运行。
- `sdaaSetDevice`：SDAARuntime接口，用于指定Kernel函数运行在设备端具体哪一个计算核心阵列SPA上，可参考[常用SDAARuntime接口](#)。
例如：此示例中`sdaaSetDevice(0)`，表示Kernel函数运行在设备端0号计算核心阵列SPA上。
- `<<< ... >>>`：通过`<<< ... >>>`调用Kernel函数。
- `sdaaDeviceSynchronize`：SDAARuntime接口，将一个计算核心阵列内`threadDim`个计算核心SPE的数据进行同步，可参考[常用SDAARuntime接口](#)。
- `printf`：设备端打印接口，使用方法可参考[打印运行时日志](#)。

3.4 步骤三：编译程序

TecoCC 是SDAA C提供的编译器，详细使用方法可参考[程序编译](#)。

在test.scpp文件路径下，执行以下命令，编译程序。

```
tecocc test.scpp
```

执行结果

编译后生成a.out文件。

3.5 步骤四：运行程序

在a.out所在路径下，执行以下命令，运行a.out文件。

```
./a.out
```

执行结果

打印输出`threadDim`个 `Hello AI Card` 。

❓ 说明:

由于应用程序中通过sdaaSSetDevice指定了SPA0运行Kernel函数，因此目标SPA的threadDim个SPE均会执行一遍Kernel函数，故会打印threadDim个Hello AI Card。

[illegible]

4. 异构计算模型

4.1 概述

CPU处理器体系偏向于快速的用户反馈和调度控制，但其计算能力有限。在面向高并发、大规模的数据计算时，需要使用异构计算模型对计算进行加速，从而节省用户大批量数据计算所消耗的时间。使用SDAA C提供的异构计算模型，用户无需感知底层的硬件细节，即可正常使用太初AI加速卡提供的强大算力。

SDAA C异构计算模型是将应用程序中大量的计算任务，由主机端发送到设备端的计算核心阵列SPA执行，充分利用太初AI加速卡的硬件资源，提高程序的运行速度，缩短程序的运行时间。

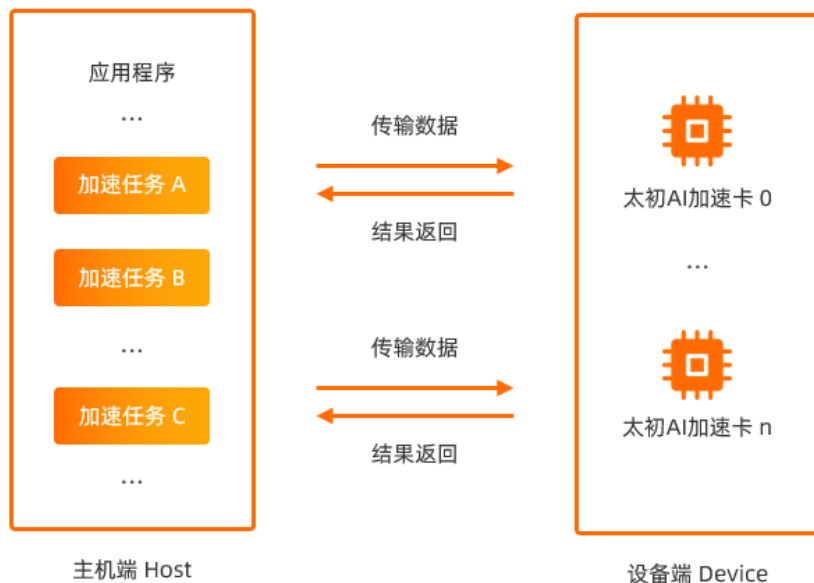


图1 异构计算模型

4.2 SPMD

SPMD (Single Program Multiple Data, 单程序多数据)，是一种用于任务并行的编程范式。其本质是将一个问题分解成若干个子问题，然后对其并行求解。

SDAA C遵循SPMD编程范式，同一计算核心阵列SPA内所有计算核心SPE运行同一份应用程序，每个SPE可以独立完成对子问题的并行求解。其核心在于对复杂计算任务的切分，并合理地将计算任务分配到不同的SPE内进行计算。

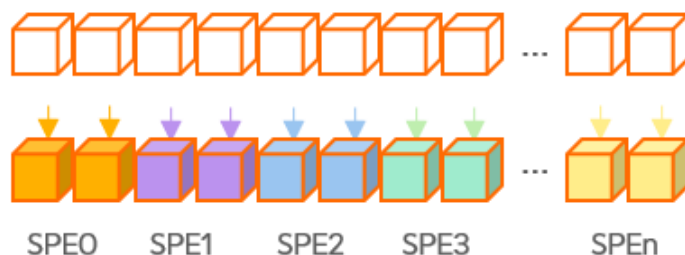


图1 基于SDAA C的SPMD编程范式

SDAA C提供了`threadIdx`和`threadDim`为每个SPE定制计算任务。其中：

- `threadIdx`：当前计算核心SPE的ID号，可以通过该关键字为目标SPE分配计算任务。
- `threadDim`：获取同一计算核心阵列SPA内计算核心SPE的总数。

编程示例

以数组计算为例，将计算任务分配到同一SPA的所有SPE内并行执行：

```
using namespace sdac;
#define SIZE 67

__device__ int g_data[SIZE];
__global__ void func()
{
    // 初始化原始数据
    for (int i = 0; i < SIZE; i++) {
        g_data[i] = i;
    }

    // SPE同步，确保所有的SPE都完成了数据的初始化
    sync_threads();

    // 可用的计算资源超过计算量时，每个SPE承担一次计算
    if (threadDim >= SIZE) {
        if (threadIdx < SIZE) {
            g_data[threadIdx]++;
        }
    } else {
        // 可用计算资源少于计算量时，每个SPE需承担多次计算
        // 例如当前计算量为67，假如共有32个SPE参与计算。则每个SPE需要至少承担两次计算任务
        int cal_time = SIZE / threadDim;
        for (int i = 0; i < cal_time; i++) {
            g_data[i * threadDim + threadIdx]++;
        }

        // 对于剩余的计算量使用部分SPE承担
        // 例如当前的计算量为67，共有32个SPE参与计算，每个SPE计算两次后，还剩余3次。则使用0, 1, 2三个SPE完成剩余的计算
        int remain_time = SIZE % threadDim;
        if ((threadIdx + 1) <= remain_time) {
            g_data[cal_time * threadDim + threadIdx]++;
        }
    }

    // SPE同步，确保所有的SPE都完成了子问题的求解
    sync_threads();

    // 打印输出计算结果
    for (int i = 0; i < SIZE; i++) {
        printf("SPE ID = %lu, g_data[%d] = %d\n", threadIdx, i, g_data[i]);
    }
}
```

上述应用程序中就通过SPMD编程范式完成对复杂任务的求解。

- SP (Single Program, 单程序)：所有参与计算的SPE都执行同一份代码，在本例中为func函数。
- MD (Multiple Data, 多数据)：通过 `threadIdx` 将数组进行切分，每个SPE仅需完成少量运算。

在本例中：SIZE为67，假如可用计算资源为32个SPE，计算资源少于计算量，每个SPE需要承担多次计算，如：SPE0需要计算数据g_data[0]、g_data[32]和g_data[64]、SPE1需要计算数据g_data[1]、g_data[33]和g_data[65]等。



图2 使用SPMD编程范式完成数组计算任务

具体计算任务的代码片段如下：

```
// 可用计算资源少于计算量时，每个SPE需承担多次计算
// 例如当前计算量为67，共有32个SPE参与计算。则每个SPE需要至少承担两次计算任务
int cal_time = SIZE / threadIdx;
for (int i = 0; i < cal_time; i++) {
    g_data[i * threadIdx + threadIdx]++;
}

// 对于剩余的计算量使用部分SPE承担
// 例如当前的计算量为67，共有32个SPE参与计算，每个SPE计算两次后，还剩余3次。则使用0, 1, 2三个SPE完成剩余的计算任务
int remain_time = SIZE % threadIdx;
if ((threadIdx + 1) <= remain_time) {
    g_data[cal_time * threadIdx + threadIdx]++;
}
```

代码中涉及部分关键字含义如下：

- `__device__`：运行空间标识符，修饰Device端的函数或变量，在Device端调用、运行，可参考 `__device__`。
- `__global__`：函数标识符，修饰Kernel函数，可参考 `__global__`。
- `sync_threads`：对同一计算核心阵列SPA内的所有或部分计算核心SPE进行同步操作，可参考 `sync_threads`。
- `printf`：打印接口，可参考 `printf`。

SPMD编程注意事项

考虑到程序的可扩展性，在为SPE分配计算任务时，应尽量考虑到以下几点：

- 负载均衡，每个SPE承担的计算量尽可能保持一致。
- 数据就近原则，SPE内用于计算的原始数据尽可能来自于当前SPE，减少同一计算核心阵列SPA内不同计算核心SPE之间的数据传递。
- 分配计算任务时，请提前做好规划，以便在计算量增加时可以做到更好地兼容与扩展。

4.3 Kernel函数

太初AI加速卡负责处理大规模的数据运算（可参考[SDAA异构并行模型](#)），计算核心SPE在SDAA C中相当于一个线程，用户可以使用SPE进行异构并行计算。

Kernel函数是由 `__global__` 关键字修饰的函数，在设备端执行（类似于线程代码）。在使用SDAA C时，主机端通过调用Kernel函数将大量的计算任务分配给SPE执行，从而提高数据的处理效率。

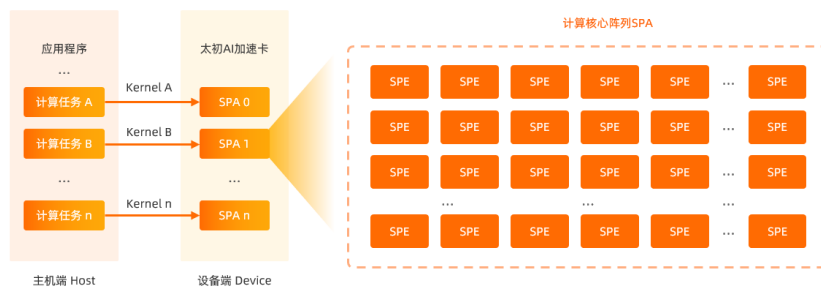


图1 Kernel函数

SDAA C编程语言：

- 通过 `__global__` 关键字标明Kernel函数，可参考[__global__关键字](#)。
- 在主程序中使用 `<<<...>>>` 语法调用Kernel函数，将计算任务发送到太初AI加速卡上执行计算，可参考[<<<...>>>语法](#)。
- 使用 `threadIdx` 标识执行计算的SPE，可参考[threadIdx关键字](#)。

以执行加法为例的Kernel函数：

```
// Kernel函数
__global__ void user_kernel(int *a)
{
    int tid = threadIdx;
    // 每个SPE执行一个加法操作，threadDim个SPE完成threadDim个加法操作
    a[tid] = a[tid] + a[tid];
    return;
}
```

4.4 存储空间

SDAA C的存储空间分为主机端内存、设备端内存、计算核心SPE片上存储空间SPM。

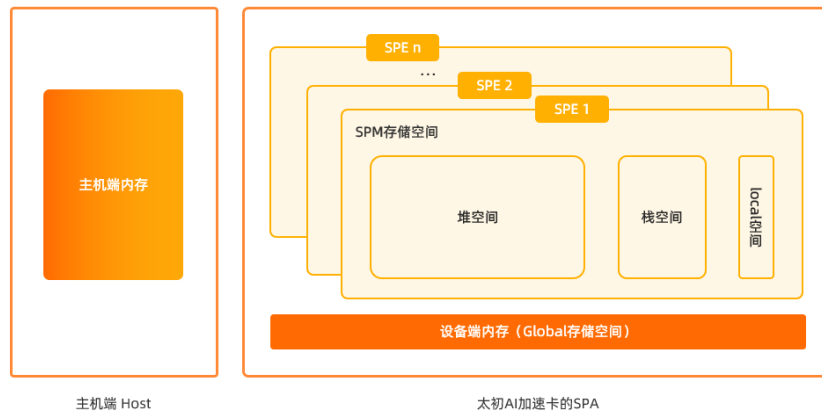


图1 存储模型

主机端内存

主机端内存是主机端的存储空间，用于存储需要在设备端进行计算的源数据。

申请和释放主机端内存：可以使用 `malloc`、`free` 函数，可参考C/C++语言的相关内容。

设备端内存（Global存储空间）

设备端内存又称设备端Global存储空间，是计算核心阵列SPA的共享存储。在使用计算核心SPE进行计算前，需要先将数据从主机端传输到设备端的Global存储空间。

Global存储空间可以使用以下方法进行管理：

- 使用 `__device__` 关键字申请Global存储空间。
- 使用 `sdaaMalloc`、`sdaaFree` 申请和释放Global存储空间。

Global存储空间使用示例：

- 使用 `__device__` 关键字分配Global存储空间。

```
// g_mem被创建在global存储空间内
__device__ int g_mem = 1;
```

- 使用 `sdaaMalloc` 申请Global存储空间。
- 使用 `sdaaFree` 释放申请的Global存储空间。

```
// 需要在Host端使用sdaaMalloc申请Global存储空间，使用sdaaFree释放申请的Global存储空间
#define size 100;
int main()
{
    int *h_mem = NULL;
    // 为h_mem申请Global存储空间
    sdaaMalloc((void **)&h_mem, size * sizeof(int));
```

```
...

// 释放h_mem
sdaaFree(h_mem);
}
```

SPM存储空间

每个计算核心SPE都有一块高速的片上本地局部数据存储空间SPM。SPM空间属于计算核心SPE的私有空间。对于计算核心SPE而言，访问Global存储空间，速度缓慢。在SPE执行计算任务的过程中，如果直接从SPM存储空间获取数据，计算效率将得到显著提升。

计算核心SPE运行时独占SPM存储空间，SPM存储空间可分为：

- **堆空间**：需要手动管理的一块动态内存空间，您可以通过**malloc**接口分配堆内存空间，通过**free**接口释放所申请的堆内存空间。可通过**get_heap_size**接口查询当前计算核心SPE的堆空间总大小。
- **栈空间**：程序运行时用于管理函数调用和局部变量的内存空间。可通过**get_stack_size**接口查询当前计算核心SPE的栈空间总大小。
- **local空间**：使用**__local__**关键字声明的空间，不支持变量初始化，可通过**get_local_size**接口查询当前计算核心SPE的local空间总大小。

? 说明：

在开启**--stack-on-global**选项后，**__local__**和**__scoped_local__**共享local空间。

SPM存储空间使用示例：

```
// 使用__local__关键字分配local空间
__local__ int a;

__device__ void user_spm()
{
    // 变量b被分配到了user_spm函数的栈空间内
    int b = 1;

    // 使用malloc接口动态分配堆空间
    int *c = (int *)malloc(16 * sizeof(int));

    // 使用free接口释放所申请的堆空间
    free(c);
}
```

存储空间可用大小

存储空间名称	存储空间大小	所属存储空间
堆空间	通过 <code>get_heap_size</code> 接口查询	SPM存储空间
栈空间	通过 <code>get_stack_size</code> 接口查询	- 默认为SPM存储空间 - 在开启<code>--stack-on-global</code>选项后，切换到Global存储空间
<code>local</code> 空间	通过 <code>get_local_size</code> 接口查询	SPM存储空间
<code>Global</code> 存储空间	16GB	Global存储空间

4.5 硬件架构

太初AI加速卡由多个计算核心阵列SPA组成，每个计算核心阵列SPA由`threadDim`个计算核心SPE和Global存储空间构成。每个计算核心SPE都拥有独立的计算、访存和控制功能，是一个独立的处理器核心，可以独立完成一个计算任务。每个计算核心SPE由FU、SU、SREG、VPU、VREG以及SPM存储空间组成。

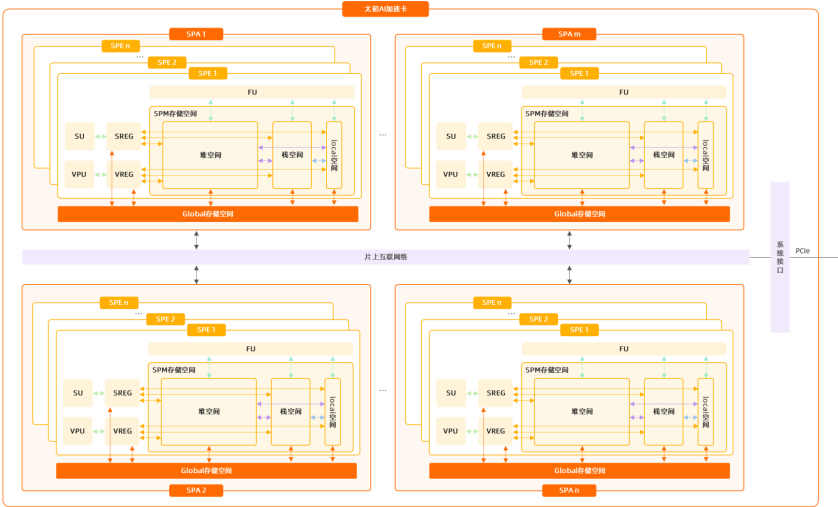


图1 太初AI加速卡

SU

SU（Scalar Unit）标量计算单元，用于执行标量运算的部件，可以完成算术运算、逻辑运算等标量运算。标量计算单元只能接收标量寄存器内的数据，在完成运算后将计算结果保存到标量寄存器内。

SREG

SREG（Scalar Register）标量寄存器，用于存储标量数据的寄存器，支持单个数据元素的读写操作。标量寄存器可以与SPM存储空间和Global存储空间交换数据。

VPU

VPU (Vector Processing Unit) 向量处理单元，用于执行向量运算的部件，可以完成众多**向量操作**，包括**向量存储操作**、**向量算术运算**、**向量类型转换**、**向量比较选择**等。向量处理单元只能接受向量寄存器内的数据，在完成向量操作后将计算结果保存到向量寄存器内。

VREG

VREG (Vector Register) 向量寄存器，用于存储向量数据的专用寄存器，支持多个标量数据元素的并行读写操作。向量寄存器可以与SPM存储空间和Global存储空间交换数据。

FU

FU (Function Unit) 运算功能单元，可以完成**矩阵乘**等计算。运算功能单元只能接收SPM存储空间的数据，在完成运算后将计算结果保存到SPM存储空间。

SPM存储空间

SPM存储空间是计算核心SPE内私有的存储空间，有较高的访存带宽，但是可用容量较小，需要合理利用有限的SPM存储空间以提升运行程序的性能。SDAA C将SPM存储空间进一步细分为：

- **堆空间**：可以与FU、SREG、VREG、栈空间、local空间以及SPA内Global存储空间交换数据。
- **栈空间**：可以与FU、SREG、VREG、堆空间、local空间以及SPA内Global存储空间交换数据。

? 说明：

在开启**--stack-on-global**选项，将函数栈从SPM存储空间切换到Global存储空间后，切换后的栈空间可以与SREG、VREG、堆空间、local空间以及Global存储空间交换数据。

- **local空间**：可以与FU、SREG、VREG、堆空间、栈空间以及SPA内Global存储空间交换数据。

Global存储空间

Global存储空间是计算核心阵列SPA内的内存空间，由计算核心阵列SPA内的所有计算核心SPE共享。Global存储空间有较大的存储容量，但是访存带宽较小，可以通过**数据搬运接口**将数据搬运到SPM存储空间内进行计算，以提升程序性能。Global存储空间可以与SREG、VREG、堆空间、栈空间以及local空间交换数据。

4.6 数据传输

数据传输管理主要包括：主机端内存与设备端内存（Global存储空间）之间的数据传递、同一计算核心阵列SPA的Global存储空间内的数据传递、Global存储空间与SPM存储空间之间的数据传递、同一计算核心阵列SPA内不同计算核心SPE之间SPM存储空间的数据传递、以及同一计算核心SPE的SPM存储空间内的数据传递。

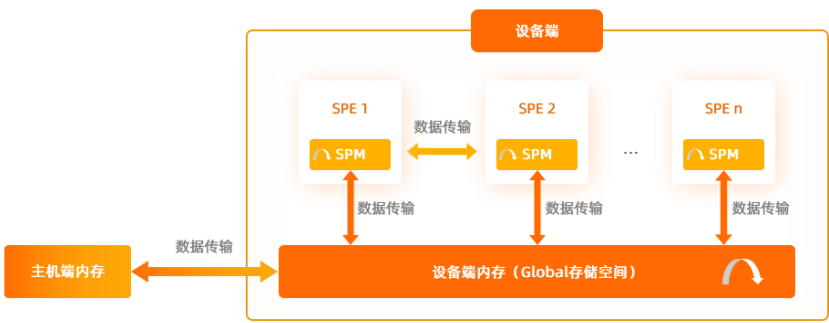


图1 数据传输

主机端内存与Global存储空间之间的数据传递

Kernel函数在设备端运行时所需要的源数据，需要从主机端获取。在进行数据传输前，需要在设备端申请相应大小的Global存储空间，主机端将数据传输到已分配的Global存储空间，完成数据计算后，再将计算结果传回主机端内存，并释放申请的Global存储空间。

主机端内存与Global存储空间的数据传递：

操作	函数
主机端内存与Global存储空间的数据传递	<code>sdaaMemcpy</code> ，可参考 sdaaMemcpy 。

同一计算核心阵列SPA的Global存储空间内的数据传递

同一计算核心阵列SPA内的Global存储空间和Global存储空间之间的数据传递，可参考[memcpy](#)。

Global存储空间与SPM存储空间之间的数据传递

为提升SPE的计算效率，需要将数据从Global存储空间传输到SPM存储空间进行计算。在进行数据传输前，需要在计算核心SPE上申请相应大小的SPM存储空间，设备端将数据传输到已分配的SPM存储空间进行计算，再将计算结果传回Global存储空间，并释放申请的SPM存储空间。

Global存储空间与SPM存储空间的数据传递：

操作	函数
Global存储空间与SPM存储空间的数据传递	<code>memcpy</code> ， <code>memcpy_async</code> 等接口，可参考 DMA数据搬运 。

不同计算核心SPE之间的数据传递

为使不同计算核心SPE之间的数据交换更加方便，SDAA C提供了RMA编程接口，用于搬运不同SPM存储空间之间的数据。RMA编程接口实现了计算核心SPE之间的高效数据搬运，且可以缓解SPM存储空间容量受限的问题，可参考[RMA数据搬运](#)。

在同一计算核心SPE的SPM存储空间内的数据传递

计算核心SPE可在其内部的SPM空间进行数据传递，可参考[memcpy](#)。

4.7 SDAARuntime接口

仅在主机端编写Kernel函数，太初AI加速卡并不能执行计算任务：

1. 没有指定运行Kernel函数的目标计算核心阵列SPA。
2. 没有申请设备端的Global存储空间。
3. 缺少计算所需的源数据。

因此，主机端通过SDAARuntime接口解决以上问题。SDAARuntime接口可以：

1. 指定Kernel函数运行在设备端具体哪一个计算核心阵列SPA上，可参考[sdaaSetDevice](#)。
2. 在设备端分配Global存储空间，申请Global存储空间可参考[sdaaMalloc](#)；释放Global存储空间可参考[sdaaFree](#)。
3. 将数据从主机端内存传递到设备端Global存储空间，可参考[sdaaMemcpy](#)。

5. 语言规范

5.1 语法

5.1.1 <<<...>>>

在主程序中使用 `<<<placeholder, stream>>>` 语法调用Kernel函数，`<<<placeholder, stream>>>` 共有两个参数：

```
size_t placeholder; // 暂无实际含义
sdaaStream stream; // SDAA流
```

其中：

- `placeholder`：目前暂无实际含义，支持设置任意值。
- `stream`：SDAA流（详细说明参考SDAA流（详情请参考《SDAARuntime用户手册》的**SDAA流**章节）），可缺省。

? 说明：

使用 `<<<...>>>` 调用kernel函数时，会同时调用`threadDim`个计算核心SPE进行计算，相当于同时启动`threadDim`个线程。

使用示例

```
__global__ void user_kernel(int *a, int *b, int *c)
{
    // 根据实际情况，编写Kernel函数
    ...
}

int main()
{
    ...
    // 调用核函数进行计算
    sdaaStream_t stream;
    sdaaStreamCreate(&stream);
    user_kernel<<<1, stream>>>(a, b, c);
    // 第二个参数可以不写: user_kernel<<<1>>>(a, b, c);
    ...
}
```

5.2 关键字

5.2.1 __host__

功能介绍

运行空间标识符，修饰Host端函数，在Host端调用、运行。

该关键字可以缺省。

使用示例

```
__host__ void add(int *a, int *b, int *c)
{
    *c = *a + *b;
}

void calc(int *a, int *b, int *c)
{
    add(a, b, c);
}
```

5.2.2 __global__

功能介绍

函数标识符，修饰Kernel函数。在Host端通过 `<<<...>>>` 调用，在Device端运行。

返回类型需要是void。

使用示例

```
__global__ void calc(int *a, int *b, int *c)
{
    ...
    *c = *a + *b;
    ...
}

int main()
{
    ...
    // 调用Kernel函数
    calc<<<1>>>>(a, b, c);
    ...
}
```

5.2.3 __device__

功能介绍

运行空间标识符，修饰Device端的函数或变量，在Device端调用、运行。可以被 `__device__` 或 `__global__` 修饰的函数调用。

使用示例

```
__device__ void add()
{
    ...
}

__device__ void add2()
{
    ...
    add();
    ...
}

__global__ void calc()
{
    ...
    add2();
    ...
}
```

5.2.4 __local__

功能介绍

变量标识符，修饰存放在SPM存储空间的变量。

只能作为全局变量使用，目前不支持初始化。可以通过`get_local_size`获取当前计算核心SPE可用`local空间`总大小。

使用示例

```
__local__ int num[512];
__local__ short n;
```

5.2.5 __scoped_local__

功能介绍

变量修饰符，修饰Device端函数栈变量。在开启`--stack-on-global`选项时，该关键字生效。生效后与`__local__`共享同一块SPM存储空间，可以通过`get_local_size`获取当前计算核心SPE可用`local空间`总大小。

? 说明:

当未启用 `--stack-on-global` 选项时，所有栈变量默认位于SPM存储空间中。

在这种情况下，编译器将忽略 `__scoped_local__` 关键字，因此被 `__scoped_local__` 修饰的变量将被处理为普通栈变量。

被 `__scoped_local__` 修饰的变量类似于普通的栈变量，在每个线程内，该变量：

- 只能在函数内访问。
- 具有函数内的生命周期。
- 支持初始化，且每次函数调用时初始化，若没有显式初始化，其初始值是未定义的。

但也与栈变量有重要区别，该变量：

- 驻留在SPM存储空间中，而不是在栈上。
- 拥有静态存储位置，其地址在程序开始运行时确定，并在程序结束前一直保持不变。
- 不支持在递归函数内使用此变量。

使用示例

```
// __global__ 或者 __device__ 修饰的Device端函数
__device__ void func()
{
    __scoped_local__ int num[512];
    __scoped_local__ short n = 1;
    ...
}
```

5.2.6 threadIdx

功能介绍

`threadIdx` 用于获取Device端计算核心SPE的ID，可以为每一个计算核心SPE定制不同的行为。

使用示例

```
__device__ void user_kernel()
{
    if (threadIdx == 0) {
        ...
    } else if (threadIdx == 1) {
        ...
    }
}
```

```
    } else {  
        ...  
    }  
}
```

5.2.7 threadDim

功能介绍

`threadDim` 用于获取同一计算核心阵列SPA内计算核心SPE的总数。

使用示例

```
__device__ void user_kernel()  
{  
    unsigned long thread_count = threadDim;  
}
```

5.3 数据类型

SDAA C支持标量数据类型和向量数据类型。

标量数据类型

SDAA C支持C/C++的基本数据类型，并在此基础上进行了扩展：

- 支持半精度浮点类型，修饰符为half，占用2字节。符合IEEE-754的存储标准，有1个符号位，5个指数位，10个尾数位。
- 支持bfloat类型，修饰符为bfloat16，占用2字节，有1个符号位，8个指数位，7个尾数位。

 说明：

bfloat16目前仅支持指针操作，具体可参考[simd_load_widen](#) 和[simd_store_narrow](#)。

向量数据类型

向量的各个元素互不相关，对当前向量元素的操作不会影响其它的向量元素。

修饰符	简介
intv16	有符号整型向量，由16个有符号整型组成。
uintv16	无符号整型向量，由16个无符号整型组成。

修饰符	简介
shortv32	有符号短整型向量，由32个有符号短整型组成。
ushortv32	无符号短整型向量，由32个无符号短整型组成。
halfv16	半精度浮点向量，由16个半精度浮点型组成。
floatv16	单精度浮点向量，由16个单精度浮点型组成。

5.4 头文件

SDAA C常用的头文件有：sdAA_atomic.h、sdAA_matmul.h、sdAA_transpose.h和sdAA_perf.h。

sdAA_atomic.h

该头文件定义了原子操作接口，详细信息可参考[原子操作](#)。

sdAA_matmul.h

该头文件定义了矩阵乘接口，详细信息可参考[矩阵乘](#)。

sdAA_transpose.h

该头文件定义了转置操作接口，详细信息可参考[转置操作](#)。

sdAA_perf.h

该头文件定义了性能相关接口，详细信息可参考[性能采样](#)。

sdAA_device_info.h

该头文件定义了设备端信息查询相关接口，详细信息可参考[设备信息查询](#)。

scl.h

该头文件定义了高层次API相关接口，详细信息可参考[高层次函数接口](#)。

5.5 命名空间

SDAA C目前提供两个命名空间：`sdAA` 和 `sdAA::scl`。

sdAA

SDAA C的基础命名空间，SDAA C提供的函数接口，如：[线程组](#)、[计算核心SPE同步](#)、[数据搬运](#)、[原子操作](#)等接口均在此命名空间内。

sdAA::scl

SDAA C为[高层次函数接口](#)所提供的命名空间。

5.6 常用SDAARuntime接口

SDAARuntime接口可以对太初AI加速卡进行设备管理、资源申请和资源释放等。

在SDAA C中常用的SDAARuntime接口如下表，更多且详细的SDAARuntime接口描述可参考SDAARuntime用户手册（详情请参考《SDAARuntime用户手册》的[API参考](#)章节）：

SDAA接口	功能介绍
sdAADeviceReset	销毁当前进程中当前设备上的所有已分配的资源并重置所有状态。
sdAASetDevice	指定Kernel函数运行在Device端具体哪一个计算核心阵列SPA上。
sdAAGetDevice	获取当前线程使用的设备。
sdAAGetDeviceCount	获取设备的数量。
sdAADeviceSynchronize	等待当前线程使用的设备完成之前的所有请求任务。
sdAAMalloc	申请设备端Global存储空间。
sdAAFree	释放设备端Global存储空间。
sdAAMemcpy	在主机端内存和设备端内存（Global存储空间）之间传输数据。
sdAAMemset	将Global存储空间的起始地址初始化为一个特定值。
sdAAStreamCreate	创建一个新的异步流。

使用示例

```
__global__ void calc(long *a)
{
    // 根据实际情况，编写Kernel函数
    ...
}

int main()
{
    int dev_count = 0;
    long a = 0;
    long *dev_a = NULL;
    // 1. 判断是否有可用设备
```

```
sdaaGetDeviceCount(&dev_count);
assert(0 < dev_count);
int device_id = 0;
// 2. 设置使用的设备
sdaaSetDevice(device_id);
// 3. 申请设备端Global存储空间
sdaaMalloc((void**>(&dev_a), sizeof(long));
// 4. 初始化Global存储空间
sdaaMemset(dev_a, 0, sizeof(long));
// 5. 从Host端内存拷贝数据到Device端的Global存储空间
sdaaMemcpy(dev_a, &a, sizeof(long), sdaaMemcpyHostToDevice);
// 6. 调用Kernel函数
calc<<<1>>>>(dev_a);
// 7. 等待Kernel函数执行结束
sdaaDeviceSynchronize();
// 8. 从Device端的Global存储空间拷贝计算结果到Host端内存
sdaaMemcpy(&a, dev_a, sizeof(long), sdaaMemcpyDeviceToHost);
// 9. 释放设备端Global存储空间
sdaaFree(dev_a);
return 0;
}
```

6. 环境变量

6.1 SDAA_SYNC_PRINT

`SDAA_SYNC_PRINT` 用于设置SPE按照何种顺序进行打印，帮助定位和解决问题。

名称	默认值	取值
SDAA_SYNC_PRINT	0	是否按照SPE编号从小到大的顺序进行打印： 0：按照SPE的执行顺序进行打印。 1：按照SPE编号从小到大的顺序进行打印。

7. 函数接口

7.1 概述

函数接口是编译器内部实现的函数，SDAA C内置了计算核心SPE同步、SPM内存分配和释放、数据搬运、原子操作、矩阵乘、向量操作和性能采样操作等函数接口，为用户提供简单、易用、功能丰富的编程接口，从而更高效地利用太初AI加速卡的强大算力。

7.2 线程组

7.2.1 概述

线程组是指对计算核心阵列SPA中的所有计算核心SPE进行组合，根据需要将若干个计算核心SPE组合为一个线程组，您可以对线程组内的计算核心SPE执行**计算核心SPE同步**、**Broadcast数据搬运**等操作。线程组由以下函数接口组成：

- **ThreadGroup**：用于配置线程组，可以将指定的计算核心SPE组合为一个线程组。
- **thread_group_set_mask**：以掩码的方式设置当前SPA内的各个SPE是否加入目标线程组。
- **thread_group_include**：在目标线程组内添加指定的计算核心SPE。
- **thread_group_exclude**：在目标线程组内删除指定的计算核心SPE。
- **thread_group_is_included**：判断指定的计算核心SPE是否在指定的线程组内。
- **thread_group_get_size**：获取指定线程组内计算核心SPE的数量。
- **thread_group_clear**：删除目标线程组内的所有计算核心SPE。

7.2.2 ThreadGroup

```
ThreadGroup()  
ThreadGroup(unsigned long *thread_group, unsigned int size)  
ThreadGroup(unsigned long mask, unsigned long position = 0)
```

功能介绍

用于配置线程组，可以将指定的计算核心SPE组合为一个线程组。

参数解释

参数名称	说明
thread_group	线程组，由计算核心SPE的ID组成。

参数名称	说明
size	参数thread_group数组的长度。  说明： size的最大长度为计算核心阵列SPA内计算核心SPE的数量，即threadDim。
mask	标识当前计算核心阵列SPA内的各个SPE是否加入指定的线程组内。每个bit位对应一个计算核心SPE，一个mask最多可表示64个SPE。 1: 目标SPE加入指定的线程组内。 0: 目标SPE不加入指定的线程组内。
position	标识SPE的范围，默认为0。position = （SPE的ID号） / 64。例如： - position为0: SPE的ID号为0~63。 - position为1: SPE的ID号为64~127。

注意事项

需要引入sdac命名空间。

使用示例1

使用数组的方式将ID为偶数的计算核心组成一个线程组。

```
using namespace sdac;
__device__ void create_thread_group()
{
    constexpr unsigned long num_thread = threadDim / 2;
    unsigned long arr[num_thread];
    for (int i = 0; i < num_thread; i++) {
        arr[i] = i * 2;
    }
    // 配置线程组，使用所有线程里的偶数号线程
    ThreadGroup thread_group(arr, num_thread);

    // 其他代码逻辑
    ...
}
```

使用示例2

使用掩码的方式将ID为偶数的计算核心组成一个线程组。

```
using namespace sdAA;
__device__ void create_thread_group()
{
    // 配置线程组，使用所有线程里的奇数号线程
    ThreadGroup thread_group(0x55555555);

    // 其他代码逻辑
    ...
}
```

7.2.3 thread_group_set_mask

```
void thread_group_set_mask(ThreadGroup &thread_group, unsigned long mask, unsigned long position = 0)
```

功能介绍

以掩码的方式设置当前SPA内的各个SPE是否加入目标线程组。

参数解释

参数	说明
thread_group	ThreadGroup 类型，标识指定的线程组。
mask	标识当前计算核心阵列SPA内的各个SPE是否加入指定的线程组内。每个bit位对应一个计算核心SPE，一个mask最多可表示64个SPE。 1：目标SPE加入指定的线程组内。 0：目标SPE不加入指定的线程组内。
position	标识SPE的范围，默认为0。position = （SPE的ID号） / 64。例如： - position为0：SPE的ID号为0~63。 - position为1：SPE的ID号为64~127。

返回值

无。

注意事项

需要引入sdAA命名空间。

使用示例

```
using namespace sdaa;
__device__ void func()
{
    ThreadGroup thread_group;
    thread_group_set_mask(thread_group, 0xFF00);    // 将SPE8~SPE15分配为一个线程组。

    // 其他代码逻辑

    thread_group_set_mask(thread_group, 0xFF0);    // 修改线程组, 将SPE4~SPE11分配到指定的线程组内

    // 其他代码逻辑
}
```

7.2.4 thread_group_include

```
void thread_group_include(ThreadGroup &thread_group, unsigned long thread_id)
```

功能介绍

在目标线程组内添加指定的计算核心SPE。

参数解释

参数名称	说明
thread_group	ThreadGroup类型，标识指定的线程组。
thread_id	需要加入目标线程组的计算核心SPE的ID（threadIdx）。

返回值

无。

注意事项

需要引入sdaa命名空间。

使用示例

```
using namespace sdaa;
__device__ void func()
{
    unsigned long arr[] = {0, 1, 2, 3};
    ThreadGroup thread_group(arr, 4);
}
```



```
// 其他代码逻辑

// 将SPE4 ~ SPE7加入线程组
for (unsigned long i = 4; i <= 7; i++) {
    thread_group_include(thread_group, i);
}

// 其他代码逻辑
}
```

7.2.5 thread_group_exclude

```
void thread_group_exclude(ThreadGroup &thread_group, unsigned long thread_id)
```

功能介绍

在目标线程组内删除指定的计算核心SPE。

参数解释

参数名称	说明
thread_group	ThreadGroup 类型，标识指定的线程组。
thread_id	需要在目标线程组删除的计算核心SPE的ID（threadIdx）。

返回值

无。

注意事项

需要引入sdac命名空间。

使用示例

```
using namespace sdAA;
__device__ void func()
{
    unsigned long arr[] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
    ThreadGroup thread_group(arr, 9);

    // 其他代码逻辑

    // 从指定线程组内删除SPE8
    thread_group_exclude(thread_group, 8);

    // 其他代码逻辑
}
```

7.2.6 thread_group_is_included

```
bool thread_group_is_included(const ThreadGroup &thread_group, unsigned long thread_id)
```

功能介绍

判断指定的计算核心SPE是否在指定的线程组内。

参数解释

参数名称	说明
thread_group	ThreadGroup 类型，标识指定的线程组。
thread_id	待确认的计算核心SPE的ID（threadIdx）。

返回值

返回值类型	说明
bool	- true：指定的计算核心SPE在当前的线程组内。 - false：指定的计算核心SPE不在当前的线程组内。

注意事项

需要引入sdAA命名空间。

使用示例

```
using namespace sdAA;
__device__ void func()
```

```
{
    unsigned long arr[] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
    ThreadGroup thread_group(arr, 9);

    // 其他代码逻辑

    // 判断当前SPE是否在指定线程组内
    if (thread_group_is_included(thread_group, threadIdx)) {
        // 其他代码逻辑
    }
}
```

7.2.7 thread_group_get_size

```
unsigned int thread_group_get_size(const ThreadGroup &thread_group)
```

功能介绍

获取指定线程组内计算核心SPE的数量。

参数解释

参数名称	说明
thread_group	ThreadGroup 类型，标识指定的线程组。

返回值

返回值类型	说明
unsigned int	指定线程组内计算核心SPE的数量。

注意事项

需要引入sdac命名空间。

使用示例

```
using namespace sdac;
__device__ void func()
{
    unsigned long arr[] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
    ThreadGroup thread_group(arr, 9);

    // 其他代码逻辑

    // 获取当前线程组内计算核心SPE的数量
    unsigned int size = thread_group_get_size(thread_group);
}
```

```
    // 其他代码逻辑
}
```

7.2.8 thread_group_clear

```
void thread_group_clear(ThreadGroup &thread_group)
```

功能介绍

删除目标线程组内的所有计算核心SPE。

参数解释

参数名称	说明
thread_group	ThreadGroup类型，标识指定的线程组。

返回值

无。

注意事项

需要引入sdac命名空间。

使用示例

```
using namespace sdac;
__device__ void func()
{
    unsigned long arr[] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
    ThreadGroup thread_group(arr, 9);

    // 其他代码逻辑

    // 清空指定的线程组
    thread_group_clear(thread_group);

    // 其他代码逻辑
}
```

7.3 计算核心SPE同步

7.3.1 概述

计算核心SPE同步可以对同一计算核心阵列SPA内的所有或者部分计算核心SPE的执行进度进行同步。计算核心SPE同步由以下函数接口组成：

- **sync_threads**：对同一计算核心阵列SPA内的所有或者部分计算核心SPE进行同步操作。

7.3.2 sync_threads

```
void sync_threads()
void sync_threads(const ThreadGroup &thread_group)
```

功能介绍

对同一计算核心阵列SPA内的所有或部分计算核心SPE进行同步操作，即当所有或设置的计算核心SPE都运行到 **sync_threads** 时才会执行下一条语句：

- 不携带参数：对同一计算核心阵列SPA内的所有计算核心SPE进行同步操作。
- 携带参数：对同一计算核心阵列SPA内由线程组指定的计算核心SPE进行同步操作。

参数解释

参数名称	说明
thread_group	ThreadGroup 类型，标识指定的线程组，包含需要同步的所有SPE的 threadIdx。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- 不携带参数：**sync_threads** 函数需要在当前所有SPE都可以执行到的代码段内执行。
- 携带参数：**sync_threads** 函数需要在指定的SPE都可以执行到的代码段内执行。

使用示例1

```
#define SIZE 100

__local__ int input[SIZE];
__global__ void func()
{
    // 为所有的计算核心SPE初始化input数组
    for (int i = 0; i < SIZE; i++) {
        input[i] = i * threadIdx;
```

```
}

// 在当前所有计算核心SPE都可以执行到的代码段内执行同步操作，以确保当所有计算核心SPE执行到该位置时，才会执行下一条语句
sdaa::sync_threads();
}
```

使用示例2

```
using namespace sdaa;
#define SIZE 100
#define THREAD_GROUP_SIZE 6

__local__ int input[SIZE];
__global__ void func()
{
    // 为所有的计算核心SPE初始化input数组
    for (int i = 0; i < SIZE; i++) {
        input[i] = i * threadIdx;
    }

    // 表示0, 5, 9, 12, 18, 22六个线程进行同步操作
    unsigned long arr[] = {0, 5, 9, 12, 18, 22};
    // 创建线程组
    ThreadGroup thread_group(arr, THREAD_GROUP_SIZE);
    // 对指定的部分计算核心SPE执行同步操作
    sync_threads(thread_group);
}
```

使用示例3

```
using namespace sdaa;
#define SIZE 100
#define THREAD_GROUP_SIZE 6

__local__ int input[SIZE];

__global__ void func()
{
    // 为所有的计算核心SPE初始化input数组
    for (int i = 0; i < SIZE; i++) {
        input[i] = i * threadIdx;
    }

    // 表示0, 5, 9, 12, 18, 22六个线程进行同步操作
    unsigned long arr[] = {0, 5, 9, 12, 18, 22};
    // 创建线程组
    ThreadGroup thread_group(arr, THREAD_GROUP_SIZE);

    // 对指定的部分计算核心SPE执行同步操作，仅需要同步的计算核心SPE执行同步操作。
    if (thread_group_is_included(thread_group, threadIdx)) {
        sync_threads(thread_group);
    }
}
```

7.4 SPM内存分配和释放

7.4.1 概述

SPM内存分配和释放接口可以对SPM内存进行精准管理。SPM内存分配和释放接口主要由以下函数接口组成：

- **malloc**：根据对SPM存储空间的使用需求，判断SPM存储空间是否可用，并根据判断结果返回失败或可用空间的起始地址。
- **free**：对于给定的SPM存储空间地址，释放以此地址为起始位置的SPM存储空间。

7.4.2 malloc

```
void *malloc(size_t size, MallocDirect direct = AddressLowToHigh)
```

功能介绍

根据给定的内存大小和分配方向，判断SPM存储空间中是否有空间满足需求。如果满足，则分配SPM存储空间并返回该段空间的起始地址；否则返回零地址。

参数解释

参数名称	说明
size	需要分配的SPM存储空间大小，单位：Byte。
direct	枚举类型MallocDirect，支持以下两种内存分配方向： • AddressLowToHigh：自低地址向高地址方向分配，默认内存分配方向。 • AddressHighToLow：自高地址向低地址方向分配。

返回值

返回值类型	说明
void *	如果SPM中有空间可以满足本次分配需求，返回可用空间的起始地址。否则，返回零地址。

注意事项

- 用户的SPM需求空间会自动64B对齐。例如，用户需求4B的SPM存储空间，接口会按64B判断并分配；用户需求100B的SPM存储空间，接口会按128B判断并分配。
- 该接口可分配的最大SPM存储空间为0x3AC00字节。

- 对于从低地址向高地址分配的需求，接口分配的起始地址为0x1400。对于从高地址向低地址分配的需求，接口分配的起始地址为0x3C000。
- `malloc` 一般与`free`配合使用。

使用示例

调用 `malloc` 分别从SPM低地址和高地址开始分配4B大小的SPM存储空间，使用后调用`free`释放这部分存储空间。

```
__device__ void test()
{
    size_t size = 4;
    // 申请4个Byte的空间，分配给指针addr_low
    void *addr_low = malloc(size);
    // 地址从高到底申请4个Byte的空间，分配给指针addr_high
    void *addr_high = malloc(size, AddressHighToLow);

    // 使用addr_low, addr_high存储空间
    ...

    // 释放申请的SPM空间的指针
    free(addr_low);
    free(addr_high);
}
```

7.4.3 free

```
void free(void *addr)
```

功能介绍

释放以给定地址为起始的SPM存储空间。

参数解释

参数名称	说明
addr	需要释放的SPM存储空间的起始地址。

返回值

无。

注意事项

- 接口会自动读取SPM存储空间的使用情况，并根据给定的地址，自动判断需要释放的空间大小，无需用户指定。

- `free` 一般与`malloc`配合使用。

使用示例

调用`malloc`分别从SPM低地址和高地址开始分配4B大小的SPM存储空间，使用后调用 `free` 释放这部分存储空间。

```
__device__ void test()
{
    size_t size = 4;
    // 申请4个Byte的空间，分配给指针addr_low
    void *addr_low = malloc(size);
    // 地址从高到底申请4个Byte的空间，分配给指针addr_high
    void *addr_high = malloc(size, AddressHighToLow);

    // 使用addr_low, addr_high存储空间
    ...

    // 释放申请的SPM空间的指针
    free(addr_low);
    free(addr_high);
}
```

7.5 内存初始化

7.5.1 memset

```
void *memset(void *ptr, int value, size_t num)
```

功能介绍

将Global存储空间或SPM存储空间初始化为指定的值。

参数解释

参数名称	说明
ptr	指向需要初始化的内存区域的指针。
value	初始化的值，通常为0或者-1。
num	初始化内存区域的大小，单位：Byte。

返回值

返回值类型	说明
void *	返回一个等同于参数 <code>ptr</code> 的指针。

注意事项

无。

使用示例1

对Global存储空间进行初始化。

```
__device__ int g_arr[100];

__global__ void func()
{
    if (threadIdx != 0) {
        return;
    }

    // 对Global内存g_arr进行初始化操作
    memset(g_arr, 0, 100 * sizeof(int));

    // 打印输出
    for (int i = 0; i < 100; i++) {
        printf("g_arr[%d] = %d\n", i, g_arr[i]);
    }
}
```

使用示例2

对SPM存储空间进行初始化。

```
__global__ void func()
{
    if (threadIdx != 0) {
        return;
    }

    // 在SPM存储空间中申请一段内存空间
    int *arr = (int *)malloc(100 * sizeof(int));

    // 对SPM内存arr进行初始化操作
    memset(arr, 0, 100 * sizeof(int));

    // 打印输出
    for (int i = 0; i < 100; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    // 释放申请的内存空间
}
```

```
    free(arr);  
}
```

7.6 数据搬运

7.6.1 DMA数据搬运

7.6.1.1 概述

DMA（Direct Memory Access）数据搬运是指同一SPA内的Global存储和Global存储之间、同一SPE内的SPM存储之间以及同一SPA内的Global存储和SPM存储之间的数据流动。

DMA数据搬运可以分为阻塞型DMA数据搬运和非阻塞型DMA数据搬运：

- 阻塞型DMA数据搬运接口：使用简单，但性能不佳。
- 非阻塞型DMA数据搬运接口：使用相对复杂，但性能较好。

阻塞型DMA数据搬运

阻塞型DMA数据搬运由以下函数接口组成：

- **memcpy**：常规阻塞型数据搬运，数据搬运完成才可以执行下一条运算。
- **check_memcpy**：验证 `memcpy` 输入参数的合法性。
- **memcpy_stride**：带跨步功能的阻塞型数据搬运，数据搬运完成才可以执行下一条运算。
- **check_memcpy_stride**：验证 `memcpy_stride` 输入参数的合法性。

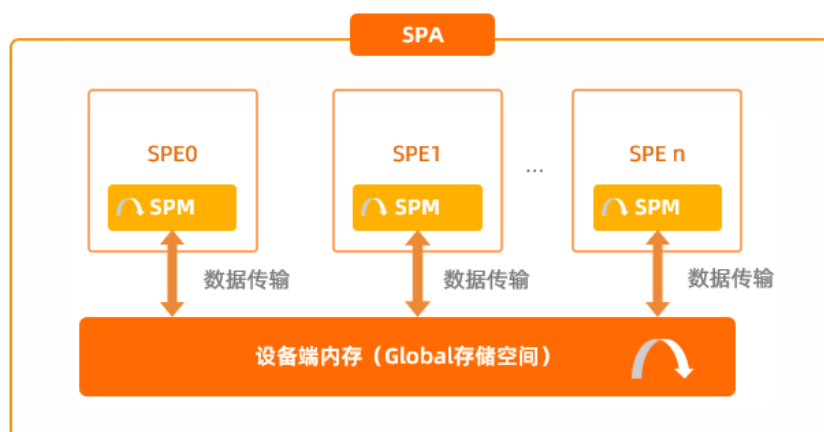


图1 阻塞型DMA数据搬运

非阻塞型DMA数据搬运

非阻塞型DMA数据搬运仅适用于Global存储和SPM存储之间的数据搬运，由以下函数接口组成：

- **MemcpyHandle**：用于控制不同的非阻塞型DMA数据搬运组。
- **memcpy_async**：非阻塞型数据搬运，数据搬运的同时可以执行下一条运算。

- `memcpy_wait`: 非阻塞型数据搬运同步接口，确保所有非阻塞数据搬运完成。
- `check_memcpy_async`: 验证`memcpy_async`输入参数的合法性。

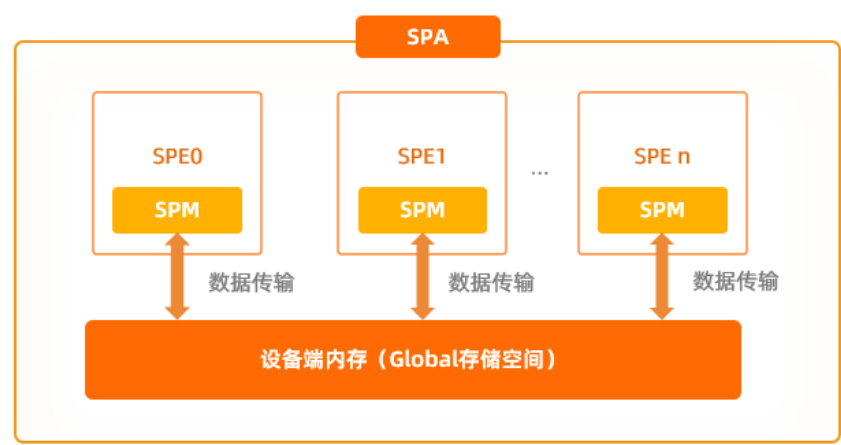


图2 非阻塞型DMA数据搬运

7.6.1.2 阻塞型DMA数据搬运

7.6.1.2.1 memcpy

```
void *memcpy(void *dst, const void *src, size_t size)
```

功能介绍

完成同一SPA内的Global存储和Global存储之间、同一SPE内的SPM存储之间以及同一SPA内的Global存储和SPM存储之间的阻塞型数据搬运。

参数解释

参数名称	说明
dst	数据搬运的目的地址，可以为Global存储或SPM存储的地址。
src	数据搬运的源地址，可以为Global存储或SPM存储的地址。

参数名称	说明
size	数据搬运量大小，单位：Byte。 数据搬运量的大小受源地址及目标地址所在空间限制，由于memcpy接口源地址和目的地址不允许有重叠，因此不同存储空间的数据搬运量大小会有所区别，存储空间可用空间大小可参照不同存储空间可用大小： - 若源地址与目标地址在同一块内存空间，则理论最大数据搬运量为：所在内存空间的二分之一。 - 若源地址与目标地址不在同一块内存空间，则理论最大数据搬运量为：空间较小者的空间大小。

返回值

返回值类型	说明
void *	返回一个等同于dst的指针。

注意事项

- 暂不支持不同SPE内的SPM存储之间的数据搬运，可参考[RMA数据搬运](#)和[Broadcast数据搬运](#)。
- 当前memcpy接口会额外占用SPM堆空间：由于Global存储和Global存储之间的数据搬运需要以SPM存储空间作为中转，会临时占用SPM 20KB的堆空间，数据搬运完成后释放。

使用示例

使用DMA数据搬运阻塞接口完成Global存储空间与SPM存储空间之间，Global存储空间之间以及SPM存储空间之间的数据搬运。

```
// 初始化Global存储上input中的数据
__device__ int input[32] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
                            17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

// 初始化Global存储上input_1中的数据
__device__ int input_1[32] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                              0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

// 初始化SPM存储中的数据
__device__ void init_array_get(int *array, int num, int val)
{
    for (int i = 0; i < num; i++) {
        array[i] = val;
    }
}

// 将数据从Global存储拷贝到SPM存储
__global__ void test_global_to_spm()
{

```

```

    int *tmp = (int *)malloc(32 * sizeof(int));
    if (threadIdx == 0) {
        memcpy(tmp, &input[0], 32 * sizeof(int));
    }
    free(tmp);
}

// 将数据从SPM存储拷贝到Global存储
__global__ void test_spm_to_global()
{
    int *tmp = (int *)malloc(32 * sizeof(int));
    init_array_get(tmp, 32, 9);
    if (threadIdx == 0) {
        memcpy(&input[0], tmp, 32 * sizeof(int));
    }
    free(tmp);
}

// 将数据从Global存储拷贝到Global存储
__global__ void test_global_to_global()
{
    if (threadIdx == 0) {
        memcpy(&input_1[0], &input[0], 32 * sizeof(int));
    }
}

// 将数据从SPM存储拷贝到同一计算核心上的SPM存储
__global__ void test_spm_to_spm()
{
    if (threadIdx == 0) {
        int *tmp = (int *)malloc(32 * sizeof(int));
        int *tmp_1 = (int *)malloc(32 * sizeof(int));
        init_array_get(tmp, 32, 9);
        init_array_get(tmp_1, 32, 0);
        memcpy(tmp, tmp_1, 32 * sizeof(int));
        free(tmp);
        free(tmp_1);
    }
}

```

7.6.1.2.2 check_memcpy

```
unsigned int check_memcpy(const void *dst, const void *src, size_t size)
```

功能介绍

验证memcpy输入参数的合法性，通过返回值判断输入参数是否满足特定的输入要求，如：源地址与目的地址是否有重叠、当前接口占用SPM堆空间是否超出剩余可用堆空间大小等。

参数解释

参数名称	说明
dst	数据搬运的目的地址，可以为Global存储或SPM存储的地址。
src	数据搬运的源地址，可以为Global存储或SPM存储的地址。
size	数据搬运量大小，单位：Byte。

返回值

返回值类型	说明
unsigned int	参数合法性验证结果，每个Bit代表不同的状态信息。对应枚举型MemcpyStatus，表示合法性验证的状态信息，当前可表示状态有： - MEMCPY_SUCC：参数无误。 - MEMCPY_ERR_ADDR_ILLEGAL：源地址或目的地址不合法，需要是有效地址。 - MEMCPY_ERR_MEM_OVERLAP：源地址和目的地址存在重叠。 - MEMCPY_ERR_SPM_OVERSIZE：当前接口占用SPM堆空间超出剩余可用堆空间。

返回的具体状态信息可参照如下表格：

第0位	第1位	第2位	第3位
0：参数无误。	第0位是0时，此位保留。	第0位是0时，此位保留。	第0位是0时，此位保留。
1：参数有误。	第0位是1时，此位： 0：正常。 1：错误。源地址或目的地址不合法，需要是有效地址。	第0位是1时，此位： 0：正常。 1：错误。源地址和目的地址存在重叠。	第0位是1时，此位： 0：正常。 1：错误。当前接口占用SPM堆空间超出剩余可用堆空间。

注意事项

需要引入sdac命名空间。

使用示例

使用不携带跨步参数的check_memcpy接口，对数据搬运参数进行安全检查。

```
__global__ void test(void *dst, const void *src, size_t size)
{
    if (threadIdx != 0) { return; }
    // 获取check_memcpy函数的返回值
    int check_val = sdaa::check_memcpy(dst, src, size);

    // 通过check_memcpy的返回值确认memcpy参数是否合法, 合法则进行memcpy操作
    if (check_val == MEMCPY_SUCC) {
        memcpy(dst, src, size);
    }
}
```

7.6.1.2.3 memcpy_stride

```
void *memcpy_stride(void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d)
```

功能介绍

完成同一SPA内的Global存储和Global存储之间、同一SPE内的SPM存储之间以及同一SPA内的Global存储和SPM存储之间带跨步功能的阻塞型数据搬运。

参数解释

参数名称	说明
dst	数据搬运的目的地址，可以为Global存储或SPM存储的地址。
src	数据搬运的源地址，可以为Global存储或SPM存储的地址。
size	数据搬运量大小，单位：Byte。  说明： 数据搬运量的大小受源地址和目的地址所在空间限制。 由于memcpy_stride接口的源地址和目的地址不允许有重叠，因此不同存储空间的数据搬运量大小会有所区别。 Global存储空间和SPM存储空间的可用空间大小，可参考存储空间章节的存储空间可用大小： - 若源地址与目的地址在同一块存储空间，则理论最大数据搬运量为：所在存储空间的二分之一。 - 若源地址与目的地址不在同一块存储空间，则理论最大数据搬运量为：空间较小者的空间大小。

参数名称	说明
section_id	一维数据搬运单元的大小，单位Byte，需要是4B的整数倍。
stride_id	MemoryStride结构体类型变量，设置源和目标存储空间的一维跨步单元大小，单位：Byte。  说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运，且当前接口的 <code>section_id</code> 参数无效。

返回值

返回值类型	说明
void *	返回一个等同于dst的指针。

注意事项

- 需要引入sdAA命名空间。
- 暂不支持不同SPE内的SPM存储之间的数据搬运，可参考[RMA数据搬运](#)和[Broadcast数据搬运](#)。
- 当前memcpy_stride接口会额外占用SPM堆空间情况：由于Global存储和Global存储之间的数据搬运需要以SPM存储空间作为中转，会临时占用SPM 20KB的堆空间，数据搬运完成后释放。
- DMA数据搬运的跨步功能不支持在SPM存储空间上设置跨步单元的大小。因此，SPM存储空间向SPM存储空间仅支持搬运连续数据。

使用示例

使用DMA数据搬运阻塞接口的跨步功能完成Global存储空间与SPM存储空间之间，Global存储空间之间以及SPM存储空间之间的数据搬运。

```
using namespace sdAA;
// 初始化Global存储上input中的数据
__device__ int input[32] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
                            17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

// 初始化Global存储上input_1中的数据
__device__ int input_1[32] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                              0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
```

```

// 初始化Global存储上input_2中的数据
__device__ int input_2[32] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
                              0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

// 初始化SPM存储中的数据
__device__ void init_array_get(int *array, int num, int val)
{
    for (int i = 0; i < num; i++) {
        array[i] = val;
    }
}

// 使用跨步功能将数据从Global存储空间拷贝到SPM存储空间
__device__ void test_global_to_spm()
{
    // 数据搬运大小是64B
    constexpr size_t size = 16 * sizeof(int);
    // 每个数据搬运单元大小是8B
    constexpr size_t section = 2 * sizeof(int);
    // Global上的跨步大小是8B
    constexpr size_t glbmem_stride = 2 * sizeof(int);
    // 对总数据块申请SPM空间
    int *spm_arr = (int *)malloc(size);
    // 在调用memcpy_stride 接口时, 填入目的地址, 源地址, 数据块大小和MemoryStride对象
    memcpy_stride(spm_arr, input, size, section, {0, glbmem_stride});

    // 其他代码逻辑
    ...
    // 释放SPM空间
    free(spm_arr);
}

// 使用跨步功能将数据从SPM存储空间拷贝到Global存储空间
__device__ void test_spm_to_global()
{
    if (threadIdx != 0) {
        return;
    }
    // 数据搬运大小是64B
    constexpr size_t size = 16 * sizeof(int);
    // 每个数据搬运单元大小是8B
    constexpr size_t section = 2 * sizeof(int);
    // Global上的跨步大小是8B
    constexpr size_t glbmem_stride = 2 * sizeof(int);
    // 对总数据块申请SPM空间
    int *spm_arr = (int *)malloc(size);
    // 初始化源数据
    init_array_get(spm_arr, 16, 10);
    // 在调用memcpy_stride 接口时, 填入目的地址, 源地址, 数据块大小和stride对象
    memcpy_stride(input_2, spm_arr, size, section, {glbmem_stride, 0});

    // 其他代码逻辑
    ...
    // 释放SPM空间
    free(spm_arr);
}

// 使用跨步功能将数据从Global存储空间拷贝到Global存储空间

```

```

__device__ void test_global_to_global()
{
    if (threadIdx != 0) {
        return;
    }
    // 数据搬运大小是64B
    constexpr size_t size = 16 * sizeof(int);
    // 每个数据搬运单元大小是8B
    constexpr size_t section = 2 * sizeof(int);
    // Global上的跨步大小是8B
    constexpr size_t glbmem_stride = 2 * sizeof(int);
    // 在调用memcpy_stride 接口时, 填入目的地址, 源地址, 数据块大小和stride对象
    memcpy_stride(input_2, input, size, section, {glbmem_stride, glbmem_stride});

    // 其他代码逻辑
    ...
}

// 使用跨步功能将数据从SPM存储空间拷贝到SPM存储空间
__device__ void test_spm_to_spm()
{
    // 数据搬运大小是64B
    constexpr size_t size = 16 * sizeof(int);
    // 每个数据搬运单元大小是8B
    constexpr size_t section = 2 * sizeof(int);
    // Global上的跨步大小是8B
    constexpr size_t glbmem_stride = 2 * sizeof(int);
    // 给源数据和目的地址数据申请SPM空间
    int *spm_arr_1 = (int *)malloc(section_num * section);
    int *spm_arr_2 = (int *)malloc(section_num * section);
    // 初始化源数据和目的地址数据
    init_array_get(spm_arr_1, 16, 10);
    init_array_get(spm_arr_2, 16, 0);
    // 在调用memcpy_stride 接口时, 填入目的地址, 源地址, 数据块大小和stride对象
    memcpy_stride(spm_arr_2, spm_arr_1, size, section, {0, 0});

    // 其他代码逻辑
    ...
    // 释放SPM空间
    free(spm_arr_2);
    free(spm_arr_1);
}

```

7.6.1.2.4 check_memcpy_stride

```

unsigned int check_memcpy_stride(const void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d)

```

功能介绍

验证**memcpy_stride**输入参数的合法性, 通过返回值判断输入参数是否满足特定的输入要求, 如: 源地址与目的地址是否有重叠、当前接口占用SPM堆空间是否超出剩余可用堆空间大小等。

参数解释

参数名称	说明
dst	数据搬运的目的地址，可以为Global存储或SPM存储的地址。
src	数据搬运的源地址，可以为Global存储或SPM存储的地址。
size	数据搬运量的大小，单位：Byte。 ? 说明： 数据搬运量的大小受源地址和目的地址所在空间限制。 由于memcpy_stride接口的源地址和目的地址不允许有重叠，因此不同存储空间的数据搬运量大小会有所区别。 Global存储空间和SPM存储空间的可用空间大小，可参考存储空间章节的存储空间可用大小： - 若源地址与目的地址在同一块存储空间，则理论最大数据搬运量为：所在存储空间的二分之一。 - 若源地址与目的地址不在同一块存储空间，则理论最大数据搬运量为：空间较小者的空间大小。
section_ld	一维数据搬运单元的大小，单位：Byte，需要是4B的整数倍。
stride_ld	MemoryStride结构体类型变量，设置源和目标存储空间的一维跨步单元大小，单位：Byte。 ? 说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运，且当前接口的 <code>section_ld</code> 参数无效。

返回值

返回值类型	说明
unsigned int	参数合法性验证结果，每个Bit代表不同的状态信息。对应枚举型MemcpyStatus，表示合法性验证的状态信息，当前可表示状态有： - MEMCPY_SUCC：参数无误。 - MEMCPY_ERR_ADDR_ILLEGAL：源地址或目的地址不合法，需要是有效地址。 - MEMCPY_ERR_MEM_OVERLAP：源地址和目的地址存在重叠。 - MEMCPY_ERR_SPM_OVERSIZE：当前接口占用SPM堆空间超出剩余可用堆空间。 - MEMCPY_ERR_STRIDE_PARAM：跨步参数错误。

返回的具体状态信息可参照如下表格：

第0位	第1位	第2位	第3位	第4位
0：参数无误。	第0位是0时，此位保留。	第0位是0时，此位保留。	第0位是0时，此位保留。	第0位是0时，此位保留。
1：参数有误。	第0位是1时，此位： 0：正常。 1：错误。源地址或目的地址不合法，需要是有效地址。	第0位是1时，此位： 0：正常。 1：错误。源地址和目的地址存在重叠。	第0位是1时，此位： 0：正常。 1：错误。当前接口占用SPM堆空间超出剩余可用堆空间。	第0位是1时，此位： 0：正常。 1：跨步参数错误。

注意事项

需要引入sdAA命名空间。

使用示例

使用带跨步参数的check_memcpy_stride接口，对数据搬运参数进行安全检查。

```
using namespace sdAA;
__global__ void test(void *dst, const void *src, size_t size)
{
    if (threadIdx != 0) {
        return;
    }
    // 每个数据搬运单元大小是8B
    constexpr size_t section = 2 * sizeof(int);
    // Global上的跨步大小是8B
    constexpr size_t glbmem_stride = 2 * sizeof(int);
    // 获取check_memcpy_stride函数的返回值
```

```
int check_val = check_memcpy_stride(dst, src, size, section, {0, glbmem_stride});

// 通过check_memcpy_stride的返回值确认memcpy参数是否合法, 合法则进行带跨步的memcpy操作
if (check_val == MEMCPY_SUCC) {
    // 在调用memcpy_stride 接口时, 填入目的地址, 源地址, 数据块大小和stride对象
    memcpy_stride(dst, src, size, section, {0, glbmem_stride});
}
}
```

7.6.1.3 非阻塞型DMA数据搬运

7.6.1.3.1 MemcpyHandle及配置接口

7.6.1.3.1.1 概述

MemcpyHandle是非阻塞DMA数据搬运的句柄, 可以控制不同**memcpy_async**搬运操作所属的非阻塞型DMA数据搬运组。MemcpyHandle由以下函数接口组成:

- **MemcpyHandle**: 控制不同**memcpy_async**搬运操作所属的非阻塞型DMA数据搬运组。

7.6.1.3.1.2 MemcpyHandle

```
MemcpyHandle::MemcpyHandle()
```

功能介绍

用于控制不同**memcpy_async**搬运操作所属的非阻塞型DMA数据搬运组。同一搬运组内的所有 **memcpy_async** 搬运操作可以只调用一次**memcpy_wait**进行同步。如果不指定 **MemcpyHandle** , 则使用系统默认的非阻塞型DMA数据搬运组。

注意事项

需要引入sdac命名空间。

使用示例

```
using namespace sdac;
__device__ void func()
{
    // 创建MemcpyHandle类型变量
    MemcpyHandle handle;

    // 执行非阻塞型DMA操作
}
```

7.6.1.3.2 memcpy_async

```
void *memcpy_async(void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d,
MemcpyDirect direct)
void *memcpy_async(void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d,
MemcpyDirect direct, MemcpyHandle &handle)
```

功能介绍

完成同一SPA内的Global存储和SPM存储之间的非阻塞型数据搬运。可以通过 `MemcpyHandle` 控制所属的非阻塞型DMA数据搬运组，若没有使用 `MemcpyHandle`，则使用系统默认的非阻塞型DMA数据搬运组。

参数解释

参数名称	说明
dst	数据搬运的目的地址，可以为Global存储或SPM存储的地址，需要4B对齐。
src	数据搬运的源地址，可以为Global存储或SPM存储的地址，需要4B对齐。
size	数据搬运量大小，单位：Byte，需要是4B的整数倍。 数据搬运量的大小受SPM存储空间大小限制，可参照不同存储空间可用大小，理论最大数据搬运量为：对应SPM存储空间的大小。
section_1d	一维数据搬运单元大小，单位Byte，需要是4B的整数倍。
stride_1d	MemoryStride 结构体类型变量，设置源和目标存储空间的一维跨步单元大小，单位：Byte。 ? 说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运，且参数 <code>section_1d</code> 无效。
direct	枚举类型MemcpyDirect，支持以下两种数据搬运模式： - MemcpyGlobalToSpm：Global存储向SPM存储的数据搬运。 - MemcpySpmToGlobal：SPM存储向Global存储的数据搬运。
handle	(可选) MemcpyHandle 类型变量，用于控制不同的非阻塞型DMA数据搬运组。

返回值

返回值类型	说明
void *	返回一个等同于dst的指针。

注意事项

- 需要引入sdac命名空间。
- 暂不支持以下功能：
 - 同一计算核心上SPM存储之间的非阻塞型数据搬运。
 - Global存储和Global存储之间的非阻塞型数据搬运。
 - 不同计算核心的SPM存储之间的数据搬运，可参考[RMA数据搬运](#)和[Broadcast数据搬运](#)。
- Global和SPM的存储空间地址需要4B对齐。
- `memcpy_async` 需要与[memcpy_wait](#)配合使用。

使用示例1

以Global存储和SPM存储上长度为32个int的数组为例，介绍如何使用 `memcpy_async` 和[memcpy_wait](#)将Global存储上的数据非阻塞拷贝到SPM存储。


```

using namespace sdAA;
// 初始化Global存储上input中的数据
__device__ int input[32] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
                            17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

// 初始化SPM存储中的数据
__device__ void init_array_get(int *array, int num, int val)
{
    for (int i = 0; i < num; i++) {
        array[i] = val;
    }
}

// 将数据从Global存储拷贝到SPM存储
__global__ void test_global_to_spm_async()
{
    int *tmp = (int *)malloc(32 * sizeof(int));
    init_array_get(tmp, 32, 9);
    if (threadIdx == 0) {
        // 非阻塞型拷贝, 填入目的地址, 源地址, 搬运数据量, 搬运方向是Global到SPM
        memcpy_async(tmp, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm);
    }

    // 等待直到完成所有的非阻塞型数据搬运
    memcpy_wait();
    free(tmp);
}

```

使用示例2

以Global存储和SPM存储上两个长度为32个int的数组为例, 介绍如何使用**MemcpyHandle**、`memcpy_async` 和 `memcpy_wait`将Global存储上的数据非阻塞拷贝到SPM存储。

```

using namespace sdAA;
// 初始化Global存储上input中的数据
__device__ int input[32] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
                            17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

// 初始化SPM存储中的数据
__device__ void init_array_get(int *array, int num, int val)
{
    for (int i = 0; i < num; i++) {
        array[i] = val;
    }
}

// 将数据从Global存储拷贝到SPM存储
__global__ void test_global_to_spm_async()
{
    // 申请SPM上的空间
    int *tmp1 = (int *)malloc(32 * sizeof(int));
    int *tmp2 = (int *)malloc(64 * sizeof(int));
    int *tmp3 = (int *)malloc(64 * sizeof(int));
    // 给申请的SPM空间做数据初始化
    init_array_get(tmp1, 32, 9);
}

```

```

init_array_get(tmp2, 32, 9);
init_array_get(tmp3, 32, 9);
// 声明2个MemcpyHandle
MemcpyHandle handle1, handle2;
if (threadIdx == 0) {
    // 非阻塞拷贝
    memcpy_async(tmp1, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm); //
    // 使用默认组
    memcpy_async(tmp2, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle1); //
    // 使用handle1组
    memcpy_async(tmp3, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle1); //
    // 使用handle1组
    memcpy_async(tmp2 + 32, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle2); //
    // 使用handle2组
    memcpy_async(tmp3 + 32, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle2); //
    // 使用handle2组
}

// 等待直到完成所有的非阻塞数据搬运
memcpy_wait(); // 等待默认组上所有非阻塞拷贝操作完成
memcpy_wait(handle1); // 等待handle1组上所有非阻塞拷贝操作完成
memcpy_wait(handle2); // 等待handle2组上所有非阻塞拷贝操作完成

free(tmp3);
free(tmp2);
free(tmp1);
}

```

使用示例3

以Global存储和SPM存储上两个长度为200、数据类型为short的数组为例，介绍如何使用**MemoryStride**、**MemcpyHandle**、**memcpy_async** 和**memcpy_wait**将SPM存储上的数据以跨步的方式搬运到Global存储空间。

```

using namespace sdac;
#define SIZE 200
#define STRIDE 4
#define SECTION 4
#define SECTION_NUM 20
#define DATA_SIZE 80
__device__ short g_dst[SIZE];

__global__ void func(short *d_dst, short *d_src)
{
    if (threadIdx != 0) {
        return;
    }

    // 在SPM存储空间申请内存
    short *spm_src = (short *)malloc(SIZE * sizeof(short));
    for (int i = 0; i < SIZE; i++) {
        g_dst[i] = d_dst[i]; // 初始化Global存储空间变量
        spm_src[i] = d_src[i]; // 初始化SPM存储空间变量
    }

    // 声明handle

```

```
MemcpyHandle handle;

// 配置跨步参数, 进行非阻塞跨步拷贝
memcpy_async(g_dst, spm_src, DATA_SIZE * sizeof(short),
             SECTION * sizeof(short), {STRIDE * sizeof(short), 0},
             MemcpySpmToGlobal, handle);
memcpy_wait(handle); // 等待handle组上的非阻塞拷贝完成

// 其他代码逻辑

free(spm_src);
}
```

7.6.1.3.3 memcpy_wait

```
void memcpy_wait()
void memcpy_wait(MemcpyHandle &handle)
```

功能介绍

非阻塞数据搬运同步，确保之前所有的数据搬运完成。可以通过 `MemcpyHandle` 控制所属的非阻塞型DMA数据搬运组，若没有使用 `MemcpyHandle`，则使用系统默认的非阻塞型DMA数据搬运组。

参数解释

参数名称	说明
handle	(可选) <code>MemcpyHandle</code> 类型变量，用于控制不同的非阻塞型DMA数据搬运组。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- `memcpy_wait` 需要与`memcpy_async`配合使用。

使用示例1

以Global存储和SPM存储上长度为32个int的数组为例，介绍如何使用`memcpy_async`和 `memcpy_wait` 将Global存储上的数据非阻塞拷贝到SPM存储。

```
using namespace sdac;
// 初始化Global存储上input中的数据
__device__ int input[32] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
                           17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};
```

```

// 初始化SPM存储中的数据
__device__ void init_array_get(int *array, int num, int val)
{
    for (int i = 0; i < num; i++) {
        array[i] = val;
    }
}

// 将数据从Global存储拷贝到SPM存储
__global__ void test_global_to_spm_async()
{
    int *tmp = (int *)malloc(32 * sizeof(int));
    init_array_get(tmp, 32, 9);
    if (threadIdx == 0) {
        // 非阻塞拷贝
        memcpy_async(tmp, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm);
    }

    // 等待默认组上所有非阻塞拷贝操作完成
    memcpy_wait();
    free(tmp);
}

```

使用示例2

以Global存储和SPM存储上两个长度为32个int的数组为例，介绍如何使用MemcpyHandle、memcpy_async和memcpy_wait 将Global存储上的数据非阻塞拷贝到SPM存储。

```

using namespace sdac;
// 初始化Global存储上input中的数据
__device__ int input[32] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
                             17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};

// 初始化SPM存储中的数据
__device__ void init_array_get(int *array, int num, int val)
{
    for (int i = 0; i < num; i++) {
        array[i] = val;
    }
}

// 将数据从Global存储拷贝到SPM存储
__global__ void test_global_to_spm_async()
{
    // 在SPM上申请数据空间
    int *tmp1 = (int *)malloc(32 * sizeof(int));
    int *tmp2 = (int *)malloc(64 * sizeof(int));
    int *tmp3 = (int *)malloc(64 * sizeof(int));
    // 给申请的数据空间初始化数据
    init_array_get(tmp1, 32, 9);
    init_array_get(tmp2, 32, 9);
    init_array_get(tmp3, 32, 9);
    MemcpyHandle handle1, handle2;
    if (threadIdx == 0) {
        // 非阻塞拷贝

```

```
    memcpy_async(tmp1, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm); //
// 使用默认组
    memcpy_async(tmp2, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle1); //
// 使用handle1组
    memcpy_async(tmp3, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle1); //
// 使用handle1组
    memcpy_async(tmp2 + 32, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle2); //
// 使用handle2组
    memcpy_async(tmp3 + 32, &input[0], 32 * sizeof(int), 0, {0, 0}, MemcpyGlobalToSpm, handle2); //
// 使用handle2组
}

// 等待直到完成所有的非阻塞数据搬运
memcpy_wait(); // 等待默认组上所有非阻塞拷贝操作完成
memcpy_wait(handle1); // 等待handle1组上所有非阻塞拷贝操作完成
memcpy_wait(handle2); // 等待handle2组上所有非阻塞拷贝操作完成

free(tmp3);
free(tmp2);
free(tmp1);
}
```

7.6.1.3.4 check_memcpy_async

```
unsigned int check_memcpy_async(const void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d, MemcpyDirect direct)
unsigned int check_memcpy_async(const void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d, MemcpyDirect direct, MemcpyHandle &handle)
```

功能介绍

验证**memcpy_async**输入参数的合法性，通过返回值判断输入参数是否满足特定的输入要求，如：源地址与目的地址是否对齐、数据搬运量是否符合要求等。

参数解释

参数名称	说明
dst	数据搬运的目的地址，可以为Global存储或SPM存储的地址，需要4B对齐。
src	数据搬运的源地址，可以为Global存储或SPM存储的地址，需要4B对齐。

参数名称	说明
size	数据搬运量的大小，单位：Byte，需要是4B的整数倍。  说明： 数据搬运量的大小受SPM存储空间大小限制，可参考存储空间可用大小章节获取对应的堆空间、栈空间、local空间等的大小。 理论最大数据搬运量为：对应SPM存储空间的大小。
section_id	一维数据搬运单元大小，单位：Byte，需要是4B的整数倍。
stride_id	MemoryStride结构体类型变量，设置源地址和目的地址存储空间的一维跨步单元大小，单位：Byte。  说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运，且当前接口的 <code>section_id</code> 参数无效。
direct	枚举类型MemcpyDirect，支持以下两种数据搬运模式： - MemcpyGlobalToSpm：Global存储空间到SPM存储空间的数据搬运。 - MemcpySpmToGlobal：SPM存储空间到Global存储空间的数据搬运。
handle	（可选）MemcpyHandle类型变量，用于控制不同的非阻塞型DMA数据搬运组。

返回值

返回值类型	说明
unsigned int	参数合法性验证结果，每个Bit代表不同的状态信息。对应枚举型MemcpyAsyncStatus，当前可表示状态有： - MEMCPY_ASYNC_SUCC：参数无误。 - MEMCPY_ASYNC_ERR_ADDR_UNALIGNED：源地址或目的地址不满足对齐要求。 - MEMCPY_ASYNC_ERR_SIZE_ILLEGAL：数据搬运量不满足要求。 - MEMCPY_ASYNC_ERR_DIRECT_ILLEGAL：数据搬运方向不满足要求。 - MEMCPY_ASYNC_ERR_STRIDE_PARAM：跨步参数错误。

返回的具体状态信息可参照如下表格：

第0位	第1位	第2位	第3位	第4位
0：参数无误	第0位是0时，此位保留	第0位是0时，此位保留	第0位是0时，此位保留	第0位是0时，此位保留
1：参数有误	第0位是1时，此位： 0：正常。 1：错误。源地址或目的地址不满足对齐要求。	第0位是1时，此位： 0：正常。 1：错误。数据搬运量不满足要求。	第0位是1时，此位： 0：正常。 1：错误。数据搬运方向不满足要求。	第0位是1时，此位： 0：正常。 1：错误。跨步参数错误。

注意事项

需要引入sdAA命名空间。

使用示例

使用带跨步参数的check_memcpy_async接口，对数据搬运参数进行安全检查。

```
using namespace sdAA;
__device__ void func(void *dst, const void *src, size_t size)
{
    if (threadIdx != 0) {
        return;
    }
    // 每个数据搬运单元大小是8B
    constexpr size_t section = 2 * sizeof(int);
    // Global上的跨步大小是8B
    constexpr size_t glbmem_stride = 2 * sizeof(int);
    // 创建MemcpyHandle类的对象
```

```
MemcpyHandle handle;
// 获取check_memcpy_async函数的返回值
int check_val = check_memcpy_async(dst, src, size, section, {0, glbmem_stride}, MemcpyGlobalToSpm,
handle);

// 通过check_memcpy_async的返回值确认memcpy_async参数是否合法, 合法则进行memcpy_async操作
if (check_val == MEMCPY_ASYNC_SUCC) {
    memcpy_async(dst, src, size, section, {0, glbmem_stride}, MemcpyGlobalToSpm, handle);

    // 其他代码逻辑
    ...

    memcpy_wait(handle);
}
}
```

7.6.2 RMA数据搬运

7.6.2.1 概述

RMA（Remote Memory Access）数据搬运是指同一计算核心阵列SPA内两个不同计算核心SPE的SPM存储之间的数据流动。

RMA数据搬运可以分为阻塞型RMA数据搬运和非阻塞型RMA数据搬运：

- 阻塞型RMA数据搬运：使用简单，但性能不佳。
- 非阻塞型RMA数据搬运：使用相对复杂，但性能较好。

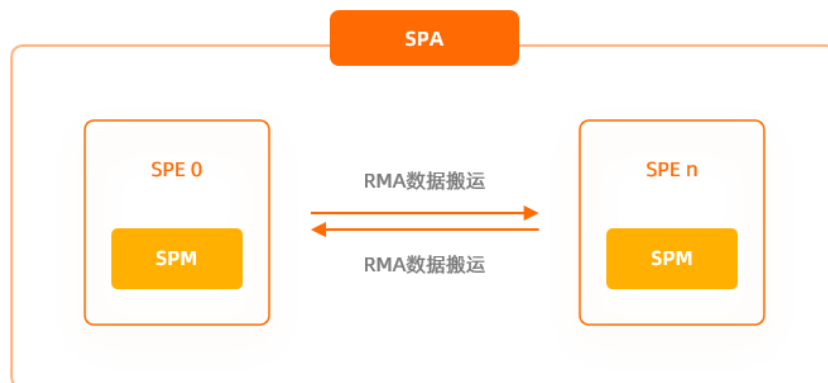


图1 RMA数据搬运

阻塞型RMA数据搬运

阻塞型RMA操作由以下函数接口组成：

- **rma_get**：在同一计算核心阵列SPA内，将远端计算核心SPE中指定的SPM存储数据以阻塞的方式搬运到当前计算核心SPE中指定的SPM存储空间内。
- **rma_put**：在同一计算核心阵列SPA内，将当前计算核心SPE中指定的SPM存储数据以阻塞的方式搬运到远端计算核心SPE中指定的SPM存储空间内。

非阻塞型RMA数据搬运

非阻塞型RMA操作由以下函数接口组成：

- **RmaHandle**：用于存储远端计算核心SPE的ID。
- **rma_set_thread_id**：配置远端计算核心SPE的ID。
- **rma_set_thread_group**：配置由多个远端计算核心SPE的ID组成的**线程组**。
- **rma_async_get**：将远端计算核心SPE中指定的SPM存储空间数据非阻塞拷贝到当前计算核心SPE中指定的SPM存储空间内。
- **rma_async_put**：将当前计算核心SPE中指定的SPM存储空间数据非阻塞拷贝到远端计算核心SPE中指定的SPM存储空间内。
- **rma_complete**：等待当前计算核心SPE完成非阻塞型RMA数据搬运。
- **rma_wait**：等待远端计算核心SPE完成非阻塞型RMA数据搬运。

7.6.2.2 阻塞型RMA数据搬运

7.6.2.2.1 rma_get

```
void rma_get(void *local_addr, const void *remote_addr, size_t size, unsigned long local_id, unsigned long remote_id)
```

功能介绍

在同一计算核心阵列SPA内，将远端计算核心SPE中指定的SPM存储数据以阻塞的方式搬运到当前计算核心SPE中指定的SPM存储空间内。

参数解释

参数名称	说明
local_addr	数据搬运的目的地址，当前计算核心的SPM存储空间地址。
remote_addr	数据搬运的源地址，远端计算核心的SPM存储空间地址。
size	数据搬运大小，单位：Byte。 数据搬运量的大小受SPM存储空间大小限制，可参照 不同存储空间可用大小 ，理论最大数据搬运量为：源地址和目标地址所在空间较小者的空间大小。
local_id	发起RMA操作的计算核心ID。
remote_id	远端计算核心的ID。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- SPM存储空间地址需要4B对齐。
- 数据搬运的数据量需要是4B的整数倍。
- 仅当前SPE和远端SPE需要执行该函数接口，其他SPE不能执行。
- 使用**malloc**分配SPM存储空间时，需要在所有涉及RMA数据搬运的SPE都可以执行到的代码区域内进行，并且要确保所有SPE分配的SPM空间首地址相同。正确的使用方法，可参考使用示例2。

提示：

在使用malloc分配SPM存储空间之前，如果已经对涉及RMA数据搬运的SPE申请过大小不同的SPM存储空间，请先释放这部分SPM内存空间，以确保所有涉及RMA数据搬运的SPE申请的SPM存储空间的首地址相同。

使用示例1

将1号计算核心中指定的SPM地址空间中的数据拷贝到0号计算核心指定的SPM地址空间内。

```
using namespace sdac;
#define SIZE 100

// 使用__local__ 分配的SPM存储空间
__local__ int local_addr[SIZE];
__local__ int remote_addr[SIZE];

__global__ void func()
{
    // 初始化0号计算核心中local_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            local_addr[i] = 0;
        }
    }

    // 初始化1号计算核心中remote_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            remote_addr[i] = i;
        }
    }

    // 仅0号计算核心和1号计算核心执行该函数接口
    if ((threadIdx == 0) || (threadIdx == 1)) {
        rma_get(local_addr, remote_addr, SIZE * sizeof(int), 0, 1);
    }
}
```

```

    }

    // 阻塞接口执行完后打印0号计算核心中local_addr的内容
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            printf("local_addr[%d] = %d\n", i, local_addr[i]);
        }
    }
}

```

使用示例2

将1号计算核心中指定的SPM地址空间中的数据拷贝到0号计算核心指定的SPM地址空间内。

```

using namespace sdac;
#define SIZE 100

__global__ void func()
{
    // 使用malloc申请SPM空间
    int *spm_addr = (int *)malloc(SIZE * sizeof(int));

    // 初始化0号计算核心中spm_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            spm_addr[i] = 0;
        }
    }

    // 初始化1号计算核心中remote_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            spm_addr[i] = i;
        }
    }

    // 仅0号计算核心和1号计算核心执行该函数接口
    if ((threadIdx == 0) || (threadIdx == 1)) {
        rma_get(spm_addr, spm_addr, SIZE * sizeof(int), 0, 1);
    }

    // 阻塞接口执行完后打印0号计算核心中spm_addr的内容
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            printf("spm_addr[%d] = %d\n", i, spm_addr[i]);
        }
    }

    free(spm_addr);
}

```

7.6.2.2.2 rma_put

```
void rma_put(const void *local_addr, void *remote_addr, size_t size, unsigned long local_id, unsigned long remote_id)
```

功能介绍

在同一计算核心阵列SPA内，将当前计算核心SPE中指定的SPM存储数据以阻塞的方式搬运到远端计算核心SPE中指定的SPM存储空间内。

参数解释

参数名称	说明
local_addr	数据搬运的源地址，当前计算核心的SPM存储空间地址。
remote_addr	数据搬运的目的地址，远端计算核心的SPM存储空间地址。
size	数据搬运大小，单位：Byte。 数据搬运量的大小受SPM存储空间大小限制，可参照不同存储空间可用大小，理论最大数据搬运量为：源地址和目标地址所在空间较小者的空间大小。
local_id	发起RMA操作的计算核心ID。
remote_id	远端计算核心的ID。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- SPM存储空间地址需要4B对齐。
- 数据搬运的数据量需要是4B的整数倍。
- 仅当前SPE和远端SPE需要执行该函数接口，其他SPE不能执行。
- 使用malloc分配SPM存储空间时，需要在所有涉及RMA数据搬运的SPE都可以执行到的代码区域内进行，并且要确保所有SPE分配的SPM空间首地址相同。正确的使用方法，可参考使用示例2。

提示：

在使用malloc分配SPM存储空间之前，如果已经对涉及RMA数据搬运的SPE申请过大小不同的SPM存储空间，请先释放这部分SPM内存空间，以确保所有涉及RMA数据搬运的SPE申请的SPM存储空间的首地址相同。

使用示例1

将0号计算核心中指定的SPM地址空间中的数据拷贝到1号计算核心指定的SPM地址空间内。

```
using namespace sdAA;
#define SIZE 100

// 使用__local__ 分配的SPM存储空间
__local__ int local_addr[SIZE];
__local__ int remote_addr[SIZE];

__global__ void func()
{
    // 初始化0号计算核心中local_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            local_addr[i] = i;
        }
    }

    // 初始化1号计算核心中remote_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            remote_addr[i] = 0;
        }
    }

    // 仅0号计算核心和1号计算核心执行该函数接口
    if ((threadIdx == 0) || (threadIdx == 1)) {
        rma_put(local_addr, remote_addr, SIZE * sizeof(int), 0, 1);
    }

    // 阻塞接口执行完后打印1号计算核心中remote_addr的内容
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            printf("remote_addr[%d] = %d\n", i, remote_addr[i]);
        }
    }
}
```

使用示例2

将0号计算核心中指定的SPM地址空间中的数据拷贝到1号计算核心指定的SPM地址空间内。

```
using namespace sdAA;
#define SIZE 100

__global__ void func()
{
    // 使用malloc申请SPM空间
    int *spm_addr= (int *)malloc(SIZE * sizeof(int));

    // 初始化0号计算核心中spm_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
```

```
        spm_addr[i] = i;
    }
}

// 初始化1号计算核心中spm_addr的值
if (threadIdx == 1) {
    for (int i = 0; i < SIZE; i++) {
        spm_addr[i] = 0;
    }
}

// 仅0号计算核心和1号计算核心执行该函数接口
if ((threadIdx == 0) || (threadIdx == 1)) {
    rma_put(spm_addr, spm_addr, SIZE * sizeof(int), 0, 1);
}

// 阻塞接口执行完后打印1号计算核心中spm_addr的内容
if (threadIdx == 1) {
    for (int i = 0; i < SIZE; i++) {
        printf("spm_addr[%d] = %d\n", i, spm_addr[i]);
    }
}

free(spm_addr);
}
```

7.6.2.3 非阻塞型RMA数据搬运

7.6.2.3.1 RmaHandle及配置接口

7.6.2.3.1.1 概述

RmaHandle是非阻塞型RMA数据搬运的句柄，由以下函数接口组成：

- **RmaHandle**：用于存储远端计算核心SPE的ID。
- **rma_set_thread_id**：配置远端计算核心SPE的ID。
- **rma_set_thread_group**：配置由多个远端计算核心SPE的ID组成的线程组。

7.6.2.3.1.2 RmaHandle

```
RmaHandle::RmaHandle()
```

功能介绍

用于存储远端计算核心SPE的ID。

注意事项

- 需要引入sdAA命名空间。

- 使用 `RmaHandle` 创建变量时需要在当前计算核心SPE和远端计算核心SPE都可以执行到的代码段内创建。
- 在创建完 `RmaHandle` 类型的变量后和执行非阻塞型RMA数据搬运之前需要使用`sync_threads`函数接口在计算核心间进行一次同步操作。

使用示例

```
using namespace sdAA;
__global__ void func()
{
    // 创建RmaHandle类型变量
    RmaHandle handle;

    // 其他代码逻辑

    // 计算核心之间进行一次同步操作
    sync_threads();

    // 执行非阻塞型RMA操作
}
```

7.6.2.3.1.3 rma_set_thread_id

```
void rma_set_thread_id(RmaHandle &handle, unsigned long remote_thread_id)
```

功能介绍

配置远端计算核心SPE的ID。

参数解释

参数名称	说明
handle	<code>RmaHandle</code> 类型变量，用于存储远端计算核心的ID。
remote_thread_id	远端计算核心的ID。

返回值

无。

注意事项

- 需要引入sdAA命名空间。
- 如果使用 `rma_set_thread_id` 前，已使用`rma_set_thread_group`为同一个`RmaHandle`类型变量配置了由多个远端SPE的ID组成的线程组，则使用 `remote_thread_id` 为`RmaHandle`类型变量配置远端SPE的ID

后，**RmaHandle**类型变量中的线程组仅保留当前配置的一个远端SPE的ID。

使用示例

SPE 0向SPE 2发起非阻塞型RMA数据搬运。

```
using namespace sdac;
__device__ void func()
{
    RmaHandle handle;

    // 其他代码逻辑

    // 计算核心之间进行一次同步操作
    sync_threads();

    if (threadIdx == 0) {
        // 为0号计算核心配置远端ID
        rma_set_thread_id(handle, 2);

        // 执行非阻塞型RMA操作
    }

    if (threadIdx == 2) {
        // 为2号计算核心配置远端ID
        rma_set_thread_id(handle, 0);

        // 执行非阻塞型RMA等待操作
    }
}
```

7.6.2.3.1.4 rma_set_thread_group

```
void rma_set_thread_group(RmaHandle &handle, ThreadGroup *remote_thread_group)
```

功能介绍

配置由多个远端计算核心SPE的ID组成的**线程组**。

? 说明：

配置由多个远端计算核心SPE的ID组成的线程组建议配合rma_wait使用，等待线程组中的SPE完成非阻塞型RMA数据搬运。

参数解释

参数名称	说明
handle	RmaHandle 类型变量，用于存储远端计算核心的ID。
remote_thread_group	由多个远端计算核心SPE的ID组成的 线程组 。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- 如果使用 `rma_set_thread_group` 为**RmaHandle**类型变量配置线程组前，已使用**rma_set_thread_id**为同一个**RmaHandle**类型变量配置了一个远端SPE的ID，则使用 `rma_set_thread_group` 配置线程组后，**RmaHandle**类型变量仅保留当前配置的线程组。

使用示例

SPE 0向SPE 2发起非阻塞型RMA数据搬运。

```
using namespace sdac;
__device__ void func()
{
    RmaHandle handle;

    // 其他代码逻辑

    // 计算核心之间进行一次同步操作
    sync_threads();

    if (threadIdx == 0) {
        // 为0号计算核心配置远端ID
        rma_set_thread_id(handle, 2);

        // 执行非阻塞型RMA操作
    }

    if (threadIdx == 1) {
        // 为1号计算核心配置远端ID
        rma_set_thread_id(handle, 2);

        // 执行非阻塞型RMA操作
    }

    if (threadIdx == 2) {
        // 声明由远端计算核心SPE的ID组成的线程组
        ThreadGroup remote_thread_group(0x3);
        // 为2号计算核心配置线程组
        rma_set_thread_group(handle, &remote_thread_group);

        // 执行非阻塞型RMA等待操作
    }
}
```

```
    }

    // 其他代码逻辑
}
```

7.6.2.3.2 rma_async_get

```
void rma_async_get(void *local_addr, const void *remote_addr, size_t size, RmaHandle &handle)
```

功能介绍

将远端计算核心SPE中指定的SPM存储空间数据非阻塞拷贝到当前计算核心SPE中指定的SPM存储空间内。

参数解释

参数名称	说明
local_addr	数据搬运的目的地址，当前计算核心的SPM存储空间地址。
remote_addr	数据搬运的源地址，远端计算核心的SPM存储空间地址。
size	数据搬运大小，单位：Byte。 数据搬运量的大小受SPM存储空间大小限制，可参照 不同存储空间可用大小 ，理论最大数据搬运量为：源地址和目标地址所在空间较小者的空间大小。
handle	RmaHandle 类型变量，用于存储远端计算核心的ID。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- SPM存储空间地址需要4B对齐。
- 数据搬运的数据量需要是4B的整数倍。
- 使用**malloc**分配SPM存储空间时，需要在所有涉及RMA数据搬运的SPE都可以执行到的代码区域内进行，并且要确保所有SPE分配的SPM空间首地址相同。使用**malloc**分配完SPM空间后，在执行非阻塞型RMA数据搬运之前，需要使用**sync_threads**进行同步。正确的使用方法，可参考使用示例2。

 提示：

在使用malloc分配SPM存储空间之前，如果已经对涉及RMA数据搬运的SPE申请过大小不同的SPM存储空间，请先释放这部分SPM内存空间，以确保所有涉及RMA数据搬运的SPE申请的SPM存储空间的首地址相同。

使用示例1

使用非阻塞型RMA操作，将1号计算核心中指定的SPM地址空间中的数据拷贝到0号计算核心指定的SPM地址空间内。

```
using namespace sdac;
#define SIZE 100

// 使用__local__ 分配的SPM存储空间
__local__ int local_addr[SIZE];
__local__ int remote_addr[SIZE];

__global__ void func()
{
    // 初始化0号计算核心中local_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            local_addr[i] = 0;
        }
    }

    // 初始化1号计算核心中remote_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            remote_addr[i] = i;
        }
    }

    // 创建用于进行非阻塞RMA操作的句柄
    RmaHandle handle;
    // 在进行非阻塞RMA操作前进行计算核心之间的同步
    sync_threads();

    // 0号计算核心发起非阻塞RMA操作
    if (threadIdx == 0) {
        // 配置1号计算核心为远端核心
        rma_set_thread_id(handle, 1);
        // 发起非阻塞RMA GET操作
        rma_async_get(local_addr, remote_addr, SIZE * sizeof(int), handle);

        // 等待当前计算核心完成非阻塞RMA GET操作
        // 使用了rma_complete接口，关于该接口用户可参考rma_complete小节介绍
        rma_complete(handle);

        // 其他代码逻辑
    }

    // 1号计算核心等待0号计算核心完成非阻塞RMA操作
    if (threadIdx == 1) {
        // 配置0号计算核心为远端核心
        rma_set_thread_id(handle, 0);
    }
}
```

```
        // 等待1号计算核心完成非阻塞RMA操作逻辑
        // 使用了rma_wait接口, 关于该接口用户可参考rma_wait小节介绍
        rma_wait(handle);
    }
}
```

使用示例2

使用非阻塞型RMA操作, 将1号计算核心中指定的SPM地址空间中的数据拷贝到0号计算核心指定的SPM地址空间内。

```
using namespace sdac;
#define SIZE 100

__global__ void func()
{
    // 使用malloc申请SPM空间
    int *spm_addr = (int *)malloc(SIZE * sizeof(int));

    // 初始化0号计算核心中spm_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            spm_addr[i] = 0;
        }
    }

    // 初始化1号计算核心中spm_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            spm_addr[i] = i;
        }
    }

    // 创建用于进行非阻塞RMA操作的句柄
    RmaHandle handle;
    // 在进行非阻塞RMA操作前进行计算核心之间的同步
    sync_threads();

    // 0号计算核心发起非阻塞RMA操作
    if (threadIdx == 0) {
        // 配置1号计算核心为远端核心
        rma_set_thread_id(handle, 1);
        // 发起非阻塞RMA GET操作
        rma_async_get(spm_addr, spm_addr, SIZE * sizeof(int), handle);

        // 等待当前计算核心完成非阻塞RMA GET操作
        // 使用了rma_complete接口, 关于该接口用户可参考rma_complete小节的介绍
        rma_complete(handle);

        // 其他代码逻辑
    }

    // 1号计算核心等待0号计算核心完成非阻塞RMA操作
    if (threadIdx == 1) {
        // 配置0号计算核心为远端核心
        rma_set_thread_id(handle, 0);
    }
}
```

```
    // 等待1号计算核心完成非阻塞RMA操作逻辑
    // 使用了rma_wait接口，关于该接口用户可参考rma_wait小节介绍
    rma_wait(handle);
}

free(spm_addr);
}
```

7.6.2.3.3 rma_async_put

```
void rma_async_put(const void *local_addr, void *remote_addr, size_t size, RmaHandle &handle)
```

功能介绍

将当前计算核心SPE中指定的SPM存储空间数据非阻塞拷贝到远端计算核心SPE中指定的SPM存储空间内。

参数解释

参数名称	说明
local_addr	数据搬运的源地址，当前计算核心的SPM存储空间地址。
remote_addr	数据搬运的目的地址，远端计算核心的SPM存储空间地址。
size	数据搬运大小，单位：Byte。 数据搬运量的大小受SPM存储空间大小限制，可参照 不同存储空间可用大小 ，理论最大数据搬运量为：源地址和目标地址所在空间较小者的空间大小。
handle	RmaHandle 类型变量，用于存储远端计算核心的ID。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- SPM存储空间地址需要4B对齐。
- 数据搬运的数据量需要是4B的整数倍。
- 使用**malloc**分配SPM存储空间时，需要在所有涉及RMA数据搬运的SPE都可以执行到的代码区域内进行，并且要确保所有SPE分配的SPM空间首地址相同。使用**malloc**分配完SPM空间后，在执行非阻塞型RMA数据搬运之前，需要使用**sync_threads**进行同步。正确的使用方法，可参考使用示例2。

 提示:

在使用malloc分配SPM存储空间之前，如果已经对涉及RMA数据搬运的SPE申请过大小不同的SPM存储空间，请先释放这部分SPM内存空间，以确保所有涉及RMA数据搬运的SPE申请的SPM存储空间的首地址相同。

使用示例1

使用非阻塞型RMA操作，将0号计算核心中指定的SPM地址空间中的数据拷贝到1号计算核心指定的SPM地址空间内。

```
using namespace sdAA;
#define SIZE 100

// 使用__local__ 分配的SPM存储空间
__local__ int local_addr[SIZE];
__local__ int remote_addr[SIZE];

__global__ void func()
{
    // 初始化0号计算核心中local_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            local_addr[i] = i;
        }
    }

    // 初始化1号计算核心中remote_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            remote_addr[i] = 0;
        }
    }

    // 创建用于进行非阻塞RMA操作的句柄
    RmaHandle handle;
    // 在进行非阻塞RMA操作前进行计算核心之间的同步
    sync_threads();

    // 0号计算核心发起非阻塞RMA操作
    if (threadIdx == 0) {
        // 配置1号计算核心为远端核心
        rma_set_thread_id(handle, 1);
        // 发起非阻塞RMA PUT操作
        rma_async_put(local_addr, remote_addr, SIZE * sizeof(int), handle);

        // 等待当前计算核心完成非阻塞RMA PUT操作
        // 使用了rma_complete接口，关于该接口用户可参考rma_complete小节介绍
        rma_complete(handle);

        // 其他代码逻辑
    }

    // 1号计算核心等待0号计算核心完成非阻塞RMA操作
    if (threadIdx == 1) {
```

```

        // 配置0号计算核心为远端核心
        rma_set_thread_id(handle, 0);

        // 等待1号计算核心完成非阻塞RMA操作逻辑
        // 使用了rma_wait接口, 关于该接口用户可参考rma_wait小节介绍
        rma_wait(handle);
    }
}

```

使用示例2

使用非阻塞型RMA操作，将0号计算核心中指定的SPM地址空间中的数据拷贝到1号计算核心指定的SPM地址空间内。

```

using namespace sdac;
#define SIZE 100

__global__ void func()
{
    // 使用malloc申请SPM空间
    int *spm_addr = (int *)malloc(SIZE * sizeof(int));

    // 初始化0号计算核心中spm_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            spm_addr[i] = i;
        }
    }

    // 初始化1号计算核心中spm_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            spm_addr[i] = 0;
        }
    }

    // 创建用于进行非阻塞RMA操作的句柄
    RmaHandle handle;
    // 在进行非阻塞RMA操作前进行计算核心之间的同步
    sync_threads();

    // 0号计算核心发起非阻塞RMA操作
    if (threadIdx == 0) {
        // 配置1号计算核心为远端核心
        rma_set_thread_id(handle, 1);
        // 发起非阻塞RMA PUT操作
        rma_async_put(spm_addr, spm_addr, SIZE * sizeof(int), handle);

        // 等待当前计算核心完成非阻塞RMA PUT操作
        // 使用了rma_complete接口, 关于该接口用户可参考rma_complete小节的介绍
        rma_complete(handle);

        // 其他代码逻辑
    }

    // 1号计算核心等待0号计算核心完成非阻塞RMA操作
    if (threadIdx == 1) {

```

```
// 配置0号计算核心为远端核心
rma_set_thread_id(handle, 0);

// 等待1号计算核心完成非阻塞RMA操作逻辑
// 使用了rma_wait接口, 关于该接口用户可参考rma_wait小节介绍
rma_wait(handle);
}

free(spm_addr);
}
```

7.6.2.3.4 rma_complete

```
void rma_complete(RmaHandle &handle, RmaOperateMode operateMode = RmaNormalMode)
```

功能介绍

等待当前计算核心SPE完成非阻塞型RMA数据搬运。

参数解释

参数名称	说明
handle	RmaHandle 类型变量，用于存储远端计算核心的ID。
operateMode	枚举类型RmaOperateMode，支持以下两种模式： - RmaNormalMode：常规模式，由编译器自动计算当前计算核心发起的非阻塞RMA数据搬运次数，该模式为默认配置模式。该模式下非阻塞RMA数据搬运的效率较低。 - RmaCustomizeMode：自定义模式，需要由用户手动写入当前计算核心发起的非阻塞RMA数据搬运次数。需配合rma_wait的rmaTime参数使用。该模式下非阻塞RMA数据搬运的效率较高。

返回值

无。

注意事项

- 需要引入sdAA命名空间。
- 要求 `rma_complete` 与**rma_wait**需要成对使用。
- `RmaNormalMode` 和 `RmaCustomizeMode` 两种模式不能混合使用。

- 使用 `RmaNormalMode` 模式，要求用户在远端SPE使用`rma_wait`时，不能写入非阻塞RMA操作次数。
- 使用 `RmaCustomizeMode` 模式，要求用户在远端SPE使用`rma_wait`时，需要准确写入非阻塞RMA操作次数。
- 执行完 `rma_complete` 函数，表示当前SPE已完成非阻塞RMA数据搬运。用户可以根据需要，对当前SPE `local_addr` 中的数据进行更改。

使用示例

使用非阻塞型RMA操作，将1号计算核心中指定的SPM地址空间中的数据拷贝到0号计算核心指定的SPM地址空间内。

```
using namespace sdac;
#define SIZE 100

__local__ int local_addr[SIZE];
__local__ int remote_addr[SIZE];

__global__ void func()
{
    // 初始化0号计算核心中local_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            local_addr[i] = 0;
        }
    }

    // 初始化1号计算核心中remote_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            remote_addr[i] = i;
        }
    }

    // 创建用于进行非阻塞RMA操作的句柄
    RmaHandle handle;
    // 在进行非阻塞RMA操作前进行计算核心之间的同步
    sync_threads();

    // 0号计算核心发起非阻塞RMA操作
    if (threadIdx == 0) {
        // 配置1号计算核心为远端核心
        rma_set_thread_id(handle, 1);
        // 发起非阻塞RMA GET操作
        rma_async_get(local_addr, remote_addr, SIZE * sizeof(int), handle);

        // 使用默认的RmaNormalMode等待当前计算核心完成非阻塞RMA GET操作
        rma_complete(handle);

        // 执行完rma_complete，当前计算核心就可以随意操作由远端获取到的数据了
        for (int i = 0; i < SIZE; i++) {
            printf("local_addr[%d] = %d\n", i, local_addr[i]);
        }
    }

    // 1号计算核心等待0号计算核心完成非阻塞RMA操作
```

```
if (threadIdx == 1) {
    // 配置0号计算核心为向当前计算核心发起非阻塞RMA操作的计算核心
    rma_set_thread_id(handle, 0);

    // 等待1号计算核心完成非阻塞RMA操作逻辑
    // 使用了rma_wait接口, 关于该接口用户可参考rma_wait小节介绍
    rma_wait(handle);
}
}
```

7.6.2.3.5 rma_wait

```
void rma_wait(RmaHandle &handle, unsigned int rmaTime = 0xffff)
```

功能介绍

等待远端计算核心SPE完成非阻塞型RMA数据搬运。

参数解释

参数名称	说明
handle	RmaHandle 类型变量, 用于存储向当前SPE发起非阻塞RMA操作的其他SPE信息。
rmaTime	其他SPE向当前SPE发起非阻塞RMA操作的次数, 默认为0xffff。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- 要求 `rma_wait` 和 `rma_complete` 需要成对使用。
- `rmaTime` 是否写入需要与 `rma_complete` 的 `RmaOperateMode` 参数严格对应。
 - 当 `RmaOperateMode` 为 `RmaNormalMode` 时:
 - `rmaTime` 需要使用默认值。
 - 当前SPE在发起 `rma_wait` 时, 不在 `handle` 配置之内的其他SPE不能向当前计算核心发起非阻塞RMA操作。
 - 当 `RmaOperateMode` 为 `RmaCustomizeMode` 时:

- 用户需要准确写入其他SPE向当前SPE发起的非阻塞RMA操作的次数，即当前SPE在完成等待上一次其他SPE向当前SPE发起的非阻塞RMA数据搬运后，到当前时间点之间，其他SPE向当前SPE发起的非阻塞RMA数据搬运次数。
- 当前SPE在发起 `rma_wait` 时，不允许其他任何SPE向当前SPE发起非阻塞RMA操作。
- 执行完 `rma_wait`，表示当前SPE已完成非阻塞RMA数据搬运，用户可以根据需要，对当前SPE `remote_addr` 中的数据进行更改。

使用示例

1号和2号计算核心使用RmaCustomizeMode分别向0号计算核心发起非阻塞型RMA PUT操作。

```
using namespace sdAA;
#define SIZE 100
#define REMOTE_SIZE 50

__local__ int remote_addr[SIZE];
__local__ int local_addr[REMOTE_SIZE];

__global__ void func()
{
    // 初始化0号计算核心中remote_addr的值
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            remote_addr[i] = 0;
        }
    }

    // 初始化1号计算核心中local_addr的值
    if (threadIdx == 1) {
        for (int i = 0; i < REMOTE_SIZE; i++) {
            local_addr[i] = i;
        }
    }

    // 初始化2号计算核心中local_addr的值
    if (threadIdx == 2) {
        for (int i = 0; i < REMOTE_SIZE; i++) {
            local_addr[i] = i * 2;
        }
    }

    // 创建用于进行非阻塞RMA操作的句柄
    RmaHandle handle;
    // 在进行非阻塞RMA操作前进行计算核心之间的同步
    sync_threads();

    // 1号计算核心向0号计算核心发起RMA PUT操作
    if (threadIdx == 1) {
        // 配置0号计算核心为远端核心
        rma_set_thread_id(handle, 0);
        // 发起非阻塞RMA PUT操作
        rma_async_put(local_addr, remote_addr, REMOTE_SIZE * sizeof(int), handle);

        // 使用RmaCustomizeMode等待当前计算核心完成非阻塞RMA PUT操作
    }
}
```

```
    rma_complete(handle, RmaCustomizeMode);
}

// 2号计算核心向0号计算核心发起RMA PUT操作
if (threadIdx == 2) {
    // 配置0号计算核心为远端核心
    rma_set_thread_id(handle, 0);
    // 发起非阻塞RMA PUT操作
    rma_async_put(local_addr, remote_addr + REMOTE_SIZE, REMOTE_SIZE * sizeof(int), handle);

    // 使用RmaCustomizeMode等待当前计算核心完成非阻塞RMA PUT操作
    rma_complete(handle, RmaCustomizeMode);
}

if (threadIdx == 0) {
    // 声明由远端计算核心SPE1和SPE2的ID组成的线程组
    ThreadGroup remote_thread_group(0x6);
    // 为0号计算核心配置线程组
    rma_set_thread_group(handle, &remote_thread_group);

    // 等待1号和2号计算核心完成非阻塞RMA操作逻辑
    // 第二个参数2，表示有两个计算核心向当前计算核心总共发起了2次的非阻塞RMA操作
    rma_wait(handle, 2);

    // 打印输出
    for (int i = 0; i < SIZE; i++) {
        printf("%d\n", remote_addr[i]);
    }
}
}
```

7.6.2.4 典型场景示例

7.6.2.4.1 单个计算核心向多个计算核心发起非阻塞RMA操作

本文主要介绍单个计算核心向多个计算核心发起非阻塞RMA操作。

编程思路

步骤一： 0号计算核心SPE分别使用RmaNormalMode模式和RmaCustomizeMode模式通过非阻塞RMA rma_async_get操作从1号计算核心SPE获取数据进行加工。

步骤二： 0号计算核心SPE完成数据加工后通过非阻塞RMA的rma_async_put操作将数据发送到2号和3号计算核心SPE。

步骤三： 2号3号计算核心SPE成功接收数据后，打印输出数据。



图1 单个计算核心向多个计算核心发起非阻塞RMA操作

RmaNormalMode模式

```

using namespace sdaa;
// 声明SPM内存空间
__local__  int local_addr[10];
__local__  int remote_addr[10];

__global__ void func()
{
    // 初始化0号计算核心SPE local_addr内存值
    if (threadIdx == 0) {
        for (int i = 0; i < 10; i++) {
            local_addr[i] = 0;
        }
    }

    // 初始化1号计算核心SPE remote_addr内存值
    if (threadIdx == 1) {
        for (int i = 0; i < 10; i++) {
            remote_addr[i] = i;
        }
    }

    // 初始化2号和3号计算核心SPE remote_addr内存值
    if ((threadIdx == 2) || (threadIdx == 3)) {
        for (int i = 0; i < 10; i++) {
            remote_addr[i] = 0;
        }
    }

    // 定义RmaHandle类型的变量
    RmaHandle handle;
    // 同步操作
    sync_threads();

    if (threadIdx == 0) {
        // 0号计算核心SPE使用非阻塞RMA的rma_async_get操作, 获取1号计算核心SPE remote_addr中的值
        rma_set_thread_id(handle, 1);
        rma_async_get(local_addr, remote_addr, 10 * sizeof(int), handle);

        // 等待0号计算核心SPE完成非阻塞RMA GET
        rma_complete(handle);

        // 0号计算核心SPE完成非阻塞RMA的rma_async_get操作后, 对数据进行二次加工
        for (int i = 0; i < 10; i++) {
            local_addr[i] *= 2;
        }
    }
}

```

```

    }

    // 0号计算核心SPE通过非阻塞RMA的rma_async_put操作, 将数据发送到2号和3号计算核心
    rma_set_thread_id(handle, 2);
    rma_async_put(local_addr, remote_addr, 10 * sizeof(int), handle);

    rma_set_thread_id(handle, 3);
    rma_async_put(local_addr, remote_addr, 10 * sizeof(int), handle);

    rma_complete(handle);
}

if (threadIdx == 1) {
    // 1号计算核心SPE等待0号计算核心SPE发起的非阻塞RMA的rma_async_get操作完成
    rma_set_thread_id(handle, 0);
    rma_wait(handle);
}

if (threadIdx == 2) {
    // 2号计算核心SPE等待0号计算核心SPE发起的非阻塞RMA的rma_async_put操作完成
    rma_set_thread_id(handle, 0);
    rma_wait(handle);

    // 2号计算核心SPE等待非阻塞RMA的rma_async_put操作完成后, 对数据进行打印输出
    printf("SPE 2: ");
    for (int i = 0; i < 10; i++) {
        printf("%d\t", remote_addr[i]);
    }
    printf("\n");
}

if (threadIdx == 3) {
    // 3号计算核心SPE等待0号计算核心SPE发起的非阻塞RMA的rma_async_put操作完成
    rma_set_thread_id(handle, 0);
    rma_wait(handle);

    // 3号计算核心SPE等待非阻塞RMA的rma_async_put操作完成后, 对数据进行打印输出
    printf("SPE 3: ");
    for (int i = 0; i < 10; i++) {
        printf("%d\t", remote_addr[i]);
    }
    printf("\n");
}
}

int main()
{
    // 设置使用0号计算核心阵列
    sdaaSetDevice(0);
    // 核函数调用
    func<<<1>>>>();
    sdaaDeviceSynchronize();
    return 0;
}

```

RmaCustomizeMode模式

```

using namespace sdAA;
// 声明SPM内存空间
__local__  int local_addr[10];
__local__  int remote_addr[10];

__global__ void func()
{
    // 初始化0号计算核心SPE local_addr内存值
    if (threadIdx == 0) {
        for (int i = 0; i < 10; i++) {
            local_addr[i] = 0;
        }
    }

    // 初始化1号计算核心SPE remote_addr内存值
    if (threadIdx == 1) {
        for (int i = 0; i < 10; i++) {
            remote_addr[i] = i;
        }
    }

    // 初始化2号和3号计算核心SPE remote_addr内存值
    if ((threadIdx == 2) || (threadIdx == 3)) {
        for (int i = 0; i < 10; i++) {
            remote_addr[i] = 0;
        }
    }

    // 定义RmaHandle类型的变量
    RmaHandle handle;
    // 同步操作
    sync_threads();

    if (threadIdx == 0) {
        // 0号计算核心SPE使用非阻塞RMA的rma_async_get操作, 获取1号计算核心SPE remote_addr中的值
        rma_set_thread_id(handle, 1);
        rma_async_get(local_addr, remote_addr, 10 * sizeof(int), handle);

        // 等待0号计算核心SPE完成非阻塞RMA的rma_async_get操作
        rma_complete(handle, RmaCustomizeMode);

        // 0号计算核心SPE完成非阻塞RMA的rma_async_get操作后, 对数据进行二次加工
        for (int i = 0; i < 10; i++) {
            local_addr[i] *= 2;
        }

        // 0号计算核心SPE通过非阻塞RMA的rma_async_put操作, 将数据发送到2号和3号计算核心
        rma_set_thread_id(handle, 2);
        rma_async_put(local_addr, remote_addr, 10 * sizeof(int), handle);

        rma_set_thread_id(handle, 3);
        rma_async_put(local_addr, remote_addr, 10 * sizeof(int), handle);

        rma_complete(handle, RmaCustomizeMode);
    }

    if (threadIdx == 1) {
        // 1号计算核心SPE等待0号计算核心SPE发起的非阻塞RMA的rma_async_get操作完成
    }
}

```

```

    rma_set_thread_id(handle, 0);
    // 1表示0号计算核心SPE通过RmaCustomizeMode向1号计算核心SPE发起了1次非阻塞RMA操作
    rma_wait(handle, 1);
}

if (threadIdx == 2) {
    // 2号计算核心SPE等待0号计算核心SPE发起的非阻塞RMA的 rma_async_put操作完成
    rma_set_thread_id(handle, 0);
    // 1表示0号计算核心SPE通过RmaCustomizeMode向2号计算核心SPE发起了1次非阻塞RMA操作
    rma_wait(handle, 1);

    // 2号计算核心SPE等待非阻塞RMA的rma_async_put操作完成后, 对数据进行打印输出
    printf("SPE 2: ");
    for (int i = 0; i < 10; i++) {
        printf("%d\t", remote_addr[i]);
    }
    printf("\n");
}

if (threadIdx == 3) {
    // 3号计算核心SPE等待0号计算核心SPE发起的非阻塞RMA的rma_async_put操作完成
    rma_set_thread_id(handle, 0);
    // 1表示0号计算核心SPE通过RmaCustomizeMode向2号计算核心SPE发起了1次非阻塞RMA操作
    rma_wait(handle, 1);

    // 3号计算核心SPE等待非阻塞RMA rma_async_put完成后, 对数据进行打印输出
    printf("SPE 3: ");
    for (int i = 0; i < 10; i++) {
        printf("%d\t", remote_addr[i]);
    }
    printf("\n");
}
}

int main()
{
    // 设置使用0号计算核心阵列
    sdaaSetDevice(0);
    // 核函数调用
    func<<<1>>>>();
    sdaaDeviceSynchronize();
    return 0;
}

```

7.6.2.4.2 多个计算核心向单个计算核心发起非阻塞RMA操作

本文主要介绍多个计算核心向单个计算核心发起非阻塞RMA操作。

编程思路

步骤一：1号计算核心SPE分别使用RmaNormalMode模式和RmaCustomizeMode模式通过非阻塞RMA的rma_async_put操作将数据发送到0号计算核心SPE上。

步骤二：0号计算核心SPE完成数据接收后, 2号3号计算核心SPE分别使用RmaNormalMode模式和RmaCustomizeMode模式通过非阻塞RMA的rma_async_put操作将数据发送到0号计算核心SPE上。

步骤三：0号计算核心SPE成功接收数据后，打印输出数据。



图1 多个计算核心向单个计算核心发起非阻塞RMA操作

RmaNormalMode模式

```

using namespace sdac;
__local__ int local_addr[10];
__local__ int remote_addr[30];

__global__ void func()
{
    // 初始化0号计算核心SPE remote_addr内存值
    if (threadIdx == 0) {
        for (int i = 0; i < 30; i++) {
            remote_addr[i] = 0;
        }
    }

    // 初始化1号, 2号和3号计算核心SPE local_addr内存值
    if ((threadIdx == 1) || (threadIdx == 2) || (threadIdx == 3)) {
        for (int i = 0; i < 10; i++) {
            local_addr[i] = i * threadIdx;
        }
    }

    // 定义RmaHandle类型的变量
    RmaHandle handle;
    // 同步操作
    sync_threads();

    if (threadIdx == 1) {
        // 1号计算核心SPE通过非阻塞RMA rma_async_put操作, 将数据发送到0号计算核心
        rma_set_thread_id(handle, 0);
        rma_async_put(local_addr, remote_addr, 10 * sizeof(int), handle);

        rma_complete(handle);
    }

    if (threadIdx == 0) {
        // 0号计算核心SPE等待1号计算核心SPE发起的非阻塞RMA的rma_async_put操作完成
        rma_set_thread_id(handle, 1);
        rma_wait(handle);
    }

    // 同步操作
    sync_threads();
}

```

```

if (threadIdx == 2) {
    // 2号计算核心SPE通过非阻塞RMA的rma_async_put操作, 将数据发送到0号计算核心SPE
    rma_set_thread_id(handle, 0);
    rma_async_put(local_addr, remote_addr + 10, 10 * sizeof(int), handle);

    rma_complete(handle);
}

if (threadIdx == 3) {
    // 3号计算核心SPE通过非阻塞RMA的rma_async_put操作, 将数据发送到0号计算核心SPE
    rma_set_thread_id(handle, 0);
    rma_async_put(local_addr, remote_addr + 20, 10 * sizeof(int), handle);
    rma_complete(handle);
}

if (threadIdx == 0) {
    // 0号计算核心SPE等待2号和3号计算核心SPE发起的非阻塞RMA的rma_async_put操作完成
    ThreadGroup remote_thread_group(0xC);
    rma_set_thread_group(handle, &remote_thread_group);
    rma_wait(handle);

    // 0号计算核心SPE等待非阻塞RMA的rma_async_put操作完成后, 进行打印输出
    printf("SPE 0: ");
    for (int i = 0; i < 30; i++) {
        printf("%d\t", remote_addr[i]);
    }

    printf("\n");
}
}

int main()
{
    // 设置使用0号计算核心阵列
    sdaaSetDevice(0);
    // 核函数调用
    func<<<1>>>>();
    sdaaDeviceSynchronize();
    return 0;
}

```

RmaCustomizeMode模式

```

using namespace sdaa;
__local__ int local_addr[10];
__local__ int remote_addr[30];

__global__ void func()
{
    // 初始化0号计算核心SPE remote_addr内存值
    if (threadIdx == 0) {
        for (int i = 0; i < 30; i++) {
            remote_addr[i] = 0;
        }
    }
}

```

```

// 初始化1号, 2号和3号计算核心SPE local_addr内存值
if ((threadIdx == 1) || (threadIdx == 2) || (threadIdx == 3)) {
    for (int i = 0; i < 10; i++) {
        local_addr[i] = i * threadIdx;
    }
}

// 定义RmaHandle类型的变量
RmaHandle handle;
// 同步操作
sync_threads();

if (threadIdx == 1) {
    // 1号计算核心SPE通过非阻塞RMA的rma_async_put操作, 将数据发送到0号计算核心SPE
    rma_set_thread_id(handle, 0);
    rma_async_put(local_addr, remote_addr, 10 * sizeof(int), handle);

    rma_complete(handle, RmaCustomizeMode);
}

if (threadIdx == 0) {
    // 0号计算核心SPE等待1号计算核心SPE发起的非阻塞RMA的rma_async_put操作完成
    rma_set_thread_id(handle, 1);
    // 1表示0号计算核心在完成上一次非阻塞RMA操作有, 其他计算核心向0号计算核心发起了1次非阻塞RMA操作
    rma_wait(handle, 1);
}

// 同步操作
sync_threads();

if (threadIdx == 2) {
    // 2号计算核心SPE通过非阻塞RMA的rma_async_put操作, 将数据发送到0号计算核心SPE
    rma_set_thread_id(handle, 0);
    rma_async_put(local_addr, remote_addr + 10, 10 * sizeof(int), handle);

    rma_complete(handle, RmaCustomizeMode);
}

if (threadIdx == 3) {
    // 3号计算核心SPE通过非阻塞RMA的rma_async_put操作, 将数据发送到0号计算核心SPE
    rma_set_thread_id(handle, 0);
    rma_async_put(local_addr, remote_addr + 20, 10 * sizeof(int), handle);

    rma_complete(handle, RmaCustomizeMode);
}

if (threadIdx == 0) {
    // 0号计算核心SPE等待2号和3号计算核心SPE发起的非阻塞RMA的rma_async_put操作完成
    ThreadGroup remote_thread_group(0xC);
    rma_set_thread_group(handle, &remote_thread_group);
    // 2表示0号计算核心在完成上一次非阻塞RMA操作有, 其他计算核心向0号计算核心发起了2次非阻塞RMA操作
    rma_wait(handle, 2);

    // 0号计算核心SPE等待非阻塞RMA PUT完成后进行打印输出
    printf("SPE 0: ");
    for (int i = 0; i < 30; i++) {
        printf("%d\t", remote_addr[i]);
    }
}

```

```
        printf("\n");
    }
}

int main()
{
    // 设置使用0号计算核心阵列
    sdaaSetDevice(0);
    // 核函数调用
    func<<<1>>>();
    sdaaDeviceSynchronize();
    return 0;
}
```

7.6.3 Broadcast数据搬运

7.6.3.1 概述

Broadcast数据搬运是指同一计算核心阵列SPA内，指定的计算核心SPE向多个SPE传输数据。Broadcast数据搬运可以通过BroadcastHandle配置SPE加入广播组，进而接收广播数据。

从调用接口上可以分为阻塞型Broadcast数据搬运和非阻塞型Broadcast数据搬运：

- 阻塞型Broadcast数据搬运接口：使用简单，但性能不佳。
- 非阻塞型Broadcast数据搬运接口：使用相对复杂，但性能较好。

BroadcastHandle及配置接口

BroadcastHandle及配置接口由以下函数接口组成：

- **BroadcastHandle**：用于配置广播组，控制接收广播的计算核心SPE。
- **broadcast_set_thread_group**：通过线程组的方式设置当前SPA内的各个SPE是否接收广播数据。

阻塞型Broadcast数据搬运

阻塞型Broadcast操作由以下函数接口组成：

- **broadcast**：在同一计算核心阵列SPA内，将指定存储空间的数据以阻塞的方式搬运到所有SPE的SPM存储空间，或搬运到由 **BroadcastHandle** 指定SPE的SPM存储空间内。

非阻塞型Broadcast数据搬运

非阻塞型Broadcast操作由以下函数接口组成：

- **broadcast_async**：在同一计算核心阵列SPA内，将指定存储空间的数据以非阻塞的方式搬运到由 **BroadcastHandle** 指定的部分或全部SPE的SPM存储空间内。

- `broadcast_wait`: 等待当前计算核心SPE上所有使用同一个 `BroadcastHandle` 的非阻塞型Broadcast数据搬运操作完成。

7.6.3.2 BroadcastHandle及配置接口

7.6.3.2.1 概述

BroadcastHandle是Broadcast数据搬运的句柄，由以下函数接口组成：

- `BroadcastHandle`: 用于配置当前广播组，控制接收广播的计算核心SPE和数据搬运过程中的跨步参数。
- `broadcast_set_thread_group`: 通过线程组的方式设置当前SPA内的各个SPE是否接收广播数据。

7.6.3.2.2 BroadcastHandle

```
BroadcastHandle::BroadcastHandle()
BroadcastHandle::BroadcastHandle(ThreadGroup *thread_group)
```

功能介绍

用于配置当前广播组。默认广播组包括当前计算核心阵列SPA内所有计算核心SPE，即所有SPE均接受广播数据。

参数解释

参数名称	说明
thread_group	<code>ThreadGroup</code> 指针类型变量，用于配置SPE是否加入广播组并接收数据。

注意事项

- 需要引入sdAA命名空间。
- 使用 `BroadcastHandle` 创建变量时需要在当前所有SPE都可以执行到的代码段内创建。
- 在创建完 `BroadcastHandle` 类型的变量后，且在执行Broadcast数据搬运之前，需要使用`sync_threads`函数接口在SPE间进行一次同步操作。

使用示例

```
using namespace sdAA;
__global__ void func()
{
    BroadcastHandle handle; // 创建BroadcastHandle类型变量，所有SPE均接收数据

    // 其他代码逻辑

    sync_threads(); // 计算核心之间进行一次同步操作
```

```
    // 执行Broadcast操作
}
```

使用示例2

```
using namespace sdac;
__global__ void func()
{
    ThreadGroup thread_group(0xFF00);
    BroadcastHandle handle(&thread_group); // 创建BroadcastHandle类型变量，仅SPE8~15接收数据

    // 其他代码逻辑

    sync_threads(); // 计算核心之间进行一次同步操作

    // 执行Broadcast操作
}
```

7.6.3.2.3 broadcast_set_thread_group

```
void broadcast_set_thread_group(BroadcastHandle &handle, ThreadGroup *thread_group)
```

功能介绍

通过线程组的方式设置当前SPA内的各个SPE是否接收广播数据。

参数解释

参数名称	说明
handle	BroadcastHandle类型变量，标识当前的广播组。
thread_group	ThreadGroup指针类型变量，用于配置SPE是否加入广播组并接收数据。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- 使用 broadcast_set_thread_group 接口为BroadcastHandle类型变量配置完线程组后，如果需要对线程组进行修改可直接使用ThreadGroup的相关配置接口。在修改完线程组后无需再次使用 broadcast_set_thread_group 为同一个BroadcastHandle类型变量进行配置。

使用示例

```
using namespace sdAA;
__global__ void func()
{
    ThreadGroup thread_group(0xFF00);    // 创建ThreadGroup类型变量, 将SPE8~SPE15组成一个线程组
    BroadcastHandle handle;               // 创建BroadcastHandle类型变量
    broadcast_set_thread_group(handle, &thread_group);    // 为handle配置线程组
    sync_threads(); // 计算核心之间进行一次同步操作

    // 执行Broadcast操作, 广播到SPE8~15

    thread_group_exclude(thread_group, 8); // 修改广播组, 仅SPE9~SPE15接收数据

    // 无需使用broadcast_set_thread_group再次对handle进行配置, 可直接执行Broadcast操作, 广播到SPE9~SPE15
}
```

7.6.3.3 阻塞型Broadcast数据搬运

7.6.3.3.1 broadcast

```
void broadcast(void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d, unsigned long root, BroadcastDirect direct)
void broadcast(void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d, unsigned long root, BroadcastDirect direct, BroadcastHandle &handle)
```

功能介绍

在同一计算核心阵列SPA内, 将指定存储空间的数据以阻塞的方式搬运到所有SPE的SPM存储空间, 或搬运到由 `BroadcastHandle` 指定SPE的SPM存储空间内。

参数解释

参数名称	说明
dst	Broadcast数据搬运的目的地址。
src	Broadcast数据搬运的源地址。
size	Broadcast数据搬运大小, 单位: Byte, 需要是4B的整数倍。 数据搬运量的大小受SPM存储空间大小限制, 可参照不同存储空间可用大小, 理论最大数据搬运量为: 源地址和目标地址所在空间较小者的空间大小。
section_1d	一维数据搬运单元大小, 单位Byte, 需要是4B的整数倍。

参数名称	说明
stride_1d	MemoryStride结构体类型变量，设置源和目标存储空间的一维跨步单元大小，单位：Byte。 ? 说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运，且当前接口的 <code>section_1d</code> 参数无效。
root	发送方的SPE的threadIdx。 数据搬运方向为BroadcastGlobalToSpm时，root需为线程组内的某个线程。
direct	枚举类型BroadcastDirect，Broadcast数据搬运方向。 - 非跨步场景：支持以下数据搬运方向： - BroadcastGlobalToSpm：Global存储空间向SPM存储空间的Broadcast数据搬运。 - BroadcastSpmToSpm：SPM存储空间到SPM存储空间的Broadcast数据搬运。 - 跨步场景：支持以下数据搬运方向中Global存储空间跨步设置： - BroadcastGlobalToSpm：Global存储空间向SPM存储空间的Broadcast数据搬运。
handle	(可选) BroadcastHandle类型变量，表示当前广播组，用于标识接收广播的SPE。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- SPM存储空间地址需要4B对齐。
- Broadcast数据搬运的数据量需要是4B的整数倍。
- 如果使用跨步功能，Global存储空间的跨步需要是4B的整数倍。

- 不使用 `BroadcastHandle` 参数时，为全广播（SPA内所有SPE均参与），此时所有SPE都必须执行 `broadcast` 函数，否则会造成同步失败，可参考使用示例1。
- 使用 `BroadcastHandle` 参数时，不要求所有SPE必须执行 `broadcast` 函数，但发送方SPE和 `BroadcastHandle` 广播组内的所有SPE都需要执行 `broadcast` 函数，否则会造成不可预知的结果（即不能保证广播完成），可参考使用示例2。
- 多次执行 `broadcast` 函数向同一地址发送不同数据时，接收方无法保证接收数据的时序准确性。您可以使用 `sync_threads` 进行数据同步，从而确保连续接收数据的安全性，可参考使用示例3。
- 使用 `malloc` 分配SPM存储空间时，需要在所有涉及Broadcast操作的SPE都可以执行到的代码区域内进行，并且要确保所有SPE分配的SPM空间首地址相同。使用 `malloc` 分配完SPM空间后，在发起Broadcast操作前，需要使用 `sync_threads` 进行同步。正确的使用方法，可参考使用示例2。

提示：

在使用 `malloc` 分配SPM存储空间之前，如果已经对涉及Broadcast数据搬运的SPE申请过大小不同的SPM存储空间，请先释放这部分SPM内存空间，以确保所有涉及Broadcast数据搬运的SPE申请的SPM存储空间的首地址相同。

使用示例1

```
using namespace sdAA;
#define SIZE 10

// 使用__local__ 分配的SPM存储空间
__local__ int dst[SIZE];
__local__ int src[SIZE];

__global__ void func()
{
    ... // 准备广播数据

    broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm); // 正确, 0号SPE广播到所有SPE

    if (threadIdx < 8) { // 假设threadDim > 8
        broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm); // 错误, 无handle的broa
dcast只能做全广播
    }
}
```

使用示例2

```
using namespace sdAA;
#define SIZE 10

__global__ void func()
{
    // 使用malloc申请SPM空间（需要确保参与广播的SPE上dst和src地址相同）
    int *dst = (int *)malloc(SIZE * sizeof(int));
    int *src = (int *)malloc(SIZE * sizeof(int));
```

```

... // 准备广播数据

BroadcastHandle handle; // 创建BroadcastHandle类型变量, 所有SPE均接收数据
sync_threads(); // 计算核心之间进行一次同步操作

broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确, 0号SPE广播
到所有SPE

if (threadIdx < 8) { // 假设threadDim > 8
    broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 错误, 只能保
证SPE0~SPE7接收完数据, 不能保证其余SPE接收完数据
}

ThreadGroup thread_group(0xFF); // 定义由SPE0~SPE7组成的线程组
broadcast_set_thread_group(handle, &thread_group); // 设置仅SPE0~SPE7接收广播

broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确, 仅SPE0~SPE
7接收数据, 其余SPE执行broadcast时将立即退出

if (threadIdx < 8) {
    broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确
}

if (threadIdx == 0 || thread_group_is_included(thread_group, threadIdx)) {
    broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确, 确保数
据发送方SPE和线程组内的SPE进入
}

if (threadIdx < 4) {
    broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 错误, 仅SPE0
~3接收完数据, 不能保证SPE4~7接收完数据
}

free(src);
free(dst);
}

```

使用示例3

```

using namespace sdAA;
#define SIZE 10

// 使用__local__ 分配的SPM存储空间
__local__ int dst[SIZE];
__local__ int src[SIZE];

__global__ void func()
{
    ... // 准备广播数据

    const unsigned int root = 0; // 发送方ID
    BroadcastHandle handle; // 创建BroadcastHandle类型变量, 所有SPE均接收数据
    sync_threads(); // 计算核心之间进行一次同步操作

    // 错误示范:

```

```

// root多次向相同地址广播数据, 由于各线程独立执行, 因此不能保证各线程执行broadcast操作的时序性。
// 例如发送方执行速度较快, 在执行第二次broadcast时, 其他线程还没执行第一次broadcast,
// 就可能导致发送方第二次发送的数据覆盖第一次发送的数据, 其他线程在执行第一次broadcast后接收到的数据
// 实际为发送方第二次发送的数据。
for (int i = 0; i < 100; i++) {
    if (threadIdx == root) {
        src[0] = i; // 修改src的值
    }
    broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, root, BroadcastSpmToSpm, handle); // 发送方线程向其他线程发送数据
    if (threadIdx != root) {
        if (dst[0] == i) ... // 获取dst的值
    }
}

// 正确做法:
// 在处理有可能因线程不同步而导致的数据冲突的问题时, 可加上线程同步操作,
// 以保证各线程执行broadcast操作的时序性。
for (int i = 0; i < 100; i++) {
    if (threadIdx == root) {
        src[0] = i; // 修改src的值
    }
    sync_threads(); // 在发送数据前加上同步, 确保其他线程进入能接收数据的状态
    broadcast(dst, src, SIZE * sizeof(int), 0, {0, 0}, root, BroadcastSpmToSpm, handle); // 发送方线程向其他线程发送数据
    if (threadIdx != root) {
        if (dst[0] == i) ... // 获取dst的值
    }
}
}

```

使用示例4

```

using namespace sdAA;

#define SRC_SIZE 100
#define DST_SIZE 50

__device__ int src[SRC_SIZE];
__global__ void func()
{
    // 仅使用SPE0初始化Global存储空间数组
    if (threadIdx == 0) {
        for (int i = 0; i < SRC_SIZE; i++) {
            src[i] = i;
        }
    }

    int *dst = (int *)malloc(DST_SIZE * sizeof(int)); // 声明目的地址
    // 初始化目标存储空间
    for (int i = 0; i < DST_SIZE; i++) {
        dst[i] = 0;
    }

    // 进行Global存储空间跨步为40B的广播

```

```
size_t global_stride = 10 * sizeof(int);
ThreadGroup thread_group(0xFF00); // 定义SPE8~SPE15组成线程组
BroadcastHandle handle(&thread_group); // 定义BroadcastHandle类型变量
sync_threads(); // 创建完handle后执行一次同步操作

// 仅发起广播的SPE和线程组内的SPE执行广播操作
if ((threadIdx == 0) || (thread_group_is_included(thread_group, threadIdx))) {
    broadcast(dst, src, DST_SIZE * sizeof(int), 10 * sizeof(int), {0, global_stride}, 0, BroadcastGlobalToSpm, handle);
}

... // 其他操作

thread_group_set_mask(thread_group, 0xFF0000); // 修改线程组, 当前线程组内由SPE16~SPE23构成
// 使用线程组配置接口修改完线程组后直接进行新一轮的广播操作
if ((threadIdx == 31) || (thread_group_is_included(thread_group, threadIdx))) {
    broadcast(dst, src, DST_SIZE * sizeof(int), 10 * sizeof(int), {0, global_stride}, 31, BroadcastGlobalToSpm, handle);
}

... // 其他操作

free(dst);
}
```

7.6.3.4 非阻塞型Broadcast数据搬运

7.6.3.4.1 broadcast_async

```
void broadcast_async(void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d, unsigned long root, BroadcastDirect direct, BroadcastHandle &handle)
void broadcast_async(void *dst, const void *src, size_t size, size_t section_1d, MemoryStride stride_1d, BroadcastDirect direct, BroadcastHandle &handle)
```

功能介绍

在同一计算核心阵列SPA内，将指定存储空间的数据以非阻塞的方式搬运到由BroadcastHandle指定的部分或全部SPE的SPM存储空间内。

参数解释

参数名称	说明
dst	Broadcast数据搬运的目的地址。
src	Broadcast数据搬运的源地址。

参数名称	说明
size	Broadcast数据搬运大小，单位：Byte，需要是4B的整数倍。 数据搬运量的大小受SPM存储空间大小限制，可参照不同存储空间可用大小，理论最大数据搬运量为：源地址和目标地址所在空间较小者的空间大小。
section_id	一维数据搬运单元大小，单位Byte，需要是4B的整数倍。
stride_id	MemoryStride结构体类型变量，设置源和目标存储空间的一维跨步单元大小，单位：Byte。 ? 说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运，且当前接口的 <code>section_id</code> 参数无效。
root	(可选) 表示发送方的SPE的threadIdx。 ? 说明： - 使用带root参数的broadcast_async接口时： - 需要发送方SPE和BroadcastHandle广播组内的SPE都需要执行该函数，否则会造成不可预知的结果（即不能保证广播完成），可参考使用示例5。 - 需要配合 <code>broadcast_wait(BroadcastHandle &handle)</code> 接口使用。 - 使用不带root参数的broadcast_async接口时： - 仅需在发送方的SPE内执行该函数，可参考使用示例2。 - 需要配合 <code>broadcast_wait(BroadcastHandle &handle, unsigned int until_times)</code> 接口使用。

参数名称	说明
direct	枚举类型BroadcastDirect，Broadcast数据搬运方向。 - 非跨步场景：支持以下数据搬运方向： - BroadcastGlobalToSpm：Global存储空间向SPM存储空间的Broadcast数据搬运。 - BroadcastSpmToSpm：SPM存储空间到SPM存储空间的Broadcast数据搬运。 - 跨步场景：支持以下数据搬运方向： - BroadcastGlobalToSpm：Global存储空间向SPM存储空间的Broadcast数据搬运。
handle	BroadcastHandle 类型变量，表示当前广播组，用于标识接收广播的SPE。

返回值

无。

注意事项

- 需要引入sdAA命名空间。
- SPM存储空间地址需要4B对齐。
- Broadcast数据搬运的数据量需要是4B的整数倍。
- 如果使用跨步功能，Global存储空间的跨步需要是4B的整数倍。
- `broadcast_async` 需要与 `broadcast_wait` 配合使用，多个 `broadcast_async` 可以用同一个 `broadcast_wait` 等待。
- 多次执行broadcast函数向同一地址发送不同数据时，接收方无法保证接收数据的时序准确性。您可以使用 `sync_threads` 进行数据同步，从而确保连续接收数据的安全性，可参考使用示例6。
- 使用 `malloc` 分配SPM存储空间时，需要在所有涉及Broadcast操作的SPE都可以执行到的代码区域内进行，并且要确保所有SPE分配的SPM空间首地址相同。使用 `malloc` 分配完SPM空间后，在发起Broadcast操作前，需要使用 `sync_threads` 进行同步。正确的使用方法，可参考使用示例1和使用示例2。

 提示：

在使用malloc分配SPM存储空间之前，如果已经对涉及Broadcast数据搬运的SPE申请过大小不同的SPM存储空间，请先释放这部分SPM内存空间，以确保所有涉及Broadcast数据搬运的SPE申请的SPM存储空间的首地址相同。

使用示例1

SP0使用带root参数的broadcast_async向SPA内所有SPE发起广播操作。

```

using namespace sdAA;
#define SIZE1 10
#define SIZE2 20

// 使用 __local__ 分配的SPM存储空间
__local__ int dst1[SIZE1];
__local__ int src1[SIZE1];
__local__ int dst2[SIZE2];
__local__ int src2[SIZE2];

__global__ void func()
{
    ... // 准备广播数据

    ThreadGroup thread_group(0xFFFFFFFF); // 使用mask的方式创建线程组
    BroadcastHandle handle(&thread_group); // 创建BroadcastHandle类型变量, 所有SPE均接收数据
    sync_threads(); // 计算核心之间进行一次同步操作

    broadcast_async(dst1, src1, SIZE1 * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 0号SP
    E广播到所有SPE

    ... // 其他代码逻辑

    broadcast_wait(handle); // 等待SPE0完成发送操作和线程组内SPE完成接收广播的操作

    thread_group_exclude(thread_group, 0); // 使用线程组接口从当前线程组内删除SPE0
    broadcast_async(dst2, src2, SIZE2 * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 0号SP
    E广播到除自身外的所有其他SPE (即自身不接收)

    ... // 其他代码逻辑

    broadcast_wait(handle); // 等待SPE0完成发送操作和线程组内SPE完成接收广播的操作
}

```

使用示例2

SPE0使用不带root参数的broadcast_async向SPA内所有SPE发起广播操作。

```

using namespace sdAA;
#define SIZE1 10

// 使用 __local__ 分配的SPM存储空间
__local__ int dst1[SIZE1];
__local__ int src1[SIZE1];

__global__ void func()
{
    ... // 准备广播数据

    ThreadGroup thread_group(0xFFFFFFFF); // 使用掩码的方式创建线程组
    BroadcastHandle handle(&thread_group); // 创建BroadcastHandle类型变量, 所有SPE均接收数据
    sync_threads(); // 计算核心之间进行一次同步操作

    // 仅线程组内的SPE0执行广播操作
    if (threadIdx == 0) {

```

```

        broadcast_async(dst1, src1, SIZE1 * sizeof(int), 0, {0, 0}, BroadcastSpmToSpm, handle); // 0号SPE广播到所有SPE
    }
    ... // 其他代码逻辑

    if (thread_group_is_included(thread_group, threadIdx)) {
        broadcast_wait(handle, 1); // 等待线程组所有SPE完成发送广播或/与接收广播的操作
    }
}

```

使用示例3

SPE0使用带root参数的broadcast_async向SPE0~SPE7发起带跨步行为的广播操作。

```

using namespace sdAA;
#define SRC_SIZE 100
#define DST_SIZE 50

__device__ int src[SRC_SIZE];
__global__ void func()
{
    // 仅使用SPE0初始化Global存储空间数组
    if (threadIdx == 0) {
        for (int i = 0; i < SRC_SIZE; i++) {
            src[i] = i;
        }
    }

    // 使用malloc申请SPM空间
    int *dst = (int *)malloc(DST_SIZE * sizeof(int));
    // 初始化目标存储空间
    for (int i = 0; i < DST_SIZE; i++) {
        dst[i] = 0;
    }

    // 进行Global存储空间跨步为40B的广播
    size_t global_stride = 10 * sizeof(int);
    ThreadGroup thread_group(0xFF); // 定义SPE0~SPE7组成线程组
    BroadcastHandle handle(&thread_group); // 定义BroadcastHandle类型变量
    sync_threads(); // 创建完handle后执行一次同步操作

    // 仅发起广播的SPE和线程组内的SPE执行广播操作
    if (thread_group_is_included(thread_group, threadIdx)) {
        broadcast_async(dst, src, DST_SIZE * sizeof(int), 10 * sizeof(int), {0, global_stride}, 0, BroadcastGlobalToSpm, handle);
    }

    ... // 其他操作

    broadcast_wait(handle);
    free(dst);
}

```

使用示例4

SPE0使用不带root参数的broadcast_async向SPE0~SPE7发起带跨步行为的广播操作。

```
using namespace sdaa;
#define SRC_SIZE 100
#define DST_SIZE 50

__device__ int src[SRC_SIZE];
__global__ void func()
{
    // 仅使用SPE0初始化Global存储空间数组
    if (threadIdx == 0) {
        for (int i = 0; i < SRC_SIZE; i++) {
            src[i] = i;
        }
    }

    // 使用malloc申请SPM空间
    int *dst = (int *)malloc(DST_SIZE * sizeof(int));
    // 初始化目标存储空间
    for(int i = 0; i < DST_SIZE; i++) {
        dst[i] = 0;
    }

    // 进行Global存储空间跨步为40B的广播
    size_t global_stride = 10 * sizeof(int);
    ThreadGroup thread_group(0xFF); // 定义SPE0~SPE7组成线程组
    BroadcastHandle handle(&thread_group); // 定义BroadcastHandle类型变量
    sync_threads(); // 创建完handle后执行一次同步操作

    // 仅线程组内的SPE0执行广播操作
    if (threadIdx == 0) {
        broadcast_async(dst, src, DST_SIZE * sizeof(int), 10 * sizeof(int), {0, global_stride}, BroadcastGlobalToSpm, handle);
    }

    ... // 其他操作

    if (thread_group_is_included(thread_group, threadIdx)) {
        broadcast_wait(handle, 1);
    }
    free(dst);
}
```

使用示例5

错误示范：展示BroadcastHandle广播组内存在SPE未执行广播操作。

```
using namespace sdaa;
#define SIZE 10

__global__ void func()
{
    // 使用malloc申请SPM空间（需要确保参与广播的SPE上dst和src地址相同）
    int *dst = (int *)malloc(SIZE * sizeof(int));
    int *src = (int *)malloc(SIZE * sizeof(int));
```

```

... // 准备广播数据

BroadcastHandle handle; // 创建BroadcastHandle类型变量, 所有SPE均接收数据
sync_threads(); // 发起广播操作之前, 计算核心之间进行一次同步操作

broadcast_async(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确, 0号SPE广播到所有SPE

if (threadIdx < 8) { // 假设threadDim > 8
    broadcast_async(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 错误, 只能保证SPE0~7接收完数据, 不能保证其余SPE接收完数据
}

ThreadGroup thread_group(0xFF); // 定义由SPE0~SPE7组成的线程组
broadcast_set_thread_group(handle, &thread_group); // 设置仅SPE0~SPE7接收广播
broadcast_async(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确, 仅SPE0~7接收数据, 其余SPE执行broadcast将立即退出

if (threadIdx < 8) {
    broadcast_async(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确
}

if (threadIdx == 0 || thread_group_is_included(thread_group, threadIdx)) {
    broadcast_async(dst, src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 正确, 确保数据发送方SPE和线程组内的SPE进入
}

if (threadIdx < 4) {
    broadcast_wait(handle); // 错误, 仅SPE0~3接收完数据, 不能保证SPE4~7接收完数据
}

broadcast_wait(handle); // 等待所有broadcast_async操作完成

free(src);
free(dst);
}

```

使用示例6

错误示范：通过广播接口向同一地址发送不同数据时，未使用sync_threads进行数据同步，无法保证连续接受数据的安全性。

```

using namespace sdac;
#define SIZE 10
__local__ int dst[SIZE];
__local__ int src[SIZE];

__global__ void func()
{
    ... // 准备广播数据

    const unsigned int root = 0; // 发送方ID
    BroadcastHandle handle; // 创建BroadcastHandle类型变量, 所有SPE均接收数据
    sync_threads(); // 计算核心之间进行一次同步操作
}

```

```
// 错误示范：
// root多次向相同地址广播数据，由于各线程独立执行，因此不能保证各线程执行broadcast_async操作的时序性。
// 例如发送方执行速度较快，在执行第二次broadcast_async时，其他线程还没执行第一次broadcast_async，
// 就可能导致发送方第二次发送的数据覆盖第一次发送的数据，其他线程在执行第一次broadcast_async后接收到的数据
// 实际为发送方第二次发送的数据。
for (int i = 0; i < 100; i++) {
    if (threadIdx == root) {
        broadcast_wait(handle); // 确保发送方发送完src上数据后再修改，同时会等待0号线程接收完成上次发送的数据
        src[0] = i; // 修改src的值
    }
    broadcast_async(dst, src, SIZE * sizeof(int), 0, {0, 0}, root, BroadcastSpmToSpm, handle); // 发
送方线程向其他线程发送数据
    if (threadIdx != root) {
        broadcast_wait(handle); // 等待dst上接收数据
        if (dst[0] == i) ... // 获取dst的值
    }
}

// 正确做法：
// 在处理有可能因线程不同步而导致的数据冲突的问题时，可加上线程同步操作，
// 以保证各线程执行broadcast_async操作的时序性。
for (int i = 0; i < 100; i++) {
    if (threadIdx == root) {
        broadcast_wait(handle); // 确保发送方发送完src上数据后再修改，同时会等待0号线程接收完成上次发送的数据
        src[0] = i; // 修改src的值
    }
    sync_threads(); // 在发送数据前加上同步，确保其他线程进入能接收数据的状态
    broadcast_async(dst, src, SIZE * sizeof(int), 0, {0, 0}, root, BroadcastSpmToSpm, handle); // 发
送方线程向其他线程发送数据
    if (threadIdx != root) {
        broadcast_wait(handle); // 等待dst上接收数据
        if (dst[0] == i) ... // 获取dst的值
    }
}
}
```

7.6.3.4.2 broadcast_wait

```
void broadcast_wait(BroadcastHandle &handle)
void broadcast_wait(BroadcastHandle &handle, unsigned int until_times)
```

功能介绍

等待当前计算核心SPE上所有使用同一个 `BroadcastHandle` 的非阻塞型Broadcast数据搬运操作完成。

参数解释

参数名称	说明
handle	<code>BroadcastHandle</code> 类型变量，表示当前广播组，用于标识接收广播的SPE。

参数名称	说明
until_time	(可选) 当前SPE接收广播数据的次数。  说明： - 不带until_time参数的broadcast_wait接口，需要搭配带root参数的broadcast_async接口使用。 - 带until_time参数的broadcast_wait接口，需要搭配不带root参数的broadcast_async接口使用。

返回值

无。

注意事项

- 需要引入sdac命名空间。
- 使用 `broadcast_wait` 时，不要求所有SPE必须执行该函数接口，但需要注意发送方或者 `BroadcastHandle` 广播组内包含的SPE若未执行 `broadcast_wait`，则不能保证发送和接收广播的操作完成，可参考使用示例。

使用示例1

```
using namespace sdac;
#define SIZE1 10
#define SIZE2 20
#define SIZE3 30
__local__ int dst1[SIZE1];
__local__ int src1[SIZE1];
__local__ int dst2[SIZE2];
__local__ int src2[SIZE2];
__local__ int dst3[SIZE3];
__local__ int src3[SIZE3];

__global__ void func()
{
    ... // 准备广播数据

    ThreadGroup thread_group(0xFFFFFFFF); // 使用mask的方式创建线程组
    BroadcastHandle handle(&thread_group); // 创建BroadcastHandle类型变量，所有SPE均接收数据
    sync_threads(); // 计算核心之间进行一次同步操作

    broadcast_async(dst1, src1, SIZE1 * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 0号SPE
    广播到所有SPE

    thread_group_exclude(thread_group, 0);
    broadcast_async(dst2, src2, SIZE2 * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 0号SPE
    广播到除自身外的所有其他SPE（即自身不接收）
```

```

... // 其他代码逻辑

broadcast_wait(handle); // 等待所有SPE完成发送广播或/与接收广播的操作

thread_group_set_mask(thread_group, 0xFF); // 设置handle, 仅SPE0~7接收广播
broadcast_async(dst3, src3, SIZE3 * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle); // 0号SPE
广播到SPE0~7
if (threadIdx <= 4) {
    broadcast_wait(handle); // 不建议, 无法保证SPE5~7接收完数据
}
if (threadIdx == 0 || thread_group_is_included(thread_group, threadIdx)) {
    broadcast_wait(handle); // 正确, 可以保证SPE0发送完数据, 以及SPE8~15接收完数据
}
broadcast_wait(handle); // 正确, 未参与广播的SPE执行后无需等待即可退出
}

```

使用示例2

```

using namespace sdAA;
#define SIZE1 10
#define SIZE2 20
__local__ int dst1[SIZE1];
__local__ int src1[SIZE1];
__local__ int dst2[SIZE2];
__local__ int src2[SIZE2];

__global__ void func()
{
    ... // 准备广播数据

    ThreadGroup thread_group(0xFFFFFFFF); // 使用mask的方式创建线程组
    BroadcastHandle handle(&thread_group); // 创建BroadcastHandle类型变量, 所有SPE均接收数据
    sync_threads(); // 计算核心之间进行一次同步操作

    if (threadIdx == 0) {
        broadcast_async(dst1, src1, SIZE1 * sizeof(int), 0, {0, 0}, BroadcastSpmToSpm, handle); // 0号SP
E广播到SPA内所有SPE
    }

    ... // 其他代码逻辑

    if (thread_group_is_included(thread_group, threadIdx)) {
        broadcast_wait(handle, 1); // 等待所有SPE完成发送广播或/与接收广播的操作
    }

    thread_group_set_mask(thread_group, 0xFF); // 设置handle, 仅SPE0~7接收广播
    if (threadIdx == 0) {
        broadcast_async(dst2, src2, SIZE2 * sizeof(int), 0, {0, 0}, BroadcastSpmToSpm, handle); // 0号SP
E广播到SPE0~7
    }
    if (threadIdx <= 4) {
        broadcast_wait(handle, 1); // 不建议, 无法保证SPE5~7接收完数据
    }
    if (threadIdx == 0 || thread_group_is_included(thread_group, threadIdx)) {
        broadcast_wait(handle, 1); // 正确, 可以保证SPE0发送完数据, 以及SPE8~15接收完数据
    }
}

```

```
}  
    broadcast_wait(handle, 1); // 错误, 未参与广播的SPE不能执行该操作  
}
```

7.6.3.5 典型场景示例

7.6.3.5.1 模拟核心阵列进行横向、纵向广播

本示例通过将部分SPE自定义为二维阵列组，并模拟行横向和列纵向广播，从而实现二维阵列组中全广播的效果。

编程思路

步骤一：给不同行、不同列的SPE划分广播组。

步骤二：每个SPE分别向同行、同列的SPE广播组上不同地址发送数据。

步骤三：此时每个SPE上应当拥有相同数据，打印比较。

代码示例

```
using namespace sdac;  
#define THREAD_NUM 16 // 设置共16个SPE参与广播  
#define ROW_NUM 4 // 划分为4行4列  
#define COL_NUM 4  
  
#define IS_SAME_ROW(i, j) (i / COL_NUM == j / COL_NUM)  
#define IS_SAME_COL(i, j) (i % COL_NUM == j % COL_NUM)  
  
// 声明SPM内存空间  
__local__ int local_value;  
__local__ int all_data1[THREAD_NUM];  
__local__ int all_data2[THREAD_NUM];  
  
__global__ void testBroadcast()  
{  
    // 计算每个SPE在当前阵列中的行列位置  
    const unsigned int row_id = threadIdx / COL_NUM;  
    const unsigned int col_id = threadIdx % COL_NUM;  
    const size_t size = sizeof(int);  
  
    ThreadGroup row_thread_group, col_thread_group;  
  
    // 每个SPE上的线程组为包括自身在内的同行同列广播组  
    for (int i = 0; i < THREAD_NUM; i++) {  
        if (IS_SAME_ROW(i, threadIdx)) {  
            thread_group_include(row_thread_group, i);  
        }  
        if (IS_SAME_COL(i, threadIdx)) {  
            thread_group_include(col_thread_group, i);  
        }  
    }  
  
    // 定义控制行列方向上广播组的BroadcastHandle类型变量
```

```

BroadcastHandle row_handle(&row_thread_group);
BroadcastHandle col_handle(&col_thread_group);
sync_threads();

local_value = threadIdx;
// 先将自身数据广播到当前行, 后将包含当前行的所有数据广播到当前列
broadcast_async(all_data1 + threadIdx, &local_value, size, 0, {0, 0}, threadIdx,
               BroadcastSpmToSpm, row_handle);

broadcast_wait(row_handle);

broadcast_async(all_data1 + row_id * COL_NUM, all_data1 + row_id * COL_NUM,
               size * COL_NUM, 0, {0, 0}, threadIdx, BroadcastSpmToSpm, col_handle);
broadcast_wait(col_handle);

// 打印结果验证
if (threadIdx == 0) {
    for (int i = 0; i < THREAD_NUM; i++) printf("%d ", all_data1[i]);
    printf("\n");
}

// 上述代码应与全广播效果相同
ThreadGroup all_thread_group(0xFFFF);
BroadcastHandle all_handle(&all_thread_group);
broadcast(all_data2 + threadIdx, &local_value, size, 0, {0, 0}, threadIdx, BroadcastSpmToSpm, all_h
andle);

// 打印结果验证
if (threadIdx == 0) {
    for (int i = 0; i < THREAD_NUM; i++) printf("%d ", all_data2[i]);
    printf("\n");
}
}

int main() {
    // 设置使用0号计算核心阵列
    sdaaSetDevice(0);
    // 核函数调用
    testBroadcast<<<1>>>();
    sdaaDeviceSynchronize();
    return 0;
}

```

7.6.4 跨步类型

7.6.4.1 概述

跨步是指从一段连续的数据中每隔一段固定的距离获取一段固定长度的数据，跨步由以下函数接口组成：

- **MemoryStride**：配置跨步数据搬运的结构体变量。

跨步方式

SDAA C目前支持以下跨步方式：

Global存储空间向SPM存储空间

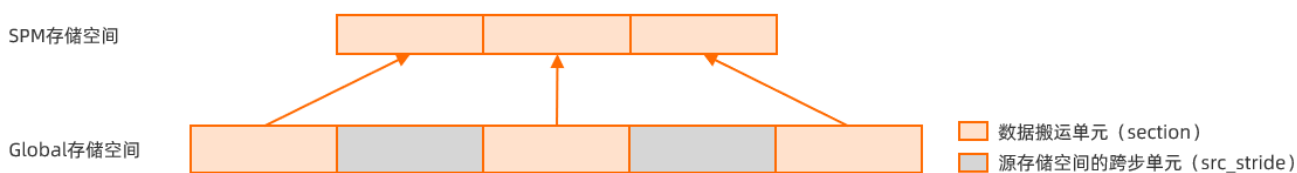


图1 Global存储空间向SPM存储空间

SPM存储空间向Global存储空间

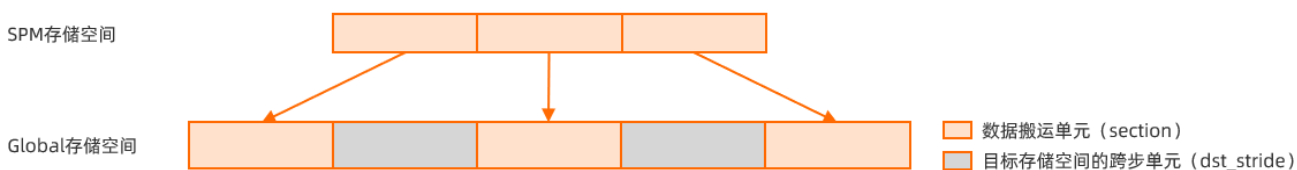


图2 SPM存储空间向Global存储空间

Global存储空间向Global存储空间

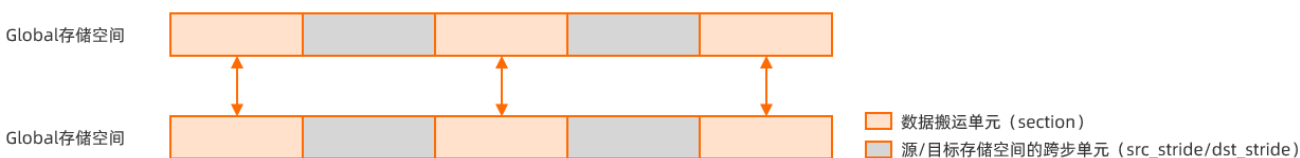


图3 Global存储空间向Global存储空间

SPM存储空间向SPM存储空间

? 说明：

不支持在SPM存储空间上设置跨步单元的大小。因此，SPM存储空间向SPM存储空间仅支持搬运连续数据。

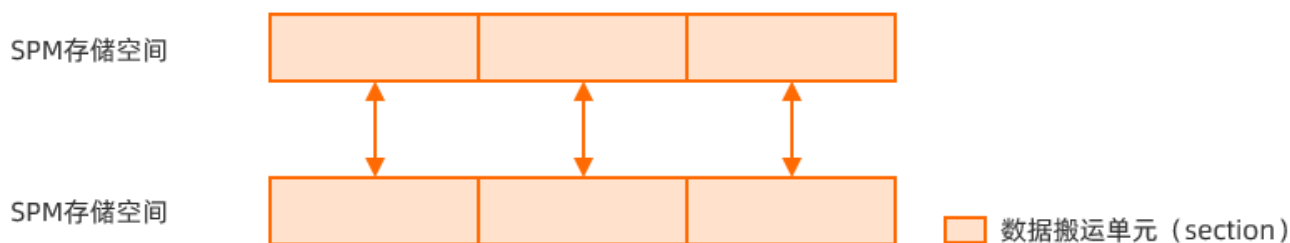


图4 SPM存储空间向SPM存储空间

7.6.4.2 MemoryStride

```
typedef struct {
    size_t dst_stride;
```



```
size_t src_stride;
} MemoryStride;
```

功能介绍

配置跨步数据搬运的结构体变量。

参数解释

变量名称	说明
dst_stride	目标存储空间的跨步单元大小，单位：Byte。  说明： - 仅支持在Global存储空间，设置该参数。 - 若目标存储空间位于SPM，则该变量值无效。
src_stride	源存储空间的跨步单元大小，单位：Byte。  说明： - 仅支持在Global存储空间，设置该参数。 - 若目标源存储空间位于SPM，则该变量值无效。

注意事项

- 需要引入sdac命名空间。
- 如果参数 `dst_stride` 和 `src_stride` 都设置为0，则为连续数据搬运。
- 受硬件限制，目前仅支持在Global存储空间，设置跨步单元大小。

使用示例1

配置跨步参数，创建MemoryStride类型变量。

```
using namespace sdAA;
__global__ void func()
{
    // 创建MemoryStride类型变量, 设置目标存储空间的跨步
    MemoryStride stride = {stride_size, 0};

    // 其他代码逻辑
}
```

使用示例2

配置跨步参数, 不创建MemoryStride类型变量。

```
using namespace sdAA;
__global__ void func()
{
    constexpr size_t stride_size = 2 * sizeof(int);
    // 其他代码逻辑
    // ...

    // 配置跨步参数, 进行非阻塞跨步拷贝
    memcpY_async(g_dst, spm_src, size, section_size, {stride_size, 0},
                 MemcpYspmToGlobal);

    // 其他代码逻辑
}
```

7.7 原子操作

7.7.1 概述

多线程编程模型中, 原子操作 (Atomic Operation) 可以保证多线程对同一块内存访问的串行性, 从而提高数据操作的安全性和正确性。SDAA C提供的原子操作主要有以下几类:

- **atomic_inc**: 读取Global上的一个数据并使其自增, 此操作不会被打断, 返回旧值。
- **atomic_add**: 使Global上的一个数据增加某值, 此操作不会被打断, 返回旧值。
- **atomic_add_noret**: 使Global上的一个数据增加某值, 此操作不会被打断, 无返回值。
- **atomic_sub**: 使Global上的一个数据减少某值, 此操作不会被打断, 返回旧值。
- **atomic_sub_noret**: 使Global上的一个数据减少某值, 此操作不会被打断, 无返回值。
- **atomic_cas**: 满足条件时, 使Global上的一个数据替换为某值, 此操作不会被打断, 返回旧值。
- **atomic_cas_bool**: 满足条件时, 使Global上的一个数据替换为某值, 此操作不会被打断, 返回布尔值。

7.7.2 atomic_inc

```
int64_t atomic_inc(int64_t* val)
```

功能介绍

对目标操作数加1，操作数得到新值，旧值作为返回值返回。

参数解释

参数名称	说明
val	待计算的操作数指针，需要位于Global存储空间。

返回值

返回值类型	说明
long	返回的旧值。

注意事项

- 需要手动包含 `sdaa_atomic.h` 头文件。
- 需要引入sdaa命名空间。

使用示例1

```
#include <sdaa_atomic.h>
using namespace sdaa;

__device__ int64_t val = 0;
__global__ void calc()
{
    ...
    int64_t ret;
    ret = atomic_inc(&val);

    ...
}

// ret 的值在不同线程上是不同的，且依次加一 例如: 1, 2, 3, 4, 5, 6... 与线程顺序无关
```

使用示例2

```
#include <sdaa_atomic.h>
using namespace sdaa;
// val 是用 sdaaMalloc 在全局内存上创建
__global__ void calc(int32_t *val)
{
```

```
...
int64_t ret;
ret = atomic_inc(val);

...
}
// ret 的值在不同线程上是不同的，且依次加一 例如: *val+1, *val+2, *val+3, *val+4, *val+5... 与线程顺序无关
```

相关推荐

更详细的原子操作使用示例，可参考[使用CAS完成自定义原子操作](#)。

7.7.3 atomic_add

```
int32_t atomic_add(int32_t* val, int32_t num)
int64_t atomic_add(int64_t* val, int64_t num)
float   atomic_add(float* val, float num)
double  atomic_add(double* val, double num)
```

功能介绍

对目标操作数加 `num`，操作数得到新值，返回旧值。

参数解释

参数名称	说明
val	待计算的操作数指针，需要位于Global存储空间。
num	增加的数值，即被加到操作数上的值。

返回值

返回旧值。

注意事项

- 需要手动包含 `sdaa_atomic.h` 头文件。
- 需要引入 `sdaa` 命名空间。

使用示例1

```
#include <sdaa_atomic.h>
using namespace sdaa;

__device__ int32_t val = 0;
__global__ void calc()
```

```
{
    ...
    int32_t old = atomic_add(&val, 10);
    ...
}
// val 的值在不同线程上是不同的，且依次加10 例如：10, 20, 30, 40, 50... 且与线程顺序无关
// old 的值则是val的值在加10之前的旧值
```

使用示例2

```
#include <sdaa_atomic.h>
using namespace sdaa;
// val 是用 sdaaMalloc 在全局内存上创建
__global__ void calc(int32_t *val)
{
    ...
    int32_t old = atomic_add(val, 10);
    ...
}
// *val 的值在不同线程上是不同的，且依次加10 例如：*val+10, *val+20, *val+30, *val+40, *val+50... 与线程顺序无关
// old 的值则是*val的值在加10之前的旧值
```

相关推荐

更详细的原子操作使用示例，可参考[使用CAS完成自定义原子操作](#)。

7.7.4 atomic_add_noret

```
void atomic_add_noret(int32_t* val, int32_t num)
void atomic_add_noret(int64_t* val, int64_t num)
```

功能介绍

对目标操作数加 `num`，操作数得到新值，无返回值。

参数解释

参数名称	说明
val	待计算的操作数指针，需要位于Global存储空间。
num	增加的数值，即被加到操作数上的值。

返回值

无。

注意事项

- 需要手动包含 `sdaa_atomic.h` 头文件。
- 需要引入 `sdaa` 命名空间。

使用示例1

```
#include <sdaa_atomic.h>
using namespace sdaa;
__device__ int32_t val = 0;
__global__ void calc()
{
    ...
    atomic_add_noret(&val, 10);
    ...
}
// val 的值在不同线程上是不同的, 且依次加10 例如: 10, 20, 30, 40, 50... 且与线程顺序无关
```

使用示例2

```
#include <sdaa_atomic.h>
using namespace sdaa;
// val 是用 sdaaMalloc 在全局内存上创建
__global__ void calc(int32_t *val)
{
    ...
    atomic_add_noret(val, 10);
    ...
}
// *val 的值在不同线程上是不同的, 且依次加10 例如: *val+10, *val+20, *val+30, *val+40, *val+50... 与线程顺序无关
```

相关推荐

更详细的原子操作使用示例, 可参考[使用CAS完成自定义原子操作](#)。

7.7.5 atomic_sub

```
int32_t atomic_sub(int32_t* val, int32_t num)
int64_t atomic_sub(int64_t* val, int64_t num)
```

功能介绍

对目标操作数减 `num`, 操作数得到新值, 返回旧值。

参数解释

参数名称	说明
val	待计算的操作数指针，需要位于Global存储空间。
num	减小的数值，即从操作数上减去的值。

返回值

返回旧值。

注意事项

- 需要手动包含 `sdaa_atomic.h` 头文件。
- 需要引入sdaa命名空间。

使用示例1

```
#include <sdaa_atomic.h>
using namespace sdaa;
__device__ int32_t val = 0;
__global__ void calc()
{
    ...
    int32_t old = atomic_sub(&val, 10);
    ...
}
// val 的值在不同线程上是不同的，且依次减10 例如：-10, -20, -30, -40, -50... 且与线程顺序无关
// old 的值则是val的值在减10之前的旧值
```

使用示例2

```
#include <sdaa_atomic.h>
using namespace sdaa;
// val 是用 sdaaMalloc 在全局内存上创建
__global__ void calc(int32_t *val)
{
    ...
    int32_t old = atomic_sub(val, 10);
    ...
}
// *val 的值在不同线程上是不同的，且依次减10 例如：*val-10, *val-20, *val-30, *val-40, *val-50... 与线程顺序无关
// old 的值则是val的值在减10之前的旧值
```

相关推荐

更详细的原子操作使用示例，可参考[使用CAS完成自定义原子操作](#)。

7.7.6 atomic_sub_noret

```
void atomic_sub_noret(int32_t* val, int32_t num)
void atomic_sub_noret(int64_t* val, int64_t num)
```

功能介绍

对目标操作数减 `num`，操作数得到新值，无返回值。

参数解释

参数名称	说明
val	待计算的操作数指针，需要位于Global存储空间。
num	减小的数值，即从操作数上减去的值。

返回值

无。

注意事项

- 需要手动包含 `sdaa_atomic.h` 头文件。
- 需要引入 `sdaa` 命名空间。

使用示例1

```
#include <sdaa_atomic.h>
using namespace sdaa;
__device__ int32_t val = 0;
__global__ void calc()
{
    ...
    atomic_sub_noret(&val, 10);
    ...
}
// val 的值在不同线程上是不同的，且依次减10 例如：-10, -20, -30, -40, -50... 且与线程顺序无关
```

使用示例2

```
#include <sdaa_atomic.h>
using namespace sdaa;
// val 是用 sdaaMalloc 在全局内存上创建
__global__ void calc(int32_t *val)
{
    ...
    atomic_sub_noret(val, 10);
    ...
}
```



```
}
// *val 的值在不同线程上是不同的，且依次减10 例如: *val-10, *val-20, *val-30, *val-40, *val-50... 与线程顺序无关
```

相关推荐

更详细的原子操作使用示例，可参考[使用CAS完成自定义原子操作](#)。

7.7.7 atomic_cas

```
int32_t atomic_cas(int32_t* val, int32_t compare, int32_t new_val)
int64_t atomic_cas(int64_t* val, int64_t compare, int64_t new_val)
```

功能介绍

如果目标操作数等于 `compare`，则把 `new_val` 赋值给操作数，返回旧值。

参数解释

参数名称	说明
val	待计算的目标操作数指针，需要位于Global存储空间。
compare	比较值，与目标操作数比较。
new_val	新值，如果目标操作数与 <code>compare</code> 相同，则将新值赋给操作数。

返回值

返回旧值。

注意事项

- 需要手动包含 `sdaa_atomic.h` 头文件。
- 需要引入sdaa命名空间。

使用示例1

```
#include <sdaa_atomic.h>
using namespace sdaa;
__device__ int32_t val = 1;
__global__ void calc()
{
    ...
    // 有一个线程返回旧值1，且把新值5赋给val。其他线程返回旧值5，且val值为5
    int32_t old = atomic_cas(&val, 1, 5);
```

```
...  
}
```

使用示例2

```
#include <sdaa_atomic.h>  
using namespace sdaa;  
// val 是用 sdaaMalloc 在全局内存上创建  
__global__ void calc(int32_t *val)  
{  
    ...  
    // 若传入的*val的值与1相等, 则有一个线程返回旧值1, 且把新值5赋给*val。其他线程返回旧值5, 且*val值为5  
    // 若传入的*val的值与1不相等, 所有线程返回*val旧值, 且*val保持旧值  
    int32_t old = atomic_cas(val, 1, 5);  
    ...  
}
```

使用示例3

```
#include <sdaa_atomic.h>  
using namespace sdaa;  
__device__ int32_t val = 1;  
__global__ void calc()  
{  
    ...  
    // 所有线程返回旧值 1, 且val的值保持1  
    int32_t old = atomic_cas(&val, 0, 5);  
    ...  
}
```

相关推荐

更详细的原子操作使用示例, 可参考[使用CAS完成自定义原子操作](#)。

7.7.8 atomic_cas_bool

```
bool atomic_cas_bool(int32_t* val, int32_t compare, int32_t new_val)  
bool atomic_cas_bool(int64_t* val, int64_t compare, int64_t new_val)
```

功能介绍

如果目标操作数等于 `compare` , 则把 `new_val` 赋值给操作数, 返回true; 否则返回 false。

参数解释

参数名称	说明
val	待计算的目标操作数指针，需要位于Global存储空间。
compare	比较值，与目标操作数比较。
new_val	新值，如果目标操作数与 <code>compare</code> 相同，则将新值赋给操作数。

返回值

返回值类型	说明
bool	赋值成功返回true；否则返回false。

注意事项

- 需要手动包含 `sdaa_atomic.h` 头文件。
- 需要引入sdaa命名空间。

使用示例1

```
#include <sdaa_atomic.h>
using namespace sdaa;
__device__ int32_t val = 1;
__global__ void calc()
{
    ...
    // 有一个线程返回true，且val值更新为5，其他线程返回false，val值为5
    bool ret = atomic_cas_bool(&val, 1, 5);
    ...
}
```

使用示例2

```
#include <sdaa_atomic.h>
using namespace sdaa;
// val 是用 sdaaMalloc 在全局内存上创建
__global__ void calc(int32_t *val)
{
    ...
    // 若传入的*val的值与1相等，则有一个线程返回true，且把新值5赋给val。其他线程返回false，且val值为5
    // 若传入的*val的值与1不相等，所有线程返回false，且*val保持旧值
    bool ret = atomic_cas_bool(val, 1, 5);
    ...
}
```

使用示例3

```
#include <sdaa_atomic.h>
using namespace sdaa;
__device__ int32_t val = 1;
__global__ void calc()
{
    ...
    // 所有线程返回false, val值保持为1
    bool ret = atomic_cas_bool(&val, 0, 5);
    ...
}
```

相关推荐

更详细的原子操作使用示例，可参考[使用CAS完成自定义原子操作](#)。

7.8 矩阵乘

7.8.1 概述

矩阵乘函数接口实现了高效的二维矩阵乘法计算，从调用接口上可分为阻塞型矩阵乘和非阻塞型矩阵乘：

- 阻塞型矩阵乘：使用简单，但性能不佳。
- 非阻塞型矩阵乘：使用相对复杂，但性能较好。

阻塞型矩阵乘

阻塞型矩阵乘由以下函数接口组成：

- **matmul**：进行矩阵乘计算，矩阵地址可以位于SPM或Global上，且矩阵的维度可变，同时支持原位操作。
- **check_matmul**：验证matmul函数输入参数的合法性，通过返回值判断输入参数是否满足特定的输入要求，如：地址对齐、当前接口占用SPM堆空间不能超出剩余可用堆空间等。

非阻塞型矩阵乘

非阻塞型矩阵乘可分为配置型操作和调用型操作。

- **配置型操作**：通过修改参数完成多种矩阵乘功能，由以下函数接口组成：
 - **MatmulHandle**：配置矩阵乘的参数，从而使用不同的矩阵乘功能。
 - **matmul_set_updating_weight**：设置是否更新权重矩阵。
 - **matmul_set_flushing_output**：设置是否可以输出计算结果到SPM空间。
 - **matmul_set_weight_row_padding**：通过掩码的方式，设置加载权重矩阵时，是否填充空行。
 - **matmul_set_output_row_offset**：设置计算结果的行偏移。
 - **matmul_wait_loading_input**：等待输入矩阵加载完成。

- `matmul_wait_loading_weight`: 等待权重矩阵加载完成。
- **调用型操作**: 是调用矩阵乘进行计算的必要步骤，由以下函数接口组成：
 - `matmul_init`: 初始化矩阵乘接口，并配置输入和输出数据的类型。
 - `matmul_load_weight`: 从SPM指定地址加载权重矩阵到矩阵乘计算单元上。
 - `matmul_compute`: 从SPM指定地址加载输入矩阵，并与加载好的权重矩阵做矩阵乘运算。
 - `matmul_store`: 将矩阵乘运算的计算结果写回到SPM指定地址上。
 - `matmul_wait`: 等待矩阵乘写回操作完成。

7.8.2 阻塞型矩阵乘

7.8.2.1 matmul

```
void matmul(MatmulDataType type, void *output, const void *input, const void *weight, unsigned int m, unsigned int k, unsigned int n)
```

功能介绍

计算两个矩阵的乘法。支持原位操作，且矩阵的维度可变。

形式： $output_{m \times n} = input_{m \times k} \times weight_{k \times n}$

参数解释

参数名称	说明
type	枚举型MatmulDataType，表示参与矩阵乘运算的数据类型，当前支持： • MatmulHalfToHalf: 16位半精度浮点型运算，输出16位半精度浮点型。 • MatmulHalfToFloat: 16位半精度浮点型运算，输出32位单精度浮点型。 • MatmulShortToShort: 16位有符号短整型运算，输出16位有符号短整型。 • MatmulShortToInt: 16位有符号短整型运算，输出32位有符号整型。
output	输出矩阵首地址，可位于SPM或Global。
input	输入矩阵首地址，可位于SPM或Global。
weight	权重矩阵首地址，可位于SPM或Global。
m	输入矩阵和输出矩阵的行数，无大小限制。

参数名称	说明
k	输入矩阵列数，也是权重矩阵行数，无大小限制。
n	权重矩阵和输出矩阵的列数，无大小限制。

返回值

无。

注意事项

输入矩阵input、权重矩阵weight和输出矩阵output可以位于Global或SPM。

- 需要手动包含sdaa_matmul.h头文件。
- 需要引入sdaa命名空间。
- 对齐要求：
 - 如果某矩阵位于Global：
 1. 该矩阵大小需为4B的整数倍，以满足DMA传输量的大小限制。
 2. 该矩阵首地址需要4B对齐，以满足DMA要求。
 - 如果某矩阵位于SPM：
 1. 如果满足 `k == 32 && n == 32`，且该矩阵为output和input，则该矩阵首地址需要64B对齐。
 2. 否则，如果满足 `k % 32 == 0`，且该矩阵为input，则该矩阵首地址需要64B对齐。
- 当前matmul接口会额外占用SPM堆空间情况：
 - 如果满足以下条件之一，需要额外占用对应矩阵数据大小的SPM堆空间；
 - 输入参数中矩阵地址位于Global。
 - `n <= 32` 且输入参数中output矩阵地址与input或weight地址空间有重叠。
 - 如果数据类型是 MatmulHalfToHalf 或 MatmulShortToShort：
 1. 满足 `k == 32 && n == 32` 时，不需要额外占用SPM堆空间。
 2. 不满足条件1，但满足 `k % 32 == 0 && n % 32 == 0` 时，需要额外占用 10KB 的SPM堆空间。
 3. 如果不满足以上两种情况，需要额外占用 18KB 的SPM堆空间。
 - 如果数据类型是 MatmulHalfToFloat 或 MatmulShortToInt：
 1. 满足 `k == 32 && n == 32` 时，不需要额外占用SPM堆空间。
 2. 不满足条件1，但满足 `k % 32 == 0 && n % 32 == 0` 时，需要额外占用 18KB 的SPM堆空间。
 3. 如果不满足以上两种情况，需要额外占用 26KB 的SPM堆空间。

使用示例1

矩阵位于Global时调用 `matmul` 。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void test(const void *data_a, const void *data_b, void *data_c)
{
    if (threadIdx != 0) { return; }
    // 设置matmul输入数据类型为half, 输出为float
    matmul(MatmulHalfToFloat, data_c, data_a, data_b, 200, 100, 100);
}

int main()
{
    ...
    half *dev_a = NULL;
    half *dev_b = NULL;
    float *dev_c = NULL;
    sdaaMalloc((void **)&dev_a, 200 * 100 * sizeof(half));
    sdaaMalloc((void **)&dev_b, 100 * 100 * sizeof(half));
    sdaaMalloc((void **)&dev_c, 200 * 100 * sizeof(float));
    ...
    // 准备矩阵数据
    ...
    test<<<1>>>>(dev_a, dev_b, dev_c);
    ...
}
```

使用示例2

矩阵位于SPM时调用 `matmul` 。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void test(const void *data_a, const void *data_b, void *data_c)
{
    if (threadIdx != 0) { return; }
    // 在SPM上申请输入矩阵数据空间
    half *input = (half *)malloc(200 * 100 * sizeof(half));
    memcpy_async(input, data_a, 200 * 100 * sizeof(half), 0, {0, 0}, MemcpyGlobalToSpm);
    // 在SPM上申请权重矩阵数据空间
    half *weight = (half *)malloc(100 * 100 * sizeof(half));
    memcpy_async(weight, data_b, 100 * 100 * sizeof(half), 0, {0, 0}, MemcpyGlobalToSpm);
    // 在SPM上申请输出矩阵数据空间
    float *output = (float *)malloc(200 * 100 * sizeof(float));
    memcpy_wait();
    // 进行同步的矩阵乘法
    matmul(MatmulHalfToFloat, output, input, weight, 200, 100, 100);
    // 将计算完成的数据从SPM上copy到Global数据空间上
    memcpy(data_c, output, 200 * 100 * sizeof(float));

    free(output);
    free(weight);
    free(input);
}
```

```

}

int main()
{
    ...
    half *dev_a = NULL;
    half *dev_b = NULL;
    float *dev_c = NULL;
    sdaaMalloc((void **)&dev_a, 200 * 100 * sizeof(half));
    sdaaMalloc((void **)&dev_b, 100 * 100 * sizeof(half));
    sdaaMalloc((void **)&dev_c, 200 * 100 * sizeof(float));
    ...
    // 准备矩阵数据
    ...
    test<<<1>>>(dev_a, dev_b, dev_c);
    ...
}

```

使用示例3

支持原位计算。

```

#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void test(void *data_a)
{
    if (threadIdx != 0) { return; }
    // 申请一块数据空间，既是输入矩阵，权重矩阵数据空间，也是输出矩阵空间
    half *spm_a = (half *)malloc(32 * 32 * sizeof(half));
    memcpy(spm_a, data_a, 32 * 32 * sizeof(half));
    // 进行同步矩阵乘法
    matmul(MatmulHalfToHalf, spm_a, spm_a, spm_a, 32, 32, 32);

    memcpy(data_a, spm_a, 32 * 32 * sizeof(half));

    free(spm_a);
}

int main()
{
    ...
    half *dev_a = NULL;
    sdaaMalloc((void **)&dev_a, 32 * 32 * sizeof(half));
    ...
    // 准备矩阵数据
    ...
    test<<<1>>>(dev_a);
    ...
}

```

7.8.2.2 check_matmul


```
unsigned int check_matmul(MatmulDataType type, void *output, const void *input, const void *weight, unsigned int m, unsigned int k, unsigned int n)
```

功能介绍

验证阻塞型矩阵乘接口输入参数的合法性。

参数解释

参数名称	说明
type	枚举型MatmulDataType，表示参与矩阵乘运算的数据类型，当前支持： - MatmulHalfToHalf：16位半精度浮点型运算，输出16位半精度浮点型。 - MatmulHalfToFloat：16位半精度浮点型运算，输出32位单精度浮点型。 - MatmulShortToShort：16位有符号短整型运算，输出16位有符号短整型。 - MatmulShortToInt：16位有符号短整型运算，输出32位有符号整型。
output	输出矩阵首地址，可位于SPM或Global。
input	输入矩阵首地址，可位于SPM或Global。
weight	权重矩阵首地址，可位于SPM或Global。
m	输入矩阵和输出矩阵的行数，无大小限制。
k	输入矩阵列数，也是权重矩阵行数，无大小限制。
n	权重矩阵和输出矩阵的列数，无大小限制。

返回值

返回值类型	说明
unsigned int	参数合法性验证结果，每个Bit代表不同的状态信息。对应枚举型 MatmulStatus，表示合法性验证的状态信息，当前可表示状态有： - MATMUL_SUCC：参数无误。 - MATMUL_ERROR_DMA_SIZE_UNALIGNED：各矩阵大小（即DMA传输量）不是4B整数倍。 - MATMUL_ERROR_GLOBAL_ADDR_UNALIGNED：输入矩阵首地址在Global上但没有4B对齐。 - MATMUL_ERROR_SPM_ADDR_UNALIGNED：输入矩阵首地址在SPM上但没有64B对齐。 - MATMUL_ERROR_SPM_OVERSIZE：当前接口占用SPM堆空间超出剩余可用堆空间。

返回的具体状态信息可参照如下表格：

第0位	第1位	第2位	第3位	第4位	第5~31位
0：参数无误。	第0位是0时，此位保留。	第0位是0时，此位保留。	第0位是0时，此位保留。	第0位是0时，此位保留。	保留
1：参数有误。	第0位是1时，此位： 0：正常。 1：错误。各矩阵大小（即DMA传输量）不是4B整数倍。	第0位是1时，此位： 0：正常。 1：错误。输入矩阵首地址在Global上但没有4B对齐。	第0位是1时，此位： 0：正常。 1：错误。输入矩阵首地址在SPM上但没有64B对齐。	第0位是1时，此位： 0：正常。 1：错误。当前接口占用SPM堆空间超出剩余可用堆空间。	

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。

使用示例

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void test(const void *data_a, const void *data_b, void *data_c)
```

```
{  
    if (threadIdx != 0) { return; }  
    // 矩阵乘参数合法化检查  
    int check_val = check_matmul(MatmulHalfToFloat, data_c, data_a, data_b, 200, 100, 100);  
    if (check_val == MATMUL_SUCC) {  
        matmul(MatmulHalfToFloat, data_c, data_a, data_b, 200, 100, 100);  
    }  
}
```

7.8.3 非阻塞型矩阵乘

7.8.3.1 使用逻辑

非阻塞型矩阵乘接口可分为调用型接口和配置型接口，其中：

- 调用型接口：是非阻塞型矩阵乘的必要接口步骤。
- 配置型接口：通过修改相关配置项修改部分调用型接口的行为。

非阻塞型矩阵乘接口的使用逻辑为：

1. **matmul_init**：初始化矩阵乘计算单元，并配置输入输出的数据类型。
2. **matmul_load_weight**：从SPM的指定地址加载权重矩阵到权重矩阵加载缓冲区。

? 说明：

可以通过配置型接口**matmul_set_weight_row_padding**设置加载权重矩阵时，是否填充空行，该接口作用于当前和之后阶段的**matmul_load_weight**。

3. **matmul_compute**：从SPM加载输入矩阵并计算与权重矩阵的乘积。

? 说明：

以下接口作用于当前和之后阶段的**matmul_compute**。

- 可以通过配置型接口**matmul_set_updating_weight**设置是否从权重矩阵加载缓冲区更新权重矩阵。
- 可以通过配置型接口**matmul_set_output_row_offset**设置计算结果写入计算结果累加缓冲区时的行偏移。
- 可以通过配置型接口**matmul_set_flushing_output**设置是否使能从累加缓冲区向SPM输出计算结果。

4. **matmul_store**：从计算结果累加缓冲区输出计算结果到SPM的指定地址。
5. **matmul_wait**：等待矩阵乘的计算结果写出完成。

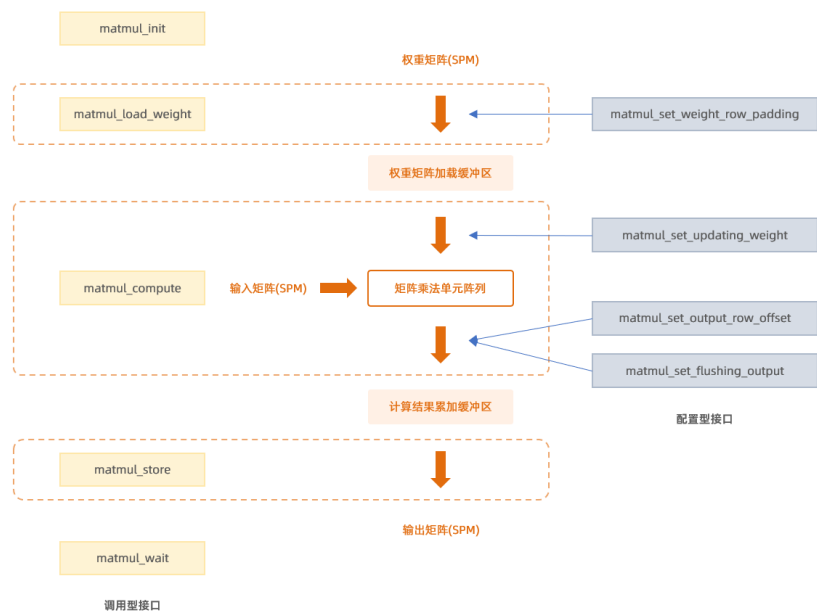


图1 非阻塞型矩阵乘的使用逻辑

7.8.3.2 调用型操作

7.8.3.2.1 matmul_init

```
void matmul_init(MatmulHandle &handle, MatmulDataType type)
```

功能介绍

初始化矩阵乘接口，并配置输入和输出数据的类型。

参数解释

参数名称	说明
handle	MatmulHandle 型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。
type	枚举型MatmulDataType，表示参与矩阵乘运算的数据类型，当前支持： - MatmulHalfToHalf：16位半精度浮点型运算，输出16位半精度浮点型。 - MatmulHalfToFloat：16位半精度浮点型运算，输出32位单精度浮点型。 - MatmulShortToShort：16位有符号短整型运算，输出16位有符号短整型。 - MatmulShortToInt：16位有符号短整型运算，输出32位有符号整型。

返回值

无。

注意事项

- 需要手动包含 `sdaa_matmul.h` 头文件。
- 需要引入 `sdaa` 命名空间。
- 此接口隐含栅栏语义，会等待之前的所有矩阵乘运算完成。
- 暂不支持 `int8` 型矩阵乘计算。

使用示例1

使用 `matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input` 共同完成 $100 \times 32 \times 32$ 的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵，32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵，100*32个half型数据，并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址，100*32个float型数据
    matmul_store(handle, output, 100, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}
```

使用示例2

使用 `matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight` 共同

完成200*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵前半, 128*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 128, MatmulK32);

    // 输出计算结果写回到指定地址, 128*32个float型数据
    matmul_store(handle, output, 128, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);

    // 不使能更新权重矩阵, 即使用上次加载的权重矩阵
    matmul_set_updating_weight(handle, false);

    // 加载输入矩阵后半, 72*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + 128 * MatmulK32, 72, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 72*32个float型数据
    matmul_store(handle, output + 128 * MatmulN32, 72, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}
```

使用示例3

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_flushing_output`共同完成100*64*32的矩阵乘运算。

```

#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 不使能计算结果输出, 即暂存入累加缓冲区
    matmul_set_flushing_output(handle, false);

    // 加载权重矩阵上半, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 跨步加载输入矩阵左半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32, 1);

    // 使能计算结果输出, 下次计算结果累加后输出
    matmul_set_flushing_output(handle, true);

    // 加载权重矩阵下半, 32*32个half型数据
    matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 跨步加载输入矩阵右半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + MatmulK32, 100, MatmulK32, 1);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 100*32个float型数据
    matmul_store(handle, output, 100, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}

```

7.8.3.2.2 matmul_load_weight

```

void matmul_load_weight(MatmulHandle &handle, void *weight, MatmulKSize k, MatmulNSize n, unsigned int
stride = 0)

```

功能介绍

从SPM指定地址加载权重矩阵到矩阵乘计算单元上。

参数解释

参数名称	说明
handle	MatmulHandle型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。
weight	权重矩阵首地址，需位于SPM。
k	权重矩阵行数。枚举类型MatmulKSize，当前支持： MatmulK32：权重矩阵行数为32。
n	权重矩阵列数。枚举类型MatmulNSize，当前支持： MatmulN32：权重矩阵列数为32。
stride	加载权重矩阵的跨步步长。 ? 说明： 预留项，目前仅支持步长为0。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。
- 当前权重矩阵、输入矩阵和输出矩阵的行列要求为：

权重矩阵	输入矩阵	输出矩阵
MatmulK32 * MatmulN32	(1~128) * MatmulK32	(1~128) * MatmulN32

使用示例1

使用matmul_init、matmul_load_weight、matmul_compute、matmul_store、matmul_wait、matmul_wait_loading_weight、matmul_wait_loading_input共同完成100*32*32的矩阵乘运算。

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void func()
{
```



```

... // 准备矩阵数据

// 创建MatmulHandle类型变量, 使用默认参数
MatmulHandle handle;

// 初始化矩阵乘接口并配置数据类型
matmul_init(handle, MatmulHalfToFloat);

// 加载权重矩阵, 32*32个half型数据
matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

// 等待权重矩阵加载完成
matmul_wait_loading_weight(handle);

... // 此时可修改weight地址上数据

// 加载输入矩阵, 100*32个half型数据, 并开始矩阵乘运算
matmul_compute(handle, input, 100, MatmulK32);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址, 100*32个float型数据
matmul_store(handle, output, 100, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}

```

使用示例2

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同完成200*32*32的矩阵乘运算。

```

#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据
}

```

```

// 加载输入矩阵前半, 128*32个half型数据, 并开始矩阵乘运算
matmul_compute(handle, input, 128, MatmulK32);

// 输出计算结果写回到指定地址, 128*32个float型数据
matmul_store(handle, output, 128, MatmulN32);

// 等待写回完成
matmul_wait(handle);

// 不使能更新权重矩阵, 即使用上次加载的权重矩阵
matmul_set_updating_weight(handle, false);

// 加载输入矩阵后半, 72*32个half型数据, 并开始矩阵乘运算
matmul_compute(handle, input + 128 * MatmulK32, 72, MatmulK32);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址, 72*32个float型数据
matmul_store(handle, output + 128 * MatmulN32, 72, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}

```

使用示例3

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_flushing_output`共同完成100*64*32的矩阵乘运算。

```

#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 不使能计算结果输出, 即暂存入累加缓冲区
    matmul_set_flushing_output(handle, false);

    // 加载权重矩阵上半, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 跨步加载输入矩阵左半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32, 1);
}

```

```
// 使能计算结果输出，下次计算结果累加后输出
matmul_set_flushing_output(handle, true);

// 加载权重矩阵下半，32*32个half型数据
matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);

// 等待权重矩阵加载完成
matmul_wait_loading_weight(handle);

... // 此时可修改weight地址上数据

// 跨步加载输入矩阵右半，100*32个half型数据，并开始矩阵乘运算
matmul_compute(handle, input + MatmulK32, 100, MatmulK32, 1);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址，100*32个float型数据
matmul_store(handle, output, 100, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}
```

7.8.3.2.3 matmul_compute

```
void matmul_compute(MatmulHandle &handle, void *input, unsigned int m, MatmulKSize k, unsigned int stride = 0)
```

功能介绍

从SPM指定地址加载输入矩阵，并与加载好的权重矩阵做矩阵乘运算。

参数解释

参数名称	说明
handle	MatmulHandle 型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。
input	输入矩阵首地址，需位于SPM。
m	输入矩阵行数。有效值：1~128。
k	输入矩阵列数。枚举类型MatmulKSize，当前支持： MatmulK32：输入矩阵列数为32。

参数名称	说明
stride	加载输入矩阵的跨步步长。 ? 说明： 跨步步长 = stride * 64Byte，例如： - stride为0时，跨步步长为0Byte。 - stride为1时，跨步步长为64Byte。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。
- 当前权重矩阵、输入矩阵和输出矩阵的行列要求为：

权重矩阵	输入矩阵	输出矩阵
MatmulK32 * MatmulN32	(1~128) * MatmulK32	(1~128) * MatmulN32

使用示例1

使用matmul_init、matmul_load_weight、matmul_compute、matmul_store、matmul_wait、matmul_wait_loading_weight、matmul_wait_loading_input共同完成100*32*32的矩阵乘运算。

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵，32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);
```

```
... // 此时可修改weight地址上数据

// 加载输入矩阵, 100*32个half型数据, 并开始矩阵乘运算
matmul_compute(handle, input, 100, MatmulK32);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址, 100*32个float型数据
matmul_store(handle, output, 100, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}
```

使用示例2

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同完成200*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵前半, 128*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 128, MatmulK32);

    // 输出计算结果写回到指定地址, 128*32个float型数据
    matmul_store(handle, output, 128, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);

    // 不使能更新权重矩阵, 即使用上次加载的权重矩阵
    matmul_set_updating_weight(handle, false);
}
```

```
// 加载输入矩阵后半, 72*32个half型数据, 并开始矩阵乘运算
matmul_compute(handle, input + 128 * MatmulK32, 72, MatmulK32);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址, 72*32个float型数据
matmul_store(handle, output + 128 * MatmulN32, 72, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}
```

使用示例3

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_flushing_output`共同完成100*64*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 不使能计算结果输出, 即暂存入累加缓冲区
    matmul_set_flushing_output(handle, false);

    // 加载权重矩阵上半, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 跨步加载输入矩阵左半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32, 1);

    // 使能计算结果输出, 下次计算结果累加后输出
    matmul_set_flushing_output(handle, true);

    // 加载权重矩阵下半, 32*32个half型数据
    matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 跨步加载输入矩阵右半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + MatmulK32, 100, MatmulK32, 1);
}
```

```
// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址, 100*32个float型数据
matmul_store(handle, output, 100, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}
```

7.8.3.2.4 matmul_store

```
void matmul_store(MatmulHandle &handle, void *output, unsigned int m, MatmulNSize n)
```

功能介绍

将矩阵乘运算的计算结果写回到SPM指定地址上。

参数解释

参数名称	说明
handle	MatmulHandle型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。
output	输出矩阵首地址，需位于SPM。
m	输出矩阵行数。有效值：1~128。
n	输出矩阵列数。枚举类型MatmulNSize，当前支持： MatmulN32：输出矩阵列数为32。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。
- 当前权重矩阵、输入矩阵和输出矩阵的行列要求为：

权重矩阵	输入矩阵	输出矩阵
MatmulK32 * MatmulN32	(1~128) * MatmulK32	(1~128) * MatmulN32

- 调用`matmul_store`时，如果上次`matmul_store`之后的所有`matmul_compute`调用都配置为不输出计算结果，则调用`matmul_store`输出计算结果时，会发生调用异常。您可以在创建`MatmulHandle`或调用`matmul_set_flushing_output`时配置输出计算结果使能`matmul_store`。

使用示例1

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`共同完成100*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵，32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵，100*32个half型数据，并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址，100*32个float型数据
    matmul_store(handle, output, 100, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}
```

使用示例2

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同

完成200*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵前半, 128*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 128, MatmulK32);

    // 输出计算结果写回到指定地址, 128*32个float型数据
    matmul_store(handle, output, 128, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);

    // 不使能更新权重矩阵, 即使用上次加载的权重矩阵
    matmul_set_updating_weight(handle, false);

    // 加载输入矩阵后半, 72*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + 128 * MatmulK32, 72, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 72*32个float型数据
    matmul_store(handle, output + 128 * MatmulN32, 72, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}
```

使用示例3

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_flushing_output`共同完成100*64*32的矩阵乘运算。

```

#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 不使能计算结果输出, 即暂存入累加缓冲区
    matmul_set_flushing_output(handle, false);

    // 加载权重矩阵上半, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 跨步加载输入矩阵左半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32, 1);

    // 使能计算结果输出, 下次计算结果累加后输出
    matmul_set_flushing_output(handle, true);

    // 加载权重矩阵下半, 32*32个half型数据
    matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 跨步加载输入矩阵右半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + MatmulK32, 100, MatmulK32, 1);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 100*32个float型数据
    matmul_store(handle, output, 100, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}

```

7.8.3.2.5 matmul_wait

```
void matmul_wait(MatmulHandle &handle)
```

功能介绍

等待矩阵乘写回操作完成。

参数解释

参数名称	说明
handle	MatmulHandle 型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。

返回值

无。

注意事项

- 需要手动包含 `sdaa_matmul.h` 头文件。
- 需要引入sdaa命名空间。

使用示例1

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`共同完成100*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵，32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵，100*32个half型数据，并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址，100*32个float型数据
    matmul_store(handle, output, 100, MatmulN32);

    // 等待写回完成
```

```
matmul_wait(handle);  
}
```

使用示例2

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同完成200*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵前半, 128*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 128, MatmulK32);

    // 输出计算结果写回到指定地址, 128*32个float型数据
    matmul_store(handle, output, 128, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);

    // 不使能更新权重矩阵, 即使用上次加载的权重矩阵
    matmul_set_updating_weight(handle, false);

    // 加载输入矩阵后半, 72*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + 128 * MatmulK32, 72, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 72*32个float型数据
    matmul_store(handle, output + 128 * MatmulN32, 72, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}
```

使用示例3

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_flushing_output`共同完成100*64*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 不使能计算结果输出, 即暂存入累加缓冲区
    matmul_set_flushing_output(handle, false);

    // 加载权重矩阵上半, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 跨步加载输入矩阵左半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32, 1);

    // 使能计算结果输出, 下次计算结果累加后输出
    matmul_set_flushing_output(handle, true);

    // 加载权重矩阵下半, 32*32个half型数据
    matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 跨步加载输入矩阵右半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + MatmulK32, 100, MatmulK32, 1);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 100*32个float型数据
    matmul_store(handle, output, 100, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}
```

7.8.3.3 配置型操作

7.8.3.3.1 MatmulHandle

```
MatmulHandle::MatmulHandle()
MatmulHandle::MatmulHandle(bool is_updating_weight, bool is_flushing_output, unsigned long weight_row_p
adding_mask = 0, MatmulOutputRowOffsetFlag output_row_offset_flag = MatmulEnableOutputRowOffset, unsign
ed int output_row_offset = 0)
```

功能介绍

配置矩阵乘的参数，从而使用不同的矩阵乘功能。创建 `MatmulHandle` 类型的变量时：

- 如果不提供参数，则使用其默认值。
- 如果提供参数，则使用参数指定的值配置矩阵乘。每个参数均设有对应的配置型函数接口。

参数解释

参数名称	说明
is_updating_weight	是否更新权重矩阵（默认更新），对应函数接口 <code>matmul_set_updating_weight</code> : • true：更新权重矩阵，切换权重矩阵加载缓冲区，使用新加载的权重矩阵进行下一次矩阵乘运算。 • false：不更新权重矩阵，不切换权重矩阵加载缓冲区，使用之前计算时使用的权重矩阵进行下一次矩阵乘运算。
is_flushing_output	是否在下次计算后输出计算结果（默认输出），对应函数接口 <code>matmul_set_flushing_output</code> : • true：下次矩阵乘输出计算结果。 • false：下次矩阵乘不输出计算结果。
weight_row_paddi ng_mask	加载权重矩阵时填充空行的掩码，默认值为0，对应函数接口 <code>matmul_set_weight_row_padding</code> 。
output_row_offset _flag	枚举型MatmulOutputRowOffsetFlag，设置计算结果行偏移标志位（默认为MatmulEnableOutputRowOffset），对应函数接口 <code>matmul_set_output_row_offset</code> : • MatmulEnableOutputRowOffset：使用指定的参数output_row_offset设置计算结果的行偏移。 • MatmulDisableOutputRowOffset：紧接着上一次计算结果之后的位置写入累加缓冲区。
output_row_offset	仅当output_row_offset_flag设置为MatmulEnableOutputRowOffset时有效，对应函数接口 <code>matmul_set_output_row_offset</code> 。有效值：0~127，对应累加缓冲区条目数，默认值为0。

注意事项

- 需要手动包含 `sdaa_matmul.h` 头文件。
- 需要引入`sdaa`命名空间。

使用示例1

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    MatmulHandle handle; // 创建MatmulHandle型变量, 使用默认参数

    ... // 执行非阻塞型矩阵乘操作
}
```

使用示例2

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    // 创建MatmulHandle类型变量, 不使能下一次计算结果输出, 不使能高精度计算结果输出
    MatmulHandle handle(false, false);

    ... // 执行非阻塞型矩阵乘操作
}
```

7.8.3.3.2 matmul_set_weight_row_padding

```
void matmul_set_weight_row_padding(MatmulHandle &handle, unsigned long weight_row_padding_mask)
```

功能介绍

通过掩码的方式，设置加载权重矩阵时，是否填充空行，默认不填充。

参数解释

参数名称	说明
handle	MatmulHandle 型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。

参数名称	说明
weight_row_padding_mask	加载权重矩阵时填充空行的掩码，默认值为0。掩码的每一个bit位对应权重矩阵的一行： - 为0时：不填充空行，加载权重矩阵对应的行。 - 为1时：填充空行，不加载权重矩阵的对应行。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。

使用示例

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void func()
{
    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;
    // 设置权重的mask，把权重矩阵的第1行屏蔽掉
    matmul_set_weight_row_padding(handle, 0x01);

    ... // 执行非阻塞型矩阵乘操作，加载权重矩阵时第1行填充0，权重矩阵从第2行开始写入
}
```

7.8.3.3.3 matmul_set_updating_weight

```
void matmul_set_updating_weight(MatmulHandle &handle, const bool &is_updating_weight)
```

功能介绍

更新权重矩阵或者使用前一次的权重矩阵进行下一次计算，默认更新权重矩阵。

参数解释

参数名称	说明
handle	MatmulHandle型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。

参数名称	说明
is_updating_weight	bool型，表示是否更新权重矩阵，默认为true： • true：更新权重矩阵。切换权重矩阵加载缓冲区，使用新加载的权重矩阵进行下一次矩阵乘运算。 • false：不更新权重矩阵。不切换权重矩阵加载缓冲区，使用前一次计算时使用的权重矩阵进行下一次矩阵乘运算。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。

使用示例

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void func()
{
    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    ... // 执行非阻塞型矩阵乘操作，使用新加载的权重矩阵

    matmul_set Updating_weight(handle, false);

    ... // 执行非阻塞型矩阵乘操作，使用前一次加载的权重矩阵
}
```

7.8.3.3.4 matmul_set_output_row_offset

```
void matmul_set_output_row_offset(MatmulHandle &handle, MatmulOutputRowOffsetFlag output_row_offset_flag, unsigned int output_row_offset)
```

功能介绍

设置将计算结果写入计算结果累加缓冲区的行偏移方式。

参数解释

参数名称	说明
handle	MatmulHandle 型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。
output_row_offset_flag	枚举型MatmulOutputRowOffsetFlag，设置计算结果写入累加缓冲区地址，默认为MatmulEnableOutputRowOffset： - MatmulEnableOutputRowOffset：使用指定的参数output_row_offset设置准备写入计算结果的累加缓冲区地址。 - MatmulDisableOutputRowOffset：紧接着上一次计算结果之后的位置写入累加缓冲区。
output_row_offset	仅当output_row_offset_flag设置为MatmulEnableOutputRowOffset时有效。有效值：0~127，对应累加缓冲区条目数，默认值为0。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。

使用示例

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void func()
{
    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;
    // 设置累加缓冲区偏移跳过第1行，从第2行开始写入并累加
    matmul_set_output_row_offset(handle, MatmulEnableOutputRowOffset, 1);

    ... // 执行非阻塞型矩阵乘操作，计算结果从累加缓冲区第2行开始写入
}
```

7.8.3.3.5 matmul_set_flushing_output

```
void matmul_set_flushing_output(MatmulHandle &handle, bool is_flushing_output)
```

功能介绍

设置在下次计算后，是否可以输出计算结果，默认可以输出。

参数解释

参数名称	说明
handle	MatmulHandle 型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。
is_flushing_output	bool型，表示下一次计算是否可以输出计算结果，默认可以输出： • true：下一次计算累加结束，使能计算结果可以输出到SPM，运算结果与累加缓冲区数据累加后切换累加缓冲区并设置为可以输出到SPM。 • false：下一次计算不用输出计算结果，不使能计算结果输出，运算结果与累加缓冲区数据累加并暂存在累加缓冲区。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。

使用示例

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void func()
{
    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    matmul_set_flushing_output(handle, false);

    ... // 执行非阻塞型矩阵乘操作，将计算结果暂存于累加缓冲区

    matmul_set_flushing_output(handle, true);

    ... // 执行非阻塞型矩阵乘操作，计算结果与上一次计算结果累加后输出
}
```

7.8.3.3.6 matmul_wait_loading_weight

```
void matmul_wait_loading_weight(MatmulHandle &handle)
```

功能介绍

等待权重矩阵加载完成。

参数解释

参数名称	说明
handle	MatmulHandle 型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。

返回值

无。

注意事项

- 需要手动包含 `sdaa_matmul.h` 头文件。
- 需要引入sdaa命名空间。

使用示例1

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`共同完成100*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵，32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵，100*32个half型数据，并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据
```

```
// 输出计算结果写回到指定地址, 100*32个float型数据
matmul_store(handle, output, 100, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}
```

使用示例2

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同完成200*32*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵前半, 128*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 128, MatmulK32);

    // 输出计算结果写回到指定地址, 128*32个float型数据
    matmul_store(handle, output, 128, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);

    // 不使能更新权重矩阵, 即使用上次加载的权重矩阵
    matmul_set_updating_weight(handle, false);

    // 加载输入矩阵后半, 72*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + 128 * MatmulK32, 72, MatmulK32);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 72*32个float型数据
```

```
    matmul_store(handle, output + 128 * MatmulN32, 72, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}
```

使用示例3

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同完成100*64*32的矩阵乘运算。

```
#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 不使能计算结果输出, 即暂存入累加缓冲区
    matmul_set_last_calculation(handle, false);

    // 加载权重矩阵上半, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 跨步加载输入矩阵左半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32, 1);

    // 使能计算结果输出, 下次计算结果累加后输出
    matmul_set_last_calculation(handle, true);

    // 加载权重矩阵下半, 32*32个half型数据
    matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 跨步加载输入矩阵右半, 100*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input + MatmulK32, 100, MatmulK32, 1);

    // 等待输入矩阵加载完成
    matmul_wait_loading_input(handle);

    ... // 此时可修改input地址上数据

    // 输出计算结果写回到指定地址, 100*32个float型数据
    matmul_store(handle, output, 100, MatmulN32);

    // 等待写回完成
}
```

```
matmul_wait(handle);
}
```

7.8.3.3.7 matmul_wait_loading_input

```
void matmul_wait_loading_input(MatmulHandle &handle)
```

功能介绍

等待输入矩阵加载完成。

参数解释

参数名称	说明
handle	MatmulHandle型变量。配置矩阵乘的参数，从而使用不同的矩阵乘功能。

返回值

无。

注意事项

- 需要手动包含 sdAA_matmul.h 头文件。
- 需要引入sdAA命名空间。

使用示例

使用matmul_init、matmul_load_weight、matmul_compute、matmul_store、matmul_wait、matmul_wait_loading_weight、matmul_wait_loading_input共同完成100*32*32的矩阵乘运算。

```
#include <sdAA_matmul.h>
using namespace sdAA;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵，32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);
}
```



```

// 等待权重矩阵加载完成
matmul_wait_loading_weight(handle);

... // 此时可修改weight地址上数据

// 加载输入矩阵, 100*32个half型数据, 并开始矩阵乘运算
matmul_compute(handle, input, 100, MatmulK32);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址, 100*32个float型数据
matmul_store(handle, output, 100, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}

```

使用示例2

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同完成200*32*32的矩阵乘运算。

```

#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量, 使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 加载权重矩阵, 32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据

    // 加载输入矩阵前半, 128*32个half型数据, 并开始矩阵乘运算
    matmul_compute(handle, input, 128, MatmulK32);

    // 输出计算结果写回到指定地址, 128*32个float型数据
    matmul_store(handle, output, 128, MatmulN32);

    // 等待写回完成
    matmul_wait(handle);
}

```

```

// 不使能更新权重矩阵，即使用上次加载的权重矩阵
matmul_set_updating_weight(handle, false);

// 加载输入矩阵后半，72*32个half型数据，并开始矩阵乘运算
matmul_compute(handle, input + 128 * MatmulK32, 72, MatmulK32);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址，72*32个float型数据
matmul_store(handle, output + 128 * MatmulN32, 72, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}

```

使用示例3

使用`matmul_init`、`matmul_load_weight`、`matmul_compute`、`matmul_store`、`matmul_wait`、`matmul_wait_loading_weight`、`matmul_wait_loading_input`、`matmul_set_updating_weight`共同完成100*64*32的矩阵乘运算。

```

#include <sdaa_matmul.h>
using namespace sdaa;
__global__ void func()
{
    ... // 准备矩阵数据

    // 创建MatmulHandle类型变量，使用默认参数
    MatmulHandle handle;

    // 初始化矩阵乘接口并配置数据类型
    matmul_init(handle, MatmulHalfToFloat);

    // 不使能计算结果输出，即暂存入累加缓冲区
    matmul_set_last_calculation(handle, false);

    // 加载权重矩阵上半，32*32个half型数据
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);

    // 跨步加载输入矩阵左半，100*32个half型数据，并开始矩阵乘运算
    matmul_compute(handle, input, 100, MatmulK32, 1);

    // 使能计算结果输出，下次计算结果累加后输出
    matmul_set_last_calculation(handle, true);

    // 加载权重矩阵下半，32*32个half型数据
    matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);

    // 等待权重矩阵加载完成
    matmul_wait_loading_weight(handle);

    ... // 此时可修改weight地址上数据
}

```

```
// 跨步加载输入矩阵右半, 100*32个half型数据, 并开始矩阵乘运算
matmul_compute(handle, input + MatmulK32, 100, MatmulK32, 1);

// 等待输入矩阵加载完成
matmul_wait_loading_input(handle);

... // 此时可修改input地址上数据

// 输出计算结果写回到指定地址, 100*32个float型数据
matmul_store(handle, output, 100, MatmulN32);

// 等待写回完成
matmul_wait(handle);
}
```

7.9 转置操作

7.9.1 概述

转置操作是指对矩阵的行和列进行互换, 即将原始矩阵的行变换为列, 或将原始矩阵的列变换为行。

转置操作分为阻塞型转置操作和非阻塞型转置操作:

- 阻塞型转置操作: 使用简单, 但性能不佳。
- 非阻塞型转置操作: 使用相对复杂, 但性能较好。

阻塞型转置操作

transpose: 进行矩阵的转置操作, 原始矩阵可以位于Global或SPM存储空间。不支持原位操作。

非阻塞型转置操作

- **TransposeHandle**: 用于控制不同的非阻塞型转置操作组。
- **transpose_async**: 非阻塞型转置操作接口。原始矩阵地址只能位于Global存储空间, 目的地址只能位于SPM存储空间。不支持原位操作。
- **transpose_wait**: 等待当前计算核心SPE上所有使用同一个 **TransposeHandle** 的非阻塞型转置操作完成。
- **check_transpose_async**: 验证 **transpose_async** 输入参数的合法性。

7.9.2 阻塞型转置

7.9.2.1 transpose

```
void transpose(void *dst, const void *src, unsigned int height, unsigned int width, size_t element_size)
```

功能介绍

完成同一SPA内的Global和Global存储空间之间、同一SPE内的SPM和SPM存储之间以及同一SPA内的Global存储和SPM存储之间的阻塞型矩阵转置操作。



图1 阻塞型矩阵转置

参数解释

参数名称	说明
dst	目的矩阵地址，可以为Global存储或SPM存储的地址。
src	源矩阵地址，可以为Global存储或SPM存储的地址。
height	源矩阵的行数。
width	源矩阵的列数。
element_size	矩阵的数据类型大小。目前仅支持数据类型大小为2B（short类型或者half类型）或者4B（int类型或者float类型）。

返回值

无。

注意事项

- 需要手动包含sdAA_transpose.h头文件
- 需要引入sdAA命名空间。

- 源矩阵地址和目的矩阵地址需要按数据类型进行对齐。例如：矩阵数据类型是int类型，则要求按4B对齐，矩阵数据类型是short类型，则要求按2B对齐。
- 转置接口不支持原位操作。
- 当前transpose接口会额外占用SPM堆空间：
 - 当数据类型大小为 2B 时，额外占用 2KB 的SPM堆内存空间。
 - 当数据类型大小为 4B 时，额外占用 1KB 的SPM堆内存空间。

使用示例

```
#include <sdaa_transpose.h>
using namespace sdaa;

#define SIZE 256      // 矩阵大小
#define HEIGHT 16     // 矩阵行数
#define WIDTH 16      // 矩阵列数

__device__ void func()
{
    // 仅使用0号SPE进行矩阵转置
    if (threadIdx != 0) {
        return;
    }

    int *src = (int *)malloc(SIZE * sizeof(int));
    int *dst = (int *)malloc(SIZE * sizeof(int));

    // 初始化源矩阵:
    // {0, 1, 2, ..., 15,
    // 16, 17, 18, ..., 31,
    // ...,
    // 240, 241, 242, ..., 255}
    for (int row = 0; row < HEIGHT; row++) {
        for (int col = 0; col < WIDTH; col++) {
            src[row * WIDTH + col] = row * WIDTH + col;
        }
    }

    // 执行转置操作
    transpose(dst, src, HEIGHT, WIDTH, sizeof(int));

    // 打印转置结果
    // {0, 16, 32, ..., 240,
    // 1, 17, 33, ..., 241,
    // ...,
    // 15, 31, 47, ..., 255}
    for (int row = 0; row < HEIGHT; row++) {
        for (int col = 0; col < WIDTH; col++) {
            printf("%d\t", dst[row * WIDTH + col]);
        }
        printf("\n");
    }
    printf("\n");
}
```

```
    free(dst);  
    free(src);  
}
```

7.9.3 非阻塞型转置

7.9.3.1 TransposeHandle及配置接口

7.9.3.1.1 概述

TransposeHandle是非阻塞型转置操作的句柄，以非阻塞型转置组的形式控制不同transpose_async操作。TransposeHandle由以下函数接口组成：

- **TransposeHandle**：控制不同transpose_async操作所属的非阻塞型转置组。

7.9.3.1.2 TransposeHandle

```
TransposeHandle::TransposeHandle()
```

功能介绍

用于控制不同transpose_async操作所属的非阻塞型转置组。同一转置组内的所有transpose_async操作可以只调用一次transpose_wait进行同步。

注意事项

- 需要手动包含sdaa_transpose.h头文件。
- 需要引入sdaa命名空间。

使用示例

使用无参构造函数创建 TransposeHandle 类型变量。

```
#include <sdaa_transpose.h>  
using namespace sdaa;  
  
__device__ void func()  
{  
    // 创建TransposeHandle类型变量  
    TransposeHandle handle;  
  
    // 执行非阻塞转置操作  
}
```

7.9.3.2 transpose_async

```
void transpose_async(void *dst, const void *src, unsigned int height, unsigned int width, size_t element_size, MemoryStride stride, TransposeDirect direct, TransposeHandle &handle)
```

功能介绍

完成同一SPA内Global存储到SPM存储的非阻塞型转置操作。



图1 非阻塞型转置

参数解释

参数名称	说明
dst	目的矩阵地址，只能为SPM存储空间。
src	源矩阵地址，只能为Global存储空间。
height	目的矩阵的行数。 - 当数据类型大小为2B时，需为32。 - 当数据类型大小为4B时，需为16。
width	目的矩阵的列数。 - 当数据类型大小为2B时，需为32。 - 当数据类型大小为4B时，需为16。
element_size	矩阵的数据类型大小。目前仅支持数据类型大小为2B（short类型或者half类型）或者4B（int类型或者float类型）。

参数名称	说明
stride	MemoryStride结构体类型变量，设置源和目标存储空间的跨步单元大小，单位：Byte。  说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运。
direct	转置方向，枚举类型TransposeDirect，目前支持： TransposeGlobalToSpm：源矩阵位于Global存储空间，目的矩阵位于SPM存储空间。
handle	TransposeHandle类型变量，用于控制不同的非阻塞型转置操作组。

返回值

无。

注意事项

- 需要手动包含sdAA_transpose.h头文件。
- 需要引入sdAA命名空间。
- Global和SPM的存储空间地址需要64B对齐。
- `transpose_async` 需要与**transpose_wait**配合使用。

使用示例1

对Global存储空间内的16×16的int类型的矩阵进行转置操作。

```
#include <sdAA_transpose.h>
using namespace sdAA;

#define SIZE 256
#define HEIGHT 16
#define WIDTH 16

// 声明地址64B对齐的Global存储空间
__device__ int src[SIZE] __attribute__((aligned(64)));
```



```
__device__ void func()
{
    // 仅使用0号SPE进行矩阵转置
    if (threadIdx != 0) {
        return;
    }

    // 在SPM存储空间申请用于存放目的矩阵的内存空间
    int *dst = (int *)malloc(SIZE * sizeof(int));

    // 初始化源矩阵:
    // {0, 1, 2, ..., 15,
    // 16, 17, 18, ..., 31,
    // ...,
    // 240, 241, 242, ..., 255}
    for (int row = 0; row < HEIGHT; row++) {
        for (int col = 0; col < WIDTH; col++) {
            src[row * WIDTH + col] = row * WIDTH + col;
        }
    }

    // 声明TransposeHandle类型变量
    TransposeHandle handle;

    // 执行转置操作
    transpose_async(dst, src, HEIGHT, WIDTH, sizeof(int), {0, 0}, TransposeGlobalToSpm, handle);

    // 其他代码逻辑

    // 等待完成非阻塞转置操作
    ...

    // 打印转置结果
    // {0, 16, 32, ..., 240,
    // 1, 17, 33, ..., 241,
    // ...,
    // 15, 31, 47, ..., 255}
    for (int row = 0; row < HEIGHT; row++) {
        for (int col = 0; col < WIDTH; col++) {
            printf("%d\t", dst[row * WIDTH + col]);
        }
        printf("\n");
    }
    printf("\n");

    // 释放申请的SPM存储空间
    free(dst);
}
```

使用示例2

对Global存储空间内的32×32的int类型的矩阵进行带跨步的转置操作。

```
#include <sdaa_transpose.h>
using namespace sdaa;
```

```

#define SIZE_SRC 1024
#define SIZE_DST 256
#define HEIGHT_SRC 32
#define WIDTH_SRC 32

#define HEIGHT_DST 16
#define WEIGHT_DST 16
#define GLBMEM_STRIDE 16

// 声明地址64B对齐的Global存储空间
__device__ int src[SIZE_SRC] __attribute__((aligned(64)));
__device__ void func()
{
    // 仅使用0号SPE进行矩阵转置
    if (threadIdx != 0) {
        return;
    }

    // 在SPM存储空间申请用于存放目的矩阵的内存空间
    int *dst = (int *)malloc(SIZE_DST * sizeof(int));

    // 初始化源矩阵:
    // {0, 1, 2, ..., 31,
    // 32, 33, 34, ..., 63,
    // ...,
    // 992, 993, 994, ..., 1023}
    for (int row = 0; row < HEIGHT_SRC; row++) {
        for (int col = 0; col < WIDTH_SRC; col++) {
            src[row * WIDTH_SRC + col] = row * WIDTH_SRC + col;
        }
    }

    // 声明转置参数
    size_t global_stride = GLBMEM_STRIDE * sizeof(int);
    // 声明TransposeHandle类型变量
    TransposeHandle handle;

    // 执行转置操作
    transpose_async(dst, src, HEIGHT_DST, WEIGHT_DST, sizeof(int), {0, global_stride}, TransposeGlobalToSpm, handle);

    // 其他代码逻辑

    // 等待完成非阻塞转置操作
    ...

    // 打印转置结果
    // {0, 32, 64, ..., 480,
    // 1, 33, 65, ..., 481,
    // ...,
    // 15, 47, 79, ..., 495}
    for (int row = 0; row < HEIGHT_DST; row++) {
        for (int col = 0; col < WEIGHT_DST; col++) {
            printf("%d\t", dst[row * WEIGHT_DST + col]);
        }
        printf("\n");
    }
    printf("\n");
}

```

```
// 释放申请的SPM存储空间
free(dst);
}
```

7.9.3.3 transpose_wait

```
void transpose_wait(TransposeHandle &handle)
```

功能介绍

等待当前计算核心SPE上所有使用同一个`TransposeHandle`的非阻塞型转置操作完成。

参数解释

参数名称	说明
handle	<code>TransposeHandle</code> 类型变量，表示当前转置操作组。

返回值

无。

注意事项

- 需要手动包含`sdaa_transpose.h`头文件。
- 需要引入`sdaa`命名空间。
- `transpose_wait` 需要与`transpose_async`配合使用。

使用示例1

对Global存储空间内的16×16的int类型的矩阵进行转置操作。

```
#include <sdaa_transpose.h>
using namespace sdaa;
#define SIZE 256
#define HEIGHT 16
#define WIDTH 16

// 声明地址64B对齐的Global存储空间
__device__ int src[SIZE] __attribute__((aligned(64)));
__device__ void func()
{
    // 仅使用0号SPE进行矩阵转置
    if (threadIdx != 0) {
        return;
    }
}
```

```

// 在SPM存储空间申请用于存放目的矩阵的内存空间
int *dst = (int *)malloc(SIZE * sizeof(int));

// 初始化源矩阵:
// {0, 1, 2, ..., 15,
// 16, 17, 18, ..., 31,
// ...,
// 240, 241, 242, ..., 255}
for (int row = 0; row < HEIGHT; row++) {
    for (int col = 0; col < WIDTH; col++) {
        src[row * WIDTH + col] = row * WIDTH + col;
    }
}

// 声明TransposeHandle类型变量
TransposeHandle handle;

// 执行转置操作
transpose_async(dst, src, HEIGHT, WIDTH, sizeof(int), {0, 0}, TransposeGlobalToSpm, handle);

// 其他代码逻辑

// 等待完成非阻塞转置操作
transpose_wait(handle);

// 打印转置结果
// {0, 16, 32, ..., 240,
// 1, 17, 33, ..., 241,
// ...,
// 15, 31, 47, ..., 255}
for (int row = 0; row < HEIGHT; row++) {
    for (int col = 0; col < WIDTH; col++) {
        printf("%d\t", dst[row * WIDTH + col]);
    }
    printf("\n");
}
printf("\n");

// 释放申请的SPM存储空间
free(dst);
}

```

使用示例2

对Global存储空间内的32×32的int类型的矩阵进行带跨步的转置操作。

```

#include <sdaa_transpose.h>
using namespace sdaa;

#define SIZE_SRC 1024
#define SIZE_DST 256
#define HEIGHT_SRC 32
#define WIDTH_SRC 32

#define HEIGHT_DST 16
#define WEIGHT_DST 16

```

```

#define GLBMEM_STRIDE 16

// 声明地址64B对齐的Global存储空间
__device__ int src[SIZE_SRC] __attribute__((aligned(64)));
__device__ void func()
{
    // 仅使用0号SPE进行矩阵转置
    if (threadIdx != 0) {
        return;
    }

    // 在SPM存储空间申请用于存放目的矩阵的内存空间
    int *dst = (int *)malloc(SIZE_DST * sizeof(int));

    // 初始化源矩阵:
    // {0, 1, 2, ..., 31,
    // 32, 33, 34, ..., 63,
    // ...,
    // 992, 993, 994, ..., 1023}
    for (int row = 0; row < HEIGHT_SRC; row++) {
        for (int col = 0; col < WIDTH_SRC; col++) {
            src[row * WIDTH_SRC + col] = row * WIDTH_SRC + col;
        }
    }

    // 声明转置参数
    size_t global_stride = GLBMEM_STRIDE * sizeof(int);
    // 声明TransposeHandle类型变量
    TransposeHandle handle;

    // 执行转置操作
    transpose_async(dst, src, HEIGHT_DST, WEIGHT_DST, sizeof(int), {0, global_stride}, TransposeGlobalToSpm, handle);

    // 其他代码逻辑

    // 等待完成非阻塞转置操作
    ...

    // 打印转置结果
    // {0, 32, 64, ..., 480,
    // 1, 33, 65, ..., 481,
    // ...,
    // 15, 47, 79, ..., 495}
    for (int row = 0; row < HEIGHT_DST; row++) {
        for (int col = 0; col < WEIGHT_DST; col++) {
            printf("%d\t", dst[row * WEIGHT_DST + col]);
        }
        printf("\n");
    }
    printf("\n");

    // 释放申请的SPM存储空间
    free(dst);
}

```

7.9.3.4 check_transpose_async

```
unsigned int check_transpose_async(void *dst, const void *src, unsigned int height, unsigned int width,
size_t element_size, MemoryStride stride, TransposeDirect direct, TransposeHandle &handle)
```

功能介绍

验证非阻塞`transpose_async`输入参数的合法性，通过返回值判断输入参数是否满足特定的输入要求，如：源地址与目的地址是否对齐、转置操作的数据量是否符合要求等。

参数解释

参数名称	说明
dst	转置操作的 <code>目的矩阵地址</code> 。
src	转置操作的 <code>源矩阵地址</code> 。
height	目的矩阵的<code>行数</code>。 - 当数据类型大小为2B时，需为32。 - 当数据类型大小为4B时，需为16。
width	目的矩阵的<code>列数</code>。 - 当数据类型大小为2B时，需为32。 - 当数据类型大小为4B时，需为16。
element_size	矩阵的 <code>数据类型大小</code> 。
stride	<code>MemoryStride</code>结构体类型变量，设置源和目标存储空间的跨步单元大小，单位：Byte。  说明： - MemoryStride的设置受硬件限制： - 目前仅支持在Global存储空间，设置跨步单元的大小。 - 跨步单元的大小，需要小于Global存储空间的大小，可参考存储空间可用大小。 - 如果MemoryStride中的 <code>dst_stride</code> 和 <code>src_stride</code> 都设置为0，则为连续数据搬运。

参数名称	说明
direct	转置方向，枚举类型TransposeDirect，目前支持： TransposeGlobalToSpm：源矩阵位于Global存储空间，目的矩阵位于SPM存储空间。
handle	TransposeHandle类型变量，用于控制不同的非阻塞型转置操作组。

返回值

返回值类型	说明
unsigned int	参数合法性验证结果，每个Bit代表不同的状态信息。对应枚举型MemcpyAsyncStatus，当前可表示的状态有： - TRANPOSE_ASYNC_SUCC：参数无误。 - TRANPOSE_ASYNC_ERR_ADDR_UNALIGNED：源地址或目的地址不满足对齐要求。 - TRANPOSE_ASYNC_ERR_SIZE_ILLEGAL：源矩阵的行数或者列数不满足要求。 - TRANPOSE_ASYNC_ERR_DIRECT_ILLEGAL：转置方向不满足要求。 - TRANPOSE_ASYNC_ERR_STRIDE_PARAM：跨步参数错误。

返回的具体状态信息，可参照如下表格：

第0位	第1位	第2位	第3位	第4位
0：参数 无误	第0位是0时，此位保留	第0位是0时，此位保留	第0位是0时，此位保留	第0位是0时，此位保留
1：参数 有误	第0位是1时，此位： 0：正常。 1：错误。源地址或目的地址不满足对齐要求。	第0位是1时，此位： 0：正常。 1：错误。源矩阵的行数或者列数不满足要求。	第0位是1时，此位： 0：正常。 1：错误。转置方向不满足要求。	第0位是1时，此位： 0：正常。 1：错误。跨步参数错误。

注意事项

- 需要手动包含sdaa_transpose.h头文件。
- 需要引入sdaa命名空间。

使用示例

```

#include <sdaa_transpose.h>
using namespace sdaa;

#define SIZE 256
#define HEIGHT 16
#define WIDTH 16

// 声明地址64B对齐的Global存储空间
__device__ int src[SIZE] __attribute__((aligned(64)));
__device__ void func()
{
    // 仅使用0号SPE进行矩阵转置
    if (threadIdx != 0) {
        return;
    }

    // 在SPM存储空间申请用于存放目的矩阵的内存空间
    int *dst = (int *)malloc(SIZE * sizeof(int));

    // 初始化源矩阵:
    // {0, 1, 2, ..., 15,
    // 16, 17, 18, ..., 31,
    // ...,
    // 240, 241, 242, ..., 255}
    for (int row = 0; row < HEIGHT; row++) {
        for (int col = 0; col < WIDTH; col++) {
            src[row * WIDTH + col] = row * WIDTH + col;
        }
    }

    // 声明TransposeHandle类型变量
    TransposeHandle handle;

    // 如果transpose_async接口参数合法, check_transpose_async返回TRANPOSE_ASYNC_SUCC
    unsigned int check_val = check_transpose_async(dst, src, HEIGHT, WIDTH, sizeof(int), {0, 0}, TransposeGlobalToSpm, handle);
    if (check_val == TRANPOSE_ASYNC_SUCC) {
        // 执行转置操作
        transpose_async(dst, src, HEIGHT, WIDTH, sizeof(int), {0, 0}, TransposeGlobalToSpm, handle);

        // 其他代码逻辑

        // 等待完成非阻塞转置操作
        transpose_wait(handle);
    }

    // 打印转置结果
    // {0, 16, 32, ..., 240,
    // 1, 17, 33, ..., 241,
    // ...,
    // 15, 31, 47, ..., 255}
    for (int row = 0; row < HEIGHT; row++) {
        for (int col = 0; col < WIDTH; col++) {
            printf("%d\t", dst[row * WIDTH + col]);
        }
        printf("\n");
    }
    printf("\n");
}

```



```
// 释放申请的SPM存储空间
free(dst);
}
```

7.10 向量操作

7.10.1 概述

SDAA C提供的向量操作类函数接口通过封装SIMD（Single Instruction Multiple Data）指令一次性处理多个数据，极大地提高了数据处理的并行性。用于向量操作的函数接口可以分为以下几类：

- 装入/存储类：描述标量和向量间的数据搬运接口。
- 运算类：描述整数、浮点向量类型的算术运算、比较和选择等运算接口。具体可细分为：
 - 标量扩展类
 - 比较选择类
 - 元素整理类
 - 符号拷贝类
 - 类型转换类
 - 算术运算类
 - 比较选择类
 - 融合乘加类
- 对于混合类型的计算，目前不支持使用操作符替代函数接口。例如，不支持以下操作：

```
floatv16 leftVal;
float16v16 rightVal;
floatv16 res = leftVal + rightVal;
```

- 对于非混合类型（同一类型）的计算，算术运算类和比较类接口可用操作符进行替代。目前支持的操作符详见下表。

? 说明：

对于比较类函数接口，当使用操作符进行替代时，需要根据目标类型增加强制类型转换。例如：

```
floatv16 leftVal;
floatv16 rightVal;
floatv16 res = floatv16(leftVal <= rightVal);
```

类别	操作符
下标运算符	[]
单目运算符	+, -, *, &, !, ~, sizeof
算术运算符	+, -, *, /, %
逻辑运算符	<, <=, >, >=, ==, !=, &, ^, , &&,
赋值运算符	=, +=, -=, *=, /=, %=, &=, ^=, =

7.10.2 装入/存储类操作

7.10.2.1 simd_load

```
void simd_load(intv16 &vector, const int *src)
void simd_load(uintv16 &vector, const unsigned int *src)
void simd_load(shortv32 &vector, const short *src)
void simd_load(ushortv32 &vector, const unsigned short *src)
void simd_load(halfv16 &vector, const half *src)
void simd_load(floatv16 &vector, const float *src)
```

功能介绍

对齐向量装入，将64字节（halfv16类型的装入是32字节）长度的数据从连续内存区域中装入到一个向量变量中。

参数解释

参数名称	说明
vector	intv16、uintv16、shortv32、ushortv32、halfv16、floatv16型向量。
src	64B对齐（halfv16要求32B对齐）的标量地址（需要是连续内存区域）。

返回值

无。

使用示例

```
// addr_0 64B 对齐, addr_1 32B 对齐
__global__ void func(int *addr_0, half *addr_1)
{
    // 64Byte整型数据装入 intv16
```

```
intv16 va_0;
simd_load(va_0, addr_0);

// 32Byte 16位浮点型数据装入 halfv16
halfv16 va_1;
simd_load(va_1, addr_1);
}
```

7.10.2.2 simd_store

```
void simd_store(intv16 vector, int *dst)
void simd_store(uintv16 vector, unsigned int *dst)
void simd_store(shortv32 vector, short *dst)
void simd_store(ushortv32 vector, unsigned short *dst)
void simd_store(halfv16 vector, half *dst)
void simd_store(floatv16 vector, float *dst)
```

功能介绍

对齐向量存储，将一个向量变量中的数据存储到64字节（halfv16类型的存储是32字节）连续内存区域中。

参数解释

参数名称	说明
vector	intv16、uintv16、shortv32、ushortv32、halfv16、floatv16型向量。
dst	64B对齐（halfv16要求32B对齐）的标量地址（需要是连续内存区域）。

返回值

无。

使用示例

```
// addr_0 64B 对齐, addr_1 32B 对齐
__global__ void func(int *addr_0, half *addr_1)
{
    // 将 intv16 的数据写入到地址addr_0
    intv16 va_0 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    simd_store(va_0, addr_0);

    // 将 halfv16 的数据写入到地址addr_1
    halfv16 va_1 = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6};
    simd_store(va_1, addr_1);
}
```

7.10.2.3 simd_loadu

```
void simd_loadu(intv16 &vector, const int *src)
void simd_loadu(uintv16 &vector, const unsigned int *src)
void simd_loadu(shortv32 &vector, const short *src)
void simd_loadu(ushortv32 &vector, const unsigned short *src)
void simd_loadu(halfv16 &vector, const half *src)
void simd_loadu(floatv16 &vector, const float *src)
```

功能介绍

不对齐向量装入，将64字节（halfv16类型的装入是32字节）长度的数据从连续内存区域中装入到一个向量变量中。

参数解释

参数名称	说明
vector	intv16、uintv16、shortv32、ushortv32、halfv16、floatv16型向量。
src	不对齐的标量地址（需要是连续内存区域）。 说明： 标量地址不要求64B（halfv16不要求32B）对齐，但是需要按标量数据类型对齐。例如标量是int类型数组则要求4B对齐，标量是short数据类型数组则要求2B对齐。

返回值

无。

使用示例

```
__global__ void func(int *addr_0, half *addr_1)
{
    // 64Byte整型数据装入 intv16
    intv16 va_0;
    simd_loadu(va_0, addr_0);

    // 32Byte 16位浮点型数据装入 halfv16
    halfv16 va_1;
    simd_loadu(va_1, addr_1);
}
```

7.10.2.4 simd_storeu

```
void simd_storeu(intv16 vector, int *dst)
void simd_storeu(uintv16 vector, unsigned int *dst)
```

```
void simd_storeu(shortv32 vector, short *dst)
void simd_storeu(ushortv32 vector, unsigned short *dst)
void simd_storeu(halfv16 vector, half *dst)
void simd_storeu(floatv16 vector, float *dst)
```

功能介绍

不对齐向量存储，将一个向量变量中的数据存储到64字节（halfv16类型的存储是32字节）连续内存区域中。

参数解释

参数名称	说明
vector	intv16、uintv16、shortv32、ushortv32、halfv16、floatv16型向量。
dst	不对齐的标量地址（需要是连续内存区域）。  说明： 标量地址不要求64B（halfv16不要求32B）对齐，但是需要按标量数据类型对齐。例如标量是int类型数组则要求4B对齐，标量是short数据类型数组则要求2B对齐。

返回值

无。

使用示例

```
__global__ void func(int *addr_0, half *addr_1)
{
    // 将 intv16 的数据写入到地址addr_0
    intv16 va_0 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    simd_storeu(va_0, addr_0);

    // 将 halfv16 的数据写入到地址addr_1
    halfv16 va_1 = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6};
    simd_storeu(va_1, addr_1);
}
```

7.10.2.5 simd_load_widen

```
void simd_load_widen(intv16 &vector, const short *src)
void simd_load_widen(shortv32 &vector, const char *src)
void simd_load_widen(floatv16 &vector, const bfloat16 *src)
```

功能介绍

向量扩展装入，将数据从连续内存区域扩展装入到一个向量变量中。

 提示：

根据所使用的数据类型，装入数据的个数可能会有所不同，如：

- `void simd_load_widen(intv16 &vector, const short *src)` 是将16个short类型数据扩展为int类型数据，并装入到intv16向量类型中。
- `void simd_load_widen(shortv32 vector, const char *src)` 是将32个char类型数据扩展为short类型数据，并装入到shortv32向量类型中。
- `void simd_load_widen(floatv16 vector, const bfloat16 *src)` 是将16个bfloat16类型数据扩展为float类型数据，并装入到floatv16向量类型中。

参数解释

参数名称	说明
vector	intv16、shortv32、floatv16型向量。
src	连续内存区域的标量地址。  说明： short*、char*、bfloat16*型的标量地址，需要32B对齐。

返回值

无。

使用示例

```
__global__ void func(short *addr_0, char *addr_1, bfloat16 *addr_2)
{
    // 将16个short类型数据扩展为int类型后装入 intv16
    intv16 va_0;
    simd_load_widen(va_0, addr_0);

    // 将32个char类型数据扩展为short类型后装入 shortv32
    shortv32 va_1;
    simd_load_widen(va_1, addr_1);

    // 将16个bfloat类型数据扩展为float类型后装入 floatv16
}
```

```
floatv16 va_2;
simd_load_widen(va_2, addr_2);}
```

7.10.2.6 simd_store_narrow

```
void simd_store_narrow(intv16 vector, short *dst)
void simd_store_narrow(shortv32 vector, char *dst)
void simd_store_narrow(floatv16 vector, bfloat16 *dst)
```

功能介绍

向量收缩存储，将一个向量变量中的数据收缩存储至连续内存区域中。

提示：

根据所使用的数据类型，存储数据的个数可能会有所不同，如：

- `void simd_store_narrow(intv16 vector, short *dst)` 是将16个int类型数据收缩为short类型数据，并存储到short类型数组中。
- `void simd_store_narrow(shortv32 vector, char *src)` 是将32个short类型数据收缩为char类型数据，并存储到char类型数组中。
- `void simd_store_narrow(floatv16 vector, bfloat16 *src)` 是将16个float类型数据收缩为bfloat类型数据，并存储到bfloat16类型数组中。

参数解释

参数名称	说明
vector	intv16、shortv32、floatv16型向量。
dst	连续内存区域的标量地址。  说明： short*、char*、bfloat16*型的标量地址，需要32B对齐。

返回值

无。

使用示例

```
__global__ void func(short* addr_0, char* addr_1, bfloat16* addr_2)
{
    intv16 va_0 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    // 将 intv16 的数据收缩为16个short类型写入到 addr_0
    simd_store_narrow(va_0, addr_0);

    shortv32 va_1 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23,
24, 25, 26, 27, 28, 29, 30, 31};
    // 将 shortv32 的数据收缩为32个char类型写入到 addr_1
    simd_store_narrow(va_1, addr_1);

    floatv16 va_2 = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6};
    // 将 floatv16 的数据收缩为16个bfloat类型写入到 addr_2
    simd_store_narrow(va_2, addr_2);
}
```

7.10.2.7 simd_loadu_widen

```
void simd_loadu_widen(intv16 &vector, const short *src)
void simd_loadu_widen(shortv32 &vector, const char *src)
void simd_loadu_widen(floatv16 &vector, const bfloat16 *src)
```

功能介绍

不对齐向量扩展装入，将数据从连续内存区域扩展装入到一个向量变量中。

提示：

根据所使用的数据类型，装入数据的个数可能会有所不同，如：

- `void simd_loadu_widen(intv16 &vector, const short *src)` 是将16个short类型数据扩展为int类型数据，并装入到intv16向量类型中。
- `void simd_loadu_widen(shortv32 vector, const char *src)` 是将32个char类型数据扩展为short类型数据，并装入到shortv32向量类型中。
- `void simd_loadu_widen(floatv16 vector, const bfloat16 *src)` 是将16个bfloat16类型数据扩展为float类型数据，并装入到floatv16向量类型中。

参数解释

参数名称	说明
vector	intv16、shortv32、floatv16型向量。

参数名称	说明
src	连续内存区域的标量地址。 ? 说明： short*、bfloat16*型的标量地址，需要2B对齐。

返回值

无。

使用示例

```
__global__ void func(short *addr_0, char *addr_1, bfloat16 *addr_2)
{
    // 将16个short类型数据扩展为int类型后装入 intv16
    intv16 va_0;
    simd_loadu_widen(va_0, addr_0);

    // 将32个char类型数据扩展为short类型后装入 shortv32
    shortv32 va_1;
    simd_loadu_widen(va_1, addr_1);

    // 将16个bfloat类型数据扩展为float类型后装入 floatv16
    floatv16 va_2;
    simd_loadu_widen(va_2, addr_2);
}
```

7.10.2.8 simd_storeu_narrow

```
void simd_storeu_narrow(intv16 vector, short *dst)
void simd_storeu_narrow(shortv32 vector, char *dst)
void simd_storeu_narrow(floatv16 vector, bfloat16 *dst)
```

功能介绍

不对齐向量收缩存储，将一个向量变量中的数据收缩存储至连续内存区域中。

i

提示：

根据所使用的数据类型，存储数据的个数可能会有所不同，如：

- `void simd_storeu_narrow(intv16 vector, short *dst)` 是将16个int类型数据收缩为short类型数据，并存储到short类型数组中。
- `void simd_storeu_narrow(shortv32 vector, char *src)` 是将32个short类型数据收缩为char类型数据，并存储到char类型数组中。
- `void simd_storeu_narrow(floatv16 vector, bfloat16 *src)` 是将16个float类型数据收缩为bfloat类型数据，并存储到bfloat16类型数组中。

参数解释

参数名称	说明
vector	intv16、shortv32、floatv16型向量。
dst	连续内存区域的标量地址。 说明： short*、bfloat16*型的标量地址，需要2B对齐。

返回值

无。

使用示例

```
__global__ void func(short* addr_0, char* addr_1, bfloat16* addr_2)
{
    intv16 va_0 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
    // 将 intv16 的数据收缩为16个short类型写入到 addr_0
    simd_storeu_narrow(va_0, addr_0);

    shortv32 va_1 = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23,
24, 25, 26, 27, 28, 29, 30, 31};
    // 将 shortv32 的数据收缩为32个char类型写入到 addr_1
    simd_storeu_narrow(va_1, addr_1);

    floatv16 va_2 = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6};
    // 将 floatv16 的数据收缩为16个bfloat类型写入到 addr_2
    simd_storeu_narrow(va_2, addr_2);
}
```

7.10.2.9 simd_set

```
intv16 simd_set(int a0, int a1, int a2, int a3, int a4, int a5, int a6, int a7, int a8, int a9,
               int a10, int a11, int a12, int a13, int a14, int a15)
shortv32 simd_set(short a0, short a1, short a2, short a3, short a4, short a5, short a6, short a7,
                  short a8, short a9, short a10, short a11, short a12, short a13, short a14,
                  short a15, short a16, short a17, short a18, short a19, short a20, short a21,
                  short a22, short a23, short a24, short a25, short a26, short a27, short a28,
                  short a29, short a30, short a31)
ushortv32 simd_set(unsigned short a0, unsigned short a1, unsigned short a2, unsigned short a3,
                  unsigned short a4, unsigned short a5, unsigned short a6, unsigned short a7,
                  unsigned short a8, unsigned short a9, unsigned short a10, unsigned short a11,
                  unsigned short a12, unsigned short a13, unsigned short a14, unsigned short a15,
                  unsigned short a16, unsigned short a17, unsigned short a18, unsigned short a19,
                  unsigned short a20, unsigned short a21, unsigned short a22, unsigned short a23,
                  unsigned short a24, unsigned short a25, unsigned short a26, unsigned short a27,
                  unsigned short a28, unsigned short a29, unsigned short a30, unsigned short a31)
floatv16 simd_set(float a0, float a1, float a2, float a3, float a4, float a5, float a6, float a7,
                  float a8, float a9, float a10, float a11, float a12, float a13, float a14, float a15)
halfv16 simd_set(half a0, half a1, half a2, half a3, half a4, half a5, half a6, half a7,
                 half a8, half a9, half a10, half a11, half a12, half a13, half a14, half a15)
```

功能介绍

对向量数据进行赋值操作。

参数解释

参数名称	说明
a0-a15/a31	int、short、unsigned short、half、float型标量。

返回值

返回值类型	说明
intv16、shortv32、ushortv32、floatv16、halfv16	参数对应的向量类型。

使用示例

```
int a0 = 1, a1 = 1, a2 = 1, a3 = 1, a4 = 1, a5 = 1, a6 = 1, a7 = 1, a8 = 1, a9 = 1,
    a10 = 1, a11 = 1, a12 = 1, a13 = 1, a14 = 1, a15 = 1;
intv16 va = simd_set(a0, a1, a2, a3, a4, a5, a6, a7, a8, a9, a10, a11, a12, a13, a14, a15);
// va 是 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

7.10.3 标量扩展类操作

7.10.3.1 simd_stretch

```
intv16 simd_stretch(int ra)
shortv32 simd_stretch(short ra)
floatv16 simd_stretch(float ra)
halfv16 simd_stretch(half ra)
```

功能介绍

将标量ra复制成16个（short 类型为32个）相同元素并组成新的向量类型并返回。

参数解释

参数名称	说明
ra	int、short、float、half类型标量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。 - 输入标量ra等于NaN时，返回元素全为NaN的向量。 - 输入标量ra等于 $\pm\infty$ 时，返回元素全为 $\pm\infty$ 的向量。

使用示例1

```
int ra = 1;
intv16 va = simd_stretch(ra);
// va 是 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例2

```
short ra = 1;
shortv32 va = simd_stretch(ra);
// va 是 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例3

```
float ra = 0.1;
floatv16 va = simd_stretch(ra);
// va 是 [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]
```

使用示例4

```
half ra = 0.1;
halfv16 va = simd_stretch(ra);
// va 是 [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]
```

7.10.4 比较选择类操作

7.10.4.1 simd_seleq

```
intv16 simd_seleq(intv16 va, intv16 vb, intv16 vc)
shortv32 simd_seleq(shortv32 va, shortv32 vb, shortv32 vc)
floatv16 simd_seleq(floatv16 va, floatv16 vb, floatv16 vc)
halfv16 simd_seleq(halfv16 va, halfv16 vb, halfv16 vc)
```

功能介绍

逐元素判断向量类型参数va，如果某元素等于0，则返回vb对应元素的值；否则返回vc对应元素的值。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。
vc	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。

注意事项

该接口支持原位操作。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
```

```
intv16 vd = simd_seleq(va, vb, vc);
// vd 是 [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
va = simd_seleq(va, vb, vc);    // 原位操作
// va is [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
shortv32 vd = simd_seleq(va, vb, vc);
// vd 是 [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
```

使用示例4

```
floatv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = {-0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1};
floatv16 vd = simd_seleq(va, vb, vc);
// vd 是 [-0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1]
```

使用示例5

```
halfv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = {-0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1};
halfv16 vd = simd_seleq(va, vb, vc);
// vd 是 [-0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1]
```

7.10.4.2 simd_selle

```
intv16 simd_selle(intv16 va, intv16 vb, intv16 vc)
shortv32 simd_selle(shortv32 va, shortv32 vb, shortv32 vc)
floatv16 simd_selle(floatv16 va, floatv16 vb, floatv16 vc)
halfv16 simd_selle(halfv16 va, halfv16 vb, halfv16 vc)
```

功能介绍

逐元素判断向量类型的参数va，如果某元素小于等于0，则返回vb对应元素的值；否则返回vc对应元素的值。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。
vc	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。

注意事项

该接口支持原位操作。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
intv16 vd = simd_selle(va, vb, vc);
// vd 是 [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
va = simd_selle(va, vb, vc); // 原位操作
// va 是 [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
shortv32 vd = simd_selle(va, vb, vc);
// vd 是 [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
```

使用示例4

```
floatv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = {-0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1};
floatv16 vd = simd_selle(va, vb, vc);
// vd 是 [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]
```

使用示例5

```
halfv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = {-0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1};
halfv16 vd = simd_selle(va, vb, vc);
// vd 是 [0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1]
```

7.10.4.3 simd_sellt

```
intv16 simd_sellt(intv16 va, intv16 vb, intv16 vc)
shortv32 simd_sellt(shortv32 va, shortv32 vb, shortv32 vc)
floatv16 simd_sellt(floatv16 va, floatv16 vb, floatv16 vc)
halfv16 simd_sellt(halfv16 va, halfv16 vb, halfv16 vc)
```

功能介绍

逐元素判断向量类型的参数va，如果某元素小于0，则返回vb对应元素的值；否则返回vc对应元素的值。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。

参数名称	说明
vb	intv16、shortv32、floatv16、halfv16型向量。
vc	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。

■ 注意事项

该接口支持原位操作。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
intv16 vd = simd_sellt(va, vb, vc);
// vd 是 [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = {-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1};
va = simd_sellt(va, vb, vc);    // 原位操作
// va 是 [-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]
```

使用示例3

[illegible]

使用示例4

```
floatv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = {-0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1};
floatv16 vd = simd_sellt(va, vb, vc);
// vd 是 [0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1]
```

使用示例5

```
halfv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = {-0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1, -0.1};
halfv16 vd = simd_sellt(va, vb, vc);
// vd 是 [0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1]
```

7.10.5 元素整理类操作

7.10.5.1 simd_concat

```
intv16  simd_concat(intv16 va, intv16 vb, const void *n)
shortv32 simd_concat(shortv32 va, shortv32 vb, const void *n)
floatv16 simd_concat(floatv16 va, floatv16 vb, const void *n)
halfv16 simd_concat(halfv16 va, halfv16 vb, const void *n)
```

功能介绍

向量按元素拼接。根据n的低6位（忽略其低1位）确定拼接字节位置，将va的高地址数据和vb的低地址数据拼接后形成新的向量。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。
n	无类型指针，其值用于控制拼接位置。 说明： 1. 对于intv16、floatv16、halfv16型向量：4字节对应一个元素。 2. 对于shortv32型向量：2字节对应一个元素。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。 - 输入向量va或vb的元素值不存在NaN，且输入向量va或vb的某个元素值是 $\pm\infty$ 时，拼接后对应位置返回 $\pm\infty$ 。 - 输入向量va或vb的某个元素值是NaN时，拼接后对应位置返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
intv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
intv16 vb = {-1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15, -16};
unsigned long n = 0x04; // 拼接位置为4B即一个int数
intv16 vc = simd_concat(va, vb, (void *)n);
// vc 是 [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, -1]
```

使用示例2

```
intv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
intv16 vb = {-1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15, -16};
unsigned long n = 0x04; // 拼接位置为4B即一个int数
va = simd_concat(va, vb, (void *)n); // 原位操作
// va 是 [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, -1]
```

使用示例3

```
shortv32 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};
shortv32 vb = {-1, -2, -3, -4, -5, -6, -7, -8, -9, -10, -11, -12, -13, -14, -15, -16, -17, -18, -19, -20, -21, -22, -23, -24, -25, -26, -27, -28, -29, -30, -31, -32};
unsigned long n = 0x04; // 拼接位置为4B即两个short数
shortv32 vc = simd_concat(va, vb, (void *)n);
// vc 是 [3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, -1, -2]
```

使用示例4

```
floatv16 va = {0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16};
```

```
floatv16 vb = {-0.01, -0.02, -0.03, -0.04, -0.05, -0.06, -0.07, -0.08, -0.09, -0.1, -0.11, -0.12, -0.13, -0.14, -0.15, -0.16};
unsigned long n = 0x04; // 拼接位置为4B即一个float数
floatv16 vc = simd_concat(va, vb, (void *)n);
// vc 是 [0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16, -0.01]
```

使用示例5

```
halfv16 va = {0.01, 0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16};
halfv16 vb = {-0.01, -0.02, -0.03, -0.04, -0.05, -0.06, -0.07, -0.08, -0.09, -0.1, -0.11, -0.12, -0.13, -0.14, -0.15, -0.16};
unsigned long n = 0x04; // 拼接位置为4B但halfv16视作一个half数
halfv16 vc = simd_concat(va, vb, (void *)n);
// vc 是 [0.02, 0.03, 0.04, 0.05, 0.06, 0.07, 0.08, 0.09, 0.1, 0.11, 0.12, 0.13, 0.14, 0.15, 0.16, -0.01]
```

7.10.5.2 simd_ins

```
intv16 simd_ins(int ra, intv16 va, int n)
shortv32 simd_ins(short ra, shortv32 va, int n)
floatv16 simd_ins(float ra, floatv16 va, int n)
halfv16 simd_ins(half ra, halfv16 va, int n)
```

功能介绍

向量按元素插入。用标量ra替代向量va中由n指定的元素，组成并返回新的向量。

参数解释

参数名称	说明
ra	int、short、float、half型标量。
va	intv16、shortv32、floatv16、halfv16型向量。
n	int型常量整数。 ? 说明： 1. 对于intv16、floatv16、halfv16型向量：n的低4位有效即0~15。 2. 对于shortv32型向量：n的低5位有效即0~31。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。 - 输入标量ra等于NaN时，插入后向量对应位置返回NaN。 - 输入标量ra等于 $\pm\infty$ 时，插入后向量对应位置返回 $\pm\infty$ 。 - 输入向量va不存在NaN，且输入向量va的某个元素值是 $\pm\infty$ 时，插入后向量对应位置返回 $\pm\infty$ 。 - 输入向量va的某个元素值是NaN时，插入后向量对应位置返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
int ra = -1;
intv16 va = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_ins(ra, va, 4);
// vc 是 [1, 1, 1, 1, -1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例2

```
int ra = -1;
intv16 va = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_ins(ra, va, 4);    // 原位操作
// va 是 [1, 1, 1, 1, -1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例3

```
short ra = -1;
shortv32 va = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_ins(ra, va, 4);
// vc 是 [1, 1, 1, 1, -1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例4

```
float ra = -1;
floatv16 va = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
floatv16 vc = simd_ins(ra, va, 4);
// vc 是 [1, 1, 1, 1, -1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例5

```
half ra = -1;
halfv16 va = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
halfv16 vc = simd_ins(ra, va, 4);
// vc 是 [1, 1, 1, 1, -1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

7.10.6 符号拷贝类操作

7.10.6.1 simd_copy_sign

```
floatv16 simd_copy_sign(floatv16 va, floatv16 vb)
halfv16  simd_copy_sign(halfv16 va, halfv16 vb)
```

功能介绍

按元素向量符号拷贝。返回值是由va各元素的符号位与vb对应元素的指数和尾数拼接得到的浮点向量类型。

参数解释

参数名称	说明
va	floatv16、halfv16型向量。
vb	floatv16、halfv16型向量。

返回值

返回值类型	说明
floatv16	floatv16、halfv16型向量。 - 输入向量va中的某个元素值是NaN或者 $+\infty$ 时，返回值为vb。 - 输入向量va中的某个元素值是 $-\infty$ 时，返回值为-vb。 - 输入向量vb中的某个元素值是NaN时，返回元素全为NaN的向量。

注意事项

该接口支持原位操作。

使用示例1

```
floatv16 va = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_copy_sign(va, vb);
// vc 是 [-0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1]
```

使用示例2

```
floatv16 va = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
va = simd_copy_sign(va, vb);    // 原位操作
// va 是 [-0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1]
```

使用示例3

```
halfv16 va = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_copy_sign(va, vb);
// vc 是 [-0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1]
```

使用示例4

```
halfv16 va = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
va = simd_copy_sign(va, vb);    // 原位操作
// va 是 [-0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1, -0.1, 0.1]
```

7.10.7 类型转换类操作

7.10.7.1 simd_cvt_f2h

```
halfv16 simd_cvt_f2h(floatv16 va)
```

功能介绍

向量类型转换。将va中的16个float浮点数转换成16个half型浮点数组成的halfv16型向量并返回。

参数解释

参数名称	说明
va	floatv16型向量。

返回值

返回值类型	说明
halfv16	halfv16型向量。 - 输入向量va的某个元素值是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$ 。 - 输入向量va的某个元素值是NaN时，该位置对应返回NaN。

注意事项

如果floatv16 va里面的float的值超过half的取值范围，则取half数据类型能表示的最大/小值。

使用示例

```
floatv16 va = {65505.0, -65506.0, -3.0, -4.0, -5.0, -6.0, -7.0, -8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
halfv16 vb = simd_cvt_f2h(va);
// vb is [65504.0, -65504.0, -3.0, -4.0, -5.0, -6.0, -7.0, -8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0]
```

7.10.7.2 simd_cvt_h2f

```
floatv16 simd_cvt_h2f(halfv16 va)
```

功能介绍

向量类型转换。将va中的16个half型浮点数转换成16个float型浮点数组成floatv16型向量并返回。

参数解释

参数名称	说明
va	halfv16型向量。

返回值

返回值类型	说明
floatv16	floatv16型向量。 - 输入向量va的某个元素值是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$ 。 - 输入向量va的某个元素值是NaN时，该位置对应返回NaN。

使用示例

```
halfv16 va = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
floatv16 vb = simd_cvt_h2f(va);
// vb is [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1]
```

7.10.8 融合乘加类操作

7.10.8.1 simd_fma

```
halfv16 simd_fma(halfv16 va, halfv16 vb, halfv16 vc)
floatv16 simd_fma(floatv16 va, floatv16 vb, floatv16 vc)
```

功能介绍

对向量va和vb进行相应元素的乘法操作后，与向量vc的相应元素相加，返回结果向量。

参数解释

参数名称	说明
va	halfv16、floatv16型向量。
vb	halfv16、floatv16型向量。
vc	halfv16、floatv16型向量。

返回值

返回值类型	说明
halfv16、floatv16	参数对应的向量类型。 - 输入向量不存在NaN，且输入向量的某个元素值是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$。 - 输入向量的某个元素值是NaN时，该位置对应返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
halfv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
halfv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
halfv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
halfv16 vd = simd_fma(va, vb, vc);
// vd 是 [-0.5, 2.5, -2.5, 4.5, -4.5, 6.5, -6.5, 8.5, -8.5, 10.5, -10.5, 12.5, -12.5, 14.5, -14.5, 16.5]
```

使用示例2

```
floatv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
floatv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
floatv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
va = simd_fma(va, vb, vc); // 原位操作
// vd 是 [-0.5, 2.5, -2.5, 4.5, -4.5, 6.5, -6.5, 8.5, -8.5, 10.5, -10.5, 12.5, -12.5, 14.5, -14.5, 16.5]
```

7.10.8.2 simd_fms

```
halfv16 simd_fms(halfv16 va, halfv16 vb, halfv16 vc)
floatv16 simd_fms(floatv16 va, floatv16 vb, floatv16 vc)
```

功能介绍

对向量va和vb进行相应元素的乘法操作后，与向量vc的相应元素相减，返回结果向量。

参数解释

参数名称	说明
va	halfv16、floatv16型向量。
vb	halfv16、floatv16型向量。
vc	halfv16、floatv16型向量。

返回值

返回值类型	说明
halfv16、floatv16	参数对应的向量类型。 - 输入向量不存在NaN，且输入向量的某个元素值是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$。 - 输入向量的某个元素值是NaN时，该位置对应返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
halfv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
halfv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
halfv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
halfv16 vd = simd_fms(va, vb, vc);
// vd 是 [-1.5, 1.5, -3.5, 3.5, -5.5, 5.5, -7.5, 7.5, -9.5, 9.5, -11.5, 11.5, -13.5, 13.5, -15.5, 15.5]
```

使用示例2

```
floatv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
floatv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
floatv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
vb = simd_fms(va, vb, vc);    // 原位操作
// vd 是 [-1.5, 1.5, -3.5, 3.5, -5.5, 5.5, -7.5, 7.5, -9.5, 9.5, -11.5, 11.5, -13.5, 13.5, -15.5, 15.5]
```

7.10.8.3 simd_fnma

```
halfv16 simd_fnma(halfv16 va, halfv16 vb, halfv16 vc)
floatv16 simd_fnma(floatv16 va, floatv16 vb, floatv16 vc)
```

功能介绍

对向量va和vb进行相应元素的乘法操作后取负，与向量vc的相应元素相加，返回结果向量。

参数解释

参数名称	说明
va	halfv16、floatv16型向量。
vb	halfv16、floatv16型向量。
vc	halfv16、floatv16型向量。

返回值

返回值类型	说明
halfv16、floatv16	参数对应的向量类型。 - 输入向量不存在NaN，且输入向量的某个元素值是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$ 。 - 输入向量的某个元素值是NaN时，该位置对应返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
halfv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
halfv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
halfv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
halfv16 vd = simd_fnma(va, vb, vc);
// vd 是 [1.5, -1.5, 3.5, -3.5, 5.5, -5.5, 7.5, -7.5, 9.5, -9.5, 11.5, -11.5, 13.5, -13.5, 15.5, -15.5]
```

使用示例2

```
floatv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
floatv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
floatv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
vc = simd_fnma(va, vb, vc); // 原位操作
// vd 是 [1.5, -1.5, 3.5, -3.5, 5.5, -5.5, 7.5, -7.5, 9.5, -9.5, 11.5, -11.5, 13.5, -13.5, 15.5, -15.5]
```

7.10.8.4 simd_fnms

```
halfv16 simd_fnms(halfv16 va, halfv16 vb, halfv16 vc)
floatv16 simd_fnms(floatv16 va, floatv16 vb, floatv16 vc)
```

功能介绍

对向量va和vb进行相应元素的乘法操作后取负，与向量vc的相应元素相减，返回结果向量。

参数解释

参数名称	说明
va	halfv16、floatv16型向量。
vb	halfv16、floatv16型向量。

参数名称	说明
vc	halfv16、floatv16型向量。

返回值

返回值类型	说明
halfv16、floatv16	参数对应的向量类型。 - 输入向量不存在NaN，且输入向量的某个元素值是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$ 。 - 输入向量的某个元素值是NaN时，该位置对应返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
halfv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
halfv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
halfv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
halfv16 vd = simd_fnms(va, vb, vc);
// vd 是 [0.5, -2.5, 2.5, -4.5, 4.5, -6.5, 6.5, -8.5, 8.5, -10.5, 10.5, -12.5, 12.5, -14.5, 14.5, -16.5]
```

使用示例2

```
floatv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
floatv16 vb = {-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1};
floatv16 vc = {0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5};
va = simd_fnms(va, vb, vc);    // 原位操作
// vd 是 [0.5, -2.5, 2.5, -4.5, 4.5, -6.5, 6.5, -8.5, 8.5, -10.5, 10.5, -12.5, 12.5, -14.5, 14.5, -16.5]
```

7.10.9 算数运算类操作

7.10.9.1 simd_add

```
intv16  simd_add(intv16 va, intv16 vb)
shortv32 simd_add(shortv32 va, shortv32 vb)
floatv16 simd_add(floatv16 va, floatv16 vb)
halfv16 simd_add(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb进行对应元素相加后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。 - 输入向量va或vb不存在NaN，且输入向量va或vb的某个元素值是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$ 。 - 输入向量va或vb的某个元素值是NaN时，该位置对应返回NaN。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `+` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_add(va, vb);
// vc 是 [2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_add(va, vb);    // 原位操作
// va 是 [2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
```

```
1, 1, 1};
shortv32 vc = simd_add(va, vb);
// vc 是 [2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1]
```

使用示例4

```
floatv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_add(va, vb);
// vc 是 [-0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1]
```

使用示例5

```
halfv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_add(va, vb);
// vc 是 [-0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1, -0.9, 0.1]
```

7.10.9.2 simd_sub

```
intv16 simd_sub(intv16 va, intv16 vb)
shortv32 simd_sub(shortv32 va, shortv32 vb)
floatv16 simd_sub(floatv16 va, floatv16 vb)
halfv16 simd_sub(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb进行对应元素相减后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。 - 输入向量va或vb不存在NaN，当输入向量va的某个元素值是 $\pm\infty$ 时： - 如果vb对应位置不是 $\pm\infty$ 时，该位置对应返回 $\pm\infty$ 。 - 如果vb对应位置是 $\pm\infty$ 且与va该位置元素符号相同时，该位置对应返回NaN。 - 如果vb对应位置是 $\pm\infty$ 且与va该位置元素符号相反时，该位置返回va对应位置的值。 - 输入向量va或vb不存在NaN，当输入向量vb的某个元素值是 $\pm\infty$ 时： - 如果va对应位置不是 $\pm\infty$ 时，该位置对应返回 $\mp\infty$ 。 - 如果va对应位置是 $\pm\infty$ 且与vb该位置元素符号相同时，该位置对应返回NaN。 - 如果va对应位置是 $\pm\infty$ 且与vb该位置元素符号相反时，该位置返回va对应位置的值。 - 输入向量va或vb的某个元素值是NaN时，该位置对应返回NaN。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `-` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_sub(va, vb);
// vc 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_sub(va, vb);    // 原位操作
// va 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

使用示例3


```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_sub(va, vb);
// vc 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

使用示例4

```
floatv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_sub(va, vb);
// vc 是 [-1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1]
```

使用示例5

```
halfv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_sub(va, vb);
// vc 是 [-1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1, -1.1, -0.1]
```

7.10.9.3 simd_mul

```
intv16 simd_mul(intv16 va, intv16 vb)
shortv32 simd_mul(shortv32 va, shortv32 vb)
floatv16 simd_mul(floatv16 va, floatv16 vb)
halfv16 simd_mul(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb进行对应元素相乘后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。 - 输入向量va或vb不存在NaN，且输入向量va或vb的某个元素值是$\pm\infty$时，该位置对应返回$\pm\infty$。 - 输入向量va或vb的某个元素值是NaN时，该位置对应返回NaN。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `*` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_mul(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_mul(va, vb);    // 原位操作
// va 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_mul(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例4

```
floatv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_mul(va, vb);
// vc 是 [-0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0]
```

使用示例5

```
halfv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_mul(va, vb);
// vc 是 [-0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0, -0.1, 0.0]
```

7.10.9.4 simd_div

```
intv16  simd_div(intv16 va, intv16 vb)
shortv32 simd_div(shortv32 va, shortv32 vb)
floatv16 simd_div(floatv16 va, floatv16 vb)
halfv16 simd_div(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb进行对应元素相除后，返回结果向量。其中，va为被除数，vb为除数。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。 - 输入向量va的某个元素值是 $\pm\infty$ 且vb对应位置的元素不是NaN时，该位置对应返回 $\pm\infty$ 。 - 输入向量vb的某个元素值是 $\pm\infty$ 且va对应位置的元素不是NaN时，该位置对应返回 ± 0 。 - 输入向量va或vb的某个元素值是NaN时，该位置对应返回NaN。

注意事项

- 该接口支持原位操作。

- 可以使用算数运算符 `/` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_div(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_div(va, vb);    // 原位操作
// va 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_div(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例4

```
floatv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_div(va, vb);
// vc 是 [-10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0]
```

使用示例5

```
halfv16 va = {-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_div(va, vb);
// vc 是 [-10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0, -10.0, 0.0]
```

7.10.9.5 simd_rem

```
intv16 simd_rem(intv16 va, intv16 vb)
shortv32 simd_rem(shortv32 va, shortv32 vb)
```

功能介绍

对向量va和vb进行对应元素取余计算后，返回结果向量。其中，va为被除数，vb为除数。

参数解释

参数名称	说明
va	intv16、shortv32型向量。
vb	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `%` 进行平替。

使用示例1

```
intv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
intv16 vb = {2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2};
intv16 vc = simd_rem(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例2

```
intv16 va = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
intv16 vb = {2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2};
va = simd_rem(va, vb);    // 原位操作
// va is [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例3

7.10.9.7 simd_redsum

```
int simd_redsum(intv16 va)
short simd_redsum(shortv32 va)
float simd_redsum(floatv16 va)
half simd_redsum(halfv16 va)
```

功能介绍

对向量va的每个元素进行逐一累加，返回结果标量。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv、shortv、float、half	参数对应的标量类型。

注意事项

- 输入向量va元素中含有NaN时，累加结果为NaN。
- 输入向量va元素中含有 $\pm\infty$ 时，累加结果为 $\pm\infty$ 。

使用示例1

```
intv16 intva = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
int a = simd_redsum(intva); // a 是 136
```

使用示例2

```
shortv32 shortva = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,
                    18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32};
short a = simd_redsum(shortva);
// a 是 528
```

使用示例3

```
floatv16 fva = {1.3, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
float a = simd_redsum(fva);
// a 是 136.3
```

使用示例4

```
halfv16 halfva = {1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
half a = simd_redsum(halfva);
// a 是 136.0
```

7.10.10 比较类操作

7.10.10.1 simd_cmplt

```
intv16 simd_cmplt(intv16 va, intv16 vb)
shortv32 simd_cmplt(shortv32 va, shortv32 vb)
floatv16 simd_cmplt(floatv16 va, floatv16 vb)
halfv16 simd_cmplt(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb的对应元素进行小于比较后，返回结果向量。如果va的元素小于vb的相对应元素，则返回1，否则返回0。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。

注意事项

- 该接口支持原位操作。

- 可以使用运算符 `<` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_cmplt(va, vb);
// vc 是 [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_cmplt(va, vb); // 原位操作
// va 是 [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_cmplt(va, vb);
// vc 是 [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

使用示例4

```
floatv16 va = {2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_cmplt(va, vb);
// vc 是 [0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0]
```

使用示例5

```
halfv16 va = {2.0, 0, 2.0, 0, 2.0, 0, 2.0, 0, 2.0, 0, 2.0, 0, 2.0, 0, 2.0, 0};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_cmplt(va, vb);
// vc 是 [0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0]
```

7.10.10.2 simd_cmple

```
intv16 simd_cmple(intv16 va, intv16 vb)
shortv32 simd_cmple(shortv32 va, shortv32 vb)
floatv16 simd_cmple(floatv16 va, floatv16 vb)
halfv16 simd_cmple(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb的对应元素进行小于等于比较后，返回结果向量。如果va的元素小于等于vb的相对应元素，则返回1，否则返回0。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用运算符 `<=` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_cmple(va, vb);
// vc 是 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_cmple(va, vb); // 原位操作
// va 是 [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

使用示例3


```
halfv16 vc = simd_cmpgt(va, vb);
// vc 是 [1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0]
```

7.10.10.4 simd_cmpge

```
intv16 simd_cmpge(intv16 va, intv16 vb)
shortv32 simd_cmpge(shortv32 va, shortv32 vb)
floatv16 simd_cmpge(floatv16 va, floatv16 vb)
halfv16 simd_cmpge(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb的对应元素进行大于等于比较后，返回结果向量。如果va的元素大于等于vb的相对应元素，则返回1，否则返回0。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用运算符 `>=` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_cmpge(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_cmpge(va, vb);    // 原位操作
// va 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_cmpeq(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例4

```
floatv16 va = {2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_cmpeq(va, vb);
// vc 是 [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

使用示例5

```
halfv16 va = {2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_cmpeq(va, vb);
// vc 是 [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]
```

7.10.10.5 simd_cmpeq

```
intv16 simd_cmpeq(intv16 va, intv16 vb)
shortv32 simd_cmpeq(shortv32 va, shortv32 vb)
floatv16 simd_cmpeq(floatv16 va, floatv16 vb)
halfv16 simd_cmpeq(halfv16 va, halfv16 vb)
intv16 simd_cmpeq(intv16 va, int b)
shortv32 simd_cmpeq(shortv32 va, short b)
floatv16 simd_cmpeq(floatv16 va, float b)
halfv16 simd_cmpeq(halfv16 va, half b)
```

功能介绍

对向量va和vb的对应元素进行等于比较后，返回结果向量。如果va的元素等于vb的相对应元素，则返回1，否则返回0。


```
intv16 simd_cmpne(intv16 va, intv16 vb)
shortv32 simd_cmpne(shortv32 va, shortv32 vb)
floatv16 simd_cmpne(floatv16 va, floatv16 vb)
halfv16 simd_cmpne(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb的对应元素进行不等于比较后，返回结果向量。如果va的元素不等于vb的相对应元素，则返回1，否则返回0。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。
vb	intv16、shortv32、floatv16、halfv16型向量。

返回值

返回值类型	说明
intv16、shortv32、floatv16、halfv16	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用运算符 `!=` 进行平替。

使用示例1

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_cmpne(va, vb);
// vc 是 [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

使用示例2

```
intv16 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_cmpne(va, vb); // 原位操作
// va 是 [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

使用示例3

```
shortv32 va = {1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_cmpne(va, vb);
// vc 是 [0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
```

使用示例4

```
floatv16 va = {2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1};
floatv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
floatv16 vc = simd_cmpne(va, vb);
// vc 是 [1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0]
```

使用示例5

```
halfv16 va = {2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1, 2.0, 0.1};
halfv16 vb = {0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1, 0.1};
halfv16 vc = simd_cmpne(va, vb);
// vc 是 [1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0]
```

7.10.10.7 simd_isnan

```
floatv16 simd_isnan(floatv16 va)
halfv16  simd_isnan(halfv16 va)
```

功能介绍

判断向量va里的浮点类元素是不是NaN（非数），返回结果向量。

- 如果是NaN，则结果向量的对应元素置为2.0。
- 如果不是NaN，则结果向量的对应元素置为0.0。

参数解释

参数名称	说明
va	floatv16、halfv16型向量。

返回值

返回值类型	说明
floatv16、halfv16	参数对应的向量类型。

注意事项

该接口支持原位操作。

使用示例

```
floatv16 fva = {1.3, NAN, NAN, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
fva = simd_isnan(fva);
// fva 是 [0.0, 2.0, 2.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

7.10.10.8 simd_max

```
floatv16 simd_max(floatv16 va, floatv16 vb)
halfv16  simd_max(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb的对应元素进行比较后取较大的值，返回结果向量。

参数解释

参数名称	说明
va	floatv16、halfv16型向量。
vb	floatv16、halfv16型向量。

返回值

返回值类型	说明
floatv16、halfv16	参数对应的向量类型。 - 输入向量va的元素中含有NaN时，该位置对应返回NaN。 - 输入向量vb的元素中含有NaN时，该位置返回向量va对应元素的数据。

注意事项

该接口支持原位操作。

使用示例1

```
floatv16 fva = {1.3, NAN, NAN, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
floatv16 fvb = {16.0, 15.0, 14.0, 13.0, 12.0, 11.0, 10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0};
fva = simd_max(fva, fvb);
// fva 是 [16.0, NAN, NAN, 13.0, 12.0, 11.0, 10.0, 9.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0]
```

使用示例2

```
floatv16 fva = {16.0, 15.0, 14.0, 13.0, 12.0, 11.0, 10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0};
floatv16 fvb = {1.3, NAN, NAN, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
fva = simd_max(fva, fvb);
// fva 是 [16.0, 15.0, 14.0, 13.0, 12.0, 11.0, 10.0, 9.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0]
```

7.10.10.9 simd_min

```
floatv16 simd_min(floatv16 va, floatv16 vb)
halfv16  simd_min(halfv16 va, halfv16 vb)
```

功能介绍

对向量va和vb的对应元素进行比较后取较小的值，返回结果向量。

参数解释

参数名称	说明
va	floatv16、halfv16型向量。
vb	floatv16、halfv16型向量。

返回值

返回值类型	说明
floatv16、halfv16	参数对应的向量类型。 - 输入向量va的元素中含有NaN时，该位置返回向量vb对应元素的数据。 - 输入向量vb的元素中含有NaN时，该位置对应返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
floatv16 fva = {1.3, NAN, NAN, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
floatv16 fvb = {16.0, 15.0, 14.0, 13.0, 12.0, 11.0, 10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0};
fva = simd_min(fva, fvb);
// fva 是 [1.3, 15.0, 14.0, 4.0, 5.0, 6.0, 7.0, 8.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0]
```

使用示例2

```
floatv16 fva = {16.0, 15.0, 14.0, 13.0, 12.0, 11.0, 10.0, 9.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0};
floatv16 fvb = {1.3, NAN, NAN, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0, 11.0, 12.0, 13.0, 14.0, 15.0, 16.0};
fva = simd_min(fva, fvb);
// fva 是 [1.3, NAN, NAN, 4.0, 5.0, 6.0, 7.0, 8.0, 8.0, 7.0, 6.0, 5.0, 4.0, 3.0, 2.0, 1.0]
```

7.10.11 逻辑运算类操作

7.10.11.1 simd_and

```
intv16 simd_and(intv16 va, intv16 vb)
shortv32 simd_and(shortv32 va, shortv32 vb)
```

功能介绍

对向量va和vb的对应元素进行逻辑与计算后，返回结果向量。

- 如果va与vb中相对应元素都不为0，则返回非0值。
- 如果va与vb中相对应元素都为0，则返回0。

参数解释

参数名称	说明
va	intv16、shortv32型向量。
vb	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `&&` 进行平替。

使用示例1

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_and(va, vb);
// vc 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

使用示例2

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_and(va, vb);    // 原位操作
// va 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

使用示例3

```
shortv32 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_and(va, vb);
// vc 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

7.10.11.2 simd_or

```
intv16 simd_or(intv16 va, intv16 vb)
shortv32 simd_or(shortv32 va, shortv32 vb)
```

功能介绍

对向量va和vb的对应元素进行逻辑或计算后，返回结果向量。

- 如果va与vb中相对应元素都不全为0，则返回非0值。
- 如果va与vb中相对应元素全为0，则返回0。

参数解释

参数名称	说明
va	intv16、shortv32型向量。
vb	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `||` 进行平替。

使用示例1

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
intv16 vc = simd_or(va, vb);
// vc 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

使用示例2

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
va = simd_or(va, vb); // 原位操作
// va 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

使用示例3

```
shortv32 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
shortv32 vb = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
shortv32 vc = simd_or(va, vb);
// vc 是 [0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1]
```

7.10.11.3 simd_not

```
intv16 simd_not(intv16 va)
shortv32 simd_not(shortv32 va)
```

功能介绍

对向量va中各元素进行逻辑取反计算后，返回结果向量。

- 如果va的元素为0，则返回非0值。
- 如果va的元素不为0，则返回0。

参数解释

参数名称	说明
va	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `!` 进行平替。

使用示例1


```
intv16 va = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
intv16 vb = simd_not(va);
// vb 是 [-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0]
```

使用示例2

```
intv16 va = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
va = simd_not(va);    // 原位操作
// va 是 [-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0]
```

使用示例3

```
shortv32 va = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
shortv32 vb = simd_not(va);
// vb 是 [-1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0, -1, 0]
```

7.10.11.4 simd_neg

```
intv16  simd_neg(intv16 va)
shortv32 simd_neg(shortv32 va)
floatv16 simd_neg(floatv16 va)
halfv16 simd_neg(halfv16 va)
```

功能介绍

对向量va中各元素的符号取反计算后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32、floatv16、halfv16型向量。

返回值


```
halfv16 va = {2.0, -0.1, 2.0, -0.1, 2.0, -0.1, 2.0, -0.1, 2.0, -0.1, 2.0, -0.1, 2.0, -0.1, 2.0, -0.1};
halfv16 vb = simd_neg(va);
// vb 是 [-2.0, 0.1, -2.0, 0.1, -2.0, 0.1, -2.0, 0.1, -2.0, 0.1, -2.0, 0.1, -2.0, 0.1, -2.0, 0.1]
```

7.10.12 按位运算类操作

7.10.12.1 simd_band

```
intv16 simd_band(intv16 va, intv16 vb)
shortv32 simd_band(shortv32 va, shortv32 vb)
```

功能介绍

对向量va和vb的对应元素进行按位与计算后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32型向量。
vb	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `&` 进行平替。

使用示例1

```
intv16 va = {1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_band(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例2

```
intv16 va = {1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_band(va, vb);    // 原位操作
// va 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

使用示例3

```
shortv32 va = {1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2, 1, 2};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_band(va, vb);
// vc 是 [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
```

7.10.12.2 simd_bxor

```
intv16 simd_bxor(intv16 va, intv16 vb)
shortv32 simd_bxor(shortv32 va, shortv32 vb)
```

功能介绍

对向量va和vb的对应元素进行按位异或计算后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32型向量。
vb	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。

- 可以使用算数运算符 `^` 进行平替。

使用示例1

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_bxor(va, vb);
// vc 是 [1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3]
```

使用示例2

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_bxor(va, vb); // 原位操作
// va 是 [1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3]
```

使用示例3

```
shortv32 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_bxor(va, vb);
// vc 是 [1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3]
```

7.10.12.3 simd_bor

```
intv16 simd_bor(intv16 va, intv16 vb)
shortv32 simd_bor(shortv32 va, shortv32 vb)
```

功能介绍

对向量va和vb的对应元素进行按位或计算后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32型向量。
vb	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `|` 进行平替。

使用示例1

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
intv16 vc = simd_bor(va, vb);
// vc 是 [1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3]
```

使用示例2

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
va = simd_bor(va, vb);    // 原位操作
// va 是 [1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3]
```

使用示例3

```
shortv32 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
shortv32 vb = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};
shortv32 vc = simd_bor(va, vb);
// vc 是 [1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3, 1, 3]
```

7.10.12.4 simd_bnor

```
intv16 simd_bnor(intv16 va, intv16 vb)
shortv32 simd_bnor(shortv32 va, shortv32 vb)
```

功能介绍

对向量vb中的各元素进行按位取反操作后，再与va对应的元素进行按位或运算，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32型向量。
vb	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `|` `~` 进行平替。

使用示例1

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
intv16 vc = simd_bnor(va, vb);
// vc 是 [-1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2]
```

使用示例2

```
intv16 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
intv16 vb = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
va = simd_bnor(va, vb);    // 原位操作
// va 是 [-1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2]
```

使用示例3

```
shortv32 va = {0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2};
shortv32 vb = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
shortv32 vc = simd_bnor(va, vb);
// vc 是 [-1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0, -1.0, -2.0]
```

7.10.12.5 simd_bnot

```
intv16  simd_bnot(intv16 va)
shortv32 simd_bnot(shortv32 va)
```

功能介绍

对向量va中各元素进行按位取反计算后，返回结果向量。

参数解释

参数名称	说明
va	intv16、shortv32型向量。

返回值

返回值类型	说明
intv16、shortv32	参数对应的向量类型。

注意事项

- 该接口支持原位操作。
- 可以使用算数运算符 `~` 进行平替。

使用示例1

```
intv16 va = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
intv16 vb = simd_bnot(va);
// vb 是 [-1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2]
```

使用示例2

```
intv16 va = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
va = simd_bnot(va);    // 原位操作
// va 是 [-1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2, -1, -2]
```

使用示例3

使用示例

以统计某段循环加法代码的执行周期为例，介绍如何使用 `perf_start` 和 `perf_stop`。

```
#include "sdaa_perf.h"
#define SIZE 1000

__device__ void sum_func(const int *arr1, const int *arr2, int len, int *res_arr)
{
    // 设置采集性能项为执行周期，并开始采集
    perf_start();
    for (int i = 0; i < len; i++) {
        res_arr[i] = arr1[i] + arr2[i];
    }
    // 结束性能采集
    (void)perf_stop();
}

__global__ void test()
{
    int *res_arr = (int *)malloc(SIZE * sizeof(int));
    int array1[SIZE];
    int array2[SIZE];
    for (int i = 0; i < SIZE; i++) {
        array1[i] = i;
        array2[i] = i * 10;
    }
    sum_func(array1, array2, SIZE, res_arr);
    free(res_arr);
}

int main()
{
    sdaaSetDevice(0);
    test<<<1>>>();
    sdaaDeviceSynchronize();
    return 0;
}
```

7.11.3 perf_stop

PerfData `perf_stop()`

功能介绍

标识性能采样的结束位置。

参数解释

无。

返回值

参数名称	说明
PerfData	性能采样的结构体数据，可参考 PerfData ，目前支持： - latency：延时拍数，单位：拍。 - cache_miss：指令缓存脱靶次数，单位：次。

注意事项

- 需要手动包含 `sdaa_perf.h` 头文件。
- `perf_stop` 需要与[perf_start](#)配合使用，并且需要一一对应，不支持嵌套使用。

使用示例

以统计某段循环加法代码的执行周期为例，介绍如何使用[perf_start](#)和 `perf_stop`。

```
#include "sdaa_perf.h"
#define SIZE 1000

__device__ void sum_func(const int *arr1, const int *arr2, int len, int *res_arr)
{
    // 设置采集性能项为执行周期，并开始采集
    perf_start();
    for (int i = 0; i < len; i++) {
        res_arr[i] = arr1[i] + arr2[i];
    }
    // 结束性能采集
    PerfData tmp_data = perf_stop();
    // 自行打印采集的性能数据
    printf("perf latency:%lu, cache miss:%lu\n",
        tmp_data.latency, tmp_data.cache_miss);
}

__global__ void test()
{
    int *res_arr = (int *)malloc(SIZE * sizeof(int));
    int array1[SIZE];
    int array2[SIZE];
    for (int i = 0; i < SIZE; i++) {
        array1[i] = i;
        array2[i] = i * 10;
    }
    sum_func(array1, array2, SIZE, res_arr);
    free(res_arr);
}

int main()
{
    sdaaSetDevice(0);
```

```
test<<<1>>>();  
sdaaDeviceSynchronize();  
return 0;  
}
```

7.11.4 perf_print

```
void perf_print()
```

功能介绍

在终端界面输出采集到的性能数据，包括：延时拍数和指令缓存脱靶次数等。

参数解释

无。

返回值

无。

注意事项

- 需要手动包含 sdaa_perf.h 头文件。
- 用户需要在采集完性能数据后才能使用 `perf_print` 输出性能数据。
- 为避免对其他代码产生影响，保证性能数据采集的准确性，请将 `perf_print` 放在Kermel函数返回前的位置。
- 每个SPE最多采集10组性能数据。如果采集数量超过10组，会保留最后的10组数据。

使用示例

以统计某段循环加法代码的执行周期为例，介绍如何使用 `perf_print` 输出采集的性能数据。

```
#include "sdaa_perf.h"  
#define SIZE 1000  
  
__device__ void sum_func(const int *arr1, const int *arr2, int len, int *res_arr)  
{  
    // 设置采集性能项为执行周期，并开始采集  
    perf_start();  
    for (int i = 0; i < len; i++) {  
        res_arr[i] = arr1[i] + arr2[i];  
    }  
    // 结束性能采集  
    (void)perf_stop();  
}
```

```
__global__ void test()
{
    int *res_arr = (int *)malloc(SIZE * sizeof(int));
    int array1[SIZE];
    int array2[SIZE];
    for (int i = 0; i < SIZE; i++) {
        array1[i] = i;
        array2[i] = i * 10;
    }
    sum_func(array1, array2, SIZE, res_arr);
    free(res_arr);
    // 输出采集的性能数据
    perf_print();
}

int main()
{
    sdaaSetDevice(0);
    test<<<1>>>();
    sdaaDeviceSynchronize();
    return 0;
}
```

性能采样结果

打印输出多个SPE的性能采样数据。不同SPE的数据理论上相同，实际执行时根据代码的执行情况可能会存在微小差异，具体请以实际情况为准。

Kernel perf data of thread 0:

```
-----
|START LINE NUM          |7|
|LATENCY                  |11013|
|CNT_L1IC_MISS            |0|
```

Kernel perf data of thread 1:

```
-----
|START LINE NUM          |7|
|LATENCY                  |11013|
|CNT_L1IC_MISS            |0|
....
```

Kernel perf data of thread 31:

```
-----
|START LINE NUM          |7|
|LATENCY                  |11013|
|CNT_L1IC_MISS            |0|
```

输出说明

字段	说明
START LINE NUM	测试开始的行号。

字段	说明
LATENCY	延迟时间，即执行周期，单位：拍。 ? 说明： 具体结果可能随着不同的硬件上下波动。
CNT_LIIC_MISS	指令缓存脱靶次数，单位：次。

7.11.5 clock

```
unsigned long clock()
```

功能介绍

获取当前SPE内时钟周期计数器的值。您可以在代码段前后进行两次插桩采样，并计算两次采样的差值，统计指定代码段的执行周期。

参数解释

无。

返回值

返回值类型	说明
unsigned long	返回当前SPE内时钟周期计数器的值。

注意事项

- 需要手动包含 `sdaa_clock.h` 头文件。
- 需要引入`sdaa`命名空间。

使用示例

以统计某段循环加法代码的执行周期为例，介绍如何使用 `clock` 。

```
__device__ void sum_func(const int *arr1, const int *arr2, int len, int *res_arr)
{
    // 在指定代码段之前插桩，获取起始时钟周期计数器的值
    unsigned long start_time = clock();

    for (int i = 0; i < len; i++) {
        res_arr[i] = arr1[i] + arr2[i];
    }
}
```

```
}

// 在指定代码段之后插桩，获取起始时钟周期计数器的值
unsigned long end_time = clock();

// 打印代码段的执行周期
printf("SPE ID: %lu, timd: %lu\n", threadIdx, end_time - start_time);
}

__global__ void test()
{
    int *res_arr = (int *)malloc(SIZE * sizeof(int));
    int array1[SIZE];
    int array2[SIZE];
    for (int i = 0; i < SIZE; i++) {
        array1[i] = i;
        array2[i] = i * 10;
    }
    sum_func(array1, array2, SIZE, res_arr);
    free(res_arr);
}

int main()
{
    sdaaSetDevice(0);
    test<<<1>>>();
    sdaaDeviceSynchronize();
    return 0;
}
```

7.11.6 PerfData

```
typedef struct _perfdata {
    uint64_t latency;
    uint64_t cache_miss;
} PerfData;
```

功能介绍

性能采样的结构体变量。

变量解释

变量名称	说明
latency	延时拍数，单位：拍。
cache_miss	指令缓存脱靶次数，单位：次。

注意事项

需要引入sdac命名空间。

使用示例

创建PerfData类型变量。

```
using namespace sdac;
__global__ void func()
{
    // 创建PerfData类型变量
    PerfData tmp_data = {0, 0};

    // 其他代码逻辑
}
```

7.12 设备信息查询

7.12.1 概述

设备信息查询接口提供了一系列接口，用于查询当前计算核心SPE的参数信息。

- get_heap_size**：获取当前计算核心SPE的堆空间总大小。
- get_stack_size**：获取当前计算核心SPE的栈空间总大小。
- get_local_size**：获取当前计算核心SPE的local空间总大小。

7.12.2 get_heap_size

```
size_t get_heap_size()
```

功能介绍

获取当前计算核心SPE的堆空间总大小。

返回值

返回值类型	说明
size_t	当前计算核心SPE的堆空间总大小，单位：Byte。

使用示例


```
__device__ void test()
{
    size_t size = get_heap_size();
}
```

7.12.3 get_stack_size

```
size_t get_stack_size()
```

功能介绍

获取当前计算核心SPE的栈空间总大小。

返回值

返回值类型	说明
size_t	当前计算核心SPE的栈空间总大小，单位：Byte。

使用示例

```
__device__ void test()
{
    size_t size = get_stack_size();
}
```

7.12.4 get_local_size

```
size_t get_local_size()
```

功能介绍

获取当前计算核心SPE的local空间总大小。

返回值

返回值类型	说明
size_t	当前计算核心SPE的local空间总大小，单位：Byte。

使用示例

```
__device__ void test()
{
    size_t size = get_local_size();
}
```

7.13 程序调试

7.13.1 概述

程序调试是应用程序开发人员进行软件开发过程中必要的纠错手段。一款优秀的程序调试工具可以协助软件开发人员快速定位代码在编译、运行时发生错误的原因和位置，提升软件的开发效率。

SDAA C提供了如下程序调试接口：

- **printf**：打印设备端运行时日志。通过打印运行时日志的方式，追踪打印计算核心SPE中代码的运行状态，从而定位出问题发生的具体原因。
- **abort**：终止设备端程序。通过终止设备端程序的方式，在设备端的函数代码中检测到无法修复的问题时，及时停止设备端程序的运行。
- **assert**：断言。通过断言的方式，检测代码中存在的错误，出现错误时打印对应的文件行号等详细信息。

7.13.2 printf

```
int printf(const char *format, ...)
```

功能介绍

打印设备端运行时日志。可以通过调用 `printf` 接口打印设备端目标变量的运行时状态。

参数解释

参数名称	说明
format	格式化字符串，指定需要输出的文本及格式化说明符。  说明： 常用格式化说明符： • %d：打印有符号十进制整数。 • %u：打印无符号十进制整数。 • %f：打印浮点数（默认6位小数）。 • %c：打印单个字符。 • %s：打印字符串。 • %x/%X：打印十六进制整数（小写/大写）。 • %p：打印指针地址。 • %%：打印百分号。
...	可变参数，对应格式化说明符的变量或值。

返回值

- 打印成功，返回打印的字符数。
- 打印失败，返回负值。

注意事项

- `printf` 接口是SDAA C提供的内置接口，使用时无需引入头文件。
- 设置环境变量`export SDAA_SYNC_PRINT=1`，可以按照SPE编号从小到大的顺序进行打印。默认按照SPE的执行顺序进行打印。

使用示例

```
__device__ void sum_func(int a, int b)
{
    int g_sum = a + b;
    printf("threadIdx: %lu, sum: %d\n", threadIdx, g_sum);
}

__global__ void bar()
{
    sum_func(1, 2);
}
```

```
int main()
{
    sdaaSetDevice(0);
    bar<<<1>>>();
    sdaaDeviceSynchronize();

    return 0;
}
```

- 不设置环境变量 `SDAA_SYNC_PRINT` 或者值默认为0时，会按照SPE的执行顺序进行打印。执行后打印结果如下：

```
threadIdx: 4, sum: 3
threadIdx: 0, sum: 3
threadIdx: 6, sum: 3
threadIdx: 7, sum: 3
.....
```

- 设置环境变量 `SDAA_SYNC_PRINT=1`，按照SPE编号从小到大的顺序进行打印。执行后打印结果如下：

```
threadIdx: 0, sum: 3
threadIdx: 1, sum: 3
threadIdx: 2, sum: 3
threadIdx: 3, sum: 3
.....
```

7.13.3 abort

```
void abort()
```

功能介绍

终止设备端程序。如果您在设备端编写的函数代码中，检查到无法修复的问题，可以通过 `abort` 接口直接终止设备端的程序运行。

参数解释

无。

返回值

无。

注意事项

- `abort` 接口是SDAA C提供的内置接口，使用时无需引入头文件。

- 通过 `abort` 函数结束运行时：
 - 如果设置了环境变量 `SDAA_ENABLE_COREDUMP_ON_EXCEPTION`（详情请参考《SDAARuntime用户手册》的 **SDAA_ENABLE_COREDUMP_ON_EXCEPTION** 章节），通知主机端生成包含错误相关上下文的 Core Dump 文件，并且终止主机端的代码执行。
 - 如果没有设置该环境变量 `SDAA_ENABLE_COREDUMP_ON_EXCEPTION`（详情请参考《SDAARuntime用户手册》的 **SDAA_ENABLE_COREDUMP_ON_EXCEPTION** 章节）：继续执行主机端的代码，后续通过执行 `sdaaDeviceSynchronize` 接口，获取对应的 `abort` 错误码。

使用示例

以检测除数不为0作为示例，当检查到除数为0时，执行 `abort` 终止SPE后续程序的运行。

```
__device__ int g_sum = 0;
__device__ int my_div(int a, int b)
{
    if (b == 0) {
        // 如果除数b为0，终止SPE后续程序的运行
        abort();
    }
    return a / b;
}

__device__ void my_compute(int a, int b)
{
    int div_result = my_div(a, b);
    g_sum = (a - b) * div_result;
}

__global__ void bar()
{
    my_compute(1, 2);
}

int main()
{
    sdaaSetDevice(0);
    bar<<<1>>>();
    sdaaDeviceSynchronize();

    return 0;
}
```

7.13.4 assert

```
void assert(bool condition)
```

功能介绍

检查代码中是否出现错误，出现错误后会打印对应错误所在的文件行号等信息，之后终止设备端程序的运行。

参数解释

参数名称	说明
condition	需要检查的条件表达式。  说明： 如果条件表达式的返回值为0/false，则触发断言失败。

返回值

无。

注意事项

- `assert` 接口是SDAA C提供的内置接口，使用时无需引入头文件。
- 通过 `assert` 函数终止设备端程序运行时：
 - 如果设置了环境变量SDAA_ENABLE_COREDUMP_ON_EXCEPTION（详情请参考《SDAARuntime用户手册》的SDAA_ENABLE_COREDUMP_ON_EXCEPTION章节）：通知主机端生成包含错误相关上下文的Core Dump文件，并且终止主机端的代码执行。
 - 如果没有设置环境变量SDAA_ENABLE_COREDUMP_ON_EXCEPTION（详情请参考《SDAARuntime用户手册》的SDAA_ENABLE_COREDUMP_ON_EXCEPTION章节）：继续执行主机端的代码，后续通过执行 `sdaaDeviceSynchronize`接口，获取对应的assert错误码。

使用示例

以检测除数不为0作为示例，`assert` 会检查除数是否为0，为0时会打印错误所在的文件行号信息并且终止SPE后续程序的执行。

```
__device__ int g_sum = 0;
__device__ void my_div(int a, int b)
{
    // 如果除数b为0，则打印错误信息并终止SPE后续程序的执行
    assert(b != 0 && "b should not be zero");
    g_sum = a / b;
}

__global__ void bar()
{
    my_div(1, 2);
}

int main()
{
    sdaaSetDevice(0);
    bar<<<1>>>();
}
```

```
sdaaDeviceSynchronize();  
  
return 0;  
}
```

8. 数学函数

8.1 概述

数学函数不需要#include相应的头文件即可使用。SDAA C内置了包括：指数计算、对数计算、正弦计算、余弦计算、正切计算、高斯误差计算等常用标量数学计算接口。为了满足用户对于应用程序运行性能的追求，SDAA C还提供了基于ULP(Unit in the Last Place)精度标识的向量数学计算接口。

8.2 单精度标量接口

8.2.1 ceilf

```
float ceilf(float x)
```

功能介绍

计算不小于给定单精度浮点数x的最小整数。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	大于等于参数x的最小整数。 - 假设y为小于x的最大整数，当x位于(y, y + 1]的时候，返回 y + 1。 - 当x取 $\pm\infty$ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
```



```
    for (int i = 0; i < len; i++) {  
        b[i]= ceilf(a[i]) + 1;  
    }  
}
```

使用示例2

```
__device__ void test(float *a, int len)  
{  
    for (int i = 0; i < len; i++) {  
        a[i]= ceilf(a[i]) + 1;    // 原位操作  
    }  
}
```

8.2.2 floorf

```
float floorf(float x)
```

功能介绍

计算不大于给定单精度浮点数x的最大整数值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	小于等于x的最大整数值。 - 假设y为小于等于x的最大整数，当x位于[y, y + 1)的时候，返回y。 - 当x取 $\pm\infty$ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i] = floorf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i] = floorf(a[i]) + 1;    // 原位操作
    }
}
```

8.2.3 fabsf

```
float fabsf(float x)
```

功能介绍

计算单精度浮点数x的绝对值。最大ULP为0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	x的绝对值。 - 当x小于0时，返回-x。 - 当x大于等于0时，返回x。 - 当x取-0时，返回0。 - 当x取 ∞ 或者 NaN 时，返回x。 - 当x取 $-\infty$ 时，返回 ∞ 。

注意事项。

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= fabsf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fabsf(a[i]) + 1;    // 原位操作
    }
}
```

8.2.4 sqrtf

```
float sqrtf(float x)
```

功能介绍

计算单精度浮点数x的平方根函数。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	对数计算结果，其中： - 当x小于0时，返回 NaN 。 - 当x取0时，返回0。 - 当x等于 ∞ 时，返回 ∞ 。 - 当x等于 $-\infty$ 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= sqrtf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= sqrtf(a[i]);    // 原位操作
    }
}
```

8.2.5 erff

```
float erff(float x)
```

功能介绍

计算单精度浮点的高斯误差函数的取值， $\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$ 。最大ULP为1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	高斯误差函数的取值，其中： - 当x取 ± 0时，返回+0。 - 当x小于等于-6时，返回-1。 - 当x大于等于6时，返回1。 - 当x取 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= erff(a[i]);
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= erff(a[i]);    // 原位操作
    }
}
```

8.2.6 expf

```
float expf(float x)
```

功能介绍

计算单精度浮点的 e^x 的取值，其中 e 为底数。最大ULP为 1。

参数解释

参数名称	说明
x	输入参数，作为e的指数。

返回值

返回值类型	说明
float	e^x 的取值，其中： - 当x小于-103.97时，返回 0。 - 当x取 ± 0 时，返回 1。 - 当x大于88.72时，返回 ∞ 。 - 当x取 NaN 或者 ∞ 时，返回x。 - 当x取 $-\infty$ 时，返回0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= expf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
```

```
        a[i]= expf(a[i]) + 1;    // 原位操作
    }
}
```

8.2.7 expm1f

```
float expm1f(float x)
```

功能介绍

计算单精度浮点的 $e^x - 1$ 的取值，其中 e 为底数。最大ULP为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	$e^x - 1$ 的取值，其中： - 当x小于-103.97时，返回 -1。 - 当x取 ± 0时，返回 0。 - 当x大于88.72时，返回 ∞ 。 - 当x取 $-\infty$ 时，返回-1。 - 当x取 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= expm1f(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= expm1f(a[i]) + 1;    // 原位操作
    }
}
```

8.2.8 isnan

```
int isnan(float x)
```

功能介绍

判断单精度浮点数x是否为NaN（非数字值）。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	如果给定参数x为非数字值NaN则返回1，否则返回0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= isnan(a[i]) + 1;
    }
}
```

使用示例2


```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= isnan(a[i]) + 1;    // 原位操作
    }
}
```

8.2.9 log1pf

```
float log1pf(float x)
```

功能介绍

计算单精度浮点x+1的对数，以 e 为底数。最大ULP 为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	对数计算结果，其中： - 当x小于-1时，返回 NaN 。 - 当x取-1时，返回 ∞ 。 - 当x取 NaN 或者0时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= log1pf(a[i]) + 1;
    }
}
```

```
    }  
}
```

使用示例2

```
__device__ void test(float *a, int len)  
{  
    for (int i = 0; i < len; i++) {  
        a[i]= log1pf(a[i]) + 1;    // 原位操作  
    }  
}
```

8.2.10 log2f

```
float log2f(float x)
```

功能介绍

计算单精度浮点的对数，以 2 为底数。最大ULP 为 5。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	对数计算结果，其中： - 当x小于0时，返回 NaN 。 - 当x取0时，返回 $-\infty$ 。 - 当x取 ∞ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= log2f(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= log2f(a[i]) + 1;    // 原位操作
    }
}
```

8.2.11 logf

```
float logf(float x)
```

功能介绍

计算单精度浮点的对数，以 e 为底数。最大ULP 为 4。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	对数计算结果，其中： - 当x小于0或者为 NaN 时，返回 NaN 。 - 当x取0时，返回 $-\infty$ 。 - 当x为 $+\infty$ 时，返回 $+\infty$ 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i] = logf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i] = logf(a[i]) + 1;    // 原位操作
    }
}
```

8.2.12 sigmoidf

```
float sigmoidf(float x)
```

功能介绍

计算单精度浮点的sigmoid函数的取值。

- 参数x的取值范围在 $(-10, 10)$ 时：最大ULP 为 3。
- 参数x取其他范围时：最大绝对误差 $5e^{-05}$ 。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	sigmoid函数的取值，其中： - 当x小于-10时，返回0。 - 当x大于10时，返回1。 - 当x取0时，返回0.5。 - 当x取 $+\infty$ 时，返回1。 - 当x取 $-\infty$ 时，返回0。 - 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= sigmoidf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= sigmoidf(a[i]) + 1;    // 原位操作
    }
}
```

8.2.13 sinf

```
float sinf(float x)
```

功能介绍

计算单精度浮点的正弦函数的取值。最大ULP为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	正弦函数的取值，其中： - 当x取 ± 0 时，返回 ± 0。 - 当x取 $\pm\infty$ 时，返回 NaN 。 - 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i] = sinf(a[i]);
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i] = sinf(a[i]);    // 原位操作
    }
}
```

8.2.14 cosf

```
float cosf(float x)
```

功能介绍

计算单精度浮点的余弦函数的取值。最大ULP为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	余弦函数的取值，其中： - 当x取 ± 0 时，返回1。 - 当x取 $\pm\infty$ 时，返回 NaN 。 - 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i] = cosf(a[i]);
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i] = cosf(a[i]);    // 原位操作
    }
}
```

8.2.15 tanf

```
float tanf(float x)
```

功能介绍

计算单精度浮点的正切函数的取值。最大ULP 为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	正切函数的取值，其中： - 当x取 ± 0 时，返回 ± 0。 - 当x取 $\pm\infty$ 时，返回 NaN 。 - 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= tanf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= tanf(a[i]) + 1;    // 原位操作
    }
}
```

8.2.16 tanhf

```
float tanhf(float x)
```


功能介绍

计算单精度浮点的双曲正切函数的取值， $\frac{e^{2x}-1}{e^{2x}+1}$ 。最大ULP 为 32。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	双曲正切函数的取值，其中： - 当x取 ± 0时，返回+0。 - 当x小于-7.6时，返回-1。 - 当x大于7.6时，返回1。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= tanhf(a[i]);
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= tanhf(a[i]);    // 原位操作
    }
}
```

8.2.17 atanf

```
float atanf(float x)
```

功能介绍

计算单精度浮点的反正切函数的取值。最大ULP 为 2。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	反正切函数的取值，其中： - 当x取 ± 0 时，返回 ± 0。 - 当x取 $\pm \infty$ 时，返回 $\pm \frac{\pi}{2}$。 - 当x取 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= atanf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= atanf(a[i]) + 1;    // 原位操作
    }
}
```

8.2.18 fmaf

```
float fmaf(float x, float y, float z)
```

功能介绍

对给定单精度浮点x和y进行乘法操作后，与z进行相加。最大ULP为0。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。
z	输入参数。

返回值

返回值类型	说明
float	$(x * y) + z$的取值。 - 当x、y、z中出现NaN时，返回NaN。 - 当x、y、z中出现∞时，返回∞。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, float *c, float *d, int len)
{
    for (int i = 0; i < len; i++) {
        d[i] = fmaf(a[i], b[i], c[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, float *b, float *c, int len)
{
```

```
for (int i = 0; i < len; i++) {
    a[i]= fmaf(a[i], b[i], c[i]) + 1;    // 原位操作
}
}
```

8.2.19 fmaxf

```
float fmaxf(float x, float y)
```

功能介绍

获取单精度浮点数x和y中的较大值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。

返回值

返回值类型	说明
float	对于给定单精度浮点数x和y： - 其中某个参数取 NaN 时，返回 NaN 。 - 其中某个参数取 $-\infty$ 时，返回另一个参数。 - 当其中某个参数取 ∞ 时，返回 ∞ 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, float *c, int len)
{
    for (int i = 0; i < len; i++) {
        c[i]= fmaxf(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fmaxf(a[i], b[i]) + 1;    // 原位操作
    }
}
```

8.2.20 fminf

```
float fminf(float x, float y)
```

功能介绍

获取单精度浮点数x和y中的较小值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。

返回值

返回值类型	说明
float	对于给定单精度浮点数x和y： - 其中某个参数取 NaN 时，返回 NaN 。 - 其中某个参数取 $-\infty$ 时，返回 $-\infty$ 。 - 其中某个参数取 ∞ 时，返回另一个参数。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, float *c, int len)
{
    for (int i = 0; i < len; i++) {
        c[i]= fminf(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fminf(a[i], b[i]) + 1;    // 原位操作
    }
}
```

8.2.21 fmodf

```
float fmodf(float x, float y)
```

功能介绍

计算两个单精度浮点数的余数，其中x为被除数，y为除数。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。

返回值

返回值类型	说明
float	两个单精度浮点数的余数。 - 当x取 NaN 或者 $\pm\infty$ 时，返回 NaN 。 - 当y取 $\pm\infty$ 时，返回x。 - 当y取 NaN 时，返回 NaN 。 - 当y取0时，返回 NaN 。 - 当x取0时，返回0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, float *c, int len)
{
    for (int i = 0; i < len; i++) {
        c[i]= fmodf(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fmodf(a[i], b[i]) + 1;    // 原位操作
    }
}
```

8.2.22 powf

```
float powf(float x, float y)
```

功能介绍

计算单精度浮点数x的y次幂，即 x^y 的取值。最大ULP 为 3。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。

返回值

返回值类型	说明
float	x^y的取值。 - 当输入数据x为$+\infty$时： - 如果输入数据y小于0，返回0。 - 如果输入数据y大于0，返回$+\infty$。 - 当输入数据x为$-\infty$时： - 如果输入数据y小于0，返回0。 - 如果输入数据y大于0且小数位不为0时，返回$+\infty$。 - 如果输入数据y大于0且小数位为0时，返回$-\infty$。 - 当输入数据y为$+\infty$时： - 如果输入数据x属于$(-1, 1)$，返回0。 - 如果输入数据x的绝对值为1，返回1。 - 如果输入数据x不属于以上两种情况，返回$+\infty$。 - 当输入数据y为$-\infty$时： - 如果输入数据x属于$(-1, 1)$，返回$+\infty$。 - 如果输入数据x的绝对值为1，返回1。 - 如果输入数据x不属于以上两种情况，返回0。 - 如果x或者y中某一个数为NaN时，返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, float *c, int len)
{
```



```
    for (int i = 0; i < len; i++) {
        c[i]= powf(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= powf(a[i], b[i]) + 1;    // 原位操作
    }
}
```

8.2.23 roundf

```
float roundf(float x)
```

功能介绍

将单精度浮点数x舍入为浮点形式表示的最近整数值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	返回浮点数的四舍五入值。 - 当x位于两个整数的正中间，则向远离0的方向舍入。 - 当x取 $\pm\infty$ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= roundf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= roundf(a[i]) + 1;    // 原位操作
    }
}
```

8.3 双精度标量接口

8.3.1 ceil

```
double ceilf(double x)
```

功能介绍

计算不小于给定双精度浮点数x的最小整数。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	大于等于参数x的最小整数。 - 假设y为小于x的最大整数，当x位于(y, y + 1]的时候，返回y + 1。 - 当x取 $\pm\infty$ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= ceil(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= ceil(a[i]) + 1;    // 原位操作
    }
}
```

8.3.2 floor

```
double floor(double x)
```

功能介绍

获取不大于给定双精度浮点数x的最大整数值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	小于等于x的最大整数值。 - 假设y为小于等于x的最大整数，当x位于[y, y + 1)的时候，返回y。 - 当x取 $\pm\infty$ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= floor(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= floor(a[i]) + 1;    // 原位操作
    }
}
```

8.3.3 fabs

```
double fabs(double x)
```

功能介绍

计算双精度浮点数x的绝对值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	x的绝对值。 - 当x小于0时，返回-x。 - 当x大于等于0时，返回x。 - 当x取-0时，返回0。 - 当x取 ∞ 或者 NaN 时，返回x。 - 当x取 $-\infty$ 时，返回 ∞ 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= fabs(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fabs(a[i]) + 1;    // 原位操作
    }
}
```

8.3.4 sqrt

```
double sqrt(double x)
```

功能介绍

计算双精度浮点数x的平方根函数。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	对数计算结果，其中： - 当x小于0时，返回 NaN 。 - 当x取0时，返回0。 - 当x等于 ∞ 时，返回 ∞ 。 - 当x等于 $-\infty$ 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= sqrt(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= sqrt(a[i]);    // 原位操作
    }
}
```

8.3.5 erf

```
double erf(double x)
```

功能介绍

计算双精度浮点的高斯误差函数的取值， $\frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$ 。最大ULP 为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	高斯误差函数的取值，其中： - 当x取 ± 0时，返回+0。 - 当x小于等于-6时，返回-1。 - 当x大于等于6时，返回1。 - 当x取 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= erf(a[i]) + 1;
    }
}
```

```
    }  
}
```

使用示例2

```
__device__ void test(double *a, int len)  
{  
    for (int i = 0; i < len; i++) {  
        a[i]= erf(a[i]) + 1;    // 原位操作  
    }  
}
```

8.3.6 exp

```
double exp(double x)
```

功能介绍

计算双精度浮点的 e^x 的取值，其中 e 为底数。最大ULP 为 149。

参数解释

参数名称	说明
x	输入参数，作为e的指数。

返回值

返回值类型	说明
double	e^x 的取值，其中： - 当x小于-103.97时，返回 0。 - 当x取 ± 0时，返回 1。 - 当x大于88.72时，返回 ∞ 。 - 当x取 NaN 或者 ∞ 时，返回x。 - 当x取 $-\infty$ 时，返回0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i] = exp(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i] = exp(a[i]) + 1;    // 原位操作
    }
}
```

8.3.7 expm1

```
double expm1(float x)
```

功能介绍

计算双精度浮点的 $e^x - 1$ 的取值，其中 e 为底数。最大ULP为1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
float	$e^x - 1$ 的取值，其中： - 当x小于-103.97时，返回 -1。 - 当x取 ± 0时，返回 0。 - 当x大于88.72时，返回 ∞ 。 - 当x取 $-\infty$ 时，返回-1。 - 当x取 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(float *a, float *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= expm1(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(float *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= expm1(a[i]) + 1;    // 原位操作
    }
}
```

8.3.8 isnan

```
int isnan(double x)
```

功能介绍

判断双精度浮点数x是否为NaN（非数字值）。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	如果给定参数x为非数字值NaN则返回1，否则返回0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= isnan(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= isnan(a[i]) + 1;    // 原位操作
    }
}
```

8.3.9 log1p

```
double log1p(double x)
```

功能介绍

计算双精度浮点x+1的对数，以 e 为底数。最大ULP 为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	对数计算结果，其中： - 当x小于-1时，返回 NaN 。 - 当x取-1时，返回 ∞ 。 - 当x取 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= log1pf(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= log1pf(a[i]) + 1;    // 原位操作
    }
}
```

8.3.10 log2

```
double log2(double x)
```

功能介绍

计算双精度浮点的对数，以 2 为底数。最大ULP 为 6。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	对数计算结果，其中： - 当x小于0时，返回 NaN 。 - 当x取0时，返回 $-\infty$ 。 - 当x取 ∞ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= log2(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= log2(a[i]) + 1;    // 原位操作
    }
}
```

8.3.11 log

```
double log(double x)
```

功能介绍

计算双精度浮点的对数，以 e 为底数。最大ULP 为 4。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	对数计算结果，其中： - 当x小于0或者为 NaN 时，返回 NaN 。 - 当x取0时，返回 $-\infty$ 。 - 当x为 $+\infty$ 时，返回 $+\infty$ 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= log(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= log(a[i]) + 1;    // 原位操作
    }
}
```

8.3.12 sigmoid

```
double sigmoid(double x)
```

功能介绍

计算双精度浮点的sigmoid函数的取值。

- 参数x的取值范围在（-10，10）时：最大ULP 为 124。
- 参数x取其他范围时：最大绝对误差 $5e^{-05}$ 。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	sigmoid函数的取值，其中： • 当x小于-10时，返回0。 • 当x大于10时，返回1。 • 当x取0时，返回0.5。 • 当x取 $+\infty$ 时，返回1。 • 当x取 $-\infty$ 时，返回0。 • 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= sigmoid(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= sigmoid(a[i]) + 1;    // 原位操作
    }
}
```

8.3.13 sin

```
double sin(double x)
```

功能介绍

计算双精度浮点的正弦函数的取值。最大ULP 为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	正弦函数的取值，其中： - 当x取 ± 0 时，返回 ± 0。 - 当x取 $\pm\infty$ 时，返回 NaN 。 - 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= sin(a[i]) + 1;
    }
}
```



```
    }  
}
```

使用示例2

```
__device__ void test(double *a, int len)  
{  
    for (int i = 0; i < len; i++) {  
        a[i]= sin(a[i]) + 1;    // 原位操作  
    }  
}
```

8.3.14 cos

```
double cos(double x)
```

功能介绍

计算双精度浮点的余弦函数的取值。最大ULP 为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	余弦函数的取值，其中： - 当x取 ± 0 时，返回1。 - 当x取 $\pm\infty$ 时，返回 NaN 。 - 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= cos(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= cos(a[i]) + 1;    // 原位操作
    }
}
```

8.3.15 tan

```
double tan(double x)
```

功能介绍

计算双精度浮点的正切函数的取值。最大ULP 为 1。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	正切函数的取值，其中： - 当x取 ± 0 时，返回 ± 0。 - 当x取 $\pm\infty$ 时，返回 NaN 。 - 当x取 NaN 时，返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= tan(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= tan(a[i]) + 1;    // 原位操作
    }
}
```

8.3.16 tanh

```
double tanh(double x)
```

功能介绍

计算双精度浮点的双曲正切函数的取值， $\frac{e^{2x}-1}{e^{2x}+1}$ 。最大ULP 为 3。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	双曲正切函数的取值，其中： - 当x取 ± 0时，返回+0。 - 当x小于-22时，返回-1。 - 当x大于22时，返回1。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= tanh(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= tanh(a[i]) + 1;    // 原位操作
    }
}
```

8.3.17 atan

```
double atan(double x)
```

功能介绍

计算双精度浮点的反正切函数的取值。最大ULP 为 2。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	反正切函数的取值，其中： - 当x取 ± 0 时，返回 ± 0。 - 当x取 $\pm\infty$ 时，返回 $\pm\frac{\pi}{2}$。 - 当x取 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i]= atan(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= atan(a[i]) + 1;    // 原位操作
    }
}
```

8.3.18 fma

```
double fma(double x, double y, double z)
```

功能介绍

对给定双精度浮点x和y进行乘法操作后，与z进行相加。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。
z	输入参数。

返回值

返回值类型	说明
double	$(x * y) + z$的取值。 - 当x、y、z中出现 NaN 时，返回 NaN 。 - 当x、y、z中出现 ∞ 时，返回 ∞ 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, double *c, double *d, int len)
{
    for (int i = 0; i < len; i++) {
        d[i]= fma(a[i], b[i], c[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, double *b, double *c, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fma(a[i], b[i], c[i]) + 1;    // 原位操作
    }
}
```

8.3.19 fmax

```
double fmax(double x, double y)
```

功能介绍

获取双精度浮点数x和y中的较大值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

参数名称	说明
y	输入参数。

返回值

返回值类型	说明
double	对于给定双精度浮点数x和y： - 其中某个参数取 NaN 时，返回 NaN 。 - 其中某个参数取 $-\infty$ 时，返回另一个参数。 - 当其中某个参数取 ∞ 时，返回 ∞ 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, double *c, int len)
{
    for (int i = 0; i < len; i++) {
        c[i]= fmax(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fmax(a[i], b[i]) + 1;    // 原位操作
    }
}
```

8.3.20 fmin

```
double fmin(double x, double y)
```

功能介绍

获取双精度浮点数x和y中的较小值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。

返回值

返回值类型	说明
double	对于给定双精度浮点数x和y： - 其中某个参数取 NaN 时，返回 NaN 。 - 其中某个参数取 $-\infty$ 时，返回 $-\infty$ 。 - 其中某个参数取 ∞ 时，返回另一个参数。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, double *c, int len)
{
    for (int i = 0; i < len; i++) {
        c[i]= fmin(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= fmin(a[i], b[i]) + 1;    // 原位操作
    }
}
```

8.3.21 fmod


```
double fmodf(double x, double y)
```

功能介绍

计算两个双精度浮点数的余数，其中x为被除数，y为除数。最大ULP为0。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。

返回值

返回值类型	说明
double	两个双精度浮点数的余数。 - 当x取NaN或者±∞时，返回NaN。 - 当y取±∞时，返回x。 - 当y取NaN时，返回NaN。 - 当y取0时，返回NaN。 - 当x取0时，返回0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, double *c, int len)
{
    for (int i = 0; i < len; i++) {
        c[i] = fmod(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        a[i] = fmod(a[i], b[i]) + 1;    // 原位操作
    }
}
```

8.3.22 pow

```
double pow(double x, double y)
```

功能介绍

计算双精度浮点数x的y次幂，即 x^y 的取值。最大ULP为1。

参数解释

参数名称	说明
x	输入参数。
y	输入参数。

返回值

返回值类型	说明
double	x^y的取值。 - 当输入数据x为$+\infty$时： - 如果输入数据y小于0，返回0。 - 如果输入数据y大于0，返回$+\infty$。 - 当输入数据x为$-\infty$时： - 如果输入数据y小于0，返回0。 - 如果输入数据y大于0且小数位不为0时，返回$+\infty$。 - 如果输入数据y大于0且小数位为0时，返回$-\infty$。 - 当输入数据y为$+\infty$时： - 如果输入数据x属于$(-1, 1)$，返回0。 - 如果输入数据x的绝对值为1，返回1。 - 如果输入数据x不属于以上两种情况，返回$+\infty$。 - 当输入数据y为$-\infty$时： - 如果输入数据x属于$(-1, 1)$，返回$+\infty$。 - 如果输入数据x的绝对值为1，返回1。 - 如果输入数据x不属于以上两种情况，返回0。 - 如果x或者y中某一个数为NaN时，返回NaN。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, double *c, int len)
{
    for (int i = 0; i < len; i++) {
        c[i]= pow(a[i], b[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, double *b, int len)
{
```

```
for (int i = 0; i < len; i++) {
    a[i] = pow(a[i], b[i]) + 1;    // 原位操作
}
```

8.3.23 round

```
double round(double x)
```

功能介绍

将双精度浮点数x舍入为浮点形式表示的最近整数值。最大ULP 为 0。

参数解释

参数名称	说明
x	输入参数。

返回值

返回值类型	说明
double	返回浮点数的四舍五入值。 - 当x位于两个整数的正中间，则向远离0的方向舍入。 - 当x取 $\pm\infty$ 或者 NaN 时，返回x。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ void test(double *a, double *b, int len)
{
    for (int i = 0; i < len; i++) {
        b[i] = round(a[i]) + 1;
    }
}
```

使用示例2

```
__device__ void test(double *a, int len)
{
    for (int i = 0; i < len; i++) {
        a[i]= round(a[i]) + 1;    // 原位操作
    }
}
```

8.4 向量接口

8.4.1 simd_exp

```
floatv16 simd_exp(floatv16 x)
```

功能介绍

计算 e 为底数的指数向量，一次计算16个float。最大ULP为 5。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	e 为底数的指数向量，其中： - 输入向量的某个值小于等于-87.33，该位置对应返回0。 - 输入向量的某个值大于等于88.72，该位置对应返回 $+\infty$ 。 - 输入向量的某个值是 $+\infty$ ，该位置对应返回 $+\infty$ 。 - 输入向量的某个值是 $-\infty$ ，该位置对应返回0。 - 输入向量的某个值是 NaN ，该位置对应返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_exp_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -87.33 + (176.0 / 16) * i;
    }
    simd_load(input, input_arr);

    result = simd_exp(input);
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_exp_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -87.33 + (176.0 / 16) * i;
    }
    simd_load(input, input_arr);

    input = simd_exp(input);    // 原位操作
    return input;
}
```

8.4.2 simd_exp128

```
void simd_exp128(float *x)
```

功能介绍

计算 e 为底数的指数数组，一次计算128个float，计算结果写回输入参数对应位置。最大ULP为175。

- 数组中元素的取值范围在 $(-19.4, 88.72)$ 时：最大ULP 为 175。
- 数组中元素取其他范围时：最大绝对误差 $3.725e^{-09}$ 。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：以 e 为底数的指数数组，写回输入参数对应位置，其中： - 输入数组的某个值小于等于-87.33，该位置对应返回0。 - 输入数组的某个值大于等于88.72，该位置对应返回 $+\infty$。 - 输入数组的某个值是 $+\infty$，该位置对应返回 $+\infty$。 - 输入数组的某个值是 $-\infty$，该位置对应返回0。 - 输入数组的某个值是 NaN，该位置对应返回 NaN。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_exp128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0};    // 64B aligned

    for (int i = 0; i < 128; i++) {
        input_arr[i] = -87.33 + (176.0 / 128) * i;
    }
    simd_exp128(input_arr); // 原位操作
}
```

8.4.3 simd_log

```
floatv16 simd_log(floatv16 x)
```

功能介绍

计算 e 为底数的对数向量，一次计算16个float。最大ULP为5。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	e 为底数的对数向量，其中： - 输入向量的某个值小于0或者为 NaN 时，该位置对应返回 NaN 。 - 输入向量的某个值等于0，该位置对应返回 $-\infty$ 。 - 输入向量的某个值是 $+\infty$ 时，该位置对应返回 $+\infty$ 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_log_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = i;
    }
    simd_load(input, input_arr);

    result = simd_log(input);
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_log_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = i;
    }
    simd_load(input, input_arr);

    input = simd_log(input);    // 原位操作
}
```



```
    return input;
}
```

8.4.4 simd_log128

```
void simd_log128(float *x)
```

功能介绍

计算 e 为底数的对数数组，一次计算128个float，计算结果写回输入参数对应位置。最大ULP为5。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：以 e 为底数的对数数组，写回输入参数对应位置，其中： - 输入数组的某个值小于0或者为 NaN 时，该位置对应返回 NaN。 - 输入数组的某个值等于0，该位置对应返回 $-\infty$。 - 输入向量的某个值是 $+\infty$ 时，该位置对应返回 $+\infty$。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_log128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0}; // 64B aligned

    for (int i = 0; i < 128; i++) {
        input_arr[i] = i;
    }
    simd_log128(input_arr); // 原位操作
}
```

8.4.5 simd_tanh

```
floatv16 simd_tanh(floatv16 x)
```

功能介绍

计算双曲正切函数向量，一次计算16个float。最大ULP为32。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	双曲正切函数向量，其中： - 输入向量的某个值小于等于-6.93，该位置对应返回-1。 - 输入向量的某个值大于等于6.93，该位置对应返回1。 - 输入向量的某个值是 $\pm\infty$，该位置对应返回 ± 1。 - 输入向量的某个值是 NaN，该位置对应返回 NaN。

注意事项

该接口支持原位操作。

使用示例

```
__device__ floatv16 simd_tanh_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -6.93 + (13.86 / 16) * i;
    }
    simd_load(input, input_arr);

    result = simd_tanh(input);
}
```

```
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_tanh_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -6.93 + (13.86 / 16) * i;
    }
    simd_load(input, input_arr);

    input = simd_tanh(input);    // 原位操作
    return input;
}
```

8.4.6 simd_tanh128

```
void simd_tanh128(float *x)
```

功能介绍

计算双曲正切函数数组，一次计算128个float，计算结果写回输入参数对应位置。最大ULP为32。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：双曲正切函数数组，写回输入参数对应位置，其中： - 输入数组的某个值小于等于-6.93，该位置对应返回-1。 - 输入数组的某个值大于等于6.93，该位置对应返回1。 - 输入数组的某个值是 $\pm\infty$，该位置对应返回 ± 1。 - 输入数组的某个值是 NaN，该位置对应返回 NaN。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_tanh128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0}; // 64B aligned

    for (int i = 0; i < 128; i++) {
        input_arr[i] = -6.93 + (13.86 / 128) * i;
    }
    simd_tanh128(input_arr); // 原位操作
}
```

8.4.7 simd_sigmoid

```
floatv16 simd_sigmoid(floatv16 x)
```

功能介绍

计算sigmoid函数向量，一次计算16个float。

- 参数x中元素的取值范围在 $(-10, 10)$ 时：最大ULP 为 8。
- 参数x中元素取其他范围时：最大绝对误差 $9.54e^{-07}$ ，最大相对误差 $9.54e^{-07}$ 。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	sigmoid函数向量，其中： - 输入向量的某个值小于等于-13.86，该位置对应返回0。 - 输入向量的某个值大于等于13.86，该位置对应返回1。 - 输入向量的某个值是 $+\infty$，该位置对应返回1。 - 输入向量的某个值是 $-\infty$，该位置对应返回0。 - 输入向量的某个值是 NaN，该位置对应返回 NaN。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_sigmoid_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -13.86 + (28.0 / 16) * i;
    }
    simd_load(input, input_arr);

    result = simd_sigmoid(input);
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_sigmoid_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -13.86 + (28.0 / 16) * i;
    }
    simd_load(input, input_arr);

    input = simd_sigmoid(input);    // 原位操作
    return input;
}
```

8.4.8 simd_sigmoid128

```
void simd_sigmoid128(float *x)
```

功能介绍

计算sigmoid函数数组，一次计算128个float，计算结果写回输入参数对应位置。

- 数组中元素的取值范围在 (-10, 10) 时：最大ULP 为 8。
- 数组中元素取其他范围时：最大绝对误差 $9.54e^{-07}$ ，最大相对误差 $9.54e^{-07}$ 。

参数解释

参数名称	说明
x	- 输入向量：首地址64字节对齐，float个数为128。 - 计算结果：sigmoid函数数组，写回输入参数对应位置，其中： - 输入数组的某个值小于等于-13.86，该位置对应返回0。 - 输入数组的某个值大于等于13.86，该位置对应返回1。 - 输入数组的某个值是 $+\infty$，该位置对应返回1。 - 输入数组的某个值是 $-\infty$，该位置对应返回0。 - 输入数组的某个值是 NaN，该位置对应返回 NaN。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_sigmoid128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0};    // 64B aligned

    for (int i = 0; i < 128; i++) {
        input_arr[i] = -13.86 + (28.0 / 128) * i;
    }
    simd_sigmoid128(input_arr);  // 原位操作
}
```

8.4.9 simd_sqrt

```
floatv16 simd_sqrt(floatv16 x)
```

功能介绍

计算平方根函数向量，一次计算16个float。最大ULP为2。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	平方根函数向量，其中 - 输入向量的某个值为负数，该位置对应返回 NaN 。 - 输入向量的某个值为0，该位置对应返回0。 - 输入向量的某个值是 $+\infty$ ，该位置对应返回 $+\infty$ 。 - 输入向量的某个值是 $-\infty$ ，该位置对应返回 NaN 。 - 输入向量的某个值是 NaN ，该位置对应返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_sqrt_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = (1000.0 / 16) * i;
    }
    simd_load(input, input_arr);

    result = simd_sqrt(input);
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_sqrt_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = (1000.0 / 16) * i;
    }
}
```

```
    }  
    simd_load(input, input_arr);  
  
    input = simd_sqrt(input);    // 原位操作  
    return input;  
}
```

8.4.10 simd_inv

```
floatv16 simd_inv(floatv16 x)
```

功能介绍

计算倒数向量，一次计算16个float。最大ULP为1。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	倒数向量，其中 - 输入向量的某个值是 $\pm\infty$，该位置对应返回 ± 0。 - 输入向量的某个值是 NaN，该位置对应返回 NaN。

注意事项

该接口支持原位操作。

使用示例

```
__device__ floatv16 simd_inv_test()  
{  
    float input_arr[16] = {0};  
    floatv16 input;  
    floatv16 result;  
  
    for (int i = 0; i < 16; i++) {  
        input_arr[i] = (1000.0 / 16) * i;  
    }  
    simd_load(input, input_arr);  
}
```



```
    result = simd_inv(input);  
    return result;  
}
```

使用示例2

```
__device__ floatv16 simd_inv_test()  
{  
    float input_arr[16] = {0};  
    floatv16 input;  
  
    for (int i = 0; i < 16; i++) {  
        input_arr[i] = (1000.0 / 16) * i;  
    }  
    simd_load(input, input_arr);  
  
    input = simd_inv(input);    // 原位操作  
    return input;  
}
```

8.4.11 simd_erf

```
floatv16 simd_erf(floatv16 x)
```

功能介绍

计算高斯误差函数向量，一次计算16个float。最大ULP为17。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	高斯误差函数向量，其中： - 输入向量的某个值小于-3.7，该位置对应返回-1。 - 输入向量的某个值大于3.7，该位置对应返回1。 - 输入向量的某个值是 $\pm\infty$，该位置对应返回 ± 1。 - 输入向量的某个值是 NaN，该位置对应返回 NaN。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_erf_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -3.7 + (8.0 / 16) * i;
    }
    simd_load(input, input_arr);

    result = simd_erf(input);
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_erf_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -3.7 + (8.0 / 16) * i;
    }
    simd_load(input, input_arr);

    input = simd_erf(input); // 原位操作
    return input;
}
```

8.4.12 simd_erf128

```
void simd_erf128(float *x)
```

功能介绍

计算高斯误差函数数组，一次计算128个float，计算结果写回输入参数对应位置。最大ULP为17。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：高斯误差函数数组，写回输入参数对应位置，其中： - 输入数组的某个值小于-3.7，该位置对应返回-1。 - 输入数组的某个值大于3.7，该位置对应返回1。 - 输入数组的某个值是 $\pm\infty$，该位置对应返回 ± 1。 - 输入数组的某个值是 NaN，该位置对应返回 NaN。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_erf128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0};    // 64B aligned

    for (int i = 0; i < 128; i++) {
        input_arr[i] = -13.86 + (28.0 / 128) * i;
    }
    simd_erf128(input_arr); // 原位操作
}
```

8.4.13 simd_atan

```
floatv16 simd_atan(floatv16 x)
```

功能介绍

计算反正切函数向量，一次计算16个float。最大ULP为5。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	反正切函数向量，其中： - 输入向量的某个值等于 ± 0，该位置对应返回 ± 0。 - 输入向量的某个值等于 $\pm\infty$，该位置对应返回 $\pm\frac{\pi}{2}$。 - 输入向量的某个值是 NaN，该位置对应返回 NaN。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_atan_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -6.93 + (13.86 / 16) * i;
    }
    simd_load(input, input_arr);

    result = simd_atan(input);
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_atan_test()
{
    float input_arr[16] = {0};
```

```
floatv16 input;

for (int i = 0; i < 16; i++) {
    input_arr[i] = -6.93 + (13.86 / 16) * i;
}
simd_load(input, input_arr);

input = simd_atan(input);    // 原位操作
return input;
}
```

8.4.14 simd_atan128

```
void simd_atan128(float *x)
```

功能介绍

计算反正切函数数组，一次计算128个float，计算结果写回输入参数对应位置。最大ULP为5。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：反正切函数数组，写回输入参数对应位置，其中： - 输入数组的某个值等于 ± 0，该位置对应返回 ± 0。 - 输入数组的某个值等于 $\pm\infty$，该位置对应返回 $\pm\frac{\pi}{2}$。 - 输入数组的某个值是 NaN，该位置对应返回 NaN。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_atan128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0};    // 64B aligned

    for (int i = 0; i < 128; i++) {
```

```
        input_arr[i] = -6.93 + (13.86 / 128) * i;
    }
    simd_atan128(input_arr); // 原位操作
}
```

8.4.15 simd_log2

```
floatv16 simd_log2(floatv16 x)
```

功能介绍

计算 2 为底数的对数向量，一次计算16个float。最大ULP为6。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	2 为底数的对数向量，其中： • 输入向量的某个值小于0，该位置对应返回 NaN 。 • 输入向量的某个值等于0，该位置对应返回 $-\infty$ 。 • 输入向量的某个值是 $+\infty$，该位置对应返回 $+\infty$ 。 • 输入向量的某个值是 $-\infty$，该位置对应返回 NaN 。 • 输入向量的某个值是 NaN，该位置对应返回 NaN 。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_log_test()
{
    float input_arr[16] = {0};
    floatv16 input;
```

```
floatv16 result;

for (int i = 0; i < 16; i++) {
    input_arr[i] = i;
}
simd_load(input, input_arr);

result = simd_log2(input);
return result;
}
```

使用示例2

```
__device__ floatv16 simd_log_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = i;
    }
    simd_load(input, input_arr);

    input = simd_log2(input);    // 原位操作
    return input;
}
```

8.4.16 simd_log2_128

```
void simd_log2_128(float *x)
```

功能介绍

计算 2 为底数的对数数组，一次计算128个float，计算结果写回输入参数对应位置。最大ULP为6。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：以 2 为底数的对数数组，写回输入参数对应位置，其中： - 输入数组的某个值小于0，该位置对应返回 NaN 。 - 输入数组的某个值等于0，该位置对应返回 $-\infty$ 。 - 输入数组的某个值是 $+\infty$ ，该位置对应返回 $+\infty$ 。 - 输入数组的某个值是 $-\infty$ ，该位置对应返回 NaN 。 - 输入数组的某个值是 NaN ，该位置对应返回 NaN 。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_log2_128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0};    // 64B aligned

    for (int i = 0; i < 128; i++) {
        input_arr[i] = i;
    }
    simd_log2_128(input_arr); // 原位操作
}
```

8.4.17 simd_sin

```
floatv16 simd_sin(floatv16 x)
```

功能介绍

计算正弦函数向量，一次计算16个float。最大ULP为2。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	正弦函数向量，其中： - 输入向量的某个值等于 $\pm\infty$，该位置对应返回 NaN。 - 输入向量的某个值等于 NaN，该位置对应返回 NaN。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_sin_test()
{
    float input_arr[16] = {0};
    floatv16 input;
    floatv16 result;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -6.93 + (13.86 / 16) * i;
    }
    simd_load(input, input_arr);

    result = simd_sin(input);
    return result;
}
```

使用示例2

```
__device__ floatv16 simd_sin_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -6.93 + (13.86 / 16) * i;
    }
    simd_load(input, input_arr);

    input = simd_sin(input);    // 原位操作
    return input;
}
```

8.4.18 simd_sin128

```
void simd_sin128(float *x)
```

功能介绍

计算正弦函数数组，一次计算128个float，计算结果写回输入参数对应位置。最大ULP为2。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：正弦函数数组，写回输入参数对应位置，其中： - 输入数组的某个值等于 $\pm\infty$，该位置对应返回 NaN。 - 输入数组的某个值等于 NaN，该位置对应返回 NaN。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_sin128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0};    // 64字节对齐
}
```

```
for (int i = 0; i < 128; i++) {  
    input_arr[i] = -6.93 + (13.86 / 128) * i;  
}  
simd_sin128(input_arr); // 原位操作  
}
```

8.4.19 simd_cos

```
floatv16 simd_cos(floatv16 x)
```

功能介绍

计算余弦函数向量，一次计算16个float。

- 参数x中元素的取值范围在（-2000，2000）时：最大ULP为 2。
- 参数x中元素取其他范围时：最大绝对误差 $1.28e^{-07}$ 。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	余弦函数向量，其中： - 输入向量的某个值等于 $\pm\infty$，该位置对应返回 NaN。 - 输入向量的某个值等于 NaN，该位置对应返回 NaN。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_cos_test()  
{  
    float input_arr[16] = {0};  
    floatv16 input;  
    floatv16 result;
```

```
for (int i = 0; i < 16; i++) {
    input_arr[i] = -6.93 + (13.86 / 16) * i;
}
simd_load(input, input_arr);

result = simd_cos(input);
return result;
}
```

使用示例2

```
__device__ floatv16 simd_cos_test()
{
    float input_arr[16] = {0};
    floatv16 input;

    for (int i = 0; i < 16; i++) {
        input_arr[i] = -6.93 + (13.86 / 16) * i;
    }
    simd_load(input, input_arr);

    input = simd_cos(input);    // 原位操作
    return input;
}
```

8.4.20 simd_cos128

```
void simd_cos128(float *x)
```

功能介绍

计算余弦函数数组，一次计算128个float，计算结果写回输入参数对应位置。

- 数组中元素的取值范围在 (-2000, 2000) 时：最大ULP 为 2。
- 数组中元素取其他范围时：最大绝对误差 $1.28e^{-07}$ 。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：余弦函数数组，写回输入参数对应位置，其中： - 输入数组的某个值等于 $\pm\infty$，该位置对应返回 NaN 。 - 输入数组的某个值等于 NaN，该位置对应返回 NaN 。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_cos128_test()
{
    float input_arr[128] __attribute__((aligned(64))) = {0};    // 64字节对齐

    for (int i = 0; i < 128; i++) {
        input_arr[i] = -6.93 + (13.86 / 128) * i;
    }
    simd_cos128(input_arr);  // 原位操作
}
```

8.4.21 simd_round

```
floatv16 simd_round(floatv16 x)
```

功能介绍

按照就近舍入的方式，将输入向量的每个浮点元素，舍入为浮点形式的整数值。中间值向远离零的方向舍入，例如：输入2.5将输出3.0，输入-2.5将输出-3.0。

一次计算16个float。最大ULP 为 0。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	就近舍入的结果向量，其中： - 输入向量的某个值等于 $\pm\infty$，该位置对应返回 $\pm\infty$。 - 输入向量的某个值等于 NaN，该位置对应返回 NaN。 - 输入向量的某个值等于 ± 0，该位置对应返回 ± 0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_round_test()
{
    floatv16 input = {0.0, -0.0, INFINITY, -INFINITY, NAN, -NAN, 2.1, 2.5,
                      2.9, 2.0, -2.1, -2.5, -2.9, -2.0, 0.5, -0.5};

    floatv16 result = simd_round(input);
    // 运算结果为 {0.0, -0.0, inf, -inf, NaN, -NaN, 2.0, 3.0,
    //              3.0, 2.0, -2.0, -3.0, -3.0, -2.0, 1.0, -1.0}

    return result;
}
```

使用示例2

```
__device__ floatv16 simd_round_test()
{
    floatv16 input = {0.0, -0.0, INFINITY, -INFINITY, NAN, -NAN, 2.1, 2.5,
                      2.9, 2.0, -2.1, -2.5, -2.9, -2.0, 0.5, -0.5};

    input = simd_round(input);    // 原位操作
    // 运算结果为 {0.0, -0.0, inf, -inf, NaN, -NaN, 2.0, 3.0,
    //              3.0, 2.0, -2.0, -3.0, -3.0, -2.0, 1.0, -1.0}

    return input;
}
```

8.4.22 simd_round128

```
void simd_round128(float *x)
```

功能介绍

按照就近舍入的方式，将输入数组的每个浮点元素，舍入为浮点形式的整数值。中间值向远离零的方向舍入，例如：输入2.5将输出3.0，输入-2.5将输出-3.0。

一次计算128个float。最大ULP 为 0。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：就近舍入的结果数组，写回输入参数对应位置，其中： - 输入数组的某个值等于 $\pm\infty$，该位置对应返回 $\pm\infty$。 - 输入数组的某个值等于 NaN，该位置对应返回 NaN。 - 输入数组的某个值等于 ± 0，该位置对应返回 ± 0。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_round128_test()
{
    float *input = (float *)malloc(128 * sizeof(float));
    for (size_t i = 0; i != 128; ++i) {
        input[i] = -20.f + (40.f / 128.f) * i;
    }

    simd_round128(input);
    free(input);
}
```

8.4.23 simd_trunc

```
floatv16 simd_trunc(floatv16 x)
```

功能介绍

按照向零舍入的方式，将输入向量的每个浮点元素，舍入为浮点形式的整数值。例如：输入2.5将输出2.0，输入-2.5将输出-2.0。

一次计算16个float。最大ULP 为 0。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	向零舍入的结果向量，其中： - 输入向量的某个值等于 $\pm\infty$，该位置对应返回 $\pm\infty$。 - 输入向量的某个值等于 NaN，该位置对应返回 NaN。 - 输入向量的某个值等于 ± 0，该位置对应返回 ± 0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_trunc_test()
{
    floatv16 input = {0.0, -0.0, INFINITY, -INFINITY, NAN, -NAN, 2.1, 2.5,
                      2.9, 2.0, -2.1, -2.5, -2.9, -2.0, 0.5, -0.5};

    floatv16 result = simd_trunc(input);
    // 运算结果为 {0.0, -0.0, inf, -inf, NaN, -NaN, 2.0, 2.0,
    //              2.0, 2.0, -2.0, -2.0, -2.0, -2.0, 0.0, -0.0}

    return result;
}
```


使用示例2

```
__device__ floatv16 simd_trunc_test()
{
    floatv16 input = {0.0, -0.0, INFINITY, -INFINITY, NAN, -NAN, 2.1, 2.5,
                     2.9, 2.0, -2.1, -2.5, -2.9, -2.0, 0.5, -0.5};

    input = simd_trunc(input);    // 原位操作
    // 运算结果为 {0.0, -0.0, inf, -inf, NaN, -NaN, 2.0, 2.0,
    //             2.0, 2.0, -2.0, -2.0, -2.0, -2.0, 0.0, -0.0}

    return input;
}
```

8.4.24 simd_trunc128

```
void simd_trunc128(float *x)
```

功能介绍

按照向零舍入的方式，将输入数组的每个浮点元素，舍入为浮点形式的整数值。例如：输入2.5将输出2.0，输入-2.5将输出-2.0。

一次计算128个float。最大ULP 为 0。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：向零舍入的结果数组，写回输入参数对应位置，其中： - 输入数组的某个值等于 $\pm\infty$，该位置对应返回 $\pm\infty$。 - 输入数组的某个值等于 NaN，该位置对应返回 NaN。 - 输入数组的某个值等于 ± 0，该位置对应返回 ± 0。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_trunc128_test()
{
    float *input = (float *)malloc(128 * sizeof(float));
    for (size_t i = 0; i != 128; ++i) {
        input[i] = -20.f + (40.f / 128.f) * i;
    }

    simd_trunc128(input);
    free(input);
}
```

8.4.25 simd_floor

```
floatv16 simd_floor(floatv16 x)
```

功能介绍

按照向负无穷舍入的方式，将输入向量的每个浮点元素，舍入为浮点形式的整数值。例如：输入2.5将输出2.0，输入-2.5将输出-3.0。

一次计算16个float。最大ULP 为 0。

参数解释

参数名称	说明
x	输入向量，float个数为16。

返回值

返回值类型	说明
floatv16	向负无穷舍入的结果向量，其中： - 输入向量的某个值等于 $\pm\infty$，该位置对应返回 $\pm\infty$。 - 输入向量的某个值等于 NaN，该位置对应返回 NaN。 - 输入向量的某个值等于 ± 0，该位置对应返回 ± 0。

注意事项

该接口支持原位操作。

使用示例1

```
__device__ floatv16 simd_floor_test()
{
    floatv16 input = {0.0, -0.0, INFINITY, -INFINITY, NAN, -NAN, 2.1, 2.5,
                     2.9, 2.0, -2.1, -2.5, -2.9, -2.0, 0.5, -0.5};

    floatv16 result = simd_floor(input);
    // 运算结果为 {0.0, -0.0, inf, -inf, NaN, -NaN, 2.0, 2.0,
    //             2.0, 2.0, -3.0, -3.0, -3.0, -2.0, 0.0, -1.0}

    return result;
}
```

使用示例2

```
__device__ floatv16 simd_floor_test()
{
    floatv16 input = {0.0, -0.0, INFINITY, -INFINITY, NAN, -NAN, 2.1, 2.5,
                     2.9, 2.0, -2.1, -2.5, -2.9, -2.0, 0.5, -0.5};

    input = simd_floor(input);    // 原位操作

    // 运算结果为 {0.0, -0.0, inf, -inf, NaN, -NaN, 2.0, 2.0,
    //             2.0, 2.0, -3.0, -3.0, -3.0, -2.0, 0.0, -1.0}

    return input;
}
```

8.4.26 simd_floor128

```
void simd_floor128(float *x)
```

功能介绍

按照向负无穷舍入的方式，将输入数组的每个浮点元素，舍入为浮点形式的整数值。例如：输入2.5将输出2.0，输入-2.5将输出-3.0。

一次计算128个float。最大ULP为0。

参数解释

参数名称	说明
x	- 输入数组：首地址64字节对齐，float个数为128。 - 计算结果：向负无穷舍入的结果数组，写回输入参数对应位置，其中： - 输入数组的某个值等于 $\pm\infty$，该位置对应返回 $\pm\infty$。 - 输入数组的某个值等于 NaN，该位置对应返回 NaN。 - 输入数组的某个值等于 ± 0，该位置对应返回 ± 0。

返回值

无。

注意事项

该接口支持原位操作。

使用示例

```
__device__ void simd_floor128_test()
{
    float *input = (float *)malloc(128 * sizeof(float));
    for (size_t i = 0; i != 128; ++i) {
        input[i] = -20.f + (40.f / 128.f) * i;
    }

    simd_floor128(input);
    free(input);
}
```

9. 高层次函数接口

9.1 概述

高层次函数接口提供：数学函数、激活函数和规约函数的批量计算接口，通过这些内置接口，您可以高效地完成数据的批量计算。您在使用高层次函数接口时，需要引入scl.h头文件和sdad::scl命名空间。

9.2 数学函数

9.2.1 batch_tanh

```
template<typename T>
void batch_tanh(T *dst, const T *src, int len)
```

功能介绍

批量计算输入数组src的双曲正切值，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。 说明： - 当输入数组src中某个数据为±0时，对应位置返回±0。 - 当输入数组src中某个数据小于等于-7.6时，对应位置返回-1。 - 当输入数组src中某个数据大于等于7.6时，对应位置返回1。 - 当输入数组src中某个数据为NaN时，对应位置返回NaN。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdaa::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len)
{
    sdaa::scl::batch_tanh(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len)
{
    sdaa::scl::batch_tanh(dst, src, len);
}
```

9.2.2 batch_sin

```
template<typename T>
void batch_sin(T *dst, const T *src, int len)
```

功能介绍

批量计算输入数组src的正弦值，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。 ? 说明: • 当输入数组src中某个数据为0时，对应位置返回0。 • 当输入数组src中某个数据为±∞时，对应位置返回NaN。 • 当输入数组src中某个数据为NaN时，对应位置返回NaN。

参数名称	说明
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len){
    sdAA::scl::batch_sin(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len){
    sdAA::scl::batch_sin(dst, src, len);
}
```

9.2.3 batch_cos

```
template<typename T>
void batch_cos(T *dst, const T *src, int len)
```

功能介绍

批量计算输入数组src的余弦值，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。 ? 说明: - 当输入数组src中某个数据为0时，对应位置返回1。 - 当输入数组src中某个数据为±∞时，对应位置返回NaN。 - 当输入数组src中某个数据为NaN时，对应位置返回NaN。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len){
    sdAA::scl::batch_cos(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len){
    sdAA::scl::batch_cos(dst, src, len);
}
```

9.2.4 batch_log


```
template<typename T>
void batch_log(T *dst, const T *src, int len)
```

功能介绍

批量计算输入数组src的对数，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。  说明: - 当输入数组src中某个小于0或者NaN时，对应位置返回NaN。 - 当输入数组src中某个数据为0时，对应位置返回-∞。 - 当输入数组src中某个数据为+∞时，对应位置返回+∞。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdaa::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len){
    sdaa::scl::batch_log(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len){
    sdaa::scl::batch_log(dst, src, len);
}
```

9.2.5 batch_pow

```
template<typename T>
void batch_pow(T *dst, const T *src1, const T *src2, int len)
```

功能介绍

批量计算输入数组src1的src2次幂，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。  说明： - 当输入数组src1中某个数为$+\infty$时： - 如果src2中对应位置的数小于0，对应位置返回0。 - 如果src2中对应位置的数大于0，对应位置返回$+\infty$。 - 当输入数组src1中某个数为$-\infty$时： - 如果src2中对应位置的数小于0，对应位置返回0。 - 如果src2中对应位置的数大于0且小数位不为0时，对应位置返回$+\infty$。 - 如果src2中对应位置的数大于0且小数位为0时，对应位置返回$-\infty$。 - 当输入数组src2中某个数为$+\infty$时： - 如果src1中对应位置的数属于$(-1, 1)$，对应位置返回0。 - 如果src1中对应位置的数的绝对值为1，对应位置返回NaN。 - 如果src1中对应位置的数不属于以上两种情况，对应位置返回$+\infty$。 - 当输入数组src2中某个数为$-\infty$时： - 如果src1中对应位置的数属于$(-1, 1)$，对应位置返回$+\infty$。 - 如果src1中对应位置的数的绝对值为1，对应位置返回NaN。 - 如果src1中对应位置的数不属于以上两种情况，对应位置返回0。 - 如果src1或者src2中某个数为NaN时，对应位置返回NaN。
src1	输入数组。
src2	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。

- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src1, float *src2, int len)
{
    sdAA::scl::batch_pow(dst, src1, src2, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src1, half *src2, int len)
{
    sdAA::scl::batch_pow(dst, src1, src2, len);
}
```

9.2.6 batch_floor

```
template<typename T>
void batch_floor(T *dst, const T *src, int len)
```

功能介绍

批量计算小于等于输入数组src的最大整数值，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。 ? 说明： • 假设x为小于等于输入数组src中某个元素的最大整数，对应位置返回x。 • 当输入数组src中某个数据为±∞或者NaN时，对应位置返回src对应位置的数据。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len)
{
    sdAA::scl::batch_floor(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len)
{
    sdAA::scl::batch_floor(dst, src, len);
}
```

9.2.7 batch_ceil

```
template<typename T>
void batch_ceil(T *dst, const T *src, int len)
```

功能介绍

批量计算大于等于输入数组src的最小整数值，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。 说明： - 假设x为小于输入数组src中某个元素的最大整数，对应位置返回 $x + 1$。 - 当输入数组src中某个数据为$\pm\infty$或者NaN时，对应位置返回src对应位置的数据。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len)
{
    sdAA::scl::batch_ceil(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len)
{
    sdAA::scl::batch_ceil(dst, src, len);
}
```

9.2.8 batch_axpy

```
template<typename T>
void batch_axpy(T *dst, const T *src1, const T *src2, T alpha, int len)
```

功能介绍

批量计算输入数组src1中的数据分别与alpha相乘后的结果，与输入数组src2中数据对应相加，并将结果存储到dst数组中：

$$dst[i] = src1[i] * alpha + src2[i]$$

参数解释

参数名称	说明
dst	输出数组。  说明： - 输入数组src1或者src2中某个元素为NaN，或者输入系数alpha为NaN时，对应位置输出NaN。 - 输入数组src1或者src2中某个元素为±∞，或者输入系数alpha为±∞时，对应位置输出±∞。
src1	输入数组。
src2	输入数组。
alpha	输入系数。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src1, float *src2, T alpha, int len)
{
    sdaa::scl::batch_axpy(dst, src1, src2, alpha, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src1, half *src2, T alpha, int len)
{
    sdaa::scl::batch_axpy(dst, src1, src2, alpha, len);
}
```

9.2.9 batch_exp

```
template<typename T>
void batch_exp(T *dst, const T *src, int len)
```

功能介绍

批量计算输入数组src的自然指数值，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。 说明： - 当输入数组src中某个数据为0时，对应位置返回1。 - 当输入数组src中某个数据小于-103.97时，对应位置返回0。 - 当输入数组src中某个数据大于88.72时，对应位置返回∞。 - 当输入数组src中某个数据为+∞或者NaN时，对应位置返回src中对应位置的数据。 - 当输入数组src中某个数据为-∞时，对应位置返回0。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len)
{
    sdAA::scl::batch_exp(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len)
{
    sdAA::scl::batch_exp(dst, src, len);
}
```

9.2.10 batch_xor

```
template<typename T>
void batch_xor(T *dst, const T *src1, const T *src2, int len)
```

功能介绍

批量计算输入数组src1与src2的按位异或值，并将计算结果存储到dst数组中。

参数解释

参数名称	说明
dst	输出数组。
src1	输入数组。

参数名称	说明
src2	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持int数据类型和short数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(int *dst, int *src1, int *src2, int len)
{
    sdAA::scl::batch_xor(dst, src1, src2, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(short *dst, short *src1, short *src2, int len)
{
    sdAA::scl::batch_pow(dst, src1, src2, len);
}
```

9.3 激活函数

9.3.1 batch_gelu

```
template<typename T>
void batch_gelu(T *dst, const T *src, int len)
```

功能介绍

批量对输入数组src进行gelu激活计算：

$$dst[i] = 0.5 * src[i] * (1 + \tanh(\sqrt{\frac{2}{\pi}}(src[i] + 0.044715 * src[i]^3)))。$$

参数解释

参数名称	说明
dst	输出数组。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdAA::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len)
{
    sdAA::scl::batch_gelu(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len)
{
    sdAA::scl::batch_gelu(dst, src, len);
}
```

9.3.2 batch_sigmoid

```
template<typename T>
void batch_sigmoid(T *dst, const T *src, int len)
```

功能介绍

批量计算输入数组src的sigmoid值，并将计算结果存储到dst数组中。

- 输入数组src的某个元素的取值范围在（-10，10）时：最大ULP 为 124。
- 输入数组src的某个元素不在（-10，10）范围内时：最大绝对误差 $4.54e^{-05}$ ，最大相对误差 $4.54e^{-05}$ 。

参数解释

参数名称	说明
dst	输出数组。  说明： • 当输入数组src中某个数据小于-10时，对应位置返回0。 • 当输入数组src中某个数据大于10时，对应位置返回1。 • 当输入数组src中某个数据为0时，对应位置返回0.5。 • 当输入数组src中某个数据为NaN时，对应位置返回NaN。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdaa::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len)
{
    sdaa::scl::batch_sigmoid(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len)
{
    sdaa::scl::batch_sigmoid(dst, src, len);
}
```

9.4 规约函数

9.4.1 sum

```
template<typename T>
void sum(float *dst, const T *src, int len)
```

功能介绍

对输入数组src中所有元素进行累计求和，将累加值保存到dst[0]中输出。

参数解释

参数名称	说明
dst	输出数组。 说明： dst[0]位置保存规约计算后的结果，其他索引位置维持原始数据。
src	输入数组。
len	数组的长度。

返回值

无。

注意事项

- 需要手动包含scl.h头文件。
- 该接口仅支持float数据类型和half数据类型。
- 需要引入sdaa::scl命名空间。

使用示例1

```
#include <scl.h>

__device__ void test(float *dst, float *src, int len)
{
    sdAA::scl::sum(dst, src, len);
}
```

使用示例2

```
#include <scl.h>

__device__ void test(half *dst, half *src, int len)
{
    sdAA::scl::sum(dst, src, len);
}
```

10. 程序编译

10.1 命令行选项

TecoCC是一个能自动为编译的目标文件传递适当的头文件及库文件的编译器驱动程序。TecoCC提供了简单且常用的命令行选项，可以简化C++及SDAA C代码的编译过程并调用不同编译阶段的工具集合执行编译流程。

选项	功能
-h/--help	显示SDAA C支持的所有命令行选项的概要帮助信息，包括命令行选项的基本用法和参数列表。
-flto	开启LTO优化模式。
-v	显示编译器的版本。
--verbose	显示编译器内部编译各过程的命令行的详细信息。
--sdaa-compile-host-device	编译SDAA C的Host端及Device端代码（默认）。
--sdaa-device-lib-path=<value>	指定SDAA C的Device端库的搜索路径。
--sdaa-device-lib=<value>	指定链接SDAA C的Device端.bc库。
--sdaa-static-device-lib=<value>	指定链接SDAA C的Device端静态库。
--sdaa-host-only	编译SDAA C代码时，指定只编译Host端代码。
--sdaa-device-only	编译SDAA C代码时，指定只编译Device端代码。
--sdaa-link	链接SDAA C的clang-offload-bundler捆绑包。
-sdaa-new-launch-api	指定开启SDAADriver v0.6.0及之后至SDAADriver v0.9.0之前版本Kernel函数的Launch接口。 ? 说明： 适用于TecoCC v1.3.0及之后至v1.3.4之前的版本。

选项	功能
<code>-fno-sdaa-new-launch-api</code>	指定开启SDAADriver v0.6.0之前的版本Kernel函数的Launch接口。  说明： 适用于TecoCC v1.3.0及之后的版本。
<code>-use-new-sdaa-driver</code>	指定开启SDAADriver v0.9.0及之后的版本Kernel函数的Launch接口。  说明： 适用于TecoCC v1.3.4及之后的版本。
<code>-fno-use-new-sdaa-driver</code>	指定开启SDAADriver0.6.0及之后至SDAADriver0.9.0之前版本Kernel函数的Launch接口。  说明： 适用于TecoCC v1.3.4及之后的版本。
<code>--sdaa-path=<value></code>	指定SDAA C库的安装路径。
<code>-sdaa-static-lib</code>	在开启LTO优化时，该选项使得TecoCC将输入的.o文件打包成Host端静态库.a文件。
<code>-nosdaainc</code>	SDAA C的Device端编译时，不添加默认添加的头文件。
<code>-nosdaalib</code>	SDAA C的Device端链接时，不链接默认链接的Device端库文件。
<code>--stack-on-global</code>	SDAA C将函数栈从SPM存储空间切换到Global存储空间。  说明： - 若对代码分阶段编译，则所有阶段必须都开启该选项。 - 同一可执行文件下的所有函数，必须同步选择是否开启，部分开启而部分不开启会导致未定义错误。
<code>--sdaa-arch=pcx_100</code>	用于生成《PCX编程指南》相关代码。

10.2 编译流程

10.2.1 LTO模式

LTO模式（Link Time Optimization）即链接时优化模式，也称模块间优化，是指在链接时进行的跨模块的优化，推迟代码生成到链接时，可以进行内联或全局优化。

在打开-flto选项（可参考[支持的命令行选项](#)）时，即开启LTO优化模式，编译流程如下（详细的编译代码，可参考[编译示例1](#)）：

1. 剥离源文件中的Host端代码，经过预处理、编译阶段生成Host端的.bc文件。
2. 剥离Device端的代码，经过预处理、编译阶段生成Device端的.bc文件。
3. （可选）如果通过-c进行编译，编译器会将源文件中Host端和Device端的.bc文件打包成一个.o文件（实质是.bc文件）。此时若做链接，会将编译生成的.o文件拆解成.bc文件，再进行步骤4的处理。
4. 链接器将多个Device端的.bc文件链接成一个Device端的二进制文件，将这个二进制文件处理成Host端的.bc文件，再与其他Host端的.bc文件一起做链接，最终生成Host端的二进制文件。

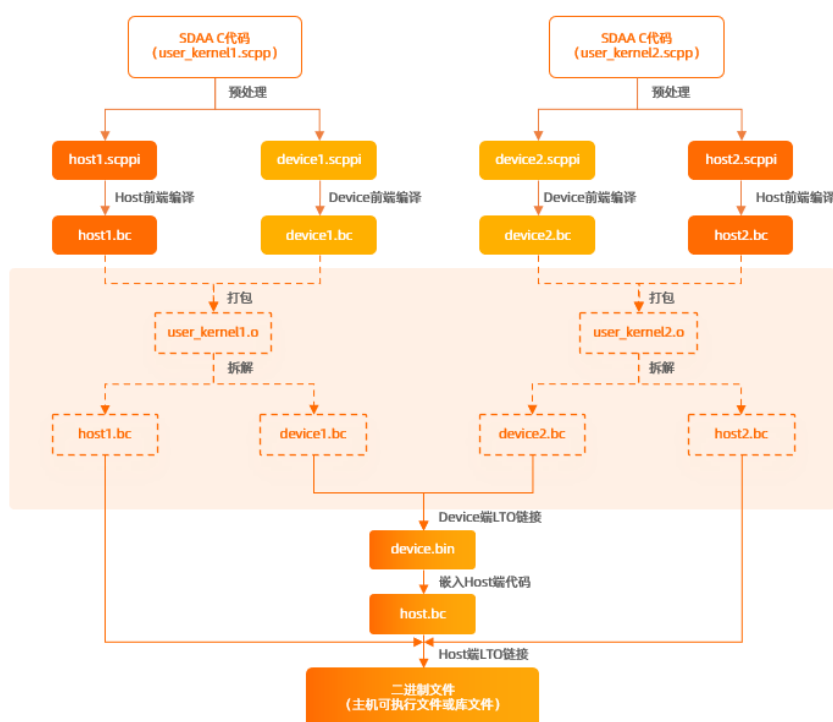


图1 LTO模式的编译流程

10.2.2 非LTO模式

非LTO模式是普通的编译模式，编译生成的.o文件是Linux下GNU兼容的.o文件，可以使用GCC链接器进行链接，编译流程如下（详细的编译代码，可参考[编译示例1](#)）：

1. 剥离将源文件中的Device端程序，经过预处理、编译、链接阶段，最终生成Device端的二进制文件。
2. 剥离Host端的代码，在Host端的代码进行前端编译时嵌入Device端的二进制，经过Host端的编译、汇编、链接阶段最终生成Host端的二进制文件。

? 说明:

Host端和Device端的编译流程均分为预处理、编译、汇编、链接四个阶段。

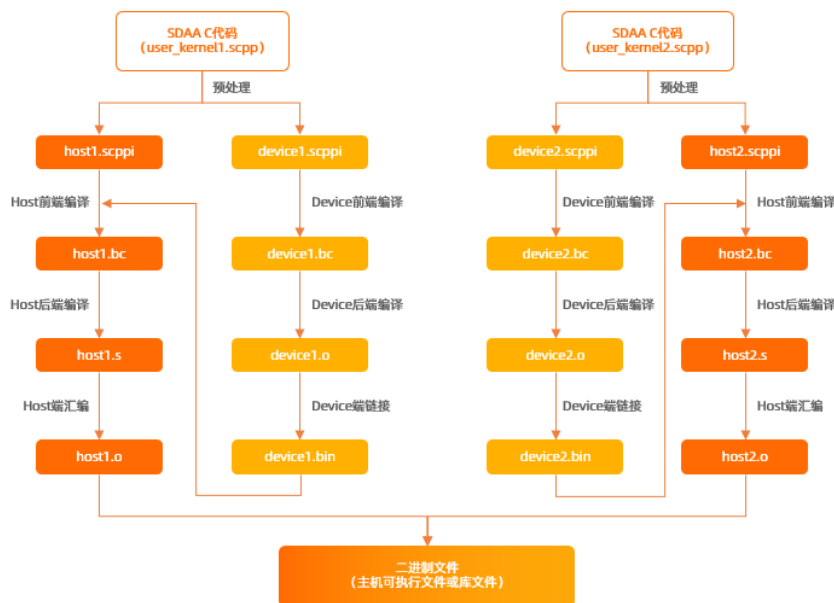


图1 非LTO模式的编译流程

10.3 编译方式

10.3.1 概述

TecoCC有三种编译方式：动态链接、静态链接和直接生成可执行文件。

- 动态链接和静态链接：主要应用于main函数单独存放于.cpp文件中的场景，可参考[编译示例2](#)。
- 直接生成可执行文件的使用场景没有限制，main函数存放于某个.scpp文件或单独存放于.cpp文件的场景均适用。

10.3.2 LTO模式

10.3.2.1 动态链接方式

1.执行以下命令，将.scpp文件编译成.o文件，详细的编译代码，可参考[编译示例2](#)。

```
# 将源文件编译成.o文件
# 使用-flto选项开启LTO优化模式
# 这里的.o文件实际上是.bc文件
# -On指定优化度的级别，您可以根据实际需求使用-00/1/2/3，数值越大，级别越高。
tecocc user_kernel1.scpp -O2 -flto -c -o user_kernel1.o
tecocc user_kernel2.scpp -O2 -flto -c -o user_kernel2.o
```

2.执行以下命令，将多个.o文件链接成动态库。

```
# 使用tecocc将所有的.o文件链接成动态库
# --sdaa-link选项指定链接SDAA C的clang-offload-bundler捆绑包
tecocc user_kernel1.o user_kernel2.o -O2 -flto -fPIC -shared --sdaa-link -o tmp.so
```

3.执行以下命令，使用main.cpp链接动态库生成可执行文件。

```
# 使用main.cpp链接动态库生成可执行文件
tecocc main.cpp tmp.so -O2 -o dynamic_lib.out
```

10.3.2.2 静态链接方式

1.执行以下命令，将.scpp文件编译成.o文件，详细的编译代码，可参考[编译示例2](#)。

```
# 将源文件编译成.o文件
tecocc user_kernel1.scpp -O2 -flto -c -o user_kernel1.o
tecocc user_kernel2.scpp -O2 -flto -c -o user_kernel2.o
```

2.执行以下命令，将多个.o文件链接成静态库。

```
# 使用tecocc将所有的.o文件链接成静态库
# -sdaa-static-lib选项指定Host端采用static模式进行链接
tecocc user_kernel1.o user_kernel2.o -O2 -flto --sdaa-link -shared -sdaa-static-lib -o tmp.a
```

3.执行以下命令，使用main.cpp链接静态库生成可执行文件。

```
# 使用main.cpp链接静态库生成可执行文件
tecocc main.cpp tmp.a -O2 -o static_lib.out
```

10.3.2.3 直接生成可执行文件方式

执行以下命令，将多个源文件直接编译生成可执行文件，详细的编译代码，可参考[编译示例2](#)。

```
# 将所有源文件一起编译生成可执行文件
tecocc user_kernel1.scpp user_kernel2.scpp main.cpp -flto --sdaa-link -O2 -fPIC -o a.out
```

? 说明：

仅LTO模式支持调用一个不在当前.scpp文件定义的__device__函数。

10.3.3 非LTO模式

10.3.3.1 动态链接方式

1.执行以下命令，将.scpp文件编译成.o文件，详细的编译代码，可参考[编译示例2](#)。

```
# 将源文件编译成.o文件
tecocc user_kernel1.scpp -O2 -fPIC -c -o user_kernel1.o
tecocc user_kernel2.scpp -O2 -fPIC -c -o user_kernel2.o
```

2.执行以下命令，将多个.o文件链接成动态库。

```
# 将所有.o文件链接成动态库
tecocc user_kernel1.o user_kernel2.o -O2 -fPIC -shared -o tmp.so
```

3.执行以下命令，使用main.cpp链接动态库生成可执行文件。

```
# 使用main.cpp链接动态库生成可执行文件
tecocc main.cpp tmp.so -O2 -o dynamic_lib.out
```

10.3.3.2 静态链接方式

1.执行以下命令，将.scpp文件编译成.o文件，详细的编译代码，可参考[编译示例2](#)。

```
# 将源文件编译成.o文件
tecocc user_kernel1.scpp -O2 -fPIC -c -o user_kernel1.o
tecocc user_kernel2.scpp -O2 -fPIC -c -o user_kernel2.o
```

2.执行以下命令，将多个.o文件链接成静态库。

```
# 使用AR打包工具将所有的.o文件链接成静态库
ar -rcs tmp.a user_kernel1.o user_kernel2.o
```

3.执行以下命令，使用main.cpp链接静态库生成可执行文件

```
# 使用main.cpp链接静态库生成可执行文件
tecocc main.cpp tmp.a -O2 -o static_lib.out
```

10.3.3.3 直接生成可执行文件方式

执行以下命令，将多个源文件直接编译生成可执行文件，详细的编译代码，可参考[编译示例2](#)。

```
# 将所有源文件一起编译生成可执行文件
tecocc user_kernel1.scpp user_kernel2.scpp main.cpp -O2 -o a.out
```

10.4 编译示例

10.4.1 编译示例1

使用场景：main函数存放于某个.scpp文件中。

user_kernel1.scpp

```
#include <stdio.h>
#include <stdlib.h>

// 定义一个Device函数，该函数仅在LTO模式下才支持被其他的.scpp文件中定义的global或device函数调用
__device__ void print()
{
    printf("this is device function in %s\n", __FILE__);
}

__global__ void test1(int a, double b)
{
    print();
    printf("this is %s! CORE NUM: %lu\n", __FILE__, threadIdx);
    // 打印Host端传过来的整型数据及浮点型数据
    printf("a = %d, b = %f\n", a, b);
    printf("\n");
}

int main1()
{
    // 指定Kernel函数运行在Device端的0号计算核心阵列SPA上
    sdAASetDevice(0);
    // 调用Kernel函数
    test1<<<1>>>(1, 3.0);
    // 等待threadDim个计算核心SPE完成之前的所有请求任务
    sdAADeviceSynchronize();
    return 0;
}
```

user_kernel2.scpp

```
#include <stdio.h>
#include <stdlib.h>

extern int main1();
// 仅LTO模式支持该用法
// extern __device__ void print();

typedef struct {
    int b;
    int c;
} st1;
st1 a = {1, 2};

__global__ void test2(int a, double b, st1 c)
{
    // 仅LTO模式支持该用法
    // print();
}
```

```
printf("this is %s! CORE NUM: %lu\n", __FILE__, threadIdx);
// 打印Host端传过来的整型数据、浮点型数据及结构体数据
printf("a = %d, b = %f, c.b = %d, c.c = %d\n", a, b, c.b, c.c);
printf("\n");
}

int main2()
{
    sdaaSetDevice(0);
    test2<<<1>>>(66, 8.0, a);
    sdaaDeviceSynchronize();
    return 0;
}

int main()
{
    // 调用user_kernel1.scpp文件中定义的main1函数
    main1();
    // 调用当前文件中定义的main2函数
    main2();
}
```

10.4.2 编译示例2

使用场景：main函数单独存放在.cpp文件中。

user_kernel1.scpp

```

#include <stdio.h>
#include <stdlib.h>

// 定义一个Device函数, 该函数仅在LTO模式下才支持被别的文件中定义的global或device函数调用
__device__ void print()
{
    printf("this is device function in %s\n", __FILE__);
}

__global__ void test1(int a, double b)
{
    print();
    printf("this is %s! CORE NUM: %lu\n", __FILE__, threadIdx);
    // 打印Host端传过来的整型数据及浮点型数据
    printf("a = %d, b = %f\n", a, b);
    printf("\n");
}

int main1()
{
    // 指定Kernel函数运行在Device端0号计算核心阵列SPA上
    sdaaSetDevice(0);
    // 调用Kernel函数
    test1<<<1>>>(1, 3.0);
    // 等待threadDim个计算核心SPE完成之前的所有请求任务
    sdaaDeviceSynchronize();
    return 0;
}

```

user_kernel2.scpp

```

#include <stdio.h>
#include <stdlib.h>

// 仅LTO模式支持该用法: extern __device__ void print();

typedef struct {
    int b;
    int c;
} st1;
st1 a = {1,2};

__global__ void test2(int a, double b, st1 c)
{
    // 仅LTO模式支持该用法: print();

    printf("this is %s! CORE NUM: %lu\n", __FILE__, threadIdx);
    // 打印Host端传过来的整型数据、浮点型数据及结构体数据
    printf("a = %d, b = %f, c.b = %d, c.c = %d\n", a, b, c.b, c.c);
    printf("\n");
}

int main2()
{
    sdaaSetDevice(0);
}

```

```
test2<<<1>>>(66, 8.0, a);  
sdaaDeviceSynchronize();  
return 0;  
}
```

main.cpp

```
extern int main1();  
extern int main2();  
  
int main()  
{  
    // 调用user_kernel1.scpp文件中定义的main1函数  
    main1();  
    // 调用user_kernel2.scpp文件中定义的main2函数  
    main2();  
}
```


11. 程序调试

11.1 概述

程序调试是应用程序开发人员进行软件开发过程中必要的纠错手段。一款优秀的程序调试工具可以协助软件开发人员快速定位代码在编译、运行时发生错误的原因和位置，提升软件的开发效率。

SDAA C为应用程序开发人员提供了配套的基本软件调试手段：

- 通过打印运行时日志的方式，追踪打印计算核心SPE中代码的运行状态，从而定位出问题发生的具体原因，可参考[打印运行时日志](#)。
- 通过TecoGDB工具调试计算核心SPE的代码，从而定位出问题发生的具体原因，可参考[TecoGDB调试工具](#)。
- 通过终止设备端程序的方式，在设备端的库函数代码中检测到无法修复的问题时，及时停止设备端程序的运行，可参考[终止设备端程序](#)。
- 通过断言的方式，检测代码中存在的错误，出现错误时打印对应的文件行号等详细信息，可参考[断言](#)。

11.2 打印运行时日志

打印运行时日志是SDAA C提供的一个代码追踪手段。用户可以通过调用[printf](#)接口打印目标变量的运行时状态。

使用方法

- `printf` 接口是SDAA C提供的内置接口，用户使用时无需引入头文件。
- `printf` 接口参数与C/C++提供的printf接口相同。
- 设置环境变量[export SDAA_SYNC_PRINT=1](#)，可以按照SPE编号从小到大的顺序进行打印。

使用示例

以打印参数的相加结果为例，示例代码如下：

```
__device__ void sum_func(int a, int b)
{
    int g_sum = a + b;
    printf("threadIdx: %lu, sum: %d\n", threadIdx, g_sum);
}

__global__ void bar()
{
    sum_func(1, 2);
}

int main()
{
    sdaaSetDevice(0);
    bar<<<1>>>>();
}
```

```
    sdaaDeviceSynchronize();  
  
    return 0;  
}
```

- 不设置环境变量 `SDAA_SYNC_PRINT` 或者值默认为0时，会按照SPE的执行顺序进行打印。执行后打印结果如下：

```
threadIdx: 4, sum: 3  
threadIdx: 0, sum: 3  
threadIdx: 6, sum: 3  
threadIdx: 7, sum: 3  
.....
```

- 设置环境变量 `SDAA_SYNC_PRINT=1`，按照SPE编号从小到大的顺序进行打印。执行后打印结果如下：

```
threadIdx: 0, sum: 3  
threadIdx: 1, sum: 3  
threadIdx: 2, sum: 3  
threadIdx: 3, sum: 3  
.....
```

11.3 TecoGDB调试工具

TecoGDB (Tecororigin GNU Debugger) 是基于GNU GDB开发的运行在Linux系统上的源码级调试工具，可以调试运行在太初AI加速卡上的程序，提供对被调试程序的运行控制和信息打印功能。该工具提供真实物理环境下的调试功能，避免仿真调试和真实环境运行的误差，方便您准确定位问题。

更多内容，可参考《TecoGDB命令行工具用户手册》。

11.4 终止设备端程序

终止设备端程序是SDAA C提供的协助调试手段。如果您在设备端编写的库函数代码中检查到无法修复的问题，可以通过`abort`接口直接终止设备端的程序运行。通过 `abort` 函数结束运行时：

- 如果设置了环境变量`SDAA_ENABLE_COREDUMP_ON_EXCEPTION`（详情请参考《SDAARuntime用户手册》的`SDAA_ENABLE_COREDUMP_ON_EXCEPTION`章节）：通知主机端生成包含错误相关上下文的Core Dump文件，并且终止主机端的代码执行。
- 如果没有设置该环境变量`SDAA_ENABLE_COREDUMP_ON_EXCEPTION`（详情请参考《SDAARuntime用户手册》的`SDAA_ENABLE_COREDUMP_ON_EXCEPTION`章节）：继续执行主机端的代码，后续通过执行`sdaaDeviceSynchronize`接口，获取对应的abort错误码。

使用方法

- `abort` 接口是SDAA C提供的内置接口，使用时无需引入头文件。
- `abort` 接口的参数与C/C++提供的 `abort` 接口和用法相同。

使用示例

以检测除数不为0作为示例，当检查到除数为0时，执行 `abort` 终止SPE后续程序的运行。

```
__device__ int g_sum = 0;
__device__ int my_div(int a, int b)
{
    if (b == 0) {
        // 如果除数为0, 终止SPE后续程序的运行
        abort();
    }
    return a / b;
}

__device__ void my_compute(int a, int b)
{
    int div_result = my_div(a, b);
    g_sum = (a - b) * div_result;
}

__global__ void bar()
{
    my_compute(1, 2);
}

int main()
{
    sdAASetDevice(0);
    bar<<<1>>>>();
    sdAADeviceSynchronize();

    return 0;
}
```

11.5 断言

断言（assert）是SDAA C提供的检测错误手段。您可以通过调用`assert`接口检查代码中是否出现错误，出现错误后会和C/C++提供的 `assert` 一样，打印对应错误所在的文件行号等信息，之后终止设备端程序的运行。通过 `assert` 函数终止设备端程序运行时：

- 如果设置了环境变量SDAA_ENABLE_COREDUMP_ON_EXCEPTION（详情请参考《SDAARuntime用户手册》的SDAA_ENABLE_COREDUMP_ON_EXCEPTION章节）：通知主机端生成包含错误相关上下文的Core Dump文件，并且终止主机端的代码执行。
- 如果没有设置环境变量SDAA_ENABLE_COREDUMP_ON_EXCEPTION（详情请参考《SDAARuntime用户手册》的SDAA_ENABLE_COREDUMP_ON_EXCEPTION章节）：继续执行主机端的代码，后续通过执行`sdAADeviceSynchronize`接口，获取对应的assert错误码。

使用方法

- `assert` 接口是SDAA C提供的内置接口，使用时无需引入头文件。

- `assert` 接口的参数与C/C++提供的 `assert` 接口和用法相同。

使用示例

以检测除数不为0作为示例，`assert` 会检查除数是否为0，为0时会打印错误所在的文件行号信息并且终止SPE后续程序的执行。

```
__device__ int g_sum = 0;
__device__ void my_div(int a, int b)
{
    // 如果除数不为0, 则打印错误信息并终止SPE后续程序的执行
    assert(b != 0 && "b should not be zero");
    g_sum = a / b;
}

__global__ void bar()
{
    my_div(1, 2);
}

int main()
{
    sdaaSetDevice(0);
    bar<<<1>>>();
    sdaaDeviceSynchronize();

    return 0;
}
```

12. 命令行工具

12.1 TecoGDB调试工具

TecoGDB (Tecororigin GNU Debugger) 是基于GNU GDB开发的运行在Linux系统上的源码级调试工具，可以调试运行在太初AI加速卡上的程序，提供对被调试程序的运行控制和信息打印功能。该工具提供真实物理环境下的调试功能，避免仿真调试和真实环境运行的误差，方便您准确定位问题。

更多内容，可参考《TecoGDB命令行工具用户手册》。

12.2 sdaacfilt符号解码工具

sdaacfilt符号解码工具是一个命令行工具，能将 SDAA C 修饰 (mangle) 后的符号解码 (demangle) 为用户可读的符号名。该工具在 c++filt 的功能基础上，增加了对 SDAA C 数据类型的支持。如果不能解码输入符号，sdaacfilt 将返回原符号名。

使用方法

```
$ sdaacfilt <symbols(s)>
```

对输入的一个或以空格隔开的多个符号进行解码，并输出解码后的符号名。

如果命令行中未输入符号，则通过交互式读取命令行输入的方式，输出解码结果。

使用示例

```
# 解码单个符号
$ sdaacfilt _Z4testDv16_fS_
test(floatv16, floatv16)

# 解码多个符号
$ sdaacfilt _Z4testDF16_DF16_ _Z4testu6__bf16u6__bf16 not_mangled
test(half, half)
test(bfloat16, bfloat16)
not_mangled

# 交互式解码
$ sdaacfilt
[Interactive Mode] Please input demangled symbol(s)
_Z4testDF16_DF16_
test(half, half)
_Z4testu6__bf16u6__bf16
test(bfloat16, bfloat16)
not_mangled
not_mangled

# 对一个文件内符号解码
$ cat test.txt
```

```
| _Z4testDF16_DF16_ *** _Z4testu6__bf16u6__bf16 *** not_mangled |  
$ sdaacfilt < test.txt  
| test(half, half) *** test(bfloat16, bfloat16) *** not_mangled |
```

通过 nm 读取符号并通过管道解码

```
$ nm --extern-only test.o | sdaacfilt  
0000000000000000 T test(half, half)  
0000000000000000 T test(halfv16, halfv16)  
0000000000000000 T test(floatv16, floatv16)  
0000000000000000 T test(intv16, intv16)  
0000000000000000 T test(uintv16, uintv16)  
0000000000000000 T test(shortv32, shortv32)  
0000000000000000 T test(ushortv32, ushortv32)  
0000000000000000 T test(float, float)  
0000000000000000 T test(bfloat16, bfloat16)
```

13. 性能调优

13.1 概述

本手册主要介绍基于SDAA C的性能调优方法，最大程度上利用SDAA异构并行计算平台的硬件资源实现最佳性能编程，主要结构如下：

名称	说明
性能度量	性能度量相关的指标和工具： - 性能指标：指令周期和访存带宽，可以在一定程度上反应计算和访存相关操作的性能。 - SDAA C支持的性能采样工具以及性能分析工具。
性能优化	具体的性能优化方法。 程序并行：从任务级、线程级（SPMD）到指令级（SIMD）不同粒度的并行化方法。 - 接口特性优化：介绍函数接口、数学函数等优化方法。 程序编译：介绍SDAA C支持的一些编译优化方法。

本手册主要面向具有一定编程经验且对程序性能有一定要求的用户。手册中讨论的内容主要基于SDAA C编程语言，因此您需要能阅读C/C++代码并且能够使用SDAA C进行编程，建议您先阅读《SDAA C编程指南》再阅读本手册以获得最佳的使用体验。

13.2 性能度量

13.2.1 性能指标

本文主要介绍衡量程序性能的关键指标：执行周期和访存带宽。

执行周期

在SDAA C中，使用执行周期作为衡量程序性能的关键指标。

程序性能公式如下：

$$\frac{1}{\text{性能}} = \frac{\text{指令}}{\text{程序}} \cdot \frac{\text{执行周期}}{\text{指令}} \cdot \frac{\text{时间}}{\text{执行周期}} = \frac{1}{\text{程序}} \cdot \text{执行周期} \cdot \text{时钟周期}$$

其中：

- 时钟周期（单位：秒）**：计算核心SPE基本的时间单位，指发生一个最小事件所需的时间，由计算核心SPE的时钟频率决定。

时钟周期的公式：
$$\text{时钟周期} = \frac{1}{\text{SPE的时钟频率}}$$

- **执行周期（单位：拍）**：程序运行所消耗的时钟周期个数（也称作拍数，一拍对应一个时钟周期）。

由于时钟周期是由计算核心SPE的时钟频率所决定的固定值，那么执行周期越短，程序的运行效率越高效，程序性能越好。因此，执行周期是衡量程序性能的关键指标。

? 说明：

- 非访存类指令的执行周期通常是固定的。
- 访存类指令的执行周期与数据量、访存方式以及时钟频率有关。

访存带宽

对于以数据读写为主的程序，访存带宽（即IO带宽）是通用的性能指标。

访存带宽是计算核心SPE在单位时间内读取或写入内存的数据量，通常以每秒传输的比特数或字节数进行衡量。

理论带宽

理论带宽的计算公式如下：

$$\text{理论带宽 (Byte/s)} = \frac{\text{内存最大频率} \times \text{内存通道数} \times 64 \text{ (每次传输位宽, 单位: bit)}}{8}$$

访存带宽

在实际使用中，由于算法效率、内存延迟等均会影响数据传输的效率，因此访存带宽通常低于理论带宽。

访存带宽的计算公式如下：

$$\text{访存带宽 (Byte/s)} = \frac{\text{数据传输量 (Byte)} \times \text{频率 (Hz)}}{\text{执行周期 (拍)}}$$

由以上公式可以看出：执行周期越短，访存带宽越大。

13.2.2 性能采样

使用 `perf_start`、`perf_stop` 和 `perf_print` 函数可以对SDAA C程序进行性能采样，采集目标代码段的执行周期。目前，性能采样工具只输出两个主要的性能数据，一个是 `perf_start`、`perf_stop` 之间的执行周期（延时拍数），另一个是指令缓存脱靶次数（Instruction Cache Miss）。

? 说明：

使用性能采样函数前，需要手动包含 `sdaa_perf.h` 头文件。

- **perf_start**：开始性能采样接口，通过此接口标识性能采样的起始位置。
- **perf_stop**：结束性能采样接口，通过此接口标识性能采样的结束位置。

- **perf_print**: 在终端界面输出采集到的性能数据（延时拍数和指令缓存脱靶次数）。

性能采样示例

? 说明:

为准确测量性能，可以循环多次调用被测程序并计算执行周期的平均值，以减少缓存未命中（Cache Miss）所带来的性能偏差。

使用 **perf_start** 和 **perf_stop** 进行性能采样的示例如下：

```
#include <sdaa_perf.h>
using namespace sdaa;

#define NUM 51200
#define FSIZE (NUM * sizeof(float))

__global__ void kernel_sig(float *a, uint32_t num)
{
    if (threadIdx == 0){
        perf_start();
        for (uint32_t i = 0; i < num; i++) {
            a[i] = sigmoidf(a[i]);
        }
        perf_stop();
        perf_print();
    }
}

int main()
{
    sdaaSetDevice(0);
    // 主机端申请数据空间
    float *host_data = (float *)malloc(FSIZE);
    for (int i = 0; i < NUM; i++) {
        host_data[i] = (float)i / NUM;
    }
    float *dev_input = NULL;
    // 申请Device端数据空间
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将主机端数据复制到Device端
    sdaaMemcpy(dev_input, host_data, FSIZE, sdaaMemcpyHostToDevice);
    // 核函数运行计算
    kernel_sig<<1>>>(dev_input, NUM);
    sdaaDeviceSynchronize();
    // 将计算好的数据从Device端复制到主机端
    sdaaMemcpy(host_data, dev_input, FSIZE, sdaaMemcpyDeviceToHost);

    free(host_data);
    sdaaFree(dev_input);
    return 0;
}
```

性能采样结果

完成性能测试后，调用 `perf_print` 函数打印出性能采样结果。性能采样结果将提供关于代码性能的详细信息：

```
Kernel perf data of thread 0:
-----
| START LINE NUM          | 10          |
| LATENCY                 | 35004840    |
| CNT_L1IC_MISS           | 51205       |
```

输出说明

字段	说明
START LINE NUM	测试开始的行号。
LATENCY	延迟时间，即执行周期，单位：拍。 说明： 具体结果可能随着不同的硬件上下波动。
CNT_LIIC_MISS	指令缓存脱靶次数。

13.3 性能优化

13.3.1 程序并行

13.3.1.1 任务级并行

本文主要介绍任务级并行的性能优化策略和示例。

优化策略

太初AI加速卡目前提供多个计算核心阵列SPA，每个SPA构成一个独立的进程。任务级并行主要指：同时并行运行多个计算核心阵列SPA完成相应的计算任务。

优化示例

计算40000个数据的sigmoid输出值。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

优化前

只使用一个SPA计算40000个数的sigmoid输出值。

```
#include <sdaa_perf.h>
using namespace sdaa;

#define DEV_NUM 4
#define NUM 10000
#define FSIZE (NUM * sizeof(float))

__global__ void kernel_sig(float *a, uint32_t num)
{
    if (threadIdx == 0){
        perf_start();
        for (uint32_t i = 0; i < num; i++) {
            *(a + i) = sigmoidf(*(a + i));
        }
        perf_stop();
        perf_print();
    }
}

int main()
{
    sdaaSetDevice(0);
    // 主机端申请数据空间
    float *host_data = (float *)malloc(DEV_NUM * FSIZE);
    for (int i = 0; i < (DEV_NUM * NUM); i++) {
        host_data[i] = (float)i / NUM;
    }
    float *dev_input = NULL;
    // 申请Device端数据空间
    sdaaMalloc((void **)&dev_input, (DEV_NUM * FSIZE));
    // 将主机端数据复制到Device端
    sdaaMemcpy(dev_input, host_data, (DEV_NUM * FSIZE), sdaaMemcpyHostToDevice);
    // 核函数运行计算
    kernel_sig<<1>>>(dev_input, (DEV_NUM * NUM));
    sdaaDeviceSynchronize();
    // 将计算好的数据从Device端复制到主机端
    sdaaMemcpy(host_data, dev_input, (DEV_NUM * FSIZE), sdaaMemcpyDeviceToHost);

    free(host_data);
    sdaaFree(dev_input);
    return 0;
}
```

优化后

将40000个数改为进程间共享数据，本文以4等份为例，同时并行运行4个SPA计算40000个数的sigmoid输出值，每个SPA计算一份数据。

```
#include <sdaa_perf.h>
#include <fcntl.h>
#include <iostream>
#include <string>
#include <sys/mman.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>
using namespace sdaa;

#define DEV_NUM 4
#define NUM 10000
#define FSIZE (NUM * sizeof(float))

__global__ void kernel_sig(float *a, uint32_t num)
{
    if(threadIdx == 0){
        perf_start();
        for (uint32_t i = 0; i < num; i++) {
            *(a + i) = sigmoidf(*(a + i));
        }
        perf_stop();
        perf_print();
    }
}

int main()
{
    float *shared_mem;
    // 创建共享内存
    int fd = shm_open("/my_shared_memory", O_CREAT | O_RDWR, S_IRUSR | S_IWUSR);
    ftruncate(fd, (DEV_NUM * FSIZE));

    // 共享内存 大小分成 4 份
    shared_mem = (float *)mmap(NULL, (DEV_NUM * FSIZE), PROT_READ | PROT_WRITE,
                               MAP_SHARED, fd, 0);

    for (int i = 0; i < (DEV_NUM * NUM); i++) {
        shared_mem[i] = (float)i / NUM;
    }

    // 创建第一个子进程
    pid_t pid1 = fork();
    if (pid1 == 0) {
        // 第一个子进程中使用第一个计算核心阵列SPA
        sdaaSetDevice(0);
        // 在Device端申请内存，大小是 共享内存的 1/4
        float *dev_input = NULL;
        sdaaMalloc((void **)&dev_input, FSIZE);
        // 将共享内存的第一个1/4数据复制到 第一个device 的内存上
        sdaaMemcpy(dev_input, shared_mem, FSIZE, sdaaMemcpyHostToDevice);
        // 运行kernel 函数 进行计算
        kernel_sig<<1>>>(dev_input, NUM);
        sdaaDeviceSynchronize();
        // 将计算过的 device 上的数据复制到第一个1/4的共享内存
    }
}
```

```

    sdaaMemcpy(shared_mem, dev_input, FSIZE, sdaaMemcpyDeviceToHost);
    sdaaFree(dev_input);
    exit(0);
}

// 创建第二个子进程
pid_t pid2 = fork();
if (pid2 == 0) {
    // 第二个子进程中使用第二个计算核心阵列SPA
    sdaaSetDevice(1);
    // 在Device端申请内存, 大小是 共享内存的 1/4
    float *dev_input = NULL;
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将共享内存的第二个1/4数据复制到 第二个device 的内存上
    sdaaMemcpy(dev_input, (shared_mem + NUM), FSIZE,
               sdaaMemcpyHostToDevice);
    // 运行kernel 函数 进行计算
    kernel_sig<<1>>>(dev_input, NUM);
    sdaaDeviceSynchronize();
    // 将计算过的 device 上的数据复制到第二个1/4的共享内存
    sdaaMemcpy((shared_mem + NUM), dev_input, FSIZE,
               sdaaMemcpyDeviceToHost);
    sdaaFree(dev_input);
    exit(0);
}

// 创建第三个子进程
pid_t pid3 = fork();
if (pid3 == 0) {
    // 第三个子进程中使用第三个计算核心阵列SPA
    sdaaSetDevice(2);
    // 在Device端申请内存, 大小是 共享内存的 1/4
    float *dev_input = NULL;
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将共享内存的第三个1/4数据复制到 第三个device 的内存上
    sdaaMemcpy(dev_input, (shared_mem + 2 * NUM), FSIZE,
               sdaaMemcpyHostToDevice);
    // 运行kernel 函数 进行计算
    kernel_sig<<1>>>(dev_input, NUM);
    sdaaDeviceSynchronize();
    // 将计算过的 device 上的数据复制到第三个1/4的共享内存
    sdaaMemcpy((shared_mem + 2 * NUM), dev_input, FSIZE,
               sdaaMemcpyDeviceToHost);
    sdaaFree(dev_input);
    exit(0);
}

// 创建第四个子进程
pid_t pid4 = fork();
if (pid4 == 0) {
    // 第四个子进程中使用第四个计算核心阵列SPA
    sdaaSetDevice(3);
    // 在Device端申请内存, 大小是 共享内存的 1/4
    float *dev_input = NULL;
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将共享内存的第四个1/4数据复制到 第四个device 的内存上
    sdaaMemcpy(dev_input, (shared_mem + 3 * NUM), FSIZE,
               sdaaMemcpyHostToDevice);
    // 运行kernel 函数 进行计算

```

```
kernel_sig<<1>>>(dev_input, NUM);
sdaaDeviceSynchronize();
// 将计算过的 device 上的数据复制到第四个1/4的共享内存
sdaaMemcpy((shared_mem + 3 * NUM), dev_input, FSIZE,
           sdaaMemcpyDeviceToHost);
sdaaFree(dev_input);
exit(0);
}

pid_t pid;
int status = 0;
// 等待子进程全部执行完成
while ((pid = wait(&status)) > 0)
    ;

// remove share memory
munmap(shared_mem, (4 * FSIZE));
shm_unlink("/my_shared_memory");

return 0;
}
```

优化效果

SPA的ID	优化前的计算任务的执行周期（拍）	优化后的计算任务的执行周期（拍）
SPA0	27351358	6818339
SPA1	N/A	6830025
SPA2	N/A	6278515
SPA3	N/A	6840136

在本示例中，同样的数据计算量，4个SPA同时并行运行的性能（取4个SPA执行周期中的最大值），是只使用1个SPA的约4倍。

13.3.1.2 线程级并行（SPMD）

本文主要介绍线程级并行（SPMD）的性能优化策略和示例。

优化策略

太初AI加速卡目前提供多个计算核心SPE，每个SPE构成一个独立的线程。线程级并行（SPMD）主要指：每个计算核心SPE并行完成不同数据的相同任务，从而发挥设备的并行处理能力。

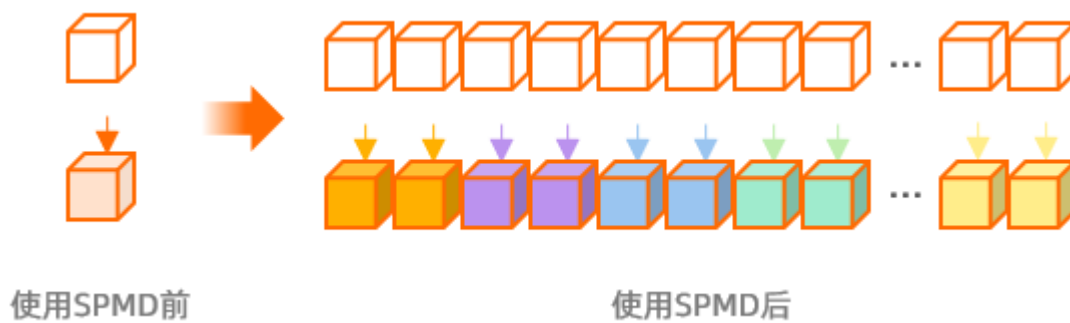


图1 线程级并行（SPMD）

如上图，使用SPMD优化前是所有线程操作同一个数据，优化后是每个线程操作不同的数据。因此，线程级并行可以大幅度提高数据处理的性能。

优化示例

计算51200个数据的sigmoid输出值。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

优化前

一个SPA的所有SPE均计算相同的51200个数的sigmoid输出值。

```
#include <sdaa_perf.h>
using namespace sdaa;

#define NUM 51200
#define FSIZE (NUM * sizeof(float))

__global__ void kernel_sig(float *a, uint32_t num)
{
    if (threadIdx == 0){
        perf_start();
        for (uint32_t i = 0; i < num; i++) {
            a[i] = sigmoidf(a[i]);
        }
        perf_stop();
        perf_print();
    }
}

int main()
{
    sdaaSetDevice(0);
    // 主机端申请数据空间
    float *host_data = (float *)malloc(FSIZE);
    for (int i = 0; i < NUM; i++) {
```

```

        host_data[i] = (float)i / NUM;
    }
    float *dev_input = NULL;
    // 申请Device端数据空间
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将主机端数据复制到Device端
    sdaaMemcpy(dev_input, host_data, FSIZE, sdaaMemcpyHostToDevice);
    // 核函数运行计算
    kernel_sig<<1>>>(dev_input, NUM);
    sdaaDeviceSynchronize();
    // 将计算好的数据从Device端复制到主机端
    sdaaMemcpy(host_data, dev_input, FSIZE, sdaaMemcpyDeviceToHost);

    free(host_data);
    sdaaFree(dev_input);
    return 0;
}

```

优化后

通过`threadIdx`给每个SPE（线程）分配不同的数据，从而实现`threadDim`个数据的并行计算。

```

#include <sdaa_perf.h>
using namespace sdaa;

#define NUM 51200
#define FSIZE (NUM * sizeof(float))

__global__ void kernel_sig(float *a, uint32_t num)
{
    perf_start();
    // 每次循环计算是跨步为 threadDim, 为每个SPE分配一个数据去计算
    for (uint32_t i = 0; i < num; i += threadDim) {
        // 用 i+threadIdx 作为坐标, 每个SPE都参与计算
        a[i + threadIdx] = sigmoidf(a[i + threadIdx]);
    }
    perf_stop();
    perf_print();
}

int main()
{
    sdaaSetDevice(0);
    // 主机端申请数据空间
    float *host_data = (float *)malloc(FSIZE);
    for (int i = 0; i < NUM; i++) {
        host_data[i] = (float)i / NUM;
    }
    float *dev_input = NULL;
    // 申请Device端数据空间
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将主机端数据复制到Device端
    sdaaMemcpy(dev_input, host_data, FSIZE, sdaaMemcpyHostToDevice);
    // 核函数运行计算
    kernel_sig<<1>>>(dev_input, NUM);
    sdaaDeviceSynchronize();
}

```



```
// 将计算好的数据从Device端复制到主机端
sdaaMemcpy(host_data, dev_input, FSIZE, sdaaMemcpyDeviceToHost);

free(host_data);
sdaaFree(dev_input);
return 0;
}
```

优化效果

SPE的ID	优化前的计算任务的执行周期（拍）	优化后的计算任务的执行周期（拍）
SPE0	35005118	897947
SPE1	N/A	897899
SPE2	N/A	897852
SPE3	N/A	898037
SPE4	N/A	902568
SPE5	N/A	902420
SPE6	N/A	897938
SPE7	N/A	898017
SPE8	N/A	902478
SPE9	N/A	897848

在本示例中展示部分SPE性能数据，使用全部的SPE做线程级并行（SPMD）优化后的性能（取所有SPE执行周期中的最大值）是优化前的约39倍。

13.3.1.3 指令级并行（SIMD）

本文主要介绍指令级并行（SIMD）的性能优化策略和示例。

优化策略

太初AI加速卡的每个计算核心SPE都有多个向量寄存器，运用SIMD（Single Instruction Multiple Data，单指令流多数据流）向量指令的加速能力，不仅可以降低功耗，还可以提高性能，满足系统需要。

优化示例

计算51200次浮点加法。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

优化前

使用SPMD编程模型，一个SPA内所有SPE分别执行1600次浮点标量计算，实现51200次浮点加法。

```
#include <sdaa_perf.h>
using namespace sdaa;

#define NUM 51200
#define FSIZE (NUM * sizeof(float))

__global__ void kernel_add(float *a, uint32_t num)
{
    perf_start();
    // 每次循环计算是跨步为 threadDim, 为每个SPE分配一个数据去计算
    for (uint32_t i = 0; i < num; i += threadDim) {
        // 用 i+threadIdx 作为坐标, 每个SPE都参与计算
        a[i + threadIdx] = a[i + threadIdx] + 10;
    }
    perf_stop();
    perf_print();
}

int main()
{
    sdaaSetDevice(0);
    // 主机端申请数据空间
    float *host_data = (float *)malloc(FSIZE);
    for (int i = 0; i < NUM; i++) {
        host_data[i] = (float)i / NUM;
    }
    float *dev_input = NULL;
    // 申请Device端数据空间
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将主机端数据复制到Device端
    sdaaMemcpy(dev_input, host_data, FSIZE, sdaaMemcpyHostToDevice);
    // 核函数运行计算
    kernel_add<<<1>>>>(dev_input, NUM);
    sdaaDeviceSynchronize();
    // 将计算好的数据从Device端复制到主机端
    sdaaMemcpy(host_data, dev_input, FSIZE, sdaaMemcpyDeviceToHost);

    free(host_data);
    sdaaFree(dev_input);
    return 0;
}
```

优化后

使用SPMD编程模型和SIMD优化，一个SPA内所有SPE分别执行100次浮点向量计算，实现51200次浮点加法。

```
#include <sdaa_perf.h>
using namespace sdaa;

#define NUM 51200
#define FSIZE (NUM * sizeof(float))
#define SIMD_NUM 16

__global__ void kernel_add(float *a, uint32_t num)
{
    perf_start();
    floatv16 va;
    // 每次循环计算是跨步为 threadDim*16, 为每个SPE分配16个数据去计算
    for (uint32_t i = 0; i < num; i += (threadDim * SIMD_NUM)) {
        // 用 i+threadIdx*SIMD_NUM
        // 作为坐标, 每个SPE都往向量寄存器装入16个float数
        simd_load(va, (a + i + threadIdx * SIMD_NUM));
        // 每个SPE 都做 add的运算
        va = va + 10;
        // 将 每个SPE的向量寄存器里的16个float数据, 存回到对应地址
        simd_store(va, (a + i + threadIdx * SIMD_NUM));
    }
    perf_stop();
    perf_print();
}

int main()
{
    sdaaSetDevice(0);
    // 主机端申请数据空间
    float *host_data = (float *)malloc(FSIZE);
    for (int i = 0; i < NUM; i++) {
        host_data[i] = (float)i / NUM;
    }
    // simd_load/simd_store 需要做数据地址的 64Byte对齐
    float __attribute__((aligned(64))) *dev_input = NULL;
    // 申请Device端数据空间
    sdaaMalloc((void **)&dev_input, FSIZE);
    // 将主机端数据复制到Device端
    sdaaMemcpy(dev_input, host_data, FSIZE, sdaaMemcpyHostToDevice);
    // 核函数运行计算
    kernel_add<<<1>>>>(dev_input, NUM);
    sdaaDeviceSynchronize();
    // 将计算好的数据从Device端复制到主机端
    sdaaMemcpy(host_data, dev_input, FSIZE, sdaaMemcpyDeviceToHost);

    free(host_data);
    sdaaFree(dev_input);
    return 0;
}
```

优化效果

SPE的ID	优化前的计算任务的执行周期（拍）	优化后的计算任务的执行周期（拍）
SPE0	638738	39531
SPE1	638702	39369
SPE2	638670	39689
SPE3	638643	39727
SPE4	638662	39014
SPE5	642580	39209
SPE6	638579	39032
SPE7	638514	38981
SPE8	638803	40509
SPE9	640998	40473

在本示例中展示部分SPE性能数据，采用指令级并行（SIMD）优化后的性能是优化前的约15.77倍。

13.3.2 函数接口

13.3.2.1 SPE同步接口

本文主要介绍使用线程组同步SPE接口的性能优化策略和示例。

优化策略

使用线程组同步SPE接口，加入线程组的SPE在满足以下条件时，性能较好：

SPE的ID号除以8所得的商或者余数相同。

例如：使用包括4个SPE的线程组进行同步时，线程组A={0, 1, 2, 3}或B={0, 8, 16, 24}的性能要优于线程组C={0, 9, 18, 27}。

优化示例

对使用线程组A={0, 1, 2, 3}、B={0, 8, 16, 24}和C={0, 9, 18, 27}同步SPE接口的执行周期进行测试并比较。

每个测试均调用1000次sync_threads并计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

__device__ void sync_perf(ThreadGroup &tg)
{
    if (!thread_group_is_included(tg, threadIdx)) return;
    perf_start();
    for (int i = 0; i < 1000; i++) {
        sync_threads(tg);
    }
    perf_stop();
    perf_print();
}

// sync among SPE 0, 1, 2, 3
__global__ void test_a()
{
    unsigned long thread_group[4] = {0, 1, 2, 3};
    ThreadGroup tg(thread_group, 4);
    sync_perf(tg);
}

// sync among SPE 0, 8, 16, 24
__global__ void test_b()
{
    unsigned long thread_group[4] = {0, 8, 16, 24};
    ThreadGroup tg(thread_group, 4);
    sync_perf(tg);
}

// sync among SPE 0, 9, 18, 27
__global__ void test_c()
{
    unsigned long thread_group[4] = {0, 9, 18, 27};
    ThreadGroup tg(thread_group, 4);
    sync_perf(tg);
}

int main()
{
    sdaaSetDevice(2);
    test_a<<<1>>>();
    sdaaDeviceSynchronize();
    test_b<<<1>>>();
    sdaaDeviceSynchronize();
    test_c<<<1>>>();
    sdaaDeviceSynchronize();
    return 0;
}
```

优化效果

线程组中所有SPE的ID	平均每次同步的执行周期（拍）
0, 1, 2, 3	226
0, 8, 16, 24	218
0, 9, 18, 27	495

在本示例中，在满足优化策略的条件时，使用线程组同步SPE接口的性能是不满足条件时的约2.19倍。

13.3.2.2 DMA数据搬运

13.3.2.2.1 SPE ID相关

本文主要介绍使用不同线程组进行DMA数据搬运的性能优化策略和示例。

优化策略

使用DMA数据搬运接口时，在数据搬运量一定的情况下，加入线程组的SPE满足以下条件时，性能较好：

- SPE的ID号除以8所得的余数尽可能不相同。
- 在无法避免需使用有相同余数的SPE情况下，启用的有相同余数的SPE的数量尽可能小。

例如：使用8个SPE的线程组进行数据搬运时：

- 线程组A={0, 1, 2, 3, 4, 5, 6, 7} 的性能要优于线程组B={0, 1, 2, 3, 8, 9, 10, 11}。
- 线程组B={0, 1, 2, 3, 8, 9, 10, 11}的性能要优于线程组C={0, 1, 8, 9, 16, 17, 24, 25}。

提示：

该优化策略适用于非阻塞型DMA数据搬运及阻塞型DMA数据搬运中非SPM存储空间之间的数据搬运。

优化示例

使用8个SPE的线程组进行非阻塞型DMA数据搬运时，测试以下几种线程组的执行周期并比较：

- A={0, 1, 2, 3, 4, 5, 6, 7}，余数均不相同，即相同余数的SPE数量最大值为1。
- B={0, 1, 2, 3, 8, 9, 10, 11}，有2组相同的余数（0, 1, 2, 3），相同余数的SPE数量最大值为2。
- C={0, 1, 8, 9, 16, 17, 24, 25}，有4组相同的余数（0, 1），相同余数的SPE数量最大值为4。

每个测例均调用10000次memcpy_async（详情请参考《SDAA C编程指南》的**memcpy_async 老接口**章节）计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率	数据搬运量
v1.2.0	2.36GHz	$8 \times 10000 \times 8$ Bytes（启用8个SPE，每个SPE处理10000个double类型的数据）

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define THREADNUM 8
#define N 10000
#define NUM 10000
#define TESTNUM 3
#define SIZE (THREADNUM * NUM)
#define CALC(x) (sin(x) * sin(x) + cos(x) * cos(x))

__device__ void dma_a(double *src, ThreadGroup &tg)
{
    if (!thread_group_is_included(tg, threadIdx)) {
        return;
    }

    unsigned long loc = 0;

    loc = threadIdx;

    double *data = (double *)malloc(NUM * sizeof(double));

    perf_start();
    for (int i = 0; i < N; i++) {
        memcpy_async(data, src + (loc * NUM), NUM * sizeof(double), 0, {0, 0},
                     MemcpyGlobalToSpm);
        memcpy_wait();
    }
    perf_stop();

    for (int i = 0; i < NUM; i++) {
        data[i] = CALC(data[i]);
    }

    perf_start();
    for (int i = 0; i < N; i++) {
        memcpy_async(src + (loc * NUM), data, NUM * sizeof(double), 0, {0, 0},
                     MemcpySpmToGlobal);
        memcpy_wait();
    }
    perf_stop();
}
```

```

    perf_print();
    free(data);
}

// use SPE 0, 1, 2, 3, 4, 5, 6, 7 to copy data
__global__ void test_a(double *src)
{
    unsigned long thread_group[THREADNUM] = {0, 1, 2, 3, 4, 5, 6, 7};
    ThreadGroup tg(thread_group, THREADNUM);
    dma_a(src, tg);
}

__device__ void dma_b(double *src, ThreadGroup &tg)
{
    if (!thread_group_is_included(tg, threadIdx)) {
        return;
    }

    unsigned long loc = 0;

    if (threadIdx < 4) {
        loc = threadIdx;
    } else {
        loc = threadIdx - 4;
    }

    double *data = (double *)malloc(NUM * sizeof(double));

    perf_start();
    for (int i = 0; i < N; i++) {
        memcpy_async(data, src + (loc * NUM), NUM * sizeof(double), 0, {0, 0},
                     MccpyGlobalToSpm);
        memcpy_wait();
    }
    perf_stop();

    for (int i = 0; i < NUM; i++) {
        data[i] = CALC(data[i]);
    }

    perf_start();
    for (int i = 0; i < N; i++) {
        memcpy_async(src + (loc * NUM), data, NUM * sizeof(double), 0, {0, 0},
                     MccpySpmToGlobal);
        memcpy_wait();
    }
    perf_stop();
    perf_print();
    free(data);
}

// use SPE 0, 1, 2, 3, 8, 9, 10, 11 to copy data
__global__ void test_b(double *src)
{
    unsigned long thread_group[THREADNUM] = {0, 1, 2, 3, 8, 9, 10, 11};
    ThreadGroup tg(thread_group, THREADNUM);
    dma_b(src, tg);
}

```



```

__device__ void dma_c(double *src, ThreadGroup &tg)
{
    if (!thread_group_is_included(tg, threadIdx)) {
        return;
    }

    unsigned long loc = 0;

    if (threadIdx % 8 == 0) {
        loc = threadIdx / 8;
    } else {
        loc = threadIdx / 8 + 4;
    }

    double *data = (double *)malloc(NUM * sizeof(double));

    perf_start();
    for (int i = 0; i < N; i++) {
        memcpy_async(data, src + (loc * NUM), NUM * sizeof(double), 0, {0, 0},
                     MemcpyGlobalToSpm);
        memcpy_wait();
    }
    perf_stop();

    for (int i = 0; i < NUM; i++) {
        data[i] = CALC(data[i]);
    }

    perf_start();
    for (int i = 0; i < N; i++) {
        memcpy_async(src + (loc * NUM), data, NUM * sizeof(double), 0, {0, 0},
                     MemcpySpmToGlobal);
        memcpy_wait();
    }
    perf_stop();
    perf_print();
    free(data);
}

// use SPE 0, 1, 8, 9, 16, 17, 24, 25 to copy data
__global__ void test_c(double *src)
{
    unsigned long thread_group[THREADNUM] = {0, 1, 8, 9, 16, 17, 24, 25};
    ThreadGroup tg(thread_group, THREADNUM);
    dma_c(src, tg);
}

void call_kernel(double *src, double (*output)[SIZE])
{
    double *dev_a = NULL;
    double *dev_b = NULL;
    double *dev_c = NULL;

    sdAASetDevice(0);
    sdAAMalloc((void **)&dev_a, SIZE * sizeof(double));
    sdAAMemcpy(dev_a, src, SIZE * sizeof(double), sdAAMemcpyHostToDevice);
    sdAAMalloc((void **)&dev_b, SIZE * sizeof(double));
    sdAAMemcpy(dev_b, src, SIZE * sizeof(double), sdAAMemcpyHostToDevice);

```

```

sdaaMalloc((void **)&dev_c, SIZE * sizeof(double));
sdaaMemcpy(dev_c, src, SIZE * sizeof(double), sdaaMemcpyHostToDevice);

test_a<<<1>>>(dev_a);
sdaaDeviceSynchronize();

test_b<<<1>>>(dev_b);
sdaaDeviceSynchronize();

test_c<<<1>>>(dev_c);
sdaaDeviceSynchronize();

sdaaMemcpy(output[0], dev_a, SIZE * sizeof(double),
            sdaaMemcpyDeviceToHost);
sdaaMemcpy(output[1], dev_b, SIZE * sizeof(double),
            sdaaMemcpyDeviceToHost);
sdaaMemcpy(output[2], dev_c, SIZE * sizeof(double),
            sdaaMemcpyDeviceToHost);
sdaaFree(dev_a);
sdaaFree(dev_b);
sdaaFree(dev_c);
}

int main()
{
    int errnum = 0;
    double data[SIZE];
    double output[TESTNUM][SIZE];

    // init data
    for (int i = 0; i < SIZE; i++) {
        data[i] = rand();
        for (int j = 0; j < TESTNUM; j++) {
            output[j][i] = 0;
        }
    }

    call_kernel(data, output);

    for (int i = 0; i < TESTNUM; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (fabs(output[i][j] - 1) > 1e-9) {
                errnum++;
                printf("data[%d] = %lf, output[%d][%d] = %lf\n", j, data[j], i,
                       j, output[i][j]);
            }
        }
    }
    if (!errnum) {
        printf("all pass!\n");
    }
    return 0;
}

```

优化效果

线程组中所有SPE的ID	Global到SPM存储空间平均每次DMA操作的执行周期（拍）	SPM到Global存储空间平均每次DMA操作的执行周期（拍）
0, 1, 2, 3, 4, 5, 6, 7	7060	5265
0, 1, 2, 3, 8, 9, 10, 11	11340	9958
0, 1, 8, 9, 16, 17, 24, 25	17466	17327

在本示例中，使用非阻塞型DMA数据搬运接口进行数据搬运时，满足优化策略的性能相比最差情况：

- Global向SPM存储空间进行数据搬运：约2.47倍。
- SPM向Global存储空间进行数据搬运：约3.29倍。

13.3.2.2.2 阻塞型DMA数据搬运

13.3.2.2.2.1 概述

阻塞型DMA数据搬运通常比非阻塞型DMA数据搬运的使用限制少、通用性高，但性能会低于非阻塞型接口。

在数据搬运量相同的情况下，阻塞型DMA数据搬运可使用以下优化策略，性能会接近非阻塞型DMA数据搬运：

- 搬运地址
- 数据搬运量
- 数据搬运方向

13.3.2.2.2.2 搬运地址

本文主要介绍阻塞型DMA数据搬运地址的性能优化策略和示例。

优化策略

阻塞型DMA数据搬运在数据搬运量相同、搬运方向相同的情况下：

- 当目的地址和源地址均满足4B对齐时，性能最好。
- 当目的地址和源地址的任意一个不满足4B对齐时，如果目标地址和源地址除以4所得的余数相同，性能较好。

优化示例

单一线程使用阻塞型DMA数据搬运的方式，从Global到SPM存储空间搬运相同的数据量时，测试以下情况的执行周期并比较：

- 目的地址和源地址均满足4B对齐。
- 目的地址和源地址的任意一个不满足4B对齐，但目的地址和源地址除以4所得的余数相同。
- 目的地址和源地址的任意一个不满足4B对齐，但目的地址和源地址除以4所得的余数不同。

每个测试均调用1000次memcpy并计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率	数据搬运量
v1.2.0	2.36GHz	128KB

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define SIZE 0x20000 // 128K

__device__ char global_arr[SIZE + 1];

__device__ void memcpy_perf(char *dst, char *src)
{
    // 打印内存拷贝的地址信息
    printf("dst addr: %p, src addr: %p\n", dst, src);
    // 数据初始化
    for (int i = 0; i < SIZE; i++) {
        dst[i] = 0;
        src[i] = 10;
    }
    // 采集完成拷贝128KB数据量的性能数据
    perf_start();
    for (int i = 0; i < 1000; i++) {
        memcpy(dst, src, SIZE * sizeof(char));
    }
    perf_stop();

    // 检查数据拷贝结果是否正确
    for (int i = 0; i < SIZE; i++) {
        if (dst[i] != 10) printf("memcpy results not correct!\n");
    }
}

// 地址为4B对齐场景下的数据拷贝
__global__ void test_aligned()
{
    if (threadIdx != 0) return;
    char *local_arr = (char *)malloc(SIZE * sizeof(char));
```

```
memcpy_perf(local_arr, global_arr);
free(local_arr);
// 输出性能数据
perf_print();
}

// 地址部分4B对齐场景下的数据拷贝
__global__ void test_partial_aligned()
{
    if (threadIdx != 0) return;
    char *local_arr = (char *)malloc((SIZE + 1) * sizeof(char));
    memcpy_perf(local_arr + 1, global_arr + 1);
    free(local_arr);
    // 输出性能数据
    perf_print();
}

// 地址不对齐场景下的数据拷贝
__global__ void test_unaligned()
{
    if (threadIdx != 0) return;
    char *local_arr = (char *)malloc((SIZE + 1) * sizeof(char));
    memcpy_perf(local_arr + 1, global_arr);
    free(local_arr);
    // 输出性能数据
    perf_print();
}

int main()
{
    // 设置使用2号SPA
    sdAASetDevice(2);
    // kernel函数调用
    test_aligned<<<1>>>>();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_partial_aligned<<<1>>>>();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_unaligned<<<1>>>>();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    return 0;
}
```

优化效果

地址对齐方式	平均每次memcpy的执行周期（拍）	对应单线程的访存带宽（GB/s）
方式一：目的地址和源地址均满足4B对齐，例如： <code>dst addr: 0x1400, src addr: 0x6100040107e0</code>	7362	45.45
方式二：目的地址和源地址的任意一个不满足4B对齐，但两者除以4所得的余数相同，例如： <code>dst addr: 0x1401, src addr: 0x6100040107e1</code>	8704	35.06
方式三：目的地址和源地址的任意一个不满足4B对齐，且两者除以4所得的余数不同，例如： <code>dst addr: 0x1401, src addr: 0x6100040107e0</code>	331260	0.92

在本示例中，阻塞型DMA数据搬运：

- 地址对齐方式一的访存带宽是方式三的约49.40倍。
- 地址对齐方式二的访存带宽是方式三的约38.10倍。

13.3.2.2.3 数据搬运量

本文主要介绍阻塞型DMA数据搬运量的性能优化策略和示例。

优化策略

阻塞型DMA数据搬运在搬运地址对齐、搬运方向相同的情况下，在满足以下条件时，性能较好：

数据搬运量4B对齐，即数据搬运量的大小是4B的整数倍。

优化示例

单一线程使用阻塞型DMA数据搬运的方式，从Global到SPM存储空间的不同4B对齐地址搬运数据时，测试以下情况的执行周期并比较：

- 数据搬运量为128KB，是4B的整数倍。
- 数据搬运量为（128KB - 1B），不是4B的整数倍。

- 数据搬运量为（128KB - 2B），不是4B的整数倍。
- 数据搬运量为（128KB - 3B），不是4B的整数倍。
- 数据搬运量为（128KB - 4B），是4B的整数倍。

每个测试均调用1000次memcpy并计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define SIZE 0x20000 // 128K

__device__ char global_arr[SIZE];

__device__ void memcpy_perf(char *dst, char *src, unsigned int data_size)
{
    // 打印内存拷贝的地址信息
    printf("dst addr: %p, src addr: %p\n", dst, src);
    // 数据初始化
    for (int i = 0; i < data_size; i++) {
        dst[i] = 0;
        src[i] = 10;
    }
    // 采集完成拷贝128KB数据量的性能数据
    perf_start();
    for (int i = 0; i < 1000; i++) {
        memcpy(dst, src, data_size * sizeof(char));
    }
    perf_stop();
    // 检查数据拷贝结果是否正确
    for (int i = 0; i < data_size; i++) {
        if (dst[i] != 10) printf("memcpy results not correct!\n");
    }
    // 输出性能数据
    perf_print();
}

// 地址为4B对齐场景下的数据拷贝
__global__ void test_128kb()
{
    if (threadIdx != 0) return;
    const unsigned int data_size = SIZE;
    char *local_arr = (char *)malloc(data_size * sizeof(char));
    memcpy_perf(local_arr, global_arr, data_size);
    free(local_arr);
}
```

```
}

// 地址为1B对齐场景下的数据拷贝
__global__ void test_128kb_m1b()
{
    if (threadIdx != 0) return;
    const unsigned int data_size = SIZE - 1;
    char *local_arr = (char *)malloc(data_size * sizeof(char));
    memcpy_perf(local_arr, global_arr, data_size);
    free(local_arr);
}

// 地址为2B对齐场景下的数据拷贝
__global__ void test_128kb_m2b()
{
    if (threadIdx != 0) return;
    const unsigned int data_size = SIZE - 2;
    char *local_arr = (char *)malloc(data_size * sizeof(char));
    memcpy_perf(local_arr, global_arr, data_size);
    free(local_arr);
}

// 地址为3B对齐场景下的数据拷贝
__global__ void test_128kb_m3b()
{
    if (threadIdx != 0) return;
    const unsigned int data_size = SIZE - 3;
    char *local_arr = (char *)malloc(data_size * sizeof(char));
    memcpy_perf(local_arr, global_arr, data_size);
    free(local_arr);
}

// 地址为4B对齐场景下的数据拷贝
__global__ void test_128kb_m4b()
{
    if (threadIdx != 0) return;
    const unsigned int data_size = SIZE - 4;
    char *local_arr = (char *)malloc(data_size * sizeof(char));
    memcpy_perf(local_arr, global_arr, data_size);
    free(local_arr);
}

int main()
{
    // 设置使用2号SPA
    sdAASetDevice(2);
    // kernel函数调用
    test_128kb_m1b();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_128kb_m2b();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_128kb_m3b();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_128kb_m4b();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
```



```
test_128kb_m3b<<<1>>>();
// 等待kernel函数执行完成
sdaaDeviceSynchronize();
// kernel函数调用
test_128kb_m4b<<<1>>>();
// 等待kernel函数执行完成
sdaaDeviceSynchronize();
return 0;
}
```

优化效果

数据搬运量	平均每次memcpy的执行周期（拍）	对应单线程的访存带宽（GB/s）
128KB	7364	41.44
128KB - 1B	8283	36.84
128KB - 2B	7890	38.67
128KB - 3B	7673	39.77
128KB - 4B	7365	41.43

在本示例中，虽然数据搬运量满足4B对齐时所搬运的数据量比数据搬运量不满足4B对齐时大，但访存带宽是不对齐的最多约1.12倍。

13.3.2.2.2.4 数据搬运方向

本文主要介绍阻塞型DMA数据搬运方向的性能优化策略和示例。

优化策略

阻塞型DMA数据搬运在数据搬运量相同、搬运地址对齐的情况下，在不同的搬运方向下，性能由高到低排列如下：

- 1. 同一SPM到SPM存储空间
- 2. Global到SPM存储空间
- 3. SPM到Global存储空间
- 4. Global到Global存储空间

优化示例

单一线程使用阻塞型DMA数据搬运的方式，在搬运数据量相同、搬运地址对齐时，测试以下不同数据搬运方向的执行周期并比较：

- 1. 同一SPM到SPM存储空间
- 2. Global到SPM存储空间
- 3. SPM到Global存储空间
- 4. Global到Global存储空间

每个测试均调用1000次memcpy并计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率	数据搬运量
v1.2.0	2.36GHz	96KB

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define SIZE 0x18000 // 96K

__device__ char global_arr0[SIZE];
__device__ char global_arr1[SIZE];

__device__ void memcpy_perf(char *dst, char *src)
{
    // 打印内存拷贝的地址信息
    printf("dst addr: %p, src addr: %p\n", dst, src);
    // 数据初始化
    for (int i = 0; i < SIZE; i++) {
        dst[i] = 0;
        src[i] = 10;
    }
    // 采集完成拷贝96KB数据量的性能数据
    perf_start();
    for (int i = 0; i < 1000; i++) {
        memcpy(dst, src, SIZE * sizeof(char));
    }
    perf_stop();
    // 检查数据拷贝结果是否正确
    for (int i = 0; i < SIZE; i++) {
        if (dst[i] != 10) printf("memcpy results not correct!\n");
    }
    // 输出性能数据
    perf_print();
}

// 从SPM空间向SPM空间进行数据拷贝
__global__ void test_s2s()
{
    if (threadIdx != 0) return;
```

```

    char *local_arr0 = (char *)malloc(SIZE * sizeof(char));
    char *local_arr1 = (char *)malloc(SIZE * sizeof(char));
    memcpy_perf(local_arr1, local_arr0);
    free(local_arr1);
    free(local_arr0);
}

// 从Global空间向SPM空间进行数据拷贝
__global__ void test_g2s()
{
    if (threadIdx != 0) return;
    char *local_arr = (char *)malloc(SIZE * sizeof(char));
    memcpy_perf(local_arr, global_arr0);
    free(local_arr);
}

// 从SPM空间向Global空间进行数据拷贝
__global__ void test_s2g()
{
    if (threadIdx != 0) return;
    char *local_arr = (char *)malloc(SIZE * sizeof(char));
    memcpy_perf(global_arr0, local_arr);
    free(local_arr);
}

// 从Global空间向Global空间进行数据拷贝
__global__ void test_g2g()
{
    // 仅使用0号SPE进行性能测试
    if (threadIdx != 0) return;
    memcpy_perf(global_arr0, global_arr1);
}

int main()
{
    // 设置使用2号SPA
    sdAASetDevice(2);
    // kernel函数调用
    test_s2s<<<1>>>>();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_g2s<<<1>>>>();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_s2g<<<1>>>>();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    // kernel函数调用
    test_g2g<<<1>>>>();
    // 等待kernel函数执行完成
    sdAADeviceSynchronize();
    return 0;
}

```

优化效果

数据搬运方向	平均每次memcpy的执行周期（拍）	对应单线程的访存带宽（GB/s）
方向一：同一SPM到SPM存储空间	3669	62.38
方向二：Global到SPM存储空间	5524	41.43
方向三：SPM到Global存储空间	5703	40.13
方向四：Global到Global存储空间	14340	15.96

在本示例中，在搬运地址4B对齐、搬运数据量为96KB时：

1. 搬运方向一的访存带宽是方向四的约3.90倍。
2. 搬运方向二的访存带宽是方向四的约2.59倍。
3. 搬运方向三的访存带宽是方向四的约2.53倍。

13.3.2.2.3 非阻塞型DMA数据搬运

本文主要介绍非阻塞型DMA数据搬运的性能优化策略和示例。

优化策略

如果搬运的数据与当前程序的后续操作不存在数据依赖时，使用**非阻塞型接口**可以更好的进行流水排布，相对**阻塞型接口**性能更好。

优化示例

在设备端测试以下情况的数据计算总执行周期并比较：

- 使用阻塞型数据搬运接口：先执行数据搬运，再执行与搬运数据无依赖的计算操作。
- 使用非阻塞型数据搬运接口：同步执行数据搬运和与搬运数据无依赖的计算操作。

每个测试均调用10000次数据搬运和与搬运数据无依赖的计算操作，从而计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率	数据搬运量
v1.2.0	2.36GHz	10KB

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;
```

```

#define SIZE 2560
#define SIZE_CAL 256
#define TIME 10000

__device__ void test_async(int *d_dst, int *d_cal)
{
    int *spm_src = (int *)malloc(SIZE * sizeof(int));
    for (int i = 0; i < SIZE; i++) {
        spm_src[i] = i;
    }
    // 统计性能数据
    perf_start();

    intv16 cal_src;
    intv16 add = 1;

    for (int time = 0; time < TIME; time++) {
        // 发起非阻塞式数据搬运
        memcpy_async(d_dst, spm_src, SIZE * sizeof(int), 0, {0, 0}, MemcpySpmToGlobal);

        // 同步进行与数据搬运无数据依赖的计算
        for (int idx = 0; idx < SIZE_CAL; idx += 16) {
            // 使用向量进行计算
            simd_load(cal_src, d_cal + idx);
            intv16 tmp = cal_src + add;
            simd_store(tmp, d_cal + idx);
        }

        // 等待完成非阻塞式数据搬运
        memcpy_wait();
    }
    // 统计性能数据
    perf_stop();
    free(spm_src);
}

__device__ void test_sync(int *d_dst, int *d_cal)
{
    int *spm_src = (int *)malloc(SIZE * sizeof(int));
    for (int i = 0; i < SIZE; i++) {
        spm_src[i] = i;
    }
    // 统计性能数据
    perf_start();

    intv16 cal_src;
    intv16 add = 1;

    for (int time = 0; time < TIME; time++) {
        // 发起阻塞式数据搬运
        memcpy(d_dst, spm_src, SIZE * sizeof(int));

        // 异步进行与数据搬运无数据依赖的计算
        for (int idx = 0; idx < SIZE_CAL; idx += 16) {
            // 使用向量进行计算
            simd_load(cal_src, d_cal + idx);
            intv16 tmp = cal_src + add;
            simd_store(tmp, d_cal + idx);
        }
    }
}

```

```

    }
    // 统计性能数据
    perf_stop();
    free(spm_src);
}

__global__ void test(int *d_dst_async, int *d_cal_async, int *d_dst_sync, int *d_cal_sync)
{
    if (threadIdx != 0) {
        return;
    }

    test_async(d_dst_async, d_cal_async);
    test_sync(d_dst_sync, d_cal_sync);

    // 打印性能数据
    perf_print();
}

int main()
{
    int h_dst_async[SIZE] = { 0 };
    int h_cal_async[SIZE_CAL];
    int h_dst_sync[SIZE] = { 0 };
    int h_cal_sync[SIZE_CAL];

    for (int i = 0; i < SIZE_CAL; i++) {
        h_cal_async[i] = i;
        h_cal_sync[i] = i;
    }

    // 设置使用0号SPA
    sdaaSetDevice(0);
    int *d_dst_async = NULL;
    int *d_cal_async = NULL;
    int *d_dst_sync = NULL;
    int *d_cal_sync = NULL;
    // 在Global内存中malloc一段空间
    sdaaMalloc((void**)&d_dst_async, SIZE * sizeof(int));
    sdaaMalloc((void**)&d_cal_async, SIZE_CAL * sizeof(int));
    sdaaMalloc((void**)&d_dst_sync, SIZE * sizeof(int));
    sdaaMalloc((void**)&d_cal_sync, SIZE_CAL * sizeof(int));

    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(d_dst_async, h_dst_async, SIZE * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_cal_async, h_cal_async, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_dst_sync, h_dst_sync, SIZE * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_cal_sync, h_cal_sync, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);

    // kernel函数调用
    test<<<1>>>>(d_dst_async, d_cal_async, d_dst_sync, d_cal_sync);

    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // 将数据从Device端拷贝到Host端
    sdaaMemcpy(h_dst_async, d_dst_async, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_cal_async, d_cal_async, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_dst_sync, d_dst_sync, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_cal_sync, d_cal_sync, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);
}

```

```
for (int i = 0; i < SIZE; i++) {
    if ((h_dst_async[i] != i) || (h_dst_sync[i] != i)) {
        printf("h_dst_async[%d] = %d, h_dst_sync[%d] = %d\n", i, h_dst_async[i], i, h_dst_sync[i]);
    }
}

for (int i = 0; i < SIZE_CAL; i++) {
    if ((h_cal_async[i] != (i + TIME)) || (h_cal_sync[i] != (i + TIME))) {
        printf("h_cal_async[%d] = %d, h_cal_sync[%d] = %d\n", i, h_cal_async[i], i, h_cal_sync[i]);
    }
}

// 释放申请的Global内存
sdaaFree(d_dst_async);
sdaaFree(d_cal_async);
sdaaFree(d_dst_sync);
sdaaFree(d_cal_sync);
return 0;
}
```

优化效果

接口类型	数据计算总执行周期（拍）
阻塞型接口	2301
非阻塞型接口	1292

在本示例中，使用非阻塞型接口，相对于阻塞型接口的数据计算总执行周期减少了1009拍左右，使用非阻塞型接口的性能是使用阻塞型接口的约1.72倍。

13.3.2.3 RMA数据搬运

13.3.2.3.1 SPE ID相关

本文主要介绍RMA数据搬运的性能优化策略和示例。

优化策略

进行RMA数据搬运的两个SPE的ID号x和y，在满足以下条件时，性能较好：

令 $D(x, y) = \text{abs}(x/8 - y/8) + \text{abs}(x\%8 - y\%8)$ ，则 $D(x, y)$ 的值越小，性能越好。

例如：SPE0 和 SPE9 的RMA数据搬运性能要优于 SPE0 和 SPE31 的RMA数据搬运性能。

优化示例

测试并比较以下两种场景的阻塞型RMA数据搬运的执行周期：

- SPE0 和 SPE9
- SPE0 和 SPE31

每个测试均调用1000次**rma_put**并计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

__local__ int src;
__local__ int dst;

__device__ void rma_perf(unsigned long target_id)
{
    if (threadIdx != 0 && threadIdx != target_id) return;
    // data init
    src = 0;
    dst = 0;
    if (threadIdx == 0) src = 100;
    ThreadGroup tg(0x1);
    thread_group_include(tg, target_id);
    sync_threads(tg);
    // start rma_put
    perf_start();
    for (int i = 0; i < 1000; i++) {
        rma_put(&src, &dst, sizeof(int), 0, target_id);
    }
    perf_stop();
    perf_print();
}

// SPE 0 put to SPE 9
__global__ void test_a()
{
    rma_perf(9);
}

// SPE 0 put to SPE 31
__global__ void test_b()
{
    rma_perf(31);
}

int main() {
    sdaaSetDevice(2);
    test_a<<<1>>>();
}
```



```
sdaaDeviceSynchronize();
test_b<<<1>>>();
sdaaDeviceSynchronize();
return 0;
}
```

优化效果

RMA数据搬运的SPE ID	平均每次RMA操作的执行周期（拍）
0, 9	186
0, 31	259

在本示例中，在满足优化策略（SPE0 和 SPE9）的性能，是不满足（SPE0 和 SPE31）的约1.39倍。

13.3.2.3.2 非阻塞型RMA数据搬运

13.3.2.3.2.1 优化策略

- 1. 如果搬运的数据与当前程序的后续操作不存在数据依赖时，使用非阻塞型接口可以更好的进行流水排布，相对阻塞型接口性能更好。
- 2. 当使用非阻塞型接口进行数据搬运时，使用RmaCustomizeMode模式相对于RmaNormalMode模式，性能更好。

13.3.2.3.2.2 rma_get 优化示例

在设备端测试以下情况的数据计算总执行周期并比较：

- 使用阻塞型数据搬运接口：先使用rma_get进行数据搬运，再执行与数搬运数据无依赖的计算操作。
- 使用非阻塞型数据搬运接口：使用rma_async_get的RmaNormalMode模式，同步执行数据搬运和与搬运数据无依赖的计算操作。
- 使用非阻塞型数据搬运接口：使用rma_async_get的RmaCustomizeMode模式，同步执行数据搬运和与搬运数据无依赖的计算操作。

每个测试均调用10000次数据搬运和与搬运数据无依赖的计算操作，从而计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit 版本	计算核心频率	数据搬运量
v1.2.0	2.36GHz	10KB

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define SIZE 2560
#define SIZE_CAL 480
#define TIME 10000

__device__ void test_sync(int *dst, int *src, int *cal)
{
    intv16 cal_src;
    intv16 add = 1;
    // 统计性能数据
    perf_start();

    for (int time = 0; time < TIME; time++) {
        // 发起阻塞型rma_get
        rma_get(dst, src, SIZE * sizeof(int), 0, 1);

        for (int idx = 0; idx < SIZE_CAL; idx += 16) {
            // 使用向量进行计算
            simd_load(cal_src, cal + idx);
            intv16 tmp = cal_src + add;
            simd_store(tmp, cal + idx);
        }
    }
    // 统计性能数据
    perf_stop();
}

__device__ void test_async_normal(int *dst, int *src, int *cal)
{
    RmaHandle handle;
    ThreadGroup thread_group(3);
    // 部分SPE同步
    sync_threads(thread_group);

    intv16 cal_src;
    intv16 add = 1;

    if ((threadIdx == 0)) {
        // 统计性能数据
        perf_start();

        for (int time = 0; time < TIME; time++) {
            // 配置远端计算核心ID
            rma_set_thread_id(handle, 1);
            // 发起非阻塞数据搬运
            rma_async_get(dst, src, SIZE * sizeof(int), handle);

            for (int idx = 0; idx < SIZE_CAL; idx += 16) {
                // 使用向量进行计算
                simd_load(cal_src, cal + idx);
                intv16 tmp = cal_src + add;
                simd_store(tmp, cal + idx);
            }
        }
    }
}
```

```

        // 本端计算核心等待数据搬运完成
        rma_complete(handle);
    }
    // 统计性能数据
    perf_stop();
}

if (threadIdx == 1) {
    // 配置远端计算核心ID
    rma_set_thread_id(handle, 0);

    // 等待当前计算核心完成RMA操作
    for (int time = 0; time < TIME; time++) {
        rma_wait(handle);
    }
}
}

__device__ void test_async_cust(int *dst, int *src, int *cal)
{
    RmaHandle handle;
    ThreadGroup thread_group(3);
    // 部分SPE同步
    sync_threads(thread_group);

    intv16 cal_src;
    intv16 add = 1;

    if ((threadIdx == 0)) {
        // 统计性能数据
        perf_start();

        for (int time = 0; time < TIME; time++) {
            // 配置远端计算核心ID
            rma_set_thread_id(handle, 1);
            // 发起非阻塞数据搬运
            rma_async_get(dst, src, SIZE * sizeof(int), handle);

            for (int idx = 0; idx < SIZE_CAL; idx += 16) {
                // 使用向量进行计算
                simd_load(cal_src, cal + idx);
                intv16 tmp = cal_src + add;
                simd_store(tmp, cal + idx);
            }
            // 本端计算核心等待数据搬运完成
            rma_complete(handle, RmaCustomizeMode);
        }
        // 统计性能数据
        perf_stop();
    }

    if (threadIdx == 1) {
        rma_set_thread_id(handle, 0);
        rma_wait(handle, TIME);
    }
}

__global__ void test(int *d_sync, int *d_async_cust, int *d_async_normal,
                    int *d_sync_cal, int *d_async_cust_cal, int *d_async_normal_cal)

```

```

{
    if ((threadIdx != 0) && (threadIdx != 1)) {
        return;
    }

    int *spm = (int *)malloc(SIZE * sizeof(int));

    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            spm[i] = 0;
        }
    }

    if (threadIdx == 1) {
        for (int i = 0; i < SIZE; i++) {
            spm[i] = i;
        }
    }

    // 配置线程组内包含SP0和SPE1
    ThreadGroup thread_group(3);

    sync_threads(thread_group);
    // 测试阻塞接口
    test_sync(spm, spm, d_sync_cal);
    if (threadIdx == 0) {
        memcpy(d_sync, spm, SIZE * sizeof(int));
    }

    // 测试非阻塞接口的RmaNormalMode模式
    test_async_normal(spm, spm, d_async_normal_cal);
    if (threadIdx == 0) {
        memcpy(d_async_normal, spm, SIZE * sizeof(int));
    }

    // 测试非阻塞接口的RmaCustomizeMode
    test_async_cust(spm, spm, d_async_cust_cal);
    if (threadIdx == 0) {
        memcpy(d_async_cust, spm, SIZE * sizeof(int));
    }

    // 打印性能数据
    perf_print();
    free(spm);
}

int main()
{
    int h_sync[SIZE] = { 0 };
    int h_async_cust[SIZE] = { 0 };
    int h_async_normal[SIZE] = { 0 };

    int h_sync_cal[SIZE_CAL] = { 0 };
    int h_async_cust_cal[SIZE_CAL] = { 0 };
    int h_async_normal_cal[SIZE_CAL] = { 0 };

    for (int i = 0; i < SIZE_CAL; i++) {
        h_sync_cal[i] = i;
        h_async_cust_cal[i] = i;
    }
}

```

```

        h_async_normal_cal[i] = i;
    }

    // 设置使用0号SPA
    sdaaSetDevice(0);
    int *d_sync = NULL;
    int *d_async_cust = NULL;
    int *d_async_normal = NULL;

    int *d_sync_cal = NULL;
    int *d_async_cust_cal = NULL;
    int *d_async_normal_cal = NULL;

    // 在Global内存中malloc一段空间
    sdaaMalloc((void**)&d_sync, SIZE * sizeof(int));
    sdaaMalloc((void**)&d_async_cust, SIZE * sizeof(int));
    sdaaMalloc((void**)&d_async_normal, SIZE * sizeof(int));

    sdaaMalloc((void**)&d_sync_cal, SIZE_CAL * sizeof(int));
    sdaaMalloc((void**)&d_async_cust_cal, SIZE_CAL * sizeof(int));
    sdaaMalloc((void**)&d_async_normal_cal, SIZE_CAL * sizeof(int));

    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(d_sync, h_sync, SIZE * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_async_cust, h_async_cust, SIZE * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_async_normal, h_async_normal, SIZE * sizeof(int), sdaaMemcpyHostToDevice);

    sdaaMemcpy(d_sync_cal, h_sync_cal, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_async_cust_cal, h_async_cust_cal, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_async_normal_cal, h_async_normal_cal, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);

    // kernel函数调用
    test<<<1>>>>(d_sync, d_async_cust, d_async_normal,
                  d_sync_cal, d_async_cust_cal, d_async_normal_cal);
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();

    // 将数据从Device端拷贝到Host端
    sdaaMemcpy(h_sync, d_sync, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_async_cust, d_async_cust, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_async_normal, d_async_normal, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);

    sdaaMemcpy(h_sync_cal, d_sync_cal, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_async_cust_cal, d_async_cust_cal, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_async_normal_cal, d_async_normal_cal, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);

    for (int i = 0; i < SIZE; i++) {
        if ((h_sync[i] != i) || (h_async_cust[i] != i) || (h_async_normal[i] != i)) {
            printf("h_sync[%d] = %d, h_async_cust[%d] = %d, h_async_normal[%d] = %d\n",
                   i, h_sync[i], i, h_async_cust[i], i, h_async_normal[i]);
        }
    }

    for (int i = 0; i < SIZE_CAL; i++) {
        if ((h_sync_cal[i] != i + TIME) || (h_async_cust_cal[i] != i + TIME) || (h_async_normal_cal[i]
!= i + TIME)) {
            printf("h_sync_cal[%d] = %d, h_async_cust_cal[%d] = %d, h_async_normal_cal[%d] = %d\n",
                   i, h_sync_cal[i], i, h_async_cust_cal[i], i, h_async_normal_cal[i]);
        }
    }

```

```
    }

    // 释放申请的Global内存
    sdaaFree(d_sync_cal);
    sdaaFree(d_async_cust_cal);
    sdaaFree(d_async_normal_cal);
    sdaaFree(d_sync);
    sdaaFree(d_async_cust);
    sdaaFree(d_async_normal);

    return 0;
}
```

优化效果

接口及模式	数据计算总执行周期（拍）
阻塞型接口 rma_get	13292
非阻塞型接口 rma_async_get 的 RmaNormalMode 模式	13036
非阻塞型接口 rma_async_get 的 RmaCustomizeMode 模式	11937

在本示例这个能，使用非阻塞型接口 rma_async_get 的 RmaCustomizeMode 模式，相对于阻塞型接口 rma_get，数据计算总执行周期减少了1355拍左右，性能是其约1.11倍。

13.3.2.3.2.3 rma_put 优化示例

在设备端测试以下情况的数据计算总执行周期并比较：

- 使用阻塞型数据搬运接口：先使用rma_put进行数据搬运，再执行与搬运数据无依赖的计算操作。
- 使用非阻塞型数据搬运接口：使用rma_async_put的RmaNormalMode模式，同步执行数据搬运和与搬运数据无依赖的计算操作。
- 使用非阻塞型数据搬运接口：使用rma_async_put的RmaCustomizeMode模式，同步执行数据搬运和与搬运数据无依赖的计算操作。

每个测试均调用10000次数据搬运和与搬运数据无依赖的计算操作，从而计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率	数据搬运量
v1.2.0	2.36GHz	10KB

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define SIZE 2560
#define SIZE_CAL 480
#define TIME 10000

__device__ void test_sync(int *dst, int *src, int *cal)
{
    intv16 cal_src;
    intv16 add = 1;
    // 统计性能数据
    perf_start();

    for (int time = 0; time < TIME; time++) {
        // 发起阻塞型rma_put
        rma_put(dst, src, SIZE * sizeof(int), 1, 0);

        for (int idx = 0; idx < SIZE_CAL; idx += 16) {
            // 使用向量进行计算
            simd_load(cal_src, cal + idx);
            intv16 tmp = cal_src + add;
            simd_store(tmp, cal + idx);
        }
    }
    // 统计性能数据
    perf_stop();
}

__device__ void test_async_normal(int *dst, int *src, int *cal)
{
    RmaHandle handle;
    ThreadGroup thread_group(3);
    // 部分SPE同步
    sync_threads(thread_group);

    intv16 cal_src;
    intv16 add = 1;

    if ((threadIdx == 1)) {
        // 统计性能数据
        perf_start();

        for (int time = 0; time < TIME; time++) {
            // 配置远端计算核心ID
            rma_set_thread_id(handle, 0);
            // 发起非阻塞数据搬运
            rma_async_put(dst, src, SIZE * sizeof(int), handle);

            for (int idx = 0; idx < SIZE_CAL; idx += 16) {
                // 使用向量进行计算
                simd_load(cal_src, cal + idx);
                intv16 tmp = cal_src + add;
                simd_store(tmp, cal + idx);
            }
            // 本端计算核心等待数据搬运完成
            rma_complete(handle);
        }
    }
}
```

```

        // 统计性能数据
        perf_stop();
    }

    if (threadIdx == 0) {
        // 配置远端计算核心ID
        rma_set_thread_id(handle, 1);

        // 等待当前计算核心完成RMA操作
        for (int time = 0; time < TIME; time++) {
            rma_wait(handle);
        }
    }
}

__device__ void test_async_cust(int *dst, int *src, int *cal)
{
    RmaHandle handle;
    ThreadGroup thread_group(3);
    // 部分SPE同步
    sync_threads(thread_group);

    intv16 cal_src;
    intv16 add = 1;

    if ((threadIdx == 1)) {
        // 统计性能数据
        perf_start();

        for (int time = 0; time < TIME; time++) {
            // 配置远端计算核心ID
            rma_set_thread_id(handle, 0);
            // 发起非阻塞数据搬运
            rma_async_put(dst, src, SIZE * sizeof(int), handle);

            for (int idx = 0; idx < SIZE_CAL; idx += 16) {
                // 使用向量进行计算
                simd_load(cal_src, cal + idx);
                intv16 tmp = cal_src + add;
                simd_store(tmp, cal + idx);
            }
            // 本端计算核心等待数据搬运完成
            rma_complete(handle, RmaCustomizeMode);
        }
        // 统计性能数据
        perf_stop();
    }

    if (threadIdx == 0) {
        rma_set_thread_id(handle, 1);
        rma_wait(handle, TIME);
    }
}

__global__ void test(int *d_sync, int *d_async_cust, int *d_async_normal,
                    int *d_sync_cal, int *d_async_cust_cal, int *d_async_normal_cal)
{
    if ((threadIdx != 0) && (threadIdx != 1)) {
        return;
    }
}

```



```

}

int *spm = (int *)malloc(SIZE * sizeof(int));

if (threadIdx == 0) {
    for (int i = 0; i < SIZE; i++) {
        spm[i] = 0;
    }
}

if (threadIdx == 1) {
    for (int i = 0; i < SIZE; i++) {
        spm[i] = i;
    }
}

// 配置线程组内包含SP0和SPE1
ThreadGroup thread_group(3);

sync_threads(thread_group);
// 测试阻塞接口
test_sync(spm, spm, d_sync_cal);
if (threadIdx == 0) {
    memcpy(d_sync, spm, SIZE * sizeof(int));
}

// 测试非阻塞接口的RmaNormalMode模式
test_async_normal(spm, spm, d_async_normal_cal);
if (threadIdx == 0) {
    memcpy(d_async_normal, spm, SIZE * sizeof(int));
}

// 测试非阻塞接口的RmaCustomizeMode
test_async_cust(spm, spm, d_async_cust_cal);
if (threadIdx == 0) {
    memcpy(d_async_cust, spm, SIZE * sizeof(int));
}

// 打印性能数据
perf_print();
free(spm);
}

int main()
{
    int h_sync[SIZE] = { 0 };
    int h_async_cust[SIZE] = { 0 };
    int h_async_normal[SIZE] = { 0 };

    int h_sync_cal[SIZE_CAL] = { 0 };
    int h_async_cust_cal[SIZE_CAL] = { 0 };
    int h_async_normal_cal[SIZE_CAL] = { 0 };

    for (int i = 0; i < SIZE_CAL; i++) {
        h_sync_cal[i] = i;
        h_async_cust_cal[i] = i;
        h_async_normal_cal[i] = i;
    }
}

```

```

// 设置使用0号SPA
sdaaSetDevice(0);
int *d_sync = NULL;
int *d_async_cust = NULL;
int *d_async_normal = NULL;

int *d_sync_cal = NULL;
int *d_async_cust_cal = NULL;
int *d_async_normal_cal = NULL;

// 在Global内存中malloc一段空间
sdaaMalloc((void**)&d_sync, SIZE * sizeof(int));
sdaaMalloc((void**)&d_async_cust, SIZE * sizeof(int));
sdaaMalloc((void**)&d_async_normal, SIZE * sizeof(int));

sdaaMalloc((void**)&d_sync_cal, SIZE_CAL * sizeof(int));
sdaaMalloc((void**)&d_async_cust_cal, SIZE_CAL * sizeof(int));
sdaaMalloc((void**)&d_async_normal_cal, SIZE_CAL * sizeof(int));

// 将数据从Host端拷贝到Device端
sdaaMemcpy(d_sync, h_sync, SIZE * sizeof(int), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_async_cust, h_async_cust, SIZE * sizeof(int), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_async_normal, h_async_normal, SIZE * sizeof(int), sdaaMemcpyHostToDevice);

sdaaMemcpy(d_sync_cal, h_sync_cal, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_async_cust_cal, h_async_cust_cal, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_async_normal_cal, h_async_normal_cal, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);

// kernel函数调用
test<<<1>>>>(d_sync, d_async_cust, d_async_normal,
              d_sync_cal, d_async_cust_cal, d_async_normal_cal);
// 等待kernel函数执行完成
sdaaDeviceSynchronize();

// 将数据从Device端拷贝到Host端
sdaaMemcpy(h_sync, d_sync, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);
sdaaMemcpy(h_async_cust, d_async_cust, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);
sdaaMemcpy(h_async_normal, d_async_normal, SIZE * sizeof(int), sdaaMemcpyDeviceToHost);

sdaaMemcpy(h_sync_cal, d_sync_cal, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);
sdaaMemcpy(h_async_cust_cal, d_async_cust_cal, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);
sdaaMemcpy(h_async_normal_cal, d_async_normal_cal, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);

for (int i = 0; i < SIZE; i++) {
    if ((h_sync[i] != i) || (h_async_cust[i] != i) || (h_async_normal[i] != i)) {
        printf("h_sync[%d] = %d, h_async_cust[%d] = %d, h_async_normal[%d] = %d\n",
               i, h_sync[i], i, h_async_cust[i], i, h_async_normal[i]);
    }
}

for (int i = 0; i < SIZE_CAL; i++) {
    if ((h_sync_cal[i] != i + TIME) || (h_async_cust_cal[i] != i + TIME) || (h_async_normal_cal[i]
!= i + TIME)) {
        printf("h_sync_cal[%d] = %d, h_async_cust_cal[%d] = %d, h_async_normal_cal[%d] = %d\n",
               i, h_sync_cal[i], i, h_async_cust_cal[i], i, h_async_normal_cal[i]);
    }
}

// 释放申请的Global内存

```

```
    sdaaFree(d_sync_cal);
    sdaaFree(d_async_cust_cal);
    sdaaFree(d_async_normal_cal);
    sdaaFree(d_sync);
    sdaaFree(d_async_cust);
    sdaaFree(d_async_normal);

    return 0;
}
```

优化效果

接口及模式	数据计算总执行周期（拍）
阻塞接口 rma_put	13291
非阻塞接口 rma_async_put 的 RmaNormalMode 模式	13035
非阻塞接口 rma_async_put 的 RmaCustomizeMode 模式	11906

在本示例中，使用非阻塞型接口 rma_async_put 的 RmaCustomizeMode 模式，相对于阻塞接口 rma_put，数据计算总执行周期减少了1385拍左右，性能是其约1.11倍。

13.3.2.4 Broadcast数据搬运

13.3.2.4.1 SPE ID相关

本文主要介绍使用线程组进行Broadcast数据搬运的性能优化策略和示例。

优化策略

使用线程组进行Broadcast数据搬运时，当线程组内的所有SPE满足以下任意一种条件时，性能较好：

- 线程组内所有SPE刚好能被划分为：{0~7}，{8~15}，{16~23}，{24~31}中的一组或多组。
- 线程组内所有SPE刚好能被划分为：{0, 8, 16, 24}，{1, 9, 17, 25}，{2, 10, 18, 26}，{3, 11, 19, 27}，{4, 12, 20, 28}，{5, 13, 21, 29}，{6, 14, 22, 30}，{7, 15, 23, 31}中的一组或多组。

例如：对包含8个SPE的线程组进行广播时，线程组A={0, 1, 2, 3, 4, 5, 6, 7}或B={0, 1, 8, 9, 16, 17, 24, 25}的性能要优于线程组C={0, 1, 9, 10, 18, 19, 27, 28}。

优化示例

对使用线程组A={0, 1, 2, 3, 4, 5, 6, 7}、B={0, 1, 8, 9, 16, 17, 24, 25}和C={0, 1, 9, 10, 18, 19, 27, 28}进行阻塞型广播的执行周期进行测试并比较。

每个测试均调用1000次broadcast（详情请参考《SDAA C编程指南》的**broadcast 老接口**章节）并计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

__local__ int src;
__local__ int dst;

__device__ void broadcast_perf(ThreadGroup &tg)
{
    if (threadIdx != 0 && !thread_group_is_included(tg, threadIdx)) return;
    // data init
    BroadcastHandle handle(&tg);
    src = 0;
    dst = 0;
    if (threadIdx == 0) src = 100;
    sync_threads(tg);
    // start broadcast
    perf_start();
    for (int i = 0; i < 1000; i++) {
        broadcast(&dst, &src, sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle);
    }
    perf_stop();
    perf_print();
}

// broadcast to 0, 1, 2, 3, 4, 5, 6, 7
__global__ void test_a()
{
    unsigned long thread_group[8] = {0, 1, 2, 3, 4, 5, 6, 7};
    ThreadGroup tg(thread_group, 8);
    broadcast_perf(tg);
}

// broadcast to 0, 1, 8, 9, 16, 17, 24, 25
__global__ void test_b()
{
    unsigned long thread_group[8] = {0, 1, 8, 9, 16, 17, 24, 25};
    ThreadGroup tg(thread_group, 8);
    broadcast_perf(tg);
}

// broadcast to 0, 1, 9, 10, 18, 19, 27, 28
__global__ void test_c()
{

```

```
    unsigned long thread_group[8] = {0, 1, 9, 10, 18, 19, 27, 28};
    ThreadGroup tg(thread_group, 8);
    broadcast_perf(tg);
}
int main()
{
    sdaaSetDevice(2);
    test_a<<<1>>>();
    sdaaDeviceSynchronize();
    test_b<<<1>>>();
    sdaaDeviceSynchronize();
    test_c<<<1>>>();
    sdaaDeviceSynchronize();
    return 0;
}
```

优化效果

线程组中所有SPE的ID	平均每次Broadcast操作的执行周期（拍）
0, 1, 2, 3, 4, 5, 6, 7	266
0, 1, 8, 9, 16, 17, 24, 25	322
0, 1, 9, 10, 18, 19, 27, 28	1362

在本示例中，满足优化策略的条件时使用线程组进行阻塞型Broadcast数据搬运的性能，是不满足条件时的约5.12倍。

13.3.2.4.2 非阻塞型Broadcast数据搬运

本文主要介绍非阻塞型Broadcast数据搬运的性能优化策略和示例。

优化策略

如果搬运的数据与当前程序的后续操作不存在数据依赖时，使用非阻塞型接口可以更好的进行流水排布，相对阻塞式接口性能更好。

优化示例

在设备端使用阻塞型广播接口将SPE0中的数据广播到SPE0~SPE7内然后执行与数据搬运无数据依赖的计算，与使用非阻塞型数据搬运接口同步执行将SPE0中的数据广播到SPE0~SPE7的数据搬运操作和与数据搬运无数据依赖的计算进行性能测试并比较。

每个测试均调用10000次数据搬运和与搬运数据无依赖的计算操作，从而计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率	数据搬运量
v1.2.0	2.36GHz	10KB

测试代码

```
#include<sdaa_perf.h>
using namespace sdaa;

#define SIZE 2560
#define SIZE_CAL 256
#define TIME 10000

__device__ __attribute__((noinline)) void test_sync(int *dst, int *bc_dst, int *bc_src, int *d_cal)
{
    // 设置线程组 SPE0~SPE7
    ThreadGroup thread_group(0xff);
    BroadcastHandle handle(&thread_group);
    // 初始化目的地址数据
    if (thread_group_is_included(thread_group, threadIdx)) {
        memset(bc_dst, 0, SIZE * sizeof(int));
    }

    intv16 cal_src;
    intv16 add = 1;
    // 执行SPE0~SPE7同步
    sync_threads(thread_group);

    if (thread_group_is_included(thread_group, threadIdx)) {
        // 统计性能数据
        perf_start();

        for (int time = 0; time < TIME; time++) {
            broadcast(bc_dst, bc_src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle);

            // 异步进行与数据搬运无数据依赖的计算
            for (int idx = 0; idx < SIZE_CAL; idx += 16) {
                // 使用向量进行计算
                simd_load(cal_src, d_cal + (SIZE_CAL * threadIdx) + idx);
                intv16 tmp = cal_src + add;
                simd_store(tmp, d_cal + (SIZE_CAL * threadIdx) + idx);
            }
        }
        // 统计性能数据
        perf_stop();
    }

    // 将广播数据拷贝到SPE0~SPE7的目的地址
    if (thread_group_is_included(thread_group, threadIdx)) {
        memcpy(dst + threadIdx * SIZE, bc_dst, SIZE * sizeof(int));
    }
}

__device__ __attribute__((noinline)) void test_async(int *dst, int *bc_dst, int *bc_src, int *d_cal)
```

```

{
    // 设置线程组 SPE0~SPE7
    ThreadGroup thread_group(0xff);
    BroadcastHandle handle(&thread_group);
    // 初始化目的地址数据
    if (thread_group_is_included(thread_group, threadIdx)) {
        memset(bc_dst, 0, SIZE * sizeof(int));
    }

    intv16 cal_src;
    intv16 add = 1;
    // 执行SPE0~SPE7同步
    sync_threads(thread_group);

    if (thread_group_is_included(thread_group, threadIdx)) {
        // 统计性能数据
        perf_start();

        for (int time = 0; time < TIME; time++) {
            // 发起非阻塞广播操作
            broadcast_async(bc_dst, bc_src, SIZE * sizeof(int), 0, {0, 0}, 0, BroadcastSpmToSpm, handle);

e);

            // 同步进行与数据搬运无数据依赖的计算
            for (int idx = 0; idx < SIZE_CAL; idx += 16) {
                // 使用向量进行计算
                simd_load(cal_src, d_cal + (SIZE_CAL * threadIdx) + idx);
                intv16 tmp = cal_src + add;
                simd_store(tmp, d_cal + (SIZE_CAL * threadIdx) + idx);
            }
            // 等待非阻塞广播操作完成
            broadcast_wait(handle);
        }
        // 统计性能数据
        perf_stop();
    }

    // 将广播数据拷贝到SPE0~SPE7的目的地址
    if (thread_group_is_included(thread_group, threadIdx)) {
        memcpy(dst + threadIdx * SIZE, bc_dst, SIZE * sizeof(int));
    }
}

__global__ void test(int *d_sync, int *d_async, int *d_sync_cal, int *d_async_cal)
{
    int *src = (int *)malloc(SIZE * sizeof(int));
    int *dst = (int *)malloc(SIZE * sizeof(int));
    if (threadIdx == 0) {
        for (int i = 0; i < SIZE; i++) {
            src[i] = i;
        }
    }
    // 阻塞接口测试
    test_sync(d_sync, dst, src, d_sync_cal);
    // 非阻塞接口测试
    test_async(d_async, dst, src, d_async_cal);

    // 打印性能数据
    perf_print();
}

```

```

    free(dst);
    free(src);
}

int main()
{
    const int thread_cnt = 8;
    int h_dst_async[SIZE * thread_cnt] = { 0 };
    int h_dst_sync[SIZE * thread_cnt] = { 0 };
    int h_sync_cal[SIZE_CAL * thread_cnt] = { 0 };
    int h_async_cal[SIZE_CAL * thread_cnt] = { 0 };

    for (int i = 0; i < SIZE_CAL * thread_cnt; i++) {
        h_sync_cal[i] = i;
        h_async_cal[i] = i;
    }

    // 设置使用0号SPA
    sdaaSetDevice(0);
    int *d_dst_async = NULL;
    int *d_dst_sync = NULL;
    int *d_sync_cal = NULL;
    int *d_async_cal = NULL;

    // 在Global内存中malloc一段空间
    sdaaMalloc((void**)&d_dst_async, SIZE * thread_cnt * sizeof(int));
    sdaaMalloc((void**)&d_dst_sync, SIZE * thread_cnt * sizeof(int));
    sdaaMalloc((void**)&d_sync_cal, SIZE_CAL * thread_cnt * sizeof(int));
    sdaaMalloc((void**)&d_async_cal, SIZE_CAL * thread_cnt * sizeof(int));

    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(d_dst_async, h_dst_async, SIZE * thread_cnt * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_dst_sync, h_dst_sync, SIZE * thread_cnt * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_sync_cal, h_sync_cal, SIZE_CAL * thread_cnt * sizeof(int), sdaaMemcpyHostToDevice);
    sdaaMemcpy(d_async_cal, h_async_cal, SIZE_CAL * thread_cnt * sizeof(int), sdaaMemcpyHostToDevice);

    // kernel函数调用
    test<<<1>>>(d_dst_sync, d_dst_async, d_sync_cal, d_async_cal);

    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // 将数据从Device端拷贝到Host端
    sdaaMemcpy(h_dst_async, d_dst_async, SIZE * thread_cnt * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_dst_sync, d_dst_sync, SIZE * thread_cnt * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_sync_cal, d_sync_cal, SIZE_CAL * thread_cnt * sizeof(int), sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_async_cal, d_async_cal, SIZE_CAL * thread_cnt * sizeof(int), sdaaMemcpyDeviceToHost);

    for (int i = 0; i < SIZE * thread_cnt; i++) {
        if ((h_dst_sync[i] != i % SIZE) || (h_dst_async[i] != i % SIZE)) {
            printf("h_dst_sync[%d] = %d, h_dst_async[%d] = %d\n", i, h_dst_sync[i], i, h_dst_async[i]);
        }
    }

    for (int i = 0; i < SIZE_CAL * thread_cnt; i++) {
        if ((h_sync_cal[i] != i + TIME) || (h_async_cal[i] != i + TIME)) {
            printf("h_sync_cal[%d] = %d, h_async_cal[%d] = %d\n",
                i, h_sync_cal[i], i, h_async_cal[i]);
        }
    }
}

```



```
// 释放申请的Global内存
sdaaFree(d_sync_cal);
sdaaFree(d_async_cal);
sdaaFree(d_dst_async);
sdaaFree(d_dst_sync);
return 0;
}
```

优化效果

接口类型	数据计算总执行周期（拍）
阻塞型接口	2301
非阻塞型接口	1395

在本示例中，使用非阻塞型接口，相对于阻塞型接口的数据计算总执行周期减少了906拍左右，性能是其约1.64倍。

13.3.2.5 原子操作

本文主要介绍原子操作的性能优化策略和示例。

原子操作除atomic_inc外，返回旧值的接口相比不返回旧值的接口会多一些操作，导致性能上的差别。

优化策略

当不需要返回指定地址上的旧值时，可以使用不返回旧值的原子操作接口提升性能：

- 使用atomic_add_noret代替atomic_add。
- 使用atomic_sub_noret代替atomic_sub。
- 使用atomic_cas_bool代替atomic_cas。

优化示例

同一SPA上的所有SPE分别使用有返回值的原子操作接口和没有返回值的原子操作接口，测试并比较以下操作的执行周期：

- 所有SPE均执行1000次atomic_add_noret和atomic_add。
- 所有SPE均执行1000次atomic_sub_noret和atomic_sub。
- 所有SPE均执行1000次atomic_cas_bool和atomic_cas。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
#include <sdaa_atomic.h>
using namespace sdaa;

__device__ int global_val;

// 使用atomic_add接口
__global__ void test_atomic_add()
{
    // 数据初始化
    const int step = 10;
    if (threadIdx == 0) global_val = 0;
    // 对每个线程执行1000次atomic_add
    sync_threads();
    perf_start();
    for (int i = 0; i < 1000; i++) {
        atomic_add(&global_val, step);
    }
    perf_stop();

    // 检查结果的正确性
    sync_threads();
    if (threadIdx != 0) return;
    if (global_val != 1000 * blockDim * step) {
        printf("Error: atomic operations not working correctly!\n");
    }
    // 输出性能数据
    perf_print();
}

// 使用atomic_add_noret接口
__global__ void test_atomic_add_noret()
{
    // 数据初始化
    const int step = 10;
    if (threadIdx == 0) global_val = 0;
    // 对每个线程执行1000次atomic_add_noret
    sync_threads();
    perf_start();
    for (int i = 0; i < 1000; i++) {
        atomic_add_noret(&global_val, step);
    }
    perf_stop();

    // 检查结果的正确性
    sync_threads();
    if (threadIdx != 0) return;
    if (global_val != 1000 * blockDim * step) {
        printf("Error: atomic operations not working correctly!\n");
    }
}
```

```
// 输出性能数据
perf_print();
}

// 使用atomic_sub接口
__global__ void test_atomic_sub()
{
    // 数据初始化
    const int step = 10;
    if (threadIdx == 0) global_val = 0;
    // 对每个线程执行1000次atomic_sub
    sync_threads();
    perf_start();
    for (int i = 0; i < 1000; i++) {
        atomic_sub(&global_val, step);
    }
    perf_stop();

    // 检查结果的正确性
    sync_threads();
    if (threadIdx != 0) return;
    if (global_val != -1000 * threadDim * step) {
        printf("Error: atomic operations not working correctly!\n");
    }
    // 输出性能数据
    perf_print();
}

// 使用atomic_sub_noret接口
__global__ void test_atomic_sub_noret()
{
    // 数据初始化
    const int step = 10;
    if (threadIdx == 0) global_val = 0;
    // 对每个线程执行1000次atomic_sub_noret
    sync_threads();
    perf_start();
    for (int i = 0; i < 1000; i++) {
        atomic_sub_noret(&global_val, step);
    }
    perf_stop();

    // 检查结果的正确性
    sync_threads();
    if (threadIdx != 0) return;
    if (global_val != -1000 * threadDim * step) {
        printf("Error: atomic operations not working correctly!\n");
    }
    // 输出性能数据
    perf_print();
}

// 使用atomic_cas接口
__global__ void test_atomic_cas()
{
    // 数据初始化
    const int step = 10;
    if (threadIdx == 0) global_val = 0;
    // 对每个线程执行1000次atomic_cas
```

```

    sync_threads();
    perf_start();
    for (int i = 0; i < 1000; i++) {
        atomic_cas(&global_val, step * i, step * (i + 1));
    }
    perf_stop();

    // 检查结果的正确性
    sync_threads();
    if (threadIdx != 0) return;
    if (global_val != 1000 * step) {
        printf("Error: atomic operations not working correctly!\n");
    }
    // 输出性能数据
    perf_print();
}

// 使用atomic_cas_bool接口
__global__ void test_atomic_cas_bool()
{
    // 数据初始化
    const int step = 10;
    if (threadIdx == 0) global_val = 0;
    // 对每个线程执行1000次atomic_cas_bool
    sync_threads();
    perf_start();
    for (int i = 0; i < 1000; i++) {
        atomic_cas_bool(&global_val, step * i, step * (i + 1));
    }
    perf_stop();

    // 检查结果的正确性
    sync_threads();
    if (threadIdx != 0) return;
    if (global_val != 1000 * step) {
        printf("Error: atomic operations not working correctly!\n");
    }
    // 输出性能数据
    perf_print();
}

int main()
{
    // 设置使用2号SPA
    sdaaSetDevice(2);
    // kernel函数调用
    test_atomic_add<<<1>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // kernel函数调用
    test_atomic_add_noret<<<1>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // kernel函数调用
    test_atomic_sub<<<1>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // kernel函数调用
    test_atomic_sub_noret<<<1>>>();
}

```

```

// 等待kernel函数执行完成
sdaaDeviceSynchronize();
// kernel函数调用
test_atomic_cas<<<1>>>();
// 等待kernel函数执行完成
sdaaDeviceSynchronize();
// kernel函数调用
test_atomic_cas_bool<<<1>>>();
// 等待kernel函数执行完成
sdaaDeviceSynchronize();
return 0;
}
```

优化效果

接口名称	平均每个SPE每次执行周期（拍）
atomic_add	125666
atomic_add_noret	7193
atomic_sub	126857
atomic_sub_noret	7259
atomic_cas	15423
atomic_cas_bool	7404

在本示例中，测试平均每个SPE每次执行原子操作的执行周期：

- **atomic_add_noret**接口的性能是**atomic_add**接口的约17.47倍。
- **atomic_sub_noret**接口的性能是**atomic_sub**接口的约17.47倍。
- **atomic_cas_bool**接口的性能是**atomic_cas**接口的约2.08倍。

13.3.2.6 矩阵乘

13.3.2.6.1 阻塞型矩阵乘

矩阵乘阻塞型接口相比非阻塞型接口具有更少的使用限制和更高的通用性，但相应地性能也会差于非阻塞型接口。当满足一定对齐条件时，矩阵乘阻塞型接口的性能可以提升到接近非阻塞型接口的性能。

优化策略

当阻塞型矩阵乘接口满足一下条件时，性能较好：

k（输入矩阵列数、权重矩阵行数）和n（权重矩阵列数）均是32的整数倍。

当使用阻塞型矩阵乘接口时，预先将矩阵分块为32大小的整数倍，可以有效提高矩阵乘接口的性能。

优化示例

单一线程使用阻塞型矩阵乘接口计算SPM上矩阵的乘积时，测试以下两种情况的执行周期并比较：

- k和n均为32的整数倍（m=128，k=64，n=32）。
- k和n不是32的整数倍（m=128，k=60，n=30）。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
#include <sdaa_matmul.h>
using namespace sdaa;

__device__ void matmul_perf(half *output, half *input, half *weight,
                           unsigned int m, unsigned int k, unsigned int n)
{
    // 获取性能数据起始值
    perf_start();
    for (int i = 0; i < 1000; i++) {
        matmul(MatmulHalfToHalf, output, input, weight, m, k, n);
    }
    // 获取性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
}

__device__ void init_matrix(half *mat, unsigned int m, unsigned int n)
{
    // 初始化矩阵数组
    for (unsigned int i = 0; i < m; i++) {
        for (unsigned int j = 0; j < n; j++) {
            mat[i * n + j] = (half) m / (half) n;
        }
    }
}

__global__ void test_aligned()
{
    if (threadIdx != 0) return;
    const unsigned int m = 128, k = 64, n = 32;
    half *input = (half *)malloc(m * k * sizeof(half));
    half *weight = (half *)malloc(k * n * sizeof(half));
    half *output = (half *)malloc(m * n * sizeof(half));
    init_matrix(input, m, k);
}
```

```
init_matrix(weight, k, n);
matmul_perf(output, input, weight, m, k, n);
free(output);
free(weight);
free(input);
}

__global__ void test_unaligned()
{
    if (threadIdx != 0) return;
    const unsigned int m = 128, k = 60, n = 30;
    half *input = (half *)malloc(m * k * sizeof(half));
    half *weight = (half *)malloc(k * n * sizeof(half));
    half *output = (half *)malloc(m * n * sizeof(half));
    init_matrix(input, m, k);
    init_matrix(weight, k, n);
    matmul_perf(output, input, weight, m, k, n);
    free(output);
    free(weight);
    free(input);
}

int main()
{
    // 设置使用0号SPA
    sdaaSetDevice(2);
    // kernel函数调用
    test_aligned<<<1>>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // kernel函数调用
    test_unaligned<<<1>>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    return 0;
}
```

优化效果

矩阵维度大小	平均每次阻塞型矩阵乘接口的执行周期（拍）
m=128, k=64, n=32	26993
m=128, k=60, n=30	146818

在本示例中，k和n非32整数倍（k=60，n=30）的矩阵乘数据量更小，但执行周期更长。k和n是32整数倍（k=64，n=32）的矩阵乘虽然多了一些数据，但性能却是非32整数倍的矩阵乘的5.43倍。出现这个性能差距的原因是非32整数倍的矩阵需要重新切分组装数据，耗时较多。

13.3.2.6.2 非阻塞型矩阵乘

本文主要介绍非阻塞型矩阵乘的性能优化策略和示例。

优化策略

如果搬运的数据与当前程序的后续操作不存在数据依赖时，使用非阻塞式接口可以更好的进行流水排布，相对阻塞式接口性能更好。

优化示例

在设备端测试以下情况的数据计算总执行周期并比较：

- 使用阻塞型矩阵乘接口：先执行矩阵乘，再执行与矩阵乘无数据依赖的计算操作。
- 使用非阻塞型矩阵乘接口：同步执行矩阵乘和与矩阵乘无数据依赖的计算操作。

每个测试均调用10000次矩阵乘操作和与矩阵乘无数据依赖的计算操作，从而计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率	输入矩阵大小	权重矩阵大小
v1.2.0	2.36GHz	128 × 32	32 × 32

测试代码

```
#include <sdaa_matmul.h>
#include <sdaa_perf.h>
using namespace sdaa;

#define WEIGHT_ROW 32
#define WEIGHT_COL 32
#define INPUT_ROW 128
#define INPUT_COL 32
#define OUTPUT_ROW 128
#define OUTPUT_COL 32
#define SIZE_CAL 256
#define TIME 10000

#define INPUT_SIZE (INPUT_ROW * INPUT_COL * sizeof(short))
#define WEIGHT_SIZE (WEIGHT_ROW * WEIGHT_COL * sizeof(short))
#define OUTPUT_SIZE (OUTPUT_ROW * OUTPUT_COL * sizeof(short))

__device__ void test_sync(short *input, short *weight, short *output, int *cal)
{
    short *ibuffer = (short *)malloc(INPUT_SIZE);
    short *wbuffer = (short *)malloc(WEIGHT_SIZE);
    short *obuffer = (short *)malloc(OUTPUT_SIZE);

    memcpy(ibuffer, input, INPUT_SIZE);
```



```

memcpy(wbuffer, weight, WEIGHT_SIZE);
memcpy(obuffer, output, OUTPUT_SIZE);

intv16 cal_src;
intv16 add = 1;
// 统计性能数据
perf_start();

for (int time = 0; time < TIME; time++) {
    matmul(MatmulShortToShort, obuffer, ibuffer, wbuffer, INPUT_ROW, WEIGHT_ROW, OUTPUT_COL);
    // 异步进行与数据搬运无数据依赖的计算
    for (int idx = 0; idx < SIZE_CAL; idx += 16) {
        // 使用向量进行计算
        simd_load(cal_src, cal + idx);
        intv16 tmp = cal_src + add;
        simd_store(tmp, cal + idx);
    }
}
// 统计性能数据
perf_stop();

memcpy(output, obuffer, OUTPUT_SIZE);

free(ibuffer);
free(wbuffer);
free(obuffer);
}

__device__ void test_async(short *input, short *weight, short *output, int *cal)
{
    short *ibuffer = (short *)malloc(INPUT_SIZE);
    short *wbuffer = (short *)malloc(WEIGHT_SIZE);
    short *obuffer = (short *)malloc(OUTPUT_SIZE);

    memcpy(ibuffer, input, INPUT_SIZE);
    memcpy(wbuffer, weight, WEIGHT_SIZE);
    memcpy(obuffer, output, OUTPUT_SIZE);

    MatmulHandle handle;
    intv16 cal_src;
    intv16 add = 1;
    // 统计性能数据
    perf_start();

    for (int time = 0; time < TIME; time++) {
        // 初始化handle
        matmul_init(handle, MatmulShortToShort);
        // 加载权重矩阵
        matmul_load_weight(handle, wbuffer, MatmulK32, MatmulN32);
        // 加载输入矩阵并与权重矩阵进行矩阵乘运算
        matmul_compute(handle, ibuffer, INPUT_ROW, MatmulK32);
        // 将矩阵乘运算结果写回到指定地址
        matmul_store(handle, obuffer, OUTPUT_ROW, MatmulN32);

        for (int idx = 0; idx < SIZE_CAL; idx += 16) {
            // 使用向量进行计算
            simd_load(cal_src, cal + idx);
            intv16 tmp = cal_src + add;
            simd_store(tmp, cal + idx);
        }
    }
}

```

```

    }
    // 等待矩阵乘写回操作完成
    matmul_wait(handle);
}
// 统计性能数据
perf_stop();

memcpy(output, obuffer, OUTPUT_SIZE);

free(ibuffer);
free(wbuffer);
free(obuffer);
}

__global__ void test(short *input, short *weight, short *output_sync,
                    short *output_async, int *cal_sync, int *cal_async)
{
    if (threadIdx != 0) {
        return;
    }

    // 测试同步接口
    test_sync(input, weight, output_sync, cal_sync);
    // 测试异步接口
    test_async(input, weight, output_async, cal_async);

    perf_print();
}

// 初始化数组
void SetArrayVal(short *array, int num, short val) {
    for (int i = 0; i < num; i++) {
        array[i] = val;
    }
}

void entry(short *h_input, short *h_weight, short *h_output_sync,
           short *h_output_async, int *h_cal_sync, int *h_cal_async)
{
    // 设置使用0号SPA
    sdAASetDevice(0);
    short *dev_input = NULL;
    short *dev_weight = NULL;
    short *dev_output_sync = NULL;
    short *dev_output_async = NULL;
    int *dev_cal_sync = NULL;
    int *dev_cal_async = NULL;

    // 在Global内存中malloc一段空间
    sdAAMalloc((void **)&dev_input, INPUT_SIZE);
    sdAAMalloc((void **)&dev_weight, WEIGHT_SIZE);
    sdAAMalloc((void **)&dev_output_sync, OUTPUT_SIZE);
    sdAAMalloc((void **)&dev_output_async, OUTPUT_SIZE);
    sdAAMalloc((void **)&dev_cal_sync, SIZE_CAL * sizeof(int));
    sdAAMalloc((void **)&dev_cal_async, SIZE_CAL * sizeof(int));

    // 将数据从Host端拷贝到Device端
    sdAAMemcpy(dev_input, h_input, INPUT_SIZE, sdAAMemcpyHostToDevice);
    sdAAMemcpy(dev_weight, h_weight, WEIGHT_SIZE, sdAAMemcpyHostToDevice);

```

```

sdaaMemcpy(dev_output_sync, h_output_sync, OUTPUT_SIZE, sdaaMemcpyHostToDevice);
sdaaMemcpy(dev_output_async, h_output_async, OUTPUT_SIZE, sdaaMemcpyHostToDevice);
sdaaMemcpy(dev_cal_sync, h_cal_sync, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);
sdaaMemcpy(dev_cal_async, h_cal_async, SIZE_CAL * sizeof(int), sdaaMemcpyHostToDevice);

// kernel函数调用
test<<<1>>>(dev_intput, dev_weight, dev_output_sync, dev_output_async,
            dev_cal_sync, dev_cal_async);
// 等待kernel函数执行完成
sdaaDeviceSynchronize();

// 将数据从Device端拷贝到Host端
sdaaMemcpy(h_output_sync, dev_output_sync, OUTPUT_SIZE, sdaaMemcpyDeviceToHost);
sdaaMemcpy(h_output_async, dev_output_async, OUTPUT_SIZE, sdaaMemcpyDeviceToHost);
sdaaMemcpy(h_cal_sync, dev_cal_sync, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);
sdaaMemcpy(h_cal_async, dev_cal_async, SIZE_CAL * sizeof(int), sdaaMemcpyDeviceToHost);

// 释放申请的Global内存
sdaaFree(dev_intput);
sdaaFree(dev_weight);
sdaaFree(dev_output_sync);
sdaaFree(dev_output_async);
sdaaFree(dev_cal_sync);
sdaaFree(dev_cal_async);
}

int main()
{
    short *h_intput = (short *)malloc(INPUT_SIZE);
    short *h_weight = (short *)malloc(WEIGHT_SIZE);
    short *h_output_sync = (short *)malloc(OUTPUT_SIZE);
    short *h_output_async = (short *)malloc(OUTPUT_SIZE);

    int *h_cal_sync = (int *)malloc(SIZE_CAL * sizeof(int));
    int *h_cal_async = (int *)malloc(SIZE_CAL * sizeof(int));

    SetArrayVal(h_intput, INPUT_ROW * INPUT_COL, 1);
    SetArrayVal(h_weight, WEIGHT_ROW * WEIGHT_COL, 2);
    memset(h_output_sync, 0, OUTPUT_SIZE);
    memset(h_output_async, 0, OUTPUT_SIZE);
    memset(h_cal_sync, 0, SIZE_CAL * sizeof(int));
    memset(h_cal_async, 0, SIZE_CAL * sizeof(int));
    const short REFEROUT = 64;

    entry(h_intput, h_weight, h_output_sync, h_output_async, h_cal_sync, h_cal_async);

    for (int row = 0; row < OUTPUT_ROW; row++) {
        for (int col = 0; col < OUTPUT_COL; col++) {
            if ((h_output_sync[row * OUTPUT_COL + col] != REFEROUT) ||
                (h_output_async[row * OUTPUT_COL + col] != REFEROUT)) {
                printf("h_output_sync[%d] = %d, h_output_async[%d] = %d\n",
                    row * OUTPUT_COL + col, h_output_sync[row * OUTPUT_COL + col],
                    row * OUTPUT_COL + col, h_output_async[row * OUTPUT_COL + col]);
            }
        }
    }

    for (int i = 0; i < SIZE_CAL; i++) {
        if ((h_cal_sync[i] != TIME) || (h_cal_async[i] != TIME)) {

```

```
        printf("h_cal_sync[%d] = %d, h_cal_async[%d] = %d\n",
               i, h_cal_sync[i], i, h_cal_async[i]);
    }
}

free(h_input);
free(h_weight);
free(h_output_sync);
free(h_output_async);
free(h_cal_sync);
free(h_cal_async);
return 0;
}
```

优化效果

矩阵乘类型	计算总执行周期（拍）
阻塞型矩阵乘	3347
非阻塞型矩阵乘	1453

在本示例中，使用非阻塞型矩阵乘，相对于使用阻塞型矩阵乘的计算总执行周期减少了1894拍左右，性能是其约2.30倍。

13.3.2.7 向量操作

13.3.2.7.1 性能数据

本文主要介绍不同向量接口的性能数据。

?

说明：

本文的向量接口性能数值会有10%左右的波动，具体请以实际情况为准。

使用SIMD接口，源地址和目的地址的数据均位于SPM存储空间，各向量接口的性能如下：

接口类别	接口名称	接口性能（执行周期，单位：拍）
装入/存储类操作	simd_load	12
	simd_loadu	16
	simd_store	7
	simd_storeu	18
	simd_load_widen	12
	simd_store_narrow	7
	simd_set	3
标量扩展类操作	simd_stretch	10
比较选择类操作	simd_seleq	15
	simd_selle	15
	simd_sellt	15
元素整理类操作	simd_concat	10
	simd_ins	7
符号拷贝类操作	simd_copy_sign	10
类型转换类操作	simd_cvt_f2h	5
	simd_cvt_h2f	5
融合乘加类操作	simd_fma	15
	simd_fms	15
	simd_fnma	15
	simd_fnms	15

接口类别	接口名称	接口性能（执行周期，单位：拍）
算数运算类操作	simd_add	12
	simd_sub	12
	simd_mul	12
	simd_div	1723
	simd_mod	1670
	simd_fabs	3
	simd_redsum	3
比较类操作	simd_cmplt	12
	simd_cmple	12
	simd_cmpgt	12
	simd_cmpge	12
	simd_cmpeq	12
	simd_cmpne	15
	simd_isnan	21
	simd_max	22
	simd_min	22
逻辑运算类操作	simd_and	100
	simd_or	16
	simd_not	11
	simd_neg	11

接口类别	接口名称	接口性能（执行周期，单位：拍）
按位运算类操作	simd_band	10
	simd_bxor	10
	simd_bor	10
	simd_bnor	11
	simd_bnot	10

13.3.2.7.2 优化策略

本文主要介绍向量操作的性能优化策略和示例。

优化策略

在使用SIMD接口时，源地址和目的地址的数据均位于SPM存储空间，性能更好。

优化示例

使用**simd_load**接口，测试并比较以下两种情况，一个SPE执行1000次的执行周期：

- 源地址和目的地址的数据均位于SPM存储空间。
- 源地址和目的地址的数据均位于Global存储空间。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;
// 在Global内存空间定义变量
volatile __device__ intv16 glb_vdata;

__device__ void test_simd_load_in_global(int *glb_data)
{
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < 1000; i++){
        simd_load(glb_vdata, glb_data);
    }
    // 收集性能数据结束值
```

```

    perf_stop();
}

__device__ void test_simd_load_in_spm(int *glb_data)
{
    int *spm_data = (int *)malloc(16 * sizeof(int));
    memcpy(spm_data, glb_data, 16 * sizeof(int));
    volatile intv16 spm_vdata;

    perf_start();
    for (int i = 0; i < 1000; i++) {
        simd_load(spm_vdata, spm_data);
    }
    perf_stop();

    free(spm_data);
}

__global__ void test_func(int *glb_data)
{
    if (threadIdx != 0) return;

    test_simd_load_in_global(glb_data);
    test_simd_load_in_spm(glb_data);
    // 输出性能数据
    perf_print();
}

int main()
{
    // 设置使用0号SPA
    sdaaSetDevice(0);
    int h_glb_data[16] = { 0 };

    for (int i = 0; i < 16; i++){
        h_glb_data[i] = i;
    }

    int *glb_data = NULL;
    // 在Global内存中malloc一段空间
    sdaaMalloc((void **)&glb_data, 16 * sizeof(int));
    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(glb_data, h_glb_data, 16 * sizeof(int), sdaaMemcpyHostToDevice);

    // kernel函数调用
    test_func<<<1>>>>(glb_data);
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();

    // 释放申请的Global内存
    sdaaFree(glb_data);
    return 0;
}

```

优化效果

接口名称	接口性能（执行周期，单位：拍） （源地址和目的地址的数据均位于Global存储空间）	接口性能（执行周期，单位：拍） （源地址和目的地址的数据均位于SPM存储空间）
simd_load	758	12

在本示例中，源地址和目的地址的数据均位于SPM存储空间，比源地址和目的地址的数据均位于Global存储空间，向量接口性能快约63倍。

13.3.3 数学函数

13.3.3.1 精度和性能数据

本文主要介绍不同数学函数接口的精度和性能。

?

说明：

本文的接口性能数值会有10%左右的波动，具体请以实际情况为准。

基础函数

类别		函数名	精度	接口性能（执行周期，单位：拍）
指数函数	双精度	exp(x)	[-103.97, 88.72]内最大ULP为150	151
	单精度	expf(x)	最大ULP为1	158
	向量	simd_exp(x)	最大ULP为5	205
	128	simd_exp128(x)	[-19.4, 88.72]内最大ULP为197 否则最大绝对误差为4e-9	满足128个输入全部小于2.77时：523 否则：1214
指数减一的函数	双精度	expm1(x)	最大ULP为1	330
	单精度	expm1f(x)	最大ULP为1	352

类别		函数名	精度	接口性能（执行周期，单位：拍）
对数函数 (以e为底)	双精度	$\log(x)$	最大ULP为4	218
	单精度	$\logf(x)$	最大ULP为4	188
	向量	$\text{simd_log}(x)$	最大ULP为5	218
	128	$\text{simd_log128}(x)$	最大ULP为5	1145
加一的对数函数（以e为底）	双精度	$\log1p(x)$	最大ULP为1	334
	单精度	$\log1pf(x)$	最大ULP为1	356
对数函数（以2为底）	双精度	$\log2(x)$	最大ULP为6	217
	单精度	$\log2f(x)$	最大ULP为5	166
	向量	$\text{simd_log2}(x)$	最大ULP为6	197
	128	$\text{simd_log2}_{128}(x)$	最大ULP为6	1146
平方根函数	双精度	$\text{sqrt}(x)$	最大ULP为0	45
	单精度	$\text{sqrtf}(x)$	最大ULP为0	31
	向量	$\text{simd_sqrt}(x)$	最大ULP为2	98
倒数函数	双精度	$1/x$	最大ULP为0	47
	单精度	$1/x$	最大ULP为0	32
	向量	$\text{simd_vinv}(x)$	最大ULP为1	69
	128	$\text{simd_vinv}_{128}(x)$	最大ULP为1	176
判断是否非数函数	双精度	$\text{isnan}(x)$	最大ULP为0	351
	单精度	$\text{isnan}(x)$	最大ULP为0	351

类别		函数名	精度	接口性能（执行周期，单位：拍）
绝对值函数	双精度	fabs(x)	最大ULP为0	14
	单精度	fabsf(x)	最大ULP为0	14
幂函数	双精度	pow(x)	最大ULP为1	1579
	单精度	powf(x)	最大ULP为3	1602

激活函数和误差函数

类别		函数名	精度	接口性能（执行周期，单位：拍）
双曲正切函数	双精度	tanh(x)	最大ULP为3	761
	单精度	tanhf(x)	最大ULP为2	251
	向量	simd_tanh(x)	最大ULP为32	251
	128	simd_tanh128(x)	最大ULP为32	635
高斯误差函数	双精度	erf(x)	最大ULP为1	[1.25, 5.999)内：1079，否则：184
	单精度	erff(x)	最大ULP为1	[1.25, 5.999)内：1103，否则：206
	向量	simd_erf(x)	最大ULP为17	253
	128	simd_erf128(x)	最大ULP为17	1083

类别		函数名	精度	接口性能（执行周期，单位：拍）
sigmoid函数	双精度	sigmoid(x)	[-10,10]内最大ULP为124，否则最大绝对误差为5e-5	219
	单精度	sigmoidf(x)	[-10,10]内最大ULP为3，否则最大绝对误差为5e-5	205
	向量	simd_sigmoid(x)	[-10,10]内最大ULP为8，否则最大绝对误差为1e-6	204
	128	simd_sigmoid128(x)	[-10,10]内最大ULP为8，否则最大绝对误差为1e-6	506

三角函数、反三角函数

类别		函数名	精度	接口性能（执行周期，单位：拍）
正弦函数	双精度	sin(x)	最大ULP为1	请以实际情况为准
	单精度	sinf(x)	最大ULP为1	请以实际情况为准
	向量	simd_sin(x)	最大ULP为2	140
	128	simd_sin128(x)	最大ULP为2	473
余弦函数	双精度	cos(x)	最大ULP为1	请以实际情况为准
	单精度	cosf(x)	最大ULP为1	请以实际情况为准
	向量	simd_cos(x)	[-2000,2000]内最大ULP为2，否则最大绝对误差为1.28e-7	278
	128	simd_cos128(x)	[-2000,2000]内最大ULP为2，否则最大绝对误差为1.28e-7	1774

类别		函数名	精度	接口性能（执行周期，单位：拍）
正切函数	双精度	$\tan(x)$	最大ULP为1	请以实际情况为准
	单精度	$\tanf(x)$	最大ULP为1	请以实际情况为准
反正切函数	双精度	$\operatorname{atan}(x)$	最大ULP为2	208
	单精度	$\operatorname{atanf}(x)$	最大ULP为2	214
	向量	$\operatorname{simd_atan}(x)$	最大ULP为5	148
	128	$\operatorname{simd_atan128}(x)$	最大ULP为5	523

浮点取整函数

类别		函数名	精度	接口性能（执行周期，单位：拍）
向上取整函数	双精度	$\operatorname{ceil}(x)$	最大ULP为0	77
	单精度	$\operatorname{ceilf}(x)$	最大ULP为0	99
向下取整函数	双精度	$\operatorname{floor}(x)$	最大ULP为0	73
	单精度	$\operatorname{floorf}(x)$	最大ULP为0	95
	向量	$\operatorname{simd_floor}(x)$	最大ULP为0	57
	128	$\operatorname{simd_floor128}(x)$	最大ULP为0	205
就近取整函数	双精度	$\operatorname{round}(x)$	最大ULP为0	72
	单精度	$\operatorname{roundf}(x)$	最大ULP为0	99
	向量	$\operatorname{simd_round}(x)$	最大ULP为0	53
	128	$\operatorname{simd_round128}(x)$	最大ULP为0	131

类别		函数名	精度	接口性能（执行周期，单位：拍）
向零取整函数	向量	simd_trunc(x)	最大ULP为0	47
	128	simd_trunc128(x)	最大ULP为0	121

二元函数

类别		函数名	精度	接口性能（执行周期，单位：拍）
浮点乘加函数	双精度	fma(x)	最大ULP为0	54
	单精度	fmaf(x)	最大ULP为0	39
最大值函数	双精度	fmax(x)	最大ULP为0	101
	单精度	fmaxf(x)	最大ULP为0	124
最小值函数	双精度	fmin(x)	最大ULP为0	101
	单精度	fminf(x)	最大ULP为0	129

13.3.3.2 优化策略

SDAA C提供的数学函数接口，除标量（单精度、双精度）外还提供了向量接口和128接口：

接口类别	输入类型	输出类型	说明
单精度标量接口	单精度浮点数	单精度浮点数	与标准库一致，计算一个单精度浮点数的数学函数值。
双精度标量接口	双精度浮点数	双精度浮点数	与标准库一致，计算一个双精度浮点数的数学函数值。
向量接口	floatv16向量类型	floatv16向量类型	利用SIMD指令，一次计算16个单精度浮点数的数学函数值。
128接口	float型指针	无	利用SIMD指令和循环展开，一次计算128个单精度浮点数的数学函数值，并返回覆盖原地址上数据。

优化策略

需要计算大量连续的单精度浮点数时，采用向量接口或者128接口相比使用单精度标量接口计算每个元素，性能更好。

优化示例

以指数函数exp为例，使用一个SPE对51200个单精度浮点数进行计算，测试并比较以下三种接口的执行周期：

- 单精度标量接口expf。
- 向量接口simd_exp。
- 128接口simd_exp128。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define NUM 51200
#define MIN -80
#define MAX 80

__device__ void init_arr(float *arr)
{
    for (int i = 0; i < NUM; i++) {
        arr[i] = ((float)(MAX - MIN) / (float)NUM) * i + MIN;
    }
}

__global__ void test_expf()
{
    if (threadIdx != 0) return;
    // 在SPM内存中申请一段空间
    float *arr = (float *)malloc(NUM * sizeof(float));
    init_arr(arr);
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < NUM; i++) {
        arr[i] = expf(arr[i]);
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
    // 释放申请的SPM内存空间
    free(arr);
}
```

```
}

__global__ void test_simd_exp()
{
    if (threadIdx != 0) return;
    // 在SPM内存中申请一段空间
    float *arr = (float *)malloc(NUM * sizeof(float));
    init_arr(arr);
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < NUM; i += 16) {
        floatv16 input;
        simd_load(input, arr + i);
        input = simd_exp(input);
        simd_store(input, arr + i);
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
    // 释放申请的SPM内存空间
    free(arr);
}

__global__ void test_simd_exp128()
{
    if (threadIdx != 0) return;
    // 在SPM内存中申请一段空间
    float *arr = (float *)malloc(NUM * sizeof(float));
    init_arr(arr);
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < NUM; i += 128) {
        simd_exp128(arr + i);
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
    // 释放申请的SPM内存空间
    free(arr);
}

int main()
{
    // 设置使用2号SPA
    sdaaSetDevice(2);
    // kernel函数调用
    test_expf<<<1>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // kernel函数调用
    test_simd_exp<<<1>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    // kernel函数调用
    test_simd_exp128<<<1>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
}
```



```
    return 0;
}
```

优化效果

接口名称	51200次数学函数计算的指令周期（拍）	平均每次数学函数计算的指令周期（拍）
单精度标量接口 expf	7885552	154.02
向量接口 simd_exp	650860	12.71
128接口 simd_exp128	354080	6.92

在本示例中，使用向量接口的性能是标量接口的约12.11倍，使用128接口的性能是标量接口的约22.26倍。

13.3.4 程序编译

-
-

13.3.4.1 循环展开

本文主要介绍使用循环展开的方式提高性能优化策略和示例。

原理介绍

循环展开是在编译程序中，通过减少循环迭代次数进而提高程序执行速度的方法。

其核心思想是将操作对象线性化，在一次迭代中访问线性数据中的一组而非单个，使程序在执行更少的迭代次数后，能够有效降低循环的开销。同时，循环展开可以减少指令的相关性，增加循环体内语句并发执行的可能性，有助于进行更好的指令调度；使用循环展开还可以减少分支预测失败的可能性。但循环展开也会使代码膨胀；代码的可读性降低，以及寄存器压力增大。

循环展开可以在程序编写时完成，也可以由编译器自动完成。由于编译器可以更好地了解目标体系结构的工作原理，并能够在有益的情况下展开循环。因此，建议您使用编译器自动展开，如果您手动展开，建议您在测试手动循环展开确实有效再进行。

优化策略

通过以下编译指示，控制是否需启用循环展开功能，以及展开的次数：

- `#pragma clang loop unroll(enable)`：启用基于自动诊断的展开功能。
- `#pragma clang loop unroll(disable)`：禁用基于自动诊断的展开功能。

- `#pragma clang loop unroll(full)`：完全展开一个循环。
- `#pragma clang loop unroll_count(N)`：显式指定展开N次，其中N是编译时的常量。

优化示例

使用不同的编译指示，在 `for` 循环内对1000个整型数进行“+1”的操作，测试以下情况的执行周期并比较：

- 不做任何操作
- `#pragma clang loop unroll(enable)`：启用基于自动诊断的展开功能。
- `#pragma clang loop unroll(disable)`：禁用基于自动诊断的展开功能。
- `#pragma clang loop unroll(full)`：完全展开一个循环。
- `#pragma clang loop unroll_count(1)`：显式指定展开1次。
- `#pragma clang loop unroll_count(2)`：显式指定展开2次。
- `#pragma clang loop unroll_count(3)`：显式指定展开3次。
- `#pragma clang loop unroll_count(4)`：显式指定展开4次。
- `#pragma clang loop unroll_count(5)`：显式指定展开5次。
- `#pragma clang loop unroll_count(6)`：显式指定展开6次。
- `#pragma clang loop unroll_count(7)`：显式指定展开7次。
- `#pragma clang loop unroll_count(8)`：显式指定展开8次。
- `#pragma clang loop unroll_count(9)`：显式指定展开9次。
- `#pragma clang loop unroll_count(10)`：显式指定展开10次。

每个测例均调用10000次“+1”的操作计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define N 10000
#define SIZE 1000

__device__ void init_data(int *data)
```

```
{
    for (int i = 0; i < SIZE; i++) {
        data[i] = i;
    }
}

__device__ void check_result(int *data)
{
    for (int i = 0; i < SIZE; i++) {
        if (data[i] != i + N) {
            printf("calculate error\n");
        }
    }
}

__device__ void loop_unroll_do_nothing()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_enable()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll(enable)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_disable()
{
    int data[SIZE];
    init_data(data);
```

```
// 收集性能数据起始值
perf_start();
for (int i = 0; i < N; i++) {
    #pragma clang loop unroll(disable)
    for (int j = 0; j < SIZE; j++) {
        data[j] = data[j] + 1;
    }
}
// 收集性能数据结束值
perf_stop();

check_result(data);
return;
}

__device__ void loop_unroll_full()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll(full)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_count_1()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll_count(1)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_count_2()
{
    int data[SIZE];
    init_data(data);
```

```
// 收集性能数据起始值
perf_start();
for (int i = 0; i < N; i++) {
    #pragma clang loop unroll_count(2)
    for (int j = 0; j < SIZE; j++) {
        data[j] = data[j] + 1;
    }
}
// 收集性能数据结束值
perf_stop();

check_result(data);
return;
}

__device__ void loop_unroll_count_3()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll_count(3)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_count_4()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll_count(4)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_count_5()
{
    int data[SIZE];
```

```
init_data(data);

// 收集性能数据起始值
perf_start();
for (int i = 0; i < N; i++) {
    #pragma clang loop unroll_count(5)
    for (int j = 0; j < SIZE; j++) {
        data[j] = data[j] + 1;
    }
}
// 收集性能数据结束值
perf_stop();

check_result(data);
return;
}

__device__ void loop_unroll_count_6()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll_count(6)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_count_7()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll_count(7)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_count_8()
{

```

```
int data[SIZE];
init_data(data);

// 收集性能数据起始值
perf_start();
for (int i = 0; i < N; i++) {
    #pragma clang loop unroll_count(8)
    for (int j = 0; j < SIZE; j++) {
        data[j] = data[j] + 1;
    }
}
// 收集性能数据结束值
perf_stop();

check_result(data);
return;
}

__device__ void loop_unroll_count_9()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll_count(9)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__device__ void loop_unroll_count_10()
{
    int data[SIZE];
    init_data(data);

    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        #pragma clang loop unroll_count(10)
        for (int j = 0; j < SIZE; j++) {
            data[j] = data[j] + 1;
        }
    }
    // 收集性能数据结束值
    perf_stop();

    check_result(data);
    return;
}

__global__ void call_kernel()
```

```
{
    if (threadIdx != 0) {
        return;
    }
    loop_unroll_do_nothing();
    loop_unroll_enable();
    loop_unroll_disable();
    loop_unroll_full();
    loop_unroll_count_1();
    loop_unroll_count_2();
    loop_unroll_count_3();
    // 输出性能数据
    perf_print();
    loop_unroll_count_4();
    loop_unroll_count_5();
    loop_unroll_count_6();
    loop_unroll_count_7();
    loop_unroll_count_8();
    loop_unroll_count_9();
    loop_unroll_count_10();
    // 输出性能数据
    perf_print();
    return;
}

int main()
{
    // 设置使用2号SPA
    sdaaSetDevice(0);
    // kernel函数调用
    call_kernel<<<1>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    return 0;
}
```

优化效果

编译指示	平均每次运算的执行周期（拍）
无	9016
#pragma clang loop_unroll_enable	2840
#pragma clang loop_unroll_disable	9016
#pragma clang loop_unroll_full	2840
#pragma clang loop unroll_count(1)	9016
#pragma clang loop unroll_count(2)	6016
#pragma clang loop unroll_count(3)	3688

编译指示	平均每次运算的执行周期（拍）
<code>#pragma clang loop unroll_count(4)</code>	3515
<code>#pragma clang loop unroll_count(5)</code>	2816
<code>#pragma clang loop unroll_count(6)</code>	5164
<code>#pragma clang loop unroll_count(7)</code>	4564
<code>#pragma clang loop unroll_count(8)</code>	2514
<code>#pragma clang loop unroll_count(9)</code>	3019
<code>#pragma clang loop unroll_count(10)</code>	3016

在本示例中，循环展开8次时的效果最佳，性能是不做循环展开的约3.58倍。

13.3.4.2 链接时优化（LTO）

本文主要介绍使用链接时优化（LTO）提高性能的优化策略和示例。

原理介绍

链接时优化（LTO）是对整个SDAA C程序在链接时进行的跨模块间优化，聚焦于分析整个程序代码，属于过程间优化（IPO）的一种。

具体实现原理是将一些pass的优化推迟到链接时再进行，从而使得这些pass具有可跨模块优化的能力。

LTO可以实现：

- **更好的内联：**LTO可以在链接阶段分析整个程序的调用关系，从而实现智能的函数内联决策，提高程序性能。
- **消除死代码：**LTO可以分析整个程序的数据流和控制流，检测并删除不会被执行的代码，从而减小程序大小。
- **传播跨模块的常量：**LTO可以在整个程序范围内传播已知的常量值，从而提高程序性能。
- **消除跨模块的重复代码：**LTO可以在整个程序范围内检测并消除重复的代码片段，从而减小程序大小。

优化策略

通过以下编译选项，控制是否需启用LTO优化：

- `-flto`：启用LTO优化，编译和链接时使用。
- `--sdaa-link`：启用LTO优化，链接时使用，指示链接SDAA C的 `clang-offload-bundler` 捆绑包。

优化示例

分别使用以下两种方式编译示例程序，测试执行周期并比较：

- 不使用LTO优化
- 使用LTO优化

每个测试均调用10000次 `foo1` 函数并计算平均值以消除初次调用时cache影响。

不使用LTO优化

执行以下编译命令，将 `lib.scpp` 编译成静态库，并使用 `main.scpp` 链接该静态库生成可执行文件：

```
# 将lib.scpp编译生成.o
tecocc lib.scpp --sdaa-device-only -c -o nonlto.o
# 将生成的.o打包成静态库
ar -rcs nonlto.a nonlto.o
# 编译main.scpp并链接静态库nonlto.a生成可执行文件
tecocc main.scpp --sdaa-static-device-lib=nonlto.a -o nonlto.out
```

使用LTO优化

执行以下编译命令，将 `lib.scpp` 编译成静态库，并使用 `main.scpp` 链接该静态库生成可执行文件：

```
# 将lib.scpp编译生成.o
tecocc lib.scpp --sdaa-device-only -flto -c -o lto.o
# 将生成的.o打包成静态库
ar -rcs lto.a lto.o
# 编译main.scpp并链接静态库lto.a生成可执行文件
tecocc main.scpp --sdaa-static-device-lib=lto.a -flto --sdaa-link -o lto.out
```

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
// main.scpp
#include <sdaa_perf.h>
using namespace sdaa;

#define N 10000

extern __device__ int foo1();

__device__ void foo4()
{
    printf("this is foo4\n");
}

__device__ void call_foo1()
```

```
{
    int retval;
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        retval = foo1();
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
    if (retval != 42) {
        printf("failed!\n");
    } else {
        printf("pass!\n");
    }
}

__global__ void call_kernel()
{
    // 仅使用0号SPE
    if (threadIdx != 0) {
        return;
    }
    call_foo1();
    return;
}

int main()
{
    // 设置使用0号SPA
    sdaaSetDevice(0);
    // kernel函数调用
    call_kernel<<<1>>>>();
    // 等待kernel函数执行完成
    sdaaDeviceSynchronize();
    return 0;
}

// lib.scpp
extern __device__ void foo4();

__device__ int val = 0;

__device__ void foo2()
{
    val = -1;
}

__device__ int foo3()
{
    foo4();
    return 10;
}

__device__ int foo1()
{
    int data = 0;
    if (val < 0) {
```

```
    data = foo3();
}
data += 42;
return data;
}
```

优化效果

是否使用LTO优化	平均每次运算的执行周期（拍）
使用	341
不使用	707

在本示例中，使用LTO优化的性能是不使用LTO优化的性能约2.07倍。

13.3.4.3 浮点融合

本文主要介绍使用浮点融合提高性能的优化策略和示例。

原理介绍

浮点融合是使用一条乘加/乘减运算操作代替乘法和加/减法两条运算的操作，从而提高程序性能并减小误差。

优化策略

通过以下编译选项控制是否需启用浮点融合功能：

- `-ffp-contract=fast`：启用浮点融合，忽略 `#pragma FP_CONTRACT`。
- `-ffp-contract=on`：根据 `#pragma FP_CONTRACT` 决定是否启用浮点融合。
- `-ffp-contract=off`：禁用浮点融合，忽略 `#pragma FP_CONTRACT`。

默认为启用浮点融合，您可以调整浮点运算写法，拼凑尽量多的乘加类操作，帮助编译器优化。

目前支持以下几种操作的融合：

- 单精度乘加（单精度同号相乘再做加法）
- 双精度乘加（双精度同号相乘再做加法）
- 单精度乘减（单精度同号相乘再做减法）
- 双精度乘减（双精度同号相乘再做减法）
- 单精度负乘加（单精度异号相乘再做加法）
- 双精度负乘加（双精度异号相乘再做加法）

- 单精度负乘减（单精度异号相乘再做减法）
- 双精度负乘减（双精度异号相乘再做减法）

优化示例

使用不同的编译选项，在 `for` 循环内进行100次乘法和加/减操作，测试以下情况的执行周期并比较：

- 单精度乘加
- 双精度乘加
- 单精度乘减
- 双精度乘减
- 单精度负乘加
- 双精度负乘加
- 单精度负乘减
- 双精度负乘减

每个测例均调用10000次乘法和加/减操作计算平均值以消除初次调用时cache影响。

测试环境

TecoDriver和TecoToolKit版本	计算核心频率
v1.2.0	2.36GHz

测试代码

```
#include <sdaa_perf.h>
using namespace sdaa;

#define N 10000
#define SIZE 100
#define NUM 4
#define MAX 200
#define MIN 10

__global__ void test_fmas(float *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = data[1 * SIZE + j] * data[2 * SIZE + j] +
```

```

        data[3 * SIZE + j];
    }
}
// 收集性能数据结束值
perf_stop();
// 输出性能数据
perf_print();
}

__global__ void test_fmss(float *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = data[1 * SIZE + j] * data[2 * SIZE + j] -
                                data[3 * SIZE + j];
        }
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
}

__global__ void test_fnmas(float *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = -((data[1 * SIZE + j]) * data[2 * SIZE + j]) +
                                data[3 * SIZE + j];
        }
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
}

__global__ void test_fnmss(float *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = -((data[1 * SIZE + j]) * data[2 * SIZE + j]) -
                                data[3 * SIZE + j];
        }
    }
}

```

```
}
// 收集性能数据结束值
perf_stop();
// 输出性能数据
perf_print();
}

__global__ void test_fmadd(double *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = data[1 * SIZE + j] * data[2 * SIZE + j] +
                                data[3 * SIZE + j];
        }
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
}

__global__ void test_fmstd(double *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = data[1 * SIZE + j] * data[2 * SIZE + j] -
                                data[3 * SIZE + j];
        }
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
}

__global__ void test_fnmad(double *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = -((data[1 * SIZE + j]) * data[2 * SIZE + j]) +
                                data[3 * SIZE + j];
        }
    }
    // 收集性能数据结束值
```

```

    perf_stop();
    // 输出性能数据
    perf_print();
}

__global__ void test_fnmsd(double *data)
{
    if (threadIdx != 0) {
        return;
    }
    // 收集性能数据起始值
    perf_start();
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < SIZE; j++) {
            data[0 * SIZE + j] = -((data[1 * SIZE + j]) * data[2 * SIZE + j]) -
                                   data[3 * SIZE + j];
        }
    }
    // 收集性能数据结束值
    perf_stop();
    // 输出性能数据
    perf_print();
}

void call_kernel_float(float *src, float (*output)[SIZE])
{
    float *dev_fmas = NULL;
    float *dev_fmss = NULL;
    float *dev_fnmas = NULL;
    float *dev_fnmss = NULL;
    // 设置使用0号SPA
    sdAASetDevice(0);
    // 在Global内存中malloc一段空间
    sdAAMalloc((void **)&dev_fmas, NUM * SIZE * sizeof(float));
    // 将数据从Host端拷贝到Device端
    sdAAMemcpy(dev_fmas, src, NUM * SIZE * sizeof(float),
               sdAAMemcpyHostToDevice);
    // 在Global内存中malloc一段空间
    sdAAMalloc((void **)&dev_fmss, NUM * SIZE * sizeof(float));
    // 将数据从Host端拷贝到Device端
    sdAAMemcpy(dev_fmss, src, NUM * SIZE * sizeof(float),
               sdAAMemcpyHostToDevice);
    // 在Global内存中malloc一段空间
    sdAAMalloc((void **)&dev_fnmas, NUM * SIZE * sizeof(float));
    // 将数据从Host端拷贝到Device端
    sdAAMemcpy(dev_fnmas, src, NUM * SIZE * sizeof(float),
               sdAAMemcpyHostToDevice);
    // 在Global内存中malloc一段空间
    sdAAMalloc((void **)&dev_fnmss, NUM * SIZE * sizeof(float));
    // 将数据从Host端拷贝到Device端
    sdAAMemcpy(dev_fnmss, src, NUM * SIZE * sizeof(float),
               sdAAMemcpyHostToDevice);

    test_fmas<<<1>>>(dev_fmas);
    test_fmss<<<1>>>(dev_fmss);
    test_fnmas<<<1>>>(dev_fnmas);
    test_fnmss<<<1>>>(dev_fnmss);

    // 将数据从Device端拷贝到Host端

```



```

    sdaaMemcpy(output[0], dev_fmas, SIZE * sizeof(float),
                sdaaMemcpyDeviceToHost);
    sdaaMemcpy(output[1], dev_fmss, SIZE * sizeof(float),
                sdaaMemcpyDeviceToHost);
    sdaaMemcpy(output[2], dev_fnmas, SIZE * sizeof(float),
                sdaaMemcpyDeviceToHost);
    sdaaMemcpy(output[3], dev_fnmss, SIZE * sizeof(float),
                sdaaMemcpyDeviceToHost);

    // 释放申请的Global内存
    sdaaFree(dev_fmas);
    sdaaFree(dev_fmss);
    sdaaFree(dev_fnmas);
    sdaaFree(dev_fnmss);
}

void call_kernel_double(double *src, double (*output)[SIZE])
{
    double *dev_fmdd = NULL;
    double *dev_fmdd = NULL;
    double *dev_fnmdd = NULL;
    double *dev_fnmdd = NULL;
    // 设置使用0号SPA
    sdaaSetDevice(0);
    // 在Global内存中malloc一段空间
    sdaaMalloc((void **)&dev_fmdd, NUM * SIZE * sizeof(double));
    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(dev_fmdd, src, NUM * SIZE * sizeof(double),
                sdaaMemcpyHostToDevice);
    // 在Global内存中malloc一段空间
    sdaaMalloc((void **)&dev_fmdd, NUM * SIZE * sizeof(double));
    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(dev_fmdd, src, NUM * SIZE * sizeof(double),
                sdaaMemcpyHostToDevice);
    // 在Global内存中malloc一段空间
    sdaaMalloc((void **)&dev_fnmdd, NUM * SIZE * sizeof(double));
    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(dev_fnmdd, src, NUM * SIZE * sizeof(double),
                sdaaMemcpyHostToDevice);
    // 在Global内存中malloc一段空间
    sdaaMalloc((void **)&dev_fnmdd, NUM * SIZE * sizeof(double));
    // 将数据从Host端拷贝到Device端
    sdaaMemcpy(dev_fnmdd, src, NUM * SIZE * sizeof(double),
                sdaaMemcpyHostToDevice);

    // kernel函数调用
    test_fmdd<<<1>>>(dev_fmdd);
    test_fmdd<<<1>>>(dev_fmdd);
    test_fmdd<<<1>>>(dev_fmdd);
    test_fmdd<<<1>>>(dev_fmdd);

    // 将数据从Device端拷贝到Host端
    sdaaMemcpy(output[0], dev_fmdd, SIZE * sizeof(double),
                sdaaMemcpyDeviceToHost);
    sdaaMemcpy(output[1], dev_fmdd, SIZE * sizeof(double),
                sdaaMemcpyDeviceToHost);
    sdaaMemcpy(output[2], dev_fmdd, SIZE * sizeof(double),
                sdaaMemcpyDeviceToHost);
    sdaaMemcpy(output[3], dev_fmdd, SIZE * sizeof(double),

```

```

        sdaaMemcpyDeviceToHost);

    // 释放申请的Global内存
    sdaaFree(dev_fmadv);
    sdaaFree(dev_fmadv);
    sdaaFree(dev_fmadv);
    sdaaFree(dev_fmadv);
}

void test_float()
{
    int errnum = 0;
    float data[NUM * SIZE];
    float dev_output[NUM][SIZE];
    float host_output[NUM][SIZE];

    // init data
    for (int i = 0; i < NUM * SIZE; i++) {
        data[i] = (((float)rand()) / (float)RAND_MAX) * (MAX - MIN) + MIN;
    }
    for (int i = 0; i < NUM; i++) {
        for (int j = 0; j < SIZE; j++) {
            dev_output[i][j] = 0;
            host_output[i][j] = 0;
        }
    }
    // calculate in host
    for (int i = 0; i < SIZE; i++) {
        host_output[0][i] = data[1 * SIZE + i] * data[2 * SIZE + i] +
            data[3 * SIZE + i];
        host_output[1][i] = data[1 * SIZE + i] * data[2 * SIZE + i] -
            data[3 * SIZE + i];
        host_output[2][i] = -(data[1 * SIZE + i] * data[2 * SIZE + i]) +
            data[3 * SIZE + i];
        host_output[3][i] = -(data[1 * SIZE + i] * data[2 * SIZE + i]) -
            data[3 * SIZE + i];
    }

    call_kernel_float(data, dev_output);

    for (int i = 0; i < NUM; i++) {
        for (int j = 0; j < SIZE; j++) {
            if (fabs(host_output[i][j] - dev_output[i][j]) > 1e-2) {
                printf("host_output[%d][%d] = %f dev_output[%d][%d] = %f\n",
                    i, j, host_output[i][j], i, j, dev_output[i][j]);
                errnum++;
            }
        }
    }

    if (!errnum) {
        printf("all pass in float!\n");
    } else {
        printf("error in float!\n");
    }
}

void test_double()
{

```

```

int errnum = 0;
double data[NUM * SIZE];
double dev_output[NUM][SIZE];
double host_output[NUM][SIZE];

// init data
for (int i = 0; i < NUM * SIZE; i++) {
    data[i] = (((double)rand()) / (double)RAND_MAX) * (MAX - MIN) + MIN;
}
for (int i = 0; i < NUM; i++) {
    for (int j = 0; j < SIZE; j++) {
        dev_output[i][j] = 0;
        host_output[i][j] = 0;
    }
}

// calculate in host
for (int i = 0; i < SIZE; i++) {
    host_output[0][i] = data[1 * SIZE + i] * data[2 * SIZE + i] +
        data[3 * SIZE + i];
    host_output[1][i] = data[1 * SIZE + i] * data[2 * SIZE + i] -
        data[3 * SIZE + i];
    host_output[2][i] = -(data[1 * SIZE + i] * data[2 * SIZE + i]) +
        data[3 * SIZE + i];
    host_output[3][i] = -(data[1 * SIZE + i] * data[2 * SIZE + i]) -
        data[3 * SIZE + i];
}

call_kernel_double(data, dev_output);

for (int i = 0; i < NUM; i++) {
    for (int j = 0; j < SIZE; j++) {
        if (fabs(host_output[i][j] - dev_output[i][j]) > 1e-9) {
            printf("host_output[%d][%d] = %lf dev_output[%d][%d] = %lf\n",
                i, j, host_output[i][j], i, j, dev_output[i][j]);
            errnum++;
        }
    }
}

if (!errnum) {
    printf("all pass in double!\n");
} else {
    printf("error in double!\n");
}
}

int main()
{
    test_float();
    test_double();
    return 0;
}

```

优化效果

编译选项	运算类型	平均每次运算的执行周期（拍）
-ffp- contract=fast	单精度乘加	35389
	单精度乘减	35421
	单精度负乘加	35389
	单精度负乘减	35392
	双精度乘加	35738
	双精度乘减	35851
	双精度负乘加	35742
	双精度负乘减	35856
-ffp- contract=on	单精度乘加	69008
	单精度乘减	69003
	单精度负乘加	69008
	单精度负乘减	69009
	双精度乘加	69752
	双精度乘减	69751
	双精度负乘加	69721
	双精度负乘减	69852
-ffp- contract=off	单精度乘加	69008
	单精度乘减	69009
	单精度负乘加	69008
	单精度负乘减	69007
	双精度乘加	69728
	双精度乘减	69727
	双精度负乘加	69756
	双精度负乘减	69847

在本示例中，使用浮点融合的性能是不使用浮点融合的性能约1.95倍，不同的数据的类型与不同的操作类型之间性能差异不大。

14. 接口示例库

为帮助您更好的理解及使用SDAA C提供的内存分配、数据搬运、矩阵乘、原子操作等函数接口以及正弦计算、余弦计算、高斯误差计算等数学计算接口，本文为每个接口提供了完整的使用示例。单击以下内容，获取相关示例的详细信息。

[下载完整示例](#)

15. 入门实战

15.1 向量运算

15.1.1 概述

本例使用LTO模式进行编译，在太初AI加速卡上实现向量相关的运算（包括向量装载、向量平方根运算、向量指数运算、向量乘、向量加、向量小于等于比较运算以及向量存储操作）。完整程序请参考[完整程序](#)。

主机端通过SDAARuntime接口分配设备端的Global存储空间，并将数据从主机端内存拷贝到设备端的Global存储空间。在设备端通过向量装载接口给向量赋值，通过向量平方根函数计算向量的平方根，通过指数运算函数对平方根运算的结果做指数运算，再通过 `*` 运算符和 `+` 运算符分别做向量乘和向量加运算，最后比较指数运算的结果是否小于等于向量乘和向量加运算的结果。在主机端通过SDAARuntime接口将Global存储空间上运算结果拷贝到主机端内存，并打印运算结果。



图1 向量运算的编程逻辑

15.1.2 编程思路

主机端

1. 分配主机端内存并赋值。
2. 主机端与设备端的数据传输。
 - a. 设置计算核心阵列SPA号。
 - b. 分配Global存储空间。
 - c. 从主机端内存向Global存储空间传输数据。
 - d. 调用Kernel函数。
 - e. 从Global存储空间拷贝计算结果到主机端内存。
 - f. 释放Global存储空间。
3. 打印运算结果。
4. 释放主机端内存。

设备端

1. 向量装载。
2. 向量平方根运算。
3. 向量指数运算。
4. 向量乘和向量加运算。
5. 向量小于等于比较。
6. 向量存储。

15.1.3 编写程序

15.1.3.1 主机端

主机端程序设计思路

1. 分配主机端内存并赋值：

- a. 在主机端通过 `malloc` 接口分配2个主机端存储空间 `h_data1, h_data2`，大小都是为 `SIZE`。其中：`SIZE` 值为 `16 * sizeof(float)`。
- b. 将已分配的2个存储空间的起始地址分别返回给指针 `*h_data1, *h_data2`。`*h_data1, *h_data2` 均为指向单精度数据类型的指针。
- c. 在Host端定义3个const常量 `COUNT`，`SIZE`，`TMPVAL`。其中：
 - `COUNT`：表示数据的个数。
 - `SIZE`：表示 `COUNT` 个单精度数据类型的字节大小。
 - `TMPVAL`：表示值为0.01的系数。
- d. 对 `h_data1` 的16个单精度赋值，依次为各元素的索引值；对 `h_weight` 的16个单精度赋值，依次为各元素索引值的平方，再乘以 `TMPVAL` 系数。

2. 主机端与设备端的数据传输：`with_sdaa` 是主机端的函数，被 `main` 函数调用。`with_sdaa` 调用 `Kernel`函数，通过SDAARuntime接口将主机端的数据与Device端Global存储空间的数据进行传输。

- a. 设置计算核阵列SPA号：调用 `sdaaSetDevice` 接口指定Kernel函数运行在设备端具体哪一个计算核心阵列SPA上。
- b. 分配Global存储空间：通过 `sdaaMalloc` 接口在设备端分配2个Global存储空间 (`d_data1, d_data2`)。
- c. 从主机端内存向Global存储空间传输数据：通过 `sdaaMemcpy` 接口将主机端 (`h_data1, h_data2`) 分别拷贝到设备端Global存储空间 (`d_data1, d_data2`)。
- d. 调用Kernel函数：设置计算核心阵列SPA的个数，调用名为 `test` 的核函数，在设备端执行计算任务。
- e. 从Global存储空间拷贝计算结果到主机端内存：计算任务完成后，调用 `sdaaDeviceSynchronize` 接口同步设备，再通过 `sdaaMemcpy` 接口将设备端Global存储空间 (`d_data1, d_data2`) 的数据分别拷贝到Host端内存 (`h_data1, h_data2`)。

f. 释放Global存储空间：通过 `sdaaFree` 接口释放设备端的Global存储空间。

主机端与设备端的数据传输程序如下：

```
/*
 * with_sdaa在设备端分配2个Global存储空间: d_data1, d_data2.
 * 将主机端h_data1, h_data2的数据拷贝至设备端Global存储空间
 * 进行向量操作和指数运算, 再将计算完的数据从设备端拷贝回来。
 */
static void with_sdaa(float *h_data1, float *h_data2, int count, int size)
{
    // 指定Kernel函数运行在2号计算核心阵列SPA上
    sdaaSetDevice(2);
    float *d_data1 = NULL;
    float *d_data2 = NULL;

    // 在设备端的Global存储空间分配大小为size的连续内存,
    // 并将已分配内存的起始位置返回给指针d_data1.
    sdaaMalloc((void **)&d_data1, size);

    // 将主机端内存h_data1的数据拷贝到设备端Global存储空间d_data1, 大小为size,
    // 方向是sdaaMemcpyHostToDevice.
    sdaaMemcpy(d_data1, h_data1, size, sdaaMemcpyHostToDevice);
    sdaaMalloc((void **)&d_data2, size);
    sdaaMemcpy(d_data2, h_data2, size, sdaaMemcpyHostToDevice);

    // 设置计算核心阵列SPA的数量为1, 调用名为test的核函数, 在设备端执行计算任务。
    // 实参d_data1和d_data2是Device端Global空间上的地址, count是一个整型的变量。
    test<<<1>>>>(d_data1, d_data2, count);

    // 同步设备
    sdaaDeviceSynchronize();

    // 将Global存储空间d_data1的数据拷贝到主机端h_data1, 大小为size,
    // 方向是sdaaMemcpyDeviceToHost.
    sdaaMemcpy(h_data1, d_data1, size, sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_data2, d_data2, size, sdaaMemcpyDeviceToHost);

    sdaaFree(d_data1); // 释放d_data1指针指向的Global存储空间
    sdaaFree(d_data2); // 释放d_data2指针指向的Global存储空间
}
```

3. 打印运算结果：打印向量运算后的结果。 `h_data1` 的数据是向量指数运算的结果， `h_data2` 的数据是向量乘加运算的结果。

4. 释放主机端内存：通过 `free` 接口释放主机端内存 `(h_data1, h_data2)` 。

主机端程序

? 说明：

Kernel函数 `test` 和Host端的代码不在同一个.scpp文件时，需要在Host端代码所在的文件声明该Kernel函数，如：

```
extern __global__ void test(float *data1, float *data2, int number); // Kernel函数的声明
```

```
// Kernel函数的声明
extern __global__ void test(float *data1, float *data2, int number);

static void with_sdaa(float *h_data1, float *h_data2, int count, int size)
{
    // 指定Kernel函数运行在2号计算核心阵列SPA上
    sdaaSetDevice(2);
    float *d_data1 = NULL;
    float *d_data2 = NULL;

    // 在设备端的Global存储空间分配大小为size的连续内存,
    // 并将已分配内存的起始位置返回给指针d_data1.
    sdaaMalloc((void **)&d_data1, size);

    // 将主机端内存h_data1的数据拷贝到设备端Global存储空间d_data1, 大小为size,
    // 方向是sdaaMemcpyHostToDevice.
    sdaaMemcpy(d_data1, h_data1, size, sdaaMemcpyHostToDevice);
    sdaaMalloc((void **)&d_data2, size);
    sdaaMemcpy(d_data2, h_data2, size, sdaaMemcpyHostToDevice);

    // 设置计算核心阵列SPA的数量为1,调用名为test的核函数, 在设备端执行计算任务。
    // 实参d_data1和d_data2是Device端Global空间上的地址, count是一个整型的变量。
    test<<<1>>>>(d_data1, d_data2, count);

    // 同步设备
    sdaaDeviceSynchronize();

    // 将Global存储空间d_data1的数据拷贝到主机端h_data1, 大小为size,
    // 方向是sdaaMemcpyDeviceToHost.
    sdaaMemcpy(h_data1, d_data1, size, sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_data2, d_data2, size, sdaaMemcpyDeviceToHost);

    sdaaFree(d_data1); // 释放d_data1指针指向的Global存储空间
    sdaaFree(d_data2); // 释放d_data2指针指向的Global存储空间
}

int main()
{
    const int COUNT = 16; // 表示每个空间单精度数据的个数
    const int SIZE = (COUNT * sizeof(float)); // 16个单精度数据类型的字节大小
    const float TMPVAL = 0.01; // 用于给h_data2空间赋值

    // 在Host端分配空间h_data1, h_data2
    // 在Host端分配大小SIZE的空间, 并把已分配空间的起始地址返回给指针h_data1.
    float *h_data1 = (float *)malloc(SIZE);
    float *h_data2 = (float *)malloc(SIZE);

    // 设置值
    for (int i = 0; i < COUNT; i++) {
        h_data1[i] = i; // 每个元素的值为各元素的索引值
        h_data2[i] = i * i * TMPVAL; // 每个元素的值为各元素索引值的
        // 平方 * TMPVAL系数
    }
}
```

```

// 调用with_sdaa实现主机端与设备端的数据传输。
// h_data1和h_data2是Host端空间上的地址，
// COUNT表示每个空间有COUNT个单精度数据，SIZE表示每个空间的大小。
with_sdaa(h_data1, h_data2, COUNT, SIZE);

printf("h_data1 :");
for (int i = 0; i < COUNT; i++) {
    printf("%f |", h_data1[i]); // 打印运算之后h_data1空间中每个元素的值
}
printf("\nh_data2 :");
for (int i = 0; i < COUNT; i++) {
    printf("%f |", h_data2[i]); // 打印运算之后h_data2空间中每个元素的值
}
printf("\n----test finish----\n");

free(h_data1); // 释放h_data1指针指向的存储空间
free(h_data2); // 释放h_data2指针指向的存储空间
return 0;
}

```

15.1.3.2 设备端

设备端程序设计思路

设备端Kernel函数指定0号计算核心SPE做向量相关运算，分别是向量装载、向量平方根运算、向量指数运算、向量乘、向量加、向量小于等于比较运算以及向量存储操作。

? 说明：

1. 使用__device__关键字修饰，说明该变量或函数是在设备端的Global存储空间，主机端不允许访问。
2. 使用__local__关键字修饰，说明该变量或函数是在设备端的SPM存储空间，主机端不允许访问。
3. 使用__global__关键字修饰，说明函数是Kernel函数，允许主机端调用，但不允许设备端调用。返回值类型仅为void类型。
4. threadIdx用于读取计算核心SPE号。

1. 向量装载：通过 `simd_load` 接口将64字节长度的数据从连续内存区域中对齐装入到向量中。
2. 向量平方根运算：通过 `simd_sqrt` 接口计算向量平方根。
3. 向量指数运算：通过 `simd_exp` 接口对平方根运算的结果做向量指数运算。
4. 向量乘和向量加运算：通过 `*` 运算符做向量乘运算， `+` 运算符做向量加运算。
5. 向量小于等于比较：Kernel函数调用设备端的 `test_vfcmp` 函数比较向量指数运算的结果是否小于等于向量乘加运算的结果。
 - a. 向量小于等于比较操作：通过 `<=` 运算符做向量小于等于比较操作。逐元素比较两个向量，如果满足条件，就返回非0值（如：-1）；否则返回0。
 - b. 打印结果：打印比较结果。

```

/*
 * 用__device__关键字修饰,说明test_vfcmp函数是在Device端global空间定义的。
 * 函数返回值为void类型,形参是floatv16数据类型。
 * 函数实现了:通过'<='操作符逐元素比较参数vdata1和vdata2,如果vdata1某元素小于等于
 * vdata2对应元素,则返回非0值(如:-1)写入vcmpres对应元素;否则返回0写入vcmpres对应元素。
 */
__device__ void test_vfcmp(floatv16 vdata1, floatv16 vdata2)
{
    // vcmpres用于记录向量小于等于比较的结果
    floatv16 vcmpres;

    // 使用'<='运算符实现向量小于等于比较
    vcmpres = (floatv16)(vdata1 <= vdata2);

    printf("向量指数运算的结果和向量乘加运算的结果做小于等于比较操作,比较的结果输出如下:\n");
    for (int i = 0; i < 16; i++) {
        printf("%f |", vcmpres[i]); // 打印向量小于等于比较结果
    }
    printf("\n\n");
}

```

6. 向量存储:通过 `simd_store` 接口将向量中的数据对齐存储到连续内存区域中。

设备端程序

```

// 在Device端SPM空间定义5个floatv16数据类型的变量
__local__ floatv16 __A, __B, vres1, vres2, vres0;

__device__ void test_vfcmp(floatv16 vdata1, floatv16 vdata2)
{
    // 逐元素比较参数vdata1和vdata2,如果vdata1某元素小于等于vdata2对应元素,
    // 则返回非零值(如:-1)写入vcmpres对应元素;否则返回0写入vcmpres对应元素。
    floatv16 vcmpres;
    // 使用'<='运算符实现向量小于等于比较
    vcmpres = (floatv16)(vdata1 <= vdata2);
    printf("向量指数运算的结果和向量乘加运算的结果做小于等于比较操作,比较的结果输出如下:\n");
    for (int i = 0; i < 16; i++) {
        printf("%f |", vcmpres[i]);
    }
    printf("\n\n");
}

__global__ void test(float *data1, float *data2, int number)
{
    // 指定0号计算核心SPE
    if (threadIdx == 0) {
        simd_load(__A, data1); // 将data1对齐装入向量__A
        simd_load(__B, data2); // 将data2对齐装入向量__B
        vres0 = simd_sqrt(__B); // 计算__B的平方根,结果写回向量vres0
        vres1 = simd_exp(vres0); // 对向量vres0做指数运算,结果写回向量vres1
        vres2 = (__A * vres0 + __B); // 向量乘和向量加运算,结果写回向量vres2

        // 调用Device端test_vfcmp函数比较vres1是否小于等于vres2
        test_vfcmp(vres1, vres2);
        simd_store(vres1, data1); // 将指数运算的结果vres1对齐存储到data1
        simd_store(vres2, data2); // 将向量乘加运算的结果vres2对齐存储到data2
    }
}

```

```

    }
}

```

15.1.3.3 完整程序

向量运算的主机端代码 `host.scpp` 如下：

```

// Kernel函数的声明
extern __global__ void test(float *data1, float *data2, int number);

static void with_sdaa(float *h_data1, float *h_data2, int count, int size)
{
    // 指定Kernel函数运行在2号计算核心阵列SPA上
    sdaaSetDevice(2);
    float *d_data1 = NULL;
    float *d_data2 = NULL;

    // 在设备端的Global存储空间分配大小为size的连续内存,
    // 并将已分配内存的起始位置返回给指针d_data1.
    sdaaMalloc((void **)&d_data1, size);

    // 将主机端内存h_data1的数据拷贝到设备端Global存储空间d_data1, 大小为size,
    // 方向是sdaaMemcpyHostToDevice.
    sdaaMemcpy(d_data1, h_data1, size, sdaaMemcpyHostToDevice);
    sdaaMalloc((void **)&d_data2, size);
    sdaaMemcpy(d_data2, h_data2, size, sdaaMemcpyHostToDevice);

    // 设置计算核心阵列SPA的数量为1,调用名为test的核函数, 在设备端执行计算任务。
    // 实参d_data1和d_data2是Device端Global空间上的地址, count是一个整型的变量。
    test<<<1>>>(d_data1, d_data2, count);

    // 同步设备
    sdaaDeviceSynchronize();

    // 将Global存储空间d_data1的数据拷贝到主机端h_data1, 大小为size,
    // 方向是sdaaMemcpyDeviceToHost.
    sdaaMemcpy(h_data1, d_data1, size, sdaaMemcpyDeviceToHost);
    sdaaMemcpy(h_data2, d_data2, size, sdaaMemcpyDeviceToHost);

    sdaaFree(d_data1); // 释放d_data1指针指向的Global存储空间
    sdaaFree(d_data2); // 释放d_data2指针指向的Global存储空间
}

int main()
{
    const int COUNT = 16; // 表示每个空间单精度数据的个数
    const int SIZE = (COUNT * sizeof(float)); // 16个单精度数据类型的字节大小
    const float TMPVAL = 0.01; // 用于给h_data2空间赋值

    // 在Host端分配空间h_data1, h_data2
    // 在Host端分配大小SIZE的空间, 并把已分配空间的起始地址返回给指针h_data1.
    float *h_data1 = (float *)malloc(SIZE);
    float *h_data2 = (float *)malloc(SIZE);

    // 设置值
    for (int i = 0; i < COUNT; i++) {

```

```

        h_data1[i] = i;           // 每个元素的值为各元素的索引值
        h_data2[i] = i * i * TMPVAL; // 每个元素的值为各元素索引值的
                                    // 平方 * TMPVAL系数
    }

    // 调用with_sdaa实现主机端与设备端的数据传输。
    // h_data1和h_data2是Host端空间上的地址，
    // COUNT表示每个空间有COUNT个单精度数据，SIZE表示每个空间的大小。
    with_sdaa(h_data1, h_data2, COUNT, SIZE);

    printf("h_data1 :");
    for (int i = 0; i < COUNT; i++) {
        printf("%f |", h_data1[i]); // 打印运算之后h_data1空间中每个元素的值
    }
    printf("\nh_data2 :");
    for (int i = 0; i < COUNT; i++) {
        printf("%f |", h_data2[i]); // 打印运算之后h_data2空间中每个元素的值
    }
    printf("\n----test finish----\n");

    free(h_data1); // 释放h_data1指针指向的存储空间
    free(h_data2); // 释放h_data2指针指向的存储空间
    return 0;
}

```

向量运算的设备端代码 `device.scpp` 如下：

```

// 在Device端SPM空间定义5个floatv16数据类型的变量
__local__ floatv16 __A, __B, vres1, vres2, vres0;

__device__ void test_vfcmp(floatv16 vdata1, floatv16 vdata2)
{
    // 逐元素比较参数vdata1和vdata2，如果vdata1某元素小于等于vdata2对应元素，
    // 则返回非零值（如：-1）写入vcmpres对应元素；否则返回0写入vcmpres对应元素。
    floatv16 vcmpres;
    // 使用 '<=' 运算符实现向量小于等于比较
    vcmpres = (floatv16)(vdata1 <= vdata2);
    printf("向量指数运算的结果和向量乘法运算的结果做小于等于比较操作，比较的结果输出如下：\n");
    for (int i = 0; i < 16; i++) {
        printf("%f |", vcmpres[i]);
    }
    printf("\n\n");
}

__global__ void test(float *data1, float *data2, int number)
{
    // 指定0号计算核心SPE
    if (threadIdx == 0) {
        simd_load(__A, data1); // 将data1对齐装入向量__A
        simd_load(__B, data2); // 将data2对齐装入向量__B
        vres0 = simd_sqrt(__B); // 计算__B的平方根，结果写回向量vres0
        vres1 = simd_exp(vres0); // 对向量vres0做指数运算，结果写回向量vres1
        vres2 = (__A * vres0 + __B); // 向量乘和向量加运算，结果写回向量vres2

        // 调用Device端test_vfcmp函数比较vres1是否小于等于vres2
        test_vfcmp(vres1, vres2);
        simd_store(vres1, data1); // 将指数运算的结果vres1对齐存储到data1
        simd_store(vres2, data2); // 将向量乘法运算的结果vres2对齐存储到data2
    }
}

```

```
}  
}
```

15.1.4 程序编译

由于 `__device__` 修饰的函数和Kernel函数 `test` 不在同一个 `.scpp` 文件，因此需要使用LTO模式进行编译，编译选项增加 `-flto --sdaa-link`。

1.执行以下编译命令，将源文件 `device.scpp`、`host.scpp` 生成可执行文件 `lto.out`。

```
tecocc device.scpp host.scpp -flto --sdaa-link -o lto.out
```

2.执行以下命令，运行程序：

```
./lto.out
```

执行结果

向量指数运算的结果和向量乘加运算的结果做小于等于比较操作，比较的结果输出如下：

```
0.000000 |0.000000 |0.000000 |0.000000 |-1.000000 |-1.000000 |-1.000000 |-1.000000 |-1.000000 |-1.000000  
0 |-1.000000 |-1.000000 |-1.000000 |-1.000000 |-1.000000 |-1.000000 |
```

```
h_data1 :1.000000 |1.105171 |1.221403 |1.349859 |1.491825 |1.648721 |1.822119 |2.013753 |2.225541 |2.45  
9603 |2.718282 |3.004166 |3.320117 |3.669297 |4.055200 |4.481688 |
```

```
h_data2 :0.000000 |0.110000 |0.440000 |0.990000 |1.760000 |2.750000 |3.960000 |5.390000 |7.040000 |8.91  
0000 |10.999999 |13.309999 |15.839999 |18.590000 |21.559999 |24.749998 |
```

```
----test finish----
```

根据向量小于等于比较结果可知，满足条件的对应元素返回非0值（-1），不满足条件的对应元素返回0。

`h_data1` 的数据是向量指数运算的结果，`h_data2` 的数据是向量乘加运算的结果。

15.2 使用SPMD完成矩阵乘

15.2.1 概述

本例将在太初AI加速卡上通过SPMD编程模型实现矩阵乘操作。完整程序请参考[完整程序](#)。

主机端通过SDAARuntime接口分配设备端的Global存储空间，并将数据从主机端拷贝到设备端的Global存储空间。在设备端通过SPMD编程模型将计算任务按照可使用的计算核心SPE的数量进行切分。每个计算核心SPE通过SDAA C提供的数据搬运接口`memcpy`将所需的数据从Global存储空间搬运到SPM存储空间进行矩阵乘运算，最后将计算结果进行组合从而完成整个计算任务的求解。设备端完成计算后通过SDAARuntime接口将Global存储空间的计算结果拷贝到主机端内存。

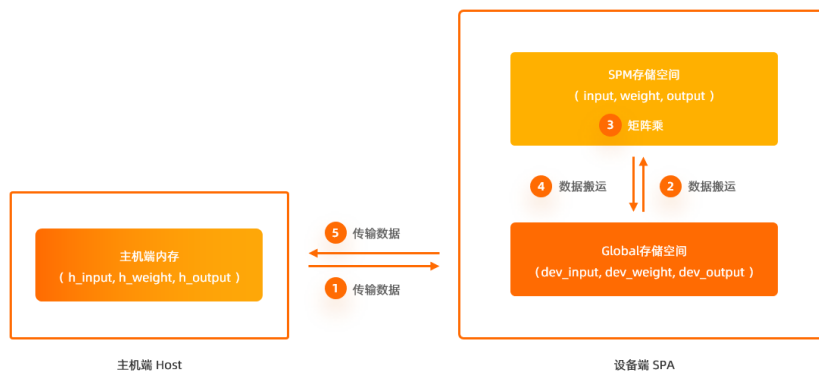


图1 矩阵乘的编程逻辑

15.2.2 编程思路

主机端

1. 分配主机端内存并通过随机数进行初始化操作。
2. 主机端与设备端的数据传输。
 - a. 设置计算核心阵列SPA号。
 - b. 分配Global存储空间。
 - c. 从主机端内存向Global存储空间传输数据。
 - d. 调用Kernel函数。
 - e. 将计算结果从Global存储空间传回主机端内存。
3. 验证计算结果的正确性。
4. 释放Global存储空间与主机端内存。

设备端

1. 计算每个计算核心SPE需要承担的计算任务。
2. 分配SPM存储空间。
3. 将计算数据从Global存储空间拷贝到SPM存储空间。
4. 执行矩阵乘操作。
5. 将计算结果从SPM存储空间拷贝到Global存储空间对应位置。
6. 释放SPM存储空间。

15.2.3 编写程序

15.2.3.1 主机端

主机端程序设计思路

1. 分配主机端内存并通过随机数进行初始化操作。

- a. 在主机端通过malloc接口分配4个主机端存储空间 `h_input` 、 `h_weight` 、 `h_output` 、 `h_output_reference` 。
 - `h_input` : 输入矩阵。
 - `h_weight` : 权重矩阵。
 - `h_output` : 输出矩阵。
 - `h_output_reference` : 输出矩阵的参考结果。
- b. 通过随机数将输入矩阵与权重矩阵进行初始化操作。
- c. 在Host端模拟矩阵乘计算, 并将计算结果存放到 `h_output_reference` 。

2. 主机端与设备端的数据传输

- a. 设置计算核心阵列SPA号: 通过`sdaaSetDevice`接口指定Kernel函数运行在哪一个计算核心阵列上。
- b. 分配Global存储空间: 通过`sdaaMalloc`接口在设备端分配3个Global存储空间 (`dev_input` 、 `dev_weight` 、 `dev_output`) 。
 - `dev_input` : Global存储空间内的输入矩阵。
 - `dev_weight` : Global存储空间内的权重矩阵。
 - `dev_output` : Global存储空间内的输出矩阵。
- c. 数据传输: 通过`sdaaMemcpy`接口将Host端内存 (`h_input` 、 `h_weight` 、 `h_output`) 对应拷贝到Device端Global存储空间 (`dev_input` 、 `dev_weight` 、 `dev_output`) 。
- d. 调用Kernel函数: 通过`<<<...>>>`调用Kernel函数。
- e. 拷贝计算结果: 通过`sdaaDeviceSynchronize`接口等待计算核心SPE完成之前的所有请求任务, 再通过`sdaaMemcpy`接口将Device端Global存储空间`dev_output`中的数据拷贝到Host内存 `h_output` 。
- f. 计算结果对比: 将Kernel的计算结果 `h_output` 与参考矩阵乘计算结果 `h_output_reference` 进行对比。
- g. 释放申请的Device端Global存储空间: 通过`sdaaFree`接口释放Device端的Global存储空间。
- h. 释放申请的Host端存储空间: 通过`free`接口释放申请的Host端存储空间。

主机端程序

```
#include <time.h>

#define M 4096          // 输入矩阵和输出矩阵的行数
#define K 64            // 输入矩阵列数, 也是权重矩阵行数
```

```

#define N 32                // 权重矩阵和输出矩阵的列数

int main()
{
    // 分配Host端空间
    // h_input为输入矩阵, h_weight为权重矩阵, h_output为计算结果矩阵
    // h_output_reference为Host模拟矩阵乘计算, 计算结果存储矩阵
    short* h_input = (short*)malloc(M * K * sizeof(short));
    short* h_weight = (short*)malloc(K * N * sizeof(short));
    short* h_output = (short*)malloc(M * N * sizeof(short));
    short* h_output_reference = (short*)malloc(M * N * sizeof(short));

    srand(time(NULL));
    // 为Host端输入矩阵进行初始化
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < K; j++) {
            h_input[i * K + j] = rand() % 10;
        }
    }

    // 为Host端权重矩阵进行初始化
    for (int i = 0; i < K; i++) {
        for (int j = 0; j < N; j++) {
            h_weight[i * N + j] = rand() % 5;
        }
    }

    // 在Host模拟矩阵乘操作, 并将结果存放到h_output_reference
    memset(h_output_reference, 0, M * N * sizeof(short));
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            for (int k = 0; k < K; k++) {
                h_output_reference[i * N + j] +=
                    h_input[i * K + k] * h_weight[k * N + j];
            }
        }
    }

    // 配置使用0号计算核心阵列
    sdAASetDevice(0);
    // 声明Device端地址
    // dev_input为输入矩阵, dev_weight为权重矩阵, dev_output为计算结果矩阵
    char* dev_input = NULL;
    char* dev_weight = NULL;
    char* dev_output = NULL;

    // 使用sdAAMalloc为Device端地址进行空间分配
    sdAAMalloc((void**)&dev_input, M * K * sizeof(short));
    sdAAMalloc((void**)&dev_weight, K * N * sizeof(short));
    sdAAMalloc((void**)&dev_output, M * N * sizeof(short));

    // 使用sdAAMemcpy将Host端内存拷贝到Device端
    sdAAMemcpy(dev_input, h_input, M * K * sizeof(short), sdAAMemcpyHostToDevice);
    sdAAMemcpy(dev_weight, h_weight, K * N * sizeof(short), sdAAMemcpyHostToDevice);

    // 启动核函数调用
    matmul_func<<<1>>>>(dev_input, dev_weight, dev_output, M, K, N);

    // 等待计算核心SPE完成之前的所有请求任务

```

```

sdaaDeviceSynchronize();

// 使用sdaaMemcpy将Device端矩阵乘计算结果拷贝到Host端
sdaaMemcpy(h_output, dev_output, M * N * sizeof(short), sdaaMemcpyDeviceToHost);

// 将Device端矩阵乘计算结果与Host端计算结果进行对比
// 验证失败则打印对应日志
for (int i = 0; i < M * N; i++) {
    if(h_output[i] != h_output_reference[i]) {
        printf("find mismatching data on line %d: ace: %d, serial: %d\n",
            i, (int)h_output[i], (int)h_output_reference[i]);
        break;
    }

    // 如果验证成功, 则打印对应日志
    if (i == M * N - 1) {
        printf("matmul result compare ok\n");
    }
}

// 释放在Device端和Host申请的内存
free(h_output_reference);
free(h_output);
free(h_weight);
free(h_input);
sdaaFree(dev_output);
sdaaFree(dev_weight);
sdaaFree(dev_input);

return 0;
}

```

15.2.3.2 设备端

设备端程序设计思路

设备端Kernel函数通过SPMD编程模型计算单个计算核心需要完成的计算子任务，每个计算核心SPE通过矩阵乘接口完成子任务的矩阵乘计算。最后将计算结果子集进行合并完成整体计算任务的求解。

? 说明：

1. 使用__device__关键字修饰，说明该变量或函数是在设备端的Global存储空间，主机端不允许访问。
2. 使用__global__关键字修饰，说明函数是Kernel函数，允许主机端调用，但不允许设备端调用。返回值类型仅为void类型。
3. threadDim用于获取计算核心阵列可用的计算核心资源数量。
4. threadIdx用于读取计算核心SPE的ID。

1. 通过SPMD编程模型将计算任务进行拆分：通过threadDim计算每个计算核心SPE所承担的计算子任务。
2. 申请SPM存储空间：通过malloc申请SPM存储空间（ input 、 weight 、 output ）。

- a. `input` : 输入矩阵。
 - b. `weight` : 权重矩阵。
 - c. `output` : 输出矩阵。
3. 从Global存储空间向SPM存储空间传输数据：通过非阻塞型DMA数据搬运接口将数据从Global存储空间拷贝到SPM存储空间。
4. 矩阵乘操作：通过非阻塞型矩阵乘接口完成子任务的矩阵乘计算。

? 说明：

调用矩阵乘接口需要include头文件 `#include <sdaa_matmul.h>`

5. 数据搬运：通过阻塞型DMA数据搬运接口`memcpy`将子任务的计算结果拷贝到Global存储空间的对应位置。
6. 释放SPM存储空间：通过`free`接口释放申请的SPM存储空间。

设备端程序

```
#include <sdaa_matmul.h>
using namespace sdaa;

__device__ void matmul_compute_func(short *input, short *weight, short *output, int m, int k, int n)
{
    // 创建MatmulHandle以配置矩阵乘接口参数。此例中使用默认配置
    MatmulHandle handle;

    // 初始化矩阵乘接口，并配置输入和输出数据的类型
    matmul_init(handle, MatmulShortToShort);

    // 设置之后的计算不为最后一次计算，
    // 即计算结果会累加并缓存在矩阵乘计算单元的累加缓冲区内
    matmul_set_flushing_output(handle, false);

    // 将权重矩阵前32行加载到矩阵乘计算单元上
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);
    // 等待权重矩阵数据加载完成，此后可覆盖weight上前32行数据
    matmul_wait_loading_weight(handle);

    // 计算加载输入矩阵的跨步步长，以64字节为单位
    const unsigned int stride = k / MatmulK32 - 1;

    // 跨步加载输入矩阵（此处相当于加载输入矩阵左半）
    // 并计算与权重矩阵的乘积
    matmul_compute(handle, input, m, MatmulK32, stride);
    // 等待输入矩阵加载完成，此后可覆盖input上左半数据
    matmul_wait_loading_input(handle);

    // 使能下一次计算为最后一次计算
    // 计算完后会输出累加缓冲区内数据
    matmul_set_flushing_output(handle, true);

    // 将权重矩阵后32行加载到矩阵乘计算单元上
```

```

matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);
// 等待权重矩阵数据加载完成, 此后可覆写weight上后32行数据
matmul_wait_loading_weight(handle);

// 跨步加载输入矩阵 (此处相当于加载输入矩阵右半)
// 并计算与权重矩阵的乘积
matmul_compute(handle, input + MatmulK32, m, MatmulK32, stride);
// 等待输入矩阵加载完成, 此后可覆写input上右半数据
matmul_wait_loading_input(handle);

// 将矩阵乘运算结果写回到SPM指定位置
matmul_store(handle, output, m, MatmulN32);

// 等待运算结果写回完成
matmul_wait(handle);
}

__global__ void matmul_func(char *data_input, char *data_weight, char *data_output, int m, int k, int
n)
{
    // 计算每个计算核心SPE所需要承担的计算任务
    unsigned int slice = m / threadDim;

    // 申请SPM存储空间
    short *input = (short*)malloc(slice * k * sizeof(short));
    short *weight = (short*)malloc(k * n * sizeof(short));
    short *output = (short*)malloc(slice * n * sizeof(short));

    // 将数据从Global搬运输入矩阵和权重矩阵到SPM
    memcpy_async(input, data_input + threadIdx * slice * k * sizeof(short), slice * k * sizeof(short),
0, {0, 0}, MemcpyGlobalToSpm);
    memcpy_async(weight, data_weight, k * n * sizeof(short), 0, {0, 0}, MemcpyGlobalToSpm);
    // 等待异步拷贝完成
    memcpy_wait();

    // 矩阵乘计算
    matmul_compute_func(input, weight, output, slice, k, n);

    // 将输出矩阵从SPM搬运回Global
    memcpy(data_output + threadIdx * slice * n * sizeof(short), output, slice * n * sizeof(short));

    // 释放在SPM申请的内存
    free(output);
    free(weight);
    free(input);
}

```

15.2.3.3 完整程序

使用SPMD完成矩阵乘完整程序 (matmul_spmd.scpp) :

```

#include <time.h>
#include <sdaa_matmul.h>
using namespace sdaa;

#define M 4096 // 输入矩阵和输出矩阵的行数

```

```
#define K 64          // 输入矩阵列数，也是权重矩阵行数
#define N 32          // 权重矩阵和输出矩阵的列数

__device__ void matmul_compute_func(short *input, short *weight, short *output, int m, int k, int n)
{
    // 创建MatmulHandle以配置矩阵乘接口参数。此例中使用默认配置
    MatmulHandle handle;

    // 初始化矩阵乘接口，并配置输入和输出数据的类型
    matmul_init(handle, MatmulShortToShort);

    // 设置之后的计算不为最后一次计算，
    // 即计算结果会累加并缓存在矩阵乘计算单元的累加缓冲区内
    matmul_set_flushing_output(handle, false);

    // 将权重矩阵前32行加载到矩阵乘计算单元上
    matmul_load_weight(handle, weight, MatmulK32, MatmulN32);
    // 等待权重矩阵数据加载完成，此后可覆盖weight上前32行数据
    matmul_wait_loading_weight(handle);

    // 计算加载输入矩阵的跨步步长，以64字节为单位
    const unsigned int stride = k / MatmulK32 - 1;

    // 跨步加载输入矩阵（此处相当于加载输入矩阵左半）
    // 并计算与权重矩阵的乘积
    matmul_compute(handle, input, m, MatmulK32, stride);
    // 等待输入矩阵加载完成，此后可覆盖input上左半数据
    matmul_wait_loading_input(handle);

    // 使能下一次计算为最后一次计算
    // 计算完后会输出累加缓冲区内数据
    matmul_set_flushing_output(handle, true);

    // 将权重矩阵后32行加载到矩阵乘计算单元上
    matmul_load_weight(handle, weight + MatmulK32 * MatmulN32, MatmulK32, MatmulN32);
    // 等待权重矩阵数据加载完成，此后可覆盖weight上后32行数据
    matmul_wait_loading_weight(handle);

    // 跨步加载输入矩阵（此处相当于加载输入矩阵右半）
    // 并计算与权重矩阵的乘积
    matmul_compute(handle, input + MatmulK32, m, MatmulK32, stride);
    // 等待输入矩阵加载完成，此后可覆盖input上右半数据
    matmul_wait_loading_input(handle);

    // 将矩阵乘运算结果写回到SPM指定位置
    matmul_store(handle, output, m, MatmulN32);

    // 等待运算结果写回完成
    matmul_wait(handle);
}

__global__ void matmul_func(char *data_input, char *data_weight, char *data_output, int m, int k, int n)
{
    // 计算每个计算核心SPE所需要承担的计算任务
    unsigned int slice = m / threadDim;

    // 申请SPM存储空间
    short *input = (short*)malloc(slice * k * sizeof(short));
```

```

short *weight = (short*)malloc(k * n * sizeof(short));
short *output = (short*)malloc(slice * n * sizeof(short));

// 将数据从Global搬运输入矩阵和权重矩阵到SPM
memcpy_async(input, data_input + threadIdx * slice * k * sizeof(short), slice * k * sizeof(short),
0, {0, 0}, MemcpyGlobalToSpm);
memcpy_async(weight, data_weight, k * n * sizeof(short), 0, {0, 0}, MemcpyGlobalToSpm);
// 等待异步拷贝完成
memcpy_wait();

// 矩阵乘计算
matmul_compute_func(input, weight, output, slice, k, n);

// 将输出矩阵从SPM搬回Global
memcpy(data_output + threadIdx * slice * n * sizeof(short), output, slice * n * sizeof(short));

// 释放在SPM申请的内存
free(output);
free(weight);
free(input);
}

int main()
{
    // 分配Host端空间
    // h_input为输入矩阵, h_weight为权重矩阵, h_output为计算结果矩阵
    // h_output_reference为Host模拟矩阵乘计算, 计算结果存储矩阵
    short* h_input = (short*)malloc(M * K * sizeof(short));
    short* h_weight = (short*)malloc(K * N * sizeof(short));
    short* h_output = (short*)malloc(M * N * sizeof(short));
    short* h_output_reference = (short*)malloc(M * N * sizeof(short));

    srand(time(NULL));
    // 为Host端输入矩阵进行初始化
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < K; j++) {
            h_input[i * K + j] = rand() % 10;
        }
    }

    // 为Host端权重矩阵进行初始化
    for (int i = 0; i < K; i++) {
        for (int j = 0; j < N; j++) {
            h_weight[i * N + j] = rand() % 5;
        }
    }

    // 在Host模拟矩阵乘操作, 并将结果存放到h_output_reference
    memset(h_output_reference, 0, M * N * sizeof(short));
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            for (int k = 0; k < K; k++) {
                h_output_reference[i * N + j] +=
                    h_input[i * K + k] * h_weight[k * N + j];
            }
        }
    }

    // 配置使用0号计算核心阵列

```

```

sdaaSetDevice(0);
// 声明Device端地址
// dev_input为输入矩阵, dev_weight为权重矩阵, dev_output为计算结果矩阵
char* dev_input = NULL;
char* dev_weight = NULL;
char* dev_output = NULL;

// 使用sdaaMalloc为Device端地址进行空间分配
sdaaMalloc((void**)&dev_input, M * K * sizeof(short));
sdaaMalloc((void**)&dev_weight, K * N * sizeof(short));
sdaaMalloc((void**)&dev_output, M * N * sizeof(short));

// 使用sdaaMemcpy将Host端内存拷贝到Device端
sdaaMemcpy(dev_input, h_input, M * K * sizeof(short), sdaaMemcpyHostToDevice);
sdaaMemcpy(dev_weight, h_weight, K * N * sizeof(short), sdaaMemcpyHostToDevice);

// 启动核函数调用
matmul_func<<<1>>>(dev_input, dev_weight, dev_output, M, K, N);

// 等待计算核心SPE完成之前的所有请求任务
sdaaDeviceSynchronize();

// 使用sdaaMemcpy将Device端矩阵乘计算结果拷贝到Host端
sdaaMemcpy(h_output, dev_output, M * N * sizeof(short), sdaaMemcpyDeviceToHost);

// 将Device端矩阵乘计算结果与Host端计算结果进行对比
// 验证失败则打印对应日志
for (int i = 0; i < M * N; i++) {
    if(h_output[i] != h_output_reference[i]) {
        printf("find mismatching data on line %d: ace: %d, serial: %d\n",
            i, (int)h_output[i], (int)h_output_reference[i]);
        break;
    }
}

// 如果验证成功, 则打印对应日志
if (i == M * N - 1) {
    printf("matmul result compare ok\n");
}
}

// 释放在Device端和Host申请的内存
free(h_output_reference);
free(h_output);
free(h_weight);
free(h_input);
sdaaFree(dev_output);
sdaaFree(dev_weight);
sdaaFree(dev_input);

return 0;
}

```

15.2.4 编译程序

程序编译

执行以下编译命令，将源文件 `matmul_spmd.scpp` 生成可执行文件 `matmul_spmd.out`。

```
tecocc matmul_spmd.scpp -o matmul_spmd.out
```

执行以下命令，运行程序：

```
./matmul_spmd.out
```

执行结果

```
matmul result compare ok
```

15.3 使用SUMMA算法实现矩阵乘

15.3.1 概述

本例将在太初AI加速卡上通过使用SUMMA算法实现矩阵乘操作。完整程序请参考[完整程序](#)。

主机端通过SDAARuntime接口分配设备端的Global存储空间，并将数据从主机端拷贝到设备端的Global存储空间。在设备端使用SDAA C提供的[DMA数据搬运接口](#)、[Broadcast数据搬运接口](#)、[矩阵乘接口](#)、[SIMD接口](#)等完成SUMMA算法，实现对大矩阵的矩阵乘运算。设备端完成计算后通过SDAARuntime接口将Global存储空间的计算结果拷贝到主机端内存。

15.3.2 编程思路

主机端

1. 分配主机端内存并通过随机数进行初始化操作。
2. 主机端与设备端的数据传输。
 - a. 设置计算核心阵列SPA号。
 - b. 分配Global存储空间。
 - c. 从主机端内存向Global存储空间传输数据。
 - d. 调用Kernel函数。
 - e. 将计算结果从Global存储空间传回主机端内存。
3. 在主机端模拟矩阵乘运算，并与设备端计算结果进行对比。
4. 释放Global存储空间与主机端内存。

设备端

1. 将计算核心阵列SPA内的SPE按照4行8列的方式抽象为一个二维数组。
2. 将输入矩阵与权重矩阵按照抽象的行列数量切分为若干个子矩阵。
3. 在SPE内为子矩阵申请SPM存储空间。
4. 通过阻塞型DMA接口**memcpy**将小矩阵拷贝到申请的SPM存储空间内。
5. 在K维度上循环并通过**阻塞型Broadcast接口**对小矩阵根据抽象的行列按行、按列进行广播操作。
6. 每个SPE在接收到广播数据后进行矩阵乘运算。
7. 将本次的矩阵乘结果上一次的矩阵乘计算结果进行累加操作。
8. 将单个SPE的计算结果通过阻塞型DMA数据搬运接口拷贝到Global存储空间。
9. 释放申请的SPM存储空间。

15.3.3 编写程序

15.3.3.1 主机端

主机端程序设计思路

1. 分配主机端内存并通过随机数进行初始化操作

a. 在主机端通过malloc接口分配3个主机端存储空间 `h_input` 、 `h_weight` 、 `h_output` 。

- `h_input` ：输入矩阵。
- `h_weight` ：权重矩阵。
- `h_output` ：输出矩阵。

b. 通过随机数将输入矩阵与权重矩阵进行初始化操作。

2. 主机端与设备端的数据传输

a. 设置计算核心阵列SPA号：通过**sdAASetDevice**接口指定Kernel函数运行在哪一个计算核心阵列上。

b. 分配Global存储空间：通过**sdAAMalloc**接口在设备端分配3个Global存储空间 (`dev_input` 、 `dev_weight` 、 `dev_output`) 。

- `dev_input` ：Global存储空间内的输入矩阵。
- `dev_weight` ：Global存储空间内的权重矩阵。
- `dev_output` ：Global存储空间内的输出矩阵。

c. 数据传输：通过**sdAAMemcpy**接口将Host端内存 (`h_input` 、 `h_weight` 、 `h_output`) 对应拷贝到Device端Global存储空间 (`dev_input` 、 `dev_weight` 、 `dev_output`) 。

d. 调用Kernel函数：通过<<<...>>>调用Kernel函数。

- e. 拷贝计算结果：通过`sdaaDeviceSynchronize`接口等待计算核心SPE完成之前的所有请求任务，再通过`sdaaMemcpy`接口将Device端Global存储空间 `dev_output` 中的数据拷贝到Host内存 `h_output` 。

3. 计算结果对比与内存输出

- a. 在Host端模拟矩阵乘计算，并将计算结果存放到 `h_output_reference` 。
- b. 计算结果对比：将Kernel的计算结果 `h_output` 与参考矩阵乘计算结果 `h_output_reference` 进行对比。
- c. 释放内存：
- 释放申请的Device端Global存储空间：通过`sdaaFree`接口释放Device端的Global存储空间。
 - 释放申请的Host端存储空间：通过`free`接口释放申请的Host端存储空间。

主机端程序

```
#include <stdio.h>

// 定义SUMMA算法的矩阵规模
#define M 64 // 输入矩阵行数
#define K 32 // 输入矩阵最大列数，权重矩阵最大行数
#define N 64 // 权重矩阵最大列数

int main()
{
    int i = 0, j = 0, k = 0;
    // 分配Host端地址
    // h_input为输入矩阵，h_weight为权重矩阵，h_output为计算结果矩阵
    half *h_input = (half *)malloc(M * K * sizeof(half));
    half *h_weight = (half *)malloc(K * N * sizeof(half));
    half *h_output = (half *)malloc(M * N * sizeof(half));

    // 设置随机种子
    srand(100);

    // 为Host端输入矩阵进行初始化
    memset(h_input, 0, M * K * sizeof(half));
    for (i = 0; i < M; i++) {
        for (j = 0; j < K; j++) {
            h_input[i * K + j] = 1.0 * rand() / RAND_MAX * 10.0 - 5;
        }
    }

    // 为Host端权重矩阵进行初始化
    memset(h_weight, 0, K * N * sizeof(half));
    for (i = 0; i < K; i++) {
        for (j = 0; j < N; j++) {
            h_weight[i * N + j] = 1.0 * rand() / RAND_MAX * 0.1 - 0.05;
        }
    }

    // 配置使用0号计算核心阵列
    sdaaSetDevice(0);
    // 声明Device端地址
```

```

// dev_input为输入矩阵, dev_weight为权重矩阵, dev_output为计算结果矩阵
char *dev_input = NULL;
char *dev_weight = NULL;
char *dev_output = NULL;

sdaaMalloc((void **)&dev_input, M * K * sizeof(half));
sdaaMalloc((void **)&dev_weight, K * N * sizeof(half));
sdaaMalloc((void **)&dev_output, M * N * sizeof(half));

// 使用sdaaMemcpy将Host端内存拷贝到Device端
sdaaMemcpy(dev_input, h_input, M * K * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(dev_weight, h_weight, K * N * sizeof(half), sdaaMemcpyHostToDevice);

// 声明Device端dev_outputcheck_val
unsigned int *dev_outputcheck_val = NULL;
// 使用sdaaMalloc为dev_outputcheck_val进行空间分配
sdaaMalloc((void **)&dev_outputcheck_val, threadDim * sizeof(unsigned int));
// 为check_val在host端进行空间分配
unsigned int *check_val = (unsigned int *)malloc(threadDim * sizeof(unsigned int));

// 核函数调用
summa<<<1>>>(dev_input, dev_weight, dev_output, dev_outputcheck_val, M, K, N);

// 等待计算核心SPE完成之前的所有请求任务
sdaaDeviceSynchronize();

// 使用sdaaMemcpy将Device端矩阵乘计算结果拷贝到Host端
sdaaMemcpy(h_output, dev_output, M * N * sizeof(half), sdaaMemcpyDeviceToHost);

// 使用sdaaMemcpy将Device端验证输入参数合法性结果拷贝到Host端
sdaaMemcpy(check_val, dev_outputcheck_val, threadDim * sizeof(unsigned int), sdaaMemcpyDeviceToHost);

// 对check结果进行合法性校验, 验证失败则打印验证失败日志, 否则打印验证成功日志
bool check_result = true;
for (i = 0; i < threadDim; i++) {
    if (check_val[i] != 0) {
        check_result = false;
        printf("thread %d check_matmul failed, returns %d\n", i, *check_val);
    }
}

if (check_result) {
    printf("all check_matmul checked done\n");

    // 在Host模拟矩阵乘操作, 并将结果存放到h_output_reference
    half *h_output_reference = (half *)malloc(M * N * sizeof(half));
    memset(h_output_reference, 0, M * N * sizeof(half));
    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++) {
            for (k = 0; k < K; k++) {
                h_output_reference[i * N + j] += h_input[i * K + k] * h_weight[k * N + j];
            }
        }
    }

    // 将Device端矩阵乘计算结果与Host端计算结果进行对比, 验证失败则打印对应日志
    for (i = 0; i < M * N; i++) {
        if ((h_output[i] - h_output_reference[i] > 1.5e-2)) {

```

```

        printf("find mismatching data Pos %d, on line %d, col %d: ace: %f, serial: %f\n",
               i, i / N, i % N, (float)h_output[i], (float)h_output_reference[i]);
        break;
    }
    if (i == M * N - 1) {
        printf("SUMMA result compare ok\n");
    }
}

// 释放h_output_reference空间
free(h_output_reference);
}

// 释放在Device端和Host申请的内存
free(check_val);
sdaaFree(dev_outputcheck_val);
sdaaFree(dev_output);
sdaaFree(dev_weight);
sdaaFree(dev_input);
free(h_output);
free(h_weight);
free(h_input);

return 0;
}

```

15.3.3.2 设备端

设备端程序设计思路

设备端Kernel函数通过SUMMA算法将输入矩阵切分为若干个子矩阵，计算核心SPE将相应的子矩阵加载到SPM存储空间后，通过SDAA C提供的**Broadcast数据搬运接口**、**矩阵乘接口**、**SIMD接口**等完成子矩阵的计算。最后将子矩阵的计算结果进行合并完成整体计算任务的求解。

? 说明：

1. 使用__device__关键字修饰，说明该变量或函数是在设备端的Global存储空间，主机端不允许访问。
2. 使用__global__关键字修饰，说明函数是Kernel函数，允许主机端调用，但不允许设备端调用。返回值类型仅为void类型。
3. threadDim用于获取计算核心阵列可用的计算核心资源数量。
4. threadIdx用于读取计算核心SPE的ID。

1. 将计算核心阵列SPA内的SPE按照4行8列的方式抽象为一个二维数组。
 2. 将输入矩阵与权重矩阵按照抽象的行列数量切分为若干个子矩阵（每个计算核心阵列SPE对应一个子矩阵）。
 3. 在计算核心SPE内为子矩阵申请SPM存储空间（ `input_spm` 、 `weight_spm` 、 `output_spm` ）。
- `input_spm` ：输入子矩阵。

- `weight_spm` : 权重子矩阵。
- `output_spm` : 输出子矩阵。

4. 通过[阻塞型DMA数据搬运接口](#)将子矩阵数据从Global存储空间拷贝到SPM存储空间。

5. 计算广播的行列掩码，并初始化[BroadcastHandle](#)类型的广播句柄。

6. 为广播地址申请SPM存储空间。

- `input_dst_spm` : 输入子矩阵多播的目的空间。
- `weight_dst_spm` : 权重子矩阵多播的目的空间。
- `input_src_spm` : 输入子矩阵多播的原始数据空间。
- `weight_dst_spm` : 权重子矩阵多播的原始数据空间。

7. 在K维度上对输入子矩阵和权重子矩阵进行循环操作。

- 将输入子矩阵的列广播到与当前计算核心SPE同行的计算核心SPE申请的 `input_dst_spm` 存储空间内。
- 将权重子矩阵的列广播到与当前计算核心SPE同列的计算核心SPE申请的 `weight_dst_spm` 存储空间内。
- 将 `input_dst_spm` 存储空间与 `weight_dst_spm` 存储空间内的数据通过[阻塞型矩阵乘接口](#)进行矩阵乘计算。
- 通过[SIMD接口](#)对每次的矩阵乘计算结果进行累加操作。

8. 计算核心SPE完成子矩阵计算后将计算结果通过[阻塞型DMA数据搬运接口](#)拷贝到Global存储空间内。

9. 释放申请的SPM存储空间。

设备端程序

```
#include <sdaa_matmul.h>
using namespace sdaa;

// 预定义SUMMA算法使用的在遍历K维时的步长
#define NB 2

// 定义当前SPA的阵列
#define MESH_ROW_NUM 4 // 行数
#define MESH_COL_NUM 8 // 列数

// 定义行多播核列多播时的枚举标记
enum Mask_Flag {
    MASK_COL = 0,
    MASK_ROW = 1
};

// 矩阵乘法的封装函数，包含参数合法性校验
__device__ unsigned int matmul_wrapper(const char *input, const char *weight, char *output, unsigned int *check_val,
                                       const int dev_m, const int dev_k, const int dev_n)
{
    // 合法性校验
```

```

*check_val += check_matmul(MatmulHalfToHalf, output, input, weight,
                           dev_m, dev_k, dev_n);
if (*check_val != MATMUL_SUCC) {
    return *check_val;
}

// 执行矩阵乘法
matmul(MatmulHalfToHalf, output, input, weight, dev_m, dev_k, dev_n);

return 0;
}

// 计算多播时需要的mask参数, 主要针对SUMMA算法中的水平方向多播和垂直方向多播
__device__ unsigned long gen_broadcast_mask(Mask_Flag is_row)
{
    // 计算当前线程所在SPA内的坐标
    const int x_pos = threadIdx % MESH_COL_NUM;
    const int y_pos = threadIdx / MESH_COL_NUM;

    unsigned long mask = 0;
    // 计算列多播时的mask
    if (is_row == MASK_COL) {
        int start_id = x_pos;
        for (int i = 0; i < threadDim; i += MESH_COL_NUM) {
            mask |= (1 << (start_id + i));
        }
    }
    // 计算行多播时的mask
    if (is_row == MASK_ROW) {
        int start_id = y_pos * MESH_COL_NUM;
        for (int i = 0; i < MESH_COL_NUM; i++) {
            mask |= (1 << (start_id + i));
        }
    }
    return mask;
}

__device__ void load_sub_block(char *src, char *dst, const int src_h, const int src_w, const int dst_h,
                              const int dst_w, const int start_x, const int start_y)
{
    const int type_size = sizeof(half);
    // 计算原数据在copy时的步长
    const int src_stride = src_w * type_size;
    // 计算原数据在copy时的起始地址
    char *src_start = src + (start_y * src_w + start_x) * type_size;
    for (int i = 0; i < dst_h; i++) {
        memcpy(dst, src_start, dst_w * type_size);
        src_start += src_stride;
        dst += dst_w * type_size;
    }
}

__device__ void gen_sub_block(char *src, char *dst, const int src_h, const int src_w, const int dst_h,
                              const int dst_w, const int start_x, const int start_y)
{
    const int type_size = sizeof(half);
    // 计算原数据在copy时的起始地址
    char *src_start = src + (start_y * src_w + start_x) * type_size;
    // 计算原数据在copy时Global存储空间的步长

```

```

    const size_t glbmem_stride = (src_w - dst_w) * type_size;

    memcpy_stride(dst, src_start, dst_w * type_size * dst_h, dst_w * type_size, {0, glbmem_stride});
}

__device__ void save_sub_block(char *dst_glb, char *src_spm, const int dst_h, const int dst_w,
                              const int src_h, const int src_w, const int start_x, const int start_y)
{
    const int type_size = sizeof(half);
    // 计算目标数据在copy时的步长
    const int dst_stride = dst_w * type_size;
    // 计算目标数据在copy时的起始地址
    char *dst_start = dst_glb + (start_y * dst_w + start_x) * type_size;
    for (int i = 0; i < src_h; i++) {
        memcpy(dst_start, src_spm, src_w * type_size);
        dst_start += dst_stride;
        src_spm += src_w * type_size;
    }
}

// summa算法的核函数
__global__ void summa(char *input, char *weight, char *output, unsigned int *check_val,
                     int dev_m, int dev_k, int dev_n)
{
    // 计算数据类型大小
    const int type_size = sizeof(half);
    // 计算当前线程所在SPA内的坐标
    const int x_pos = threadIdx % MESH_COL_NUM;
    const int y_pos = threadIdx / MESH_COL_NUM;

    // 计算输入矩阵分块后子矩阵的行数
    const int input_sub_m = dev_m / MESH_ROW_NUM;
    // 计算输入矩阵分块后子矩阵的列数
    const int input_sub_k = dev_k / MESH_COL_NUM;
    // 计算权重矩阵分块后子矩阵的行数
    const int weight_sub_k = dev_k / MESH_ROW_NUM;
    // 计算权重矩阵分块后子矩阵的列数
    const int weight_sub_n = dev_n / MESH_COL_NUM;
    // 计算输入矩阵子阵的大小
    const int input_spm_num = input_sub_m * input_sub_k;
    // 计算权重矩阵子阵的大小
    const int weight_spm_num = weight_sub_k * weight_sub_n;
    // 计算输出矩阵子阵的大小
    const int output_spm_num = input_sub_m * weight_sub_n;

    // 输入子矩阵多播的数据量
    const int input_broad_num = NB * input_sub_m;
    // 权重子矩阵多播的数据量
    const int output_broad_num = NB * weight_sub_n;

    // 判断每次多播数据是否合法, 必须是4字节的倍数
    if (((input_broad_num & 1) != 0) || ((output_broad_num & 1) != 0)) {
        printf("the broadcast data number must be the times of 4 bytes is not meet!");
        return;
    }

    // 在SPM上申请内存
    // 输入矩阵分块后的子矩阵空间
    char *input_spm = (char *)malloc(input_spm_num * type_size);

```



```

// 权重矩阵分块后的子矩阵空间
char *weight_spm = (char *)malloc(weight_spm_num * type_size);
// 输出矩阵分块后的子矩阵空间
char *output_spm = (char *)malloc(output_spm_num * type_size);
// 输入矩阵和权重矩阵多播后每次临时矩阵计算后的结果空间
char *output_rst_spm = (char *)malloc(output_spm_num * type_size);

// 初始化output_spm为0
memset(output_spm, 0, output_spm_num * type_size);
// 初始化check_val为0
memset(check_val, 0, threadDim * sizeof(unsigned int));

// 从Global加载输入矩阵的子矩阵到SPM
gen_sub_block(input, input_spm, dev_m, dev_k, input_sub_m, input_sub_k,
              input_sub_k * x_pos, input_sub_m * y_pos);
// 从Global加载权重矩阵的子矩阵到SPM
gen_sub_block(weight, weight_spm, dev_k, dev_n, weight_sub_k, weight_sub_n,
              weight_sub_n * x_pos, weight_sub_k * y_pos);

// 计算多播相关参数
unsigned long row_mask = gen_broadcast_mask(MASK_ROW);
unsigned long col_mask = gen_broadcast_mask(MASK_COL);

ThreadGroup row_thread_group(row_mask);
ThreadGroup col_thread_group(col_mask);

// 设置多播mask参数
BroadcastHandle handle_row(&row_thread_group);
BroadcastHandle handle_col(&col_thread_group);

// 广播的源地址和目的地址
// 输入矩阵分块后的子矩阵多播的目的空间
char *input_dst_spm = (char *)malloc(input_broad_num * type_size);
// 权重矩阵分块后的子矩阵多播的目的空间
char *weight_dst_spm = (char *)malloc(output_broad_num * type_size);
// 输入矩阵分块后的子矩阵多播的原数据空间
char *input_src_spm = (char *)malloc(input_broad_num * type_size);
// 权重矩阵分块后的子矩阵多播的原数据空间
char *weight_src_spm = (char *)malloc(output_broad_num * type_size);

// 同一计算核心阵列SPA内所有计算核心SPE同步操作
sync_threads();

// 在K维度上进行遍历, 每次计算最终结果的一部分
for (int i = 0; i < dev_k; i += NB) {
    // 计算本次水平广播的发起方ID
    unsigned root_row = (y_pos * MESH_COL_NUM) + (i / input_sub_k);
    // 准备水平广播的数据
    if (root_row == threadIdx) {
        // 由于输入矩阵每次多播的数据为部分列, 所以需要
        // 将输入矩阵的多播数据从不连续的子块空间, 拷贝到连续的地址空间
        load_sub_block(input_spm, input_src_spm, input_sub_m, input_sub_k, input_sub_m, NB,
                      i % input_sub_k, 0);
    }
    // 调用广播同步接口, 输入矩阵进行水平方向广播
    broadcast(input_dst_spm, input_src_spm, input_broad_num * type_size, 0, {0, 0}, root_row, BroadcastSpmToSpm, handle_row);

    // 计算本次垂直广播的发起方ID

```

```

unsigned root_col = x_pos + ((i / weight_sub_k) * MESH_COL_NUM);
// 准备本次垂直广播的原地址空间数据, 因为数据是连续的, 调用memcpy一次即可
memcpy(weight_src_spm, weight_spm + ((i % weight_sub_k) * weight_sub_n * type_size),
        output_broad_num * type_size);
// 调用广播同步接口, 矩阵B进行垂直方向广播
broadcast(weight_dst_spm, weight_src_spm, output_broad_num * type_size, 0, {0, 0}, root_col, BroadcastSpmToSpm, handle_col);

// 执行同步操作, 确保接收方接收到数据
sync_threads();

// 对广播收到的输入矩阵的子矩阵和权重矩阵的子矩阵进行乘法运算
matmul_wrapper(input_dst_spm, weight_dst_spm, output_rst_spm, check_val + threadIdx,
               input_sub_m, NB, weight_sub_n);
// 将每次临时计算的结果进行累加
halfv16 v_dst, v_src;
for (int i = 0; i < output_spm_num; i) {
    int current_num = ((i + 16) <= output_spm_num) ? 16 : (output_spm_num - i);
    if (16 == current_num) {
        // 如果当前计算16个连续的数据要求, 调用并行指令
        simd_load(v_dst, output_spm + i * type_size);
        simd_load(v_src, output_rst_spm + i * type_size);
        v_dst += v_src;
        simd_store(v_dst, output_spm + i * type_size);
        i += 16;
    } else {
        // 对单个数据, 进行串行计算
        half *src = (half *)output_rst_spm;
        half *dst = (half *)output_spm;
        *(dst + i) += *(src + i);
        i++;
    }
}
}
// 从SPM保存输出矩阵的子矩阵到Global
save_sub_block(output, output_spm, dev_m, dev_n, input_sub_m, weight_sub_n,
               weight_sub_n * x_pos, input_sub_m * y_pos);

// 释放在SPM上申请的内存
free(input_spm);
free(weight_spm);
free(output_spm);
free(output_rst_spm);
free(weight_src_spm);
free(input_src_spm);
free(weight_dst_spm);
free(input_dst_spm);
}

```

15.3.3.3 完整程序

使用SUMMA算法完成矩阵乘完整程序 (matmul_summa.scpp) :

```

#include <stdio.h>
#include <sdaa_matmul.h>
using namespace sdaa;

```

```

// 定义SUMMA算法的矩阵规模
#define M 64 // 输入矩阵行数
#define K 32 // 输入矩阵最大列数, 权重矩阵最大行数
#define N 64 // 权重矩阵最大列数

// 预定义SUMMA算法使用的在遍历K维时的步长
#define NB 2

// 定义当前SPA的阵列
#define MESH_ROW_NUM 4 // 行数
#define MESH_COL_NUM 8 // 列数

// 定义行多播核列多播时的枚举标记
enum Mask_Flag {
    MASK_COL = 0,
    MASK_ROW = 1
};

// 矩阵乘法的封装函数, 包含参数合法性校验
__device__ unsigned int matmul_wrapper(const char *input, const char *weight, char *output, unsigned int *check_val,
                                       const int dev_m, const int dev_k, const int dev_n)
{
    // 合法性校验
    *check_val += check_matmul(MatmulHalfToHalf, output, input, weight,
                              dev_m, dev_k, dev_n);
    if (*check_val != MATMUL_SUCC) {
        return *check_val;
    }

    // 执行矩阵乘法
    matmul(MatmulHalfToHalf, output, input, weight, dev_m, dev_k, dev_n);

    return 0;
}

// 计算多播时需要的mask参数, 主要针对SUMMA算法中的水平方向多播和垂直方向多播
__device__ unsigned long gen_broadcast_mask(Mask_Flag is_row)
{
    // 计算当前线程所在SPA内的坐标
    const int x_pos = threadIdx % MESH_COL_NUM;
    const int y_pos = threadIdx / MESH_COL_NUM;

    unsigned long mask = 0;
    // 计算列多播时的mask
    if (is_row == MASK_COL) {
        int start_id = x_pos;
        for (int i = 0; i < threadDim; i += MESH_COL_NUM) {
            mask |= (1 << (start_id + i));
        }
    }
    // 计算行多播时的mask
    if (is_row == MASK_ROW) {
        int start_id = y_pos * MESH_COL_NUM;
        for (int i = 0; i < MESH_COL_NUM; i++) {
            mask |= (1 << (start_id + i));
        }
    }
}

```

```

    return mask;
}

__device__ void load_sub_block(char *src, char *dst, const int src_h, const int src_w, const int dst_h,
                               const int dst_w, const int start_x, const int start_y)
{
    const int type_size = sizeof(half);
    // 计算原数据在copy时的步长
    const int src_stride = src_w * type_size;
    // 计算原数据在copy时的起始地址
    char *src_start = src + (start_y * src_w + start_x) * type_size;
    for (int i = 0; i < dst_h; i++) {
        memcpy(dst, src_start, dst_w * type_size);
        src_start += src_stride;
        dst += dst_w * type_size;
    }
}

__device__ void gen_sub_block(char *src, char *dst, const int src_h, const int src_w, const int dst_h,
                              const int dst_w, const int start_x, const int start_y)
{
    const int type_size = sizeof(half);
    // 计算原数据在copy时的起始地址
    char *src_start = src + (start_y * src_w + start_x) * type_size;
    // 计算原数据在copy时Global存储空间的步长
    const size_t glbmem_stride = (src_w - dst_w) * type_size;

    memcpy_stride(dst, src_start, dst_w * type_size * dst_h, dst_w * type_size, {0, glbmem_stride});
}

__device__ void save_sub_block(char *dst_glb, char *src_spm, const int dst_h, const int dst_w,
                               const int src_h, const int src_w, const int start_x, const int start_y)
{
    const int type_size = sizeof(half);
    // 计算目标数据在copy时的步长
    const int dst_stride = dst_w * type_size;
    // 计算目标数据在copy时的起始地址
    char *dst_start = dst_glb + (start_y * dst_w + start_x) * type_size;
    for (int i = 0; i < src_h; i++) {
        memcpy(dst_start, src_spm, src_w * type_size);
        dst_start += dst_stride;
        src_spm += src_w * type_size;
    }
}

// summa算法的核函数
__global__ void summa(char *input, char *weight, char *output, unsigned int *check_val,
                      int dev_m, int dev_k, int dev_n)
{
    // 计算数据类型大小
    const int type_size = sizeof(half);
    // 计算当前线程所在SPA内的坐标
    const int x_pos = threadIdx % MESH_COL_NUM;
    const int y_pos = threadIdx / MESH_COL_NUM;

    // 计算输入矩阵分块后子矩阵的行数
    const int input_sub_m = dev_m / MESH_ROW_NUM;
    // 计算输入矩阵分块后子矩阵的列数
    const int input_sub_k = dev_k / MESH_COL_NUM;

```

```

// 计算权重矩阵分块后子矩阵的行数
const int weight_sub_k = dev_k / MESH_ROW_NUM;
// 计算权重矩阵分块后子矩阵的列数
const int weight_sub_n = dev_n / MESH_COL_NUM;
// 计算输入矩阵子阵的大小
const int input_spm_num = input_sub_m * input_sub_k;
// 计算权重矩阵子阵的大小
const int weight_spm_num = weight_sub_k * weight_sub_n;
// 计算输出矩阵子阵的大小
const int output_spm_num = input_sub_m * weight_sub_n;

// 输入子矩阵多播的数据量
const int input_broad_num = NB * input_sub_m;
// 权重子矩阵多播的数据量
const int output_broad_num = NB * weight_sub_n;

// 判断每次多播数据是否合法，必须是4字节的倍数
if (((input_broad_num & 1) != 0) || ((output_broad_num & 1) != 0)) {
    printf("the broadcast data number must be the times of 4 bytes is not meet!");
    return;
}

// 在SPM上申请内存
// 输入矩阵分块后的子矩阵空间
char *input_spm = (char *)malloc(input_spm_num * type_size);
// 权重矩阵分块后的子矩阵空间
char *weight_spm = (char *)malloc(weight_spm_num * type_size);
// 输出矩阵分块后的子矩阵空间
char *output_spm = (char *)malloc(output_spm_num * type_size);
// 输入矩阵和权重矩阵多播后每次临时矩阵计算后的结果空间
char *output_rst_spm = (char *)malloc(output_spm_num * type_size);

// 初始化output_spm为0
memset(output_spm, 0, output_spm_num * type_size);
// 初始化check_val为0
memset(check_val, 0, threadDim * sizeof(unsigned int));

// 从Global加载输入矩阵的子矩阵到SPM
gen_sub_block(input, input_spm, dev_m, dev_k, input_sub_m, input_sub_k,
              input_sub_k * x_pos, input_sub_m * y_pos);
// 从Global加载权重矩阵的子矩阵到SPM
gen_sub_block(weight, weight_spm, dev_k, dev_n, weight_sub_k, weight_sub_n,
              weight_sub_n * x_pos, weight_sub_k * y_pos);

// 计算多播相关参数
unsigned long row_mask = gen_broadcast_mask(MASK_ROW);
unsigned long col_mask = gen_broadcast_mask(MASK_COL);

ThreadGroup row_thread_group(row_mask);
ThreadGroup col_thread_group(col_mask);

// 设置多播mask参数
BroadcastHandle handle_row(&row_thread_group);
BroadcastHandle handle_col(&col_thread_group);

// 广播的源地址和目的地址
// 输入矩阵分块后的子矩阵多播的目的空间
char *input_dst_spm = (char *)malloc(input_broad_num * type_size);
// 权重矩阵分块后的子矩阵多播的目的空间

```

```

char *weight_dst_spm = (char *)malloc(output_broad_num * type_size);
// 输入矩阵分块后的子矩阵多播的原数据空间
char *input_src_spm = (char *)malloc(input_broad_num * type_size);
// 权重矩阵分块后的子矩阵多播的原数据空间
char *weight_src_spm = (char *)malloc(output_broad_num * type_size);

// 同一计算核心阵列SPA内所有计算核心SPE同步操作
sync_threads();

// 在K维度上进行遍历，每次计算最终结果的一部分
for (int i = 0; i < dev_k; i += NB) {
    // 计算本次水平广播的发起方ID
    unsigned root_row = (y_pos * MESH_COL_NUM) + (i / input_sub_k);
    // 准备水平广播的数据
    if (root_row == threadIdx) {
        // 由于输入矩阵每次多播的数据为部分列，所以需要
        // 将输入矩阵的多播数据从不连续的子块空间，拷贝到连续的地址空间
        load_sub_block(input_spm, input_src_spm, input_sub_m, input_sub_k, input_sub_m, NB,
            i % input_sub_k, 0);
    }
    // 调用广播同步接口，输入矩阵进行水平方向广播
    broadcast(input_dst_spm, input_src_spm, input_broad_num * type_size, 0, {0, 0}, root_row, BroadcastSpmToSpm, handle_row);

    // 计算本次垂直广播的发起方ID
    unsigned root_col = x_pos + ((i / weight_sub_k) * MESH_COL_NUM);
    // 准备本次垂直广播的原地址空间数据，因为数据是连续的，调用memcpy一次即可
    memcpy(weight_src_spm, weight_spm + ((i % weight_sub_k) * weight_sub_n * type_size),
        output_broad_num * type_size);
    // 调用广播同步接口，矩阵B进行垂直方向广播
    broadcast(weight_dst_spm, weight_src_spm, output_broad_num * type_size, 0, {0, 0}, root_col, BroadcastSpmToSpm, handle_col);

    // 执行同步操作，确保接收方接收到数据
    sync_threads();

    // 对广播收到的输入矩阵的子矩阵和权重矩阵的子矩阵进行乘法运算
    matmul_wrapper(input_dst_spm, weight_dst_spm, output_rst_spm, check_val + threadIdx,
        input_sub_m, NB, weight_sub_n);
    // 将每次临时计算的结果进行累加
    halfv16 v_dst, v_src;
    for (int i = 0; i < output_spm_num; i) {
        int current_num = ((i + 16) <= output_spm_num) ? 16 : (output_spm_num - i);
        if (16 == current_num) {
            // 如果当前计算16个连续的数据要求，调用并行指令
            simd_load(v_dst, output_spm + i * type_size);
            simd_load(v_src, output_rst_spm + i * type_size);
            v_dst += v_src;
            simd_store(v_dst, output_spm + i * type_size);
            i += 16;
        } else {
            // 对单个数据，进行串行计算
            half *src = (half *)output_rst_spm;
            half *dst = (half *)output_spm;
            *(dst + i) += *(src + i);
            i++;
        }
    }
}
}

```

```

// 从SPM保存输出矩阵的子矩阵到Global
save_sub_block(output, output_spm, dev_m, dev_n, input_sub_m, weight_sub_n,
               weight_sub_n * x_pos, input_sub_m * y_pos);

// 释放在SPM上申请的内存
free(input_spm);
free(weight_spm);
free(output_spm);
free(output_rst_spm);
free(weight_src_spm);
free(input_src_spm);
free(weight_dst_spm);
free(input_dst_spm);
}

int main()
{
    int i = 0, j = 0, k = 0;
    // 分配Host端地址
    // h_input为输入矩阵, h_weight为权重矩阵, h_output为计算结果矩阵
    half *h_input = (half *)malloc(M * K * sizeof(half));
    half *h_weight = (half *)malloc(K * N * sizeof(half));
    half *h_output = (half *)malloc(M * N * sizeof(half));

    // 设置随机种子
    srand(100);

    // 为Host端输入矩阵进行初始化
    memset(h_input, 0, M * K * sizeof(half));
    for (i = 0; i < M; i++) {
        for (j = 0; j < K; j++) {
            h_input[i * K + j] = 1.0 * rand() / RAND_MAX * 10.0 - 5;
        }
    }
    // 为Host端权重矩阵进行初始化
    memset(h_weight, 0, K * N * sizeof(half));
    for (i = 0; i < K; i++) {
        for (j = 0; j < N; j++) {
            h_weight[i * N + j] = 1.0 * rand() / RAND_MAX * 0.1 - 0.05;
        }
    }

    // 配置使用0号计算核心阵列
    sdAASetDevice(0);
    // 声明Device端地址
    // dev_input为输入矩阵, dev_weight为权重矩阵, dev_output为计算结果矩阵
    char *dev_input = NULL;
    char *dev_weight = NULL;
    char *dev_output = NULL;

    sdAAMalloc((void **)&dev_input, M * K * sizeof(half));
    sdAAMalloc((void **)&dev_weight, K * N * sizeof(half));
    sdAAMalloc((void **)&dev_output, M * N * sizeof(half));

    // 使用sdAAMemcpy将Host端内存拷贝到Device端
    sdAAMemcpy(dev_input, h_input, M * K * sizeof(half), sdAAMemcpyHostToDevice);
    sdAAMemcpy(dev_weight, h_weight, K * N * sizeof(half), sdAAMemcpyHostToDevice);

    // 声明Device端dev_outputcheck_val

```

```

unsigned int *dev_outputcheck_val = NULL;
// 使用sdaaMalloc为dev_outputcheck_val进行空间分配
sdaaMalloc((void **)&dev_outputcheck_val, threadDim * sizeof(unsigned int));
// 为check_val在host端进行空间分配
unsigned int *check_val = (unsigned int *)malloc(threadDim * sizeof(unsigned int));

// 核函数调用
summa<<<1>>>(dev_input, dev_weight, dev_output, dev_outputcheck_val, M, K, N);

// 等待计算核心SPE完成之前的所有请求任务
sdaaDeviceSynchronize();

// 使用sdaaMemcpy将Device端矩阵乘计算结果拷贝到Host端
sdaaMemcpy(h_output, dev_output, M * N * sizeof(half), sdaaMemcpyDeviceToHost);

// 使用sdaaMemcpy将Device端验证输入参数合法性结果拷贝到Host端
sdaaMemcpy(check_val, dev_outputcheck_val, threadDim * sizeof(unsigned int), sdaaMemcpyDeviceToHost);

// 对check结果进行合法性校验, 验证失败则打印验证失败日志, 否则打印验证成功日志
bool check_result = true;
for (i = 0; i < threadDim; i++) {
    if (check_val[i] != 0) {
        check_result = false;
        printf("thread %d check_matmul failed, returns %d\n", i, *check_val);
    }
}

if (check_result) {
    printf("all check_matmul checked done\n");

    // 在Host模拟矩阵乘操作, 并将结果存放到h_output_reference
    half *h_output_reference = (half *)malloc(M * N * sizeof(half));
    memset(h_output_reference, 0, M * N * sizeof(half));
    for (i = 0; i < M; i++) {
        for (j = 0; j < N; j++) {
            for (k = 0; k < K; k++) {
                h_output_reference[i * N + j] += h_input[i * K + k] * h_weight[k * N + j];
            }
        }
    }

    // 将Device端矩阵乘计算结果与Host端计算结果进行对比, 验证失败则打印对应日志
    for (i = 0; i < M * N; i++) {
        if ((h_output[i] - h_output_reference[i] > 1.5e-2)) {
            printf("find mismatching data Pos %d, on line %d, col %d: ace: %f, serial: %f\n",
                i, i / N, i % N, (float)h_output[i], (float)h_output_reference[i]);
            break;
        }
        if (i == M * N - 1) {
            printf("SUMMA result compare ok\n");
        }
    }

    // 释放h_output_reference空间
    free(h_output_reference);
}

// 释放在Device端和Host申请的内存

```



```
    free(check_val);
    sdaaFree(dev_outputcheck_val);
    sdaaFree(dev_output);
    sdaaFree(dev_weight);
    sdaaFree(dev_input);
    free(h_output);
    free(h_weight);
    free(h_input);

    return 0;
}
```

15.3.4 编译程序

程序编译

执行以下编译命令，将源文件 `matmul_summa.scpp` 生成可执行文件 `matmul_summa.out`。

```
tecocc matmul_summa.scpp -o matmul_summa.out
```

执行以下命令，运行程序：

```
./matmul_summa.out
```

执行结果

```
all check_matmul checked done
SUMMA result compare ok
```

15.4 使用CAS完成自定义原子操作

15.4.1 概述

CAS（Compare And Swap，比较并交换）是一种无锁机制，也称为乐观锁。比较Global存储空间上指定内存位置的值与预期值是否相等，如果相等，则将新值赋予指定内存，如果不相等，则保持目标内存的值不变。当多个SPE同时对同一个内存地址的值进行操作时，CAS可以保证只有一个SPE能够成功更新数据，因此，通过CAS可以实现原子操作。

本例使用CAS完成浮点类型的原子加操作。

程序分为主机端和设备端，主机端负责设备调用，空间申请及释放，数据传输；设备端主要负责计算。



图1 原子操作的编程逻辑

15.4.2 编程思路

主机端

1. 分配主机端内存 `host_a` 并初始化。
2. 主机端与设备端的数据传输。
 - a. 设置计算核心阵列SPA号。
 - b. 分配Global存储空间 `dev_a`。
 - c. 从主机端内存 `host_a` 向Global存储空间 `dev_a` 传输数据。
 - d. 调用Kernel函数进行计算。
 - e. 将计算结果从Global存储空间 `dev_a` 传回主机端内存 `host_a`。
3. 打印结果并释放Global存储空间 `dev_a`。

设备端

1. 利用函数接口 `atomic_cas_bool`，实现浮点类型的原子加函数。
 - a. 创建变量 `address`，用于转换指针类型，以便传入 `atomic_cas_bool`。
 - b. 创建变量 `old`，用于获取目标内存的数值。
 - c. 创建变量 `is_ok`，用于判断交换是否成功。
 - d. 构建循环，创建浮点变量 `expect`，保存期望的浮点加结果。调用 `atomic_cas_bool` 进行交换：
 - 满足交换条件：交换 `expect` 到目标内存，`atomic_cas_bool` 接口返回true。
 - 不满足交换条件时：`atomic_cas_bool` 接口返回false。继续循环，直到完成一次交换。
2. 利用步骤1的原子加函数，对 `dev_a` 的数据做运算。

15.4.3 编写程序

15.4.3.1 主机端

主机端程序设计思路

1. 分配主机端内存 `host_a` 并初始化。

为了简单展示程序效果，这里使用一个浮点型数组存储两个数字，一个用于原子计算，一个普通计算，两个结果用于对比。

```
#define SIZE 2
float host_a[SIZE] = {0, 0};
```

2. 主机端与设备端的数据传输。

a. 设置计算核心阵列SPA号：通过**`sdaaSetDevice`**接口指定Kernel函数运行在哪一个计算核心阵列上。

```
sdaaSetDevice(0);
```

b. 分配Global存储空间：通过**`sdaaMalloc`**接口在设备端分配给 `dev_a` 。

```
int size = SIZE * sizeof(float);
float *dev_a = NULL;
sdaaMalloc((void**>(&dev_a), size);
```

c. 数据传输：通过**`sdaaMemcpy`**接口将Host端内存 `host_a` 对应拷贝到Device端Global存储空间 `dev_a` 。

```
sdaaMemcpy(dev_a, &host_a, size, sdaaMemcpyHostToDevice);
```

d. 调用Kernel函数：通过**`<<<...>>>`**调用Kernel函数。

```
calc<<<1, stream>>>(dev_a);
```

e. 拷贝计算结果：通过**`sdaaMemcpy`**接口将Device端Global存储空间 `dev_a` 中的数据拷贝到Host内存 `host_a` 。

```
sdaaMemcpy(&host_a, dev_a, size, sdaaMemcpyDeviceToHost);
```

3. 打印结果并释放Global存储空间 `dev_a` 。

a. 使用 `printf` 打印 `host_a` 中的数据。

```
printf("host_a[0]=%f, host_a[1]=%f\n", host_a[0], host_a[1]);
```

b. 释放申请的Device端Global存储空间：通过**`sdaaFree`**接口释放Device端的Global存储空间。

```
sdaaFree(dev_a);
```

主机端程序

```

#include <stdio.h>
#include <stdlib.h>
#define SIZE 2
int main()
{
    float host_a[SIZE] = {0, 0};

    // 指定Kernel函数运行在0号计算核心阵列SPA上
    sdaaSetDevice(0);
    // 在设备端的Global存储空间分配大小为size的连续内存
    int size = SIZE * sizeof(float);
    float *dev_a = NULL;
    sdaaMalloc((void**)&dev_a, size);

    // 将主机端内存host_a的数据拷贝到设备端Global存储空间, 大小为size
    sdaaMemcpy(dev_a, &host_a, size, sdaaMemcpyHostToDevice);

    // 调用名为calc的核函数, 在设备端执行计算任务。
    sdaaStream_t stream;
    sdaaStreamCreate(&stream);
    calc<<<1, stream>>>(dev_a);
    sdaaDeviceSynchronize();

    // 将Global存储空间dev_a的数据拷贝到主机端host_a, 大小为size
    sdaaMemcpy(&host_a, dev_a, size, sdaaMemcpyDeviceToHost);
    printf("host_a[0]=%f, host_a[1]=%f\n", host_a[0], host_a[1]);

    // 释放dev_a指针指向的Global存储空间
    sdaaFree(dev_a);

    return 0;
}

```

15.4.3.2 设备端

设备端程序设计思路

设备端先利用atomic_cas_bool实现float的原子加, 然后再使用该原子加进行计算。为了对比, 同时使用普通加法进行计算。

? 说明:

1. 使用__device__关键字修饰, 说明该变量或函数是在设备端的Global存储空间, 主机端不允许访问。
2. 使用__global__关键字修饰, 说明函数是Kernel函数, 允许主机端调用, 但不允许设备端调用。返回值类型仅为void类型。

1. 利用函数接口atomic_cas_bool, 实现浮点类型的原子加函数。

- a. 创建变量 `address`, 用于转换指针类型, 以便传入 `atomic_cas_bool`。
- b. 创建变量 `old`, 用于获取目标内存的数值。

c. 创建变量 `is_ok`，用于判断交换是否成功。

d. 构建循环，创建浮点变量 `expect`，保存期望的浮点加结果。调用 `atomic_cas_bool` 进行交换：

- 满足交换条件：交换 `expect` 到目标内存，`atomic_cas_bool` 接口返回true。
- 不满足交换条件时：`atomic_cas_bool` 接口返回false。继续循环，直到完成一次交换。

```
__device__ void atomic_add_float(float* val, float num)
{
    int* address = (int*)val;
    int old = *address;
    bool is_ok = true;
    do {
        old = *address;
        float expect = num + *val;
        is_ok = atomic_cas_bool(address, old, *(int *)&expect);
    } while (!is_ok);
    return;
}
```

其中，`float expect = num + *val;` 即为要做的浮点加法，通过 `atomic_cas_bool` 交换到 `address`，达到原子操作的目的。

此处可以使用其它自定义计算，实现不同的自定义原子操作。

2. 利用步骤1的原子加函数，对 `dev_a` 的数据做运算。

其中，对 `dev_a[0]` 做原子加运算，对 `dev_a[1]` 做普通加法。

此处做1000次加法，假如当前可用计算核心数量为32，每个SPE计算加2.0，期待的结果是浮点数64000.000000。

```
// 所有SPE进行计算
__global__ void calc(float *dev_a)
{
    // 原子计算
    for(int i = 0; i < 1000; i++) {
        atomic_add_float(&dev_a[0], 2.0);
    }
    // 普通计算
    for(int i = 0; i < 1000; i++) {
        dev_a[1] = dev_a[1] + 2.0;
    }
}
```

设备端程序

```
#include <sdaa_atomic.h>
using namespace sdaa;
```

```
// 使用CAS实现原子浮点加
__device__ void atomic_add_float(float* val, float num)
{
    int* address = (int*)val;
    int old = *address;
    bool is_ok = true;
    do {
        old = *address;
        float expect = num + *val;
        is_ok = atomic_cas_bool(address, old, *(int *)&expect);
    } while (!is_ok);
    return;
}

// 所有SPE进行计算
__global__ void calc(float *dev_a)
{
    // 原子计算
    for(int i = 0; i < 1000; i++) {
        atomic_add_float(&dev_a[0], 2.0);
    }
    // 普通计算
    for(int i = 0; i < 1000; i++) {
        dev_a[1] = dev_a[1] + 2.0;
    }
}
```

15.4.3.3 完整程序

以下是完整代码（cas_sample.scpp）：

```
#include <stdio.h>
#include <stdlib.h>
#include <sdaa_atomic.h>
using namespace sdaa;

// 使用CAS实现原子浮点加
__device__ void atomic_add_float(float* val, float num)
{
    int* address = (int*)val;
    int old = *address;
    bool is_ok = true;
    do {
        old = *address;
        float expect = num + *val;
        is_ok = atomic_cas_bool(address, old, *(int *)&expect);
    } while (!is_ok);
    return;
}

// 所有SPE进行计算
__global__ void calc(float *dev_a)
{
    // 原子计算
    for(int i = 0; i < 1000; i++) {
```

```
        atomic_add_float(&dev_a[0], 2.0);
    }
    // 普通计算
    for(int i = 0; i < 1000; i++) {
        dev_a[1] = dev_a[1] + 2.0;
    }
}
#define SIZE 2
int main()
{
    float host_a[SIZE] = {0, 0};

    // 指定Kernel函数运行在0号计算核心阵列SPA上
    sdaaSetDevice(0);
    // 在设备端的Global存储空间分配大小为size的连续内存
    int size = SIZE * sizeof(float);
    float *dev_a = NULL;
    sdaaMalloc((void**)&dev_a, size);

    // 将主机端内存host_a的数据拷贝到设备端Global存储空间, 大小为size
    sdaaMemcpy(dev_a, &host_a, size, sdaaMemcpyHostToDevice);

    // 调用名为calc的核函数, 在设备端执行计算任务。
    sdaaStream_t stream;
    sdaaStreamCreate(&stream);
    calc<<<1, stream>>>(dev_a);
    sdaaDeviceSynchronize();

    // 将Global存储空间dev_a的数据拷贝到主机端host_a, 大小为size
    sdaaMemcpy(&host_a, dev_a, size, sdaaMemcpyDeviceToHost);
    printf("host_a[0]=%f, host_a[1]=%f\n", host_a[0], host_a[1]);

    // 释放dev_a指针指向的Global存储空间
    sdaaFree(dev_a);
    return 0;
}
```

15.4.4 编译程序

程序编译

执行以下编译命令, 将源文件 `cas_sample.scpp` 生成可执行文件 `cas_sample.out`。

```
tecocc cas_sample.scpp -o cas_sample.out
```

执行以下命令, 运行程序:

```
./cas_sample.out
```

执行结果

此处可以执行多次, 以便更清晰地对比出原子操作效果。

其中 `host_a[0]` 是原子操作的计算结果，符合预期，`host_a[1]` 由于不是原子操作，每次结果都不能保证一样。

```
$ ./cas_sample
host_a[0]=64000.000000, host_a[1]=42000.000000
$ ./cas_sample.out
host_a[0]=64000.000000, host_a[1]=58000.000000
$ ./cas_sample.out
host_a[0]=64000.000000, host_a[1]=62000.000000
$ ./cas_sample.out
host_a[0]=64000.000000, host_a[1]=60000.000000
```


16. 进阶实战

16.1 概述

进阶实战是SDAA C的进阶使用，即如何使用SDAA C完成算子的开发和性能调优。本章节将会通过算子开发实战的方式，从算子的基本概念、算子的开发流程以及如何对算子的性能进行调优等多个维度，展示如何通过SDAA C完成一个优质算子的开发全流程。本章节的具体内容如下：

- **算子开发实战**：介绍算子的基本概念和开发流程。
 - **开发加法算子**：介绍加法算子从分析设计到实现的全流程。
 - **开发矩阵乘算子**：介绍矩阵乘算子从分析设计到实现的全流程。
- **算子性能调优**：介绍常用的算子调优方法。
- **算子调优实战**：介绍如何使用算子调优方法优化矩阵乘算子的性能。

16.2 算子开发实战

16.2.1 什么是算子开发

算子（Operator）通常是指在深度学习、机器学习或其他数值计算领域中，用于执行特定数学操作的函数，例如：加法、矩阵乘、卷积等。

算子是模型训练推理的重要组成部分，主要用于构建、部署深度学习神经网络模型等。一个高性能的算子可以提升模型的性能，加速模型的训练和推理。

在以下场景，需要进行算子开发：

- 训练场景下，将模型迁移到太初AI加速卡上，存在不支持的算子。
- 推理场景下，将模型转换为太初推理框架支持的模型时，存在不支持的算子。
- 提升模型性能，需要开发更高性能的算子。

16.2.2 算子基本概念

算子类型（Type）

算子类型用于描述实现逻辑相同的一类算子，例如：卷积类算子、池化类算子等。

说明：

- 在一个神经网络中，同一类型的算子可能会存在多个，例如：在某个神经网络中，存在2个卷积类算子 conv1, conv2。

- 同一类型算子的实现可能会存在差别，例如：conv1, conv2都属于卷积类算子，但实现会根据具体的使用情况而不同。

算子名称 (Name)

神经网络中具体算子的名称，算子名称在同一神经网络中需要保持唯一。

算子数据结构 (Tensor)

张量 (Tensor) 是存储算子输入数据和输出数据的基本数据结构。张量的属性主要包括：名称 (name)、形状 (shape)、数据类型 (dtype) 和数据排布格式 (format)。

张量名称 (name)

张量的名称，可以根据名称对张量进行索引。

张量形状 (shape)

张量的形状用于描述张量的维度及其大小，表述形式：(S1, S2, ...SN)，其中SN是正整数。例如：

- (6)：有6个正整数的一维张量即向量，如：(1,2,3,4,5,6)
- (2,3)：有2个维度的张量，右起第一个维度矩阵宽度 (W) 是3，第二个维度矩阵高度 (H) 是2，是一个 2×3 的矩阵，如： $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$
- (3,2,3)：有3个维度的张量，右起第一个维度矩阵宽度 (W) 是3，第二个维度矩阵高度 (H) 是2，第三个维度矩阵个数 (N) 是3，表示3个 2×3 矩阵

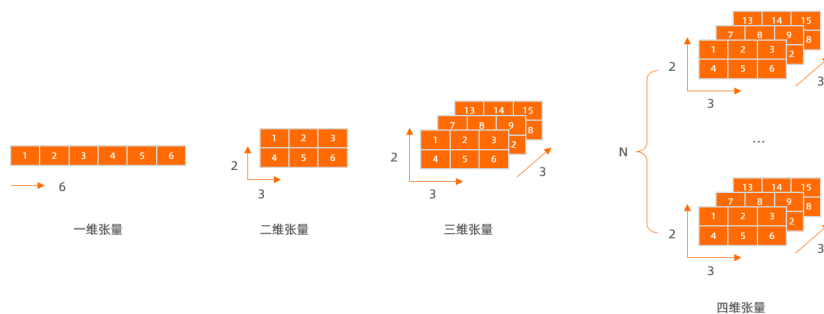


图1 张量形状

张量的数据类型 (dtype)

张量中存储的数据的类型，取值范围：int8、int16、float16、float32等。

张量的数据排布格式 (format)

在内存中张量数据的存储格式，不同的数据排布格式会影响数据的访问模式、内存利用率和算子的性能。

以四维张量(N, H, W, C)为例，四个维度结合图像方面的物理含义如下：

- N (Batch)：一个批次内图片的数量，即一次处理的图片数量。
- H (Height)：特征图的高，即垂直高度方向的像素个数。
- W (Width)：特征图的宽，即水平宽度方向的像素个数。
- C (Channels)：特征图的通道数，例如：灰度图像为通道数1，彩色RGB图像为3。

由于内存中只能进行线性存储，因此对于四维张量需要根据不同的顺序进行存储，例如：



图2 张量的数据排布格式

16.2.3 算子开发流程

算子开发的整体流程主要分为：算子分析、算子实现和编译运行三个阶段。

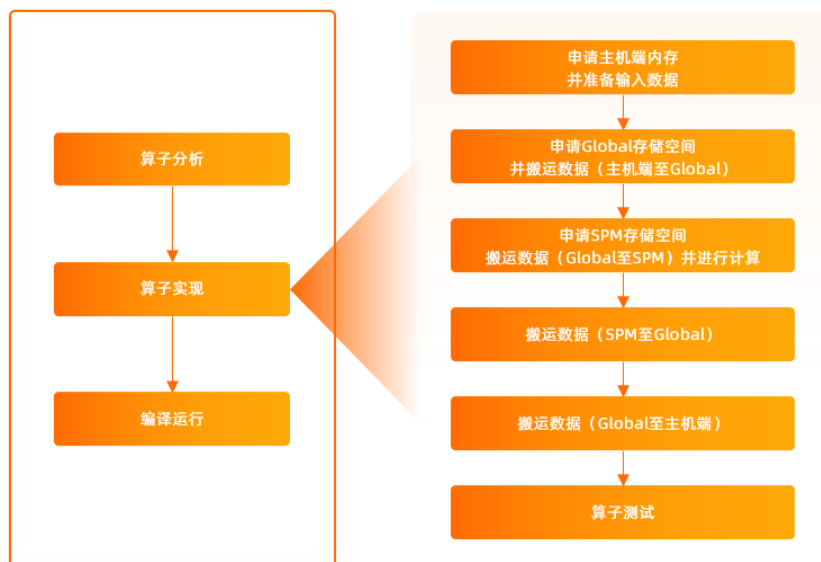


图1 算子开发流程

1. **算子分析**：主要是分析算子的数学计算逻辑、输入输出数据，并明确算子的实现流程。
2. **算子实现**：实现算子计算逻辑的核心步骤，包括：
 - a. 申请主机端内存，并搬运输入数据
 - b. 申请Global存储空间，并搬运输入数据（主机端至Global）

- c. 申请SPM存储空间，并搬运输入数据（Global至SPM）到SPM进行计算，计算结果存储在SPM。
- d. 搬运输出数据（SPM至Global）
- e. 搬运输出数据（Global至主机端）
- f. 算子测试

3. **编译运行**：开发完成的算子，编译运行后可以获得目标结果。

16.2.4 开发加法算子

16.2.4.1 算子分析

算子分析主要是分析算子的数学计算逻辑、输入输出数据，并明确算子的实现流程。本文以二维张量的加法为例，介绍如何进行算子分析。

确定算子名称（name）

算子的名称需要体现算子的功能，本文的加法算子的名称以 `kernel_add` 为例，即Kernel函数的名称为 `kernel_add()`。

明确算子任务

计算两个二维张量的加法，即将两个输入矩阵的每个数据进行一一对应的相加：

```
z = x + y
```

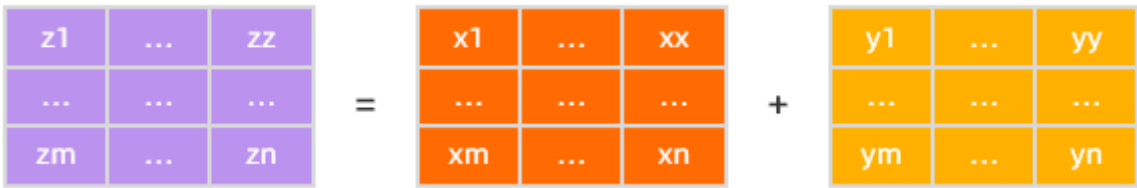


图1 加法计算逻辑

确定算子数据结构(Tensor)

加法算子中有两组输入数据，一组输出数据。两组输入数据的名称分别为 `x`，`y`，输出数据的名称为 `z`；输入输出数据的格式为2维(H,W)张量，输入输出数据的形状信息为(256,32)，即256行32列的矩阵。数据类型为单精度浮点型float，详细信息如下：

输入输出	名称 (name)	形状 (shape)	数据类型（data type）	数据排布格式（format）
输入数据	x	(256,32)	float	(H,W)

输入输出	名称 (name)	形状 (shape)	数据类型 (data type)	数据排布格式 (format)
输入数据	y	(256,32)	float	(H,W)
输出数据	z	(256,32)	float	(H,W)

16.2.4.2 算子实现

16.2.4.2.1 实现流程

算子实现是实现算子计算逻辑的核心步骤，其主要流程如下：

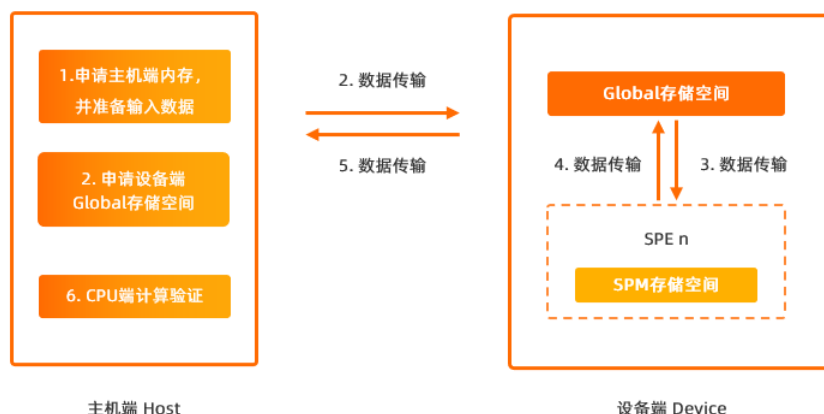


图1 算子开发流程

- 1. 申请主机端内存，并准备输入数据：**在主机端申请内存空间，并准备输入数据。
- 2. 申请Global存储空间，并传输数据（主机端至Global）：**申请Global存储空间，并将输入数据从主机端搬运至Global存储空间。
- 3. 申请SPM存储空间，并传输数据（Global至SPM）到SPM进行计算：**申请SPM存储空间，将数据从Global存储空间搬运至SPM存储空间，进行加速计算，并得出计算结果。
- 4. 传输数据（SPM至Global）：**将计算结果从SPM存储空间搬运至Global存储空间。
- 5. 传输数据（Global至主机端）：**将计算结果从Global存储空间搬运至主机端。
- 6. 算子测试：**算子的精度测试，通常是在主机端（业内认可度高、精度高的硬件芯片，如：X86架构的高精度芯片）执行相同的计算，将计算结果与设备端传输回的计算结果进行比对验证。

? 说明：

其中，第1, 2, 5, 6步骤是主机端需要设计和执行的，代码都在主机端运行。而第3, 4步骤是设备端需要设计和执行的，代码在设备端运行。

16.2.4.2.2 步骤一：申请主机端内存并初始化输入数据

本文主要介绍如何申请主机端内存，以及如何初始化输入数据 `x`，`y`。

编程思路

1. 主机端代码结构与CPU端代码结构相同，从 `main()` 函数开始。

```
int main { }
```

2. 在 `main()` 函数中，设置设备端进行加法计算的SDAA计算设备编号，本文以0号SDAA计算设备为例。

```
int main {  
    // 设置0号SDAA计算设备，用于进行加法计算  
    sdaaSetDevice(0);  
}
```

3. 申请主机端输入数据和输出数据的内存空间。

```
float *hdata_x = (float *)malloc(FSIZE);  
float *hdata_y = (float *)malloc(FSIZE);  
float *hdata_z = (float *)malloc(FSIZE);
```

4. 初始化输入数据 `x`，`y`，初始化方式可以根据实际情况自行选择。

```
for (int i = 0; i < TOTAL_NUM; i++) {  
    hdata_x[i] = (float)i / TOTAL_NUM;  
    hdata_y[i] = sin((float)i / TOTAL_NUM);  
}
```

16.2.4.2.3 步骤二：申请Global存储空间并搬运数据（主机端至Global）

本文主要介绍如何申请Global存储空间，以及如何将输入数据 `x`，`y` 从主机端搬运至Global存储空间。

编程思路

1. 申请Global存储空间。

```
// 定义设备端指针初始化为NULL  
float *dev_data_x = NULL;  
float *dev_data_y = NULL;
```

```
float *dev_data_z = NULL;

// 申请Global存储空间, 并返回地址给之前定义的指针
sdaaMalloc((void **)&dev_data_x, FSIZE);
sdaaMalloc((void **)&dev_data_y, FSIZE);
sdaaMalloc((void **)&dev_data_z, FSIZE);
```

2. 将输入数据 `x` , `y` 从主机端搬运至Global存储空间。

```
// 将主机端数据复制到Global存储空间; 需要配置sdaaMemcpyHostToDevice, 数据传输方向是主机端至设备端
sdaaMemcpy(dev_data_x, hdata_x, FSIZE, sdaaMemcpyHostToDevice);
sdaaMemcpy(dev_data_y, hdata_y, FSIZE, sdaaMemcpyHostToDevice);
```

16.2.4.2.4 步骤三：申请SPM存储空间并搬运数据（Global至SPM）至SPM进行计算

本文主要介绍如何申请SPM存储空间，并将数据从Global存储空间搬运至SPM存储空间进行加速计算，得出计算结果。

数据切分

SDAA C遵循SPMD编程范式，同一计算核心阵列SPA内所有计算核心SPE运行同一份应用程序，每个SPE可以独立完成对子问题的并行求解。其核心在于对复杂计算任务的切分，并合理地将计算任务分配到不同的SPE内进行计算。

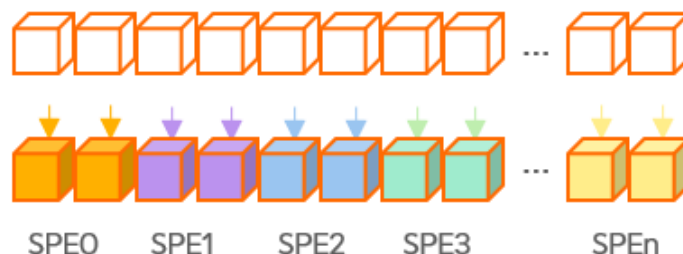


图1 基于SDAA C的SPMD编程范式

SDAA C提供了`threadIdx`和`threadDim`为每个SPE定制计算任务。其中：

- `threadIdx`：当前计算核心SPE的ID号，可以通过该关键字为目标SPE分配计算任务。
- `threadDim`：同一计算核心阵列SPA内计算核心SPE的总数。

编程思路

1. 根据SPE的总数 `threadDim` 切分数据，每个SPE分得 `tile_num` 个float数据，并计算出每块数据的大小，单位：字节。

```
uint32_t tile_num = total_num / threadDim;
uint32_t spm_size = tile_num * sizeof(float);
```

2. 申请SPM存储空间，大小为步骤一中每块数据的字节数目。

```
float *spm_x = (float *)malloc(spm_size);
float *spm_y = (float *)malloc(spm_size);
float *spm_z = (float *)malloc(spm_size);
```

3. 将数据从Global存储空间搬运至SPM存储空间。

```
memcpy(spm_x, (x + threadIdx * tile_num), spm_size);
memcpy(spm_y, (y + threadIdx * tile_num), spm_size);
```

4. 每个SPE处理 `tile_num` 个float数据，且循环执行 `tile_num` 次的加法计算。

```
for (uint32_t i = 0; i < tile_num; i++) {
    spm_z[i] = spm_x[i] + spm_y[i];
}
```

16.2.4.2.5 步骤四：搬运数据（SPM至Global）

本文主要介绍如何将计算结果从SPM存储空间搬运至Global存储空间。

编程思路

将数据从SPM存储空间搬运至Global存储空间。

```
memcpy((z + threadIdx * tile_num), spm_z, spm_size);
```

设备端Kernel函数

```
// x, y: 表示输入的Global地址, z: 表示计算结果的Global地址
__global__ void kernel_add(float *x, float *y, float *z, uint32_t total_num)
{
    // 将所有数据切分为threadDim块, 每块含有tile_num个数据, 分给不同的SPE去计算
    uint32_t tile_num = total_num / threadDim;
    // 计算出tile_num个数据占用多少字节的数据空间
    uint32_t spm_size = tile_num * sizeof(float);
    // 在SPE为x, y, z申请SPM存储空间
    // 使用SPM存储空间可以加快数据的访问速度, 提升性能
    float *spm_x = (float *)malloc(spm_size);
    float *spm_y = (float *)malloc(spm_size);
    float *spm_z = (float *)malloc(spm_size);
    // 使用输入数据x, y的首地址 + threadIdx * tile_num 偏移坐标作为搬运数据的源地址
    // 对Global存储空间的数据进行切分后, 搬运到SPM存储空间
    memcpy(spm_x, (x + threadIdx * tile_num), spm_size);
```



```
memcpy(spm_y, (y + threadIdx * tile_num), spm_size);

// 每个计算核心SPE相隔tile_num个float数据, 且循环tile_num次, 进行加法计算
for (uint32_t i = 0; i < tile_num; i++) {
    // 将SPM上所有输入数据相加, 并将计算结果输出到对应的SPM存储空间
    spm_z[i] = spm_x[i] + spm_y[i];
}

// 用输出数据z的首地址 + threadIdx * tile_num 偏移坐标作为搬运数据的目的地址,
// 将SPM存储空间上的计算结果搬运到Global存储空间
memcpy((z + threadIdx * tile_num), spm_z, spm_size);
}
```

16.2.4.2.6 步骤五：搬运数据（Global至主机端）

本文主要介绍如何将计算结果从Global存储空间搬运至主机端。

编程思路

在主机端执行以下代码，将计算结果从Global存储空间搬运至主机端。

```
sdaaMemcpy(hdata_z, dev_data_z, FSIZE, sdaaMemcpyDeviceToHost);
```

16.2.4.2.7 步骤六：算子测试

算子的精度测试，通常是在主机端（业内认可度高、精度高的硬件芯片，如：X86架构的高精度芯片）执行相同的计算，将计算结果与设备端传回的计算结果进行比对验证，从而确定计算结果的精度。

本文主要介绍如何验证算子的精度。

确定测试精度

测试精度通常根据算子的计算逻辑和输入输出数据类型，进行分析确认。

本示例的add加法算子，输入输出的数据类型是float（单精度浮点数），其单位精度是 2^{-23} ，约为1.19e-07。因此，本文将绝对值误差定为5e-07。即：CPU计算的结果，和SDAA计算设备计算的结果之间的绝对值误差不超过 5e-07。

? 说明：

您也可以根据实际情况对主机端和设备端的绝对值误差进行调整。

精度调测

精度调测是将设备端输出的计算结果和主机端（X86架构的高精度芯片或业界其他标杆硬件）的计算数据结果进行逐一对比。

```
// 申请主机端的内存空间，用于存放主机端的加法计算结果
float *golden_z = (float *)malloc(FSIZE);
for (int i = 0; i < TOTAL_NUM; i++) {
    // 在主机端执行加法计算
    golden_z[i] = hdata_x[i] + hdata_y[i];
    // 主机端计算结果和设备端计算结果的绝对值误差如果大于5e-7，则提示精度不达标
    if(abs(golden_z[i] - hdata_z[i]) > 5e-7) {
        printf(" %dth value: %f is wrong! \n", i, hdata_z[i]);
        break;
    }
    if(i == TOTAL_NUM - 1) {
        printf(" SUCCESS! \n");
    }
}
```

? 说明：

- 可以通过数据的标号确认精度存在误差的数据。
- 如果绝对值误差不达标（大于 $5e-7$ ），可以从计算和数据存储两个主要维度，分析问题的原因：
 - 计算：
 - 查看算子的计算逻辑是否有误。
 - 使用的计算指令是否正确恰当。如果指令本身有问题或者使用有问题，也会引起精度问题。
 - 数据存储：
 - 检查使用的数据地址是否正确。
 - 异步数据操作时，操作同步不当，引起的数据踩踏的问题，也会导致精度错误。

主机端测试代码

在主机端编写和设备端Kernel函数相同计算逻辑的验证函数。针对 `kernel_add()` 函数，主机端的测试代码如下：

```
#define TOTAL_NUM (256 * 32)
#define FSIZE (TOTAL_NUM * sizeof(float))

int main()
{
    // 申请主机端的输入数据和输出数据的内存空间
    float *hdata_x = (float *)malloc(FSIZE);
    float *hdata_y = (float *)malloc(FSIZE);

    // 初始化输入数据x, y的值，初始化方式根据实际情况自行选择
    for (int i = 0; i < TOTAL_NUM; i++) {
        hdata_x[i] = (float)(i / TOTAL_NUM);
        hdata_y[i] = (float)sin(i / TOTAL_NUM);
    }
}
```

```

// 申请主机端的内存空间，用于存放主机端的加法计算结果
float *golden_z = (float *)malloc(FSIZE);
for (int i = 0; i < TOTAL_NUM; i++) {
    // 在主机端执行加法计算
    golden_z[i] = hdata_x[i] + hdata_y[i];
    // 主机端计算结果和设备端计算结果的绝对值误差如果大于5e-7，则提示精度不达标
    if(abs(golden_z[i] - hdata_z[i]) > 5e-7) {
        printf(" %dth value: %f is wrong! \n", i, hdata_z[i]);
        break;
    }
    if(i == TOTAL_NUM - 1) {
        printf(" SUCCESS! \n");
    }
}
}

```

16.2.4.2.8 完整程序

加法算子 `kernel_add()` 的完整程序如下：

```

// 使用sdaa空间名
using namespace sdaa;
#define TOTAL_NUM (256 * 32)
#define FSIZE (TOTAL_NUM * sizeof(float))

// x, y: 表示输入的Global地址, z: 表示计算结果的Global地址
__global__ void kernel_add(float *x, float *y, float *z, uint32_t total_num)
{
    // 将所有数据切分为threadDim块，每块含有tile_num个数据，分给不同的SPE去计算
    uint32_t tile_num = total_num / threadDim;
    // 计算出tile_num个数据占用多少字节的数据空间
    uint32_t spm_size = tile_num * sizeof(float);
    // 在SPE为x, y, z申请SPM存储空间
    // 使用SPM存储空间可以加快数据的访问速度，提升性能
    float *spm_x = (float *)malloc(spm_size);
    float *spm_y = (float *)malloc(spm_size);
    float *spm_z = (float *)malloc(spm_size);
    // 使用输入数据x, y的首地址 + threadIdx * tile_num 偏移坐标作为搬运数据的源地址
    // 对Global存储空间的数据进行切分后，搬运到SPM存储空间
    memcpy(spm_x, (x + threadIdx * tile_num), spm_size);
    memcpy(spm_y, (y + threadIdx * tile_num), spm_size);

    // 每个计算核心SPE相隔tile_num个float数据，且循环tile_num次，进行加法计算
    for (uint32_t i = 0; i < tile_num; i++) {
        // 将SPM上所有输入数据相加，并将计算结果输出到对应的SPM存储空间
        spm_z[i] = spm_x[i] + spm_y[i];
    }
    // 用输出数据z的首地址 + threadIdx * tile_num 偏移坐标作为搬运数据的目的地址
    // 将SPM存储空间上的计算结果搬运到Global存储空间
    memcpy((z + threadIdx * tile_num), spm_z, spm_size);
}

int main()
{

```

```

// 设置0号SDAA计算设备, 用于进行加法计算
sdaaSetDevice(0);
// 申请主机端输入数据和输出数据的内存空间
float *hdata_x = (float *)malloc(FSIZE);
float *hdata_y = (float *)malloc(FSIZE);
float *hdata_z = (float *)malloc(FSIZE);
// 初始化输入数据x, y的值, 初始化方式可以根据实际情况自行选择
for (int i = 0; i < TOTAL_NUM; i++) {
    hdata_x[i] = (float)(i / TOTAL_NUM);
    hdata_y[i] = (float)sin(i / TOTAL_NUM);
}
// 定义设备端指针初始化为NULL
float *dev_data_x = NULL;
float *dev_data_y = NULL;
float *dev_data_z = NULL;

// 申请Global存储空间, 并返回地址给之前定义的指针
sdaaMalloc((void **)&dev_data_x, FSIZE);
sdaaMalloc((void **)&dev_data_y, FSIZE);
sdaaMalloc((void **)&dev_data_z, FSIZE);

// 将主机端数据复制到Global存储空间; 需要配置sdaaMemcpyHostToDevice, 数据传输方向是主机端至设备端
sdaaMemcpy(dev_data_x, hdata_x, FSIZE, sdaaMemcpyHostToDevice);
sdaaMemcpy(dev_data_y, hdata_y, FSIZE, sdaaMemcpyHostToDevice);

// 调用Kernel函数, 将实际输入数据、输出数据的地址填入函数参数, 进行计算
kernel_add<<1>>>(dev_data_x, dev_data_y, dev_data_z, TOTAL_NUM);
// 进行同步, 等待所有SPE运行完Kernel函数
sdaaDeviceSynchronize();
// 将计算完成的数据从Global搬运到主机端内存
sdaaMemcpy(hdata_z, dev_data_z, FSIZE, sdaaMemcpyDeviceToHost);

// 申请主机端的内存空间, 用于存放主机端的加法计算结果
float *golden_z = (float *)malloc(FSIZE);
for (int i = 0; i < TOTAL_NUM; i++) {
    // 在主机端执行加法计算
    golden_z[i] = hdata_x[i] + hdata_y[i];
    // 主机端计算结果和设备端计算结果的绝对值误差如果大于5e-7, 则提示精度不达标
    if(abs(golden_z[i] - hdata_z[i]) > 5e-7) {
        printf(" %dth value: %f is wrong! \n", i, hdata_z[i]);
        break;
    }
    if(i == TOTAL_NUM - 1) {
        printf(" SUCCESS! \n");
    }
}

// 数据使用完后, 释放主机端内存
free(hdata_x);
free(hdata_y);
free(hdata_z);
free(golden_z);
// 数据使用完后, 释放Global存储空间
sdaaFree(dev_data_x);
sdaaFree(dev_data_y);
sdaaFree(dev_data_z);

return 0;
}

```

16.2.4.3 编译运行

算子代码编写完成后，编译并运行可以获得算子的计算结果。本文主要介绍如何进行编译和运行。

操作步骤

1. 将完整程序中的代码复制到 `example.scpp` 文件中。
2. 执行以下编译命令，将 `example.scpp` 生成可执行文件 `a.out`。

```
tecocc example.scpp
```

3. 执行以下命令，运行程序。

```
./a.out
```

执行结果

输出以下内容，表示算子可以正常运行，且精度达标，即：主机端计算结果和设备端计算结果的绝对值误差小于 $5e-7$ 。

```
SUCCESS!
```

16.2.5 开发矩阵乘算子

16.2.5.1 算子分析

本例将在太初AI加速卡上使用SDAA C开发矩阵乘算子。

确定算子名称 (name)

矩阵乘算子的名称为 `kernel_matmul`，即Kernel函数的名称为 `kernel_matmul()`。

确定算子任务

计算两个矩阵的乘积，计算公式：

$$C = A \times B$$

确定算子数据结构(Tensor)

矩阵乘法算子中有两组输入数据，一组输出数据。输入矩阵分别为A和B，输出矩阵为C，详细信息如下：

输入输出	名称 (name)	形状 (shape)	数据类型 (data type)	数据排布格式 (format)
输入矩阵	A	(32,128,256)	int16	(N, H, W)
输入矩阵	B	(32,256,32)	int16	(N, H, W)
输出矩阵	C	(32,128,32)	int16	(N, H, W)

算子实现逻辑分析

矩阵切分

为提升矩阵乘算子的计算性能，可以对矩阵A和B进行数据切分。对于单个Batch (1,H,W)，将矩阵A和矩阵B切分后：

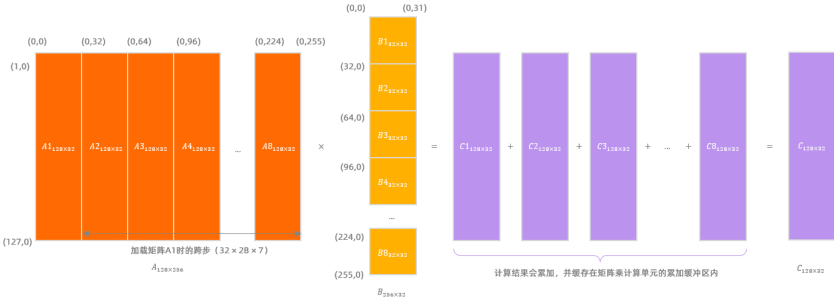
$$[A \times B] = [A1 \ A2 \ \dots \ A8] \times \begin{bmatrix} B1 \\ B2 \\ \vdots \\ B8 \end{bmatrix} = [A1 \times B1 + A2 \times B2 + \dots + A8 \times B8]$$


图1 矩阵切分

其中：

- A矩阵(128,256)，分为8个(128,32)的小矩阵；B矩阵(256,32)分为8个(32,32)的小矩阵。A和B矩阵切分后：
 - A矩阵，每次加载的数据量是 $m \times k$ ，其中：行数m是128，列数k是32。
 - B矩阵，每次加载的数据量是 $k \times n$ ，其中：行数k是32，列数n是32。
- 切分后的矩阵相乘的计算结果会累加，并缓存在矩阵乘计算单元的累加缓冲区内。

坐标转换

矩阵是二维向量，而在实际内存中数据的排布是一维线性的。因此，在进行二维矩阵乘法时，需要将二维矩阵的元素坐标换算为内存中的真实地址。

例如：

- 矩阵A的维度是(M, K)，某一矩阵元素 A_{mk} 的坐标是(m,k)，A矩阵第一行第一列的元素 A_{00} 在内存中的地址为 $pBase_a$ ，则 A_{mk} 在内存中的真实地址：

$$pAddr_a = pBase_a + (m \cdot K + k)$$

- 矩阵B的维度是(K, N)，某一矩阵元素 B_{kn} 的坐标是(k,n)，B矩阵第一行第一列的元素 B_{00} 在内存中的地址为 $pBase_b$ ，则 B_{kn} 在内存中的真实地址：

$$pAddr_b = pBase_b + (k \cdot N + n)$$

16.2.5.2 算子实现

16.2.5.2.1 设备端矩阵乘

核心编程思路

- 申请SPM存储空间**：使用SPM存储空间进行矩阵乘运算，性能更好。申请SPM存储空间，用于存储输入矩阵A和B，以及输出矩阵C的数据。
- 矩阵切分**：将切分后的8次矩阵乘划分为4组，每次做2个矩阵乘，共循环4次，循环次数用j表示，取值范围0~3。
- 配置矩阵乘的参数**，例如：是否需要刷出累加缓冲区数据，输入矩阵的行数偏移等，可参考SDAA C编程指南的[非阻塞型矩阵乘](#)章节。
- 加载权重矩阵B**：矩阵B每次循环的起始元素为 B_{k0} ，其中：第一个矩阵 $k = (2 \times j) \times 32$ ，第二个矩阵 $k = (2 \times j + 1) \times 32$ 。

由矩阵B任意元素在内存中真实地址的计算公式 $pAddr_b = pBase_b + (k \cdot N + n)$ 可以得出，两个矩阵起始元素在内存中的真实地址是：

$$pBase_b + ((2 \times j) \times 32 \times 32 + 0) \text{ 和 } pBase_b + ((2 \times j + 1) \times 32 \times 32 + 0)。$$

获取到矩阵B起始元素在内存中的真实地址后，传入`matmul_load_weight`接口，从SPM的指定地址加载权重矩阵到矩阵乘计算单元上。

- 加载输入矩阵A**：矩阵A每次循环的起始元素为 A_{0k} ，其中：第一个矩阵 $k = (2 \times j) \times 32$ ，第二个矩阵 $k = (2 \times j + 1) \times 32$ 。

由矩阵A任意元素在内存中真实地址的计算公式 $pAddr_a = pBase_a + (m \cdot K + k)$ 可以得出，两个矩阵起始元素在内存中的真实地址是：

$$pBase_a + (0 + (2 \times j) \times 32) \text{ 和 } pBase_a + (0 + (2 \times j + 1) \times 32)。$$

获取到矩阵A起始元素在内存中的真实地址后，传入`matmul_compute`接口，从SPM的指定地址加载输入矩阵，并与加载好的权重矩阵做矩阵乘运算。

 说明：

对于矩阵A1，第一行数据和第二行数据在内存里并不是连续的，所以在加载第一行以后的数据时，每次换行均需要跨过 32×7 个int16数据。而跨步设置的基本单位是64B，正好是32个int16数据，因此跨步的步长是 $32 \times 7 \times 2B / 64B = 7$ 。

6. 计算结果累加并输出：将计算结果从累加缓冲区输出到SPM存储空间（`matmul_store`），再从SPM存储空间搬运到Global存储空间。

代码实现

设备端矩阵乘的代码如下：

```
#include <sdaa_matmul.h>

using namespace sdaa;

// B 数据batch大小
#define B 32
// M维度大小
#define M 128
// N维度大小
#define N 32
// K维度大小
#define K 256
// K维度切分的最小颗粒度
#define K_BLK 32

// 矩阵乘的Kernel函数，参数是输入矩阵a, b和输出矩阵c的地址
__global__ void kernel_matmul(char *dataA, char *dataB, char *dataC) {

    // 计算每个SPE 要平均分配多少batch的数据量
    int32_t batch_spe = B/threadDim;
    // 一个SPE分别被分得的 矩阵a, b, c的batch数的总数据量大小
    int32_t a_size_spe = batch_spe * M * K * sizeof(int16_t);
    int32_t b_size_spe = batch_spe * K * N * sizeof(int16_t);
    int32_t c_size_spe = batch_spe * M * N * sizeof(int32_t);
    // 给输入矩阵a（输入矩阵）申请SPM存储空间
    int16_t *spm_input = (int16_t *)malloc(a_size_spe);
    // 通过threadIdx做地址偏移，将当前spe分得的输入矩阵a的数据从Global内存上复制到
    // SPM上申请的a矩阵数据空间
    memcpy_async(spm_input, dataA + threadIdx * a_size_spe, a_size_spe, MemcpyGlobalToSpm);

    // 给输入矩阵b（权重矩阵）申请SPM存储空间
    int16_t *spm_weight = (int16_t *)malloc(b_size_spe);
    // 通过threadIdx做地址偏移，将当前spe分得的输入矩阵b的数据从Global内存上复制到
    // SPM上申请的b矩阵数据空间
    memcpy_async(spm_weight, dataB + threadIdx * b_size_spe, b_size_spe, MemcpyGlobalToSpm);
    // 等待矩阵a, b数据复制完成
    memcpy_wait();
    // 给输出矩阵c 申请SPM存储空间
    int32_t *spm_output = (int32_t *)malloc(c_size_spe);

    // 定义一个矩阵乘的句柄，可以将矩阵乘的一些配置项设置到句柄里。这样矩阵乘执行时，
    // 可以按照句柄里的配置来进行计算
    MatmulHandle handle;
    // 配置输入数据是short 类型，输出数据是int型
```



```

matmul_init(handle, MatmulShortToInt);
// 计算一次矩阵乘 K 的切分是32一块，一个循环计算2次。计算出循环次数
uint32_t chunks = K / (K_BLK * 2);

for (int j = 0; j < chunks; j++) {
    // 等待输入矩阵a矩阵数据加载完成，若之前没有加载动作，等待也会自动跳过
    matmul_wait_loading_input(handle);
    // 等待权重矩阵b矩阵数据加载完成，若之前没有加载动作，等待也会自动跳过
    matmul_wait_loading_weight(handle);
    // 设置本次matmul计算完成后，是否需要把数据从累加buffer 刷出到SPM上
    // 本次计算不是最后一次，所以不需要设置刷出数据
    matmul_set_flushing_output(handle, false);
    // 设置输入矩阵a的偏移量，通常是输入矩阵行数不够128，且没有从128行的第一行
    // 开始加载时，才需要设置的偏移量。此处输入是128行，且从128行里的第一行开始
    // 所以设置为0
    matmul_set_output_row_offset(handle, MatmulEnableOutputRowOffset, 0);
    // 加载权重矩阵b，由之前计算出循环的第一次地址是 pBase_b + ((2 * j) * 32 * 32 + 0)，
    // 对应到代码中是，spm_weight + K_BLK * K_BLK * j * 2
    matmul_load_weight(handle, spm_weight + K_BLK * K_BLK * j * 2, MatmulK32, MatmulN32);
    // 加载输入矩阵并进行计算，由之前计算出第一次地址是 pBase_a + (2 * j) * 32，
    // 对应到代码中是，spm_input + K_BLK * j * 2，注意跨步的基本单位是 64B，
    // 跨步的步长是7(即 K / K_BLK - 1)
    matmul_compute(handle, spm_input + K_BLK * j * 2, M, MatmulK32, (K / K_BLK - 1));

    // 设置本次matmul计算完成后，是否需要把数据从累加buffer 刷出到SPM上。
    // 当j == (chunks - 1)，即最后一次计算后，需要把数据刷出到SPM上
    matmul_set_flushing_output(handle, (j == (chunks - 1)));
    // 设置输入矩阵a的偏移量，通常是输入矩阵行数不够128，且没有从128行的第一行
    // 开始加载时，才需要设置的偏移量。此处输入是128行，且从128行里的第一行开始
    // 所以设置为0
    matmul_set_output_row_offset(handle, MatmulEnableOutputRowOffset, 0);
    // 加载权重矩阵b，由之前计算出循环的第一次地址是 pBase_b + ((2 * j + 1) * 32 * 32 + 0)，
    // 对应到代码中是，spm_weight + K_BLK * K_BLK * (j * 2 + 1)
    matmul_load_weight(handle, spm_weight + K_BLK * K_BLK * (j * 2 + 1),
                        MatmulK32, MatmulN32);
    // 加载输入矩阵并进行计算，由之前计算出第一次地址是 pBase_a + (2 * j + 1) * 32，
    // 对应到代码中是，spm_input + K_BLK * (j * 2 + 1)，注意跨步的基本单位是 64B，
    // 跨步的步长是7(即 K / K_BLK - 1)
    matmul_compute(handle, spm_input + K_BLK * (j * 2 + 1), M, MatmulK32,
                    (K / K_BLK - 1));
}
// 把累加buffer 上的数据，输出到已申请的SPM地址 spm_output上
matmul_store(handle, spm_output, M, MatmulN32);
// 等待数据刷出完成
matmul_wait(handle);
// 通过threadIdx做地址偏移，将当前spe计算出的输出矩阵c的数据复制到Global上
memcpy(dataC + threadIdx * c_size_spe, spm_output, c_size_spe);
// 释放申请的SPM存储空间
free(spm_output);
free(spm_weight);
free(spm_input);
}

```

16.2.5.2.2 主机端矩阵乘

在主机端同样做一次矩阵A和矩阵B的乘法计算，生成用于对比的数据。主机端矩阵乘的代码如下：

```

#include <time.h>

// B 数据batch大小
#define B 32
// M维度大小
#define M 128
// N维度大小
#define N 32
// K维度大小
#define K 256
// K维度切分的最小颗粒度
#define K_BLK 32

int main() {
    int b, i, j, k;

    int a_size = B * M * K;
    int b_size = B * K * N;
    int c_size = B * M * N;

    unsigned int local_seed = time(NULL);

    // 在Host 端申请输入矩阵a的内存空间
    int16_t *host_a = (int16_t *)malloc(a_size * sizeof(int16_t));
    // 在Host 端申请输入矩阵b的内存空间
    int16_t *host_b = (int16_t *)malloc(b_size * sizeof(int16_t));
    // 在Host 端申请需要从设备端复制数据到Host端输出矩阵c的内存空间
    int *host_c = (int *)malloc(c_size * sizeof(int));
    // 在Host 端申请Host 自己计算的输出矩阵c 的对比数据的内存空间
    int *true_c = (int *)malloc(c_size * sizeof(int));
    // 初始化矩阵a, b的输入数据
    memset(host_a, 0, a_size * sizeof(int16_t));
    memset(host_b, 0, b_size * sizeof(int16_t));
    for (b = 0; b < B; b++) {
        for (i = 0; i < M; i++) {
            for (j = 0; j < K; j++) {
                host_a[b * M * K + i * K + j] = rand_r(&local_seed) % 100 - 50;
            }
        }
        for (i = 0; i < K; i++) {
            for (j = 0; j < N; j++) {
                host_b[b * K * N + i * N + j] = rand_r(&local_seed) % 5 - 2;
            }
        }
    }

    memset(true_c, 0, c_size * sizeof(int));
    // Host端计算 用于对比的输出矩阵c的数据
    for (b = 0; b < B; b++) {
        for (i = 0; i < M; i++) {
            for (j = 0; j < N; j++) {
                for (k = 0; k < K; k++) {
                    true_c[b * M * N + i * N + j] +=
                        ((int)host_a[b * M * K + i * K + k])
                        * ((int)host_b[b * K * N + k * N + j]);
                }
            }
        }
    }
}

```

```

    }
}

// 释放Host端内存空间
free(true_c);
free(host_c);
free(host_b);
free(host_a);

return 0;
}

```

16.2.5.2.3 完整程序

将设备端计算出的结果复制到主机端，并和主机端生成数据，进行逐一对比。完整代码如下：

```

#include <time.h>
#include <sdaa_matmul.h>

using namespace sdaa;

// B 数据batch大小
#define B 32
// M维度大小
#define M 128
// N维度大小
#define N 32
// K维度大小
#define K 256
// K维度切分的最小颗粒度
#define K_BLK 32

// 矩阵乘的核函数，参数是输入矩阵a, b和输出矩阵c的地址
__global__ void kernel_matmul(char *dataA, char *dataB, char *dataC) {

    // 计算每个SPE 要平均分配多少batch的数据量
    int32_t batch_spe = B/threadDim;
    // 一个SPE分别被分得的 矩阵a, b, c的batch数的总数据量大小
    int32_t a_size_spe = batch_spe * M * K * sizeof(int16_t);
    int32_t b_size_spe = batch_spe * K * N * sizeof(int16_t);
    int32_t c_size_spe = batch_spe * M * N * sizeof(int32_t);
    // 给输入矩阵a（输入矩阵）申请SPM存储空间
    int16_t *spm_input = (int16_t *)malloc(a_size_spe);
    // 通过threadIdx做地址偏移，将当前spe分得的输入矩阵a的数据从Global内存上复制到
    // SPM上申请的a矩阵数据空间
    memcpy_async(spm_input, dataA + threadIdx * a_size_spe, a_size_spe, MemcpyGlobalToSpm);

    // 给输入矩阵b（权重矩阵）申请SPM存储空间
    int16_t *spm_weight = (int16_t *)malloc(b_size_spe);
    // 通过threadIdx做地址偏移，将当前spe分得的输入矩阵b的数据从Global内存上复制到
    // SPM上申请的b矩阵数据空间
    memcpy_async(spm_weight, dataB + threadIdx * b_size_spe, b_size_spe, MemcpyGlobalToSpm);
    // 等待矩阵a, b数据复制完成
    memcpy_wait();
    // 给输出矩阵c 申请SPM存储空间
    int32_t *spm_output = (int32_t *)malloc(c_size_spe);

```

```

// 定义一个矩阵乘的句柄，可以将矩阵乘的一些配置项设置到句柄里。这样矩阵乘执行时，
// 可以按照句柄里的配置来进行计算
MatmulHandle handle;
// 配置输入数据是short 类型，输出数据是int型
matmul_init(handle, MatmulShortToInt);
// 计算一次矩阵乘 K 的切分是32一块，一个循环计算2次。计算出循环次数
uint32_t chunks = K / (K_BLK * 2);

for (int j = 0; j < chunks; j++) {
    // 等待输入矩阵a矩阵数据加载完成，若之前没有加载动作，等待也会自动跳过
    matmul_wait_loading_input(handle);
    // 等待权重矩阵b矩阵数据加载完成，若之前没有加载动作，等待也会自动跳过
    matmul_wait_loading_weight(handle);
    // 设置本次matmul计算完成后，是否需要把数据从累加buffer 刷出到SPM上
    // 本次计算不是最后一次，所以不需要设置刷出数据
    matmul_set_flushing_output(handle, false);
    // 设置输入矩阵a的偏移量，通常是输入矩阵行数不够128，且没有从128行的第一行
    // 开始加载时，才需要设置的偏移量。此处输入是128行，且从128行里的第一行开始
    // 所以设置为0
    matmul_set_output_row_offset(handle, MatmulEnableOutputRowOffset, 0);
    // 加载权重矩阵b，由之前计算出循环的第一次地址是 pBase_b + ((2 * j) * 32 * 32 + 0)，
    // 对应到代码中是，spm_weight + K_BLK * K_BLK * j * 2
    matmul_load_weight(handle, spm_weight + K_BLK * K_BLK * j * 2, MatmulK32, MatmulN32);
    // 加载输入矩阵并进行计算，由之前计算出第一次地址是 pBase_a + (2 * j) * 32，
    // 对应到代码中是，spm_input + K_BLK * j * 2，注意跨步的基本单位是 64B，
    // 跨步的步长是7(即 K / K_BLK - 1)
    matmul_compute(handle, spm_input + K_BLK * j * 2, M, MatmulK32, (K / K_BLK - 1));

    // 设置本次matmul计算完成后，是否需要把数据从累加buffer 刷出到SPM上。
    // 当j == (chunks - 1)，即最后一次计算后，需要把数据刷出到SPM上
    matmul_set_flushing_output(handle, (j == (chunks - 1)));
    // 设置输入矩阵a的偏移量，通常是输入矩阵行数不够128，且没有从128行的第一行
    // 开始加载时，才需要设置的偏移量。此处输入是128行，且从128行里的第一行开始
    // 所以设置为0
    matmul_set_output_row_offset(handle, MatmulEnableOutputRowOffset, 0);
    // 加载权重矩阵b，由之前计算出循环的第一次地址是 pBase_b + ((2 * j + 1) * 32 * 32 + 0)，
    // 对应到代码中是，spm_weight + K_BLK * K_BLK * (j * 2 + 1)
    matmul_load_weight(handle, spm_weight + K_BLK * K_BLK * (j * 2 + 1),
                        MatmulK32, MatmulN32);
    // 加载输入矩阵并进行计算，由之前计算出第一次地址是 pBase_a + (2 * j + 1) * 32，
    // 对应到代码中是，spm_input + K_BLK * (j * 2 + 1)，注意跨步的基本单位是 64B，
    // 跨步的步长是7(即 K / K_BLK - 1)
    matmul_compute(handle, spm_input + K_BLK * (j * 2 + 1), M, MatmulK32,
                    (K / K_BLK - 1));
}
// 把累加buffer 上的数据，输出到已申请的SPM地址 spm_output上
matmul_store(handle, spm_output, M, MatmulN32);
// 等待数据刷出完成
matmul_wait(handle);
// 通过threadIdx做地址偏移，将当前spe计算出的输出矩阵c的数据复制到Global上
memcpy(dataC + threadIdx * c_size_spe, spm_output, c_size_spe);
// 释放申请的SPM存储空间
free(spm_output);
free(spm_weight);
free(spm_input);
}

int main() {

```

```

int b, i, j, k;

char *dev_a = NULL;
char *dev_b = NULL;
char *dev_c = NULL;

int a_size = B * M * K;
int b_size = B * K * N;
int c_size = B * M * N;

unsigned int local_seed = time(NULL);

// 设置需要用到的设备号
sdaaSetDevice(1);
// 在Device端申请输入矩阵a 的Global空间
sdaaMalloc((void **)&dev_a, a_size * sizeof(int16_t));
// 在Device端申请输入矩阵b 的Global空间
sdaaMalloc((void **)&dev_b, b_size * sizeof(int16_t));
// 在Device端申请输出矩阵c 的Global空间
sdaaMalloc((void **)&dev_c, c_size * sizeof(int));
// 在Host 端申请输入矩阵a的内存空间
int16_t *host_a = (int16_t *)malloc(a_size * sizeof(int16_t));
// 在Host 端申请输入矩阵b的内存空间
int16_t *host_b = (int16_t *)malloc(b_size * sizeof(int16_t));
// 在Host 端申请需要从设备端复制数据到Host端输出矩阵c的内存空间
int *host_c = (int *)malloc(c_size * sizeof(int));
// 在Host 端申请Host 自己计算的输出矩阵c 的对比数据的内存空间
int *true_c = (int *)malloc(c_size * sizeof(int));
// 初始化矩阵a, b的输入数据
memset(host_a, 0, a_size * sizeof(int16_t));
memset(host_b, 0, b_size * sizeof(int16_t));
for (b = 0; b < B; b++) {
    for (i = 0; i < M; i++) {
        for (j = 0; j < K; j++) {
            host_a[b * M * K + i * K + j] = rand_r(&local_seed) % 100 - 50;
        }
    }
    for (i = 0; i < K; i++) {
        for (j = 0; j < N; j++) {
            host_b[b * K * N + i * N + j] = rand_r(&local_seed) % 5 - 2;
        }
    }
}

// 将初始化好的输入矩阵a数据从Host 端复制到Device 端
sdaaMemcpy(dev_a, host_a, a_size * sizeof(int16_t), sdaaMemcpyHostToDevice);
// 将初始化好的输入矩阵b数据从Host 端复制到Device 端
sdaaMemcpy(dev_b, host_b, b_size * sizeof(int16_t), sdaaMemcpyHostToDevice);

// Kernel函数kernel_matmul运行
kernel_matmul<<<1>>>(dev_a, dev_b, dev_c);
sdaaDeviceSynchronize();
// 将Device 端计算好的 输出矩阵c的数据复制到Host 端
sdaaMemcpy(host_c, dev_c, c_size * sizeof(int), sdaaMemcpyDeviceToHost);

memset(true_c, 0, c_size * sizeof(int));
// Host端计算 用于对比的输出矩阵c的数据
for (b = 0; b < B; b++) {
    for (i = 0; i < M; i++) {

```

```

        for (j = 0; j < N; j++) {
            for (k = 0; k < K; k++) {
                true_c[b * M * N + i * N + j] +=
                    ((int)host_a[b * M * K + i * K + k])
                    * ((int)host_b[b * K * N + k * N + j]);
            }
        }
    }
}

// 将 从Device 端复制过到Host 端的输出矩阵c 的数据和Host端自己计算的
// 输出矩阵c的数据做逐一对比, 如有不同报错
for (i = 0; i < c_size; i++) {
    if (host_c[i] != true_c[i]) {
        printf(
            "error: find mismatching data on %dth value, device: %d, host: %d\n",
            i, (int)host_c[i], (int>true_c[i]);
        break;
    }
    if (i == c_size - 1) {
        printf("matmul compare finished\n");
    }
}

// 释放Host端内存空间
free(true_c);
free(host_c);
free(host_b);
free(host_a);

// 释放Device端内存空间
sdaaFree(dev_c);
sdaaFree(dev_b);
sdaaFree(dev_a);

return 0;
}

```

16.2.5.3 编译运行

程序编译

1. 将完整程序中的代码复制到 `example.scpp` 文件中。
2. 执行以下编译命令, 将 `example.scpp` 生成可执行文件 `a.out`。

```
tecocc example.scpp
```

3. 执行以下命令, 运行程序。

```
./a.out
```

执行结果

输出以下内容, 表示算子可以正常运行, 且设备端和主机端的结算结果一致。

```
matmul compare finished
```

16.3 算子性能调优

16.3.1 概述

本文主要介绍如何开发高效的并行算法，或对已有算法进行性能优化，并结合太初AI加速卡提供了实际的性能优化案例。整体结构如下：

名称	说明
性能度量	介绍了两大性能指标及其计算办法：浮点性能FLOPS和IO带宽。
性能优化	- 介绍了计算相关优化办法及其示例：向量指令、指令流水线、矩阵乘法加速单元。 - 介绍了访存相关优化办法及其示例：双缓冲设计、广播机制。
优化实战	以求解下三角线性方程组为例，从串行实现入手，逐步融合多种优化手段，综合演示性能优化的思路与方法。

16.3.2 性能度量

性能指标

本文对性能的定义主要从工程的角度出发，主要关注程序在处理器上运行的直接性能表现，即程序的执行速度，包含两方面：

- 以浮点运算为主的算法，采用浮点性能FLOPS（每秒执行的浮点运算数）作为关键指标。
- 以数据读写为主的算法，采用IO带宽（通常以GB/s为单位）作为关键指标。

计算方法

为了实际测算具体算法的性能指标，最常用的方法是测量算法在计算给定问题上的耗时，然后根据计算过程所需的浮点运算数FLOPs或数据读写量来计算浮点性能与IO带宽。性能采样工具的使用，可以参考SDAA C编程指南的[性能采样](#)章节。

计算公式：

- 浮点性能（FLOPS）= 浮点运算次数 / 运算时间（秒）
- IO带宽（GB/s）= 传输的数据量（字节） / 传输时间（秒）

? 说明:

针对设备端的性能优化，暂不考虑主机端执行、主机-设备之间数据传输的开销，只需测量设备端Kernel函数或其中关键代码段的执行用时。

16.3.3 性能优化

16.3.3.1 概述

并行算法优化过程并不保证浮点性能与IO带宽两种指标同时接近硬件的峰值，这是由特定算法的计算密度决定的。针对特定算法进行优化，可以根据算法的计算密度，选择相应的优化方式。

计算密度

计算密度，也称为计算访存比，指的是一个算法在执行过程中，进行的浮点运算次数与读写数据量的比值，单位：FLOPs/Byte。计算密度越大，说明算法越集中于浮点计算，而非数据读写。

例如，对于一个输入张量形状为NHWC，使用形状为3x3的2D卷积核，输出张量形状为NPQK的卷积运算，假设所有数据类型均为float，其计算密度 I 为：

$$I = \frac{NPQK * (2 * 3 * 3 - 1) * C}{(NPQK + NHWC + C * 3 * 3 * K) * \text{sizeof}(\text{float})}$$

假设理想情况下，算法的浮点性能和IO带宽都能达到硬件的峰值，分别记为 BW_{comp} 和 BW_{IO} ，那么算法用于浮点运算和数据读写的时间之比即为：

$$I * \frac{BW_{IO}}{BW_{comp}}$$

算法类别

对于特定硬件，峰值IO带宽与浮点性能的比值是确定的。因此，算法用于浮点运算和数据读写的时间之比完全取决于计算密度。据此，可以将不同的算法进行分类：

- 运算时间远大于数据读写时间，称为计算密集型算法。
- 运算时间远小于数据读写时间，称为访存密集型算法。
- 运算与数据读写时间相当，称为均衡型算法。

其中：

- 对于密集型算法，其执行速度由对应行为的性能指标决定。因此对其展开优化，应针对相应的性能指标，使其尽量接近硬件的峰值性能。
- 对于均衡型算法，其整体性能同时受计算和访存两方面影响，两方面的性能指标优化都能带来明显的收益。同时，如果能利用硬件的特性，设法让运算与数据读写过程并发执行，可以进一步降低算法的执行用时，提升整体性能。

后续章节将分别介绍计算和访存两方面的优化手段，并给出相应的优化实践案例。基于这些优化方法，用户可以结合自己的需求，既可用于相应的密集型算法的优化，也可同时运用于均衡型算法的性能提升。

16.3.3.2 计算相关优化

并行程序设计的首要原则是尽量将任务拆分成可以并行处理的子任务，分配给处理器的多个计算单元并行处理。

每个计算核心在执行运算时，应当充分利用现代向量处理器的向量计算能力和流水线设计，尽可能实现数据并行和指令并行，才能发挥硬件的峰值性能。

此外，除了通用运算和控制功能外，某些处理器会提供针对特殊运算的硬件，在合适的场景下，能够获得明显优于通用运算的性能表现。

本章将对这三方面依次进行介绍：

- 向量指令使用
- 指令流水线优化
- 矩阵乘法加速单元

16.3.3.2.1 向量指令使用

优化策略

单指令多数据（SIMD）是一种典型的数据并行方法，指的是通过一条指令同时对多个不同的数据进行运算的设计思想。在现代向量处理器中，SIMD就是由各种向量指令支持的。

例如，普通的一条标量加法指令只能完成一个加数与一个被加数的求和运算。而一条向量加法指令则可以一次完成多个加数与等量的被加数的求和。虽然两种指令的延迟可能存在一定差异，但由于向量指令能一次完成更多的运算，其整体运算速度是明显高于标量运算的。

因此，在算法设计以及性能调优的过程中，应当尽可能采用SIMD的思路，广泛使用向量指令进行加速。

需注意，SIMD中并行的数据，需要相互不存在数据依赖。因此，使用SIMD时，需要设法解除原本的依赖关系。在加法示例中，对N个输入整数求和，顺序求和算法，意味着每次加法依赖于之前求和的结果，所有数据间产生了依赖关系，因此无法进行数据并行。但是利用加法的结合律，将N个输入均分为若干组，每一组可以无依赖地分别求和，最终对每组的结果求和得到输出。这样就实现了各组之间的数据并行，可以使用向量指令进行加速。

因此，在算法设计层面，需要开发者充分理解SIMD的思想，主动消除数据依赖关系，才能广泛地应用向量指令实现加速。在太初AI加速卡上支持的向量指令的功能和使用方法可以参考SDAA C编程指南中的[向量操作](#)章节。

优化示例

本节以一元定积分为例，介绍如何解除数据依赖，使用向量指令加速浮点数运算过程。

普通实现

给定一个一元函数 $f(x)$ 和一个闭区间 $[a, b]$ ，计算该区间上的定积分 $I = \int_a^b f(x)dx$ ，可以通过矩形公式（0阶近似）离散计算 I 的近似值，将积分区间均分为 N 等分：

$$I = \sum_{i=1}^N f(x_i) \Delta$$

其中：

$$\Delta = (b - a) / N$$

$$x_i = a + (i - 1) \Delta$$

当 N 足够大时，离散积分的结果可以作为定积分精确值的高度近似。因此，将积分区间拆分到各个计算核心上，每个核心负责一个子区间的积分，最终由一个核心（假设为SPE0）将各子区间的积分结果求和，即可输出最终的积分值。伪代码实现如下：

```
// Calculate the width of each subinterval
float delta = (b - a) / N;

// Determine the starting point of the subinterval for the current thread
float x_beg = a + delta * threadIdx * N / threadDim;

// Determine the ending point of the subinterval for the current thread
float x_end = a + delta * (1 + threadIdx) * N / threadDim;

// Compute the integral over the subinterval assigned to the current thread
float sub_integral = integrate(x_beg, x_end, N / threadDim);

// Synchronize all threads to ensure all sub_integrals are computed
sync_threads();

if (0 == threadIdx) {
    // If this is the first thread (threadIdx == 0),
    // gather sub_integral values from all other threads using RMA (Remote Memory Access)
    RMA get sub_integral from all other threads;

    // Sum up all the sub_integral values to get the total integral
    float sum = sum all sub_integral;

    // Write back the final result of the integration
    write back sum;
}
```

其中，单个子区间的积分是通过函数 `integrate()` 的矩形公式实现的。

以 $f(x) = x^2$ ， $[a, b] = [-3, 3]$ ， $N = 10^6$ 为输入参数，使用上述并行策略，在SPE主频为2.36GHz的条件下，使用SDAA C编程指南的**性能采样功能**计算Kernel函数中并行积分计算代码的执行用时（不包括RMA通讯用时），平均约为107us。

优化实现

基于上述并行版本，可以进一步优化，即在负责每个计算核心的子区间积分过程中，进行数据并行：将子区间内所有参与离散积分的点的积分过程拆分为16组，即：

$$I = \sum_{i=1}^{16} \sum_{j=1}^{N/16} f(x_i + 16(j-1)\Delta)\Delta$$

这16组积分过程是独立的，只需要最终将分别积分的结果累加即可。这样就构造了16组不存在数据依赖的计算过程，可以用太初AI加速卡的单精度浮点数向量指令来同时运算。因此，`integrate()` 函数的实现如下：

```
__device__ float integrate_v1(const float a, const float b, const int N) {
    // discrete interval
    const float dx = (b - a) / N;
    // compute integral, using 0-th order approximation
    float integral = 0.0f;
    floatv16 vx = a;
    floatv16 vdx = dx;
    {
        floatv16 offset =
            {0.0f, 1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f, 9.0f, 10.0f, 11.0f, 12.0f, 13.0f, 14.0f, 15.0f};
        vx += offset * vdx;
    }
    floatv16 vres = 0.0f;

    // loop and use SIMD
    int i;
    for (i = 0; i <= N - 16; i += 16) {
        vres += vx * vx;
        vx += vdx;
    }

    // loop the rest part
    float x = a + i * dx;
    while (i < N) {
        integral += x * x;
        x += dx;
        i++;
    }

    // reduce result
    float res[16];
    simd_store(vres, res);
    for (i = 0; i < 16; i++) {
        integral += res[i];
    }
    integral *= dx;
    // over
    return integral;
}
```

优化效果

采用上述向量指令优化后，在相同条件下测得的Kernel执行用时，降低到平均8us，也就是获得了约13倍的加速比。

16.3.3.2.2 指令流水线优化

优化策略

处理器的每个硬件核心通常有多个指令流水线，是实现多指令（MI）并行设计思想的基础。

所谓指令流水线，是将一条指令在处理器上执行的过程，分解为多个串行的阶段，而非一个不可拆分整体，指令（及其数据）随着时钟信号依次通过所有阶段。通过流水线化，处理器可以并行处理多个指令，仅轻微增加每条指令的延迟。

例如，处理器连续处理3条相同功能的指令：

- 图1是非流水线化的执行过程：每条指令必须等待前一条完全执行完成才能得到执行，总的延迟是每条指令延迟的3倍。



图1 非流水线化指令处理

- 图2是三阶段流水线化的执行过程，每条指令只需要等待前一条指令的第一阶段执行完毕就能开始执行，总的延迟明显低于非流水线的情况。

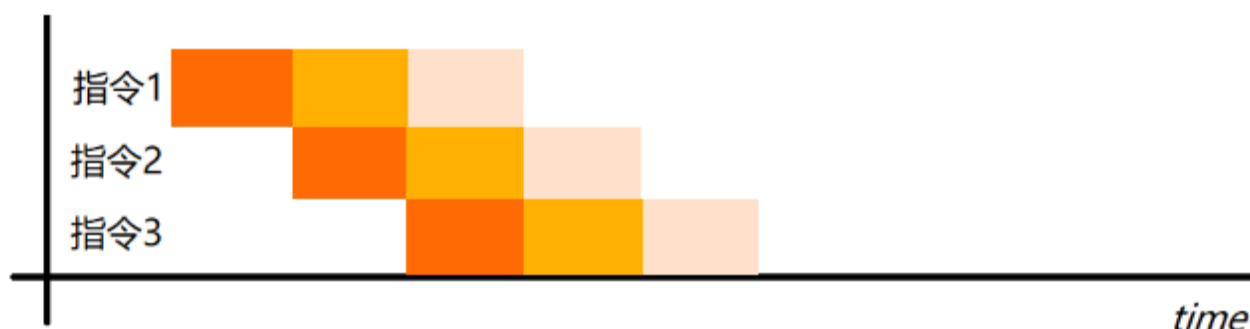


图2 流水线化指令处理

指令流水线通过增加指令执行的阶段划分，提升了处理器的指令吞吐量（单位时间内能执行的指令条数）。此外，现代处理器广泛采用多发射流水线技术：在流水线的每个阶段，可以同时处理不止一条指令，这同样可以大幅度提升处理器吞吐量。

因此，为了充分利用处理器流水线提供的指令吞吐量，开发者应当在同一时间提供足够数量的相同指令（在寄存器等资源足够的前提下），由处理器并行地执行这些指令，将指令流水线的各个阶段尽量占满。

优化示例

在上节使用向量指令优化后的算法中，计算子区间积分的循环每次只有一条浮点向量乘加指令在执行。而太初AI加速卡的每个计算核心都支持多级指令流水线，当多条无依赖的指令同时执行时，流水线才能被这些指令填满，达到最佳运算性能。因此，可以尝试进一步的优化：在循环内同时执行多条浮点向量乘加指令，也就是循环展开。示例选择将循环展开两次，即每轮循环内同时执行2条乘加指令，相应的Kernel代码如下所示：

```

__device__ float integrate_v2(const float a, const float b, const int N) {
    // discrete interval
    const float dx = (b - a) / N;
    // compute integral, using 0-th order approximation
    float integral = 0.0f;
    floatv16 vx[2], vres[2], vdx = dx;
    vres[0] = vres[1] = 0.0f;
    {
        vx[0] = a;
        vx[1] = a;
        floatv16 offset0 =
            {0.0f, 1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f, 9.0f, 10.0f, 11.0f, 12.0f, 13.0f, 14.0f, 15.0f};
        floatv16 offset1 = offset0 + (floatv16)16.0f;
        vx[0] += offset0 * vdx;
        vx[1] += offset1 * vdx;
    }

    // loop and use SIMD
    int i;
    for (i = 0; i <= N - 32; i += 32) {
        vres[0] += vx[0] * vx[0];
        vres[1] += vx[1] * vx[1];

        vx[0] += vdx;
        vx[1] += vdx;
    }

    // loop the rest part
    float x = a + i * dx;
    while (i < N) {
        integral += x * x;
        x += dx;
        i++;
    }

    // reduce result
    vres[0] += vres[1];
    float res[16];
    simd_store(vres[0], res);
    for (i = 0; i < 16; i++) {
        integral += res[i];
    }
    integral *= dx;
    // over
    return integral;
}

```

优化效果

在同样的场景下测试，循环展开后的积分过程，耗时可以降低约25%。

16.3.3.2.3 矩阵乘法加速单元

优化策略

不同处理器硬件由于面向的应用场景不同，往往会提供一些针对特定问题（例如特殊函数、矩阵乘法等）加速的计算硬件，被称为特殊计算单元（Special Function Unit，或SFU）。SFU虽然只能用于特定计算，但由于其加速基于硬件实现，而非处理器通用指令的软件实现，其计算速度远远高于通用运算过程。因此，用户可以尽可能增加SFU的利用，而非通用指令。

在太初AI加速卡中，最重要的SFU是矩阵乘法单元，专门用于进行矩阵的乘加运算（Matrix Multiply-Accumulate，或MMA）。通过矩阵乘法单元实现的MMA计算，比依靠处理器的通用乘加指令要快得多。在需要大量计算MMA的算法中，例如GEMM、卷积等算子，合理使用矩阵乘法单元，可以获得很高的浮点运算性能。具体的使用方法可以参考SDAA C编程指南的[矩阵乘](#)章节。

优化示例

本节示例将展示如何使用矩阵乘法单元，在SPM上计算两个矩阵的乘法。

示例中的输入输出数据类型均为half类型，左矩阵形为128×128，右矩阵形为128×32。由于矩阵乘接口每次进行的是一个128×32×32的MMA，因此，需要预先将左矩阵拆分为若干128×32的分块存储，方便矩阵乘接口的加载。相应的Kernel代码如下所示：

```
__global__ void mma_using_ace(void) {
    half* left_mat = (half*)malloc(sizeof(half) * 128 * (32 * 4));
    half* right_mat = (half*)malloc(sizeof(half) * 32 * (32 * 4));
    half* output = (half*)malloc(sizeof(half) * 128 * 32);

    MatmulHandle handle;
    matmul_init(handle, MatmulHalfToHalf);
    matmul_set_flushing_output(handle, false);

    // the 1st three rounds
    for (int i = 0; i < 3; i++) {
        matmul_load_weight(handle, right_mat + i * 1024, MatmulK32, MatmulN32);
        matmul_compute(handle, left_mat + i * 128 * 32, 128, MatmulK32);
    }
    // the last round
    matmul_set_flushing_output(handle, true);
    matmul_load_weight(handle, right_mat + 3 * 1024, MatmulK32, MatmulN32);
    matmul_compute(handle, left_mat + 3 * 128 * 32, 128, MatmulK32);

    matmul_store(handle, output, 128, MatmulN32);

    free(left_mat);
    free(right_mat);
    free(output);
    // over
    return;
}
```

优化效果

在SPE主频为2.36GHz的条件下，测得其浮点性能约为29TFLOPS，而通用运算指令的浮点性能峰值约为2.5TFLOPS。因此，在条件允许的情况下，应当尽量充分利用矩阵乘法单元进行矩阵乘计算。

需要指出的是，上述实验远没有达到太初AI加速卡乘加运算的浮点性能峰值，在合适的应用场景下，充分利用矩阵乘法单元可以获得更高的浮点计算性能。

16.3.3.3 访存相关优化

太初AI加速卡设备端的存储器模型可以分为3层：Global存储、SPM存储和寄存器。

- Global存储是所有片上所有核心共享的大容量存储器。
- SPM存储是每个计算核心的本地高速缓存，属于计算核心的私有资源。
- 寄存器是核心执行运算和控制时，使用的特殊存储资源。

从访问速度而言，寄存器>SPM存储>Global存储。从存储容量而言，Global存储>SPM存储>寄存器。

而一个算法运行过程的数据流动通常包括如下过程：

1. 从主机端将输入数据传输到Global存储。
2. 计算核心从Global存储读取数据到自己的SPM中。
3. 计算核心使用SPM中的数据进行计算，这个过程会根据指令执行的需要，使用各种寄存器，各种中间结果和阶段性输出都暂存在SPM中。
4. 计算完成后，SPM上的输出结果写回Global存储。
5. 主机端从Global存储中取回输出结果。

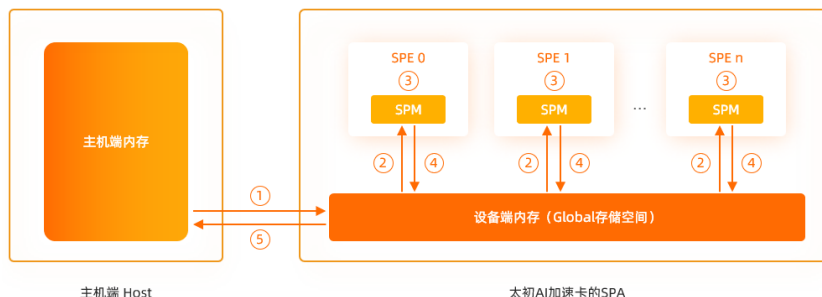


图1 数据流向示意图

其中，

- 阶段1和5，主机端与Global存储之间的数据交换可以参考SDAA C编程指南中的[数据传输](#)章节进行调用和优化。
- 阶段3，计算过程中访问SPM以及使用寄存器的过程，通常不需要特殊的控制。
- 阶段2和4，由于Global存储访问速度最慢，容量最大，输入输出阶段都需要对其进行大量的数据读写，因此是需要集中优化的阶段。

Global存储的具有单次访问延迟很高，但吞吐量（即单次访问的数据量）很大的特点。因此，它不适合像SPM或寄存器一样，以高频多次少量的方式进行读写。Global存储最佳的读写模式是一次读写大量数据，控制访问频次。因此，可以考虑尽可能实现：

- 合并读取：将多次计算所需的输入数据作为一个整体，通过一次对Global存储的访问操作，读入SPM。

- 合并写出：使用SPM上的数据进行计算，输出结果暂存在SPM上，将SPM上的结果整体写回Global存储。

16.3.3.3.1 双缓冲优化

优化策略

主机与设备间的数据传输可以与设备上Kernel的执行并行，互不影响。这意味着：主机端可以通过异步的数据传输接口发起传输，然后立即发起Kernel的执行操作，实现访存和计算的并行，进一步缩短程序耗时。同样的，使用Global存储与SPM间的**非阻塞型DMA**接口，可以实现访存和计算的并行。

这意味着开发者需要解除并行数据传输与计算过程的数据依赖，也就是计算过程不能依赖于同一时刻正在传输的数据。为了实现这一点，开发者需要在SPM上设置至少2个数据缓冲区。

- 计算缓冲区：存储计算过程依赖的数据。
- 传输缓冲区：存储异步传输的数据。

当计算和访存都完成后，再交换两个缓冲区的职能，就能始终确保计算过程与数据传输过程无依赖。

正是因为这种同时需要2个数据缓冲区的特点，此优化方法被称为双缓冲优化。

优化示例

本节以ID卷积算法为例，演示如何使用双缓冲技术优化Global存储的数据访问。即输入一个长度为N的一维信号（单通道），使用一个长度为3的卷积核对其进行滤波，输出长度仍为N的滤波结果。

普通实现

算法的基本设计思路：

1. 首先，需要根据SPM大小，确定每次最多能计算输出信号的长度dN。据此将计算过程分解为若干部分，由不同的SPE并行。
2. 在每个SPE上，给定需要计算的输出信号范围后，先从Global存储中将计算过程所依赖的输入信号读入SPM，然后进行计算，并将结果从SPM的输出缓冲区写回Global存储的对应位置。

以伪代码描述各个SPE的并行计算过程如下：

```
for (i = threadIdx; i < N / dN; i += threadDim) {
    Determine the output signal range that needs to be computed: y[i*dN, (i+1)*dN]
    Determine the input signal range needed for computation: x[i*dN-1, (i+1)*dN]
    Read the input signal range x[i*dN-1, (i+1)*dN] from Global storage into the input buffer spm_x in SPM
    Perform the convolution filtering on spm_x and store the result in the output buffer spm_y in SPM
    Write the spm_y output buffer back to Global storage
}
```

在 `stride = 1`，`padding = 1`，预留位置填充0的情况下，一维卷积（3点平滑）的基础版本Kernel函数如下：


```

__global__ void test_3pt_smooth_baseline(float* x, const int N, float* y) {
    constexpr int max_tile_size = 8192;

    // parallelize
    const int tile_num = (N + max_tile_size - 1) / max_tile_size;
    const int last_tile_size = N - (tile_num - 1) * max_tile_size;

    // allocate SPM
    float* tile_x = (float*)malloc(sizeof(float) * (2 + max_tile_size));
    float* tile_y = (float*)malloc(sizeof(float) * max_tile_size);

    // traverse all tiles
    for (int i = threadIdx; i < tile_num; i += threadDim) {
        const int tile_size = (i < tile_num - 1) ? max_tile_size : last_tile_size;

        // read tile of x
        memcpy(tile_x, x + i * tile_size - 1, sizeof(float) * (tile_size + 1));

        // perform 3-point smoothing
        if (i == 0) {
            tile_x[0] = 0.0f;
        }
        if (i == tile_num - 1) {
            tile_x[tile_size] = 0.0f;
        }
        perform_3pt_smooth(tile_x, tile_size, tile_y);

        // output tile of y
        memcpy(y + i * tile_size, tile_y, sizeof(float) * tile_size);
    }

    // free SPM
    free(tile_x);
    free(tile_y);

    // over
    return;
}

```

优化实现

对上述基础版本进行双缓冲优化，伪代码如下：

```

Determine the first segment of the output signal range that the current SPE needs to compute: y[threadIdx*dN, (threadIdx+1)*dN]
Determine the corresponding input signal range: x[threadIdx*dN-1, (threadIdx+1)*dN]
Set the buffer flag for computation to flag = 0
Use an asynchronous interface to read x[threadIdx*dN-1, (threadIdx+1)*dN] from Global storage into one of the double buffers spm_x[flag]
for (i = threadIdx; i < N / dN; i++) {
    Wait for the previously initiated data transfer to complete
    Determine the next segment of the output signal range to compute: y[(i+threadDim)*dN, (i+threadDim+1)*dN]
    Determine the corresponding input signal range: x[(i+threadDim)*dN-1, (i+threadDim+1)*dN]
    Use an asynchronous API to read the next segment of the input signal into the other buffer spm_x[fla

```

```

g ^ 1]
    Perform the convolution filtering on spm_x[flag], and write the result into the output buffer spm_y
in SPM
    Write spm_y back to Global storage
    Swap the buffer flag, i.e., flag = flag ^ 1
}

```

双缓冲版本的Kernel函数如下:

```

__global__ void test_3pt_smooth_v1(float* x, const int N, float* y) {
    constexpr int max_tile_size = 8192;

    // parallelize
    const int tile_num = (N + max_tile_size - 1) / max_tile_size;
    const int last_tile_size = N - (tile_num - 1) * max_tile_size;

    // allocate SPM
    int flag = 0;
    float* tile_x[2];
    tile_x[0] = (float*)malloc(sizeof(float) * (2 + max_tile_size));
    tile_x[1] = (float*)malloc(sizeof(float) * (2 + max_tile_size));

    float* tile_y = (float*)malloc(sizeof(float) * max_tile_size);

    if (threadIdx < tile_num) {
        const int tile_size = (threadIdx < tile_num - 1) ? max_tile_size : last_tile_size;
        // read tile of x
        memcpy_async(tile_x[flag], x + threadIdx * tile_size - 1,
                     sizeof(float) * (tile_size + 1), MemcpyGlobalToSpm);
    }
    // traverse all tiles
    for (int i = threadIdx; i < tile_num; i += blockDim) {
        const int tile_size = (i < tile_num - 1) ? max_tile_size : last_tile_size;

        // wait async memcpy to finish
        memcpy_wait();
        // read next tile of x
        if (i + blockDim < tile_num) {
            const int next_i = i + blockDim;
            const int next_tile_size = (next_i < tile_num - 1) ? max_tile_size : last_tile_size;
            memcpy_async(tile_x[flag ^ 1], x + next_i * next_tile_size - 1,
                         sizeof(float) * (next_tile_size + 1), MemcpyGlobalToSpm);
        }

        // perform 3-point smoothing
        if (i == 0) {
            tile_x[flag][0] = 0.0f;
        }
        if (i == tile_num - 1) {
            tile_x[flag][tile_size] = 0.0f;
        }
        perform_3pt_smooth(tile_x[flag], tile_size, tile_y);

        // output tile of y
        memcpy(y + i * tile_size, tile_y, sizeof(float) * tile_size);

        // update
    }
}

```

```
        flag ^= 1;
    }

    // free SPM
    free(tile_x[0]);
    free(tile_x[1]);
    free(tile_y);

    // over
    return;
}
```

优化效果

在SPE主频为2.36GHz，输入输出数据皆为float类型，输入信号长度 `N = 512k` 的场景下，测试得到双缓冲版本的耗时只有基础版本的78%左右。双缓冲设计通过将输入信号的读取过程与卷积计算过程并行，从而实现加速。

16.3.3.3.2 广播优化

优化策略

针对多个计算核心，需要同时读取Global存储中同一块数据的场景，可以利用广播（Broadcast）进行优化。

例如，在稠密矩阵与向量乘（GEMV）的计算中，可以将输入矩阵的不同行分配到不同计算核心上，然后分别与输入向量相乘，最终再将结果分别输出。其中，不同计算核心需要从Global存储中读取同一个输入向量。

此时，如果各个计算核心分别读取同样的Global存储空间，会产生明显的浪费。但广播机制，允许一个计算核心将Global存储上的数据读入自己的SPM存储器，然后通过广播同时传输给多个不同计算核心的SPM存储器。

合理利用广播机制，可以减少对Global存储器的访问次数，提升数据读取的效率。具体使用方法，可以参考SDAA C编程指南的[Broadcast数据搬运](#)章节。

优化示例

本节以一个稠密矩阵与向量乘积的算法为例，演示广播机制对Global存储上重复数据访问的性能优化。

普通实现

假设输入矩阵的行数等于SPA中所有SPE的总数，输入矩阵A的各行分给不同的SPE，每个SPE每次只需要负责A的某一行与输入向量x的内积计算。其Kernel函数如下：

```
__global__ void test_gemv_32xN_baseline(float* A, float* x, const int N, float* y) {
    // max size of one tile
    constexpr int max_tile_size = 16384;
    // allocate SPM
    float* tile_A = (float*)malloc(sizeof(float) * max_tile_size);
    float* tile_x = (float*)malloc(sizeof(float) * max_tile_size);
    // parallelize
    float* pA = A + threadIdx * N;
    float* px = x;
```

```

// iterate
int rest = N;
float result = 0.0f;
while (rest > 0) {
    const int tile_size = rest > max_tile_size ? max_tile_size : rest;
    // load tile of A into SPM
    memcpy(tile_A, pA, sizeof(float) * tile_size);
    // load tile of x into SPM
    memcpy(tile_x, px, sizeof(float) * tile_size);
    // compute dot product
    result += dev_dot_product(tile_A, tile_x, tile_size);
    // update
    rest -= tile_size;
    pA += tile_size;
    px += tile_size;
}

// output
y[threadIdx] = result;
// free SPM
free(tile_A);
free(tile_x);
// over
return;
}

```

其中dev_dot_product是负责向量内积计算的device函数（出于简便考虑，未处理边界情况）：

```

__device__ float dev_dot_product(const float* x, const float* y, const int N) {
    const float *px = x, *py = y;
    floatv16 vres[4];
    floatv16 vx[4], vy[4];
    vres[0] = vres[1] = vres[2] = vres[3] = 0;
    for (int i = 0; i < N; i += 64) {
        simd_load(vx[0], px);
        simd_load(vy[0], py);
        vres[0] += vx[0] * vy[0];

        simd_load(vx[1], px + 16);
        simd_load(vy[1], py + 16);
        vres[1] += vx[1] * vy[1];

        simd_load(vx[2], px + 32);
        simd_load(vy[2], py + 32);
        vres[2] += vx[2] * vy[2];

        simd_load(vx[3], px + 48);
        simd_load(vy[3], py + 48);
        vres[3] += vx[3] * vy[3];

        px += 64;
        py += 64;
    }

    vres[0] += vres[2];
    vres[1] += vres[3];
    vres[0] += vres[1];
}

```

```

float res[16];
simd_store(vres[0], res);
// stride = 8
res[0] += res[0 + 8];
res[1] += res[1 + 8];
res[2] += res[2 + 8];
res[3] += res[3 + 8];
res[4] += res[4 + 8];
res[5] += res[5 + 8];
res[6] += res[6 + 8];
res[7] += res[7 + 8];
// stride = 4
res[0] += res[0 + 4];
res[1] += res[1 + 4];
res[2] += res[2 + 4];
res[3] += res[3 + 4];
// stride = 2
res[0] += res[0 + 2];
res[1] += res[1 + 2];
// stride = 1
res[0] += res[0 + 1];

// over
return res[0];
}

```

优化实现

由于输入向量x是所有SPE计算过程中都要用到的，因此，可以采用广播机制避免对Global存储的重复访问，改善访存带宽。具体做法是先由一个SPE（假定为SPE0）从Global存储中读取x，然后通过一次全阵列广播，将x分发给所有SPE。相应的Kernel函数代码如下：

```

__global__ void test_gemv_32xN_v1(float* A, float* x, const int N, float* y) {
    // max size of one tile
    constexpr int max_tile_size = 16384;
    // allocate SPM
    float* tile_A = (float*)malloc(sizeof(float) * max_tile_size);
    float* tile_x = (float*)malloc(sizeof(float) * max_tile_size);
    // parallelize
    float* pA = A + threadIdx * N;
    float* px = x;

    BroadcastHandle handle;
    ThreadGroup thread_group(0xFFFFFFFF);
    thread_group_exclude(thread_group, 0);
    broadcast_set_thread_group(handle, &thread_group);
    // iterate
    int rest = N;
    float result = 0.0f;
    while (rest > 0) {
        const int tile_size = rest > max_tile_size ? max_tile_size : rest;

        if (0 == threadIdx) {
            memcpy(tile_x, px, sizeof(float) * tile_size);
        }
    }
}

```

```
    sync_threads();
    broadcast_async(tile_x, tile_x, sizeof(float) * tile_size, 0, BroadcastSpmToSpm, handle);

    // load tile of A into SPM
    memcpy(tile_A, pA, sizeof(float) * tile_size);

    broadcast_wait(handle);
    // compute dot product
    result += dev_dot_product(tile_A, tile_x, tile_size);
    // update
    rest -= tile_size;
    pA += tile_size;
    px += tile_size;
}

// output
y[threadIdx] = result;
// free SPM
free(tile_A);
free(tile_x);
// over
return;
}
```

优化效果

对于输入矩阵为 `32×32768` 的场景，在SPE主频2.36GHz的情况下，测试得到使用广播优化后的耗时可以降低到优化前82%左右。

16.3.4 优化实战

16.3.4.1 优化思路

本章将以求解下三角线性方程组为例，演示如何综合运用此前各章节介绍的优化技术。

求解下三角线性方程组

求解线性方程组的一个常用方法是LU分解：将系数矩阵分解为下三角和上三角矩阵的乘积，然后依次求解上下三角线性方程组。

由于三角化后的线性方程组可以很容易地逐行消元求解，因此，这种方法的效率很高，适用于大多数线性方程组的求解。

算法设计阶段，通常可以先从串行版本开始，构建算法的整体逻辑，然后在此基础上进行并行改造，最后做进一步的优化。

优化前：串行实现

假设待求解的下三角线性方程组记为： $Lx = b$

其中， L 是 N 阶的下三角矩阵， x 和 b 均为 N 维列向量。

按照三角方程组的逐行消元法，串行版本伪代码如下：

```
for (i = 0; i < N; i++) {
    // Initialize the sum to zero for each row i
    sum = 0;

    // Compute the dot product of the already solved parts with the corresponding coefficients in row i
    for (j = 0; j < i; j++) {
        sum += L[i][j] * x[j];
    }

    // Solve for x[i] using the computed sum and the known value b[i], then normalize by the diagonal element L[i][i]
    x[i] = (b[i] - sum) / L[i][i];
}
```

串行版本需要逐行消元求解，各行相互依赖，无法并行，因此很难获得好的性能。

基础优化：并行实现

串行版本实际上就是利用已求解的部分，对尚未求解的方程进行消元。因此，每当求解出 x 的一个分量，就可以立即对所有剩下的方程消元，这样就可以将消元的过程并行化，并行版本伪代码如下：

```
for (i = 0; i < N; i++) {
    // Solve for x[i] directly using the diagonal element L[i][i]
    x[i] = b[i] / L[i][i];

    // Update the remaining elements of b by subtracting the effect of the newly solved x[i]
    for (j = i + 1; j < N; j++) { // parallelizable loop
        b[j] -= L[j][i] * x[i];
    }
}
```

其中，第二个循环是可以并行化的。后续示例将在太初AI加速卡上，以此为基础，展开逐步优化。

16.3.4.2 优化程序

本节将综合应用计算和访存方面的优化手段，对示例进行逐步分析与优化。

优化点1：初步并行化

加速卡上各个SPE可以并行执行消元循环，同时由一个固定的SPE（假设为SPE0）负责求解消元后的第一个待求方程的解，伪代码如下：

```
i_beg = 0;
while (i_beg < N) {
    if (0 == threadIdx) {
        // SPE0 (threadIdx == 0) solves the first equation in the current segment
        b[i_beg] /= L[i_beg][i_beg];
    }
    // Synchronize threads
}
```

```

sync_threads();

// Store the latest solved value
latest_x = b[i_beg];

// Parallel loop: each thread performs elimination on a subset of equations
for (i = 1 + i_beg + threadIdx; i < N; i += threadDim) {
    b[i] -= L[i][i_beg] * latest_x;
}
i_beg++;
}

```

优化点2：改善吞吐量

初步并行化版本在执行并行消元时，每个线程最多只能对一个待求解方程的一项消元。在太初AI加速卡上，至少有两点不足：

- 一项消元只需要一次标量乘加运算，无法充分利用SPE的向量乘加指令，造成浮点性能的浪费。
- 对一个待求解方程的一项消元只需要访问该项系数，因此，对Global存储的访问只能以少量多次的方式进行，无法发挥Global存储的访存带宽。

针对以上两点问题，可以考虑增加并行消元过程中每个线程的吞吐量：每个线程最好能同时对多个待求解方程的连续若干项进行消元，进行进一步优化。即每一轮需要多求解几个方程，得到若干个连续的解，然后再多线程并行从剩余的待求解方程中，同时消去这些解对应的系数。

假设每次求解连续16个方程的解，每个线程并行按照每次16个的速度从剩余待求解方程中消去这16个解对应的系数。伪代码如下（假设N是16的整数倍）：

```

i_beg = 0;
while (i_beg < N) {
    if (0 == threadIdx) {
        // SPE0 loads a 16x16 block of L corresponding to the next 16 equations
        load lt[0:15][0:15] = L[i_beg:i_beg+15][i_beg:i_beg+15];

        // SPE0 solves the 16 equations in the block using matrix inversion
        solve x[0:15] = inv(lt) * b[i_beg:i_beg+15];
    }

    // Update the global array b with the newly solved values
    b[i_beg:i_beg+15] = x[0:15];

    // Synchronize threads
    sync_threads();

    // Load the solved values into all threads
    load x[0:15] = b[i_beg:i_beg+15];

    // Parallel loop: each thread performs elimination on a subset of remaining equations
    for (i = i_beg + 16 * (1 + threadIdx); i < N; i += 16 * threadDim) {
        // Load a 16x16 block of L corresponding to the current set of equations
        load lt[0:15][0:15] = L[i:i+15][i_beg:i_beg+15];

        // Load the corresponding portion of b for the current set of equations

```



```

    load lb[0:15] = b[i:i+15];

    // Perform vectorized elimination on the loaded block
    for (j = 0; j < 16; j++) {
        lb[i:i+15] -= lt[0:15][j] * x[j]; // vectorizable operation
    }

    // Store the updated values back to the global array b
    b[i:i+15] = lb[0:15];
}
i_beg += 16;
}

```

当前版本中，每个线程都需要对连续16个待求解方程的连续16项进行消元，解决了上述两个问题：

- 由于系数矩阵采用了列优先存储，可以直接向量加载同一个解的系数，然后进行一次向量乘加运算将这些系数消去。
- 由于每次需要消去16×16个系数，每个线程都可以一次性从Global存储中加载这256个系数，可以明显改善Global存储的访存带宽。

16.3.4.3 完整程序

优化点1：初步并行化

初步并行化的Kernel函数实现如下：

```

__global__ void test_solve_LT_v1(float* LT, float* b, const int N) {
    // assert: shape of LT = N x N, N is multiple of 16
    int i_beg = 0;

    while (i_beg < N) {
        // current equation to solve: LT(i_beg:N-1, i_beg:N-1)*x(i_beg:N-1) = b(i_beg:N-1);
        if (0 == threadIdx) {
            b[i_beg] /= LT[get_index(i_beg, i_beg, N)];
        }
        sync_threads();

        // update
        float latest_x = b[i_beg];

        // elimination process (parallelizable)
        for (int i = 1 + i_beg + threadIdx; i < N; i += threadDim) {
            b[i] -= LT[get_index(i, i_beg, N)] * latest_x;
        }

        i_beg++;
    }
    // over
    return;
}

```

其中，为了将矩阵元素的2元脚标映射为列主元存储的1维索引，设计了一个映射函数：

```
__device__ inline int get_index(const int i, const int j, const int N) {
    return (((2 * N - i + 1) * i) >> 1) + j - i;
}
```

优化点2：改善吞吐量

实现Kernel函数的过程中，可以在上节伪代码的基础上，新增了两项优化：

1. 在并行消元的过程中，由于所有线程都要使用新求得的16个解。因此，可以通过一次广播将其分享给所有线程，避免了对Global存储的重复访问。
2. 当N较大时，在一轮之内，每个线程必须进行多次消元（每次16个方程）。此时，可以采用双缓冲思想，在对当前16个方程进行消元的同时，预取接下来16个方程的系数。

其Kernel函数的代码实现如下：

```
__global__ void test_solve_LT_v2(float* LT, float* b, const int N) {
    // assert: shape of LT = N x N, N is multiple of 16
    // allocate SPM
    float* local_lt = (float*)malloc(sizeof(float) * ((1+16)*16/2));

    int flag = 0;
    float* local_eq_coeff_mem = (float*)malloc(2 * (sizeof(float) * 16 * 16));
    float* local_eq_coeff[2];
    local_eq_coeff[0] = local_eq_coeff_mem;
    local_eq_coeff[1] = local_eq_coeff_mem + 16 * 16;

    float* local_b_x_mem = (float*)malloc(3 * sizeof(float) * 16);
    float* local_b[2];
    local_b[0] = local_b_x_mem;
    local_b[1] = local_b_x_mem + 16;
    float* local_x = local_b_x_mem + 32;

    // broadcast handle
    BroadcastHandle handle;

    // main loop
    int i_beg = 0;
    while (i_beg < N) {
        // current equation to solve: LT(i_beg:N-1, i_beg:N-1)*x(i_beg:N-1) = b(i_beg:N-1);
        if (0 == threadIdx) {
            // load LT(i_beg:i_beg+15, i_beg:i_beg+15) into local_lt
            // load b(i_beg:i_beg+15) into local_b
            {
                auto dst = local_lt;
                for (int j = 0; j < 16; j++) {
                    auto src = LT + get_index(i_beg + j, i_beg + j, N);
                    memcpy_async(dst, src, sizeof(float) * (16 - j), MemcpyGlobalToSpm);
                    // update
                    dst += (16 - j);
                }
            }

            memcpy(local_b[0], b + i_beg, sizeof(float) * 16);
```

```

    }
    // wait for load to be finish
    memcpy_wait();

    // solve x(i_beg:i_beg+15) = inv(local_lt)*local_b
    solve_16x16_LT(local_lt, local_b[0]);

    // write back to update b(i_beg:i_beg+15)
    memcpy_async(b + i_beg, local_b[0], sizeof(float) * 16, MemcpySpmToGlobal);
}
sync_threads();

// RMA broadcast local_b (from thread-0) to local_x (of all threads)
broadcast_async(local_x, local_b[0], sizeof(float) * 16, 0, BroadcastSpmToSpm, handle);
broadcast_wait(handle);
// wait for write back to finish
memcpy_wait();
sync_threads();

// eliminate local_x[] from rest equations
// load coefficients of the 1st 16 equations
// load rhs (b) of the corresponding equations
if (i_beg + 16 * (1 + threadIdx) < N) {
    const int i = i_beg + 16 * (1 + threadIdx);
    auto dst = local_eq_coeff[flag];
    for (int j = 0; j < 16; j++) {
        auto src = LT + get_index(i, i_beg + j, N);
        memcpy_async(dst, src, sizeof(float) * 16, MemcpyGlobalToSpm);
        // update
        dst += 16;
    }

    memcpy_async(local_b[flag], b + i, sizeof(float) * 16, MemcpyGlobalToSpm);
}

// loop to cancel coefficients (parallelizable)
for (int i = i_beg + 16 * (1 + threadIdx); i < N; i += 16 * threadDim) {
    // wait for previous memcpy to finish
    memcpy_wait();

    // load coefficients of the next 16 equations
    // load rhs (b) of the corresponding equations
    if (i + 16 * threadDim < N) {
        const int i_next = i + 16 * threadDim;
        auto dst = local_eq_coeff[flag ^ 1];
        for (int j = 0; j < 16; j++) {
            auto src = LT + get_index(i_next, i_beg + j, N);
            memcpy_async(dst, src, sizeof(float) * 16, MemcpyGlobalToSpm);
            // update
            dst += 16;
        }

        memcpy_async(local_b[flag ^ 1], b + i_next, sizeof(float) * 16, MemcpyGlobalToSpm);
    }

    // eliminate local_x[] from equations LT(i:i+15, i_beg:N-1)*x = b
    eliminate_16x16(local_eq_coeff[flag], local_x, local_b[flag]);

    // write back to update b(i:i+15)

```

```

        memcpy(b + i, local_b[flag], sizeof(float) * 16);

        // switch
        flag ^= 1;
    }
    // update
    i_beg += 16;
}

// release
free(local_lt);
free(local_eq_coeff_mem);
free(local_b_x_mem);

// over
return;
}

```

其中，求解当前下三角方程组前16个解的计算过程被封装为device函数实现：

```

__device__ void solve_16x16_LT(float* lt, float* b) {
    auto diag_ptr = lt;
    for (int i_beg = 0; i_beg < 32; i_beg++) {
        // solve
        float x = -b[i_beg] / diag_ptr[0];
        b[i_beg] = -x;
        // eliminate
        diag_ptr++;
        for (int j = i_beg + 1; j < 32; j++) {
            b[j] += diag_ptr[0] * x;
            diag_ptr++;
        }
    }
    // over
    return;
}

```

同样，负责对连续16个待求解方程进行消元的计算过程也被封装为一个独立的device函数：

```

__device__ void eliminate_16x16(float* lt_blk, float* x, float* b) {
    floatv16 vec_b[2];
    vec_b[0] = vec_b[1] = 0.0f;

    for (int j = 0; j < 16; j += 2) {
        // expand to vector
        floatv16 vec_x[2];
        vec_x[0] = -x[j];
        vec_x[1] = -x[j + 1];

        // load vector
        floatv16 vec_lt[2];
        simd_load(vec_lt[0], lt_blk + j * 16);
        simd_load(vec_lt[1], lt_blk + (j + 1) * 16);

        // FMA
        vec_b[0] += vec_lt[0] * vec_x[0];
    }
}

```

```
    vec_b[1] += vec_lt[1] * vec_x[1];
}

floatv16 vres;
simd_load(vres, b);
vec_b[0] += vec_b[1];
vres += vec_b[0];
simd_store(vres, b);

// over
return;
}
```

这里为了充分利用SPE的浮点算力，使用了向量指令加速消元过程。同时，分配了两组向量寄存器，构成两组数据无依赖的计算流水线，手动进行循环展开。

优化效果

在SPE主频为2.36GHz的硬件上测试，对一个2048阶的下三角方程组进行求解时，最终优化版本相较于初步优化版本的加速比约为15.6。

16.4 算子调优实战

16.4.1 GEMM基础实现

16.4.1.1 实现原理

本文以GEMM（General Matrix Multiply，通用矩阵乘法）为例，介绍如何使用不同的优化方法，如：SIMD，矩阵乘法单元，广播，双缓冲等方法提升GEMM算子性能。

本章主要介绍未使用优化方法的GEMM算子的基础实现（仅用单线程进行计算），效率低且耗时长，性能不佳。

计算原理

GEMM用于执行两个矩阵的乘法，并将结果累加到第三个矩阵上。

计算公式

$$C = \alpha \times op(A) \times op(B) + \beta \times C$$

其中：

- α 和 β 是标量， A 、 B 、 C 是矩阵， $op()$ 是可选的转置操作，本示例中：矩阵A和矩阵B均未转置， α 与 β 取值分别为1和0。
- 矩阵A的维度是M×K、矩阵B的维度是K×N、矩阵C的维度是M×N。

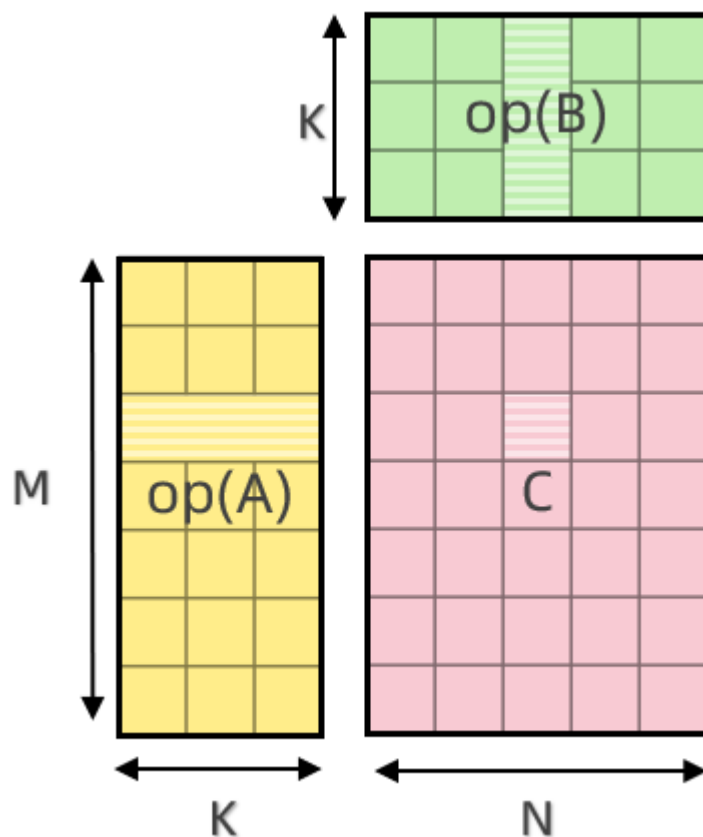


图1 GEMM矩阵乘法

设备端计算流程

1. 读取输入数据：将数据从Global存储空间读取到SPM存储空间。
2. 矩阵计算：在SPM存储空间，进行矩阵乘计算。
3. 传输计算结果：将矩阵乘的计算结果，从SPM存储空间传输到Global存储空间。

性能优化目标

设备端计算流程的总用时最少，即：读取输入数据、矩阵乘计算和传输计算结果的总用时最少。

基础实现

基础实现是通过三层嵌套循环，依次在M、N和K维度遍历矩阵 A 和矩阵 B 的元素，按照矩阵乘法的规则计算结果，从而得到矩阵C。

以下是GEMM基础实现的代码实现：

1. 参数 `GEMMArgs` 中存放矩阵 `A` 、 `B` 、 `C` 的维度信息和存储步长信息。
 - a. 维度信息：描述矩阵的大小，即矩阵的行数和矩阵的列数；
 - b. 存储步长信息：描述使用二维数组在内存中存储矩阵时，同一列中相邻元素之间的偏移量；
2. 将矩阵 `A` 、 `B` 、 `C` 的指针类型转换为 `half` 类型。

3. 使用三层嵌套的 `for` 循环，执行矩阵乘法操作：

- 最外层的 `for` 循环，遍历矩阵 `A` 的行，索引为 `idx`。
- 中间层的 `for` 循环，遍历矩阵 `B` 的列，索引为 `idy`。
- 最内层的 `for` 循环，遍历矩阵 `A` 的列和矩阵 `B` 的行，索引为 `idz`。并将矩阵 `A` 的第 `idx` 行和矩阵 `B` 的第 `idy` 列对应元素相乘，将结果累加到 `temp` 中。

4. `temp` 的结果，存储到矩阵 `C` 的 `idx` 行、`idy` 列位置。

示例代码如下：

```
__device__ void tecoKernelGemmFT16SingleThreadImpl(GEMMArgs pGemm)
{
    // 矩阵的维度
    int M, N, K;
    // 矩阵的存储步长
    int lda, ldb, ldc;
    // 指向矩阵 A、B、C 的指针
    const half *pA, *pB;
    half *pC;
    // 存储中间计算结果
    float temp;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    M = pGemm.m;
    N = pGemm.n;
    K = pGemm.k;
    // 获取矩阵 A、B、C 的存储步长
    lda = pGemm.lda;
    ldb = pGemm.ldb;
    ldc = pGemm.ldc;
    // 将矩阵 A、B、C 的指针转换为 half 类型
    pA = (const half *)pGemm.A; // [M][K]
    pB = (const half *)pGemm.B; // [K][N]
    pC = (half *)pGemm.C;        // [M][N]
    int idx, idy, idz;

    // 遍历矩阵 A 的行
    for (idx = 0; idx < M; idx++) {
        // 遍历矩阵 B 的列
        for (idy = 0; idy < N; idy++) {
            // 初始化临时结果为 0
            temp = 0;
            // 进行矩阵乘法的核心计算，将矩阵 A 的第 idx 行与矩阵 B 的第 idy 列元素相乘并累加
            for (idz = 0; idz < K; idz++) {
                temp += (float)pA[idx * lda + idz] * (float)pB[idz * ldb + idy];
            }
            // 将计算结果存储到矩阵 C 的对应位置
            pC[idx * ldc + idy] = (half)temp;
        }
    }
}
```

16.4.1.2 完整程序

GEMM基础实现的完整程序（gemm_base.scpp）：

```
#include <sdaa_perf.h>
#include <type_traits>

using namespace sdaa;

// GEMMArgs包含矩阵乘法所需的参数
typedef struct GEMMArgs {
    int m; // 矩阵 A 与矩阵 C 的行数
    int n; // 矩阵 B 与矩阵 C 的列数
    int k; // 矩阵 A 的列数与矩阵 B 的行数
    int lda; // 矩阵 A 的列间距
    int ldb; // 矩阵 B 的列间距
    int ldc; // 矩阵 C 的列间距
    int bM; // 分块矩阵的行数
    int bN; // 分块矩阵的列数
    int bK; // 分块矩阵的中间维度
    const void *A; // 矩阵 A 的指针
    const void *B; // 矩阵 B 的指针
    void *C; // 矩阵 C 的指针
} GEMMArgs;

__global__ void tecoKernelGemmImpl(GEMMArgs pGemm)
{
    // 矩阵的维度
    int M, N, K;
    // 矩阵的存储步长
    int lda, ldb, ldc;
    // 指向矩阵 A、B、C 的指针
    const half *pA, *pB;
    half *pC;
    // 存储中间计算结果
    float temp;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    M = pGemm.m;
    N = pGemm.n;
    K = pGemm.k;
    // 获取矩阵 A、B、C 的存储步长
    lda = pGemm.lda;
    ldb = pGemm.ldb;
    ldc = pGemm.ldc;

    // 开始性能采样
    perf_start();

    pA = (const half *)pGemm.A; // [M][K]
    pB = (const half *)pGemm.B; // [K][N]
    pC = (half *)pGemm.C; // [M][N]
    int idx, idy, idz;

    // 遍历矩阵 A 的行
    for (idx = 0; idx < M; idx++) {
        // 遍历矩阵 B 的列
        for (idy = 0; idy < N; idy++) {
            // 初始化临时结果为 0
            temp = 0;

```



```

        // 进行矩阵乘法的核心计算，将矩阵 A 的第 idx 行与矩阵 B 的第 idy 列元素相乘并累加
        for (idz = 0; idz < K; idz++) {
            temp += (float)pA[idx * lda + idz] * (float)pB[idz * ldb + idy];
        }
        // 将计算结果存储到矩阵 C 的对应位置
        pC[idx * ldc + idy] = (half)temp;
    }
}

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();
}

int main()
{
    sdaaSetDevice(0);

    GEMMArgs args;

    // 设置矩阵维度参数
    args.m = 1024;
    args.n = 256;
    args.k = 1024;

    args.lda = args.k;
    args.ldb = args.n;
    args.ldc = args.n;
    args.bM = 512;
    args.bN = 256;
    args.bK = 256;

    // 在主机上为矩阵 A、B 和 C 分配内存
    half *h_A = (half *)malloc(args.m * args.k * sizeof(half));
    half *h_B = (half *)malloc(args.k * args.n * sizeof(half));
    half *h_C = (half *)malloc(args.m * args.n * sizeof(half));

    // 为矩阵 A、B 和 C 设置初始值
    for (size_t i = 0; i < args.m * args.k; ++i) {
        h_A[i] = static_cast<half>(1);
    }

    for (size_t i = 0; i < args.k * args.n; ++i) {
        h_B[i] = static_cast<half>(1);
    }

    for (size_t i = 0; i < args.m * args.n; ++i) {
        h_C[i] = static_cast<half>(0);
    }

    // 在设备上为矩阵 A、B 和 C 分配内存
    half *d_A, *d_B, *d_C;

    sdaaMalloc((void **)&d_A, args.m * args.k * sizeof(half));
    sdaaMalloc((void **)&d_B, args.k * args.n * sizeof(half));
    sdaaMalloc((void **)&d_C, args.m * args.n * sizeof(half));

    sdaaMemcpy(d_A, h_A, args.m * args.k * sizeof(half), sdaaMemcpyHostToDevice);

```

```
sdaaMemcpy(d_B, h_B, args.k * args.n * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_C, h_C, args.m * args.n * sizeof(half), sdaaMemcpyHostToDevice);

args.A = d_A;
args.B = d_B;
args.C = d_C;

tecoKernelGemmImpl<<<1>>>(args);

sdaaDeviceSynchronize();

// 将结果矩阵 C 从设备内存复制到主机内存
sdaaMemcpy(h_C, d_C, args.m * args.n * sizeof(half), sdaaMemcpyDeviceToHost);

// 释放设备端内存
sdaaFree(d_A);
sdaaFree(d_B);
sdaaFree(d_C);

// 释放主机端内存
free(h_A);
free(h_B);
free(h_C);

return 0;
}
```

16.4.1.3 性能数据

程序编译

1. 执行以下编译命令，将 `gemm_base.scpp` 生成可执行文件 `gemm_base.out`。

```
tecocc gemm_base.scpp -o gemm_base.out
```

2. 执行以下命令，运行程序：

```
./gemm_base.out
```

执行结果

在本示例中展示部分SPE的性能数据：

SPE的ID	计算任务的执行周期（拍）
SPE0	104112003140
SPE1	103771006233
SPE2	103884208956

SPE的ID	计算任务的执行周期（拍）
SPE3	104056261869
SPE4	104129126819
SPE5	106351733623
SPE6	103884205730
SPE7	104056261867
SPE8	106712005833
SPE9	106466259128

性能数据

GEMM算子基础实现的性能数据：平均执行周期为104812341117拍。

16.4.2 GEMM优化策略

16.4.2.1 SIMD

16.4.2.1.1 实现原理

本章主要介绍如何在GEMM算子的基础实现上，使用SIMD优化策略，提升算子性能。

优化原理

SIMD（Single Instruction Multiple Data）单指令多数据，一条指令可以执行多个数据。

SPE上有多个向量寄存器，支持向量指令加速。可以将计算过程中的标量操作转换为向量操作，通过SIMD向量处理的能力实现多数据并行操作。

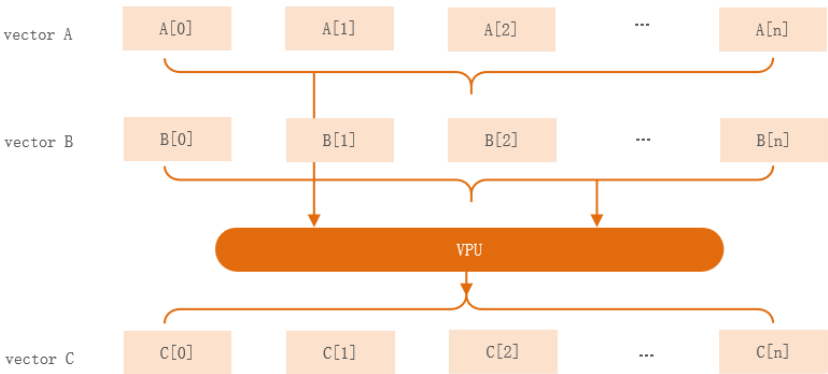


图1: 单指令多数据（SIMD）

在GEMM实现的过程中，数据加载和计算过程可以通过 SIMD 优化策略，将计算过程中常见的标量操作转换为向量操作，从而对数据进行批量处理，提升GEMM的性能。

代码逻辑

SIMD优化通过 `tecoKernelGemmSIMDImpl` 函数实现，该函数使用 SIMD 指令集高效地执行矩阵乘法操作。

1. 从参数 `GEMMArgs` 中获取矩阵 `A`、`B`、`C` 的维度信息和存储步长信息。
 - a. 维度信息：描述矩阵的大小，即矩阵的行数和矩阵的列数；
 - b. 存储步长信息：描述使用二维数组在内存中存储矩阵时，同一列中相邻元素之间的偏移量；
2. 使用 `for` 循环，遍历矩阵的分块：
 - a. 根据 `idx` 计算当前分块的行索引 `idM` 和列索引 `idN`，并获取 `pCurrC` 的位置。
 - b. 对每个 `K` 维度的分块：
 - i. 获取矩阵 `A` 和 `B` 中当前分块的位置 `pCurrA` 和 `pCurrB`。
 - ii. 使用 `memcpy_stride` 将数据从Global存储空间复制到SPM存储空间，并对 `LocalB` 进行转置存储在 `LocalB_T` 中。
 - iii. 嵌套的 `for` 循环遍历SPM存储空间内矩阵分块后的行和列，使用 `reduce_Sdot` 计算矩阵乘法。
 - `reduce_Sdot` 第一个 `for` 循环处理长度为 64 的元素块，每次迭代加载并计算 64 个元素的点积，分四组 16 个元素进行处理，每组使用 `simd_load` 加载数据，计算并累加点积结果。
 - 第二个 `for` 循环处理长度为 16 的元素块，使用 `simd_load` 加载数据，计算并累加点积结果。
 - 最后合并中间结果。
 - iv. 结果存储在 `TempC` 中并累加。
3. 根据结果矩阵的类型，将结果存储在 `LocalC` 中。
4. 将结果复制到矩阵 `C` 的对应位置，并释放分配的SPM存储空间。

示例代码如下：

```
// 点积计算的宏定义
// 该宏根据数据长度分不同阶段进行点积计算
// 先处理长度大于等于 64 的部分，然后处理长度大于等于 16 的部分，最后将部分结果累加
#define unroll_dot(load_func_a, load_func_b, type, type_v) \
do { \
    type *px1 = (type *)x1; \
    type *px2 = (type *)x2; \
    i = 0; \
    int up; \
    type_v a0, a1, a2, a3; \
    type_v b0, b1, b2, b3; \
    up = len - 64; \
    for (; i <= up; i += 64) { \
        load_func_a(a0, px1 + 0); \
        load_func_a(a1, px1 + 16); \

```

```

        load_func_a(a2, px1 + 32);          \
        load_func_a(a3, px1 + 48);          \
        load_func_b(b0, px2 + 0);           \
        load_func_b(b1, px2 + 16);          \
        load_func_b(b2, px2 + 32);          \
        load_func_b(b3, px2 + 48);          \
        ans_v0 += a0 * b0;                  \
        ans_v1 += a1 * b1;                  \
        ans_v2 += a2 * b2;                  \
        ans_v3 += a3 * b3;                  \
        px1 += 64;                          \
        px2 += 64;                          \
    }                                         \
    up = len - 16;                          \
    for (; i <= up; i += 16) {               \
        load_func_a(a0, px1 + 0);           \
        load_func_b(b0, px2 + 0);           \
        ans_v0 += a0 * b0;                  \
        px1 += 16;                          \
        px2 += 16;                          \
    }                                         \
    ans_v0 += ans_v1;                       \
    ans_v2 += ans_v3;                       \
    ans_v0 += ans_v2;                       \
} while (0)

// 定义点积计算函数的宏定义
// 该宏根据指针的对齐情况选择不同的加载函数进行点积计算
// 最后将部分结果累加并处理剩余元素, 得到最终点积结果
#define function_dot(func_name, type, type_v) \
    static __device__ void func_name(const type *x1, const type *x2, int len, float *ans) \
    { \
        int i = 0; \
        type_v ans_v0 = 0, ans_v1 = 0, ans_v2 = 0, ans_v3 = 0; \
        int align1 = is_aligned(x1, sizeof(type_v)); \
        int align2 = is_aligned(x2, sizeof(type_v)); \
        if (align1) { \
            if (align2) \
                unroll_dot(simd_load, simd_load, type, type_v); \
            else \
                unroll_dot(simd_load, simd_loadu, type, type_v); \
        } else { \
            if (align2) \
                unroll_dot(simd_loadu, simd_load, type, type_v); \
            else \
                unroll_dot(simd_loadu, simd_loadu, type, type_v); \
        } \
        float tans = 0; \
        floatv16 f_ans_v0; \
        f_ans_v0 = ans_v0; \
        for (int k = 0; k < 16; ++k) { \
            tans += ((float *)&f_ans_v0)[k]; \
        } \
        for (; i < len; ++i) { \
            tans += x1[i] * x2[i]; \
        } \
        *ans = tans; \
    }

```

```

function_dot(reduce_Sdot, float, floatv16);

__global__ void tecoKernelGemmSIMDImpl(GEMMArgs pGemm)
{
    // 线程块的行索引和列索引
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;
    const int K = pGemm.k;

    // 从 GEMMArgs 结构体中获取矩阵 A、B、C 的存储步长
    const int lda = pGemm.lda;
    const int ldb = pGemm.ldb;
    const int ldc = pGemm.ldc;

    // 获取矩阵 A、B、C 的指针
    const half *A = (const half *)pGemm.A;
    const half *B = (const half *)pGemm.B;
    half *C = (half *)pGemm.C;

    // 外部分块的大小
    const int bM = pGemm.bM;
    const int bN = pGemm.bN;
    const int bK = pGemm.bK;

    // 外部分块的数量
    int nM = M / bM;
    int nN = N / bN;
    int nK = K / bK;

    // 内部块的大小
    int LocalbM = bM / 8;
    int LocalbN = bN / 4;
    int LocalbK = bK;

    // 内部块的位置
    const half *pA = A + cid * LocalbM * lda;
    const half *pB = B + rid * LocalbN;
    half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

    // 指向当前操作元素的指针
    const half *pCurrA;
    const half *pCurrB;
    half *pCurrC;

    // 计算存储数据所需的长度和步长, 考虑元素大小和存储布局
    const int LenA = LocalbM * LocalbK * sizeof(half);
    const int BsizeA = LocalbK * sizeof(half);
    const int StrideA = (lda - LocalbK) * sizeof(half);

    const int LenB = LocalbK * LocalbN * sizeof(half);
    const int BsizeB = LocalbN * sizeof(half);
    const int StrideB = (ldb - LocalbN) * sizeof(half);

    const int LenC = LocalbM * LocalbN * sizeof(half);

```

```

const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);

// 计算所需的SPM存储空间大小
int spm_size =
    LenA + 2 * LenB + LenC + 2 * LocalbK * sizeof(float) + LocalbM * LocalbN * sizeof(float);

// 分配SPM内存空间
half *LocalA = (half *)malloc(LenA);
half *LocalB = (half *)malloc(LenB);
half *LocalB_T = (half *)malloc(LenB);
half *LocalC = (half *)malloc(LenC);
float *TempA = (float *)malloc(LocalbK * sizeof(float));
float *TempB = (float *)malloc(LocalbK * sizeof(float));
float *TempC = (float *)malloc(LocalbM * LocalbN * sizeof(float));

int idM, idN, idK;

// 开始性能采样
perf_start();

// 遍历矩阵的分块
for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算当前分块的行索引和列索引
    idM = idx / nN;
    idN = idx % nN;
    // 获取当前分块在矩阵 C 中的起始位置
    pCurrC = pC + idM * bM * ldc + idN * bN;
    // 初始化临时结果为 0
    memset(TempC, 0, LocalbM * LocalbN * sizeof(float));
    // 遍历 K 维度的分块
    for (idK = 0; idK < nK; ++idK) {
        // 获取矩阵 A 和 B 中当前分块的位置
        pCurrA = pA + idM * bM * lda + idK * bK; // A[idM][idK]
        pCurrB = pB + idK * bK * ldb + idN * bN; // B[idK][idN]
        // 将数据复制到SPM内存空间, 考虑存储布局
        memcpy_stride(LocalA, pCurrA, LenA, BsizeA, {0, (size_t)StrideA});
        memcpy_stride(LocalB, pCurrB, LenB, BsizeB, {0, (size_t)StrideB});
        // 转置矩阵 B 的SPM内存空间部分
        for (int tbk = 0; tbk < LocalbK; tbk++) {
            for (int tbn = 0; tbn < LocalbN; tbn++) {
                LocalB_T[tbn * LocalbK + tbk] = LocalB[tbk * LocalbN + tbn];
            }
        }
        // 遍历本地分块的行
        for (int ibx = 0; ibx < LocalbM; ibx++) {
            // 将数据从半精度转换为单精度
            batch_H2S(LocalA + ibx * LocalbK, TempA, LocalbK);
            // 遍历本地分块的列
            for (int iby = 0; iby < LocalbN; iby++) {
                float temp = 0;
                // 将数据从半精度转换为单精度
                batch_H2S(LocalB_T + iby * LocalbK, TempB, LocalbK);
                // 执行点积操作
                reduce_Sdot(TempA, TempB, LocalbK, &temp);
                // 累加结果
                TempC[ibx * LocalbN + iby] += temp;
            }
        }
    }
}

```

```

    }

    // 在存储结果之前，将计算结果进行类型转换
    batch_S2H(TempC, LocalC, LocalbM * LocalbN);

    // 将结果复制回矩阵 C 的对应位置
    memcpy_stride(pCurrC, LocalC, LenC, BsizeC, {(size_t)StrideC, 0});
} // Loop idx

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();

// 释放分配的SPM内存空间
free(LocalA);
free(LocalB);
free(LocalB_T);
free(LocalC);
free(TempA);
free(TempB);
free(TempC);
}

```

16.4.2.1.2 完整程序

使用SIMD优化策略的完整程序（gemm_simd.scpp）：

```

#include <sdaa_perf.h>
#include <type_traits>

using namespace sdaa;

// GEMMArgs包含矩阵乘法所需的参数
typedef struct GEMMArgs {
    int m; // 矩阵 A 与矩阵 C 的行数
    int n; // 矩阵 B 与矩阵 C 的列数
    int k; // 矩阵 A 的列数与矩阵 B 的行数
    int lda; // 矩阵 A 的列间距
    int ldb; // 矩阵 B 的列间距
    int ldc; // 矩阵 C 的列间距
    int bM; // 分块矩阵的行数
    int bN; // 分块矩阵的列数
    int bK; // 分块矩阵的中间维度
    const void *A; // 矩阵 A 的指针
    const void *B; // 矩阵 B 的指针
    void *C; // 矩阵 C 的指针
} GEMMArgs;

// 判断指针 x 是否按 alignment 字节对齐

```



```

#define is_aligned(x, alignment) (((size_t)(x) & ((alignment)-1)) == 0)

// 用于处理数据的批量操作，根据数据长度分不同阶段进行处理
// 先处理长度大于等于 128 的部分，然后处理长度大于等于 64 的部分，
// 接着处理长度大于等于 16 的部分，最后处理剩余部分
#define __align_batch(type_1, type_2, vec_type_1, vec_type_2, x, y, len, load_func, store_func, \
    op_func) \
{ \
    vec_type_1 vx0, vx1, vx2, vx3, vx4, vx5, vx6, vx7; \
    vec_type_2 vy0, vy1, vy2, vy3, vy4, vy5, vy6, vy7; \
    int __i = 0; \
    type_1 *px = (type_1 *) (x); \
    type_2 *py = (type_2 *) (y); \
    for (__i = 0; __i <= len - 128; __i += 128) { \
        load_func(vx0, px); \
        load_func(vx1, px + 16); \
        load_func(vx2, px + 32); \
        load_func(vx3, px + 48); \
        load_func(vx4, px + 64); \
        load_func(vx5, px + 80); \
        load_func(vx6, px + 96); \
        load_func(vx7, px + 112); \
        \
        vy0 = op_func(vx0); \
        vy1 = op_func(vx1); \
        vy2 = op_func(vx2); \
        vy3 = op_func(vx3); \
        vy4 = op_func(vx4); \
        vy5 = op_func(vx5); \
        vy6 = op_func(vx6); \
        vy7 = op_func(vx7); \
        \
        store_func(vy0, py); \
        store_func(vy1, py + 16); \
        store_func(vy2, py + 32); \
        store_func(vy3, py + 48); \
        store_func(vy4, py + 64); \
        store_func(vy5, py + 80); \
        store_func(vy6, py + 96); \
        store_func(vy7, py + 112); \
        \
        px += 128; \
        py += 128; \
    } \
    \
    for (; __i <= len - 64; __i += 64) { \
        load_func(vx0, px); \
        load_func(vx1, px + 16); \
        load_func(vx2, px + 32); \
        load_func(vx3, px + 48); \
        \
        vy0 = op_func(vx0); \
        vy1 = op_func(vx1); \
        vy2 = op_func(vx2); \
        vy3 = op_func(vx3); \
        \
        store_func(vy0, py); \
        store_func(vy1, py + 16); \
        store_func(vy2, py + 32); \
    } \
}

```

```

        store_func(vy3, py + 48);

        px += 64;
        py += 64;
    }

    for (; __i <= len - 16; __i += 16) {
        load_func(vx0, px);
        vy0 = op_func(vx0);
        store_func(vy0, py);

        px += 16;
        py += 16;
    }

    for (; __i < len; __i++) {
        *py = *px;
        px++;
        py++;
    }
}

// 根据数据长度和指针对齐情况选择不同的处理方式
// 如果长度小于 32, 直接逐个处理元素
// 如果指针都对齐, 调用 __align_batch 宏进行批量对齐操作
// 如果指针有未对齐情况, 根据偏移量处理部分元素后再调用 __align_batch 宏
#define __batch_unary_ops(name, type_1, type_2, vec_type_1, vec_type_2, op_func)
static __device__ inline __attribute__((always_inline)) void name(const type_1 *x, type_2 *y,
                                                                    int len)
{
    if (len < 32) {
        for (int i = 0; i < len; i++) {
            y[i] = x[i];
        }
        return;
    }
    const int align_n1 = sizeof(vec_type_1);
    const int align_n2 = sizeof(vec_type_2);
    if (((((size_t)x) & (align_n1 - 1)) == 0) && (((size_t)y) & (align_n2 - 1)) == 0)) {
        __align_batch(type_1, type_2, vec_type_1, vec_type_2, x, y, len, simd_load,
                      simd_store, op_func);
        return;
    } else {
        int padding_x = ((size_t)(x) & (align_n1 - 1)) / sizeof(type_1);
        int padding_y = ((size_t)(y) & (align_n2 - 1)) / sizeof(type_2);
        if (padding_x == padding_y) {
            for (int i = 0; i < 16 - padding_x; i++) {
                y[i] = x[i];
            }
            __align_batch(type_1, type_2, vec_type_1, vec_type_2, x + 16 - padding_x,
                          y + 16 - padding_y, len - 16 + padding_x, simd_load, simd_store,
                          op_func);
            return;
        } else {
            __align_batch(type_1, type_2, vec_type_1, vec_type_2, x, y, len, simd_loadu,
                          simd_storeu, op_func);
            return;
        }
    }
}

```

```

}

// 定义批量单精度浮点数转半精度浮点数的操作函数
// 调用 __batch_unary_ops 宏, 使用 simd_cvt_f2h 函数进行转换
__batch_unary_ops(batch_S2H, float, half, floatv16, halfv16, simd_cvt_f2h);
// 定义批量半精度浮点数转单精度浮点数的操作函数
// 调用 __batch_unary_ops 宏, 使用 simd_cvt_h2f 函数进行转换
__batch_unary_ops(batch_H2S, half, float, halfv16, floatv16, simd_cvt_h2f);

#undef __align_batch
#undef __batch_unary_ops

// 展开点积计算的宏定义
// 该宏根据数据长度分不同阶段进行点积计算
// 先处理长度大于等于 64 的部分, 然后处理长度大于等于 16 的部分, 最后将部分结果累加
#define unroll_dot(load_func_a, load_func_b, type, type_v) \
do { \
    type *px1 = (type *)x1; \
    type *px2 = (type *)x2; \
    i = 0; \
    int up; \
    type_v a0, a1, a2, a3; \
    type_v b0, b1, b2, b3; \
    up = len - 64; \
    for (; i <= up; i += 64) { \
        load_func_a(a0, px1 + 0); \
        load_func_a(a1, px1 + 16); \
        load_func_a(a2, px1 + 32); \
        load_func_a(a3, px1 + 48); \
        load_func_b(b0, px2 + 0); \
        load_func_b(b1, px2 + 16); \
        load_func_b(b2, px2 + 32); \
        load_func_b(b3, px2 + 48); \
        ans_v0 += a0 * b0; \
        ans_v1 += a1 * b1; \
        ans_v2 += a2 * b2; \
        ans_v3 += a3 * b3; \
        px1 += 64; \
        px2 += 64; \
    } \
    up = len - 16; \
    for (; i <= up; i += 16) { \
        load_func_a(a0, px1 + 0); \
        load_func_b(b0, px2 + 0); \
        ans_v0 += a0 * b0; \
        px1 += 16; \
        px2 += 16; \
    } \
    ans_v0 += ans_v1; \
    ans_v2 += ans_v3; \
    ans_v0 += ans_v2; \
} while (0)

// 定义点积计算函数的宏定义
// 该宏根据指针的对齐情况选择不同的加载函数进行点积计算
// 最后将部分结果累加并处理剩余元素, 得到最终点积结果
#define function_dot(func_name, type, type_v) \
static __device__ void func_name(const type *x1, const type *x2, int len, float *ans) \
{

```

```

int i = 0;
type_v ans_v0 = 0, ans_v1 = 0, ans_v2 = 0, ans_v3 = 0;
int align1 = is_aligned(x1, sizeof(type_v));
int align2 = is_aligned(x2, sizeof(type_v));
if (align1) {
    if (align2)
        unroll_dot(simd_load, simd_load, type, type_v);
    else
        unroll_dot(simd_load, simd_loadu, type, type_v);
} else {
    if (align2)
        unroll_dot(simd_loadu, simd_load, type, type_v);
    else
        unroll_dot(simd_loadu, simd_loadu, type, type_v);
}

float tans = 0;
floatv16 f_ans_v0;
f_ans_v0 = ans_v0;
for (int k = 0; k < 16; ++k) {
    tans += ((float *)&f_ans_v0)[k];
}
for (; i < len; ++i) {
    tans += x1[i] * x2[i];
}
*ans = tans;
}

```

```
function_dot(reduce_Sdot, float, floatv16);
```

```
__global__ void tecoKernelGemmSIMDImpl(GEMMArgs pGemm)
{
```

```

    // 线程块的行索引和列索引
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;
    const int K = pGemm.k;

    // 从 GEMMArgs 结构体中获取矩阵 A、B、C 的存储步长
    const int lda = pGemm.lda;
    const int ldb = pGemm.ldb;
    const int ldc = pGemm.ldc;

    // 获取矩阵 A、B、C 的指针
    const half *A = (const half *)pGemm.A;
    const half *B = (const half *)pGemm.B;
    half *C = (half *)pGemm.C;

    // 外部分块的大小
    const int bM = pGemm.bM;
    const int bN = pGemm.bN;
    const int bK = pGemm.bK;

    // 外部分块的数量
    int nM = M / bM;
    int nN = N / bN;

```

```

int nK = K / bK;

// 内部块的大小
int LocalbM = bM / 8;
int LocalbN = bN / 4;
int LocalbK = bK;

// 内部块的位置
const half *pA = A + cid * LocalbM * lda;
const half *pB = B + rid * LocalbN;
half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

// 指向当前操作元素的指针
const half *pCurrA;
const half *pCurrB;
half *pCurrC;

// 计算存储数据所需的长度和步长, 考虑元素大小和存储布局
const int LenA = LocalbM * LocalbK * sizeof(half);
const int BsizeA = LocalbK * sizeof(half);
const int StrideA = (lda - LocalbK) * sizeof(half);

const int LenB = LocalbK * LocalbN * sizeof(half);
const int BsizeB = LocalbN * sizeof(half);
const int StrideB = (ldb - LocalbN) * sizeof(half);

const int LenC = LocalbM * LocalbN * sizeof(half);
const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);

// 计算所需的SPM存储空间大小
int spm_size =
    LenA + 2 * LenB + LenC + 2 * LocalbK * sizeof(float) + LocalbM * LocalbN * sizeof(float);

// 分配SPM内存空间
half *LocalA = (half *)malloc(LenA);
half *LocalB = (half *)malloc(LenB);
half *LocalB_T = (half *)malloc(LenB);
half *LocalC = (half *)malloc(LenC);
float *TempA = (float *)malloc(LocalbK * sizeof(float));
float *TempB = (float *)malloc(LocalbK * sizeof(float));
float *TempC = (float *)malloc(LocalbM * LocalbN * sizeof(float));

int idM, idN, idK;

// 开始性能采样
perf_start();

// 遍历矩阵的分块
for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算当前分块的行索引和列索引
    idM = idx / nN;
    idN = idx % nN;
    // 获取当前分块在矩阵 C 中的起始位置
    pCurrC = pC + idM * bM * ldc + idN * bN;
    // 初始化临时结果为 0
    memset(TempC, 0, LocalbM * LocalbN * sizeof(float));
    // 遍历 K 维度的分块
    for (idK = 0; idK < nK; ++idK) {

```

```

// 获取矩阵 A 和 B 中当前分块的位置
pCurrA = pA + idM * bM * lda + idK * bK; // A[idM][idK]
pCurrB = pB + idK * bK * ldb + idN * bN; // B[idK][idN]
// 将数据复制到SPM内存空间, 考虑存储布局
memcpy_stride(LocalA, pCurrA, LenA, BsizeA, {0, (size_t)StrideA});
memcpy_stride(LocalB, pCurrB, LenB, BsizeB, {0, (size_t)StrideB});
// 转置矩阵 B 的SPM内存空间部分
for (int tbk = 0; tbk < LocalbK; tbk++) {
    for (int tbn = 0; tbn < LocalbN; tbn++) {
        LocalB_T[tbn * LocalbK + tbk] = LocalB[tbk * LocalbN + tbn];
    }
}
// 遍历本地分块的行
for (int ibx = 0; ibx < LocalbM; ibx++) {
    // 将数据从半精度转换为单精度
    batch_H2S(LocalA + ibx * LocalbK, TempA, LocalbK);
    // 遍历本地分块的列
    for (int iby = 0; iby < LocalbN; iby++) {
        float temp = 0;
        // 将数据从半精度转换为单精度
        batch_H2S(LocalB_T + iby * LocalbK, TempB, LocalbK);
        // 执行点积操作
        reduce_Sdot(TempA, TempB, LocalbK, &temp);
        // 累加结果
        TempC[ibx * LocalbN + iby] += temp;
    }
}
}

// 在存储结果之前, 将计算结果进行类型转换
batch_S2H(TempC, LocalC, LocalbM * LocalbN);

// 将结果复制回矩阵 C 的对应位置
memcpy_stride(pCurrC, LocalC, LenC, BsizeC, {(size_t)StrideC, 0});
} // Loop idx

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();

// 释放分配的SPM内存空间
free(LocalA);
free(LocalB);
free(LocalB_T);
free(LocalC);
free(TempA);
free(TempB);
free(TempC);
}

int main()
{
    sdAASetDevice(0);

    GEMMArgs args;

    // 设置矩阵维度参数
    args.m = 1024;

```

```
args.n = 256;
args.k = 1024;

args.lda = args.k;
args.ldb = args.n;
args ldc = args.n;
args.bM = 512;
args.bN = 256;
args.bK = 256;

// 在主机上为矩阵 A、B 和 C 分配内存
half *h_A = (half *)malloc(args.m * args.k * sizeof(half));
half *h_B = (half *)malloc(args.k * args.n * sizeof(half));
half *h_C = (half *)malloc(args.m * args.n * sizeof(half));

// 为矩阵 A、B 和 C 设置初始值
for (size_t i = 0; i < args.m * args.k; ++i) {
    h_A[i] = static_cast<half>(1);
}

for (size_t i = 0; i < args.k * args.n; ++i) {
    h_B[i] = static_cast<half>(1);
}

for (size_t i = 0; i < args.m * args.n; ++i) {
    h_C[i] = static_cast<half>(0);
}

// 在设备上为矩阵 A、B 和 C 分配内存
half *d_A, *d_B, *d_C;

sdAAMalloc((void **)&d_A, args.m * args.k * sizeof(half));
sdAAMalloc((void **)&d_B, args.k * args.n * sizeof(half));
sdAAMalloc((void **)&d_C, args.m * args.n * sizeof(half));

sdAAMemcpy(d_A, h_A, args.m * args.k * sizeof(half), sdAAMemcpyHostToDevice);
sdAAMemcpy(d_B, h_B, args.k * args.n * sizeof(half), sdAAMemcpyHostToDevice);
sdAAMemcpy(d_C, h_C, args.m * args.n * sizeof(half), sdAAMemcpyHostToDevice);

args.A = d_A;
args.B = d_B;
args.C = d_C;

// 执行kernel函数
tecoKernelGemmSIMDImpl<<1>>>(args);

sdAADeviceSynchronize();

// 将结果矩阵 C 从设备内存复制到主机内存
sdAAMemcpy(h_C, d_C, args.m * args.n * sizeof(half), sdAAMemcpyDeviceToHost);

// 释放设备端内存
sdAAFree(d_A);
sdAAFree(d_B);
sdAAFree(d_C);

// 释放主机端内存
free(h_A);
free(h_B);
```

```
    free(h_C);

    return 0;
}
```

16.4.2.1.3 性能数据

程序编译

1. 执行以下编译命令，将 `gemm_simd.scpp` 生成可执行文件 `gemm_simd.out`。

```
tecocc gemm_simd.scpp -o gemm_simd.out
```

2. 执行以下命令，运行程序：

```
./gemm_simd.out
```

执行结果

在本示例中展示部分SPE的性能数据：

SPE的ID	优化前的计算任务的执行周期（拍）	优化后的计算任务的执行周期（拍）
SPE0	104112003140	13611000
SPE1	103771006233	13589858
SPE2	103884208956	13612086
SPE3	104056261869	13611950
SPE4	104129126819	13509469
SPE5	106351733623	13581135
SPE6	103884205730	13496047
SPE7	104056261867	13509516
SPE8	106712005833	13612917
SPE9	106466259128	13600914

性能优化

优化策略	平均执行周期（拍）
基础实现	104812341117
SIMD	13573489

使用SIMD优化策略后的性能，是基础实现的约7843倍。

16.4.2.2 矩阵乘法单元

16.4.2.2.1 实现原理

本章主要介绍如何在SIMD优化策略的基础上，使用矩阵乘法单元，提升算子性能。

优化原理

矩阵乘法单元是专门用于执行矩阵乘加运算的模块。通过矩阵乘法单元实现的矩阵乘加计算，比依靠处理器的通用乘加指令实现的乘加计算更加高效。

在 GEMM 算法里，存在大量的矩阵乘加计算，因此若能合理地使用矩阵乘法单元，可以显著提升浮点运算的性能。

SDAA C从调用接口上可分为阻塞型矩阵乘和非阻塞型矩阵乘。非阻塞型矩阵乘虽然使用相对复杂，但性能较好。因此，本文将使用非阻塞型矩阵乘进行性能优化，非阻塞型矩阵乘接口的详细使用方法，可参考非阻塞型矩阵乘的[使用逻辑](#)章节。

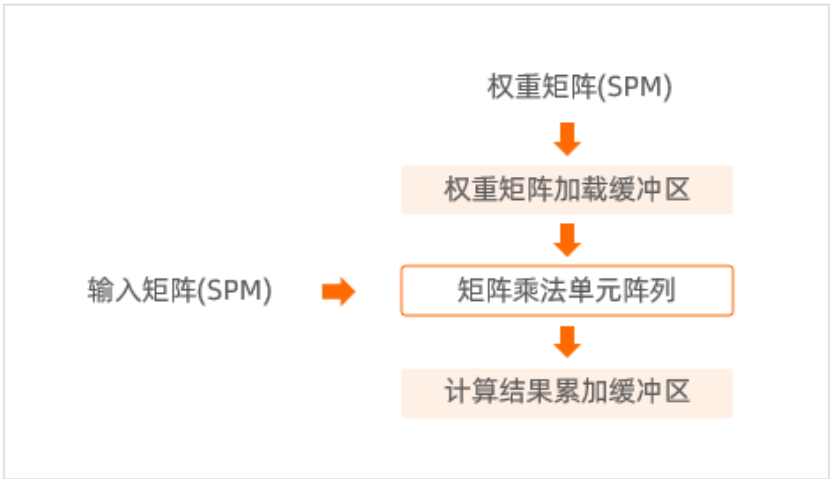


图1 阻塞型矩阵乘接口的使用逻辑

代码逻辑

非阻塞型矩阵乘通过 `tecoKernelGemmMatmulImpl` 函数实现，该函数在 SIMD 的基础上使用矩阵乘法单元进行性能优化。

1. 根据 `threadIdx` 指定矩阵的行索引 `rid` 和列索引 `cid`。
2. 从参数 `GEMMArgs` 中获取矩阵 `A`、`B`、`C` 的维度信息和存储步长信息。

- a. 维度信息：描述矩阵的大小，即矩阵的行数和矩阵的列数；
 - b. 存储步长信息：描述使用二维数组在内存中存储矩阵时，同一列中相邻元素之间的偏移量；
3. 分配SPM存储空间，包括 `LocalCompA`、`LocalCompB`、`LocalTB`、`LocalC`、`LocalCompC` 和 `tempC`。
 4. 配置 `MatmulHandle` 接口参数，并初始化矩阵乘计算单元。
 5. 使用 `for` 循环遍历矩阵的分块：
 - a. 根据 `idx` 计算当前分块的行索引 `idM` 和列索引 `idN`，并获取 `pCurrA` 和 `pCurrB` 的位置。
 - b. 等待矩阵乘法句柄加载矩阵 `A` 和 `B`，将数据复制到SPM存储空间并进行 SIMD 操作。
 - c. 使用 `matmul_compute` 进行矩阵乘法计算操作。
 - d. 使用 `matmul_wait` 等待运算结果写回。
 6. 对结果 `C` 进行置换操作，使用 SIMD 操作存储结果。
 7. 根据结果矩阵的类型决定是否进行数据类型转换，将结果存储在 `LocalC` 中。
 8. 将结果复制到矩阵 `C` 的对应位置，并释放分配的SPM存储空间。

代码示例如下：

```
__global__ void tecoKernelGemmMatmulImpl(GEMMArgs pGemm)
{
    // 计算线程行索引和列索引
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;
    const int K = pGemm.k;

    // 从 GEMMArgs 结构体中获取矩阵 A、B、C 的存储步长
    const int lda = pGemm.lda;
    const int ldb = pGemm.ldb;
    const int ldc = pGemm.ldc;

    // 获取矩阵 A、B、C 的指针
    const half *A = (const half *)pGemm.A;
    const half *B = (const half *)pGemm.B;
    half *C = (half *)pGemm.C;

    // 外部分块的大小
    const int bM = pGemm.bM;
    const int bN = pGemm.bN;
    const int bK = pGemm.bK;

    // 外部分块的数量
    int nM = M / bM;
```

```

int nN = N / bN;
int nK = K / bK;

// 内部块的大小
int LocalbM = bM / 8;
int LocalbN = bN / 4;
int LocalbK = bK;

// 内部块的位置
const half *pA = A + cid * LocalbM * lda;
const half *pB = B + rid * LocalbN;
half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

// 指向当前操作元素的指针
const half *pCurrA;
const half *pCurrB;
half *pCurrC;

// 计算存储数据所需的长度和步长, 考虑元素大小和存储布局
const int LenA = LocalbM * LocalbK * sizeof(half);
const int BsizeA = LocalbK * sizeof(half);
const int StrideA = (lda - LocalbK) * sizeof(half);

const int LenB = LocalbK * LocalbN * sizeof(half);
const int BsizeB = LocalbN * sizeof(half);
const int StrideB = (ldb - LocalbN) * sizeof(half);

const int LenC = LocalbM * LocalbN * sizeof(half);
const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);

// 计算所需的SPM存储空间大小
int spm_size = LenA + 2 * LenB + LenC + LenC / sizeof(half) * sizeof(float) * 2;

// 分配SPM存储空间
half *LocalCompA = (half *)malloc(LenA);
half *LocalCompB = (half *)malloc(LenB * 2);
// 是存储 B 矩阵的转置
half *LocalTB = LocalCompB + LocalbK * LocalbN;
half *LocalC = (half *)malloc(LenC);
// 存储中间结果
float *LocalCompC = (float *)malloc(LocalbM * LocalbN * sizeof(float) * 2);
float *tempC = LocalCompC + LocalbM * LocalbN;
half *pCA = LocalCompA;
half *pCB = LocalCompB;

int im, in, ik;
int idM, idN, idK;

SIMDType vtb[SIMDSIZE];

// 开始性能采样
perf_start();

// 初始化矩阵乘法句柄
MatmulHandle handleMMA;
matmul_init(handleMMA, MatmulHalfToFloat);

// 遍历矩阵的分块

```

```

for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算当前分块的行索引和列索引
    idM = idx / nN;
    idN = idx % nN;
    for (idK = 0; idK < nK; ++idK) {
        // 获取矩阵 A 和 B 中当前分块的位置
        pCurrA = pA + idM * BM * lda + idK * bK; // A[idM][idK]
        pCurrB = pB + idK * bK * ldb + idN * bN; // B[idK][idN]

        // 等待加载输入到矩阵乘法句柄
        matmul_wait_loading_input(handleMMA);
        // 将数据复制到SPM存储空间
        memcpy_stride(pCA, pCurrA, LenA, BsizeA, {0, (size_t)StrideA});

        // 等待加载权重到矩阵乘法句柄
        matmul_wait_loading_weight(handleMMA);
        // 将数据复制到SPM存储空间
        memcpy_stride(pCB, pCurrB, LenB, BsizeB, {0, (size_t)StrideB});

        // 使用 SIMD 加载和存储数据, 对 B 矩阵进行转置存储
        for (ik = 0; ik < LocalbK; ik += 8) {
            simd_load(vtb[0], (float*)(pCB + ik * LocalbN));
            simd_load(vtb[1], (float*)(pCB + ik * LocalbN + 32));
            simd_load(vtb[2], (float*)(pCB + (ik + 1) * LocalbN));
            simd_load(vtb[3], (float*)(pCB + (ik + 1) * LocalbN + 32));
            simd_load(vtb[4], (float*)(pCB + (ik + 2) * LocalbN));
            simd_load(vtb[5], (float*)(pCB + (ik + 2) * LocalbN + 32));
            simd_load(vtb[6], (float*)(pCB + (ik + 3) * LocalbN));
            simd_load(vtb[7], (float*)(pCB + (ik + 3) * LocalbN + 32));
            simd_load(vtb[8], (float*)(pCB + (ik + 4) * LocalbN));
            simd_load(vtb[9], (float*)(pCB + (ik + 4) * LocalbN + 32));
            simd_load(vtb[10], (float*)(pCB + (ik + 5) * LocalbN));
            simd_load(vtb[11], (float*)(pCB + (ik + 5) * LocalbN + 32));
            simd_load(vtb[12], (float*)(pCB + (ik + 6) * LocalbN));
            simd_load(vtb[13], (float*)(pCB + (ik + 6) * LocalbN + 32));
            simd_load(vtb[14], (float*)(pCB + (ik + 7) * LocalbN));
            simd_load(vtb[15], (float*)(pCB + (ik + 7) * LocalbN + 32));
            simd_store(vtb[0], (float*)(LocalTB + ik * 32));
            simd_store(vtb[1], (float*)(LocalTB + 32 * LocalbK + ik * 32));
            simd_store(vtb[2], (float*)(LocalTB + (ik + 1) * 32));
            simd_store(vtb[3], (float*)(LocalTB + 32 * LocalbK + (ik + 1) * 32));
            simd_store(vtb[4], (float*)(LocalTB + (ik + 2) * 32));
            simd_store(vtb[5], (float*)(LocalTB + 32 * LocalbK + (ik + 2) * 32));
            simd_store(vtb[6], (float*)(LocalTB + (ik + 3) * 32));
            simd_store(vtb[7], (float*)(LocalTB + 32 * LocalbK + (ik + 3) * 32));
            simd_store(vtb[8], (float*)(LocalTB + (ik + 4) * 32));
            simd_store(vtb[9], (float*)(LocalTB + 32 * LocalbK + (ik + 4) * 32));
            simd_store(vtb[10], (float*)(LocalTB + (ik + 5) * 32));
            simd_store(vtb[11], (float*)(LocalTB + 32 * LocalbK + (ik + 5) * 32));
            simd_store(vtb[12], (float*)(LocalTB + (ik + 6) * 32));
            simd_store(vtb[13], (float*)(LocalTB + 32 * LocalbK + (ik + 6) * 32));
            simd_store(vtb[14], (float*)(LocalTB + (ik + 7) * 32));
            simd_store(vtb[15], (float*)(LocalTB + 32 * LocalbK + (ik + 7) * 32));
        }

        // 根据是否是最后一个 K 维度的分块进行不同的处理
        if (idK < nK - 1) {
            for (ik = 0; ik + 32 - 1 < LocalbK; ik += 32) {
                matmul_set_flushing_output(handleMMA, false);
            }
        }
    }
}

```

```

        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        // 加载权重
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        // 执行矩阵乘法计算
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
} else {
    for (ik = 0; ik + 64 - 1 < LocalbK; ik += 32) {
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
    matmul_set_flushing_output(handleMMA, false);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
    matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

    matmul_set_flushing_output(handleMMA, true);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
    matmul_load_weight(handleMMA, LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
        MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    // 存储矩阵乘法结果
    matmul_store(handleMMA, LocalCompC, LocalbM * 2, MatmulN32);
}
} // loop idk
// 等待矩阵乘法操作完成
matmul_wait(handleMMA);

// 对结果 C 进行置换操作
for (im = 0; im < LocalbM; im++) {
    simd_load(vtb[0], LocalCompC + im * 32);
    simd_load(vtb[1], LocalCompC + im * 32 + 16);
    simd_load(vtb[2], LocalCompC + (im + LocalbM) * 32);
    simd_load(vtb[3], LocalCompC + (im + LocalbM) * 32 + 16);
    simd_store(vtb[0], tempC + im * 64);
    simd_store(vtb[1], tempC + im * 64 + 16);
    simd_store(vtb[2], tempC + im * 64 + 32);
    simd_store(vtb[3], tempC + im * 64 + 48);
}

// 在存储结果之前, 将计算结果进行类型转换
batch_S2H(tempC, (half *)LocalC, 4096);

```

```

        // 将结果复制回矩阵 C 的对应位置
        memcpy_stride(pC + idM * bM * ldc + idN * bN, LocalC, LenC, BsizeC, {(size_t)StrideC, 0});
    } // Loop idx

    // 性能采样
    perf_stop();
    // 打印出性能采样结果
    perf_print();

    // 释放分配的内存
    free(LocalCompA);
    free(LocalCompB);
    free(LocalC);
    free(LocalCompC);
}

```

16.4.2.2.2 完整程序

使用矩阵乘法单元优化策略的完整程序（gemm_matmul.scpp）：

```

#include <sdaa_matmul.h>
#include <sdaa_perf.h>
#include <type_traits>

using namespace sdaa;

#define REDUITEM 32
#define SIMDSIZE 32
typedef floatv16 SIMDType;

// GEMMArgs包含矩阵乘法所需的参数
typedef struct GEMMArgs {
    int m; // 矩阵 A 与矩阵 C 的行数
    int n; // 矩阵 B 与矩阵 C 的列数
    int k; // 矩阵 A 的列数与矩阵 B 的行数
    int lda; // 矩阵 A 的列间距
    int ldb; // 矩阵 B 的列间距
    int ldc; // 矩阵 C 的列间距
    int bM; // 分块矩阵的行数
    int bN; // 分块矩阵的列数
    int bK; // 分块矩阵的中间维度
    const void *A; // 矩阵 A 的指针
    const void *B; // 矩阵 B 的指针
    void *C; // 矩阵 C 的指针
} GEMMArgs;

// 设备函数，将浮点数数组转换为半精度浮点数数组，并根据对齐情况进行处理
__device__ void align_batch_function(float* x, half* y, int len)
{
    // 定义向量变量
    floatv16 vx0, vx1, vx2, vx3, vx4, vx5, vx6, vx7;
    halfv16 vy0, vy1, vy2, vy3, vy4, vy5, vy6, vy7;
    int i = 0;

    // 外层循环，以 128 个元素为一组处理数据
    // 每次循环处理完一组后，指针移动 128 个元素位置继续处理下一组

```

```
for (i = 0; i <= len - 128; i += 128) {
    // 使用 simd_load 函数加载数据
    simd_load(vx0, x);
    simd_load(vx1, x + 16);
    simd_load(vx2, x + 32);
    simd_load(vx3, x + 48);
    simd_load(vx4, x + 64);
    simd_load(vx5, x + 80);
    simd_load(vx6, x + 96);
    simd_load(vx7, x + 112);

    // 使用 simd_cvt_f2h 函数将加载的浮点型向量数据转换为半精度浮点型
    vy0 = simd_cvt_f2h(vx0);
    vy1 = simd_cvt_f2h(vx1);
    vy2 = simd_cvt_f2h(vx2);
    vy3 = simd_cvt_f2h(vx3);
    vy4 = simd_cvt_f2h(vx4);
    vy5 = simd_cvt_f2h(vx5);
    vy6 = simd_cvt_f2h(vx6);
    vy7 = simd_cvt_f2h(vx7);

    // 使用 simd_store 函数将转换后的半精度浮点型向量数据存储到内存中
    simd_store(vy0, y);
    simd_store(vy1, y + 16);
    simd_store(vy2, y + 32);
    simd_store(vy3, y + 48);
    simd_store(vy4, y + 64);
    simd_store(vy5, y + 80);
    simd_store(vy6, y + 96);
    simd_store(vy7, y + 112);

    // 更新指针
    x += 128;
    y += 128;
}

// 中层循环, 处理长度小于 128 但大于或等于 64 的剩余数据
// 每次以 64 个元素为一组进行处理
// 每次循环处理完一组后, 指针移动 64 个元素位置继续处理下一组
for (; i <= len - 64; i += 64) {
    simd_load(vx0, x);
    simd_load(vx1, x + 16);
    simd_load(vx2, x + 32);
    simd_load(vx3, x + 48);

    vy0 = simd_cvt_f2h(vx0);
    vy1 = simd_cvt_f2h(vx1);
    vy2 = simd_cvt_f2h(vx2);
    vy3 = simd_cvt_f2h(vx3);

    simd_store(vy0, y);
    simd_store(vy1, y + 16);
    simd_store(vy2, y + 32);
    simd_store(vy3, y + 48);

    x += 64;
    y += 64;
}
```

```

// 内层循环, 处理长度小于 64 但大于或等于 16 的剩余数据
// 每次以 16 个元素为一组进行处理
for (; i <= len - 16; i += 16) {
    simd_load(vx0, x);
    vy0 = simd_cvt_f2h(vx0);
    simd_store(vy0, y);

    x += 16;
    y += 16;
}

// 最内层循环, 处理长度小于 16 的剩余数据
// 逐元素进行赋值操作
for (; i < len; i++) {
    *y = *x;
    x++;
    y++;
}
}

// 根据长度和对齐情况调用 align_batch_function 进行浮点数到半精度浮点数的转换
__device__ void batch_S2H(float* x, half* y, int len)
{
    if (len < 32) {
        for (int i = 0; i < len; i++) {
            y[i] = x[i];
        }
        return;
    }
    const int align_n1 = sizeof(floatv16);
    const int align_n2 = sizeof(halfv16);
    if (((size_t)x & (align_n1 - 1)) == 0 && (((size_t)y & (align_n2 - 1)) == 0)) {
        align_batch_function((float*)x, y, len);
        return;
    } else {
        int padding_x = ((size_t)(x) & (align_n1 - 1)) / sizeof(float);
        int padding_y = ((size_t)(y) & (align_n2 - 1)) / sizeof(half);
        if (padding_x == padding_y) {
            for (int i = 0; i < 16 - padding_x; i++) {
                y[i] = x[i];
            }
            align_batch_function(x + 16 - padding_x, y + 16 - padding_y, len - 16 + padding_x);
            return;
        } else {
            align_batch_function((float*)x, y, len);
            return;
        }
    }
}

__global__ void tecoKernelGemmMatmulImpl(GEMMArgs pGemm)
{
    // 计算线程行索引和列索引
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;

```



```

const int K = pGemm.k;

// 从 GEMMArgs 结构体中获取矩阵 A、B、C 的存储步长
const int lda = pGemm.lda;
const int ldb = pGemm.ldb;
const int ldc = pGemm.ldc;

// 获取矩阵 A、B、C 的指针
const half *A = (const half *)pGemm.A;
const half *B = (const half *)pGemm.B;
half *C = (half *)pGemm.C;

// 外部分块的大小
const int bM = pGemm.bM;
const int bN = pGemm.bN;
const int bK = pGemm.bK;

// 外部分块的数量
int nM = M / bM;
int nN = N / bN;
int nK = K / bK;

// 内部块的大小
int LocalbM = bM / 8;
int LocalbN = bN / 4;
int LocalbK = bK;

// 内部块的位置
const half *pA = A + cid * LocalbM * lda;
const half *pB = B + rid * LocalbN;
half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

// 指向当前操作元素的指针
const half *pCurrA;
const half *pCurrB;
half *pCurrC;

// 计算存储数据所需的长度和步长, 考虑元素大小和存储布局
const int LenA = LocalbM * LocalbK * sizeof(half);
const int BsizeA = LocalbK * sizeof(half);
const int StrideA = (lda - LocalbK) * sizeof(half);

const int LenB = LocalbK * LocalbN * sizeof(half);
const int BsizeB = LocalbN * sizeof(half);
const int StrideB = (ldb - LocalbN) * sizeof(half);

const int LenC = LocalbM * LocalbN * sizeof(half);
const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);

// 计算所需的SPM存储空间大小
int spm_size = LenA + 2 * LenB + LenC + LenC / sizeof(half) * sizeof(float) * 2;

// 分配SPM存储空间
half *LocalCompA = (half *)malloc(LenA);
half *LocalCompB = (half *)malloc(LenB * 2);
// 是存储 B 矩阵的转置
half *LocalTB = LocalCompB + LocalbK * LocalbN;
half *LocalC = (half *)malloc(LenC);

```

```

// 存储中间结果
float *LocalCompC = (float *)malloc(LocalbM * LocalbN * sizeof(float) * 2);
float *tempC = LocalCompC + LocalbM * LocalbN;
half *pCA = LocalCompA;
half *pCB = LocalCompB;

int im, in, ik;
int idM, idN, idK;

SIMDType vtb[SIMDSIZE];

// 开始性能采样
perf_start();

// 初始化矩阵乘法句柄
MatmulHandle handleMMA;
matmul_init(handleMMA, MatmulHalfToFloat);

// 遍历矩阵的分块
for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算当前分块的行索引和列索引
    idM = idx / nN;
    idN = idx % nN;
    for (idK = 0; idK < nK; ++idK) {
        // 获取矩阵 A 和 B 中当前分块的位置
        pCurrA = pA + idM * bM * lda + idK * bK; // A[idM][idK]
        pCurrB = pB + idK * bK * ldb + idN * bN; // B[idK][idN]

        // 等待加载输入到矩阵乘法句柄
        matmul_wait_loading_input(handleMMA);
        // 将数据复制到SPM存储空间
        memcpy_stride(pCA, pCurrA, LenA, BsizeA, {0, (size_t)StrideA});

        // 等待加载权重到矩阵乘法句柄
        matmul_wait_loading_weight(handleMMA);
        // 将数据复制到SPM存储空间
        memcpy_stride(pCB, pCurrB, LenB, BsizeB, {0, (size_t)StrideB});

        // 使用 SIMD 加载和存储数据, 对 B 矩阵进行转置存储
        for (ik = 0; ik < LocalbK; ik += 8) {
            simd_load(vtb[0], (float *) (pCB + ik * LocalbN));
            simd_load(vtb[1], (float *) (pCB + ik * LocalbN + 32));
            simd_load(vtb[2], (float *) (pCB + (ik + 1) * LocalbN));
            simd_load(vtb[3], (float *) (pCB + (ik + 1) * LocalbN + 32));
            simd_load(vtb[4], (float *) (pCB + (ik + 2) * LocalbN));
            simd_load(vtb[5], (float *) (pCB + (ik + 2) * LocalbN + 32));
            simd_load(vtb[6], (float *) (pCB + (ik + 3) * LocalbN));
            simd_load(vtb[7], (float *) (pCB + (ik + 3) * LocalbN + 32));
            simd_load(vtb[8], (float *) (pCB + (ik + 4) * LocalbN));
            simd_load(vtb[9], (float *) (pCB + (ik + 4) * LocalbN + 32));
            simd_load(vtb[10], (float *) (pCB + (ik + 5) * LocalbN));
            simd_load(vtb[11], (float *) (pCB + (ik + 5) * LocalbN + 32));
            simd_load(vtb[12], (float *) (pCB + (ik + 6) * LocalbN));
            simd_load(vtb[13], (float *) (pCB + (ik + 6) * LocalbN + 32));
            simd_load(vtb[14], (float *) (pCB + (ik + 7) * LocalbN));
            simd_load(vtb[15], (float *) (pCB + (ik + 7) * LocalbN + 32));
            simd_store(vtb[0], (float *) (LocalTB + ik * 32));
            simd_store(vtb[1], (float *) (LocalTB + 32 * LocalbK + ik * 32));
            simd_store(vtb[2], (float *) (LocalTB + (ik + 1) * 32));

```

```

    simd_store(vtb[3], (float*)(LocalTB + 32 * LocalbK + (ik + 1) * 32));
    simd_store(vtb[4], (float*)(LocalTB + (ik + 2) * 32));
    simd_store(vtb[5], (float*)(LocalTB + 32 * LocalbK + (ik + 2) * 32));
    simd_store(vtb[6], (float*)(LocalTB + (ik + 3) * 32));
    simd_store(vtb[7], (float*)(LocalTB + 32 * LocalbK + (ik + 3) * 32));
    simd_store(vtb[8], (float*)(LocalTB + (ik + 4) * 32));
    simd_store(vtb[9], (float*)(LocalTB + 32 * LocalbK + (ik + 4) * 32));
    simd_store(vtb[10], (float*)(LocalTB + (ik + 5) * 32));
    simd_store(vtb[11], (float*)(LocalTB + 32 * LocalbK + (ik + 5) * 32));
    simd_store(vtb[12], (float*)(LocalTB + (ik + 6) * 32));
    simd_store(vtb[13], (float*)(LocalTB + 32 * LocalbK + (ik + 6) * 32));
    simd_store(vtb[14], (float*)(LocalTB + (ik + 7) * 32));
    simd_store(vtb[15], (float*)(LocalTB + 32 * LocalbK + (ik + 7) * 32));
}

// 根据是否是最后一个 K 维度的分块进行不同的处理
if (idK < nK - 1) {
    for (ik = 0; ik + 32 - 1 < LocalbK; ik += 32) {
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        // 加载权重
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        // 执行矩阵乘法计算
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
} else {
    for (ik = 0; ik + 64 - 1 < LocalbK; ik += 32) {
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
    matmul_set_flushing_output(handleMMA, false);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
    matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

    matmul_set_flushing_output(handleMMA, true);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
    matmul_load_weight(handleMMA, LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
        MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    // 存储矩阵乘法结果
    matmul_store(handleMMA, LocalCompC, LocalbM * 2, MatmulN32);
}

```

```

    } // loop idk
    // 等待矩阵乘法操作完成
    matmul_wait(handleMMA);

    // 对结果 C 进行置换操作
    for (im = 0; im < LocalbM; im++) {
        simd_load(vtb[0], LocalCompC + im * 32);
        simd_load(vtb[1], LocalCompC + im * 32 + 16);
        simd_load(vtb[2], LocalCompC + (im + LocalbM) * 32);
        simd_load(vtb[3], LocalCompC + (im + LocalbM) * 32 + 16);
        simd_store(vtb[0], tempC + im * 64);
        simd_store(vtb[1], tempC + im * 64 + 16);
        simd_store(vtb[2], tempC + im * 64 + 32);
        simd_store(vtb[3], tempC + im * 64 + 48);
    }

    // 在存储结果之前, 将计算结果进行类型转换
    batch_S2H(tempC, (half *)LocalC, 4096);

    // 将结果复制回矩阵 C 的对应位置
    memcpy_stride(pC + idM * bM * ldc + idN * bN, LocalC, LenC, BsizeC, {(size_t)StrideC, 0});
} // Loop idx

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();

// 释放分配的内存
free(LocalCompA);
free(LocalCompB);
free(LocalC);
free(LocalCompC);
}

int main()
{
    sdaaSetDevice(0);

    GEMMArgs args;

    // 设置矩阵维度参数
    args.m = 1024;
    args.n = 256;
    args.k = 1024;

    args.la = args.k;
    args.lb = args.n;
    args.lc = args.n;
    args.bM = 512;
    args.bN = 256;
    args.bK = 256;

    // 在主机上为矩阵 A、B 和 C 分配内存
    half *h_A = (half *)malloc(args.m * args.k * sizeof(half));
    half *h_B = (half *)malloc(args.k * args.n * sizeof(half));
    half *h_C = (half *)malloc(args.m * args.n * sizeof(half));

    // 为矩阵 A、B 和 C 设置初始值

```

```

for (size_t i = 0; i < args.m * args.k; ++i) {
    h_A[i] = static_cast<half>(1);
}

for (size_t i = 0; i < args.k * args.n; ++i) {
    h_B[i] = static_cast<half>(1);
}

for (size_t i = 0; i < args.m * args.n; ++i) {
    h_C[i] = static_cast<half>(0);
}

// 在设备上为矩阵 A、B 和 C 分配内存
half *d_A, *d_B, *d_C;

sdaaMalloc((void **>(&d_A), args.m * args.k * sizeof(half));
sdaaMalloc((void **>(&d_B), args.k * args.n * sizeof(half));
sdaaMalloc((void **>(&d_C), args.m * args.n * sizeof(half));

sdaaMemcpy(d_A, h_A, args.m * args.k * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_B, h_B, args.k * args.n * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_C, h_C, args.m * args.n * sizeof(half), sdaaMemcpyHostToDevice);

args.A = d_A;
args.B = d_B;
args.C = d_C;

tecoKernelGemmMatmulImpl<<<1>>>(args);

sdaaDeviceSynchronize();

// 将结果矩阵 C 从设备内存复制到主机内存
sdaaMemcpy(h_C, d_C, args.m * args.n * sizeof(half), sdaaMemcpyDeviceToHost);

// 释放设备端内存
sdaaFree(d_A);
sdaaFree(d_B);
sdaaFree(d_C);

// 释放主机端内存
free(h_A);
free(h_B);
free(h_C);

return 0;
}

```

16.4.2.2.3 性能数据

程序编译

1. 执行以下编译命令，将 `gemm_matmul.scpp` 生成可执行文件 `gemm_matmul.out`。

```
tecocc gemm_matmul.scpp -o gemm_matmul.out
```

2. 执行以下命令，运行程序：

```
./gemm_matmul.out
```

执行结果

在本示例中展示部分SPE性能数据：

SPE的ID	优化前（SIMD）的计算任务的执行周期（拍）	优化后的计算任务的执行周期（拍）
SPE0	13611000	505021
SPE1	13589858	543540
SPE2	13612086	540672
SPE3	13611950	531941
SPE4	13509469	545714
SPE5	13581135	540365
SPE6	13496047	546281
SPE7	13509516	524844
SPE8	13612917	537730
SPE9	13600914	503830

性能优化

优化策略	平均执行周期（拍）
基础实现	104812341117
SIMD	13573489
矩阵乘法单元	531993

使用矩阵乘法单元优化策略后的性能，是使用SIMD的约25倍。

16.4.2.3 广播

16.4.2.3.1 实现原理

本章主要介绍如何在矩阵乘法单元的基础上，使用广播优化策略，提升算子性能。

优化原理

针对多个计算核心同时读取Global存储空间中同一块数据的场景，可以使用广播（Broadcast）进行优化：一个计算核心将Global存储上的数据读入自己的SPM存储空间后，通过广播的方式，将数据同时传输到多个计算核心的SPM存储空间。

在 GEMM 计算过程中，根据实际情况，对矩阵 A 和 B 进行或列划分，并将划分后的数据分配到不同的计算核心，进行矩阵乘计算。如果各计算核心均独立地从 Global 存储空间读取相同的数据，会导致资源的浪费。因此，一个计算核心可以先将 Global 存储中的数据读取到自身的 SPM 存储空间，再通过广播策略，同步将该数据传输到多个计算核心的 SPM 存储空间，从而优化数据读取流程，减少不必要的资源浪费，提升整体的计算效率与资源利用率。

SDAA C中广播接口使用方法，可参考[Broadcast数据搬运](#)章节。

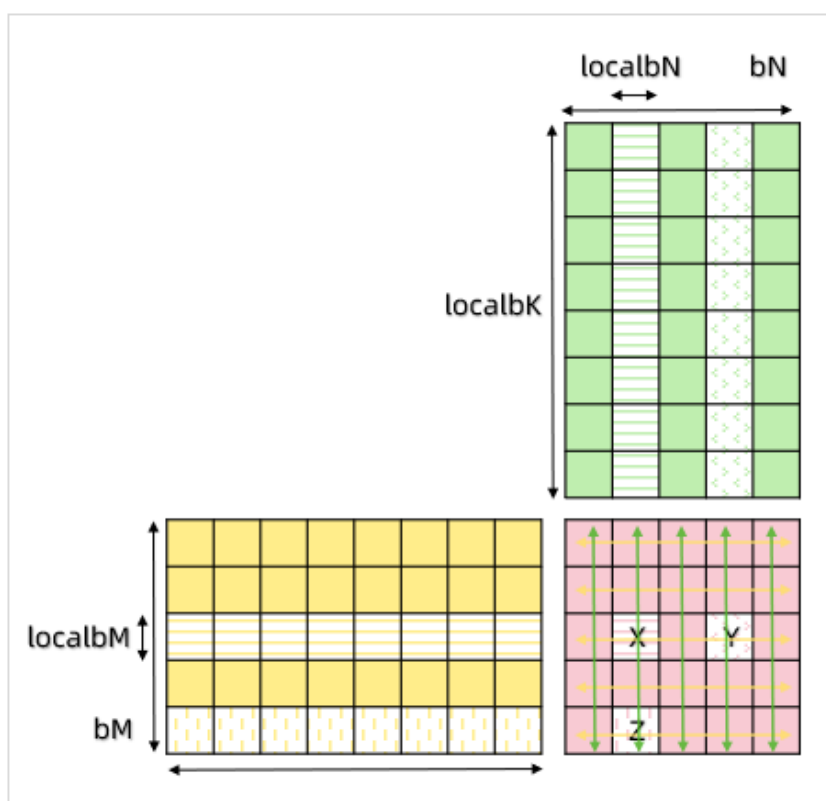


图1 广播的实现原理

代码逻辑

广播通过 `tecoKernelGemmBroadcastImpl` 函数实现，该函数在矩阵乘法单元的基础上，使用广播进行性能优化。

1. 根据 `threadIdx` 指定矩阵的行索引 `rid` 和列索引 `cid`。
2. 从 `pGemm` 结构体中获取矩阵 A 和 B 的维度信息。

3. 分配SPM存储空间，包括 `LocalCompA` 、 `LocalCompB` 、 `LocalTB` 、 `LocalC` 、 `LocalCompC` 和 `tempC` 。
4. 配置 `BroadcastHandle` 接口参数。
5. 计算用于广播的线程索引 `typeAThreads` 和 `typeBThreads` ，并创建线程组 `col_thread_group` 和 `row_thread_group` 以及相应的广播句柄 `col_handle` 和 `row_handle` 。
6. 调用 `broadcast_async` 函数，分别对矩阵 `A` 和 `B` 执行非阻塞型广播操作。
 - 对于矩阵 `A` ：
 - `pDA` 是目标缓冲区的指针，即需要将数据广播到的本地缓冲区位置，用于暂存广播来的数据。
 - `(void *)pA` 是数据源指针，即要广播的数据所在的原始内存位置。
 - `BsizeA` 表示每次广播的数据大小，确定每次传输的数据量规模。
 - `typeAThreads` 用于指定参与此次列广播操作的线程信息。
 - `BroadcastGlobalToSpm` 表示广播方向的参数。
 - `col_handle` 用于管理矩阵 `A` 列广播操作的句柄，用于控制整个广播过程。
 - 对于矩阵 `B` ：参数含义与矩阵 `A` 的类似，只是对应的是行广播相关的参数。
 - `pDB` 是目标缓冲区的指针，即需要将数据广播到的本地缓冲区位置，用于暂存广播来的数据。
 - `(void *)pB` 是数据源指针，即要广播的数据所在的原始内存位置。
 - `BsizeB` 表示每次广播的数据大小，确定每次传输的数据量规模。
 - `typeBThreads` 用于指定参与此次行广播操作的线程信息。
 - `BroadcastGlobalToSpm` 表示广播方向的参数。
 - `row_handle` 用于管理矩阵 `B` 行广播操作的句柄，用于控制整个广播过程。
7. 执行矩阵乘法计算，存储计算结果。

示例代码如下：

```
__global__ void tecoKernelGemmBroadcastImpl(GEMMArgs pGemm)
{
    // 计算线程行索引和列索引
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;
    const int K = pGemm.k;

    // 从 GEMMArgs 结构体中获取矩阵 A、B、C 的存储步长
    const int lda = pGemm.lda;
    const int ldb = pGemm.ldb;
    const int ldc = pGemm.ldc;
```



```
// 获取矩阵 A、B、C 的指针
const half *A = (const half *)pGemm.A;
const half *B = (const half *)pGemm.B;
half *C = (half *)pGemm.C;

// 外部分块的大小
const int bM = pGemm.bM;
const int bN = pGemm.bN;
const int bK = pGemm.bK;

// 外部分块的数量
int nM = M / bM;
int nN = N / bN;
int nK = K / bK;

// 内部块的大小
int LocalbM = bM / 8;
int LocalbN = bN / 4;
int LocalbK = bK;

// 内部块的位置
const half *pA = A + cid * LocalbM * lda;
const half *pB = B + rid * LocalbN;
half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

// 指向当前操作元素的指针
const half *pCurrA;
const half *pCurrB;
half *pCurrC;

// 计算存储数据所需的长度和步长
const int LenA = LocalbM * LocalbK * sizeof(half);
const int BsizeA = LocalbK * sizeof(half);
const int StrideA = (lda - LocalbK) * sizeof(half);

const int LenB = LocalbK * LocalbN * sizeof(half);
const int BsizeB = LocalbN * sizeof(half);
const int StrideB = (ldb - LocalbN) * sizeof(half);

const int LenC = LocalbM * LocalbN * sizeof(half);
const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);
// 计算所需的SPM存储空间大小
int spm_size = LenA + 2 * LenB + LenC + LenC / sizeof(half) * sizeof(float) * 2;

// 分配本SPM存储空间
half *LocalCompA = (half *)malloc(LenA);
half *LocalCompB = (half *)malloc(LenB * 2);
half *LocalTB = LocalCompB + LocalbK * LocalbN;
half *LocalC = (half *)malloc(LenC);
// 存储中间结果
float *LocalCompC = (float *)malloc(LocalbM * LocalbN * sizeof(float) * 2);
float *tempC = LocalCompC + LocalbM * LocalbN;
half *pCA = LocalCompA;
half *pCB = LocalCompB;
int im, in, ik;
int idM, idN, idK;
```

```

SIMDType vtb[SIMDSIZE];

// 开始性能采样
perf_start();

// 初始化矩阵乘法句柄
MatmulHandle handleMMA;
matmul_init(handleMMA, MatmulHalfToFloat);

// 计算用于广播的线程索引, 列广播
int typeAThreads = (3 - cid % 4) * 8 + cid;
// 计算用于广播的线程索引, 行广播
int typeBThreads = rid * 8 + rid;

// 用于列广播的线程掩码
uint64_t mask = 0;
mask = 0x01010101 << cid;
ThreadGroup col_thread_group(mask);

// 用于行广播的线程掩码
mask = 0xFF << (8 * rid);
ThreadGroup row_thread_group(mask);

// 列广播的句柄
BroadcastHandle col_handle(&col_thread_group);
// 行广播的句柄
BroadcastHandle row_handle(&row_thread_group);

// 遍历矩阵的分块
for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算当前分块的行索引和列索引
    idM = idx / nN;
    idN = idx % nN;
    for (idK = 0; idK < nK; ++idK) {
        // 获取矩阵 A 和 B 中当前分块的位置
        pCurrA = pA + idM * bM * lda + idK * bK; // A[idM][idK]
        pCurrB = pB + idK * bK * ldb + idN * bN; // B[idK][idN]

        // 等待矩阵乘法句柄加载输入
        matmul_wait_loading_input(handleMMA);
        // 同步列广播的线程组
        sync_threads(col_thread_group);
        // 执行列广播操作, 将数据从Global内存广播到SPM内存
        broadcast(pCA, (void *)pCurrA, LenA, BsizeA, {0, (size_t)StrideA}, typeAThreads, BroadcastGlobalToSpm, col_handle);

        // 等待矩阵乘法句柄加载权重
        matmul_wait_loading_weight(handleMMA);
        // 同步行广播的线程组
        sync_threads(row_thread_group);
        // 执行广播操作, 将数据从Global内存广播到SPM内存
        broadcast(pCB, (void *)pCurrB, LenB, BsizeB, {0, (size_t)StrideB}, typeBThreads, BroadcastGlobalToSpm, row_handle);

        // 使用 SIMD 加载和存储数据, 对 B 矩阵进行转置存储
        for (ik = 0; ik < LocalbK; ik += 8) {
            simd_load(vtb[0], (float *) (pCB + ik * LocalbN));
            simd_load(vtb[1], (float *) (pCB + ik * LocalbN + 32));
            simd_load(vtb[2], (float *) (pCB + (ik + 1) * LocalbN));
        }
    }
}

```

```

    simd_load(vtb[3], (float*)(pCB + (ik + 1) * LocalbN + 32));
    simd_load(vtb[4], (float*)(pCB + (ik + 2) * LocalbN));
    simd_load(vtb[5], (float*)(pCB + (ik + 2) * LocalbN + 32));
    simd_load(vtb[6], (float*)(pCB + (ik + 3) * LocalbN));
    simd_load(vtb[7], (float*)(pCB + (ik + 3) * LocalbN + 32));
    simd_load(vtb[8], (float*)(pCB + (ik + 4) * LocalbN));
    simd_load(vtb[9], (float*)(pCB + (ik + 4) * LocalbN + 32));
    simd_load(vtb[10], (float*)(pCB + (ik + 5) * LocalbN));
    simd_load(vtb[11], (float*)(pCB + (ik + 5) * LocalbN + 32));
    simd_load(vtb[12], (float*)(pCB + (ik + 6) * LocalbN));
    simd_load(vtb[13], (float*)(pCB + (ik + 6) * LocalbN + 32));
    simd_load(vtb[14], (float*)(pCB + (ik + 7) * LocalbN));
    simd_load(vtb[15], (float*)(pCB + (ik + 7) * LocalbN + 32));
    simd_store(vtb[0], (float*)(LocalTB + ik * 32));
    simd_store(vtb[1], (float*)(LocalTB + 32 * LocalbK + ik * 32));
    simd_store(vtb[2], (float*)(LocalTB + (ik + 1) * 32));
    simd_store(vtb[3], (float*)(LocalTB + 32 * LocalbK + (ik + 1) * 32));
    simd_store(vtb[4], (float*)(LocalTB + (ik + 2) * 32));
    simd_store(vtb[5], (float*)(LocalTB + 32 * LocalbK + (ik + 2) * 32));
    simd_store(vtb[6], (float*)(LocalTB + (ik + 3) * 32));
    simd_store(vtb[7], (float*)(LocalTB + 32 * LocalbK + (ik + 3) * 32));
    simd_store(vtb[8], (float*)(LocalTB + (ik + 4) * 32));
    simd_store(vtb[9], (float*)(LocalTB + 32 * LocalbK + (ik + 4) * 32));
    simd_store(vtb[10], (float*)(LocalTB + (ik + 5) * 32));
    simd_store(vtb[11], (float*)(LocalTB + 32 * LocalbK + (ik + 5) * 32));
    simd_store(vtb[12], (float*)(LocalTB + (ik + 6) * 32));
    simd_store(vtb[13], (float*)(LocalTB + 32 * LocalbK + (ik + 6) * 32));
    simd_store(vtb[14], (float*)(LocalTB + (ik + 7) * 32));
    simd_store(vtb[15], (float*)(LocalTB + 32 * LocalbK + (ik + 7) * 32));
}

// 根据是否是最后一个 K 维度的分块进行不同的处理
if (idK < nK - 1) {
    for (ik = 0; ik + 32 - 1 < LocalbK; ik += 32) {
        // 设置矩阵乘法句柄
        matmul_set_flushing_output(handleMMA, false);
        // 设置矩阵乘法句柄的输出行偏移
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        // 加载权重到矩阵乘法句柄
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        // 执行矩阵乘法计算
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
} else {
    for (ik = 0; ik + 64 - 1 < LocalbK; ik += 32) {
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
    }
}

```

```

        matmul_load_weight(handleMMA,
                           LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
                           MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
    matmul_set_flushing_output(handleMMA, false);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
    matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

    matmul_set_flushing_output(handleMMA, true);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
    matmul_load_weight(handleMMA, LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
                       MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    // 存储矩阵乘法结果
    matmul_store(handleMMA, LocalCompC, LocalbM * 2, MatmulN32);
}
} // loop idk
// 等待矩阵乘法操作完成
matmul_wait(handleMMA);

// 对结果 C 进行置换操作
for (im = 0; im < LocalbM; im++) {
    simd_load(vtb[0], LocalCompC + im * 32);
    simd_load(vtb[1], LocalCompC + im * 32 + 16);
    simd_load(vtb[2], LocalCompC + (im + LocalbM) * 32);
    simd_load(vtb[3], LocalCompC + (im + LocalbM) * 32 + 16);
    simd_store(vtb[0], tempC + im * 64);
    simd_store(vtb[1], tempC + im * 64 + 16);
    simd_store(vtb[2], tempC + im * 64 + 32);
    simd_store(vtb[3], tempC + im * 64 + 48);
}

// 在存储结果之前, 将计算结果进行类型转换
batch_S2H(tempC, (half *)LocalC, 4096);
// 将结果复制回矩阵 C 的对应位置
memcpy_stride(pC + idM * bM * ldc + idN * bN, LocalC, LenC, BsizeC, {(size_t)StrideC, 0});
} // loop idx

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();

// 释放分配的内存
free(LocalCompA);
free(LocalCompB);
free(LocalC);
free(LocalCompC);
}

```

16.4.2.3.2 完整程序

使用广播优化策略的完整程序 (gemm_broadcast.scpp) :

```

#include <sdaa_matmul.h>
#include <type_traits>
#include <sdaa_perf.h>

using namespace sdaa;

#define REDUITEM 32
#define SIMDSIZE 32
typedef floatv16 SIMDType;

// GEMMArgs包含矩阵乘法所需的参数
typedef struct GEMMArgs {
    int m; // 矩阵 A 与矩阵 C 的行数
    int n; // 矩阵 B 与矩阵 C 的列数
    int k; // 矩阵 A 的列数与矩阵 B 的行数
    int lda; // 矩阵 A 的列间距
    int ldb; // 矩阵 B 的列间距
    int ldc; // 矩阵 C 的列间距
    int bM; // 分块矩阵的行数
    int bN; // 分块矩阵的列数
    int bK; // 分块矩阵的中间维度
    const void *A; // 矩阵 A 的指针
    const void *B; // 矩阵 B 的指针
    void *C; // 矩阵 C 的指针
} GEMMArgs;

// 设备函数，将浮点数数组转换为半精度浮点数数组，并根据对齐情况进行处理
__device__ void align_batch_function(float* x, half* y, int len)
{
    // 定义向量变量
    floatv16 vx0, vx1, vx2, vx3, vx4, vx5, vx6, vx7;
    halfv16 vy0, vy1, vy2, vy3, vy4, vy5, vy6, vy7;
    int i = 0;

    // 外层循环，以 128 个元素为一组处理数据
    // 每次循环处理完一组后，指针移动 128 个元素位置继续处理下一组
    for (i = 0; i <= len - 128; i += 128) {
        // 使用 simd_load 函数加载数据
        simd_load(vx0, x);
        simd_load(vx1, x + 16);
        simd_load(vx2, x + 32);
        simd_load(vx3, x + 48);
        simd_load(vx4, x + 64);
        simd_load(vx5, x + 80);
        simd_load(vx6, x + 96);
        simd_load(vx7, x + 112);

        // 使用 simd_cvt_f2h 函数将加载的浮点型向量数据转换为半精度浮点型
        vy0 = simd_cvt_f2h(vx0);
        vy1 = simd_cvt_f2h(vx1);
        vy2 = simd_cvt_f2h(vx2);
        vy3 = simd_cvt_f2h(vx3);
        vy4 = simd_cvt_f2h(vx4);
        vy5 = simd_cvt_f2h(vx5);
        vy6 = simd_cvt_f2h(vx6);
        vy7 = simd_cvt_f2h(vx7);

        // 使用 simd_store 函数将转换后的半精度浮点型向量数据存储到内存中
    }
}

```

```

    simd_store(vy0, y);
    simd_store(vy1, y + 16);
    simd_store(vy2, y + 32);
    simd_store(vy3, y + 48);
    simd_store(vy4, y + 64);
    simd_store(vy5, y + 80);
    simd_store(vy6, y + 96);
    simd_store(vy7, y + 112);

    // 更新指针
    x += 128;
    y += 128;
}

// 中层循环, 处理长度小于 128 但大于或等于 64 的剩余数据
// 每次以 64 个元素为一组进行处理
// 每次循环处理完一组后, 指针移动 64 个元素位置继续处理下一组
for (; i <= len - 64; i += 64) {
    simd_load(vx0, x);
    simd_load(vx1, x + 16);
    simd_load(vx2, x + 32);
    simd_load(vx3, x + 48);

    vy0 = simd_cvt_f2h(vx0);
    vy1 = simd_cvt_f2h(vx1);
    vy2 = simd_cvt_f2h(vx2);
    vy3 = simd_cvt_f2h(vx3);

    simd_store(vy0, y);
    simd_store(vy1, y + 16);
    simd_store(vy2, y + 32);
    simd_store(vy3, y + 48);

    x += 64;
    y += 64;
}

// 内层循环, 处理长度小于 64 但大于或等于 16 的剩余数据
// 每次以 16 个元素为一组进行处理
for (; i <= len - 16; i += 16) {
    simd_load(vx0, x);
    vy0 = simd_cvt_f2h(vx0);
    simd_store(vy0, y);

    x += 16;
    y += 16;
}

// 最内层循环, 处理长度小于 16 的剩余数据
// 逐元素进行赋值操作
for (; i < len; i++) {
    *y = *x;
    x++;
    y++;
}
}

// 根据长度和对齐情况调用 align_batch_function 进行浮点数到半精度浮点数的转换
__device__ void batch_S2H(float* x, half* y, int len)

```

```

{
    if (len < 32) {
        for (int i = 0; i < len; i++) {
            y[i] = x[i];
        }
        return;
    }
    const int align_n1 = sizeof(floatv16);
    const int align_n2 = sizeof(halfv16);
    if (((size_t)x & (align_n1 - 1)) == 0) && (((size_t)y & (align_n2 - 1)) == 0) {
        align_batch_function((float*)x, y, len);
        return;
    } else {
        int padding_x = ((size_t)(x) & (align_n1 - 1)) / sizeof(float);
        int padding_y = ((size_t)(y) & (align_n2 - 1)) / sizeof(half);
        if (padding_x == padding_y) {
            for (int i = 0; i < 16 - padding_x; i++) {
                y[i] = x[i];
            }
            align_batch_function(x + 16 - padding_x, y + 16 - padding_y, len - 16 + padding_x);
            return;
        } else {
            align_batch_function((float*)x, y, len);
            return;
        }
    }
}

__global__ void tecoKernelGemmBroadcastImpl(GEMMArgs pGemm)
{
    // 计算线程行索引和列索引
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;
    const int K = pGemm.k;

    // 从 GEMMArgs 结构体中获取矩阵 A、B、C 的存储步长
    const int lda = pGemm.lda;
    const int ldb = pGemm.ldb;
    const int ldc = pGemm.ldc;

    // 获取矩阵 A、B、C 的指针
    const half *A = (const half *)pGemm.A;
    const half *B = (const half *)pGemm.B;
    half *C = (half *)pGemm.C;

    // 外部分块的大小
    const int bM = pGemm.bM;
    const int bN = pGemm.bN;
    const int bK = pGemm.bK;

    // 外部分块的数量
    int nM = M / bM;
    int nN = N / bN;
    int nK = K / bK;

```

```

// 内部块的大小
int LocalbM = bM / 8;
int LocalbN = bN / 4;
int LocalbK = bK;

// 内部块的位置
const half *pA = A + cid * LocalbM * lda;
const half *pB = B + rid * LocalbN;
half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

// 指向当前操作元素的指针
const half *pCurrA;
const half *pCurrB;
half *pCurrC;

// 计算存储数据所需的长度和步长
const int LenA = LocalbM * LocalbK * sizeof(half);
const int BsizeA = LocalbK * sizeof(half);
const int StrideA = (lda - LocalbK) * sizeof(half);

const int LenB = LocalbK * LocalbN * sizeof(half);
const int BsizeB = LocalbN * sizeof(half);
const int StrideB = (ldb - LocalbN) * sizeof(half);

const int LenC = LocalbM * LocalbN * sizeof(half);
const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);
// 计算所需的SPM存储空间大小
int spm_size = LenA + 2 * LenB + LenC + LenC / sizeof(half) * sizeof(float) * 2;

// 分配本SPM存储空间
half *LocalCompA = (half *)malloc(LenA);
half *LocalCompB = (half *)malloc(LenB * 2);
half *LocalTB = LocalCompB + LocalbK * LocalbN;
half *LocalC = (half *)malloc(LenC);
// 存储中间结果
float *LocalCompC = (float *)malloc(LocalbM * LocalbN * sizeof(float) * 2);
float *tempC = LocalCompC + LocalbM * LocalbN;
half *pCA = LocalCompA;
half *pCB = LocalCompB;
int im, in, ik;
int idM, idN, idK;

SIMDType vtb[SIMDSIZE];

// 开始性能采样
perf_start();

// 初始化矩阵乘法句柄
MatmulHandle handleMMA;
matmul_init(handleMMA, MatmulHalfToFloat);

// 计算用于广播的线程索引, 列广播
int typeAThreads = (3 - cid % 4) * 8 + cid;
// 计算用于广播的线程索引, 行广播
int typeBThreads = rid * 8 + rid;

// 用于列广播的线程掩码
uint64_t mask = 0;

```



```

mask = 0x01010101 << cid;
ThreadGroup col_thread_group(mask);

// 用于行广播的线程掩码
mask = 0xFF << (8 * rid);
ThreadGroup row_thread_group(mask);

// 列广播的句柄
BroadcastHandle col_handle(&col_thread_group);
// 行广播的句柄
BroadcastHandle row_handle(&row_thread_group);

// 遍历矩阵的分块
for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算当前分块的行索引和列索引
    idM = idx / nN;
    idN = idx % nN;
    for (idK = 0; idK < nK; ++idK) {
        // 获取矩阵 A 和 B 中当前分块的位置
        pCurrA = pA + idM * BM * lda + idK * bK; // A[idM][idK]
        pCurrB = pB + idK * bK * ldb + idN * bN; // B[idK][idN]

        // 等待矩阵乘法句柄加载输入
        matmul_wait_loading_input(handleMMA);
        // 同步列广播的线程组
        sync_threads(col_thread_group);
        // 执行列广播操作, 将数据从Global内存广播到SPM内存
        broadcast(pCA, (void *)pCurrA, LenA, BsizeA, {0, (size_t)StrideA}, typeAThreads, BroadcastG
lbalToSpm, col_handle);

        // 等待矩阵乘法句柄加载权重
        matmul_wait_loading_weight(handleMMA);
        // 同步行广播的线程组
        sync_threads(row_thread_group);
        // 执行广播操作, 将数据从Global内存广播到SPM内存
        broadcast(pCB, (void *)pCurrB, LenB, BsizeB, {0, (size_t)StrideB}, typeBThreads, BroadcastG
lbalToSpm, row_handle);

        // 使用 SIMD 加载和存储数据, 对 B 矩阵进行转置存储
        for (ik = 0; ik < LocalbK; ik += 8) {
            simd_load(vtb[0], (float *) (pCB + ik * LocalbN));
            simd_load(vtb[1], (float *) (pCB + ik * LocalbN + 32));
            simd_load(vtb[2], (float *) (pCB + (ik + 1) * LocalbN));
            simd_load(vtb[3], (float *) (pCB + (ik + 1) * LocalbN + 32));
            simd_load(vtb[4], (float *) (pCB + (ik + 2) * LocalbN));
            simd_load(vtb[5], (float *) (pCB + (ik + 2) * LocalbN + 32));
            simd_load(vtb[6], (float *) (pCB + (ik + 3) * LocalbN));
            simd_load(vtb[7], (float *) (pCB + (ik + 3) * LocalbN + 32));
            simd_load(vtb[8], (float *) (pCB + (ik + 4) * LocalbN));
            simd_load(vtb[9], (float *) (pCB + (ik + 4) * LocalbN + 32));
            simd_load(vtb[10], (float *) (pCB + (ik + 5) * LocalbN));
            simd_load(vtb[11], (float *) (pCB + (ik + 5) * LocalbN + 32));
            simd_load(vtb[12], (float *) (pCB + (ik + 6) * LocalbN));
            simd_load(vtb[13], (float *) (pCB + (ik + 6) * LocalbN + 32));
            simd_load(vtb[14], (float *) (pCB + (ik + 7) * LocalbN));
            simd_load(vtb[15], (float *) (pCB + (ik + 7) * LocalbN + 32));
            simd_store(vtb[0], (float *) (LocalTB + ik * 32));
            simd_store(vtb[1], (float *) (LocalTB + 32 * LocalbK + ik * 32));
            simd_store(vtb[2], (float *) (LocalTB + (ik + 1) * 32));

```

```

    simd_store(vtb[3], (float*)(LocalTB + 32 * LocalbK + (ik + 1) * 32));
    simd_store(vtb[4], (float*)(LocalTB + (ik + 2) * 32));
    simd_store(vtb[5], (float*)(LocalTB + 32 * LocalbK + (ik + 2) * 32));
    simd_store(vtb[6], (float*)(LocalTB + (ik + 3) * 32));
    simd_store(vtb[7], (float*)(LocalTB + 32 * LocalbK + (ik + 3) * 32));
    simd_store(vtb[8], (float*)(LocalTB + (ik + 4) * 32));
    simd_store(vtb[9], (float*)(LocalTB + 32 * LocalbK + (ik + 4) * 32));
    simd_store(vtb[10], (float*)(LocalTB + (ik + 5) * 32));
    simd_store(vtb[11], (float*)(LocalTB + 32 * LocalbK + (ik + 5) * 32));
    simd_store(vtb[12], (float*)(LocalTB + (ik + 6) * 32));
    simd_store(vtb[13], (float*)(LocalTB + 32 * LocalbK + (ik + 6) * 32));
    simd_store(vtb[14], (float*)(LocalTB + (ik + 7) * 32));
    simd_store(vtb[15], (float*)(LocalTB + 32 * LocalbK + (ik + 7) * 32));
}

// 根据是否是最后一个 K 维度的分块进行不同的处理
if (idK < nK - 1) {
    for (ik = 0; ik + 32 - 1 < LocalbK; ik += 32) {
        // 设置矩阵乘法句柄
        matmul_set_flushing_output(handleMMA, false);
        // 设置矩阵乘法句柄的输出行偏移
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        // 加载权重到矩阵乘法句柄
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        // 执行矩阵乘法计算
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
} else {
    for (ik = 0; ik + 64 - 1 < LocalbK; ik += 32) {
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
    matmul_set_flushing_output(handleMMA, false);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
    matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

    matmul_set_flushing_output(handleMMA, true);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
    matmul_load_weight(handleMMA, LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
        MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    // 存储矩阵乘法结果

```

```

        matmul_store(handleMMA, LocalCompC, LocalbM * 2, MatmulN32);
    }
} // loop idk
// 等待矩阵乘法操作完成
matmul_wait(handleMMA);

// 对结果 C 进行置换操作
for (im = 0; im < LocalbM; im++) {
    simd_load(vtb[0], LocalCompC + im * 32);
    simd_load(vtb[1], LocalCompC + im * 32 + 16);
    simd_load(vtb[2], LocalCompC + (im + LocalbM) * 32);
    simd_load(vtb[3], LocalCompC + (im + LocalbM) * 32 + 16);
    simd_store(vtb[0], tempC + im * 64);
    simd_store(vtb[1], tempC + im * 64 + 16);
    simd_store(vtb[2], tempC + im * 64 + 32);
    simd_store(vtb[3], tempC + im * 64 + 48);
}

// 在存储结果之前, 将计算结果进行类型转换
batch_S2H(tempC, (half *)LocalC, 4096);
// 将结果复制回矩阵 C 的对应位置
memcpy_stride(pC + idM * bM * ldc + idN * bN, LocalC, LenC, BsizeC, {(size_t)StrideC, 0});
} // Loop idx

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();

// 释放分配的内存
free(LocalCompA);
free(LocalCompB);
free(LocalC);
free(LocalCompC);
}

int main()
{
    sdaaSetDevice(0);

    GEMMArgs args;

    // 设置矩阵维度参数
    args.m = 1024;
    args.n = 256;
    args.k = 1024;

    args.lda = args.k;
    args.ldb = args.n;
    args.ldc = args.n;
    args.bM = 512;
    args.bN = 256;
    args.bK = 256;

    // 在主机上为矩阵 A、B 和 C 分配内存
    half *h_A = (half *)malloc(args.m * args.k * sizeof(half));
    half *h_B = (half *)malloc(args.k * args.n * sizeof(half));
    half *h_C = (half *)malloc(args.m * args.n * sizeof(half));

```

```

// 为矩阵 A、B 和 C 设置初始值
for (size_t i = 0; i < args.m * args.k; ++i) {
    h_A[i] = static_cast<half>(1);
}

for (size_t i = 0; i < args.k * args.n; ++i) {
    h_B[i] = static_cast<half>(1);
}

for (size_t i = 0; i < args.m * args.n; ++i) {
    h_C[i] = static_cast<half>(0);
}

// 在设备上为矩阵 A、B 和 C 分配内存
half *d_A, *d_B, *d_C;

sdaaMalloc((void **)&d_A, args.m * args.k * sizeof(half));
sdaaMalloc((void **)&d_B, args.k * args.n * sizeof(half));
sdaaMalloc((void **)&d_C, args.m * args.n * sizeof(half));

sdaaMemcpy(d_A, h_A, args.m * args.k * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_B, h_B, args.k * args.n * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_C, h_C, args.m * args.n * sizeof(half), sdaaMemcpyHostToDevice);

args.A = d_A;
args.B = d_B;
args.C = d_C;

tecokernelGemmBroadcastImpl<<<1>>>(args);

sdaaDeviceSynchronize();

// 将结果矩阵 C 从设备内存复制到主机内存
sdaaMemcpy(h_C, d_C, args.m * args.n * sizeof(half), sdaaMemcpyDeviceToHost);

// 释放设备端内存
sdaaFree(d_A);
sdaaFree(d_B);
sdaaFree(d_C);

// 释放主机端内存
free(h_A);
free(h_B);
free(h_C);

return 0;
}

```

16.4.2.3.3 性能数据

程序编译

1. 执行以下编译命令，将源文件 `gemm_broadcast.scpp` 生成可执行文件 `gemm_broadcast.out`。

```
tecocc gemm_broadcast.scpp -o gemm_broadcast.out
```

2. 执行以下命令，运行程序：

```
./gemm_broadcast.out
```

执行结果

在本示例中展示部分SPE性能数据：

SPE的ID	优化前的计算任务的执行周期（拍）	优化后的计算任务的执行周期（拍）
SPE0	505021	208188
SPE1	543540	208949
SPE2	540672	208949
SPE3	531941	208118
SPE4	545714	208188
SPE5	540365	207964
SPE6	546281	208983
SPE7	524844	207802
SPE8	537730	208802
SPE9	503830	208141

性能优化

优化策略	平均执行周期（拍）
基础实现	104812341117
SIMD	13573489
矩阵乘法单元	531993
广播	208336

使用广播优化策略后的性能，是使用矩阵乘法单元的约2.5倍。

16.4.2.4 双缓冲

16.4.2.4.1 实现原理

本章主要介绍如何在广播的基础上，使用双缓冲优化策略，提升算子性能。

优化原理

主机与设备间的数据传输，可以与设备上Kernel的执行并行，互不影响。这说明：主机端可以通过异步的数据传输接口发起数据传输后，立即发起Kernel的执行操作，实现访存和计算的并行，进一步缩短程序耗时。使用Global存储与SPM存储间的**非阻塞型DMA**接口，可以实现计算和数据传输的并行。

为实现计算和数据传输的并行，两者的数据不能相互依赖，即计算过程的数据不能依赖于同一时刻正在传输的数据。因此，需要在SPM存储空间上，设置至少2个数据缓冲区：

- **计算缓冲区**：存储计算过程依赖的数据。
- **传输缓冲区**：存储异步传输过程依赖的数据。

当计算和数据传输都完成后，再交换两个缓冲区的数据，即可始终确保计算过程与数据传输过程的数据无依赖。

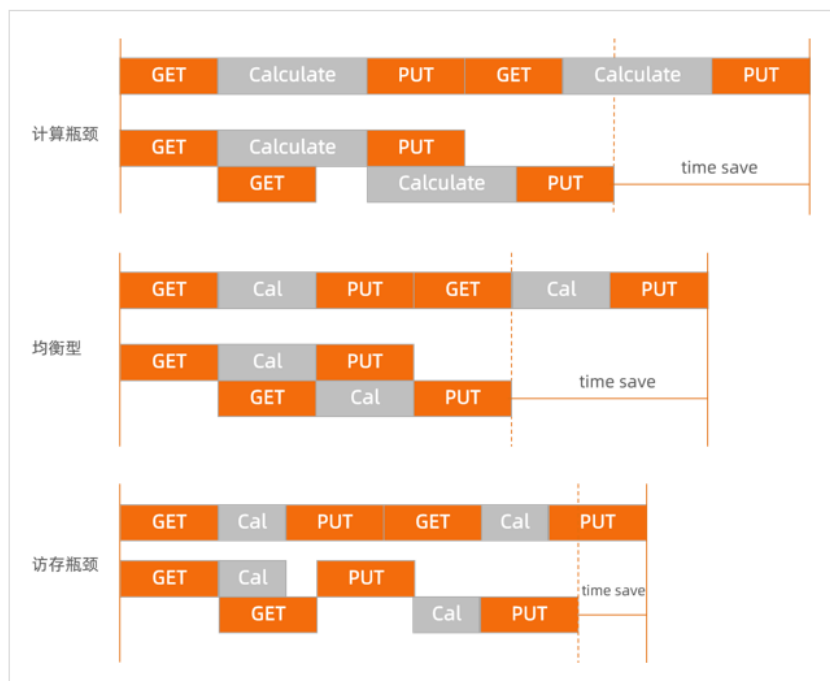


图1 双缓冲的实现原理

代码逻辑

双缓冲通过 `tecoKernelGemmDoubleBufferImpl` 函数实现，该函数在广播的基础上，使用双缓冲进行性能优化。

1. 创建缓冲区，并初始化指针

- 为矩阵 `A`、`B` 和结果矩阵 `C` 的子块分配SPM缓冲区，用于后续的计算、数据暂存等操作。
- 初始化两组指针：
 - `pDA` 和 `pCA` 分别指向步骤 a 分配的矩阵 `A` 的传输缓冲区 `LocalA` 和计算缓冲区 `LocalCompA`。

- `pDB` 和 `pCB` 分别指向步骤 a 分配的矩阵 `B` 的传输缓冲区 `LocalB` 和计算缓冲区 `LocalCompB`。

2. 创建广播句柄，并初始化广播相关的数据结构和变量

调用 `BroadcastHandle` 创建广播句柄，并初始化广播相关的数据结构和变量，包括：行广播线程组、列广播线程组、跨步对象、以及传输缓冲区和计算缓冲区。

3. 初始化矩阵乘接口

调用 `matmul_init` 初始化矩阵乘接口，并配置输入和输出数据的类型。

4. 进行非阻塞型广播

- 计算用于广播的线程索引 `typeAThreads` 和 `typeBThreads`，并创建线程组 `col_thread_group` 和 `row_thread_group` 以及相应的广播句柄 `col_handle` 和 `row_handle`。
- 调用 `sync_threads()` 函数，同步所有相关线程。
- 通过 `broadcast_async` 函数，分别对矩阵 `A` 和 `B` 进行非阻塞型广播操作。
 - 将矩阵 `A` 的数据，按照指定的块大小 `BsizeA` 和线程数量 `typeAThreads`，从Global存储空间广播到对应的缓冲区（`pDA` 指向的缓冲区）。
 - 将矩阵 `B` 的数据，按照指定的块大小 `BsizeB` 和线程数量 `typeBThreads`，从Global存储空间广播到对应的缓冲区（`pDB` 指向的缓冲区）。

5. 执行矩阵乘法计算，并等待相关操作的执行完成

- 进行计算并更新指针：循环遍历所有需要处理的矩阵 `A` 和 `B` 的组合。
 - 在每次外层循环中，先通过 `idx` 计算出当前在 `M`、`N` 维度上的块索引 `index_M` 和 `index_N`。再通过 `index_K` 遍历矩阵在 `K` 维度上的块。
 - 在循环内部，根据当前的块索引 `index_M`、`index_K` 和 `index_N`，更新指向矩阵 `A` 和 `B` 当前块的指针 `pCurrA` 和 `pCurrB`，以及指向下一个块（在 `K` 维度上的下一个位置，或者根据边界情况进行特殊处理后的下一个块位置）的指针 `pNextA` 和 `pNextB`。
- 交换计算缓冲区和传输缓冲区指针：
 - 对于矩阵 `A`：交换 `pCA` 和 `pDA`。
 - 对于矩阵 `B`：交换 `pCB` 和 `pDB`。

6. 等待上一次缓存任务完成

调用 `broadcast_wait` 和 `matmul_wait_loading_input` 等函数，等待矩阵 `A` 的广播操作和矩阵乘法的输入数据加载完成，确保数据准备好后再进行下一步操作。

7. 继续发起下一次访存任务，同时并行矩阵乘计算

在等待当前块的相关操作完成后，使用 `broadcast_async` 函数异步广播矩阵 `A` 和 `B` 的下一个块数据到对应的缓冲区，提前准备好后续计算所需的数据。

8. 执行矩阵乘法计算，并存储计算结果

示例代码如下：

```
__global__ void tecoKernelGemmDoubleBufferImpl(GEMMArgs pGemm)
{
    // 划分线程的行和列
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;
    const int K = pGemm.k;

    // 从 GEMMArgs 结构体中获取矩阵的存储步长
    const int lda = pGemm.lda;
    const int ldb = pGemm.ldb;
    const int ldc = pGemm.ldc;

    // 指向输入矩阵 A、B 和输出矩阵 C 的指针
    const half *A = (const half *)pGemm.A;
    const half *B = (const half *)pGemm.B;
    half *C = (half *)pGemm.C;

    // 外部块的大小
    const int bM = pGemm.bM;
    const int bN = pGemm.bN;
    const int bK = pGemm.bK;

    // 外部块的数量
    int nM = M / bM;
    int nN = N / bN;
    int nK = K / bK;

    // 内部块的大小
    int LocalbM = bM / 8;
    int LocalbN = bN / 4;
    int LocalbK = bK;

    // 内部块的位置
    const half *pA = A + cid * LocalbM * lda;
    const half *pB = B + rid * LocalbN;
    half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

    // A、B、C 的当前块和下一个块的指针
    const half *pCurrA, *pNextA;
    const half *pCurrB, *pNextB;
    half *pCurrC, *pNextC;

    // 计算内存步长
    const int LenA = LocalbM * LocalbK * sizeof(half);
    const int BsizeA = LocalbK * sizeof(half);
    const int StrideA = (lda - LocalbK) * sizeof(half);

    const int LenB = LocalbK * LocalbN * sizeof(half);
    const int BsizeB = LocalbN * sizeof(half);
    const int StrideB = (ldb - LocalbN) * sizeof(half);

    const int LenC = LocalbM * LocalbN * sizeof(half);
```



```

const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);

int spm_size = 2 * LenA + 3 * LenB + LenC + LenC / sizeof(half) * sizeof(float) * 2;

// 为 A、B 的子块和结果 C 分配SPM缓冲区, 包括双缓冲区
half *LocalA = (half *)malloc(LenA * 2);
half *LocalCompA = LocalA + LocalbM * LocalbK;
half *LocalB = (half *)malloc(LenB * 3);
half *LocalCompB = LocalB + LocalbK * LocalbN;
half *LocalTB = LocalB + LocalbK * LocalbN * 2;
half *LocalC = (half *)malloc(LenC);
float *LocalCompC = (float *)malloc(LocalbM * LocalbN * sizeof(float) * 2);
float *tempC = LocalCompC + LocalbM * LocalbN;

// 中间计算的临时变量
half *pTemp;

// 矩阵A、B 访存和计算缓冲区的指针
half *pDA = LocalA, *pCA = LocalCompA;
half *pDB = LocalB, *pCB = LocalCompB;

int typeAThreads = (3 - cid % 4) * 8 + cid; // 列广播
int typeBThreads = rid * 8 + rid;         // 行广播

// 遍历块和块内元素的循环变量
int im, in, ik;
int index_M, index_N, index_K;

// B 置换的准备
floatv16 vtb[32];

// 开始性能采样
perf_start();

// 初始化矩阵乘法操作
MatmulHandle handleMMA;
matmul_init(handleMMA, MatmulHalfToFloat);

// 广播的准备
uint64_t mask = 0;
mask = 0x01010101 << cid;
ThreadGroup col_thread_group(mask);

mask = 0xFF << (8 * rid);
ThreadGroup row_thread_group(mask);

// 创建广播句柄
BroadcastHandle col_handle(&col_thread_group);
BroadcastHandle row_handle(&row_thread_group);
sync_threads();

// 非阻塞广播矩阵 A 和 B
broadcast_async(pDA, (void *)pA, LenA, BsizeA, {0, (size_t)StrideA}, typeAThreads, BroadcastGlobalToSpm, col_handle);
broadcast_async(pDB, (void *)pB, LenB, BsizeB, {0, (size_t)StrideB}, typeBThreads, BroadcastGlobalToSpm, row_handle);

// 遍历矩阵 A 和 B 的块, 计算块积并累加到矩阵 C 中

```

```

for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算 M、N 维度的块索引
    index_M = idx / nN;
    index_N = idx % nN;
    // 遍历 K 维度, 将 A 和 B 的块相乘并将结果累加到 TempC 中
    for (index_K = 0; index_K < nK; ++index_K) {
        // 更新矩阵 A 和 B 的当前块的指针
        pCurrA = pA + index_M * bM * lda + index_K * bK; // A[index_M][index_K]
        pCurrB = pB + index_K * bK * ldb + index_N * bN; // B[index_K][index_N]
        // 定位 pNextA 和 pNextB
        if (index_K < nK - 1) {
            pNextA = pA + index_M * bM * lda + (index_K + 1) * bK; // A[index_M][index_K+1]
            pNextB = pB + (index_K + 1) * bK * ldb + index_N * bN; // B[index_K+1][index_N]
        } else {
            if (index_N == nN - 1) {
                pNextA = pA + (index_M + 1) * bM * lda; // A[index_M+1][0]
                pNextB = pB; // B[0][0]
                if (idx == (nM * nN - 1)) pNextA = pA;
            } else {
                pNextA = pA + index_M * bM * lda; // A[index_M][0]
                pNextB = pB + (index_N + 1) * bN; // B[0][index_N+1]
            }
        }
    }

    // 交换 pCA 和 pDA
    pTemp = pCA;
    pCA = pDA;
    pDA = pTemp;

    // 交换 pCB 和 pDB
    pTemp = pCB;
    pCB = pDB;
    pDB = pTemp;

    // 等待 A 广播和矩阵乘法数据加载
    broadcast_wait(col_handle);
    matmul_wait_loading_input(handleMMA);

    // 同步列线程组内的线程
    sync_threads(col_thread_group);

    // 非阻塞广播矩阵 A 的下一个块
    broadcast_async(pDA, (void *)pNextA, LenA, BsizeA, {0, (size_t)StrideA}, typeAThreads, BroadcastGlobalToSpm,
        col_handle);

    // 对矩阵 B 进行相同操作
    broadcast_wait(row_handle);
    matmul_wait_loading_weight(handleMMA);

    sync_threads(row_thread_group);

    broadcast_async(pDB, (void *)pNextB, LenB, BsizeB, {0, (size_t)StrideB}, typeBThreads, BroadcastGlobalToSpm,
        row_handle);

    // 置换矩阵 B
    for (ik = 0; ik < LocalbK; ik += 8) {
        simd_load(vtb[0], (float *) (pCB + ik * LocalbN));
    }
}

```

```

    simd_load(vtb[1], (float *) (pCB + ik * LocalbN + 32));
    simd_load(vtb[2], (float *) (pCB + (ik + 1) * LocalbN));
    simd_load(vtb[3], (float *) (pCB + (ik + 1) * LocalbN + 32));
    simd_load(vtb[4], (float *) (pCB + (ik + 2) * LocalbN));
    simd_load(vtb[5], (float *) (pCB + (ik + 2) * LocalbN + 32));
    simd_load(vtb[6], (float *) (pCB + (ik + 3) * LocalbN));
    simd_load(vtb[7], (float *) (pCB + (ik + 3) * LocalbN + 32));
    simd_load(vtb[8], (float *) (pCB + (ik + 4) * LocalbN));
    simd_load(vtb[9], (float *) (pCB + (ik + 4) * LocalbN + 32));
    simd_load(vtb[10], (float *) (pCB + (ik + 5) * LocalbN));
    simd_load(vtb[11], (float *) (pCB + (ik + 5) * LocalbN + 32));
    simd_load(vtb[12], (float *) (pCB + (ik + 6) * LocalbN));
    simd_load(vtb[13], (float *) (pCB + (ik + 6) * LocalbN + 32));
    simd_load(vtb[14], (float *) (pCB + (ik + 7) * LocalbN));
    simd_load(vtb[15], (float *) (pCB + (ik + 7) * LocalbN + 32));
    simd_store(vtb[0], (float *) (LocalTB + ik * 32));
    simd_store(vtb[1], (float *) (LocalTB + 32 * LocalbK + ik * 32));
    simd_store(vtb[2], (float *) (LocalTB + (ik + 1) * 32));
    simd_store(vtb[3], (float *) (LocalTB + 32 * LocalbK + (ik + 1) * 32));
    simd_store(vtb[4], (float *) (LocalTB + (ik + 2) * 32));
    simd_store(vtb[5], (float *) (LocalTB + 32 * LocalbK + (ik + 2) * 32));
    simd_store(vtb[6], (float *) (LocalTB + (ik + 3) * 32));
    simd_store(vtb[7], (float *) (LocalTB + 32 * LocalbK + (ik + 3) * 32));
    simd_store(vtb[8], (float *) (LocalTB + (ik + 4) * 32));
    simd_store(vtb[9], (float *) (LocalTB + 32 * LocalbK + (ik + 4) * 32));
    simd_store(vtb[10], (float *) (LocalTB + (ik + 5) * 32));
    simd_store(vtb[11], (float *) (LocalTB + 32 * LocalbK + (ik + 5) * 32));
    simd_store(vtb[12], (float *) (LocalTB + (ik + 6) * 32));
    simd_store(vtb[13], (float *) (LocalTB + 32 * LocalbK + (ik + 6) * 32));
    simd_store(vtb[14], (float *) (LocalTB + (ik + 7) * 32));
    simd_store(vtb[15], (float *) (LocalTB + 32 * LocalbK + (ik + 7) * 32));
}

if (index_K < nK - 1) {
    for (ik = 0; ik + 32 - 1 < LocalbK; ik += 32) {
        // 执行实际的矩阵乘法
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
            MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
} else {
    for (ik = 0; ik + 64 - 1 < LocalbK; ik += 32) {
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA,
            LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,

```

```

        MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    }
    // 最后的矩阵乘法计算并存储结果
    matmul_set_flushing_output(handleMMA, false);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
    matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

    matmul_set_flushing_output(handleMMA, true);
    matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
    matmul_load_weight(handleMMA, LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
        MatmulK32, MatmulN32);
    matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
    matmul_store(handleMMA, LocalCompC, LocalbM * 2, MatmulN32);
}
} // loop index_K

// 等待矩阵乘法操作完成
matmul_wait(handleMMA);

// 将矩阵 C 的已计算部分结果置换
for (im = 0; im < LocalbM; im++) {
    simd_load(vtb[0], LocalCompC + im * 32);
    simd_load(vtb[1], LocalCompC + im * 32 + 16);
    simd_load(vtb[2], LocalCompC + (im + LocalbM) * 32);
    simd_load(vtb[3], LocalCompC + (im + LocalbM) * 32 + 16);
    simd_store(vtb[0], tempC + im * 64);
    simd_store(vtb[1], tempC + im * 64 + 16);
    simd_store(vtb[2], tempC + im * 64 + 32);
    simd_store(vtb[3], tempC + im * 64 + 48);
}

// 在存储结果之前, 将计算结果进行类型转换
batch_S2H(tempC, (half *)LocalC, 4096);

// 将计算得到的块复制回全局内存
memcpy_stride(pC + index_M * bM * ldc + index_N * bN, LocalC, LenC, BsizeC, {(size_t)StrideC,
0});
} // Loop idx

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();

// 释放内存
free(LocalA);
free(LocalB);
free(LocalC);
free(LocalCompC);
}

```

16.4.2.4.2 完整程序

使用双缓冲优化策略的完整程序（gemm_doublebuffer.scpp）：

```
#include <sdaa_matmul.h>
#include <type_traits>
#include <sdaa_perf.h>

using namespace sdaa;

#define REDUITEM 32

// GEMMArgs包含矩阵乘法所需的参数
typedef struct GEMMArgs {
    int m; // 矩阵 A 与矩阵 C 的行数
    int n; // 矩阵 B 与矩阵 C 的列数
    int k; // 矩阵 A 的列数与矩阵 B 的行数
    int lda; // 矩阵 A 的列间距
    int ldb; // 矩阵 B 的列间距
    int ldc; // 矩阵 C 的列间距
    int bM; // 分块矩阵的行数
    int bN; // 分块矩阵的列数
    int bK; // 分块矩阵的中间维度
    const void *A; // 矩阵 A 的指针
    const void *B; // 矩阵 B 的指针
    void *C; // 矩阵 C 的指针
} GEMMArgs;

// 设备函数，将浮点数数组转换为半精度浮点数数组，并根据对齐情况进行处理
__device__ void align_batch_function(float* x, half* y, int len)
{
    // 定义向量变量
    floatv16 vx0, vx1, vx2, vx3, vx4, vx5, vx6, vx7;
    halfv16 vy0, vy1, vy2, vy3, vy4, vy5, vy6, vy7;
    int i = 0;

    // 外层循环，以 128 个元素为一组处理数据
    // 每次循环处理完一组后，指针移动 128 个元素位置继续处理下一组
    for (i = 0; i <= len - 128; i += 128) {
        // 使用 simd_load 函数加载数据
        simd_load(vx0, x);
        simd_load(vx1, x + 16);
        simd_load(vx2, x + 32);
        simd_load(vx3, x + 48);
        simd_load(vx4, x + 64);
        simd_load(vx5, x + 80);
        simd_load(vx6, x + 96);
        simd_load(vx7, x + 112);

        // 使用 simd_cvt_f2h 函数将加载的浮点型向量数据转换为半精度浮点型
        vy0 = simd_cvt_f2h(vx0);
        vy1 = simd_cvt_f2h(vx1);
        vy2 = simd_cvt_f2h(vx2);
        vy3 = simd_cvt_f2h(vx3);
        vy4 = simd_cvt_f2h(vx4);
        vy5 = simd_cvt_f2h(vx5);
        vy6 = simd_cvt_f2h(vx6);
        vy7 = simd_cvt_f2h(vx7);
    }
}
```

```
// 使用 simd_store 函数将转换后的半精度浮点型向量数据存储到内存中
simd_store(vy0, y);
simd_store(vy1, y + 16);
simd_store(vy2, y + 32);
simd_store(vy3, y + 48);
simd_store(vy4, y + 64);
simd_store(vy5, y + 80);
simd_store(vy6, y + 96);
simd_store(vy7, y + 112);

// 更新指针
x += 128;
y += 128;
}

// 中层循环, 处理长度小于 128 但大于或等于 64 的剩余数据
// 每次以 64 个元素为一组进行处理
// 每次循环处理完一组后, 指针移动 64 个元素位置继续处理下一组
for (; i <= len - 64; i += 64) {
    simd_load(vx0, x);
    simd_load(vx1, x + 16);
    simd_load(vx2, x + 32);
    simd_load(vx3, x + 48);

    vy0 = simd_cvt_f2h(vx0);
    vy1 = simd_cvt_f2h(vx1);
    vy2 = simd_cvt_f2h(vx2);
    vy3 = simd_cvt_f2h(vx3);

    simd_store(vy0, y);
    simd_store(vy1, y + 16);
    simd_store(vy2, y + 32);
    simd_store(vy3, y + 48);

    x += 64;
    y += 64;
}

// 内层循环, 处理长度小于 64 但大于或等于 16 的剩余数据
// 每次以 16 个元素为一组进行处理
for (; i <= len - 16; i += 16) {
    simd_load(vx0, x);
    vy0 = simd_cvt_f2h(vx0);
    simd_store(vy0, y);

    x += 16;
    y += 16;
}

// 最内层循环, 处理长度小于 16 的剩余数据
// 逐元素进行赋值操作
for (; i < len; i++) {
    *y = *x;
    x++;
    y++;
}
}

// 根据长度和对齐情况调用 align_batch_function 进行浮点数到半精度浮点数的转换
```

```

__device__ void batch_S2H(float* x, half* y, int len)
{
    if (len < 32) {
        for (int i = 0; i < len; i++) {
            y[i] = x[i];
        }
        return;
    }
    const int align_n1 = sizeof(floatv16);
    const int align_n2 = sizeof(halfv16);
    if (((size_t)x & (align_n1 - 1)) == 0) && (((size_t)y & (align_n2 - 1)) == 0) {
        align_batch_function((float*)x, y, len);
        return;
    } else {
        int padding_x = ((size_t)(x) & (align_n1 - 1)) / sizeof(float);
        int padding_y = ((size_t)(y) & (align_n2 - 1)) / sizeof(half);
        if (padding_x == padding_y) {
            for (int i = 0; i < 16 - padding_x; i++) {
                y[i] = x[i];
            }
            align_batch_function(x + 16 - padding_x, y + 16 - padding_y, len - 16 + padding_x);
            return;
        } else {
            align_batch_function((float*)x, y, len);
            return;
        }
    }
}

__global__ void tecoKernelGemmDoubleBufferImpl(GEMMArgs pGemm)
{
    // 划分线程的行和列
    const int rid = threadIdx / 8;
    const int cid = threadIdx % 8;

    // 从 GEMMArgs 结构体中获取矩阵的维度信息
    const int M = pGemm.m;
    const int N = pGemm.n;
    const int K = pGemm.k;

    // 从 GEMMArgs 结构体中获取矩阵的存储步长
    const int lda = pGemm.lda;
    const int ldb = pGemm.ldb;
    const int ldc = pGemm.ldc;

    // 指向输入矩阵 A、B 和输出矩阵 C 的指针
    const half *A = (const half *)pGemm.A;
    const half *B = (const half *)pGemm.B;
    half *C = (half *)pGemm.C;

    // 外部块的大小
    const int bM = pGemm.bM;
    const int bN = pGemm.bN;
    const int bK = pGemm.bK;

    // 外部块的数量
    int nM = M / bM;
    int nN = N / bN;
    int nK = K / bK;

```

```

// 内部块的大小
int LocalbM = bM / 8;
int LocalbN = bN / 4;
int LocalbK = bK;

// 内部块的位置
const half *pA = A + cid * LocalbM * lda;
const half *pB = B + rid * LocalbN;
half *pC = C + cid * LocalbM * ldc + rid * LocalbN;

// A、B、C 的当前块和下一个块的指针
const half *pCurrA, *pNextA;
const half *pCurrB, *pNextB;
half *pCurrC, *pNextC;

// 计算内存步长
const int LenA = LocalbM * LocalbK * sizeof(half);
const int BsizeA = LocalbK * sizeof(half);
const int StrideA = (lda - LocalbK) * sizeof(half);

const int LenB = LocalbK * LocalbN * sizeof(half);
const int BsizeB = LocalbN * sizeof(half);
const int StrideB = (ldb - LocalbN) * sizeof(half);

const int LenC = LocalbM * LocalbN * sizeof(half);
const int BsizeC = LocalbN * sizeof(half);
const int StrideC = (ldc - LocalbN) * sizeof(half);

int spm_size = 2 * LenA + 3 * LenB + LenC + LenC / sizeof(half) * sizeof(float) * 2;

// 为 A、B 的子块和结果 C 分配SPM缓冲区, 包括双缓冲区
half *LocalA = (half *)malloc(LenA * 2);
half *LocalCompA = LocalA + LocalbM * LocalbK;
half *LocalB = (half *)malloc(LenB * 3);
half *LocalCompB = LocalB + LocalbK * LocalbN;
half *LocalTB = LocalB + LocalbK * LocalbN * 2;
half *LocalC = (half *)malloc(LenC);
float *LocalCompC = (float *)malloc(LocalbM * LocalbN * sizeof(float) * 2);
float *tempC = LocalCompC + LocalbM * LocalbN;

// 中间计算的临时变量
half *pTemp;

// 矩阵A、B 访存和计算缓冲区的指针
half *pDA = LocalA, *pCA = LocalCompA;
half *pDB = LocalB, *pCB = LocalCompB;

int typeAThreads = (3 - cid % 4) * 8 + cid; // 列广播
int typeBThreads = rid * 8 + rid;         // 行广播

// 遍历块和块内元素的循环变量
int im, in, ik;
int index_M, index_N, index_K;

// B 置换的准备
floatv16 vtb[32];

// 开始性能采样

```



```

perf_start();

// 初始化矩阵乘法操作
MatmulHandle handleMMA;
matmul_init(handleMMA, MatmulHalfToFloat);

// 广播的准备
uint64_t mask = 0;
mask = 0x01010101 << cid;
ThreadGroup col_thread_group(mask);

mask = 0xFF << (8 * rid);
ThreadGroup row_thread_group(mask);

// 创建广播句柄
BroadcastHandle col_handle(&col_thread_group);
BroadcastHandle row_handle(&row_thread_group);
sync_threads();

// 非阻塞广播矩阵 A 和 B
broadcast_async(pDA, (void *)pA, LenA, BsizeA, {0, (size_t)StrideA}, typeAThreads, BroadcastGlobalToSpm, col_handle);
broadcast_async(pDB, (void *)pB, LenB, BsizeB, {0, (size_t)StrideB}, typeBThreads, BroadcastGlobalToSpm, row_handle);

// 遍历矩阵 A 和 B 的块, 计算块积并累加到矩阵 C 中
for (int idx = 0; idx < nM * nN; ++idx) {
    // 计算 M、N 维度的块索引
    index_M = idx / nN;
    index_N = idx % nN;
    // 遍历 K 维度, 将 A 和 B 的块相乘并将结果累加到 TempC 中
    for (index_K = 0; index_K < nK; ++index_K) {
        // 更新矩阵 A 和 B 的当前块的指针
        pCurrA = pA + index_M * bM * lda + index_K * bK; // A[index_M][index_K]
        pCurrB = pB + index_K * bK * ldb + index_N * bN; // B[index_K][index_N]
        // 定位 pNextA 和 pNextB
        if (index_K < nK - 1) {
            pNextA = pA + index_M * bM * lda + (index_K + 1) * bK; // A[index_M][index_K+1]
            pNextB = pB + (index_K + 1) * bK * ldb + index_N * bN; // B[index_K+1][index_N]
        } else {
            if (index_N == nN - 1) {
                pNextA = pA + (index_M + 1) * bM * lda; // A[index_M+1][0]
                pNextB = pB; // B[0][0]
                if (idx == (nM * nN - 1)) pNextA = pA;
            } else {
                pNextA = pA + index_M * bM * lda; // A[index_M][0]
                pNextB = pB + (index_N + 1) * bN; // B[0][index_N+1]
            }
        }
    }

    // 交换 pCA 和 pDA
    pTemp = pCA;
    pCA = pDA;
    pDA = pTemp;

    // 交换 pCB 和 pDB
    pTemp = pCB;
    pCB = pDB;
    pDB = pTemp;
}

```

```

// 等待 A 广播和矩阵乘法数据加载
broadcast_wait(col_handle);
matmul_wait_loading_input(handleMMA);

// 同步列线程组内的线程
sync_threads(col_thread_group);

// 非阻塞广播矩阵 A 的下一个块
broadcast_async(pDA, (void *)pNextA, LenA, BsizeA, {0, (size_t)StrideA}, typeAThreads, Broa
dcastGlobalToSpm,
                col_handle);

// 对矩阵 B 进行相同操作
broadcast_wait(row_handle);
matmul_wait_loading_weight(handleMMA);

sync_threads(row_thread_group);

broadcast_async(pDB, (void *)pNextB, LenB, BsizeB, {0, (size_t)StrideB}, typeBThreads, Broa
dcastGlobalToSpm,
                row_handle);

// 置换矩阵 B
for (ik = 0; ik < LocalbK; ik += 8) {
    simd_load(vtb[0], (float *) (pCB + ik * LocalbN));
    simd_load(vtb[1], (float *) (pCB + ik * LocalbN + 32));
    simd_load(vtb[2], (float *) (pCB + (ik + 1) * LocalbN));
    simd_load(vtb[3], (float *) (pCB + (ik + 1) * LocalbN + 32));
    simd_load(vtb[4], (float *) (pCB + (ik + 2) * LocalbN));
    simd_load(vtb[5], (float *) (pCB + (ik + 2) * LocalbN + 32));
    simd_load(vtb[6], (float *) (pCB + (ik + 3) * LocalbN));
    simd_load(vtb[7], (float *) (pCB + (ik + 3) * LocalbN + 32));
    simd_load(vtb[8], (float *) (pCB + (ik + 4) * LocalbN));
    simd_load(vtb[9], (float *) (pCB + (ik + 4) * LocalbN + 32));
    simd_load(vtb[10], (float *) (pCB + (ik + 5) * LocalbN));
    simd_load(vtb[11], (float *) (pCB + (ik + 5) * LocalbN + 32));
    simd_load(vtb[12], (float *) (pCB + (ik + 6) * LocalbN));
    simd_load(vtb[13], (float *) (pCB + (ik + 6) * LocalbN + 32));
    simd_load(vtb[14], (float *) (pCB + (ik + 7) * LocalbN));
    simd_load(vtb[15], (float *) (pCB + (ik + 7) * LocalbN + 32));
    simd_store(vtb[0], (float *) (LocalTB + ik * 32));
    simd_store(vtb[1], (float *) (LocalTB + 32 * LocalbK + ik * 32));
    simd_store(vtb[2], (float *) (LocalTB + (ik + 1) * 32));
    simd_store(vtb[3], (float *) (LocalTB + 32 * LocalbK + (ik + 1) * 32));
    simd_store(vtb[4], (float *) (LocalTB + (ik + 2) * 32));
    simd_store(vtb[5], (float *) (LocalTB + 32 * LocalbK + (ik + 2) * 32));
    simd_store(vtb[6], (float *) (LocalTB + (ik + 3) * 32));
    simd_store(vtb[7], (float *) (LocalTB + 32 * LocalbK + (ik + 3) * 32));
    simd_store(vtb[8], (float *) (LocalTB + (ik + 4) * 32));
    simd_store(vtb[9], (float *) (LocalTB + 32 * LocalbK + (ik + 4) * 32));
    simd_store(vtb[10], (float *) (LocalTB + (ik + 5) * 32));
    simd_store(vtb[11], (float *) (LocalTB + 32 * LocalbK + (ik + 5) * 32));
    simd_store(vtb[12], (float *) (LocalTB + (ik + 6) * 32));
    simd_store(vtb[13], (float *) (LocalTB + 32 * LocalbK + (ik + 6) * 32));
    simd_store(vtb[14], (float *) (LocalTB + (ik + 7) * 32));
    simd_store(vtb[15], (float *) (LocalTB + 32 * LocalbK + (ik + 7) * 32));
}

```

```

    if (index_K < nK - 1) {
        for (ik = 0; ik + 32 - 1 < LocalbK; ik += 32) {
            // 执行实际的矩阵乘法
            matmul_set_flushing_output(handleMMA, false);
            matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
            matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
            matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

            matmul_set_flushing_output(handleMMA, false);
            matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
            matmul_load_weight(handleMMA,
                               LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
                               MatmulK32, MatmulN32);
            matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
        }
    } else {
        for (ik = 0; ik + 64 - 1 < LocalbK; ik += 32) {
            matmul_set_flushing_output(handleMMA, false);
            matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
            matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
            matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

            matmul_set_flushing_output(handleMMA, false);
            matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
            matmul_load_weight(handleMMA,
                               LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
                               MatmulK32, MatmulN32);
            matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
        }
        // 最后的矩阵乘法计算并存储结果
        matmul_set_flushing_output(handleMMA, false);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, 0);
        matmul_load_weight(handleMMA, LocalTB + ik * LocalbN / 2, MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);

        matmul_set_flushing_output(handleMMA, true);
        matmul_set_output_row_offset(handleMMA, MatmulEnableOutputRowOffset, LocalbM);
        matmul_load_weight(handleMMA, LocalTB + LocalbN * LocalbK / 2 + ik * LocalbN / 2,
                           MatmulK32, MatmulN32);
        matmul_compute(handleMMA, pCA + ik, LocalbM, MatmulK32, LocalbK / REDUITEM - 1);
        matmul_store(handleMMA, LocalCompC, LocalbM * 2, MatmulN32);
    }
} // loop index_K

// 等待矩阵乘法操作完成
matmul_wait(handleMMA);

// 将矩阵 C 的已计算部分结果置换
for (im = 0; im < LocalbM; im++) {
    simd_load(vtb[0], LocalCompC + im * 32);
    simd_load(vtb[1], LocalCompC + im * 32 + 16);
    simd_load(vtb[2], LocalCompC + (im + LocalbM) * 32);
    simd_load(vtb[3], LocalCompC + (im + LocalbM) * 32 + 16);
    simd_store(vtb[0], tempC + im * 64);
    simd_store(vtb[1], tempC + im * 64 + 16);
    simd_store(vtb[2], tempC + im * 64 + 32);
    simd_store(vtb[3], tempC + im * 64 + 48);
}

```

```
// 在存储结果之前，将计算结果进行类型转换
batch_S2H(tempC, (half *)LocalC, 4096);

// 将计算得到的块复制回全局内存
memcpy_stride(pC + index_M * bM * ldc + index_N * bN, LocalC, LenC, BsizeC, {(size_t)StrideC,
0});
} // Loop idx

// 性能采样
perf_stop();
// 打印出性能采样结果
perf_print();

// 释放内存
free(LocalA);
free(LocalB);
free(LocalC);
free(LocalCompC);
}

int main()
{
    sdaaSetDevice(0);

    GEMMArgs args;

    // 设置矩阵维度参数
    args.m = 1024;
    args.n = 256;
    args.k = 1024;

    args.lda = args.k;
    args.ldb = args.n;
    args.ldc = args.n;
    args.bM = 512;
    args.bN = 256;
    args.bK = 256;

    // 在主机上为矩阵 A、B 和 C 分配内存
    half *h_A = (half *)malloc(args.m * args.k * sizeof(half));
    half *h_B = (half *)malloc(args.k * args.n * sizeof(half));
    half *h_C = (half *)malloc(args.m * args.n * sizeof(half));

    // 为矩阵 A、B 和 C 设置初始值
    for (size_t i = 0; i < args.m * args.k; ++i) {
        h_A[i] = static_cast<half>(1);
    }

    for (size_t i = 0; i < args.k * args.n; ++i) {
        h_B[i] = static_cast<half>(1);
    }

    for (size_t i = 0; i < args.m * args.n; ++i) {
        h_C[i] = static_cast<half>(0);
    }

    // 在设备上为矩阵 A、B 和 C 分配内存
    half *d_A, *d_B, *d_C;
```

```
sdaaMalloc((void **>(&d_A), args.m * args.k * sizeof(half));
sdaaMalloc((void **>(&d_B), args.k * args.n * sizeof(half));
sdaaMalloc((void **>(&d_C), args.m * args.n * sizeof(half));

sdaaMemcpy(d_A, h_A, args.m * args.k * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_B, h_B, args.k * args.n * sizeof(half), sdaaMemcpyHostToDevice);
sdaaMemcpy(d_C, h_C, args.m * args.n * sizeof(half), sdaaMemcpyHostToDevice);

args.A = d_A;
args.B = d_B;
args.C = d_C;

tecoKernelGemmDoubleBufferImpl<<1>>>(args);

sdaaDeviceSynchronize();

// 将结果矩阵 C 从设备内存复制到主机内存
sdaaMemcpy(h_C, d_C, args.m * args.n * sizeof(half), sdaaMemcpyDeviceToHost);

// 释放设备端内存
sdaaFree(d_A);
sdaaFree(d_B);
sdaaFree(d_C);

// 释放主机端内存
free(h_A);
free(h_B);
free(h_C);

return 0;
}
```

16.4.2.4.3 性能数据

程序编译

1. 执行以下编译命令，将源文件 `gemm_doublebuffer.scpp` 生成可执行文件 `gemm_doublebuffer.out`。

```
tecocc gemm_doublebuffer.scpp -o gemm_doublebuffer.out
```

2. 执行以下命令，运行程序：

```
./gemm_doublebuffer.out
```

执行结果

在本示例中展示部分SPE性能数据：

SPE的ID	优化前的计算任务的执行周期（拍）	优化后的计算任务的执行周期（拍）
SPE0	208188	146487
SPE1	208949	147221
SPE2	208949	147817
SPE3	208118	147636
SPE4	208188	147640
SPE5	207964	147725
SPE6	208983	147722
SPE7	207802	147169
SPE8	208802	147081
SPE9	208141	148512

性能优化

优化策略	平均执行周期（拍）
基础实现	104812341117
SIMD	13573489
矩阵乘法单元	531993
广播	208336
双缓冲	147501

使用双缓冲优化策略后的性能，是使用广播的约1.4倍，是基础实现的710587倍。

17. 常见问题

17.1 如何使用CMake构建？

为使CMake可以构建后缀为.scpp的文件，需要先使用 `set_source_files_properties` 命令，设置 `LINKER_LANGUAGE` 的属性为 `CXX`。CMake构建分为LTO模式和非LTO模式。

- **LTO模式**：在生成动态库或者可执行文件时，用到的所有.scpp文件都在同一个编译模块里，即：一个.scpp里定义的变量和函数，可以在另一个.scpp文件里使用。
- **非LTO模式**：不同的.scpp文件是不同的编译模块。在不同编译模块中的设备端代码，相互之间没有依赖关系，即：在一个编译模块里定义的变量和函数，在另一个编译模块里不能对其进行访问。

17.1.1 LTO模式

17.1.1.1 示例代码

- main2.scpp使用main1.scpp定义的 `__device__ void print()`。
- main.cpp链接生成的静态库或者动态库，生成可执行文件。

```
//main1.scpp
#include <stdio.h>
#include <stdlib.h>

__device__ void print()
{
    printf("this is device function in %s\n", __FILE__);
}

__global__ void test1(int a, double b)
{
    print();
    printf("this is %s!\n", __FILE__);
    printf("a = %d, b = %lf\n", a, b);
    printf("\n");
}

int main1()
{
    sdAASetDevice(0);

    test1<<<1>>>(1, 3.0);

    sdAADeviceSynchronize();

    return 0;
}
```

```
//main2.scpp
#include <stdio.h>
#include <stdlib.h>

extern __device__ void print();

typedef struct {
    int b;
    int c;
} st1;
st1 a = {1, 2};

__global__ void test2(int a, double b, st1 c)
{
    print();
    printf("this is %s!\n", __FILE__);
    printf("a = %d, b = %lf, c.b = %d, c.c = %d\n", a, b, c.b, c.c);
    printf("\n");
}

int main2()
{
    sdaaSetDevice(0);

    test2<<<1>>>(66, 8.0, a);

    sdaaDeviceSynchronize();

    return 0;
}
```

```
//main.cpp
extern int main1();
extern int main2();

int main()
{
    main1();
    main2();
}
```

17.1.1.2 静态库

1. 编写CMakeLists.txt文件。

 说明:

1. 指定add_library为OBJECT类型。
2. 调用add_custom_target自定义target生成静态库，需要使用-flto --sdaa-link -shared -sdaa-static-lib -fuse-ld=lld链接选项。

```
cmake_minimum_required(VERSION 3.13)

project(lto-static-lib)

#设置编译器为TecoCC
set(CMAKE_CXX_COMPILER "tecocc")

#libtmp
#步骤1: 生成objs
file(GLOB_RECURSE SRC_TMP "*.scpp")
add_library(tmp_objs OBJECT ${SRC_TMP})
set_source_files_properties(${SRC_TMP} PROPERTIES LANGUAGE CXX)
target_compile_options(tmp_objs PRIVATE -flto)

#步骤2: 生成lib
#ALL, 默认构建libtmp, 否则可能只生成objs
add_custom_target(tmp ALL
    COMMAND tecocc $<TARGET_OBJECTS:tmp_objs> -flto --sdaa-link -shared -sdaa-static-lib -fuse-ld=lld
    -o libtmp.a
    COMMAND_EXPAND_LISTS
)
add_dependencies(tmp tmp_objs)

#生成可执行文件, 链接libtmp
#tmp是custom target, 通过add_dependencies添加依赖
add_executable(static_test main.cpp)
add_dependencies(static_test tmp)
target_link_libraries(static_test PRIVATE -L. -ltmp -fuse-ld=lld)
```

2. 执行以下命令，创建并进入build目录。

```
mkdir build && cd build
```

3. 执行以下命令，生成Makefile文件。

```
cmake ..
```

4. 执行以下命令，生成可执行文件。

```
make -j2
```

17.1.1.3 动态库

1. 编写CMakeLists.txt文件。

 说明：

1. 指定add_library为OBJECT类型。
2. 调用add_custom_target自定义target生成动态库，需要使用-flto --sdaa-link -shared -fuse-ld=lld链接选项。

```
cmake_minimum_required(VERSION 3.13)

project(lto-dynamic-lib)

#设置编译器为TecoCC
set(CMAKE_CXX_COMPILER "tecocc")

#libtmp
# 步骤1: 生成objs
file(GLOB_RECURSE SRC_TMP "*.scpp")
add_library(tmp_objs OBJECT ${SRC_TMP})
set_source_files_properties(${SRC_TMP} PROPERTIES LANGUAGE CXX)
target_compile_options(tmp_objs PRIVATE -flto)

# 步骤2: 生成lib
# ALL, 默认构建libtmp, 否则可能只生成objs
add_custom_target(tmp ALL
    COMMAND tecocc $<TARGET_OBJECTS:tmp_objs> -flto --sdaa-link -shared -fuse-ld=lld
    -o libtmp.so
    COMMAND_EXPAND_LISTS
)
add_dependencies(tmp tmp_objs)

#生成可执行文件, 链接libtmp
add_executable(dynamic_test main.cpp)
add_dependencies(dynamic_test tmp)
target_link_libraries(dynamic_test PRIVATE -L. -ltmp -fuse-ld=lld)
```

2. 执行以下命令，创建并进入build目录。

```
mkdir build && cd build
```

3. 执行以下命令，生成Makefile文件。

```
cmake ..
```

4. 执行以下命令，生成可执行文件。

```
make -j2
```

17.1.2 非LTO模式

17.1.2.1 示例代码

- main1.scpp和main2.scpp是两个独立的模块，没有依赖。
- main.cpp链接生成的静态库或者动态库，生成可执行文件。

```
// main1.scpp
#include <stdio.h>
#include <stdlib.h>

__device__ void print()
{
    printf("this is device function in %s\n", __FILE__);
}

__global__ void test1(int a, double b)
{
    print();
    printf("this is %s!\n", __FILE__);
    printf("a = %d, b = %lf\n", a, b);
    printf("\n");
}

int main1()
{
    sdAASetDevice(0);
    test1<<<1>>>(1, 3.0);
    sdAADeviceSynchronize();

    return 0;
}
```

```
// main2.scpp
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int b;
    int c;
} st1;
st1 a = {1, 2};

__global__ void test2(int a, double b, st1 c)
{
    printf("this is %s!\n", __FILE__);
    printf("a = %d, b = %lf, c.b = %d, c.c = %d\n", a, b, c.b, c.c);
    printf("\n");
}
```

```
int main2()
{
    sdaaSetDevice(0);
    test2<<<1>>>(66, 8.0, a);
    sdaaDeviceSynchronize();

    return 0;
}
```

```
// main.cpp
extern int main1();
extern int main2();

int main()
{
    main1();
    main2();
}
```

17.1.2.2 静态库

1. 编写CMakeLists.txt文件。

```
cmake_minimum_required(VERSION 3.13)

project (static-lib)

#设置编译器为TecoCC
set(CMAKE_CXX_COMPILER "tecocc")

#libtmp
add_library(tmp STATIC
    main1.scpp
    main2.scpp
)

set_source_files_properties(main1.scpp main2.scpp PROPERTIES LANGUAGE CXX)
target_compile_options(tmp PRIVATE -fPIC)

#生成可执行文件，链接libtmp
add_executable(static_test main.cpp)
target_link_libraries(static_test tmp)
target_link_options(static_test PUBLIC -L$ENV{SDAA_HOME}/lib64 -lsdaart -lsddev)
```

2. 执行以下命令，创建并进入build目录。

```
mkdir build && cd build
```

3. 执行以下命令，生成Makefile文件。

```
cmake ..
```

4. 执行以下命令，生成可执行文件。

```
make -j2
```

17.1.2.3 动态库

1. 编写CMakeLists.txt文件。

 说明：

修改add_library为SHARED。

```
cmake_minimum_required(VERSION 3.13)

project(dynamic-lib)

#设置编译器为TecoCC
set(CMAKE_CXX_COMPILER "tecocc")

#libtmp, 动态库
add_library(tmp SHARED
    main1.scpp
    main2.scpp
)

set_source_files_properties(main1.scpp main2.scpp PROPERTIES LANGUAGE CXX)
target_compile_options(tmp PRIVATE -fPIC)

#生成可执行文件，链接libtmp
add_executable(dynamic_test main.cpp)
target_link_libraries(dynamic_test tmp)
target_link_options(dynamic_test PUBLIC -L$ENV{SDAA_HOME}/lib64 -lsdaart -lsddev)
```

2. 执行以下命令，创建并进入build目录。

```
mkdir build && cd build
```

3. 执行以下命令，生成Makefile文件。

```
cmake ..
```

4. 执行以下命令，生成可执行文件。

```
make -j2
```

17.2 链接LTO模式编译的库报错，libxxx.a: archive has no index; run ranlib to add one，如何解决？

根因分析

通常是使用AR工具将目标文件打包成.a静态库文件libxxx.a所导致。

解决方案

- 如果静态库中只包含Device端代码：
 - 编译.o文件时，增加 `--sdaa-device-only` 选项。
 - 链接libxxx.a时，增加 `--sdaa-static-device-lib=libxxx.a` 选项。
- 如果同时包含Device端和Host端的代码：不使用AR工具将目标文件打包成.a静态库文件libxxx.a，推荐用法参考 [LTO模式静态库](#)。

17.3 CUDA程序迁移

17.3.1 如何解决线程编号不对应？

问题描述

CUDA编程时，线程编号是通过gridDim、blockIdx、blockDim、threadIdx确定的，但在SDAA C编程中只有threadDim和threadIdx，两者该如何进行对应？

例如：以下代码中每次i相加的步长该如何进行对应？

```
for (size_t i = blockIdx.x * blockDim.x + threadIdx.x; i < (size); i += blockDim.x * gridDim.x)
```

解决方案

CUDA编程模型中线程层级为grid-block-thread，SDAA C在单个计算核心阵列SPA中只使用thread粒度，对应计算核心SPE，简化并行层级。

在单SPA编程中，对线程的标识为预定义变量threadIdx。通过将任务划分为子任务，每个线程在运行相同程序时执行不同子任务，此即SPMD（Single Program Multiple Data，单程序多数据）编程范式。

以上CUDA程序改写为SDAA C程序如下：

```
for (size_t i = threadIdx; i < (size); i += threadDim)
```

17.3.2 如何解决调用Kernel函数不对应？

问题描述

CUDA调用Kernel函数需要指定block id和thread id，但使用SDAA C调用Kernel函数只有<<<placeholder, stream>>>，两者如何对应？

例如：以下代码如何修改为SDAA C的程序？

```
MomentumUpdateVariableKernel<<<GET_BLOCKS(size), GET_THREADS, 0, cuda_stream>>>
```

解决方案

SDAA C中Kernel函数调用单计算核心阵列SPA，分配固定的硬件资源，即会同时调用同一个SPA内的threadDim个计算核心SPE进行计算，相当于同时启动threadDim个线程。

以上CUDA程序修改为SDAA C程序时，保留 placeholder 和 stream 参数即可：

```
MomentumUpdateVariableKernel<<<0, sdAA_stream>>>
```

17.3.3 SDAA C中是否有与CUDA对应的线程束Warp？

问题描述

使用CUDA进行编程时，有线程束Warp的概念，SDAA C中是否有与之对应的概念？

例如：在以下CUDA程序中，改写为SDAA C程序时，warp_start_row该如何处理？

```
int shm_row_id = (threadIdx >> 5);
int shm_col_id = (threadIdx % 32);
int block_start_row = blockIdx.x * rows_per_block;
int block_end_row = block_start_row + rows_per_block;
block_end_row = block_end_row < m ? block_end_row : m;
int warp_start_row = blockIdx.x * rows_per_block + shm_row_id * rows_per_warp;
int warp_end_row = warp_start_row + rows_per_warp;
int real_rows_per_warp = warp_end_row < block_end_row ? rows_per_warp : block_end_row - warp_start_row;
```

解决方案

SIMT的架构是以Warp为单位，可以由一个指令一次驱动32个thread，并且多Blocks、多Warps可以并行计算。

由于SDAA C使用SPMD架构，与SIMT架构并不相同。在SDAA C中，没有Block和Warp的相关概念。

在SDAA C中，一个计算核心阵列SPA内有threadDim个计算核心SPE（类比CUDA的Thread）是并行计算的。可以使用threadidx，获取Device端计算核心SPE的ID，为每一个计算核心SPE定制不同的行为。

17.3.4 如何在Kernel函数中调用数学函数接口？

数学函数不需要#include相应的头文件即可使用。

数学函数的接口信息，可参考[数学函数](#)的详细内容；数学库的使用示例，可参考[接口使用示例](#)。

17.3.5 CUDA的blockId如何修改为SDAA C代码？

SDAA C没有Grid, Block等概念。

在SDAA C中，启动Kernel函数时，默认会启动`threadDim`个计算核心，您可以通过`threadIdx`获取当前正在执行任务的计算核心SPE的ID，并为每个计算核心SPE定制不同的行为。

17.3.6 CUDA的share memory如何修改为SDAA C代码？

SDAA C中没有提供share memory，可以用的存储空间包括Global存储空间和SPM存储空间：

- Global存储空间，是计算核心阵列SPA的共享存储。
- SPM存储空间，是计算核心SPE的私有空间。

对于计算核心SPE而言，直接访问Global存储空间，速度缓慢。在SPE执行计算任务的过程中，如果从SPM存储空间获取数据，计算效率将得到显著提升。

您可以通过[RMA数据搬运](#)实现同一计算核心阵列SPA内两个不同计算核心SPE的SPM存储之间的数据流动。

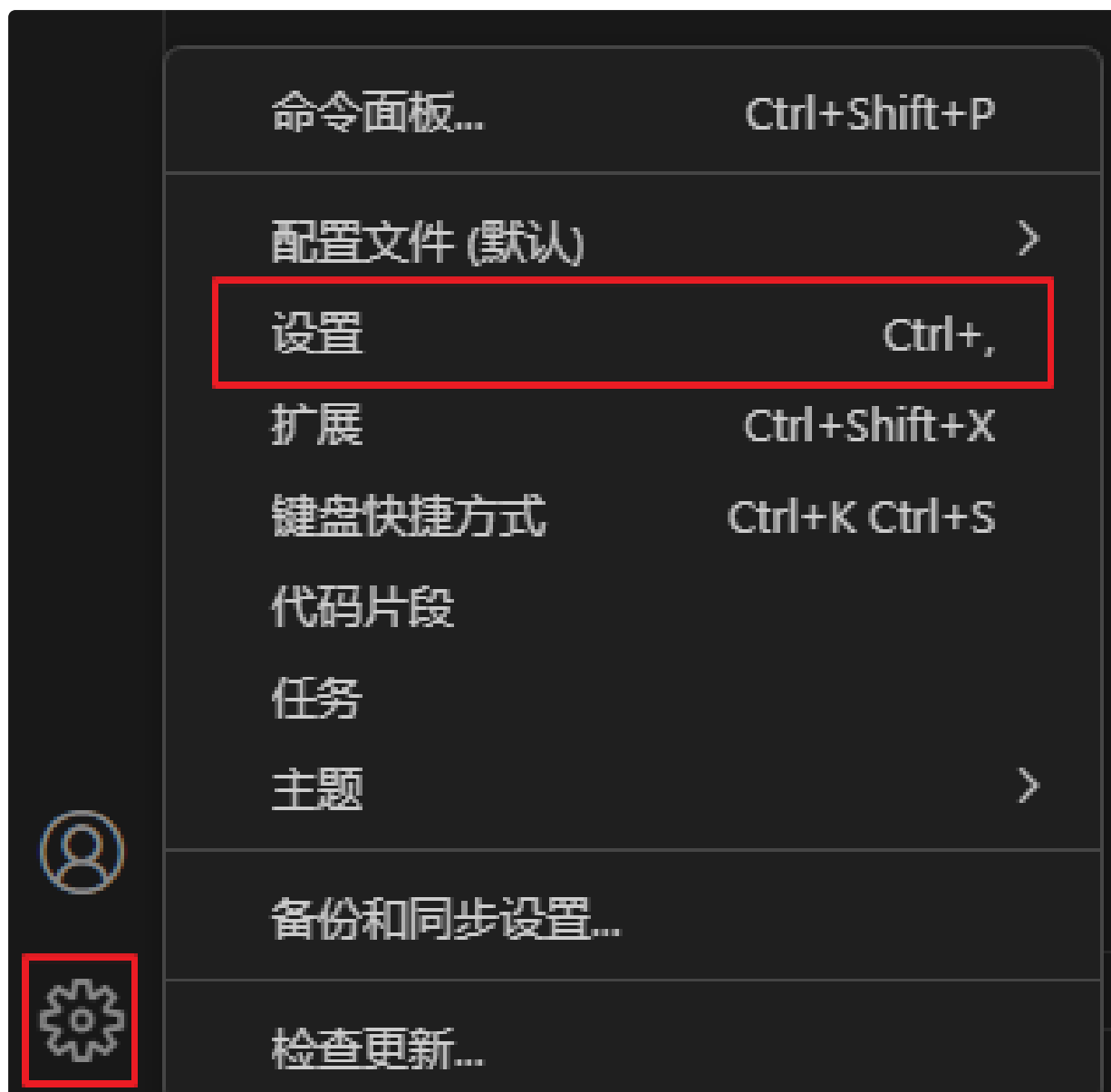
17.4 使用VSCode打开SDAA C程序时，如何实现代码高亮、代码自动补全和代码高亮提醒等功能？

问题描述

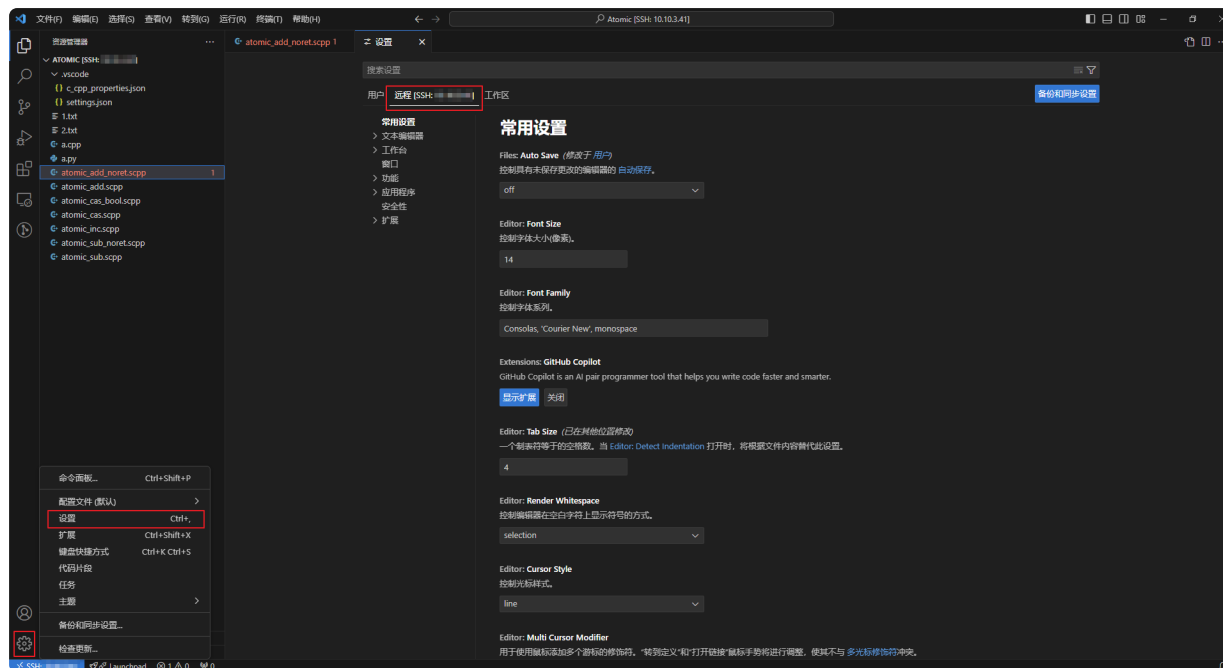
使用VSCode打开SDAA C程序时，无法使用代码高亮、代码自动补全和代码高亮提醒等功能。

解决方案

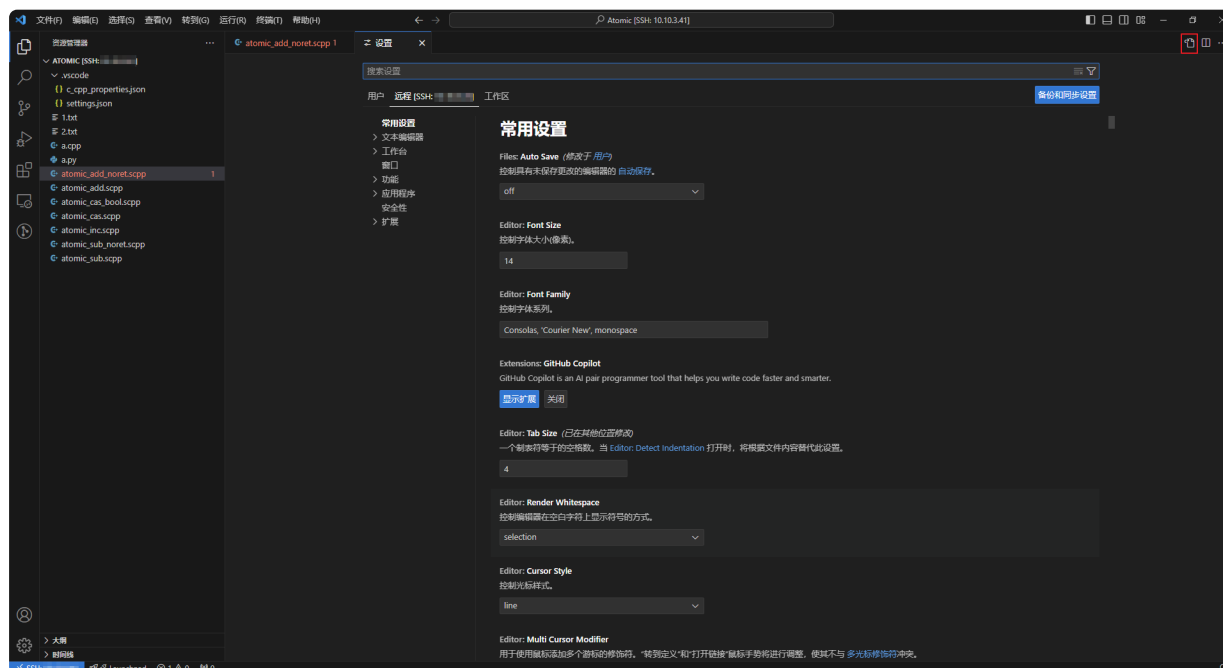
1. 单击页面左下角的 > 设置。



2. 在弹出的页面中，选择目标远程端口页签。



3. 单击页签右上角的 .



4. 在 `setting.json` 文件中，插入以下字段。

```
"files.associations": {
  "scpp": "cpp"
},
"C_Cpp.default.forcedInclude": [
  "/opt/tecoai/extras/llvm/lib/clang/11.0.0/include/__clang_sdaa_runtime_wrapper.h",
],
"C_Cpp.default.includePath": [
  "/opt/tecoai/include/",
  "/opt/tecoai/extras/llvm/lib/clang/11.0.0/include/",
],
"C_Cpp.default.defines": [
```

```
    "__SDAA__",  
    "__clang__"  
],
```

5. 保存退出。

执行结果

使用VSCode打开SDAA C程序时，可以正常使用代码高亮、代码自动补全和代码高亮提醒等功能。

```
33 // 2. 通过SDAA 寄存器上的原子操作进行原子加操作，无返回结果。  
34 // 3. 将原子操作的结果存储在dev_data，返回给host。  
35  
36 #include "sdaa_runtime.h"  
37 #include "sdaa_atomic.h"  
38  
39 using namespace sdaa;  
40 #define DSIZE (1)  
41  
42  
43 __global__ void atomic_op(int64_t *a)  
44 {  
45     // 对a[0]进行原子加操作，无返回值。  
46     atomic_add(&a[0], 2);  
47 }  
48  
49 int main()  
50 {  
51     // 配置使用0号计算核心阵列  
52     sdaaSetDevice(0);  
53     // 初始化操作数据  
54     int64_t host_data[DSIZE] = {0};  
55     int size64 = DSIZE * sizeof(int64_t);  
56     // 申请device内存  
57     int64_t *dev_data = NULL;  
58     // 使用sdaaMalloc为device端地址进行空间分配  
59     sdaaMalloc((void **)(&dev_data), size64);  
60     // 使用sdaaMemcpy从host端复制到device端  
61     sdaaMemcpy(dev_data, host_data, size64, sdaaMemcpyHostToDevice);  
62  
63     // 启动并行计算  
64     atomic_op(<<1>>>(dev_data);  
65  
66     // 等待计算核心SPR完成之操作所有操作任务  
67     sdaaDeviceSynchronize();  
68     // 使用sdaaMemcpy从device端复制到host端  
69     sdaaMemcpy(host_data, dev_data, size64, sdaaMemcpyDeviceToHost);  
70  
71     // 查看操作操作结果是否正确  
72     if (host_data[0] != 64) {  
73         printf("atomic add error: %lu\n", host_data[0]);  
74     }  
75  
76     // 释放Device端内存  
77     sdaaFree(dev_data);  
78     return 0;  
79 }
```